movo

TOMO II

Documentación de Producto

PROYECTO FINAL 2026 · 5K3 · GRUPO 27

OBRA EN TRES TOMOS

Movo: Plataforma de logística colaborativa P2P.

Proyecto Final 2026 · Ingeniería en Sistemas de Información · Facultad Regional Córdoba.

Tomo I	Documentación de Proyecto
Tomo II	Documentación de Producto ESTE TOMO
Tomo III	Manuales

movo

II

TOMO

PROYECTO FINAL 2026 · 5K3 · GRUPO 27

Documentación de Producto

Épicas e historias de usuario con su trazabilidad, modelos de diseño, arquitectura del sistema y registro de decisiones.

INTEGRANTES

ARIZA, Alena	95359
BORDINO BLANCHE, Juan Cruz	95008
DALMAGRO, Lucas	94366
VERGARA, Tomás Ignacio	94197
YORLANO, Pedro	95197

EQUIPO DOCENTE

Ing. Sergio Quinteros · Profesor Adjunto
Ing. Cecilia Trettel · JTP

ENTREGA

Turno Noche · Proyecto independiente
Córdoba, noviembre de 2026

UTN FRC FACULTAD REGIONAL CÓRDOBA

D·Sistemas Departamento de Ingeniería en Sistemas de Información

CÁTEDRA PROYECTO FINAL

Movo: plataforma de logística colaborativa p2p
Tomo II: Documentación de Producto

© 2026, Alena Ariza, Juan Cruz Bordino Blanche, Lucas Dalmagro, Tomás Ignacio Vergara y Pedro Yorlano.

1.ª edición, entrega final del 2 de noviembre de 2026.

Edita: Movo, proyecto independiente. Córdoba, Argentina. movosend.app

ISBN, ISSN y depósito legal: no corresponden (edición académica de circulación no comercial).

Proyecto Final · Ingeniería en Sistemas de Información · Universidad Tecnológica Nacional, Facultad Regional Córdoba. Curso 5K3, turno Noche, grupo 27, 2026.

Profesor Adjunto: Ing. Sergio Quinteros. Jefa de Trabajos Prácticos: Ing. Cecilia Trettel.

Autoría y desarrollo	Alena Ariza; Juan Cruz Bordino Blanche; Lucas Dalmagro; Tomás Ignacio Vergara; Pedro Yorlano
Identidad visual	Manual de Marca de Movo
Diseño de tapas	Equipo Movo, con Claude Design
Fotos del equipo	Sitio institucional de Movo
Composición	HTML y CSS de impresión (WeasyPrint) a partir del repositorio del proyecto. Texto en Inter, código y datos en JetBrains Mono. Diagramas en PlantUML, notación UML 2.

Ariza, Alena; Bordino Blanche, Juan Cruz; Dalmagro, Lucas; Vergara, Tomás Ignacio; Yorlano, Pedro.
Movo: plataforma de logística colaborativa p2p. Tomo II: Documentación de Producto / Alena Ariza, Juan Cruz Bordino Blanche, Lucas Dalmagro, Tomás Ignacio Vergara y Pedro Yorlano. 1.ª ed. Córdoba: Universidad Tecnológica Nacional, Facultad Regional Córdoba, 2026.
1. Logística colaborativa. 2. Ingeniería de software. 3. Proyecto final de carrera. I. Título.

Cómo citar. Ariza, A.; Bordino Blanche, J. C.; Dalmagro, L.; Vergara, T. I.; Yorlano, P. (2026). *Movo: plataforma de logística colaborativa p2p. Tomo II: Documentación de Producto.* Proyecto Final, Ingeniería en Sistemas de Información. Universidad Tecnológica Nacional, Facultad Regional Córdoba.

Reservados todos los derechos. Prohibida su reproducción total o parcial sin el permiso previo de los autores.

Esta documentación describe el sistema tal como quedó construido al 2 de noviembre de 2026; donde un documento previo y el código difieren, los tomos reflejan el código. Los datos de usuarios, envíos y pagos que aparecen en ejemplos, capturas y casos de prueba corresponden a cuentas y entornos de prueba, no a personas ni operaciones reales.

Los nombres de productos y servicios de terceros mencionados (Mercado Pago, Google, Didit, Twilio, Amazon Web Services, entre otros) pertenecen a sus respectivos titulares y se citan solo para describir las integraciones.

UTN FRC Facultad Regional Córdoba

D·Sistemas
Departamento de Ingeniería en Sistemas de Información

Presentación del tomo II

Este tomo describe el **producto** Movo tal como quedó construido. Parte del backlog (épicas e historias de usuario agrupadas por tema) y, para cada tema, conecta cada historia con su descripción y criterios de aceptación, el diagrama de secuencia que muestra cómo la resuelve el sistema y los casos de prueba que la verifican. Completan el tomo los modelos transversales de diseño: diagrama de clases, patrones de diseño, diagramas de transición de estados, modelo de datos (DER) y la arquitectura del sistema, con sus decisiones registradas como ADR y como decisiones de diseño (DD).

Los diagramas siguen la notación UML 2 (y el modelo C4 para la vista de arquitectura) y fueron construidos a partir del código fuente del repositorio, que es la fuente de verdad: donde un documento previo y el código difieren, este tomo refleja el código.

La **audiencia** es el tribunal evaluador, para valorar la calidad técnica, y los futuros mantenedores del sistema. Se asume conocimiento de ingeniería de software (UML, arquitectura de microservicios, bases de datos relacionales y APIs REST).

Organización de la documentación

Tomo	Contenido
Tomo I	**Documentación de Proyecto.** Ficha para Biblioteca, Estudio Inicial, Plan de Proyecto, Informe de Impacto Ambiental, Paper y Póster, y la Gestión del Proyecto desde el Sprint 0 hasta el último sprint, con la retrospectiva final.
Tomo II ESTE TOMO	**Documentación de Producto.** Listado de épicas e historias de usuario, temas agrupadores y trazabilidad de cada historia (criterios de aceptación, diagramas de secuencia y casos de prueba), diagrama de clases, patrones de diseño, diagramas de transición de estados, DER, arquitectura del sistema (despliegue, ambiente de implementación y registro completo de ADR), registro de decisiones de diseño y glosario.
Tomo III	**Manuales.** Manual de Procedimientos (proceso de negocio con el sistema integrado, roles y responsabilidades, procedimientos de administración), Manual de Instalación y Operación, y Manual de Usuario de la aplicación móvil y del panel de administración.

Índice general

01 **Épicas e Historias de Usuario** 13

1.1 Épicas e Historias de Usuario 15

1.1.1 Resumen por épica 15

1.1.2 EP-01 · Identidad y Confianza 16

1.1.3 EP-02 · Publicación y Descubrimiento 17

1.1.4 EP-03 · Asignación y Negociación 17

1.1.5 EP-04 · Ejecución del Envío 18

1.1.6 EP-05 · Pagos y Economía 19

1.1.7 EP-06 · Reputación y Comunidad 19

1.1.8 EP-07 · Gestión y Administración 20

1.1.9 Trabajo técnico, de corrección y de gestión 20

02 **EP-01 · Identidad y Confianza** 27

2.1 Descripción del Tema 29

2.2 Historias de Usuario y Trazabilidad 30

2.2.1 MOVO-7 · Registro de usuario con verificación de identidad 30

2.2.2 MOVO-63 · Inicio de sesión y gestión de la sesión 36

2.2.3 MOVO-64 · Recuperación de contraseña 41

2.2.4 MOVO-14 · Perfil del usuario 44

2.2.5 MOVO-31 · Edición de datos personales y de la cuenta 45

2.2.6 MOVO-15 · Verificación de la licencia de conducir del transportista 47

2.2.7 MOVO-39 · Baja de la cuenta 48

2.2.8 MOVO-32 · Revisión de casos de KYC en revisión manual (procedimiento operativo) 49

03 **EP-02 · Publicación y Descubrimiento** 51

3.1 Descripción del Tema 53

3.2 Historias de Usuario y Trazabilidad 54

3.2.1 MOVO-8 · Creación de un envío 54

3.2.2 MOVO-16 · Confirmación o rechazo del envío por el receptor 59

3.2.3 MOVO-18 · Declaración y gestión de viajes 62

3.2.4 MOVO-19 · Feed de paquetes disponibles 67

3.2.5 MOVO-29 · Cancelación de un envío por el emisor 69

3.2.6 MOVO-27 · Historial de envíos 71

3.2.7 MOVO-191 · Envíos activos en el inicio de la app 72

3.2.8 MOVO-259 · Rediseño de "Mis viajes" para el transportista 73
3.2.9 MOVO-138 · Motor de precios dinámico 74

04 EP-03 · Asignación y Negociación 77

4.1 Descripción del Tema 79
4.2 Historias de Usuario y Trazabilidad 80
4.2.1 MOVO-23 · Oferta del transportista 80
4.2.2 MOVO-17 · Lista de ofertas y elección del transportista 85
4.2.3 MOVO-109 · Seguimiento de las ofertas del transportista 88
4.2.4 MOVO-102 · Modelo y máquina de estados de la oferta 90

05 EP-04 · Ejecución del Envío 91

5.1 Descripción del Tema 93
5.2 Historias de Usuario y Trazabilidad 94
5.2.1 MOVO-6 · Handshake criptográfico de retiro y entrega 94
5.2.2 MOVO-10 · Mapa de ruta optimizada multi-parada 100
5.2.3 MOVO-11 · Seguimiento en tiempo real del transportista 103
5.2.4 MOVO-21 · Evidencia fotográfica en retiro y entrega 106
5.2.5 MOVO-24 · Notificaciones push en los eventos del envío 108
5.2.6 MOVO-239 · Configuración de notificaciones 111
5.2.7 MOVO-240 · Catálogo de notificaciones faltantes 112
5.2.8 MOVO-26 · Chat en tiempo real entre las partes 113
5.2.9 MOVO-275 · Transferencia de la recepción a otra persona 118

06 EP-05 · Pagos y Economía 123

6.1 Descripción del Tema 125
6.2 Historias de Usuario y Trazabilidad 126
6.2.1 MOVO-12 · Reserva automática de fondos antes del retiro 126
6.2.2 MOVO-13 · Pago automático al transportista (split payment) 132
6.2.3 MOVO-28 · Historial de pagos y cobros 136
6.2.4 MOVO-37 · Pago de la comisión en efectivo 137
6.2.5 MOVO-65 · Tarjetas guardadas 138
6.2.6 MOVO-110 · Vinculación de la cuenta de Mercado Pago del transportista 141
6.2.7 MOVO-226 · Penalidad de cancelación y cuenta corriente 143

07 EP-06 · Reputación y Comunidad 149

7.1 Descripción del Tema 151
7.2 Historias de Usuario y Trazabilidad 152

7.2.1 MOVO-22 · Calificación entre las partes de un envío ... 152
7.2.2 MOVO-25 · Score de reputación ponderado en el perfil ... 155
7.2.3 MOVO-175 · Reportar y bloquear usuarios ... 157

08 EP-07 · Gestión y Administración ... 161

8.1 Descripción del Tema ... 163
8.2 Historias de Usuario y Trazabilidad ... 164
8.2.1 MOVO-286 · Acceso al panel con segundo factor ... 164
8.2.2 MOVO-293 · Búsqueda de usuarios y ficha completa ... 169
8.2.3 MOVO-36 · Suspensión e inhabilitación de cuentas ... 171
8.2.4 MOVO-33 · Monitoreo de envíos desde el panel ... 176
8.2.5 MOVO-30 · Disputa sobre un envío ... 177
8.2.6 MOVO-34 · Gestión y resolución de disputas por el administrador ... 181
8.2.7 MOVO-35 · Métricas de oferta y demanda ... 182
8.2.8 MOVO-319 · Soporte dentro de la aplicación ... 183
8.2.9 MOVO-329 · Corrección manual del estado de un envío ... 187
8.2.10 MOVO-224 · Legal y compliance (Términos, Privacidad y Ley 25.326) ... 189

09 Diagrama de Clases ... 191

9.1 Modelo de Clases del Dominio ... 193
9.2 Contexto de Identidad ... 194
9.3 Contexto de Envíos ... 195
9.4 Contexto de Pagos ... 196
9.5 Estructura en Capas e Integración con Proveedores ... 198

10 Patrones de Diseño ... 201

10.1 Criterio de Selección ... 203
10.2 Resumen ... 204
10.3 Creacionales y Estructurales ... 206
10.3.1 Strategy y Factory Method: proveedores intercambiables ... 206
10.3.2 Adapter y Anti-Corruption Layer: clientes de otros servicios ... 206
10.3.3 Repository ... 206
10.3.4 API Gateway y Proxy ... 207
10.3.5 Inyección de dependencias mediante fábricas ... 207
10.3.6 DTO y contrato compartido ... 207
10.4 De Comportamiento ... 208
10.4.1 State: máquinas de estados explícitas ... 208
10.4.2 Publicación y suscripción para el tiempo real ... 208

10.5 De Consistencia e Integración Distribuida ... 209
10.5.1 Saga con compensación: asignación y reserva de fondos ... 209
10.5.2 Bloqueo optimista (compare-and-swap) ... 209
10.5.3 Receptor idempotente ... 209
10.5.4 Historial de solo agregar filas ... 209
10.6 De Resiliencia y Operación ... 210
10.6.1 Barrido periódico con candado distribuido ... 210
10.6.2 Degradación controlada: fail-open y fail-closed ... 210

11 Diagramas de Transición de Estados ... 211
11.1 Diagramas de Transición de Estados ... 213

12 Modelo de Datos (DER) ... 223
12.1 Modelo de Datos y Diccionario ... 225

13 Arquitectura del Sistema ... 273
13.1 Visión General ... 275
13.2 Vista de Contexto ... 276
13.3 Vista de Contenedores ... 278
13.4 Vista de Despliegue ... 280
13.4.1 Integración y despliegue continuos ... 280
13.5 Ambiente de Implementación ... 282
13.5.1 Software ... 282
13.5.2 Hardware y red ... 282
13.5.3 Operación ... 282
13.6 Seguridad ... 283
13.7 Limitaciones Aceptadas ... 284
13.8 Registro de Decisiones de Arquitectura ... 285
13.8.1 Índice de ADR ... 285
13.8.2 ADR-001 · Patrón Arquitectónico del Backend ... 286
13.8.3 ADR-002 · Lenguaje y Framework del Backend ... 288
13.8.4 ADR-003 · Motor y Estrategia de Base de Datos ... 289
13.8.5 ADR-004 · Mecanismo de Autenticación y Autorización ... 291
13.8.6 ADR-005 · Diseño y Protocolo de la API ... 293
13.8.7 ADR-006 · Infraestructura y Estrategia de Despliegue ... 294
13.8.8 ADR-007 · Estrategia de Almacenamiento de Imágenes y Archivos Multimedia ... 296
13.8.9 ADR-008 · Proveedor de Datos de Distancia para el Motor Logístico ... 298
13.8.10 ADR-009 · Infraestructura como Código (Terraform) y Gestión de DNS ... 300

13.8.11 ADR-010 · Confianza en la Red Interna del API Gateway 302
13.8.12 ADR-011 · Adopción de Prisma como ORM para los servicios de MOVO 303
13.8.13 ADR-012 · Twilio como proveedor de SMS para verificación de OTP 304
13.8.14 ADR-013 · Refresh token con TTL de 90 días, rotación de un solo uso y detección de reuso 306
13.8.15 ADR-014 · Google Maps como proveedor de geocoding detrás de GeocodingProvider 307
13.8.16 ADR-015 · Migración de Distance Matrix API (Legacy) a Routes API (Compute Route Matrix) 308
13.8.17 ADR-016 · Resend como proveedor de email detrás de EmailProvider 310
13.8.18 ADR-017 · Statelessness de movo-svc-pricing-logistics y ubicación de la entidad Offer 311
13.8.19 ADR-018 · Handshake de custodia: firma ECDSA P-256 generada en el dispositivo . 313
13.8.20 ADR-019 · Canal de tiempo real – WebSocket nativo (@fastify/websocket) 314
13.8.21 ADR-020 · Tracking en tiempo real: ingesta por HTTP, difusión por WebSocket 317
13.8.22 ADR-021 · Motor de precios dinámico: combustible desde datos abiertos, demanda por zona y contrato simplificado 318
13.8.23 ADR-022 · Arquitectura de cobro: Payments API + OAuth Connect + application_fee . 321
13.8.24 ADR-023 · Chat propio sobre el canal WebSocket (MOVO-26) 323

14 Registro de Decisiones de Diseño 325

14.1 Registro de Decisiones de Diseño 327

15 Glosario 359

15.1 Lenguaje del Dominio 361

01

PRODUCTO · BACKLOG

Épicas e Historias de Usuario

Listado vigente del backlog de Movo, agrupado por épicas y por temas, con el estado real de cada historia.

CONTENIDO

1.1 Épicas e Historias de Usuario 15

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: listado completo de épicas e historias, con su estado al cierre del Sprint 8, derivado del tablero de Linear.

Introducción

Este documento lista las épicas y las historias de usuario de Movo en su versión vigente, con el estado real de cada una al momento de la entrega. Es el índice del resto del tomo: cada épica da lugar a un tema con la trazabilidad detallada de sus historias.

Audiencia

El tribunal evaluador y los futuros mantenedores, que necesitan ubicar rápidamente qué funcionalidad existe, en qué estado está y dónde se documenta.

1.1 Épicas e Historias de Usuario

El backlog de Movo se organiza en siete **épicas** alineadas con los contextos delimitados (*bounded contexts*) de la arquitectura de microservicios. Cada épica agrupa las historias de usuario de un mismo dominio funcional y da origen a un **tema** de este tomo: los siete documentos siguientes presentan, historia por historia, los criterios de aceptación, el diagrama de secuencia y los casos de prueba.

1.1.1 Resumen por épica

La estimación inicial proviene del Sprint 0 (32 historias, en horas lineales de desarrollo). Las columnas finales muestran el estado real de las historias que componen hoy cada épica, según Linear.

Tabla 1.1. Resumen del backlog por épica.

ID	Épica	Historias iniciales	Horas iniciales	Historias hoy	Completadas	En curso o pendientes
EP-01	Identidad y Confianza	6	66	8	7	1
EP-02	Publicación y Descubrimiento	6	64	9	9	0
EP-03	Asignación y Negociación	2	18	4	4	0
EP-04	Ejecución del Envío	6	92	9	8	1
EP-05	Pagos y Economía	4	42	7	5	2
EP-06	Reputación y Comunidad	3	24	3	3	0
EP-07	Gestión y Administración	5	46	10	4	6
	Total	**32**	**352**	**50**	**40**	**10**

El backlog creció de 32 historias iniciales a 50 historias agrupadas en los siete temas (algunas de enfoque técnico, como el modelo de la oferta o el motor de precios, que el equipo gestionó como historia): varias historias se subdividieron al refinarse (por ejemplo, la creación de envíos pasó a incluir cotización congelada, fotos y confirmación del receptor), y otras surgieron de necesidades que el Sprint 0 no había previsto (chat, transferencia de receptor, penalidades de cancelación, soporte, sanciones, acceso al panel con segundo factor). De esas 50 historias, 40 están completas; el resto está en curso, por hacer o en backlog y se documenta con su estado real.

1.1.2 EP-01 · Identidad y Confianza

Tabla 1.2. Historias de la épica EP-01.

Historia	Enunciado	Estado	Sprint	Tickets
MOVO-7	Como usuario nuevo, quiero completar el proceso de registro para poder operar en la plataforma	Completada	Sprint 2, Sprint 3, Sprint 4	8
MOVO-63	Como usuario registrado, quiero iniciar sesión en la aplicación con mis credenciales, para acceder a mi cuenta y operar en la plataforma sin tener que registrarme nuevamente.	Completada	Sprint 2, Sprint 6	4
MOVO-64	Como usuario registrado, quiero recuperar el acceso a mi cuenta si olvido mi contraseña, para poder volver a operar en la plataforma sin necesidad de crear una cuenta nueva.	Completada	Sprint 4	2
MOVO-14	Como usuario registrado, quiero ver mi perfil para ver mi información.	Completada	Sprint 2, Sprint 3	2
MOVO-31	Como usuario registrado, quiero editar mis datos personales y de contacto en mi perfil, para mantener mi información actualizada dentro de la plataforma.	Completada	Sprint 3, Sprint 4	5
MOVO-15	Como transportista quiero verificar mi licencia de conducir, para aumentar mi perfil de confianza y obtener la insignia de transportista verificado.	Completada	Sprint 3	0
MOVO-39	Como usuario registrado, quiero solicitar la baja de mi cuenta para ejercer mi derecho de supresión conforme a la normativa de protección de datos vigente.	Completada	Sprint 4	0
MOVO-32	Como administrador, quiero revisar y aprobar o rechazar los casos de KYC pendientes, para asegurar que solo usuarios verificados operen en la plataforma.	Backlog	sin asignar	0
MOVO-66	Como equipo de desarrollo, quiero el schema, las migraciones y la capa de persistencia de movo-svc-users, para que las historias de identidad tengan una base de datos versionada y reproducible.	Completada	Sprint 2	6
MOVO-67	Como equipo de desarrollo, quiero una librería compartida de JWT, tipos de dominio y contrato de errores en movo-shared, para que todos los servicios validen identidad y respondan errores de forma homogénea sin duplicar código.	Completada	Sprint 2	0
MOVO-68	Como equipo de desarrollo, quiero el middleware de autenticación, autorización por rol y rate limiting en movo-api-gateway, para que todos los endpoints protegidos validen identidad en un único punto de entrada.	Completada	Sprint 2	0
MOVO-69	Como equipo de desarrollo, quiero el setup de testing automatizado y los quality gates en CI, para que ninguna PR se mergee a develop sin tests en verde y podamos medir cobertura real por sprint.	Completada	Sprint 2	0

Las historias `MOVO-66` a `MOVO-69` son historias *habilitadoras* (las pide el equipo, no un usuario final): persistencia de usuarios, librería compartida, autenticación en el gateway y puesta a punto del testing automatizado.

1.1.3 EP-02 · Publicación y Descubrimiento

Tabla 1.3. Historias de la épica EP-02.

Historia	Enunciado	Estado	Sprint	Tickets
`MOVO-8`	Como emisor, quiero crear un envio para iniciar el proceso de envio de forma clara y estructurada	Completada	Sprint 3	9
`MOVO-16`	Como receptor quiero aceptar o rechazar un envío para evitar recibir paquetes indeseados.	Completada	Sprint 3, Sprint 4	4
`MOVO-18`	Como transportista quiero declarar un viaje para ver paquetes compatibles con mi ruta.	Completada	Sprint 4, Sprint 5, Sprint 6	8
`MOVO-19`	Como transportista, quiero ver los paquetes disponibles en mi ciudad/región para encontrar oportunidades de transporte.	Completada	Sprint 4, Sprint 5	3
`MOVO-29`	Como emisor, quiero cancelar un envío publicado antes de que sea asignado a un transportista, para poder corregir errores o desistir del envío sin penalización.	Completada	Sprint 3	0
`MOVO-27`	Como emisor, quiero consultar el historial de mis envíos con estados y línea de tiempo, para hacer seguimiento de todos mis envíos pasados y actuales desde un solo lugar.	Completada	Sprint 3, Sprint 6	3
`MOVO-191`	Como usuario, quiero ver y operar mis envíos activos desde el inicio de la app, para saber qué tengo que hacer hoy sin buscar en el historial.	Completada	Sprint 5, Sprint 7	3
`MOVO-259`	[Transversal] Rediseño de “Mis viajes” para el transportista	Completada	Sprint 6, Sprint 7	4
`MOVO-138`	[svc-pricing-logistics] Motor de precios dinámico real (Post-Spike MOVO-216)	Completada	Sprint 6	0

1.1.4 EP-03 · Asignación y Negociación

Tabla 1.4. Historias de la épica EP-03.

Historia	Enunciado	Estado	Sprint	Tickets
`MOVO-23`	Como transportista, quiero crear una oferta indicando cuánto quiero cobrar por un envío, para participar en la selección del emisor con el precio que considero justo.	Completada	Sprint 4	4
`MOVO-17`	Como emisor quiero ver la lista de ofertas de transportistas para elegir la opción más conveniente.	Completada	Sprint 4	2

Historia	Enunciado	Estado	Sprint	Tickets
MOVO-109	Como transportista, quiero ver el estado de mis ofertas activas, para hacer seguimiento de mis propuestas sobre los envíos.	Completada	Sprint 4, Sprint 5, Sprint 6	10
MOVO-102	[svc-shipments] Schema y máquina de estados de la oferta (Offer)	Completada	Sprint 3	0

1.1.5 EP-04 · Ejecución del Envío

Tabla 1.5. Historias de la épica EP-04.

Historia	Enunciado	Estado	Sprint	Tickets
MOVO-6	Como transportista, quiero ejecutar el Cryptographic Handshake para confirmar retiro y entrega segura.	Completada	Sprint 4, Sprint 5, Sprint 6	5
MOVO-10	[movo-mobile] Pantalla y mapa de ruta optimizada multi-parada	Completada	Sprint 5, Sprint 6	5
MOVO-11	Como emisor o receptor, quiero ver la ubicación del transportista en tiempo real en un mapa interactivo, para saber exactamente dónde está mi paquete en cada momento del envío.	Completada	Sprint 5, Sprint 6	9
MOVO-21	Como transportista, quiero registrar fotografías al retirar y entregar cada paquete, para dejar evidencia visual del estado del paquete.	Completada	Sprint 5, Sprint 6	4
MOVO-24	Como emisor o receptor, quiero recibir notificaciones push en los eventos clave del envío, para estar informado durante el mismo.	Completada	Sprint 3, Sprint 6, Sprint 7, Sprint 8	7
MOVO-239	[svc-users] [movo-mobile] Notificaciones: pantalla de configuración con toggle maestro y desglose granular (push existentes)	Completada	Sprint 6	2
MOVO-240	Notificaciones: catálogo de notificaciones faltantes	Por hacer	Sprint 8	0
MOVO-26	[svc-shipments] Chat en tiempo real entre las partes de un envío (emisor, receptor y transportista)	Completada	Sprint 7	0
MOVO-275	Transferencia de receptor: el receptor designado puede pasarle la recepción a otra persona	Completada	Sprint 7	0

1.1.6 EP-05 · Pagos y Economía

Tabla 1.6. Historias de la épica EP-05.

Historia	Enunciado	Estado	Sprint	Tickets
MOVO-12	Como emisor, quiero que mis fondos se reserven automáticamente antes del retiro del paquete, para que el pago solo se ejecute cuando el paquete sea entregado.	Completada	Sprint 7, Sprint 8	6
MOVO-13	Como transportista, quiero recibir el pago automático con split payment tras confirmarse la entrega, para cobrar inmediatamente después de entregar sin gestiones adicionales.	Completada	Sprint 7, Sprint 8	3
MOVO-28	Como usuario, quiero ver el historial de mis pagos y cobros, para llevar registro de lo que gasté y cobré en envíos.	Completada	Sprint 7	3
MOVO-37	Como emisor, quiero declarar que voy a pagar la comisión de Movo en efectivo al momento de la entrega, para poder pagar sin una tarjeta de crédito.	Por hacer	Sprint 8	0
MOVO-65	Como emisor, quiero guardar mis tarjetas en la app y pagar un envío ingresando solo el código de seguridad (CVV), para no cargar todos los datos de la tarjeta cada vez que acepto una oferta.	Completada	Sprint 7	3
MOVO-110	Como transportista, quiero vincular mi cuenta de Mercado Pago a Movo, para poder cobrar mis envíos vía split payment	Completada	Sprint 7	3
MOVO-226	Definir e implementar la mecánica de penalidad de cancelación con Transportista asignado (holds/ liberación de pagos)	En curso	Sprint 7, Sprint 8	12

1.1.7 EP-06 · Reputación y Comunidad

Tabla 1.7. Historias de la épica EP-06.

Historia	Enunciado	Estado	Sprint	Tickets
MOVO-22	Como usuario, quiero calificar y dejar un comentario a las otras partes post-entrega, para contribuir al sistema de confianza de la red.	Completada	Sprint 4	2
MOVO-25	Como usuario, quiero ver el score de reputación ponderado en el perfil de cualquier otro usuario, para tomar decisiones informadas.	Completada	Sprint 4, Sprint 5, Sprint 6	10
MOVO-175	[movo-mobile][svc-users] Reportar y bloquear usuarios	Completada	Sprint 6	1

1.1.8 EP-07 · Gestión y Administración

Tabla 1.8. Historias de la épica EP-07.

Historia	Enunciado	Estado	Sprint	Tickets
MOVO-286	Como administrador, quiero ingresar al panel con un segundo factor para operar sobre la plataforma de forma segura	Completada	Sprint 7	5
MOVO-293	Como administrador, quiero buscar un usuario y ver su ficha completa, para entender su actividad antes de tomar una decisión sobre su cuenta	Completada	Sprint 8	2
MOVO-36	Como administrador, quiero suspender o inhabilitar una cuenta de usuario ante comportamiento fraudulento o violación de términos, para proteger la integridad de la red.	En curso	Sprint 8	5
MOVO-33	Como administrador, quiero consultar los envíos de la plataforma con su estado y el detalle de cada uno, para monitorear la operación y detectar incidentes.	Completada	Sprint 8	4
MOVO-30	Como parte de un envío, quiero iniciar una disputa sobre un envío finalizado o en curso, para reclamar formalmente ante un problema con el paquete o con la entrega.	Backlog	Sprint 8	2
MOVO-34	Como administrador, quiero gestionar las disputas abiertas para actuar como árbitro entre las partes	Backlog	sin asignar	3
MOVO-35	Como administrador, quiero ver métricas de oferta y demanda por ruta y región, para tomar decisiones operativas.	Backlog	sin asignar	0
MOVO-319	Como usuario, quiero escribirle al equipo de soporte desde la app, con o sin un envío asociado, para resolver un problema sin salir de Movo.	Backlog	Sprint 8	5
MOVO-329	Como administrador, quiero corregir el estado de un envío trabado o inconsistente, para destrabar casos que el flujo normal no resuelve.	Backlog	Sprint 8	4
MOVO-224	Legal y compliance: Términos y Condiciones, Política de Privacidad y protección de datos personales (Ley 25.326)	Completada	Sprint 5, Sprint 7, Sprint 8	5

1.1.9 Trabajo técnico, de corrección y de gestión

Además de las historias de usuario, el tablero de Linear registra trabajo que no corresponde a una funcionalidad visible para el usuario: infraestructura, modelo de datos, investigaciones (*spikes*), evolución de funcionalidades ya entregadas, corrección de errores y entregables académicos. Se lista aquí para que el backlog quede completo; su trazabilidad está en los informes de cada sprint (Tomo I).

Infraestructura, CI/CD y base técnica

Tabla 1.9. Infraestructura, CI/CD y base técnica.

Ticket	Descripción	Estado	Sprint
MOVO-43	Configuración inicial de repositorios	Completada	Sprint 1
MOVO-44	Creación de cuentas de AWS	Completada	Sprint 1
MOVO-53	Completar Verificación de Status de Repositorios y Flujos de CI/CD	Completada	Sprint 2
MOVO-54	Creación del entorno completo en AWS con Terraform (dev y prod)	Completada	Sprint 1
MOVO-59	Configurar pipeline de CI/CD para movo-admin y movo-mobile	Completada	Sprint 2
MOVO-114	[svc-users] Acotar permisos IAM del role EC2 en movo-shipment-media al prefijo profile-photos/*	Completada	Sprint 4
MOVO-115	[svc-users] Race condition (TOCTOU) en confirmPhoto()/deletePhoto() puede dejar objetos huérfanos en S3	Completada	Sprint 4
MOVO-122	CI: aumentar command_timeout de migraciones (dev/prod) para dos servicios Prisma	Completada	Sprint 3
MOVO-156	[infra] Fotos de envío rotas en S3: permisos IAM del role EC2 acotados solo a profile-photos/* (regresión de MOVO-114)	Completada	Sprint 4
MOVO-232	Fijar identificadores de app store (android.package / ios.bundleIdentifier) para EAS/TestFlight	Completada	Sprint 5
MOVO-241	[CI/CD] Build automático de movo-mobile en EAS al crear un tag (development build + TestFlight)	Completada	Sprint 6
MOVO-267	[svc-payments] Setup base del servicio: schema payments (Prisma), cliente del SDK de MP, credenciales por ambiente y ruta en el gateway	Completada	Sprint 7
MOVO-91	Alinear enums de users.users (rol/KYC) con @movo/shared	Completada	Sprint 2
MOVO-92	Chore actualización de la Entidad User	Completada	Sprint 2
MOVO-93	Adoptar Prisma como ORM en movo-svc-users	Completada	Sprint 2

Modelo de datos e integraciones iniciales

Tabla 1.10. Modelo de datos e integraciones iniciales.

Ticket	Descripción	Estado	Sprint
MOVO-41	Creación de DER de Usuarios	Completada	Sprint 1
MOVO-42	Creación de DER de Admin	Completada	Sprint 1

Ticket	Descripción	Estado	Sprint
MOVO-46	Creación de DER de Shipments	Completada	Sprint 1
MOVO-51	Creación de DER de Payments	Completada	Sprint 1
MOVO-57	Actualizar documentación - Cambios por el DER (shipments)	Completada	Sprint 4
MOVO-58	Armar .MD del DER	Completada	Sprint 2
MOVO-60	Diagrama de Estado User	Completada	Sprint 2
MOVO-48	Didit.me	Completada	Sprint 2
MOVO-49	MercadoPago (Auth & Capture + Split)	Completada	Sprint 3
MOVO-50	OR-Tools VRPTW	Completada	Sprint 3
MOVO-216	[Spike] Investigación de algoritmos de pricing dinámico y consumo de API de combustible	Completada	Sprint 6
MOVO-217	[svc-pricing-logistics] Setup de OR-Tools, cliente Redis y RoutesProvider	Completada	Sprint 5
MOVO-218	[svc-pricing-logistics] POST /routes/evaluate-candidates con cache en Redis	Completada	Sprint 5
MOVO-219	[svc-shipments] Integrar desvío marginal en el feed de viajes (GET /trips/:id/matches)	Completada	Sprint 5
MOVO-264	[Spike] Viajes recurrentes: recurrencia como atributo del Trip con generación del siguiente viaje (ADR-042)	Completada	Sprint 7

Evolución de funcionalidades existentes

Tabla 1.11. Evolución de funcionalidades existentes.

Ticket	Descripción	Estado	Sprint
MOVO-116	[svc-shipments] Bloquear declarar viaje y crear oferta sin licencia aprobada + Mercado Pago vinculado	Completada	Sprint 7
MOVO-117	[movo-mobile] UI de requisitos pendientes para operar como transportista (licencia + Mercado Pago)	Completada	Sprint 7
MOVO-167	[movo-mobile] Gating del tab Transportar por licencia no verificada y pulido de UI del header	Completada	Sprint 7
MOVO-168	[movo-mobile] Estados de carga granulares y skeletons por componente en vistas principales	Completada	Sprint 8
MOVO-169	[movo-mobile] Tutorial interactivo de uso paso a paso con pop-ups guiados	Completada	Sprint 8

Ticket	Descripción	Estado	Sprint
MOVO-184	[movo-mobile] Actividad reciente del home no prioriza envíos por confirmar y no expira los cancelados	Completada	Sprint 6
MOVO-208	[svc-shipments] Extensión del set canónico de estados: `assigned_unfunded` y `completed` (migración, @movo/shared, DTE, DER)	Completada	Sprint 5
MOVO-249	[movo-mobile] Onboarding: carrusel explicativo de primer uso	Completada	Sprint 7
MOVO-253	Elegir otro receptor tras un rechazo: acción en “Requiere tu atención” y vencimiento a cancelado	Completada	Sprint 6
MOVO-254	[movo-mobile] Badge “Alta demanda en tu zona” en el detalle del envío (highDemand de MOVO-138)	Completada	Sprint 6
MOVO-255	[svc-shipments + movo-mobile] Cotización real y congelada en el resumen del wizard de envío	Completada	Sprint 6
MOVO-258	[svc-shipments] Expiración y cierre automático de envíos y viajes en todos sus estados	Completada	Sprint 6
MOVO-265	[svc-shipments] Viajes recurrentes: atributo de recurrencia en Trip y generación de la siguiente ocurrencia	Completada	Sprint 7
MOVO-266	[movo-mobile] Viajes recurrentes: opción “Se repite” al declarar/editar y badge de serie	Completada	Sprint 7
MOVO-270	[svc-shipments] Tracking GPS antes del retiro y atribución de responsabilidad del no-retiro; revisar conteo de oferta de ADR-025	Completada	Sprint 8
MOVO-276	[svc-pricing-logistics] Rework del motor de precios: factores de paquete (peso facturable, medidas, categoría) y límites duros	Completada	Sprint 7
MOVO-277	[svc-shipments][movo-mobile] Iniciar viaje: validar fecha de salida y exigir paquetes ejecutables (fuente única de “paquete vivo”)	Completada	Sprint 7
MOVO-278	[svc-shipments][movo-mobile] Salida para envíos in_transit anómalos: visibilidad en la card y paso automático a disputa	Completada	Sprint 7
MOVO-279	[movo-mobile] Exigir ubicación en segundo plano para operar como transportista	Completada	Sprint 7
MOVO-284	[movo-mobile] Pantalla de chat del envío: mensajes en tiempo real entre las partes	Completada	Sprint 7
MOVO-285	svc-payments/movo-mobile: persistir el fee real de Mercado Pago y mostrar el monto cobrado exacto	Completada	Sprint 8
MOVO-301	Chat del envío: animación de mensajes, rediseño del detalle del envío y fixes de navegación/contador	Completada	Sprint 7
MOVO-317	[movo-mobile][svc-shipments][svc-pricing] Peso y dimensiones libres en publicar envío: wizard, sin límites duros y recalibración de precio	En curso	Sprint 8

Ticket	Descripción	Estado	Sprint
MOVO-318	Mobile: el sheet de cancelación debe usar el monto real del hold como “reservado”	Por hacer	Sprint 8
MOVO-280	[svc-shipments + movo-mobile] Cotas de precio en las ofertas del transportista	Backlog	sin asignar
MOVO-96	[svc-users] Investigar si el DNI se puede extraer del resultado de Didit y eliminar el step manual del wizard	Backlog	sin asignar

Correcciones de errores

Tabla 1.12. Correcciones de errores.

Ticket	Descripción	Estado	Sprint
MOVO-113	[movo-mobile] Paquete de fixes por bugs en UI	Completada	Sprint 4
MOVO-244	[Bugs] Batch: KYC, TyC obligatorios, sincronización de estado y varios de UI	Completada	Sprint 6
MOVO-271	[movo-mobile] Paquete de fixes #2: ofertas, seguimiento en vivo y detalle de envío	Completada	Sprint 6
MOVO-340	[Bugs] Paquete de fixes #3: teclado con pantalla externa, hora sin unidades, doble redirect, búsqueda sin tildes, chat tras transferencia, fecha por defecto, cancelación del mismo día y búsqueda por coordenadas	Completada	Sprint 8

Gestión del proyecto y entregables académicos

Tabla 1.13. Gestión del proyecto y entregables académicos.

Ticket	Descripción	Estado	Sprint
MOVO-40	Diseño y Entrega de Paper	Completada	Sprint 2
MOVO-45	Diseño y Entrega del Póster	Completada	Sprint 2
MOVO-90	Creación de skill en Claude para facilitar la review de US	Completada	Sprint 2
MOVO-99	[Spike] Investigar automatización de pruebas UAT en simuladores mobile (iOS/Android), evaluando uso de IA	Completada	Sprint 3
MOVO-164	Entrega de video del proyecto	Completada	Sprint 4
MOVO-165	Presentación de estado actual	Completada	Sprint 5
MOVO-223	Revisión del Ficha/Selector de Auto	Completada	Sprint 5
MOVO-272	Feria de Proyectos	Completada	Sprint 6

Ticket	Descripción	Estado	Sprint
`MOVO-273`	Documentación	Completada	Sprint 6

02

TEMA 1 DE 7 · TRAZABILIDAD DE HISTORIAS DE USUARIO

EP-01 · Identidad y Confianza

Registro, verificación de identidad (KYC), sesión, perfil, licencia del transportista y ciclo de vida de la cuenta.

CONTENIDO

2.1	Descripción del Tema	29
2.2	Historias de Usuario y Trazabilidad	30

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: trazabilidad de las historias del tema, con criterios de aceptación de Linear, diagramas de secuencia derivados del código y casos de prueba derivados de las pruebas automatizadas.

Introducción

Este documento presenta la trazabilidad de las historias de usuario del tema *Identidad y Confianza*: para cada historia, su descripción y criterios de aceptación, los tickets que la implementaron, el diagrama de secuencia que muestra cómo la resuelve el sistema y los casos de prueba que la verifican.

Audiencia

El tribunal evaluador y los futuros mantenedores. Se asume conocimiento de las convenciones del proyecto y de UML.

2.1 Descripción del Tema

EP-01 · Identidad y Confianza agrupa todo lo que permite que dos desconocidos confíen el uno en el otro a través de Movo: el registro de cuentas, la verificación del teléfono y de la identidad (KYC), el inicio y la gestión de la sesión, el perfil y sus insignias, la verificación de la licencia de conducir del transportista y el ciclo de vida de la cuenta (cambio de datos, recuperación de acceso y baja).

Es la primera de las cuatro capas de confianza del producto (identidad, custodia, pagos y reputación): ningún usuario opera sin haber demostrado quién es. Se resuelve en `movo-svc-users` (identidad, sesión y KYC), en el API Gateway (autenticación del token en cada pedido) y en la aplicación móvil (wizard de registro, pantallas de sesión y perfil).

Servicios involucrados: `gateway`, `movo-svc-users`, `movo-mobile`; **externos:** Didit.me (KYC), proveedor de SMS (Twilio, detrás de la interfaz `SmsProvider`, ADR-012) y de email (Resend, `EmailProvider`, ADR-016).

2.2 Historias de Usuario y Trazabilidad

Tabla 2.1. Historias de usuario del tema.

Historia	Descripción	Estado	Sprint
MOVO-7	Registro de usuario con verificación de identidad	Completada	Sprint 2, Sprint 3, Sprint 4
MOVO-63	Inicio de sesión y gestión de la sesión	Completada	Sprint 2, Sprint 6
MOVO-64	Recuperación de contraseña	Completada	Sprint 4
MOVO-14	Perfil del usuario	Completada	Sprint 2, Sprint 3
MOVO-31	Edición de datos personales y de la cuenta	Completada	Sprint 3, Sprint 4
MOVO-15	Verificación de la licencia de conducir del transportista	Completada	Sprint 3
MOVO-39	Baja de la cuenta	Completada	Sprint 4
MOVO-32	Revisión de casos de KYC en revisión manual (procedimiento operativo)	Backlog	sin asignar

2.2.1 MOVO-7 · Registro de usuario con verificación de identidad

MOVO-7 COMPLETADA SPRINT 2, SPRINT 3, SPRINT 4 31 SP

Como usuario nuevo, quiero completar el proceso de registro para poder operar en la plataforma

Alcance final. La historia padre se implementó en cuatro tickets: alta de usuario con validaciones (MOVO-70), verificación del teléfono por OTP (MOVO-71), integración con Didit.me (MOVO-72) y las pantallas del wizard de registro y del KYC embebido (MOVO-73). El registro además acepta los Términos y la Política de Privacidad en su versión vigente (MOVO-228). El criterio de la tarjeta de pago se retiró en el refinamiento porque lo cubren MOVO-65 (emisor) y MOVO-110 (transportista).

Criterios de aceptación

1. El usuario puede registrarse ingresando nombre, email y numero de telefono.
2. El sistema valida que el email no este previamente registrado. Si lo esta, muestra error claro.
3. El sistema valida que el numero de telefono sea valido (formato argentino).
4. Al finalizar el formulario, el sistema invoca la API de Didit.me y redirige al flujo de KYC sin salir de la app.
5. Si Didit.me devuelve 'approved', el usuario queda habilitado para operar.
6. Si Didit.me devuelve 'rejected', el usuario ve un mensaje claro indicando que no pudo ser verificado.

7. Si Didit.me devuelve 'manual_review', el usuario ve un mensaje indicando que su verificacion esta pendiente.
8. El sistema no almacena imagenes de documentos ni datos biometricos en servidores de Movo.

Trazabilidad de implementación

Tabla 2.2. Tickets de implementación de MOVO-7.

Ticket	Alcance	Sprint	Estado
`MOVO-70`	[svc-users] Endpoint de registro de usuario: validaciones, hash de contraseña y persistencia	Sprint 2	Completada
`MOVO-71`	[svc-users] Verificación de teléfono por OTP con almacenamiento en Redis y adapter de SMS	Sprint 2	Completada
`MOVO-72`	[svc-users] Integración con Didit.me: creación de sesión KYC, webhook de resultado y máquina de estados	Sprint 2	Completada
`MOVO-73`	[movo-mobile] Pantallas de registro, verificación de OTP y flujo de KYC embebido	Sprint 2	Completada
`MOVO-94`	[svc-users] Revisar autenticación de POST /kyc/session y GET /kyc/status (hoy públicas, sin JWT)	Sprint 2	Completada
`MOVO-95`	[svc-users] register() emite tokens de sesión, AC10/AC3 de MOVO-70 desactualizados (PR #51)	Sprint 2	Completada
`MOVO-97`	[svc-users] Foto de perfil: subida a S3 con presigned URLs, confirmación y borrado	Sprint 3	Completada
`MOVO-98`	[movo-mobile] Paso de foto de perfil al cerrar el onboarding y edición desde el perfil	Sprint 3	Completada

Diagrama de secuencia

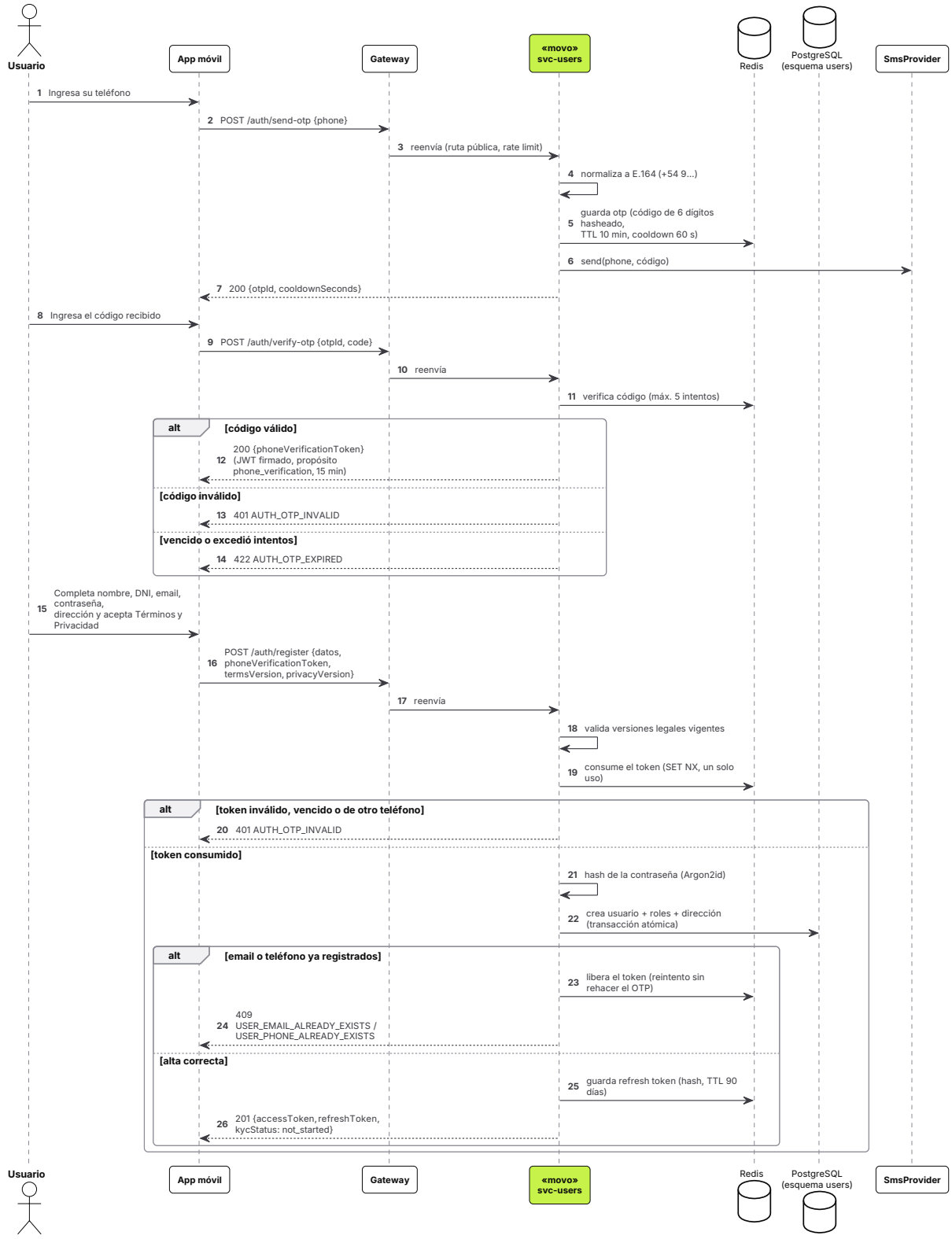


Figura 2.1. Registro con verificación de teléfono por OTP (MOVO-7, MOVO-70, MOVO-71).

Diagrama de secuencia

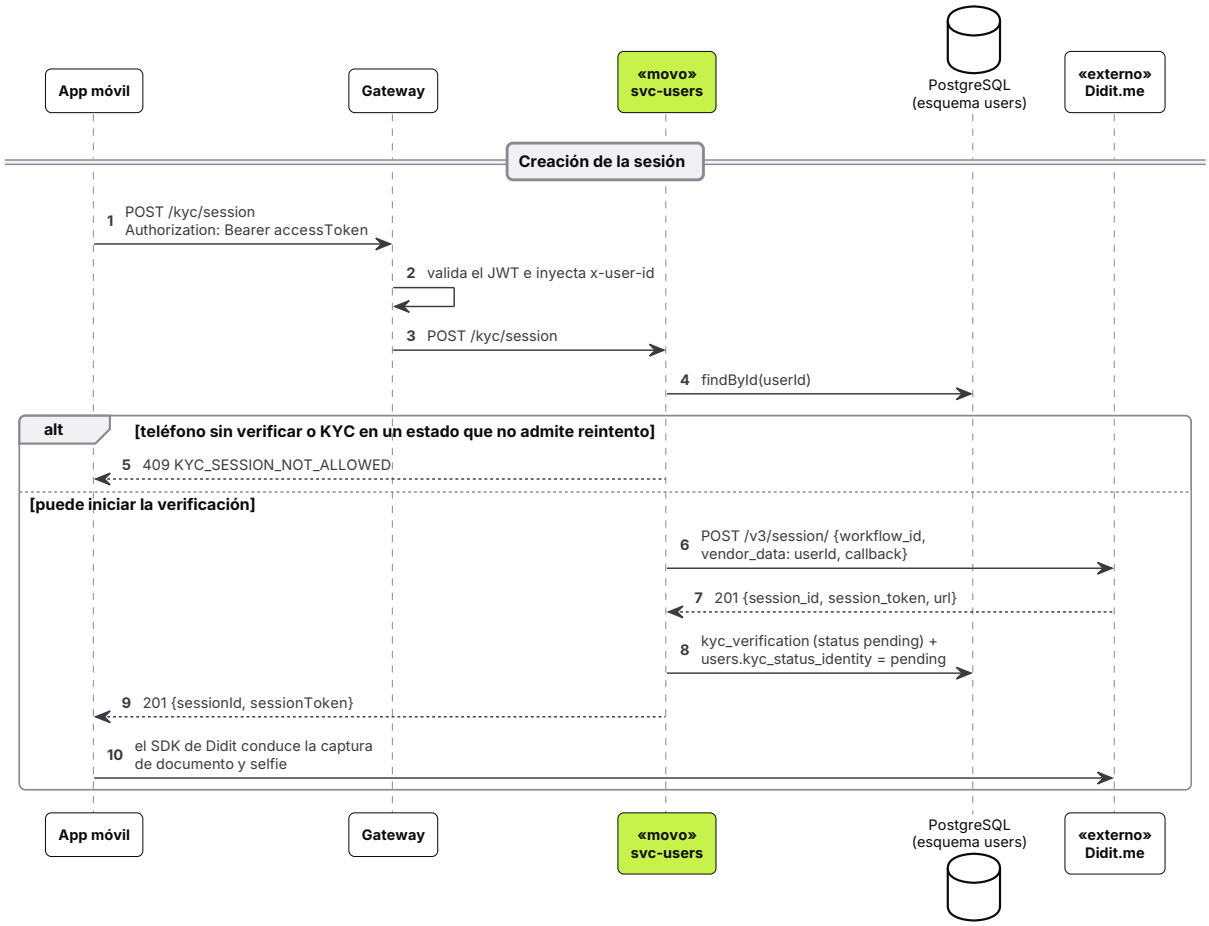


Figura 2.2. Verificación de identidad (KYC) delegada en Didit.me: sesión y webhook (MOVO-72). Parte 1 de 3: Creación de la sesión.

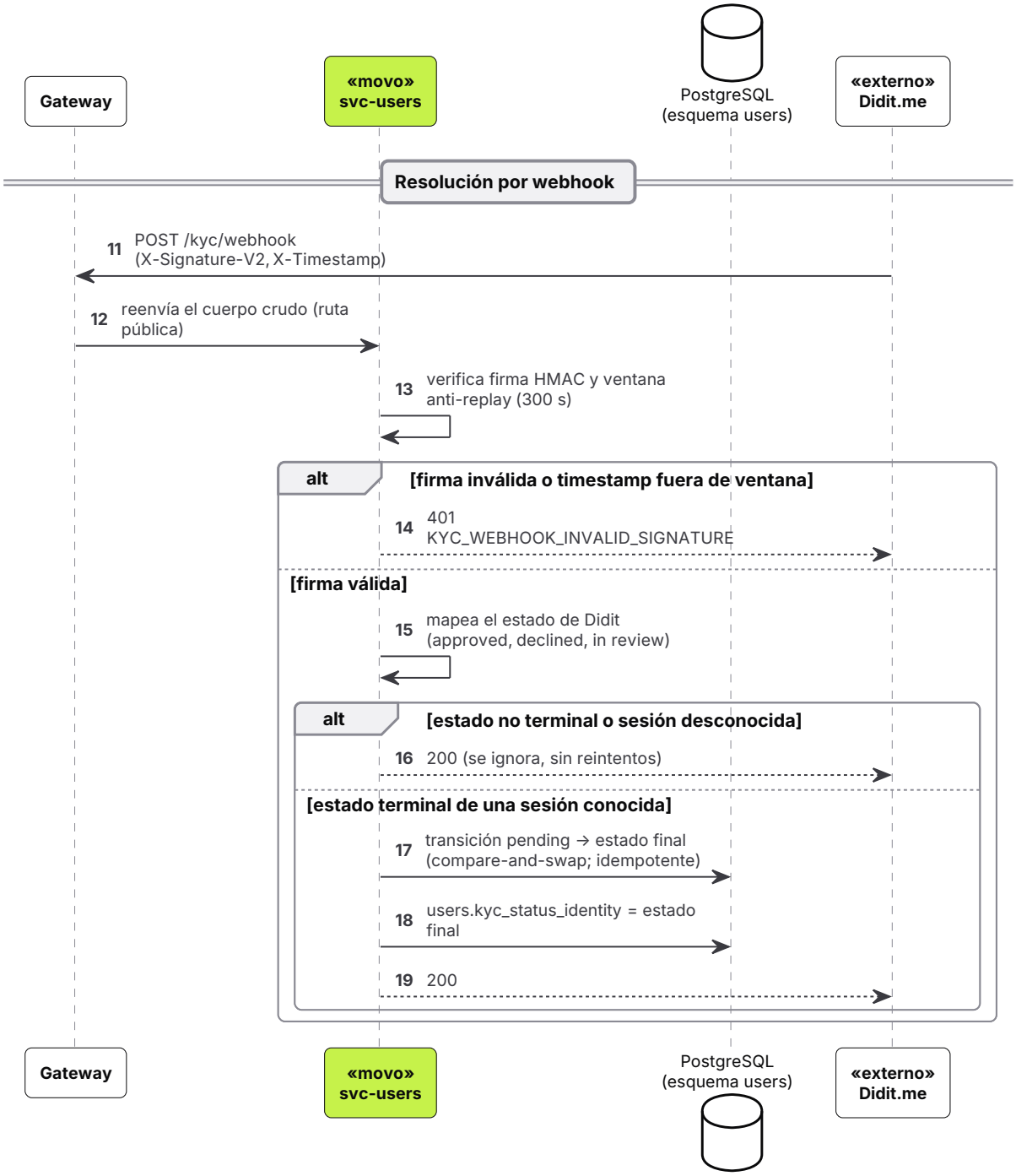


Figura 2.3. Verificación de identidad (KYC) delegada en Didit.me: sesión y webhook (MOVO-72). Parte 2 de 3: Resolución por webhook.

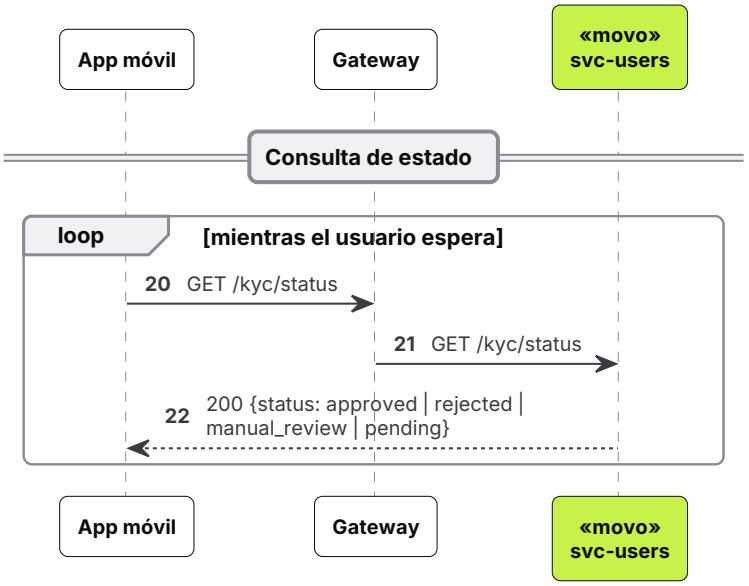


Figura 2.4. Verificación de identidad (KYC) delegada en Didit.me: sesión y webhook (MOVO-72). Parte 3 de 3: Consulta de estado.

Casos de prueba

Tabla 2.3. Casos de prueba de MOVO-7 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-7.01	1	**Registro con datos válidos** *Pre:* El teléfono tiene un phoneVerificationToken vigente. *Pasos:* Enviar POST /auth/register con nombre, apellido, DNI, email, contraseña, dirección y versiones legales vigentes.	201 con tokens de sesión; el usuario queda con roles por defecto, phoneVerified=true y kycStatus not_started.	auth.register.integration.test.ts	APROBADO
CP-7.02	1, 3	**Normalización del teléfono argentino** *Pre:* Ninguna. *Pasos:* Registrar con variantes válidas del formato argentino (+54 9..., 0351..., 15...) y con un teléfono de formato inválido.	Las variantes válidas se normalizan al mismo E.164; el formato inválido responde 400 VALIDATION_FAILED.	auth.register.integration.test.ts	APROBADO
CP-7.03	2	**Email o teléfono ya registrados** *Pre:* Existe un usuario con ese email (o teléfono). *Pasos:* Registrar un segundo usuario con el mismo email, o con el mismo teléfono escrito en otro formato.	409 USER_EMAIL_ALREADY_EXISTS o USER_PHONE_ALREADY_EXISTS; el token OTP se libera para reintentar sin rehacer el código.	auth.register.integration.test.ts auth.register-token-release.test.ts	APROBADO
CP-7.04	3	**Verificación del teléfono por OTP** *Pre:* Teléfono válido sin cuenta. *Pasos:* Pedir el código (send-otp), ingresarlo (verify-otp) y reintentar con código erróneo hasta agotar los 5 intentos.	Código correcto: 200 con phoneVerificationToken. Código erróneo: 401 AUTH_OTP_INVALID; al sexto intento 422. Un reenvío dentro de los 60 s devuelve 429.	auth.otp.integration.test.ts	APROBADO
CP-7.05	3	**El token de verificación es de un solo uso y atado al teléfono** *Pre:* Un phoneVerificationToken ya consumido en un registro. *Pasos:* Reutilizarlo en un segundo registro, o usarlo con otro número.	401 AUTH_OTP_INVALID en ambos casos, sin crear el usuario.	auth.register.integration.test.ts	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-7.06	4	**Creación de la sesión KYC** *Pre:* Usuario con teléfono verificado y KYC not_started. *Pasos:* POST /kyc/session con el token de acceso.	201 con sessionId y sessionToken; se crea el registro kyc_verification en pending y el caché del usuario pasa a pending. Con teléfono sin verificar o KYC ya aprobado: 409 KYC_SESSION_NOT_ALLOWED.	`kyc.session.integration.test.ts`	APROBADO
CP-7.07	5, 6, 7	**Resolución del KYC por webhook** *Pre:* Sesión KYC en pending. *Pasos:* Didit envía el webhook firmado con estado Approved, Declined o In Review.	El usuario pasa a approved, rejected o manual_review; GET /kyc/status refleja el estado y, en revisión manual, el motivo. Un webhook duplicado no cambia nada.	`kyc.webhook.integration.test.ts` `kyc.status.integration.test.ts`	APROBADO
CP-7.08	5	**Webhook con firma inválida o fuera de ventana** *Pre:* Sesión KYC en pending. *Pasos:* Enviar el webhook con firma errónea, sin cabeceras, o con timestamp a más de 300 s.	401 KYC_WEBHOOK_INVALID_SIGNATURE y ningún cambio de estado (anti-replay).	`didit-signature.test.ts` `kyc.webhook.integration.test.ts`	APROBADO
CP-7.09	8	**No se almacenan documentos ni biometría** *Pre:* Sesión KYC resuelta. *Pasos:* Inspeccionar lo persistido tras el webhook.	Solo se guardan el estado, el identificador de sesión y la decisión redactada (advertencias); nunca imágenes ni datos biométricos.	`kyc.webhook.integration.test.ts`	APROBADO

2.2.2 MOVO-63 · Inicio de sesión y gestión de la sesión

MOVO-63 COMPLETADA SPRINT 2, SPRINT 6 9 SP

Como usuario registrado, quiero iniciar sesión en la aplicación con mis credenciales, para acceder a mi cuenta y operar en la plataforma sin tener que registrarme nuevamente.

> **Decisión de diseño.** El access token dura 60 minutos y es un JWT; el refresh token es opaco, de un solo uso y vive 90 días en Redis (ADR-004 y ADR-013). Si un refresh token ya canjeado se vuelve a presentar, se interpreta como robo y se revocan todas las sesiones del usuario. El inicio de sesión usa el teléfono como identificador.

Criterios de aceptación

1. El usuario puede iniciar sesión ingresando email/teléfono y contraseña.
2. El sistema valida las credenciales contra el usuario registrado.
3. Si las credenciales son incorrectas, el sistema muestra un error claro sin especificar si el email existe o no (seguridad).
4. Al iniciar sesión correctamente, el sistema genera un access token y un refresh token, almacenando el estado de sesión en Redis.

5. Si el usuario no completó el KYC, el login lo redirige al flujo de verificación pendiente en lugar de al home.
6. El sistema maneja la expiración del access token renovándolo automáticamente vía refresh token mientras la sesión sea válida.
7. Si el refresh token expira o es inválido, el usuario es redirigido al login.

Trazabilidad de implementación

Tabla 2.4. Tickets de implementación de MOVO-63.

Ticket	Alcance	Sprint	Estado
`MOVO-74`	[svc-users] Endpoint de login: validación de credenciales y emisión de access + refresh token	Sprint 2	Completada
`MOVO-75`	[svc-users] Endpoints de refresh token con rotación y de logout con revocación en Redis	Sprint 2	Completada
`MOVO-76`	[movo-mobile] Pantalla de login, secure storage de tokens, refresh automático y guard de navegación	Sprint 2	Completada
`MOVO-247`	[movo-mobile] Splash screen (prototipo Claude Design)	Sprint 6	Completada

Diagrama de secuencia

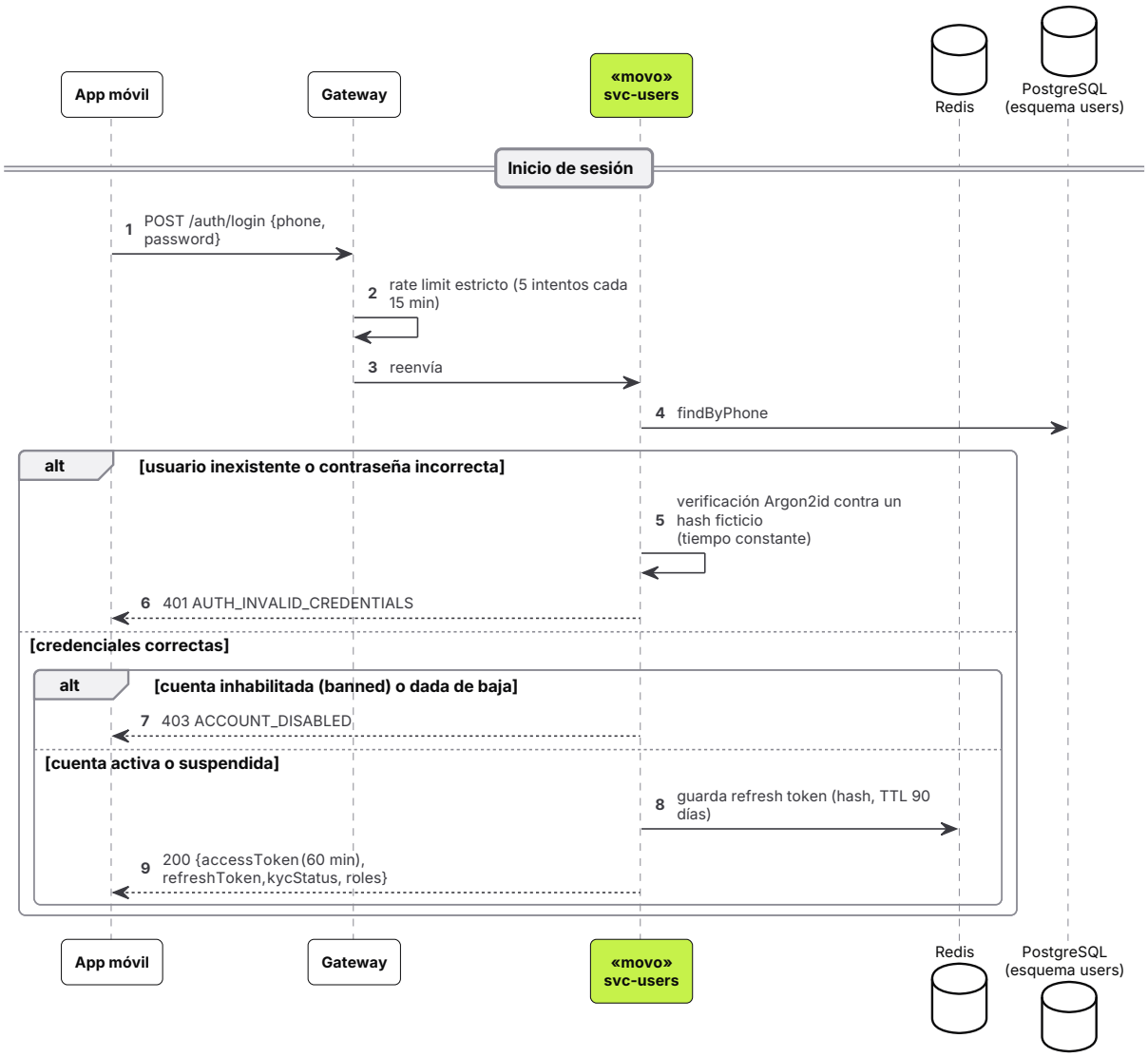


Figura 2.5. Inicio de sesión, uso del token de acceso y renovación con rotación (MOVO-63, MOVO-74, MOVO-75). Parte 1 de 3: Inicio de sesión.

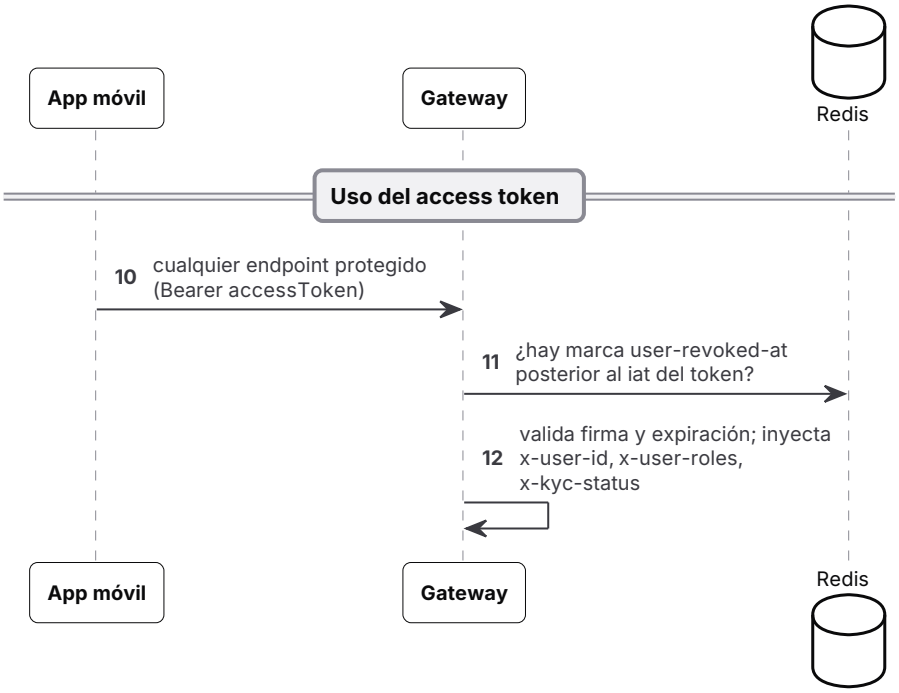


Figura 2.6. Inicio de sesión, uso del token de acceso y renovación con rotación (MOVO-63, MOVO-74, MOVO-75). Parte 2 de 3: Uso del access token.

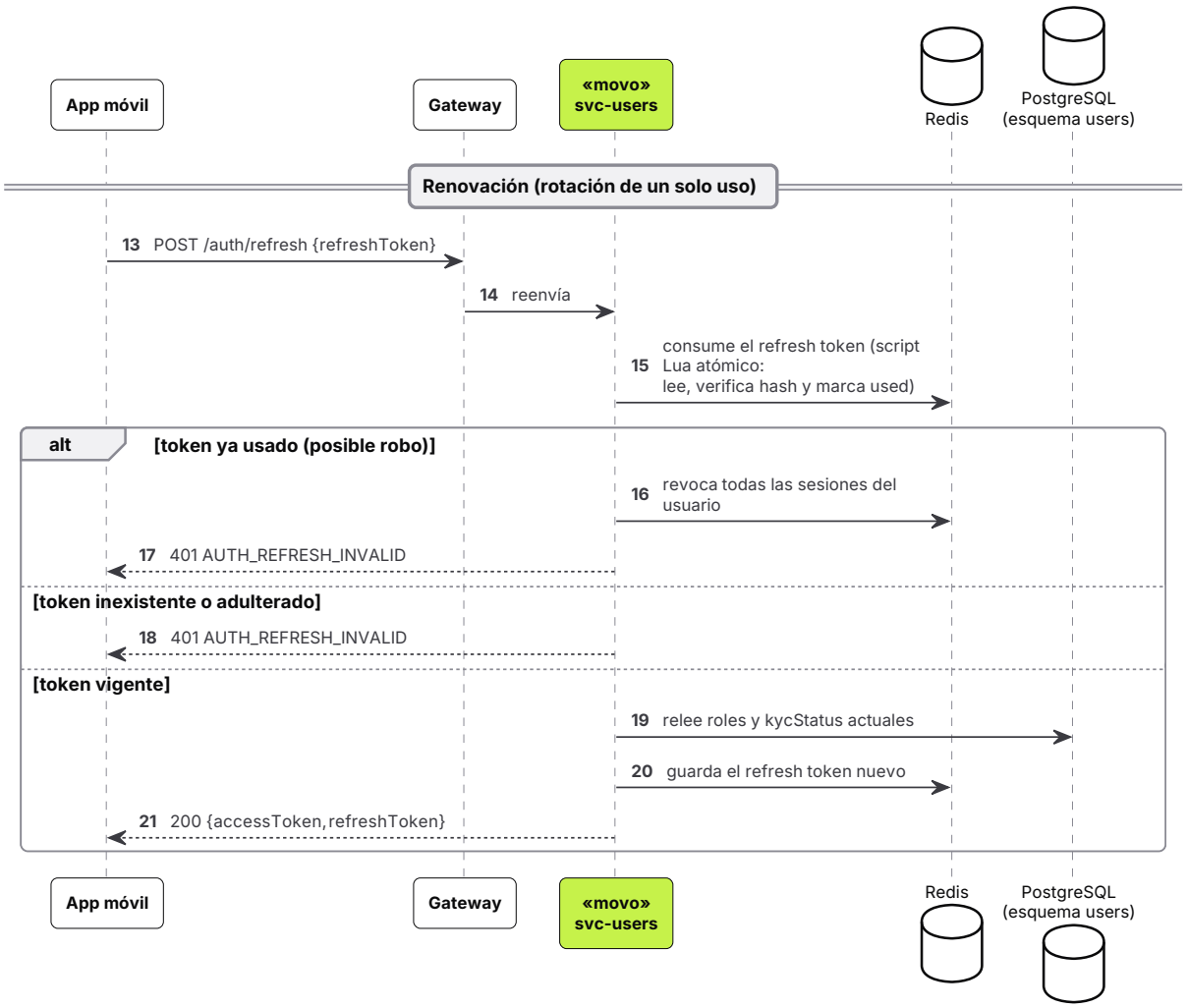


Figura 2.7. Inicio de sesión, uso del token de acceso y renovación con rotación (MOVO-63, MOVO-74, MOVO-75). Parte 3 de 3: Renovación (rotación de un solo uso).

Casos de prueba

Tabla 2.5. Casos de prueba de MOVO-63 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-63.01	1, 2, 4	**Login con credenciales válidas** *Pre:* Usuario registrado y activo. *Pasos:* POST /auth/login con teléfono y contraseña correctos.	200 con accessToken, refreshToken, kycStatus y roles; el refresh token queda en Redis con TTL de 90 días.	`auth.login.integration.test.ts`	APROBADO
CP-63.02	3	**Credenciales inválidas sin revelar si la cuenta existe** *Pre:* Un usuario existente y un teléfono sin cuenta. *Pasos:* Login con contraseña incorrecta y con un teléfono inexistente.	Ambos responden 401 AUTH_INVALID_CREDENTIALS con la misma forma; la verificación corre contra un hash ficticio para igualar el tiempo.	`auth.login.integration.test.ts`	APROBADO
CP-63.03	5	**Usuario con KYC pendiente puede iniciar sesión** *Pre:* Usuario con kycStatus pending. *Pasos:* Login.	200; la app lo lleva al flujo de verificación pendiente en lugar del home.	`auth.login.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-63.04	4	**Cuenta inhabilitada o suspendida** *Pre:* Cuenta con estado banned; otra con estado suspended. *Pasos:* Login con cada una.	banned: 403 ACCOUNT_DISABLED. suspended: el login es exitoso y el gateway bloquea después las operaciones.	`auth.login.integration.test.ts`	APROBADO
CP-63.05	6	**Renovación del token con rotación** *Pre:* Refresh token vigente. *Pasos:* POST /auth/refresh.	200 con un par de tokens nuevo; roles y kycStatus se releen de la base; el refresh anterior queda inutilizable.	`auth.refresh.integration.test.ts`	APROBADO
CP-63.06	7	**Reuso de un refresh token y token inválido** *Pre:* Refresh token ya canjeado, adulterado o inexistente. *Pasos:* POST /auth/refresh con cada uno; además, dos refresh concurrentes con el mismo token.	401 AUTH_REFRESH_INVALID; el reuso revoca todas las sesiones del usuario; con concurrencia solo uno rota (script Lua atómico).	`auth.refresh.integration.test.ts`	APROBADO
CP-63.07	4	**Cierre de sesión** *Pre:* Usuario con dos sesiones abiertas. *Pasos:* POST /auth/logout con el refresh de una; luego POST /auth/logout-all.	204; logout revoca solo esa sesión y es idempotente; logout-all revoca todas.	`auth.logout.integration.test.ts`	APROBADO
CP-63.08	2	**El gateway autentica y no confía en la identidad del cliente** *Pre:* Gateway en ejecución. *Pasos:* Llamar a una ruta protegida sin token, con token vencido, malformado y con un x-user-id falsificado.	401 AUTH_TOKEN_INVALID / AUTH_TOKEN_EXPIRED; el x-user-id del cliente se descarta y el gateway inyecta el verdadero.	`auth.test.ts` `auth-plugin.test.ts`	APROBADO

2.2.3 MOVO-64 · Recuperación de contraseña

MOVO-64 COMPLETADA SPRINT 4 5 SP

Como usuario registrado, quiero recuperar el acceso a mi cuenta si olvido mi contraseña, para poder volver a operar en la plataforma sin necesidad de crear una cuenta nueva.

Criterios de aceptación

1. Desde la pantalla de login, un enlace **"¿Olvidaste tu contraseña?"** lleva a un flujo de recuperación dentro de la app (hoy es un texto inerte que dice "Recuperar contraseña (próximamente)", `movo-mobile/app/(auth)/login.tsx:161`).
2. El usuario se identifica con **su teléfono o su email**, indistintamente, en un único campo.
3. Recibe un código de 6 dígitos por el canal que usó para identificarse, con la misma vigencia y las mismas reglas de reenvío que el OTP del registro (10 min, reenvío con cooldown de 60s, 5 intentos).
4. Con el código válido, define una contraseña nueva sujeta a **las mismas reglas de política que el registro**.
5. **El flujo nunca revela si una cuenta existe**: la respuesta es indistinguible -en status, forma y latencia- para un identificador registrado y uno que no. Una cuenta suspendida o dada de baja se comporta como inexistente.

6. Al completar el cambio se cierran **todas** las sesiones activas del usuario, en todos sus dispositivos.
7. El usuario recibe un aviso de que su contraseña fue cambiada, con indicación de contactar a soporte si no fue él. El aviso va **por todos los canales verificados de la cuenta** (SMS siempre; email además, si está verificado), no solo por el que se usó para recuperar: es lo que le permite al dueño real enterarse de un intento ajeno.
8. Al terminar con éxito el usuario **no** queda logueado automáticamente: vuelve al login con un aviso de éxito. Es coherente con el punto 6 y evita emitir tokens en un flujo sin segundo factor real.
9. Si el proceso se vence a mitad de camino, la app lo explica y permite volver a pedir un código desde cero.

Trazabilidad de implementación

Tabla 2.6. Tickets de implementación de MOVO-64.

Ticket	Alcance	Sprint	Estado
`MOVO-140`	[svc-users] Recuperación de contraseña: OTP por SMS o email, señuelo anti-enumeración y passwordResetToken single-use	Sprint 4	Completada
`MOVO-141`	[movo-mobile] Pantalla “¿Olvidaste tu contraseña?”: wizard de 3 pasos con OTP por SMS o email	Sprint 4	Completada

Diagrama de secuencia

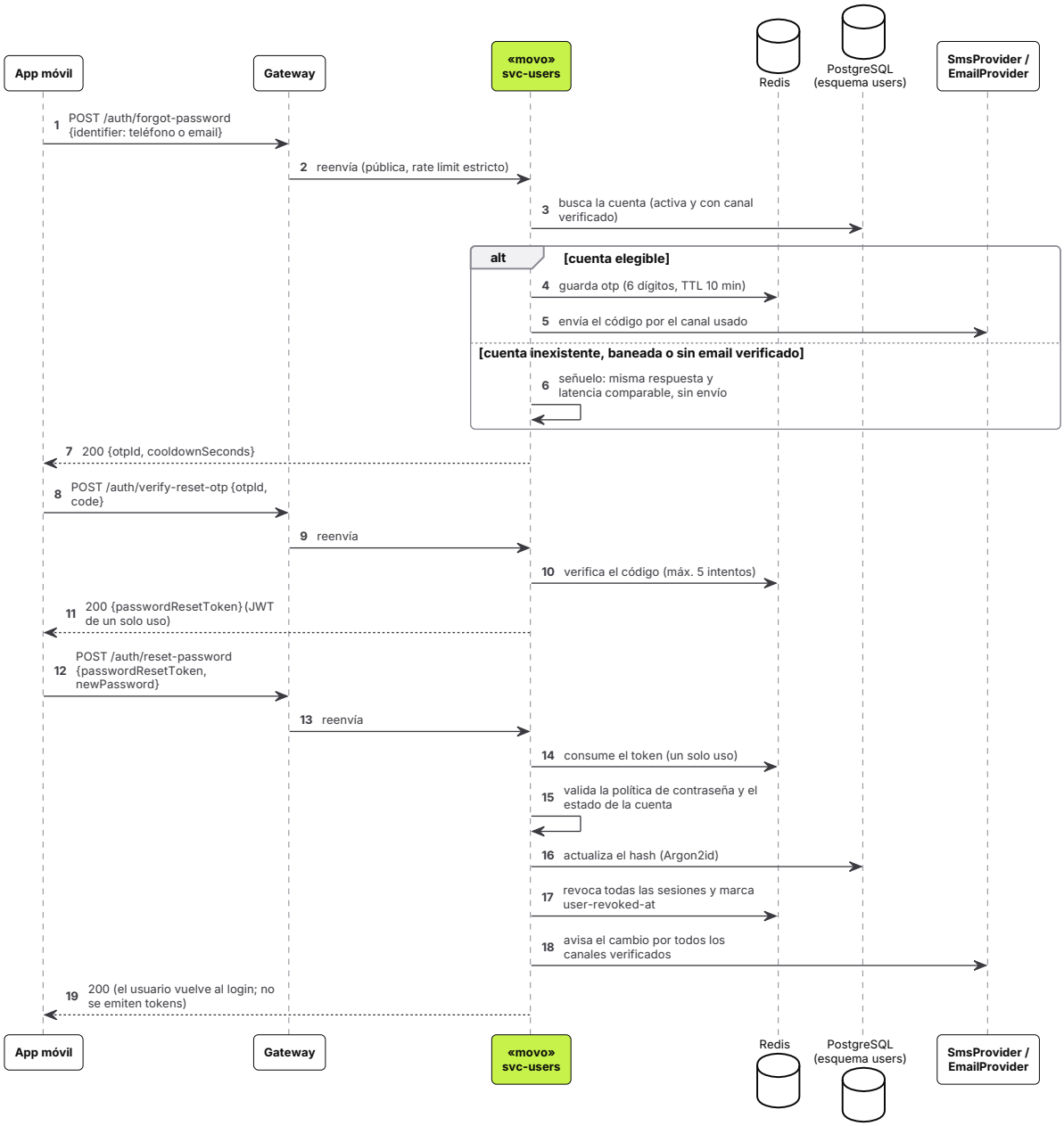


Figura 2.8. Recuperación de contraseña por código (MOVO-64, MOVO-140).

Casos de prueba

Tabla 2.7. Casos de prueba de MOVO-64 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-64.01	1, 2, 3	**Pedido del código por teléfono o por email** *Pre:* Cuenta activa con teléfono (o email verificado). *Pasos:* POST /auth/forgot-password con el teléfono o el email.	200 {otpId, cooldownSeconds}; se envía un código de 6 dígitos por el canal usado.	`auth.password-reset.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-64.02	5	**La respuesta no revela si la cuenta existe** *Pre:* Identificadores de una cuenta real, de una inexistente, de una baneada y de un email sin verificar. *Pasos:* POST /auth/forgot-password con cada uno.	Misma respuesta (estado y forma) en todos; solo la cuenta elegible recibe el código.	auth.password-reset.integration.test.ts	APROBADO
CP-64.03	4	**Definir la contraseña nueva** *Pre:* Código verificado con un passwordResetToken vigente. *Pasos:* POST /auth/reset-password con una contraseña que cumple la política, y otra que no.	Cumple: 200 y el hash cambia. No cumple: 400 VALIDATION_FAILED.	auth.password-reset.integration.test.ts	APROBADO
CP-64.04	6, 8	**Cierre de todas las sesiones y sin login automático** *Pre:* Usuario con sesiones abiertas en dos dispositivos. *Pasos:* Completar el reseteo.	Se revocan todas las sesiones y los access tokens previos; no se emiten tokens nuevos (el usuario vuelve al login).	auth.password-reset.integration.test.ts	APROBADO
CP-64.05	9	**Token vencido o ya usado** *Pre:* passwordResetToken ya consumido o vencido. *Pasos:* POST /auth/reset-password.	401 AUTH_OTP_INVALID; la app explica el vencimiento y permite pedir un código desde cero.	auth.password-reset.integration.test.ts	APROBADO

2.2.4 MOVO-14 · Perfil del usuario

MOVO-14 COMPLETADA SPRINT 2, SPRINT 3 8 SP

Como usuario registrado, quiero ver mi perfil para ver mi información.

Criterios de aceptación

1. El usuario puede acceder a su perfil desde el menu principal.
2. El perfil muestra: nombre, foto (si cargada), email, score de reputacion, cantidad de transacciones y estado de verificacion KYC.
3. El perfil diferencia lo que es visible publicamente de lo que es solo visible para el propio usuario.
4. El estado de verificacion KYC es visible (verificado / pendiente / no verificado).
5. El perfil muestra las insignias activas del usuario (KYC, licencia de conducir si aplica).

Trazabilidad de implementación

Tabla 2.8. Tickets de implementación de MOVO-14.

Ticket	Alcance	Sprint	Estado
MOVO-77	[svc-users] Endpoints GET /users/me y GET /users/:id con proyecciones separadas privada y pública	Sprint 2	Completada
MOVO-78	[movo-mobile] Pantalla de perfil propio con estado de KYC, insignias y logout	Sprint 3	Completada

Casos de prueba

Tabla 2.9. Casos de prueba de MOVO-14 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-14.01	1, 2, 3	**Perfil propio completo y perfil público acotado** *Pre:* Usuario autenticado y otro usuario. *Pasos:* GET /users/me y GET /users/:id del otro usuario.	El perfil propio incluye email y teléfono; el público omite email y teléfono y expone solo la proyección pública (nombre, foto, reputación, insignias, estado de la cuenta).	`users.profile.integration.test.ts`	APROBADO
CP-14.02	4, 5	**Estado de verificación e insignias** *Pre:* Usuarios con KYC de identidad aprobado, con licencia aprobada, con ambos y con ninguno. *Pasos:* GET /users/me de cada uno.	Aparecen las insignias kyc_verified y license_verified según corresponda; sin KYC aprobado, isVerified es falso y badges está vacío.	`users.profile.integration.test.ts`	APROBADO
CP-14.03	1	**Acceso sin identidad o con usuario inexistente** *Pre:* Sin header x-user-id, o con un id que no existe. *Pasos:* GET /users/me.	401 AUTH_TOKEN_INVALID o 404 USER_NOT_FOUND.	`users.profile.integration.test.ts`	APROBADO

2.2.5 MOVO-31 · Edición de datos personales y de la cuenta

MOVO-31 COMPLETADA SPRINT 3, SPRINT 4 13 SP

Como usuario registrado, quiero editar mis datos personales y de contacto en mi perfil, para mantener mi información actualizada dentro de la plataforma.

Criterios de aceptación

1. El usuario puede editar su **nombre y apellido**, salvo que su KYC de identidad esté aprobado, en ese caso quedan bloqueados, porque fueron validados contra su documento por Didit.
2. El usuario puede cambiar su **foto de perfil**. *(Ya implementado por* MOVO-97/98*, se reusa, no se rehace.)*
3. El usuario puede cambiar su **número de teléfono**, y el sistema envía un código de verificación **al número nuevo**: el cambio no se aplica hasta que ese código se verifica.
4. El usuario puede cambiar su **email**, y el cambio exige una verificación previa: se envía un código **al teléfono ya verificado de la cuenta**. *(Ver "Decisiones de refinamiento", el AC original pedía verificar el email nuevo por email.)*
5. El usuario **no puede editar su estado de KYC** directamente por ningún camino: la API rechaza el campo y la única forma de cambiarlo es reiniciar la verificación.
6. Un teléfono o email que ya pertenece a otra cuenta se rechaza con un mensaje claro, sin dejar la pantalla en un estado roto.
7. Todo cambio guardado tiene **confirmación visual** explícita, y el perfil muestra el dato actualizado al volver.
8. El usuario puede **cambiar su contraseña** ingresando la actual; al hacerlo se cierran las sesiones de los demás dispositivos.

9. El usuario puede **dar de baja su cuenta** confirmando con su contraseña; la acción es irreversible y anonimiza sus datos personales.

Trazabilidad de implementación

Tabla 2.10. Tickets de implementación de MOVO-31.

Ticket	Alcance	Sprint	Estado
`MOVO-133`	[svc-users] PATCH /users/me y cambio verificado de teléfono y email por OTP	Sprint 3	Completada
`MOVO-134`	[svc-users] Cambio de contraseña y baja de cuenta (Cuenta y seguridad)	Sprint 3	Completada
`MOVO-135`	[movo-mobile] Pantalla “Editar perfil” con cambio verificado de teléfono y email	Sprint 4	Completada
`MOVO-136`	[movo-mobile] Pantalla “Cuenta y seguridad”: cambiar contraseña y dar de baja la cuenta	Sprint 3	Completada
`MOVO-139`	[svc-users] EmailProvider (Resend) y verificación de email por OTP	Sprint 3	Completada

Casos de prueba

Tabla 2.11. Casos de prueba de MOVO-31 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
`CP-31.01`	1	**Editar nombre y apellido** *Pre:* Usuario con KYC sin aprobar. *Pasos:* PATCH /users/me con firstName y lastName.	200 con el perfil actualizado. Con KYC de identidad aprobado y un nombre distinto: 409 PROFILE_NAME_LOCKED_BY_KYC.	`users.profile-edit.integration.test.ts`	APROBADO
`CP-31.02`	5	**Campos no editables** *Pre:* Usuario autenticado. *Pasos:* PATCH /users/me con email, phone, kycStatus, roles o photoUrl.	400 VALIDATION_FAILED sin ningún efecto.	`users.profile-edit.integration.test.ts`	APROBADO
`CP-31.03`	3, 4, 6	**Cambio de teléfono o email con verificación** *Pre:* Usuario con teléfono verificado. *Pasos:* Pedir el código de cambio al valor nuevo, verificarlo; probar con un dato que ya es de otra cuenta.	El cambio se aplica solo tras verificar el código enviado al valor nuevo; un teléfono o email ya registrado se rechaza con 409 PHONE_ALREADY_IN_USE o EMAIL_ALREADY_IN_USE (también si la colisión aparece recién al verificar); al cambiar el email se avisa al anterior.	`users.profile-edit.integration.test.ts`	APROBADO
`CP-31.04`	2	**Foto de perfil** *Pre:* Usuario autenticado. *Pasos:* Pedir URL de subida, subir y confirmar; reemplazar y eliminar la foto.	Se persiste photo_url bajo profile-photos/{userId}/; el reemplazo borra el objeto anterior; DELETE es idempotente. Se rechazan tipos fuera de la lista y archivos mayores a 5 MB.	`users.photo.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-31.05	8	**Cambio de contraseña** *Pre:* Usuario con dos sesiones. *Pasos:* POST /users/me/password con la actual correcta, y otra vez con la actual incorrecta.	Correcta: la contraseña cambia, las demás sesiones se cierran y se devuelve un par nuevo. Incorrecta: 401 sin efectos. A la sexta llamada en la ventana: 429.	`users.account-settings.integration.test.ts`	APROBADO

2.2.6 MOVO-15 · Verificación de la licencia de conducir del transportista

MOVO-15 COMPLETADA SPRINT 3 3 SP

Como transportista quiero verificar mi licencia de conducir, para aumentar mi perfil de confianza y obtener la insignia de transportista verificado.

Refinamiento. Los criterios de la tarjeta de pago de la versión original se removieron: estaban duplicados con MOVO-65 (método de pago del emisor) y, para el transportista, el cobro se hace por Split Payment sobre su cuenta de Mercado Pago (MOVO-110). La verificación se delega a Didit.me con un *workflow* propio para licencias (`DIDIT_WORKFLOW_ID_LICENSE`).

Criterios de aceptación

1. El usuario puede subir la foto de su licencia de conducir desde la app (frente).
2. El sistema verifica que la imagen sea legible antes de procesarla.
3. Al completar la verificación, el perfil muestra la insignia «Transportista verificado».
4. Un intento de verificación de licencia es independiente del KYC de identidad.

Casos de prueba

Tabla 2.12. Casos de prueba de MOVO-15 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-15.01	1	**Crear la sesión de licencia** *Pre:* Usuario con teléfono verificado. *Pasos:* POST /kyc/license/session.	201; se inserta una verificación de tipo license y kyc_status_license pasa a pending sin tocar kyc_status_identity.	`kyc.license.integration.test.ts`	APROBADO
CP-15.02	4	**Independencia respecto del KYC de identidad** *Pre:* Identidad en pending o not_started. *Pasos:* Abrir una sesión de licencia con identidad pendiente; abrirla con la licencia ya aprobada.	Con identidad pending se permite; con la licencia approved: 409 KYC_SESSION_NOT_ALLOWED.	`kyc.license.integration.test.ts`	APROBADO
CP-15.03	3	**Licencia aprobada habilita la insignia** *Pre:* Sesión de licencia en pending. *Pasos:* Webhook Approved de tipo license con fecha de vencimiento.	kyc_status_license pasa a approved, se crea el registro drivers_license (con o sin vencimiento) y el perfil muestra license_verified.	`kyc.webhook.integration.test.ts` `users.profile.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-15.04	4	**Estado de la licencia en revisión manual** *Pre:* Licencia en manual_review. *Pasos:* GET /kyc/license/status.	Devuelve el estado y el motivo del intento de licencia más reciente, no el de identidad.	`kyc.license.integration.test.ts`	APROBADO

2.2.7 MOVO-39 · Baja de la cuenta

MOVO-39 COMPLETADA SPRINT 4 2 SP

Como usuario registrado, quiero solicitar la baja de mi cuenta para ejercer mi derecho de supresión conforme a la normativa de protección de datos vigente.

Criterios de aceptación

1. El usuario puede solicitar la baja de su cuenta.
2. El sistema no permite la baja si el usuario tiene envíos activos o disputas abiertas.
3. El usuario debe confirmar la decision con un paso adicional (ingreso de contraseña o confirmación por email).
4. Tras confirmar, el sistema elimina los datos de identificación personal del usuario.
5. Los registros de transacciones completadas se anonimizan pero no se eliminan (auditoria).
6. El usuario recibe confirmación por email de que la baja fue procesada.
7. La baja es irreversible: el usuario no puede recuperar la cuenta luego de confirmar.

Casos de prueba

Tabla 2.13. Casos de prueba de MOVO-39 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-39.01	1, 3	**Baja con confirmación de contraseña** *Pre:* Usuario sin envíos activos ni disputas abiertas. *Pasos:* DELETE /users/me con la contraseña correcta.	La cuenta queda en estado deleted, anonimizada y sin sesiones; los registros de KYC y licencia se eliminan.	`users.account-settings.integration.test.ts`	APROBADO
CP-39.02	2	**La baja se rechaza con operaciones en curso** *Pre:* Usuario con un envío activo o una disputa abierta. *Pasos:* DELETE /users/me.	409 ACCOUNT_HAS_ACTIVE_DISPUTES o ACCOUNT_HAS_ACTIVE_SHIPMENTS; la cuenta no se modifica. Una baja ya en curso responde 409 ACCOUNT_DELETION_IN_PROGRESS.	`users.account-settings.integration.test.ts` `users.deletion-status.integration.test.ts`	APROBADO
CP-39.03	5, 7	**Anonimización irreversible** *Pre:* Cuenta dada de baja. *Pasos:* Intentar iniciar sesión con la cuenta eliminada y registrar una cuenta nueva con el mismo email y teléfono.	El login responde 403; la cuenta no se recupera; el email y el teléfono quedan liberados; se borran la foto, los datos de KYC y licencia y la clave de dispositivo; la baja es idempotente (204).	`users.account-settings.integration.test.ts`	APROBADO

2.2.8 MOVO-32 · Revisión de casos de KYC en revisión manual (procedimiento operativo)

MOVO-32 BACKLOG SIN ASIGNAR

Como administrador, quiero revisar y aprobar o rechazar los casos de KYC pendientes, para asegurar que solo usuarios verificados operen en la plataforma.

> **Estado: pendiente.** La historia quedó en el *backlog*. Su resolución técnica ya existe (el webhook de Didit aplica la decisión manual, MOVO-72), pero falta el procedimiento operativo documentado y el acceso restringido a la consola. Los casos siguientes son los **previstos** a partir de los criterios de aceptación.

Criterios de aceptación

1. **Procedimiento operativo documentado** (`docs/operations/kyc-revision-manual.md` o equivalente): quién revisa, cómo se accede a la consola de Didit, qué mirar en cada caso y cómo dejar el motivo de rechazo para que le llegue al usuario.
2. **Acceso a la consola restringido** a los admins designados. La consola muestra imágenes del documento y biométricas, así que el acceso se trata como dato sensible: lista nominal de quién tiene acceso, sin cuentas compartidas. La Política de Privacidad tiene que reflejar que el proveedor y personal autorizado revisan esos casos.
3. **La decisión manual se aplica en Movo:** cuando un caso en revisión manual se aprueba o rechaza en la consola, el webhook de `MOVO-72` actualiza el estado cacheado (`kyc_status_identity` o `kyc_status_license`) y agrega una fila nueva en `KYC_VERIFICATION` (append-only). Verificar con un caso real de sandbox que Didit manda el webhook al cambiar el estado desde la consola, y con qué payload. Si no lo manda, definir el mecanismo alternativo (por ejemplo, consulta del estado de la sesión) antes de implementar.
4. **Notificación al usuario:** al aprobarse, push y el usuario queda habilitado para operar. Al rechazarse, push con el motivo. `MOVO-274` ya cubre el push de resultado; verificar que el motivo del rechazo manual llegue y se muestre en la app.
5. **Sin apelación:** si se rechaza, el usuario **puede reintentar** el KYC desde la app. La pantalla de rechazo explica el motivo y ofrece el reintento.
6. Mientras el caso está en revisión manual, la app muestra el estado “En revisión” (ya existe para identidad; confirmar que también aplica a la licencia).

Casos de prueba

Tabla 2.14. Casos de prueba de MOVO-32 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-32.01	3	**La decisión manual se refleja en Movo** *Pre:* Usuario en manual_review. *Pasos:* Un administrador aprueba o rechaza el caso en la consola de Didit.	El webhook actualiza kyc_status_identity y el usuario queda habilitado o rechazado.	`kyc.webhook.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-32.02	4, 5	**Notificación y reintento** *Pre:* Caso resuelto manualmente. *Pasos:* Aprobar y, en otro caso, rechazar.	Push de resultado al usuario; tras un rechazo la app explica el motivo y permite reintentar.	`kyc.push.test.ts`	APROBADO
CP-32.03	1, 2	**Procedimiento operativo y acceso restringido** *Pre:* Administradores designados. *Pasos:* Revisar el procedimiento documentado y la lista de accesos a la consola.	Existe el procedimiento y solo acceden los administradores designados.	–	PENDIENTE

03

TEMA 2 DE 7 · TRAZABILIDAD DE HISTORIAS DE USUARIO

EP-02 · Publicación y Descubrimiento

Cotización y creación de envíos, confirmación del receptor, viajes del transportista, feed de paquetes disponibles, cancelación e historial.

CONTENIDO

3.1	Descripción del Tema	53
3.2	Historias de Usuario y Trazabilidad	54

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: trazabilidad de las historias del tema, con criterios de aceptación de Linear, diagramas de secuencia derivados del código y casos de prueba derivados de las pruebas automatizadas.

Introducción

Este documento presenta la trazabilidad de las historias de usuario del tema *Publicación y Descubrimiento*: para cada historia, su descripción y criterios de aceptación, los tickets que la implementaron, el diagrama de secuencia que muestra cómo la resuelve el sistema y los casos de prueba que la verifican.

Audiencia

El tribunal evaluador y los futuros mantenedores. Se asume conocimiento de las convenciones del proyecto y de UML.

3.1 Descripción del Tema

EP-02 · Publicación y Descubrimiento cubre el lado de la oferta y la demanda antes de que exista un acuerdo: el emisor cotiza y publica un envío, el receptor lo confirma o lo rechaza, el transportista declara sus viajes y descubre los paquetes compatibles con su ruta, y cualquiera de las partes puede consultar sus envíos activos y su historial o cancelar lo que aún no se ejecutó.

El envío nace en `awaiting_receiver_confirmation` y recién pasa a `published`, visible para los transportistas, cuando el receptor lo acepta. El precio sugerido proviene de `movo-svc-pricing-logistics` y se congela por 15 minutos (DD-028) para que el emisor pague lo que vio. La negociación y la asignación siguen en el tema *Asignación y Negociación*.

Servicios involucrados: `gateway`, `movo-svc-shipments` (envíos y viajes), `movo-svc-pricing-logistics` (cotización y compatibilidad de rutas), `movo-svc-users` (validación del receptor) y `movo-mobile`; **externos:** Google Routes API (distancias, ADR-015), proveedor de precios de combustible y Amazon S3 (fotos del paquete, ADR-007).

3.2 Historias de Usuario y Trazabilidad

Tabla 3.1. Historias de usuario del tema.

Historia	Descripción	Estado	Sprint
MOVO-8	Creación de un envío	Completada	Sprint 3
MOVO-16	Confirmación o rechazo del envío por el receptor	Completada	Sprint 3, Sprint 4
MOVO-18	Declaración y gestión de viajes	Completada	Sprint 4, Sprint 5, Sprint 6
MOVO-19	Feed de paquetes disponibles	Completada	Sprint 4, Sprint 5
MOVO-29	Cancelación de un envío por el emisor	Completada	Sprint 3
MOVO-27	Historial de envíos	Completada	Sprint 3, Sprint 6
MOVO-191	Envíos activos en el inicio de la app	Completada	Sprint 5, Sprint 7
MOVO-259	Rediseño de “Mis viajes” para el transportista	Completada	Sprint 6, Sprint 7
MOVO-138	Motor de precios dinámico	Completada	Sprint 6

3.2.1 MOVO-8 · Creación de un envío

MOVO-8 COMPLETADA SPRINT 3 32 SP

Como emisor, quiero crear un envio para iniciar el proceso de envio de forma clara y estructurada

Decisión de diseño. El precio sugerido se cotiza antes de confirmar y queda congelado 15 minutos en Redis bajo un `quoteId` de un solo uso, atado al usuario y a los datos que afectan el precio (DD-028). Si el servicio de precios falla, el envío se crea igual con “precio a estimar”. El paquete tiene límites duros de 30 kg, 150 cm por lado y 300 cm de suma de lados (MOVO-276). El envío nace en `awaiting_receiver_confirmation` y no llega al mercado hasta que el receptor lo acepta.

Criterios de aceptación

1. El emisor puede iniciar la creacion de un envio desde el tab ‘Enviar’.
2. El formulario solicita: tipo de paquete, dimensiones, peso aproximado, descripcion opcional.
3. El emisor identificar al receptor buscando por nombre de usuario registrado en Movo.
4. El emisor define fecha y franja horaria en la que el paquete puede ser retirado.
5. El emisor puede tomar 2 fotos del paquete al crear el envio (requerido para evidencia).
6. El sistema calcula y muestra el precio sugerido una vez que el emisor completa datos de peso, dimensiones y destino.

7. El precio sugerido se muestra en ARS antes de confirmar la publicacion del envio.
8. El calculo de precio sugerido considera al menos distancia, peso y tipo de paquete.
9. Si alguno de los datos necesarios no esta completo, el sistema muestra 'precio a estimar' en lugar de un valor.
10. El sistema muestra un resumen del envio antes de confirmar la publicación.
11. Al confirmar, el sistema notifica al receptor para que acepte o rechace el envio.
12. El envio no se publica al mercado de transportistas hasta que el receptor lo acepte.
13. El envio queda en estado 'Esperando confirmacion del receptor'.

Trazabilidad de implementación

Tabla 3.2. Tickets de implementación de MOVO-8.

Ticket	Alcance	Sprint	Estado
`MOVO-79`	[svc-shipments] Schema, migraciones y máquina de estados del ciclo de vida del envío	Sprint 3	Completada
`MOVO-80`	[svc-shipments] Endpoint de creación de envío, búsqueda de receptor y consulta de detalle	Sprint 3	Completada
`MOVO-81`	[svc-shipments] Carga de fotos del paquete con presigned URLs de S3 y registro de evidencia	Sprint 3	Completada
`MOVO-82`	[svc-pricing-logistics] Contrato de precio sugerido con implementación lineal provisoria y fallback	Sprint 3	Completada
`MOVO-83`	[movo-mobile] Wizard de creación de envío con carga de fotos y precio sugerido	Sprint 3	Completada
`MOVO-119`	[movo-svc-users] CRUD de direcciones guardadas (/addresses)	Sprint 3	Completada
`MOVO-120`	[movo-svc-users] Proxy de Google Places Autocomplete (/places)	Sprint 3	Completada
`MOVO-121`	[movo-mobile] Pantalla de gestión de direcciones guardadas (Perfil)	Sprint 3	Completada
`MOVO-123`	[movo-svc-shipments] Endpoint de ruta (polyline) origen→destino para el mapa del wizard de envíos	Sprint 3	Completada

Diagrama de secuencia

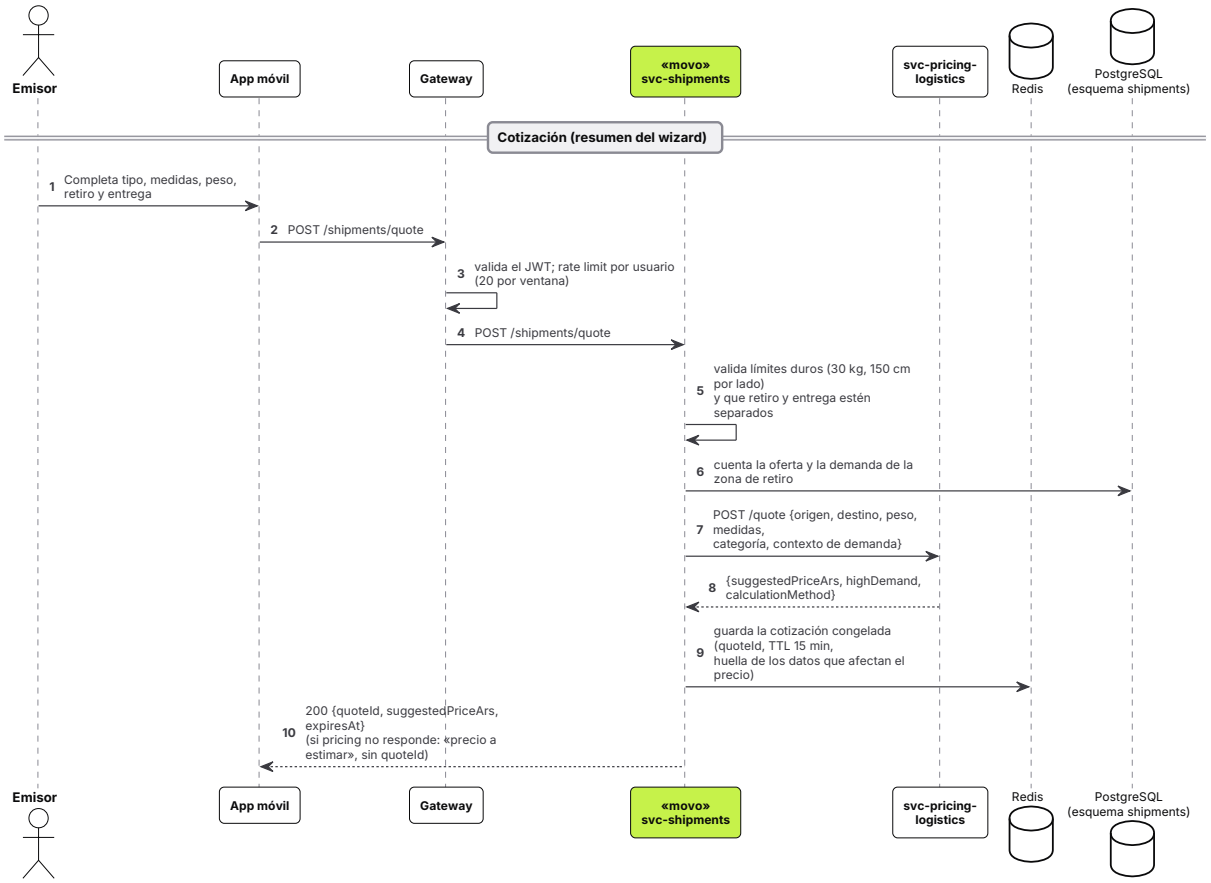


Figura 3.1. Cotización congelada y creación del envío, a la espera del receptor (MOVO-8, MOVO-82, MOVO-83). Parte 1 de 3: Cotización (resumen del wizard).

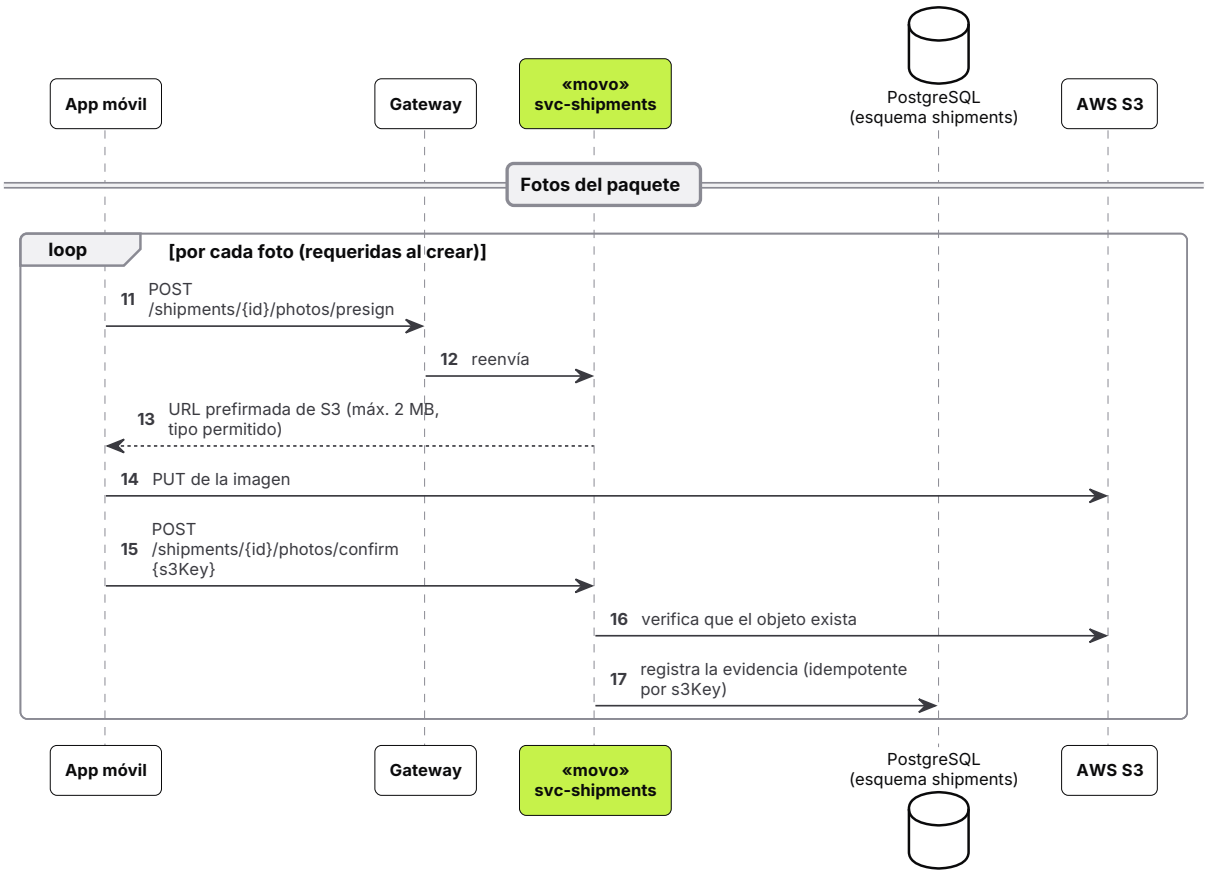


Figura 3.2. Cotización congelada y creación del envío, a la espera del receptor (MOVO-8, MOVO-82, MOVO-83). Parte 2 de 3: Fotos del paquete.

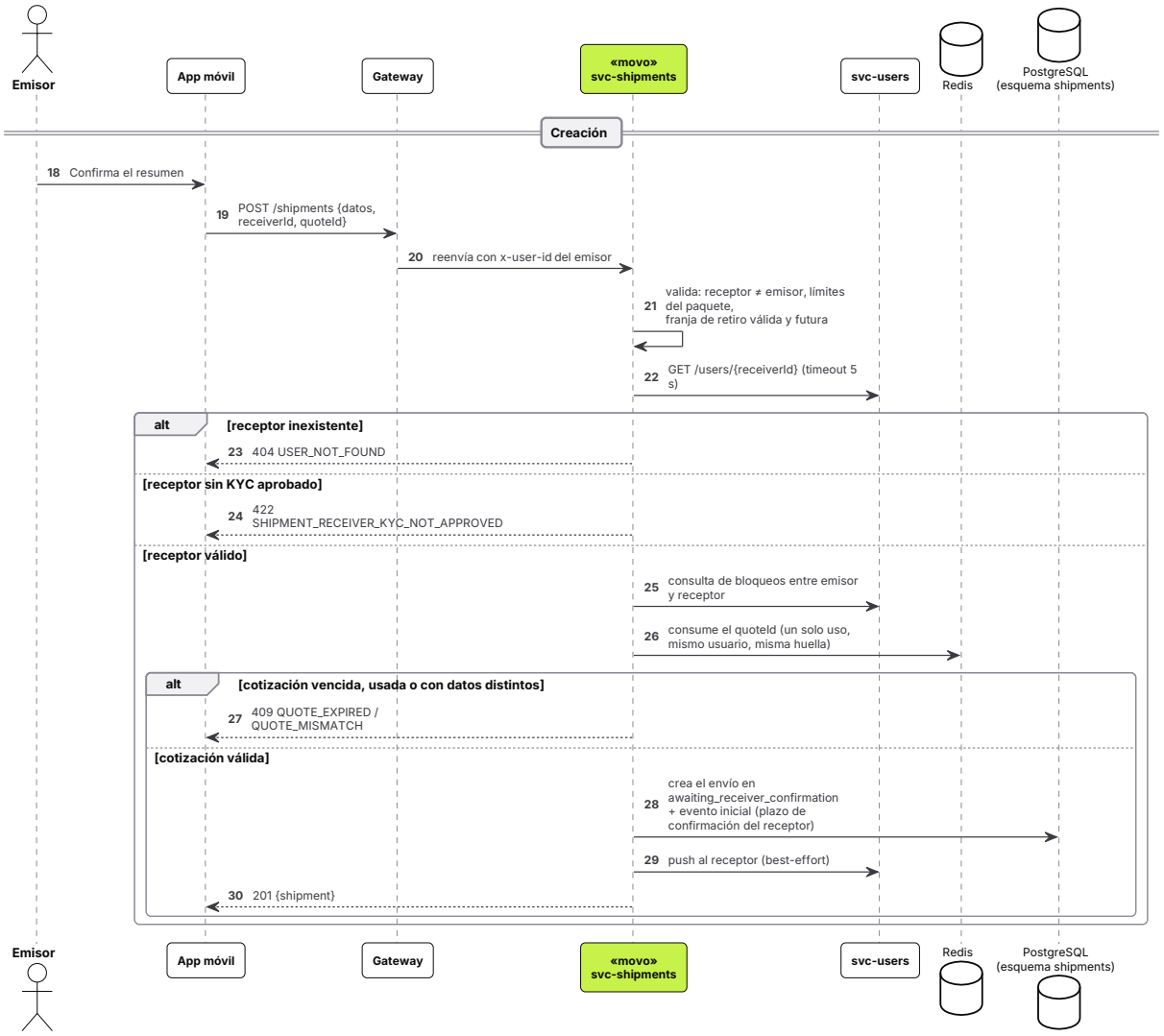


Figura 3.3. Cotización congelada y creación del envío, a la espera del receptor (MOVO-8, MOVO-82, MOVO-83). Parte 3 de 3: Creación.

Casos de prueba

Tabla 3.3. Casos de prueba de MOVO-8 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-8.01	1, 2, 11, 13	**Creación válida** *Pre:* Emisor autenticado y receptor con KYC aprobado. *Pasos:* POST /shipments con paquete, direcciones, franja de retiro y receptor.	201; el envío queda en awaiting_receiver_confirmation con su evento inicial y el receptor recibe la notificación. Si el servicio de notificaciones falla, el envío se crea igual.	`shipments-create.integration.test.ts`	APROBADO
CP-8.02	3	**Validación del receptor** *Pre:* Emisor autenticado. *Pasos:* Crear designándose a sí mismo, a un usuario inexistente y a uno sin KYC aprobado.	422 para el autoenvío y para el receptor sin KYC; 404 para el inexistente. No se crea ningún envío.	`shipments-create.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-8.03	4	**Franja de retiro en el pasado** *Pre:* Emisor autenticado. *Pasos:* Crear con una franja de retiro anterior al momento actual.	422 y ningún envío creado.	`shipments-create.integration.test.ts`	APROBADO
CP-8.04	2	**Límites duros del paquete** *Pre:* Emisor autenticado. *Pasos:* Crear o cotizar un paquete de más de 30 kg, con un lado de más de 150 cm o con suma de lados mayor a 300 cm.	422 con el detalle del límite excedido; al cotizar no se llama al servicio de precios.	`shipments-create.integration.test.ts` `package-limits.test.ts` `shipments-quote.integration.test.ts`	APROBADO
CP-8.05	-	**La identidad del emisor sale del header** *Pre:* Pedido con un senderId falsificado en el body, y otro sin `x-user-id`. *Pasos:* POST /shipments.	El senderId del body se ignora y se usa el del header; sin header, 401.	`shipments-create.integration.test.ts`	APROBADO
CP-8.06	6, 7, 8, 9	**Precio sugerido persistido y degradación** *Pre:* Servicio de precios disponible, y luego caído. *Pasos:* Crear un envío en cada situación.	Con precios disponibles se persisten suggestedPriceArs, calculationMethod y highDemand. Con precios caídos el envío se crea con precio nulo (“precio a estimar”).	`shipments-create.integration.test.ts` `shipment-quote.test.ts`	APROBADO
CP-8.07	6, 7	**Cotización congelada de un solo uso** *Pre:* Cotización vigente de un usuario. *Pasos:* Crear el envío con el quoteId, aunque el precio haya cambiado entre medio; reusar el quoteId; usarlo desde otro usuario o con datos distintos.	Se respeta el precio cotizado sin volver a cotizar. Reuso o vencimiento: 409 QUOTE_EXPIRED. Otro usuario: no puede usarlo y no lo consume. Datos distintos: 409 QUOTE_MISMATCH. Descripción, dirección escrita y franja no invalidan la cotización.	`shipments-quote.integration.test.ts` `quote-store.test.ts`	APROBADO
CP-8.08	5	**Fotos de evidencia del paquete** *Pre:* Envío en creación. *Pasos:* Pedir la URL prefirmada, subir el archivo y confirmar la foto; probar tipo de archivo, tamaño y permisos inválidos.	Se registra la foto (idempotente al confirmar dos veces). Tipo no permitido, tamaño mayor a 2 MB, clave ajena o tercero sin permiso: rechazo. Máximo de fotos por etapa: 422.	`photos.integration.test.ts`	APROBADO

3.2.2 MOVO-16 · Confirmación o rechazo del envío por el receptor

MOVO-16 COMPLETADA SPRINT 3, SPRINT 4 11 SP

Como receptor quiero aceptar o rechazar un envío para evitar recibir paquetes indeseados.

Decisión de diseño. El receptor solo puede aceptar o rechazar, nunca editar. El plazo de respuesta (MOVO-130, 48 horas por defecto) lo hace cumplir un barrido periódico con lock en Redis, de modo que corra una sola réplica. Si el receptor rechaza, el emisor tiene 48 h para designar a otro (DD-027).

Criterios de aceptación

1. El receptor recibe una notificación push cuando un emisor lo designa como destinatario.
2. La notificación muestra: quien es el emisor, descripción del paquete y fecha estimada de retiro.
3. El receptor puede ver el perfil y score de reputacion del emisor antes de decidir.
4. El receptor puede aceptar o rechazar el envio desde la notificacion o desde la app.
5. Si el receptor rechaza, el emisor recibe notificacion y el envio no se publica.
6. Si el receptor no responde en un plazo definido, el envio se cancela automaticamente y el emisor es notificado.
7. El receptor no puede editar los datos del envio, solo aceptar o rechazar.

Trazabilidad de implementación

Tabla 3.4. Tickets de implementación de MOVO-16.

Ticket	Alcance	Sprint	Estado
`MOVO-129`	[svc-shipments] Endpoints de aceptación y rechazo del envío por el receptor	Sprint 3	Completada
`MOVO-130`	[svc-shipments] Expiración automática del envío no confirmado por el receptor (timeout + cancelación)	Sprint 3	Completada
`MOVO-131`	[movo-mobile] Vista de receptor en el detalle del envío: perfil del emisor y acciones de aceptar/rechazar	Sprint 3	Completada
`MOVO-132`	[movo-mobile] Envíos que recibo: distinción de rol en el listado y entrada desde la notificación push	Sprint 4	Completada

Diagrama de secuencia

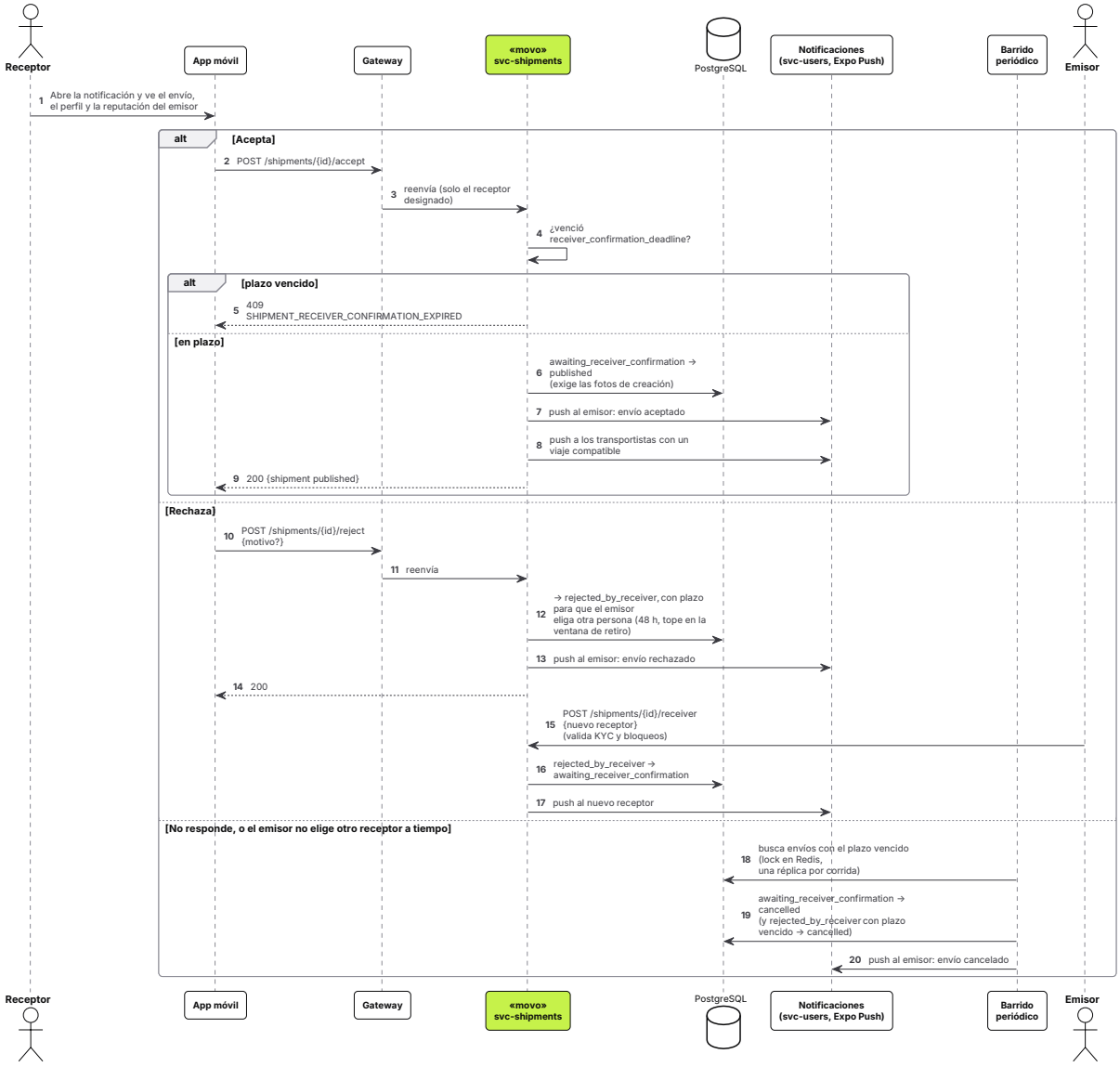


Figura 3.4. Decisión del receptor, plazo de respuesta y nueva designación (MOVO-16, MOVO-130, MOVO-253).

Casos de prueba

Tabla 3.5. Casos de prueba de MOVO-16 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-16.01	4, 7	**El receptor acepta el envío** *Pre:* Envío en awaiting_receiver_confirmation con fotos de creación. *Pasos:* POST /shipments/:id/accept con y sin campos extra en el body.	El envío pasa a published; los campos extra se ignoran. Sin fotos de creación o ya aceptado: 409.	shipments-accept-reject.integration.test.ts	APROBADO
CP-16.02	4, 5	**El receptor rechaza el envío** *Pre:* Envío en awaiting_receiver_confirmation. *Pasos:* POST /shipments/:id/reject con y sin motivo.	El envío pasa a rejected_by_receiver, con el motivo si se informó. Si ya fue cancelado o rechazado: 409.	shipments-accept-reject.integration.test.ts	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-16.03	7	**Solo el receptor decide** *Pre:* Envío esperando confirmación. *Pasos:* Aceptar y rechazar como emisor, como admin y como tercero; con envío inexistente; sin identidad.	403 para emisor, admin y tercero; 404 inexistente; 401 sin x-user-id.	`shipments-accept-reject.integration.test.ts`	APROBADO
CP-16.04	6	**Plazo de respuesta vencido** *Pre:* Envío cuyo plazo de confirmación del receptor ya venció. *Pasos:* Aceptar o rechazar; luego correr el barrido de vencimiento.	409 al responder tarde, aunque el barrido no haya corrido. El barrido cancela el envío y avisa al emisor; no corre si otra réplica tiene el lock.	`shipments-accept-reject.integration.test.ts` `receiver-confirmation-sweep.test.ts` `shipment-expiration.integration.test.ts`	APROBADO
CP-16.05	5	**El emisor elige otro receptor** *Pre:* Envío rechazado, dentro de las 48 h. *Pasos:* POST /shipments/:id/receiver con un receptor nuevo; probar uno que ya rechazó, uno sin KYC, uno bloqueado, uno mismo, plazo vencido y dos elecciones concurrentes.	El envío vuelve a esperar confirmación y el receptor nuevo recibe la notificación; las elecciones inválidas responden 409 o 422 y, ante concurrencia, solo una gana.	`shipments-redesignate-receiver.integration.test.ts`	APROBADO
CP-16.06	5, 6	**Match automático con viajes al publicarse** *Pre:* Transportista con un viaje declarado compatible. *Pasos:* Aceptar el envío.	Se envía exactamente una push trip_match al transportista del viaje compatible; sin viaje compatible, o con viaje no viable según ruteo, no se envía ninguna.	`shipments-accept-reject.integration.test.ts`	APROBADO

3.2.3 MOVO-18 · Declaración y gestión de viajes

MOVO-18 COMPLETADA SPRINT 4, SPRINT 5, SPRINT 6 28 SP

Como transportista quiero declarar un viaje para ver paquetes compatibles con mi ruta.

> **Alcance final.** Además de declarar, editar y cancelar, el viaje tiene ciclo de vida propio (declarado, activo, completado, cancelado, vencido), inicio explícito (MOVO-221), recurrencia semanal (MOVO-265) y vencimiento automático. Solo puede haber un viaje activo por transportista, garantizado por un índice único parcial en la base de datos.

Criterios de aceptación

1. El transportista puede declarar un viaje indicando: origen, destino, fecha y hora de salida, tipo de vehículo.
2. El sistema muestra los paquetes disponibles compatibles con la ruta declarada (dentro de un radio configurable de desvío).
3. El transportista puede editar o cancelar un viaje declarado mientras no tenga paquetes aceptados.

4. Si el transportista cancela un viaje con paquetes ya aceptados, se sigue el flujo de cancelacion con transportista asignado.
5. La declaración de viaje no genera obligación hasta que el transportista acepte un paquete especifico.

Trazabilidad de implementación

Tabla 3.6. Tickets de implementación de MOVO-18.

Ticket	Alcance	Sprint	Estado
MOVO-161	[svc-shipments] CRUD de Viaje declarado + matching de paquetes compatibles por radio de desvío	Sprint 4	Completada
MOVO-162	[movo-mobile] Declarar/editar/cancelar viaje (“Mis viajes”)	Sprint 4	Completada
MOVO-163	[movo-mobile] Feed de paquetes compatibles con el viaje declarado	Sprint 4	Completada
MOVO-179	[svc-shipments] Push notification al publicarse un envío compatible con un viaje declarado	Sprint 5	Completada
MOVO-221	[svc-shipments] [movo-mobile] Rediseño de estados de viaje (declarado/activo/realizado) y límite de 1 viaje activo por transportista	Sprint 5	Completada
MOVO-234	[svc-shipments] Auto-crear Trip al aceptar una oferta sin viaje asociado	Sprint 6	Completada
MOVO-236	[movo-mobile] Aviso de viaje auto-creado tras aceptar una oferta sin viaje asociado	Sprint 6	Completada
MOVO-238	[svc-shipments] Expiración automática de viajes declared sin ofertas aceptadas al llegar departureAt	Sprint 6	Completada

Diagrama de secuencia

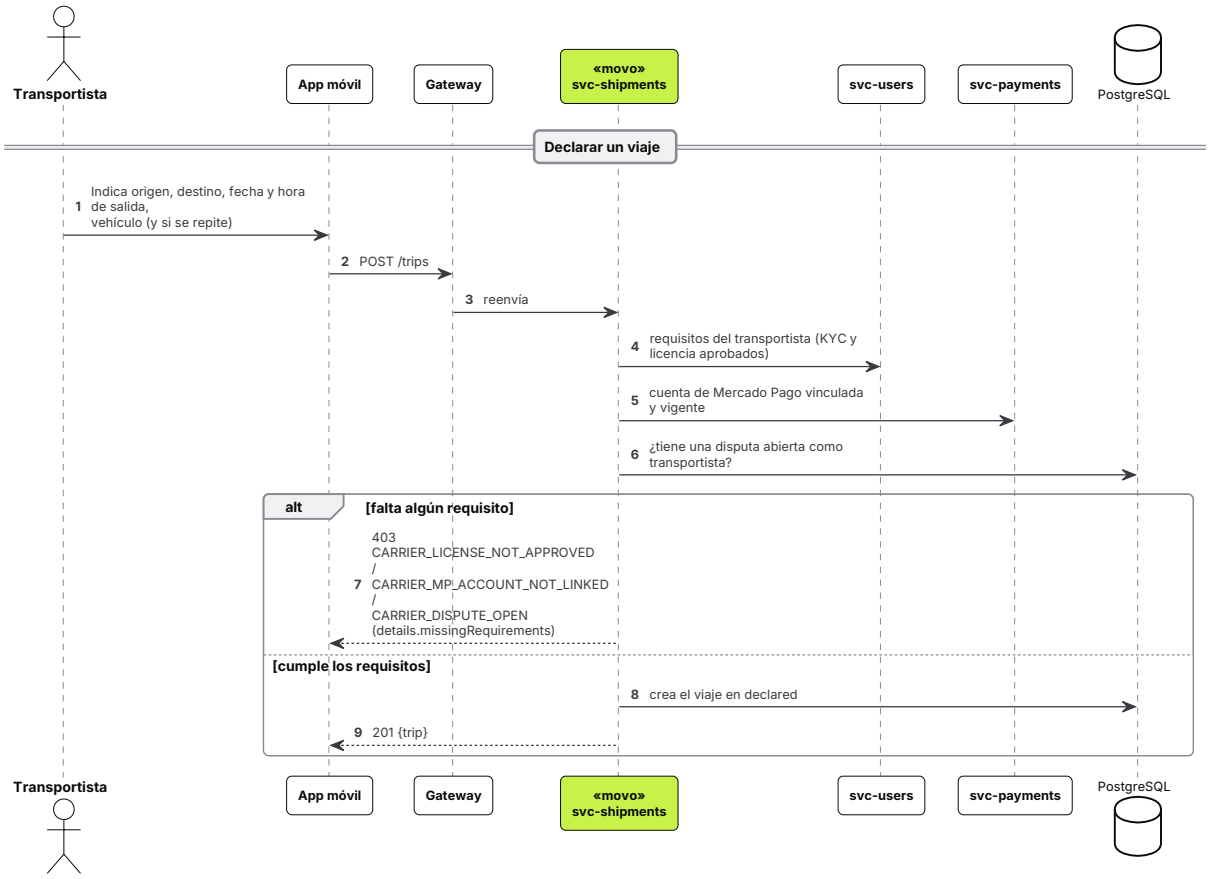


Figura 3.5. Declaración del viaje y consulta de paquetes compatibles (MOVO-18, MOVO-19). Parte 1 de 3: Declarar un viaje.

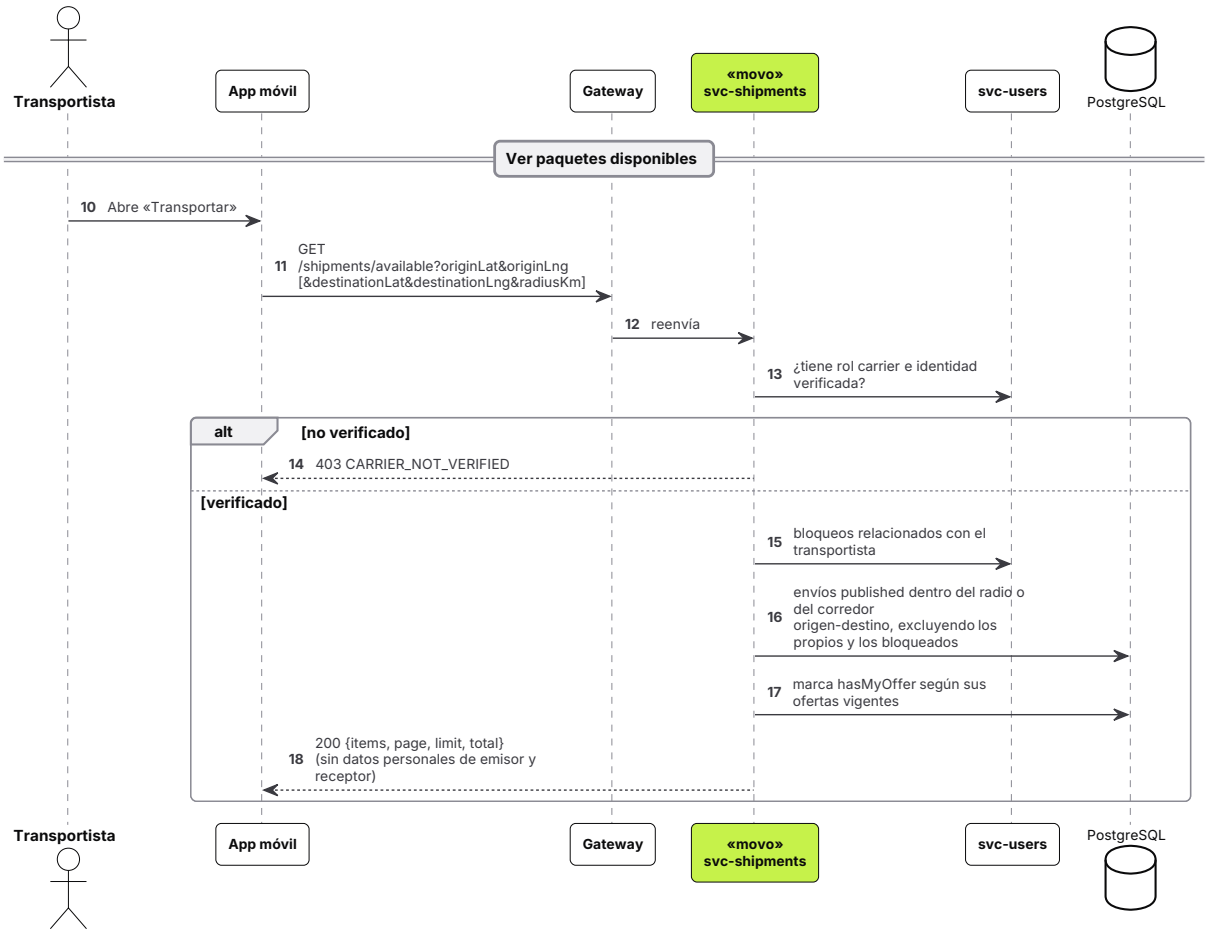


Figura 3.6. Declaración del viaje y consulta de paquetes compatibles (MOVO-18, MOVO-19). Parte 2 de 3: Ver paquetes disponibles.

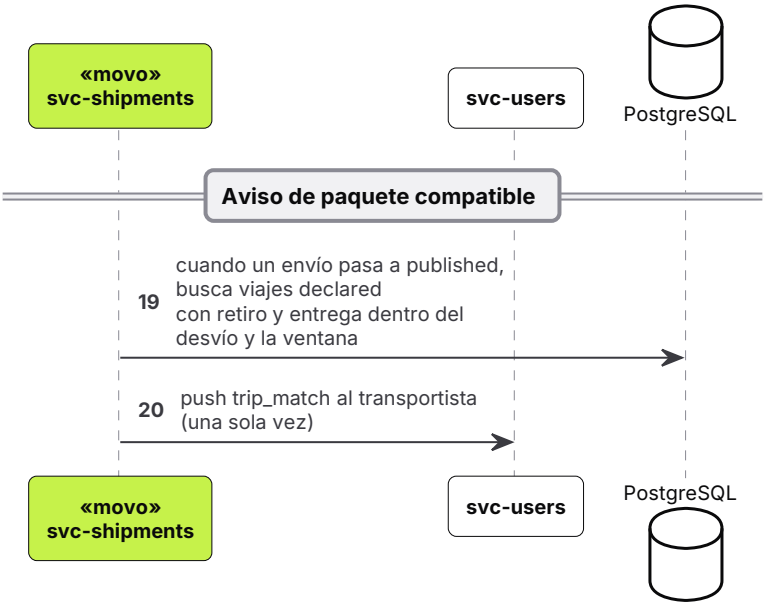


Figura 3.7. Declaración del viaje y consulta de paquetes compatibles (MOVO-18, MOVO-19). Parte 3 de 3: Aviso de paquete compatible.

Casos de prueba

Tabla 3.7. Casos de prueba de MOVO-18 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-18.01	1	**Declarar un viaje** *Pre:* Transportista verificado. *Pasos:* POST /trips con origen, destino, salida y vehículo; luego con un body inválido.	201 con el viaje en declared; body inválido: 400 VALIDATION_FAILED.	`trips.routes.test.ts`	APROBADO
CP-18.02	3	**Editar y eliminar un viaje declarado** *Pre:* Viaje declared del transportista. *Pasos:* PATCH /trips/:id y DELETE /trips/:id.	200 con el viaje actualizado y 204 al eliminar. El estado nunca se modifica por PATCH.	`trips.routes.test.ts`	APROBADO
CP-18.03	3, 4	**Cancelar un viaje** *Pre:* Viaje declared, con y sin paquetes aceptados. *Pasos:* POST /trips/:id/cancel.	Sin paquetes aceptados pasa a cancelled y desasocia las ofertas pendientes. Con un paquete aceptado se rechaza (TripHasAcceptedPackagesError). Cancelar dos veces o un viaje activo: 409 TRIP_NOT_DECLARED.	`trip-cancel.integration.test.ts` `trips.routes.test.ts`	APROBADO

CP-18.04	5	**Iniciar el viaje**
		Pre: Viaje declared con al menos un paquete asignado.
		Pasos: POST /trips/:id/start; repetir; iniciar otro viaje del mismo transportista; iniciar dos viajes en paralelo.
			Pasa a active y las ofertas pendientes se desasocian. Sin paquetes: error; segundo inicio: 409. Con otro viaje activo: 409 TRIP_ALREADY_HAS_ACTIVE_TRIP; con concurrencia exactamente uno gana, y el índice único lo garantiza aun escribiendo directo en la base.	`trip-lifecycle.integration.test.ts`
				`trip-start.integration.test.ts`
				`trips.routes.test.ts`
					APROBADO

CP-18.05	2	**Paquetes compatibles con la ruta** *Pre:* Viaje declarado y envíos publicados. *Pasos:* GET /trips/:id/matches.	Devuelve los paquetes dentro del corredor de la ruta, con hasMyOffer y fechas formateadas. Si falla el servicio de ruteo responde 502 (política No-Fallback).	`trips.routes.test.ts` `trip-compatibility.test.ts`	APROBADO
CP-18.06	-	**Vencimiento de viajes** *Pre:* Viaje declared con salida vencida. *Pasos:* Correr el barrido de vencimiento.	Un viaje vencido sin paquetes se cancela; uno con paquete aceptado queda intacto; uno activo nunca se toca. El barrido respeta el tamaño de lote.	`trip-expiry.integration.test.ts` `trip-expiry-sweep.test.ts`	APROBADO
CP-18.07	-	**Recurrencia semanal** *Pre:* Viaje con recurrencia por días de la semana. *Pasos:* Correr el generador de la próxima ocurrencia en distintas situaciones.	Declara la siguiente a la misma hora, es idempotente, respeta recurrenceUntil, pausa si la cuenta está suspendida o con deuda vencida y corta la serie si faltan requisitos o la cuenta se dio de baja.	`trip-recurrence.integration.test.ts` `trip-recurrence.test.ts`	APROBADO

3.2.4 MOVO-19 · Feed de paquetes disponibles

MOVO-19 COMPLETADA SPRINT 4, SPRINT 5 11 SP

Como transportista, quiero ver los paquetes disponibles en mi ciudad/región para encontrar oportunidades de transporte.

> **Decisión de diseño.** El filtro es siempre `status = published` y por cercanía geográfica al origen (y opcionalmente al destino). El feed no expone datos personales del emisor ni del receptor, solo direcciones aproximadas, ventana horaria y distancias, para no revelar la puerta antes del acuerdo.

Criterios de aceptación

1. **AC1: Filtrado Geográfico por Ciudad / Región / Coordenadas**
2. El transportista puede consultar paquetes disponibles filtrando por `city` (ciudad), `province` (provincia) o mediante coordenadas (`lat` , `lng`) y un radio en kilómetros (`radius_km`).
3. Por defecto, si no se especifican filtros, el feed utiliza la ubicación del perfil del usuario o su última ubicación conocida.
4. **AC2: Información Detallada en cada Item del Feed**
5. Cada item retornado en la lista debe incluir:
6. **Ubicación aproximada**: Barrio/Ciudad de origen y destino (sin revelar la dirección exacta de puerta por seguridad previa al acuerdo).
7. **Propiedades del paquete**: Tipo de paquete (`DOCUMENTO` , `ESTANDAR` , `OBJETO_FRAGIL`), peso (`weight_kg`), dimensiones (`length` , `width` , `height`), descripción corta e indicador de urgencia (`urgent`).
8. **Ventana temporal**: Franja horaria y fechas límite de retiro (`pickup_date_start` a `pickup_date_end`).
9. **Precio ofrecido**: Monto propuesto por el emisor (`suggested_price` / `offered_price`).
10. **Información del emisor**: Nombre público del emisor, avatar/foto y score de reputación ponderado (`rating_overall`).
11. **AC3: Filtrado de Estados (Solo Envíos Disponibles)**
12. Únicamente se muestran paquetes en estado `published` (disponibles para recibir ofertas).
13. Quedan automáticamente excluidos del listado los envíos en cualquier otro estado del enum canónico definido en el AC6 de MOVO-79 (extendido de 9 a 11 valores por MOVO-208): `awaiting_receiver_confirmation` , `rejected_by_receiver` , `assignment_pending` , `assigned_unfunded` , `assigned` , `in_transit` , `delivered` , `completed` , `cancelled` y `disputed` .
14. Los estados `draft` y `expired` que listaba la versión anterior de este AC **no existen** en el enum canónico, ver el comentario de alineación al pie de este issue.
15. **Alineación** MOVO-208 **(12/09/2026)**: `assigned_unfunded` y `completed` (agregados al set canónico por esa US) tampoco aparecen nunca en este feed, el filtro sigue siendo exclusivamente `status = 'published'` , así que cualquier valor nuevo del enum queda excluido por construcción, sin necesidad de tocar código de `movo-svc-shipments` .
16. **AC4: Paginación y Rendimiento**

17. El endpoint implementa paginación (cursor-based o offset-based, por defecto `limit = 20`, max `50`) para asegurar latencias de respuesta < 200ms y bajo consumo de datos móviles.
18. **AC5: Actualización y Mecanismo de Recarga**
19. En la aplicación móvil se habilita pull-to-refresh para actualizar el feed a demanda.
20. El contrato incluye un campo `updated_at` / timestamp de consulta para control de concurrencia y refresco.
21. **AC6: Vista de Detalle del Paquete**
22. Al seleccionar un item del feed, el transportista puede consultar el detalle completo (`GET /api/v1/shipments/:id`), incluyendo la descripción extendida, requerimientos de manipulación y fotografías adjuntas (`package_photo`).
23. **AC7: Restricción de Seguridad, Rol y Verificación KYC**
24. Acceso restringido exclusivamente a usuarios autenticados con JWT válido y rol `CARRIER` (o usuarios con perfil mixto sender/carrier).
25. **Requisito de KYC de Identidad**: El transportista **debe poseer** `kyc_status_identity = 'approved'`. Si el estado es distinto (`not_started`, `pending`, `rejected`, `expired`), el backend responde `403 FORBIDDEN` con código `KYC_REQUIRED` y el cliente móvil redirige/notifica para completar el onboarding KYC.
26. Un emisor no puede ver sus propios paquetes publicados en el feed de oportunidades de transporte como si fuera un tercero.

Trazabilidad de implementación

Tabla 3.8. Tickets de implementación de MOVO-19.

Ticket	Alcance	Sprint	Estado
`MOVO-142`	[svc-shipments] GET /shipments/available: descubrimiento por radio geográfico y apertura del detalle al transportista	Sprint 4	Completada
`MOVO-148`	[movo-mobile] Tab Transportar: listado de envíos disponibles cerca	Sprint 4	Completada
`MOVO-183`	[movo-mobile] Rediseño del tab Transportar: header, accesos y card de envío	Sprint 5	Completada

Casos de prueba

Tabla 3.9. Casos de prueba de MOVO-19 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-19.01	20-26	**Acceso restringido al transportista verificado** *Pre:* Usuarios sin rol carrier, con rol carrier no verificado y verificado. *Pasos:* GET /shipments/available con cada uno.	403 CARRIER_NOT_VERIFIED para los dos primeros; 200 para el verificado, sin exigir datos de licencia.	`shipments-available.integration.test.ts`	APROBADO
CP-19.02	11-15	**Solo envíos publicados y ajenos** *Pre:* Envíos en varios estados y envíos propios del caller. *Pasos:* GET /shipments/available.	Solo aparecen los published; se excluyen los propios (como emisor o receptor).	`shipments-available.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-19.03	4-10	**Contenido del item** *Pre:* Envío publicado. *Pasos:* Inspeccionar la respuesta del feed.	Incluye direcciones, ventana horaria y distancias, sin datos personales de emisor ni receptor. hasMyOffer es true solo con una oferta pendiente propia.	`shipments-available.integration.test.ts`	APROBADO
CP-19.04	2, 3	**Filtro geográfico** *Pre:* Envíos a distintas distancias. *Pasos:* Consultar sin radio, con radiusKm=500, sin coordenadas de origen, con destino incompleto y con destino completo.	Radio por defecto 50 km; tope duro de 200 (400 si se excede); sin origen o con destino a medias: 400 VALIDATION_FAILED. Con destino, un envío a mitad de un trayecto largo aparece aunque esté lejos de ambos extremos por separado.	`shipments-available.integration.test.ts`	APROBADO
CP-19.05	16, 17	**Paginación** *Pre:* Más envíos que el tamaño de página. *Pasos:* Consultar con limit y page.	Contrato {items, page, limit, total}.	`shipments-available.integration.test.ts`	APROBADO

3.2.5 MOVO-29 · Cancelación de un envío por el emisor

MOVO-29 COMPLETADA SPRINT 3 3 SP

Como emisor, quiero cancelar un envío publicado antes de que sea asignado a un transportista, para poder corregir errores o desistir del envío sin penalización.

Decisión de diseño. La penalidad depende del estado y de la cercanía del retiro: sin fondos confirmados no hay cargo, y desde `assigned` solo hay cargo (25 % del precio acordado) si faltan menos de 12 horas para el retiro (MOVO-307, DD-047); el mecanismo de cobro se detalla en el tema Pagos y Economía. La cancelación es lógica e irreversible.

Criterios de aceptación

1. El emisor puede cancelar mientras el envío esté en `awaiting_receiver_confirmation`, `published` o `assignment_pending` (ver enum canónico de MOVO-79). Ninguno de estos tres estados tiene fondos confirmados todavía, `assignment_pending` es el hold de MercadoPago en curso, no un hold ya exitoso, así que la cancelación en cualquiera de los tres **no genera penalización**.
2. Al cancelar desde `published` o `assignment_pending`, todos los transportistas que habían ofertado reciben notificación de que el envío fue cancelado; en `assignment_pending` además debe dispararse la liberación/cancelación del hold en curso en `svc-payments`.
3. Si el emisor cancela un envío que ya está en `assigned` (fondos ya confirmados, `carrier_id` y `agreed_price` fijados), se aplica la política de cancelación **con penalización**.
4. El estado del envio cambia a 'cancelado' y no puede ser reactivado.
5. El emisor ve el envio cancelado en su historial con el motivo si lo indico.

Diagrama de secuencia

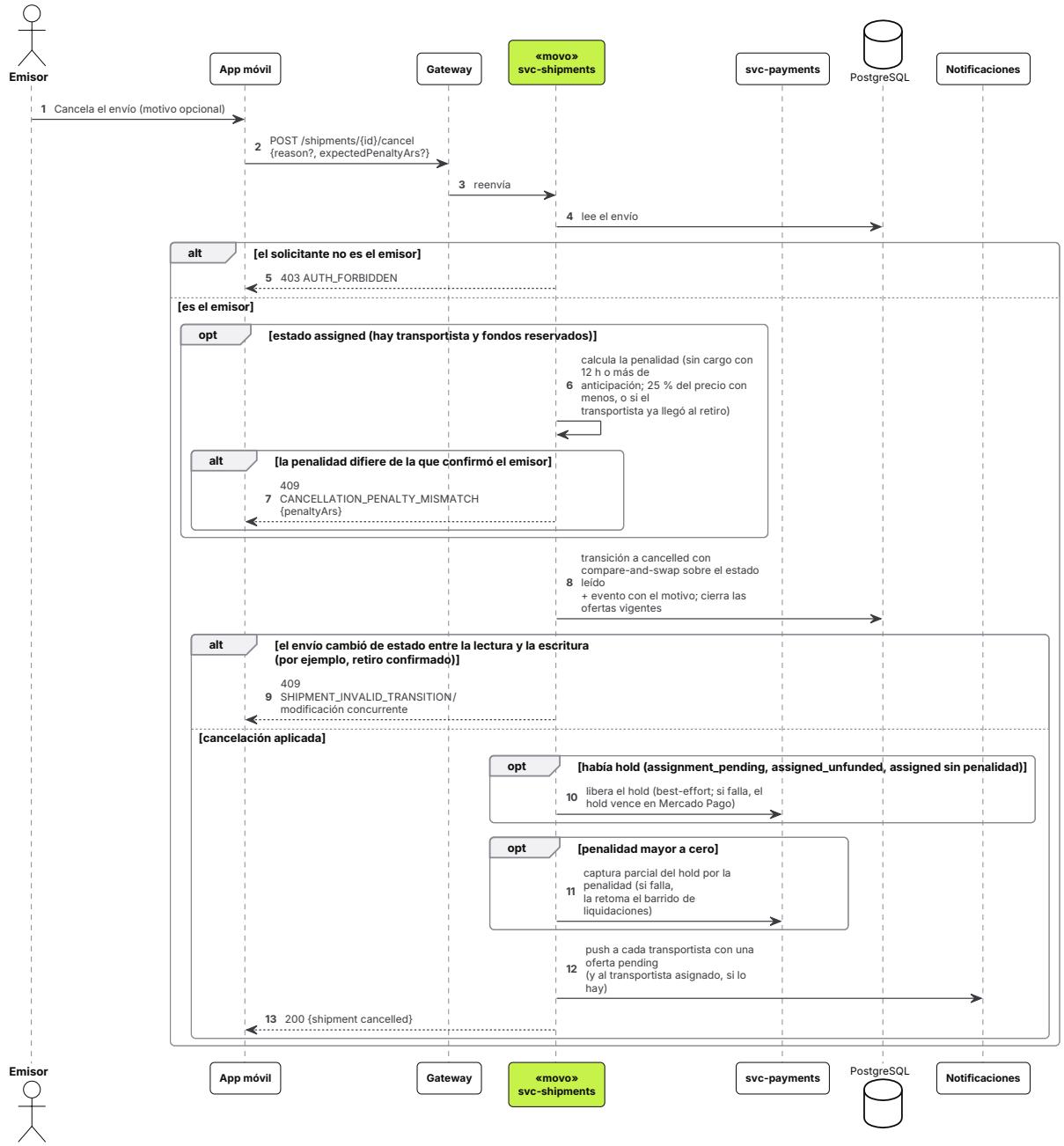


Figura 3.8. Cancelación del envío según su estado, con y sin penalidad (MOVO-29, MOVO-307).

Casos de prueba

Tabla 3.10. Casos de prueba de MOVO-29 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-29.01	1, 4, 5	**Cancelar antes de la asignación** *Pre:* Envío en awaiting_receiver_confirmation. *Pasos:* POST /shipments/:id/cancel con motivo.	El envío pasa a cancelled y el motivo queda en el historial; no se consultan ofertas.	`shipments-cancel.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-29.02	2	**Aviso a los transportistas con oferta** *Pre:* Envío published o assignment_pending con ofertas pendientes de dos transportistas y una retirada. *Pasos:* Cancelar el envío.	Se notifica solo a quienes tenían oferta pendiente. En assignment_pending se libera además el hold en curso. Un fallo de notificaciones no bloquea la cancelación.	`shipments-cancel.integration.test.ts`	APROBADO
CP-29.03	3	**Cancelar un envío asignado** *Pre:* Envío assigned con fondos confirmados. *Pasos:* Cancelar con 12 h o más de anticipación y con menos.	Con 12 h o más: sin cargo. Con menos de 12 h y la ventana abierta: se cobra el 25 % del precio acordado, que el emisor debe confirmar antes de cancelar (DD-047).	`shipments-cancel.integration.test.ts` `shipments-cancel-penalty.integration.test.ts`	APROBADO
CP-29.04	1, 4	**Permisos y estados inválidos** *Pre:* Envío del emisor. *Pasos:* Cancelar como otro usuario, sin identidad, un envío inexistente y uno ya cancelado.	403, 401, 404 y 409 SHIPMENT_INVALID_TRANSITION respectivamente.	`shipments-cancel.integration.test.ts`	APROBADO

3.2.6 MOVO-27 · Historial de envíos

MOVO-27 COMPLETADA SPRINT 3, SPRINT 6 8 SP

Como emisor, quiero consultar el historial de mis envíos con estados y línea de tiempo, para hacer seguimiento de todos mis envíos pasados y actuales desde un solo lugar.

Criterios de aceptación

1. El emisor puede acceder al historial de todos sus envios (activos, completados, cancelados).
2. Cada envio en el historial muestra: id, origen/destino, estado actual, fecha de creacion y fecha de entrega (si aplica).
3. Al seleccionar un envio, el emisor ve la linea de tiempo con todos los cambios de estado y sus timestamps.
4. El historial permite filtrar por estado y ordenar por fecha.
5. El emisor puede acceder a las fotos de evidencia de cada envio desde el historial.
6. El historial muestra envios de al menos los ultimos 12 meses.

Trazabilidad de implementación

Tabla 3.11. Tickets de implementación de MOVO-27.

Ticket	Alcance	Sprint	Estado
`MOVO-127`	[movo-mobile] Pantalla de detalle de un envío específico (ruta, paquete, receptor, línea de tiempo)	Sprint 3	Completada
`MOVO-128`	[svc-shipments] Endpoint GET /shipments/:id/events, historial de estados para la línea de tiempo	Sprint 3	Completada
`MOVO-257`	[movo-mobile] Rediseño de “Mis envíos” para emisor y receptor: accesos por rol, En curso/Historial y cards con acción	Sprint 6	Completada

Casos de prueba

Tabla 3.12. Casos de prueba de MOVO-27 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-27.01	1, 2, 4	**Listado propio paginado** *Pre:* Usuario emisor y receptor en varios envíos. *Pasos:* GET /shipments/mine con filtros y paginación.	Lista solo los envíos donde participa como emisor o receptor, paginados; 401 sin identidad.	`shipments-list-mine.integration.test.ts`	APROBADO
CP-27.02	-	**Conteo de ofertas vigentes** *Pre:* Envío publicado con ofertas. *Pasos:* Listar como emisor y como receptor.	El emisor ve el conteo de ofertas vigentes; el receptor no lo ve; es nulo si el envío no está publicado.	`shipments-list-mine.integration.test.ts`	APROBADO
CP-27.03	3	**Línea de tiempo del envío** *Pre:* Envío con varios cambios de estado. *Pasos:* GET /shipments/:id/events.	Devuelve los eventos con su estado y marca de tiempo, solo a los participantes.	`shipments-events.integration.test.ts`	APROBADO
CP-27.04	5	**Fotos de evidencia** *Pre:* Envío con fotos. *Pasos:* Listar fotos como emisor, receptor, transportista asignado, admin y terceros.	Participantes y admin ven las fotos con URL de lectura; un tercero o un transportista rechazado: 403.	`photos.integration.test.ts`	APROBADO

3.2.7 MOVO-191 · Envíos activos en el inicio de la app

MOVO-191 COMPLETADA SPRINT 5, SPRINT 7 8 SP

Como usuario, quiero ver y operar mis envíos activos desde el inicio de la app, para saber qué tengo que hacer hoy sin buscar en el historial.

Criterios de aceptación

1. Desde el inicio de la app, el usuario ve sus envíos activos agrupados en tres secciones según su rol en cada transacción: **“Estoy enviando”**, **“Estoy transportando”** y **“Voy a recibir”**.
2. Un mismo usuario puede aparecer en las tres secciones simultáneamente (puede ser emisor de un envío y transportista de otro el mismo día).
3. Cada ítem muestra el estado del envío, la fecha y ventana de retiro, y un badge cuando el retiro es hoy o cuando la ventana ya venció.
4. Cada ítem expone la **acción contextual correspondiente al rol y al estado**, que es el punto de entrada al flujo de custodia:
5. emisor + `assigned` → “Generar retiro” (abre `MOVO-159`, generación de QR)
6. transportista + `assigned` → “Retirar paquete” (abre el wizard de retiro)
7. transportista + `in_transit` → “Entregar paquete” (abre el wizard de entrega)
8. receptor + `in_transit` → “Confirmar recepción” (abre `MOVO-160`, escaneo de QR)
9. Si el usuario no tiene envíos activos en un rol, esa sección no se renderiza (no se muestra vacía).
10. La misma acción contextual está disponible también desde el detalle del envío, para que el usuario no dependa exclusivamente del home.
11. La UI respeta el Manual de Marca de Movo.

Trazabilidad de implementación

Tabla 3.13. Tickets de implementación de MOVO-191.

Ticket	Alcance	Sprint	Estado
MOVO-192	[svc-shipments] Endpoints de envíos activos por rol (/sending, /transporting, /receiving)	Sprint 5	Completada
MOVO-193	[movo-mobile] Home operativo: envíos activos por rol y acción contextual de custodia	Sprint 5	Completada
MOVO-194	[movo-mobile][svc-shipments] Detalle del envío: completar la vista del transportista y del receptor	Sprint 7	Completada

Casos de prueba

Tabla 3.14. Casos de prueba de MOVO-191 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-191.01	1, 2	**Tres secciones por rol** *Pre:* Usuario emisor de un envío y transportista de otro. *Pasos:* Consultar los endpoints de enviando, transportando y recibiendo.	Aparece solo en las secciones que corresponden y nunca ve envíos activos de otro usuario.	shipments-active.integration.test.ts	APROBADO
CP-191.02	9	**Sin envíos activos** *Pre:* Usuario sin envíos activos en un rol. *Pasos:* Consultar ese rol.	200 con lista vacía (no 404); sin identidad, 401.	shipments-active.integration.test.ts	APROBADO
CP-191.03	-	**Qué cuenta como activo** *Pre:* Envíos en published, assignment_pending, assigned_unfunded, assigned e in_transit. *Pasos:* Listar activos.	published y assignment_pending no cuentan; assigned_unfunded, assigned e in_transit sí.	shipments-active.integration.test.ts	APROBADO
CP-191.04	3	**Badge de retiro y ventana vencida** *Pre:* Envíos con retiro hoy y con ventana vencida. *Pasos:* Listar activos.	isToday es verdadero solo si el retiro cae hoy en la zona horaria argentina; pickupWindowExpired solo en assigned, nunca en in_transit. Orden por fecha y hora de retiro ascendente.	shipments-active.integration.test.ts	APROBADO
CP-191.05	4	**Contraparte según rol y estado** *Pre:* Envíos assigned e in_transit. *Pasos:* Consultar cada sección.	La contraparte es el transportista para emisor y receptor; para el transportista, el emisor antes del retiro y el receptor durante el tránsito. No se exponen ids crudos.	shipments-active.integration.test.ts	APROBADO

3.2.8 MOVO-259 · Rediseño de “Mis viajes” para el transportista

MOVO-259 COMPLETADA SPRINT 6, SPRINT 7 9 SP

[Transversal] Rediseño de “Mis viajes” para el transportista

> **Alcance final.** Sin criterios de aceptación en Linear: el alcance quedó definido por sus cuatro tickets hijos. Separa el estado `expired` de `cancelled` con cancelación lógica y listado por alcance (MOVO-260), muestra los paquetes aceptados en el detalle (MOVO-261), organiza la pantalla en pestañas "Próximos" e "Historial" (MOVO-262) y añade el detalle del viaje (MOVO-263).

Trazabilidad de implementación

Tabla 3.15. Tickets de implementación de MOVO-259.

Ticket	Alcance	Sprint	Estado
`MOVO-260`	[svc-shipments] Estado `expired` separado de `cancelled`, cancelación lógica de viajes y listado por scope (próximos/historial)	Sprint 6	Completada
`MOVO-261`	[svc-shipments] Paquetes aceptados en el detalle del viaje (GET /trips/:id)	Sprint 6	Completada
`MOVO-262`	[movo-mobile] Mis viajes: tabs Próximos/Historial y pills de estado legibles	Sprint 7	Completada
`MOVO-263`	[movo-mobile] Pantalla de detalle de viaje: paquetes aceptados, búsqueda de paquetes y cancelación	Sprint 7	Completada

Casos de prueba

Tabla 3.16. Casos de prueba de MOVO-259 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-259.01	-	**Listado por alcance** *Pre:* Viajes declared, active, completed, cancelled y expired de dos transportistas. *Pasos:* GET /trips?scope=upcoming y scope=history.	upcoming: solo declared y active, por salida ascendente. history: completed, cancelled y expired, por salida descendente. Nunca mezcla viajes de otro transportista.	`trip-cancel.integration.test.ts` `trips.routes.test.ts`	APROBADO
CP-259.02	-	**Detalle del viaje** *Pre:* Viaje con paquetes aceptados. *Pasos:* GET /trips/:id.	Devuelve el viaje con sus paquetes aceptados.	`trips.routes.test.ts` `trip-repository.integration.test.ts`	APROBADO
CP-259.03	-	**Cancelación lógica** *Pre:* Viaje declared. *Pasos:* POST /trips/:id/cancel.	El viaje no se borra: pasa a cancelled con cancelledAt; con stopRecurrence se corta la serie recurrente.	`trip-cancel.integration.test.ts` `trips.routes.test.ts`	APROBADO

3.2.9 MOVO-138 · Motor de precios dinámico

MOVO-138 COMPLETADA SPRINT 6 5 SP

[svc-pricing-logistics] Motor de precios dinámico real (Post-Spike MOVO-216)

> **Evolución del método.** El criterio original (`demand_fuel_routes_v1` : distancia de Routes API, combustible y alta demanda) fue reemplazado por `package_tapered_v2` (DD-041), que cobra por peso facturable (el mayor entre el real y el volumétrico) con tarifa decreciente por tramos. Se conservó el contrato `POST /quote` , el recargo por alta demanda y la degradación a Haversine si Routes API falla.

Criterios de aceptación

1. **Reemplazo de** `euclidean_linear_v1` **:**
2. Incorporar el cálculo de distancia real mediante `RoutesProvider` (ADR-015).
3. Nuevo `PriceCalculationMethod` en `@movo/shared` y Python: `DEMAND_FUEL_ROUTES_V1` .
4. **Componente de Combustible:**
5. Integración con la API de precios de combustible seleccionada en MOVO-216.
6. Cacheo del precio por litro para resguardar la performance del endpoint.
7. **Factor de Alta Demanda:**
8. Cálculo del multiplicador de alta demanda si la relación entre paquetes y transportistas en la zona supera el umbral definido en el Spike.
9. **Simplificación de la Respuesta (Contrato):**
10. El endpoint `POST /quote` retorna:
11. `suggestedPriceArs` : monto final calculado.
12. `highDemand` : boolean indicando si aplica recargo por alta demanda.
13. `calculationMethod` : `DEMAND_FUEL_ROUTES_V1` .
14. **Sin desglose aritmético complejo** visible al usuario (el emisor solo ve el precio sugerido y si rige alta demanda).
15. **Compatibilidad:**
16. Mantener compatibilidad con el consumidor en `movo-svc-shipments` (`pricing-client.ts`).
17. Cobertura de tests unitarios y de integración al 100% sobre los módulos de pricing.
18. **Persistencia de** `highDemand` **(decidido en** MOVO-216**):**
19. Nueva columna `shipments.high_demand BOOLEAN NULL` (migración Prisma en `movo-svc-shipments`), escrita junto a `suggested_price_ars` y `calculation_method` al crear el envío. Es una foto de la cotización y no se recalcula.
20. `NULL` = sin cotización (“precio a estimar”) o envío anterior a `DEMAND_FUEL_ROUTES_V1` . No equivale a `false` , y el badge solo se muestra con `true` .
21. `QuoteResult.highDemand` en `pricing-client.ts` , expuesto en el detalle del envío para el emisor.
22. **Degradación si falla Google Routes API en** `POST /quote` **(decidido en** MOVO-216**):**
23. Si `RoutesProvider` falla (caído, timeout o cuota agotada), `/quote` **no** propaga el 502: calcula la distancia con Haversine × 1,3, cotiza igual y registra `distanceSource: "haversine_fallback"` en el log `pricing_quote_computed` .
24. Excepción acotada a `/quote` : la política No-Fallback (MOVO-205) sigue vigente para `/optimize/route` y `/routes/evaluate-candidates` .
25. Timeout hacia Google de ~1,5 s, para que el fallback llegue antes de los 3 s de `pricing-client.ts` .
26. `calculationMethod` se mantiene en `DEMAND_FUEL_ROUTES_V1` .

Casos de prueba

Tabla 3.17. Casos de prueba de MOVO-138 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-138.01	2, 3	**Fórmula y recargos** *Pre:* Parámetros de precio por configuración. *Pasos:* Cotizar con y sin contexto de demanda, paquete frágil y distintos combustibles.	El precio escala con el combustible, el recargo frágil se aplica al subtotal, el multiplicador de demanda sigue la tabla del spike y sin contexto es neutro. El resultado se redondea a múltiplos de 10.	`test_quote.py`	APROBADO
CP-138.02	4-8	**Combustible con caché y degradación** *Pre:* API de combustible disponible, lenta, caída o con valores absurdos. *Pasos:* Pedir el precio del combustible.	Se cachea y se sirve el último valor bueno ante falla; un valor absurdo se rechaza; sin Redis se usa el de configuración; consultas concurrentes llaman a la API una sola vez.	`test_fuel_price.py`	APROBADO
CP-138.03	2, 22-26	**Distancia por Routes API y degradación a Haversine** *Pre:* Proveedor de rutas disponible, caído, lento o sin API key. *Pasos:* POST /quote en cada situación.	Se usa la distancia de Routes API; si falla, es cero o excede el timeout corto, cotiza con Haversine por factor de ruta en lugar de propagar el error.	`test_quote.py`	APROBADO
CP-138.04	-	**Propiedades del precio (grilla de referencia)** *Pre:* Grilla de casos de referencia. *Pasos:* Evaluar la grilla y propiedades de monotonía.	Cada celda cae dentro de su banda; el precio nunca baja al aumentar distancia, peso o volumen; es cóncavo en la distancia; el peso volumétrico y el real se aplican según corresponda; los límites del paquete se rechazan fuera de rango.	`test_pricing_golden.py`	APROBADO
CP-138.05	15, 16	**Compatibilidad con el consumidor** *Pre:* Cliente de pricing en svc-shipments. *Pasos:* Cotizar desde svc-shipments.	El cliente interpreta el contrato (suggestedPriceArs, highDemand, calculationMethod) y degrada a precio nulo si pricing no responde.	`pricing-client.test.ts` `shipment-quote.test.ts`	APROBADO

04

TEMA 3 DE 7 · TRAZABILIDAD DE HISTORIAS DE USUARIO

EP-03 · Asignación y Negociación

Ofertas de los transportistas, elección por parte del emisor, seguimiento de ofertas y asignación del envío.

CONTENIDO

4.1	Descripción del Tema	79
4.2	Historias de Usuario y Trazabilidad	80

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: trazabilidad de las historias del tema, con criterios de aceptación de Linear, diagramas de secuencia derivados del código y casos de prueba derivados de las pruebas automatizadas.

Introducción

Este documento presenta la trazabilidad de las historias de usuario del tema *Asignación y Negociación*: para cada historia, su descripción y criterios de aceptación, los tickets que la implementaron, el diagrama de secuencia que muestra cómo la resuelve el sistema y los casos de prueba que la verifican.

Audiencia

El tribunal evaluador y los futuros mantenedores. Se asume conocimiento de las convenciones del proyecto y de UML.

4.1 Descripción del Tema

EP-03 · Asignación y Negociación es el mercado de Movo: sobre un envío publicado, los transportistas verificados proponen cuánto quieren cobrar, el emisor compara las ofertas por precio o por reputación y elige una, y el sistema asigna el envío de forma atómica. A diferencia de un precio fijo, el emisor conserva el control de la elección, y el transportista conoce en todo momento su posición frente a la competencia.

La oferta es una entidad propia con una máquina de estados de seis valores. Aceptarla no cobra: deja el envío en `assignment_pending` o `assigned_unfunded` según la **saga de asignación** (DD-037), que se completa en el tema *Pagos y Economía* con la reserva de fondos. Una asignación concurrente se resuelve con bloqueo optimista: solo una aceptación puede ganar.

Servicios involucrados: `gateway`, `movo-svc-shipments` (ofertas, viajes y asignación), `movo-svc-users` (reputación y bloqueos), `movo-svc-payments` (cuenta de cobro del transportista) y `movo-mobile`.

4.2 Historias de Usuario y Trazabilidad

Tabla 4.1. Historias de usuario del tema.

Historia	Descripción	Estado	Sprint
`MOVO-23`	Oferta del transportista	Completada	Sprint 4
`MOVO-17`	Lista de ofertas y elección del transportista	Completada	Sprint 4
`MOVO-109`	Seguimiento de las ofertas del transportista	Completada	Sprint 4, Sprint 5, Sprint 6
`MOVO-102`	Modelo y máquina de estados de la oferta	Completada	Sprint 3

4.2.1 MOVO-23 · Oferta del transportista

MOVO-23 COMPLETADA SPRINT 4 11 SP

Como transportista, quiero crear una oferta indicando cuánto quiero cobrar por un envío, para participar en la selección del emisor con el precio que considero justo.

> **Decisión de diseño.** El transportista ingresa el monto **neto** que quiere cobrar. El sistema calcula el **bruto** (neto más comisión de Movo, tasa fija de configuración) y persiste el bruto en `price_offered`, que es lo que ve el emisor. La oferta puede proponer una fecha de retiro dentro de un máximo de 3 días posteriores a la ventana del envío (MOVO-177) y vence automáticamente; el vencimiento se resuelve en lectura, sin proceso en segundo plano.

Criterios de aceptación

1. El transportista puede crear una oferta sobre un envío publicado (`shipment.status = 'published'`) indicando el monto que quiere cobrar y, opcionalmente, un mensaje.
2. El monto que ingresa el transportista es lo que **él** quiere cobrar (neto). Antes de confirmar, el sistema le muestra también cuánto pagaría el emisor en total (bruto = neto + comisión de Movo), para que quede claro qué número está ofertando y qué número va a ver el emisor.
3. La tasa de comisión usada para este cálculo es un valor fijo de configuración por ahora, no existe todavía integración con el modelo real de comisiones de `Payment_Transaction` / MercadoPago Split Payment.
4. El monto bruto calculado es el que se persiste en `price_offered` (`Offer`, MOVO-102) y el que ve el emisor en la lista de ofertas (MOVO-17), en la misma unidad que `suggested_price_ars` del envío.
5. El mensaje es opcional, texto libre, con un límite razonable de caracteres (ej. 280).
6. **Nota técnica:** el schema de `Offer` definido en MOVO-102 no incluye hoy un campo de mensaje (`id`, `shipment_id`, `carrier_id`, `price_offered`, `offered_date`, `status`, `created_at`,

`responded_at`, `expires_at`, `carrier_rating_at_offer`). Hay que sumar una columna `message` (nullable) ahí antes o junto con esta implementación, queda coordinarlo con MOVO-102.

7. El transportista no puede crear una oferta sobre un envío que no está en estado `published` (no existe, ya asignado, cancelado, etc.). Devuelve un error de negocio explícito, no un 500 ni un 404 genérico salvo que el `shipment_id` no exista.
8. El transportista no puede tener más de una oferta `pending` activa sobre el mismo envío. Esto ya está resuelto a nivel de base en MOVO-102 (índice único parcial); acá corresponde traducir la violación de ese índice en un error de negocio claro, no en un error genérico de base de datos.
9. Si el transportista tuvo una oferta previa `rejected`, `withdrawn`, `expired` o `superseded` sobre el mismo envío, puede crear una nueva oferta sin restricción.
10. `carrier_id` se toma del header `x-user-id` inyectado por el gateway, nunca del body (mismo criterio que MOVO-102 AC12 y MOVO-80 AC10).
11. Crear una oferta exige `kyc_status_identity = 'approved'` del transportista (mismo criterio que MOVO-102 AC13).
12. `offered_date` debe caer dentro del rango `pickup_date_start` – `pickup_date_end` del envío (mismo criterio que MOVO-102 AC10); si no, error de negocio claro.
13. La oferta creada queda en estado `pending`.
14. Ruta protegida, documentada en Swagger.

Trazabilidad de implementación

Tabla 4.2. Tickets de implementación de MOVO-23.

Ticket	Alcance	Sprint	Estado
`MOVO-143`	[svc-shipments] POST /shipments/:id/offers y retiro de oferta	Sprint 4	Completada
`MOVO-149`	[movo-mobile] Detalle del envío disponible y creación de la oferta	Sprint 4	Completada
`MOVO-166`	[movo-mobile] Detalle del envío disponible para el transportista (solo lectura)	Sprint 4	Completada
`MOVO-177`	Plan de mejora: creación de oferta del transportista (mobile)	Sprint 4	Completada

Diagrama de secuencia

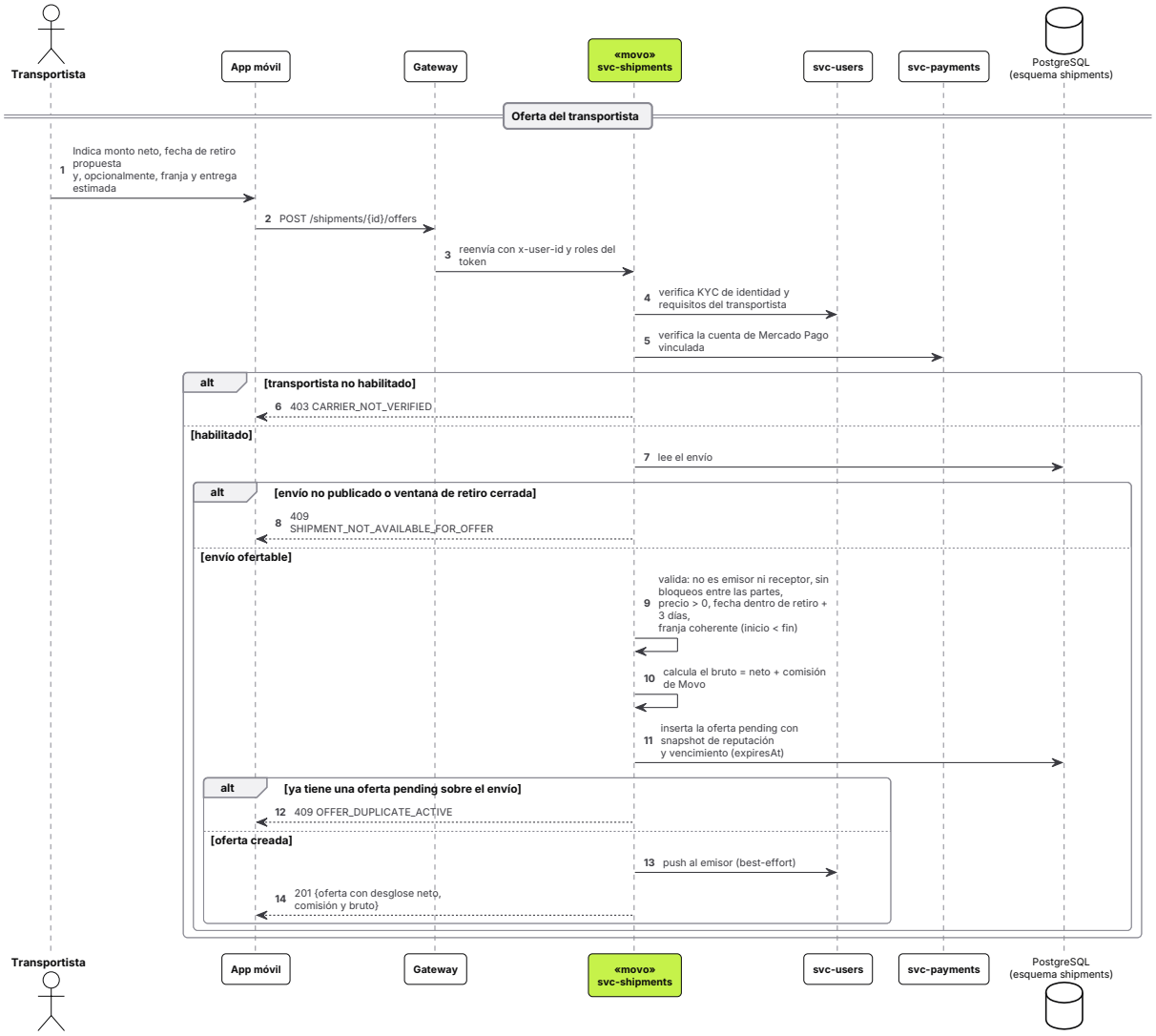


Figura 4.1. Oferta del transportista, selección por el emisor y seguimiento (MOVO-23, MOVO-17, MOVO-109, MOVO-188, MOVO-189). Parte 1 de 3: Oferta del transportista.

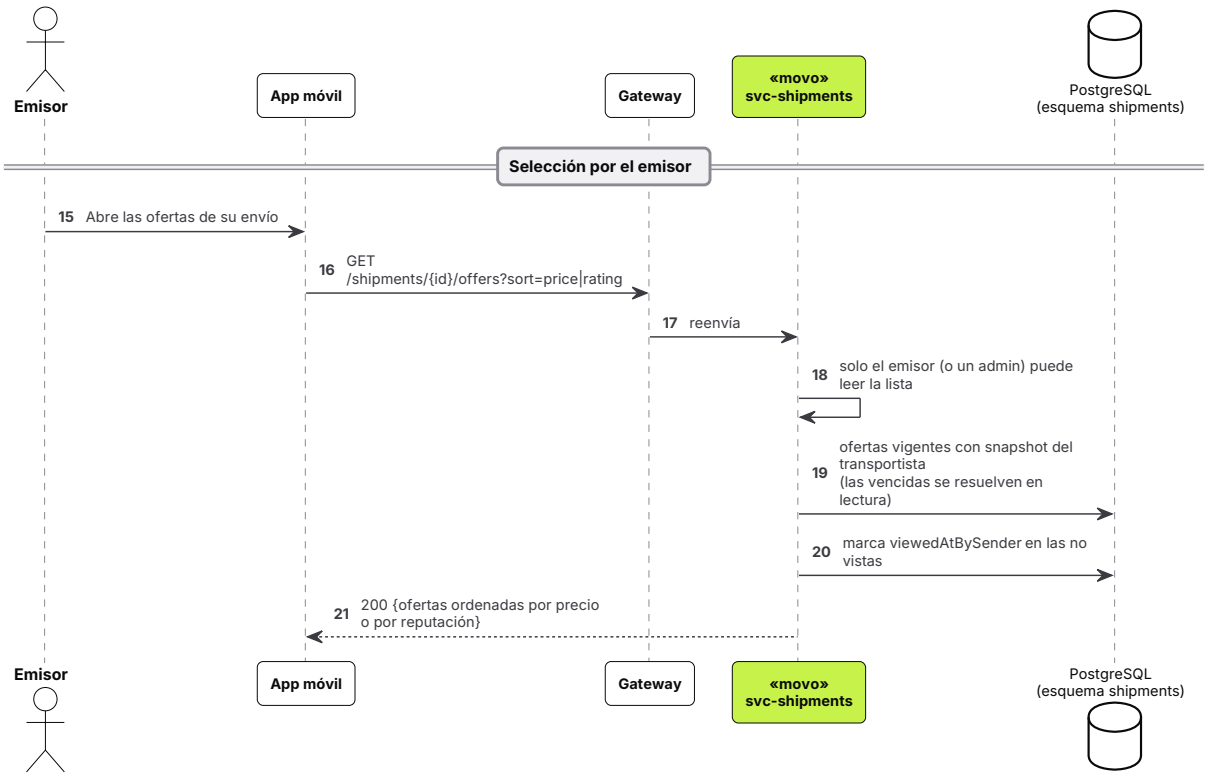


Figura 4.2. Oferta del transportista, selección por el emisor y seguimiento (MOVO-23, MOVO-17, MOVO-109, MOVO-188, MOVO-189). Parte 2 de 3: Selección por el emisor.

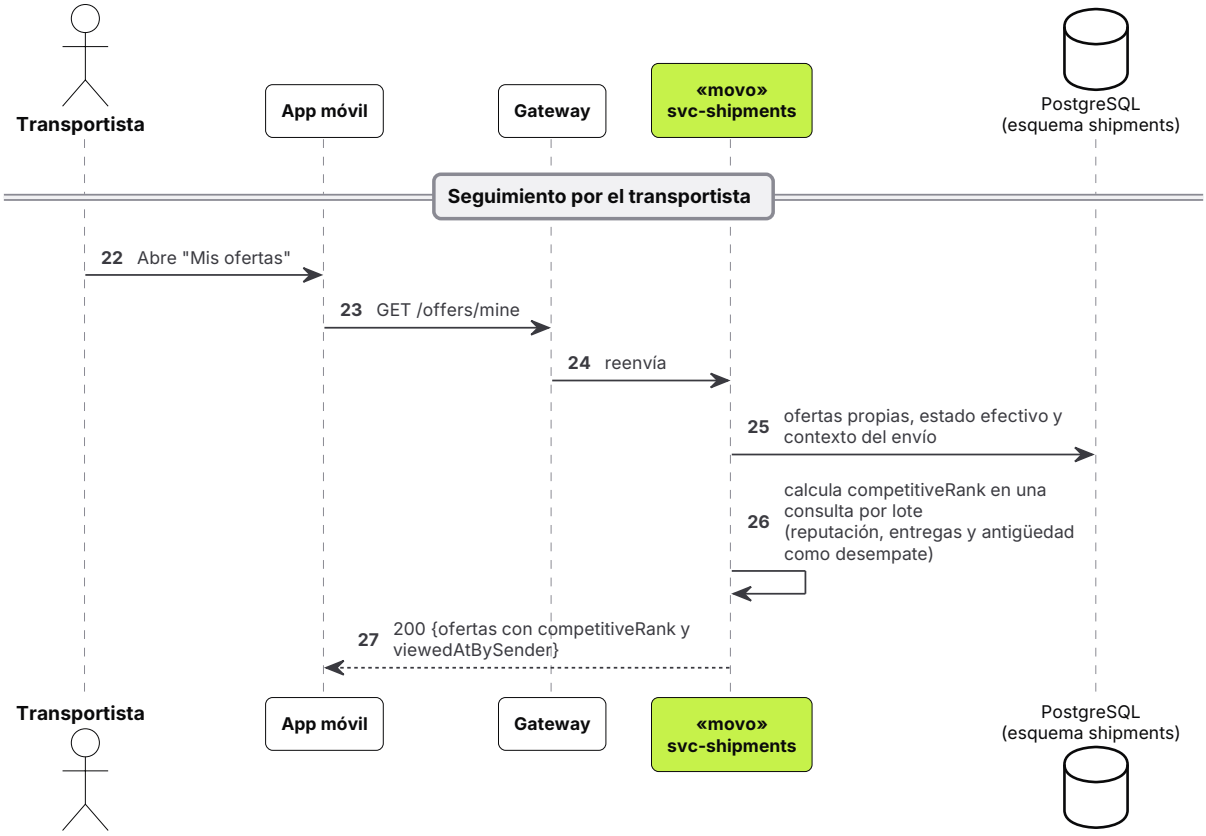


Figura 4.3. Oferta del transportista, selección por el emisor y seguimiento (MOVO-23, MOVO-17, MOVO-109, MOVO-188, MOVO-189). Parte 3 de 3: Seguimiento por el transportista.

Casos de prueba

Tabla 4.3. Casos de prueba de MOVO-23 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-23.01	1, 2, 4, 13	**Crear una oferta** *Pre:* Transportista habilitado y envío published. *Pasos:* POST /shipments/:id/offers con el monto neto y la fecha de retiro propuesta.	201 con la oferta en pending, el desglose neto, comisión y bruto, y notificación al emisor.	`shipments-offers-create.integration.test.ts`	APROBADO
CP-23.02	11	**Solo transportistas habilitados** *Pre:* Usuario sin rol carrier y otro sin KYC de identidad aprobado. *Pasos:* Ofertar con cada uno.	403 CARRIER_NOT_VERIFIED.	`shipments-offers-create.integration.test.ts`	APROBADO
CP-23.03	7	**Envío no ofertable** *Pre:* Envío no publicado, o con la ventana de retiro cerrada. *Pasos:* Ofertar sobre cada uno.	409 SHIPMENT_NOT_AVAILABLE_FOR_OFFER; el envío inexistente responde 404.	`shipments-offers-create.integration.test.ts`	APROBADO
CP-23.04	-	**Emisor y receptor no ofertan** *Pre:* Envío published. *Pasos:* El emisor y luego el receptor intentan ofertar.	403 AUTH_FORBIDDEN en ambos casos.	`shipments-offers-create.integration.test.ts`	APROBADO
CP-23.05	8, 9	**Una sola oferta pendiente por envío** *Pre:* El transportista ya tiene una oferta activa, o una previa rechazada. *Pasos:* Ofertar de nuevo sobre el mismo envío.	Con oferta activa: 409 OFFER_DUPLICATE_ACTIVE. Tras un rechazo: se crea una fila nueva. Distintos transportistas pueden ofertar a la vez.	`shipments-offers-create.integration.test.ts` `offer-repository.integration.test.ts`	APROBADO
CP-23.06	12	**Fecha y franja propuestas** *Pre:* Envío con ventana de retiro. *Pasos:* Ofertar con fecha anterior al retiro, hasta 3 días después, más allá de 3 días, y con la franja incompleta o invertida.	422 OFFER_DATE_OUT_OF_RANGE fuera del rango; 201 dentro; 422 VALIDATION_FAILED con un solo extremo de la franja; 422 OFFER_PICKUP_WINDOW_INVALID si termina antes de empezar. El vencimiento usa la franja propuesta.	`shipments-offers-create.integration.test.ts`	APROBADO
CP-23.07	10	**Identidad tomada del header** *Pre:* Pedido con un carrierId en el body. *Pasos:* POST /shipments/:id/offers.	Se usa el carrierId del header inyectado por el gateway; el precio menor o igual a 0 responde 400 VALIDATION_FAILED.	`shipments-offers-create.integration.test.ts`	APROBADO
CP-23.08	-	**Snapshot de reputación y degradación** *Pre:* Emisor y transportista con y sin calificaciones; servicio de usuarios caído. *Pasos:* Crear la oferta en cada situación.	Se guarda la reputación del transportista y del emisor al ofertar (nula si no hay calificaciones); si svc-users falla, la oferta se crea igual con el snapshot vacío.	`shipments-offers-create.integration.test.ts` `offer-repository.integration.test.ts`	APROBADO

4.2.2 MOVO-17 · Lista de ofertas y elección del transportista

MOVO-17 COMPLETADA SPRINT 4 6 SP

Como emisor quiero ver la lista de ofertas de transportistas para elegir la opción más conveniente.

Alcance final. La historia no tiene criterios de aceptación en Linear; su alcance quedó en sus dos tickets hijos (lista de ofertas y aceptar o rechazar) y en los refinamientos posteriores: orden por precio o reputación, marca de oferta vista por el emisor (MOVO-189) y asignación atómica. Aceptar no cobra: deja el envío en `assignment_pending` (retiro cercano) o `assigned_unfunded` (retiro lejano) según la saga de asignación (DD-037); el hold de fondos se resuelve en el tema Pagos.

Trazabilidad de implementación

Tabla 4.4. Tickets de implementación de MOVO-17.

Ticket	Alcance	Sprint	Estado
`MOVO-144`	[svc-shipments] GET /shipments/:id/offers y aceptación/rechazo de oferta por el emisor	Sprint 4	Completada
`MOVO-150`	[movo-mobile] Ofertas recibidas: listado, comparación y elección del transportista	Sprint 4	Completada

Diagrama de secuencia

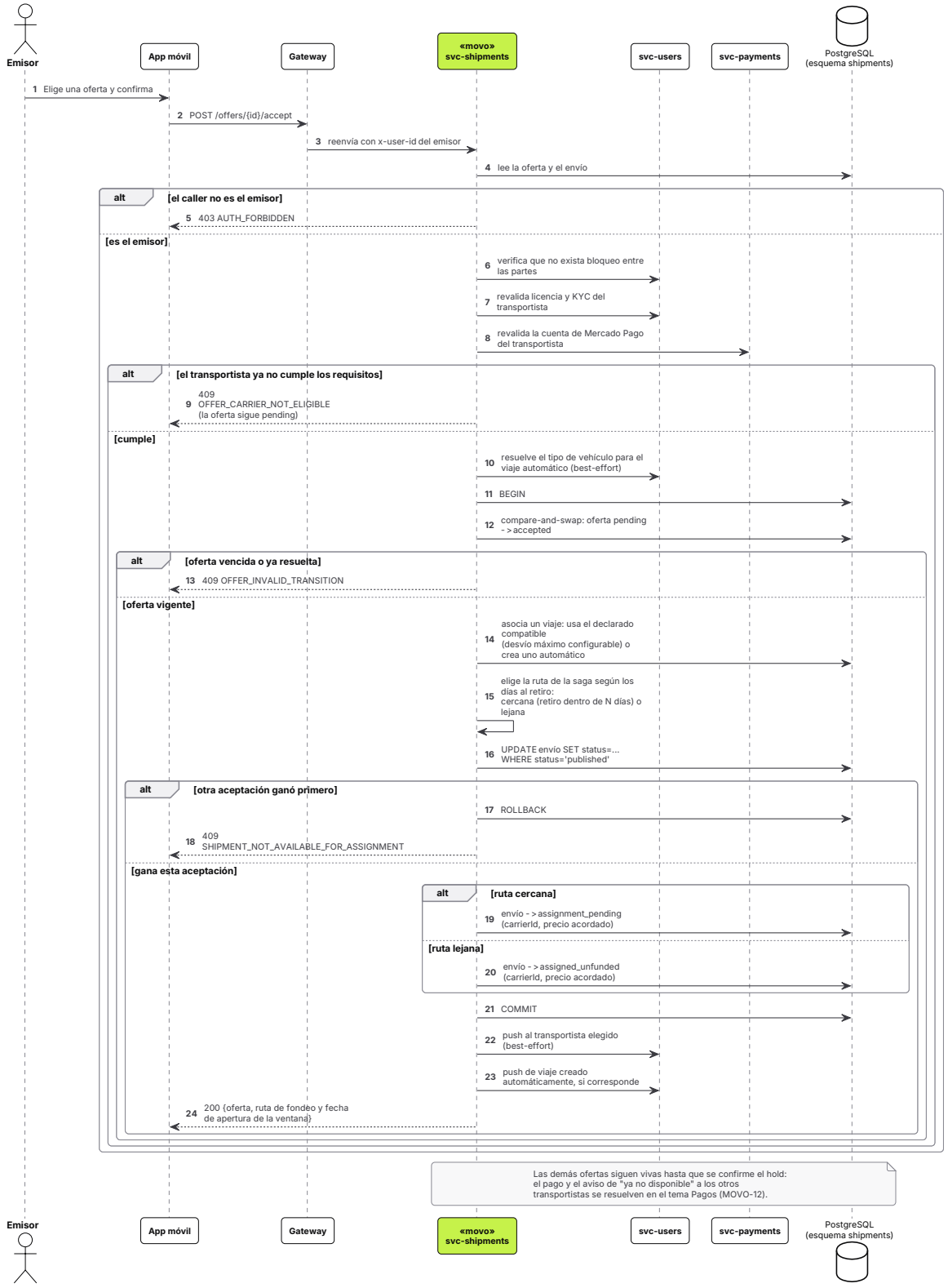


Figura 4.4. Aceptación de una oferta y asignación del transportista (MOVO-17, MOVO-210, MOVO-234).

Casos de prueba

Tabla 4.5. Casos de prueba de MOVO-17 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-17.01	-	**Lista de ofertas del envío** *Pre:* Envío con ofertas pendientes de varios transportistas. *Pasos:* GET /shipments/:id/offers como emisor, como receptor, como tercero y como admin.	El emisor y el admin ven la lista con el snapshot del transportista; el receptor y los terceros reciben 403; un envío inexistente responde 404.	`shipments-offers-list.integration.test.ts`	APROBADO
CP-17.02	-	**Orden y alcance de la lista** *Pre:* Ofertas vigentes, vencidas y de un envío ya cancelado. *Pasos:* Listar con sort por precio y por rating, y con includeResolved.	Por defecto, precio ascendente; por rating, descendente con los nulos al final. Solo vigentes salvo includeResolved=true; una oferta pendiente de un envío cancelado no aparece como vigente.	`shipments-offers-list.integration.test.ts`	APROBADO
CP-17.03	-	**Oferta vista por el emisor** *Pre:* Ofertas aún no leídas. *Pasos:* El emisor lee la lista dos veces; luego llega una oferta nueva; luego la lee un admin.	La primera lectura marca viewedAtBySender y la segunda no lo cambia; una oferta posterior queda sin marcar; el admin no marca nada.	`shipments-offers-list.integration.test.ts`	APROBADO
CP-17.04	-	**Aceptar una oferta** *Pre:* Envío published con una oferta pendiente. *Pasos:* POST /offers/:id/accept como emisor.	El envío pasa a assignment_pending con carrierId y precio acordado, la oferta queda accepted, las demás se mantienen vivas hasta el pago, y la respuesta desglosa neto y comisión. Se avisa al transportista elegido.	`offers-accept-reject.integration.test.ts` `funding.integration.test.ts`	APROBADO
CP-17.05	-	**Aceptación concurrente o vencida** *Pre:* Oferta vencida, o dos aceptaciones simultáneas. *Pasos:* Aceptar la oferta vencida; aceptar dos ofertas al mismo tiempo.	Oferta vencida: 409. Con doble aceptación concurrente una gana y la otra recibe 409.	`offers-accept-reject.integration.test.ts`	APROBADO
CP-17.06	-	**Solo el emisor decide** *Pre:* Oferta pendiente. *Pasos:* Aceptar y rechazar como receptor, como admin, sin identidad, y una oferta inexistente.	403 para receptor y admin, 401 sin identidad, 404 inexistente.	`offers-accept-reject.integration.test.ts`	APROBADO
CP-17.07	-	**Rechazar una oferta** *Pre:* Oferta pendiente. *Pasos:* POST /offers/:id/reject; luego el mismo transportista oferta de nuevo.	El envío sigue published, se avisa al rechazado y puede volver a ofertar con una fila nueva. Rechazar una oferta ya resuelta: 409.	`offers-accept-reject.integration.test.ts`	APROBADO
CP-17.08	-	**Viaje asociado a la asignación** *Pre:* Oferta sin viaje previo, o con un viaje compatible del transportista. *Pasos:* Aceptar la oferta.	Si no tenía viaje, se crea uno declared con origen, destino y salida del envío y se avisa al transportista; si hay un viaje compatible dentro del desvío máximo, se suma a él. Si el viaje queda fuera de rango, se crea uno nuevo.	`offers-accept-reject.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-17.09	-	**Vigencia de los requisitos del transportista** *Pre:* Transportista que perdió la licencia o la cuenta de cobro después de ofertar. *Pasos:* Aceptar su oferta.	409 OFFER_CARRIER_NOT_ELIGIBLE sin revelar qué le falta; la oferta sigue pending para que el emisor elija otra.	`offers-accept-reject.integration.test.ts`	APROBADO

4.2.3 MOVO-109 · Seguimiento de las ofertas del transportista

MOVO-109 COMPLETADA SPRINT 4, SPRINT 5, SPRINT 6 25 SP

Como transportista, quiero ver el estado de mis ofertas activas, para hacer seguimiento de mis propuestas sobre los envíos.

Alcance final. Además del listado, cada oferta muestra su posición competitiva (`competitiveRank`), calculada en una consulta por lote: gana la mejor reputación como transportista y, a igual reputación, quien entregó más envíos y luego la oferta más antigua (MOVO-188). El transportista puede retirar o modificar una oferta pendiente.

Criterios de aceptación

1. El transportista puede ver un listado de todas las ofertas que creó, no solo las `pending` .
2. Cada oferta del listado muestra al menos: envío asociado (referencia/origen-destino), precio ofertado, fecha de la oferta y estado actual.
3. Los seis estados posibles son los definidos en MOVO-102: `pending` , `accepted` , `rejected` , `withdrawn` , `expired` , `superseded` .
4. El estado `expired` se resuelve en lectura (lectura perezosa, mismo criterio que MOVO-102 AC11): una oferta con `expires_at` vencido se muestra como `expired` aunque nadie haya corrido un proceso que la actualice.
5. El transportista puede distinguir visualmente entre ofertas activas (`pending`) y finalizadas (`accepted` , `rejected` , `withdrawn` , `expired` , `superseded`).
6. Un transportista solo puede ver sus propias ofertas, el filtro por `carrier_id` sale del header `x-user-id` inyectado por el gateway, nunca de un parámetro de la request.
7. Ruta protegida, documentada en Swagger.

Trazabilidad de implementación

Tabla 4.6. Tickets de implementación de MOVO-109.

Ticket	Alcance	Sprint	Estado
`MOVO-145`	[svc-shipments] GET /offers/mine: listado de las ofertas propias del transportista	Sprint 4	Completada
`MOVO-151`	[movo-mobile] “Mis ofertas”: seguimiento de las propuestas del transportista	Sprint 6	Completada
`MOVO-181`	[svc-shipments] Modificar una oferta pending (monto y fecha/horario de retiro)	Sprint 5	Completada

Ticket	Alcance	Sprint	Estado
MOVO-182	[movo-mobile] Detalle de oferta: desglose completo, retirar y modificar	Sprint 5	Completada
MOVO-185	[svc-shipments] Contexto enriquecido de envío en “mis ofertas”: distancia y resumen del paquete	Sprint 5	Completada
MOVO-186	[svc-shipments] Desglose neto/comisión en todas las respuestas de oferta (no solo al crear)	Sprint 5	Completada
MOVO-187	[svc-shipments] Snapshot de identidad y confianza del emisor visible al transportista	Sprint 5	Completada
MOVO-188	[svc-shipments] Ranking competitivo de la oferta propia (posición, mínimo y máximo)	Sprint 5	Completada
MOVO-189	[svc-shipments] Tracking de “vista” del emisor sobre una oferta recibida	Sprint 5	Completada
MOVO-190	[svc-shipments] GET /offers/:id, detalle de una oferta propia	Sprint 5	Completada

Casos de prueba

Tabla 4.7. Casos de prueba de MOVO-109 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-109.01	1, 6	**Solo las ofertas propias** *Pre:* Ofertas de dos transportistas. *Pasos:* GET /offers/mine con y sin un carrierId en el query.	Lista únicamente las del transportista autenticado e ignora cualquier carrierId enviado; sin identidad, 401.	offers-mine.integration.test.ts	APROBADO
CP-109.02	2, 3, 4	**Estado efectivo y contexto del envío** *Pre:* Ofertas pending vencidas, aceptadas, rechazadas y de un envío cancelado. *Pasos:* Listar sin filtro y con ?status=expired.	Una pending vencida sale como expired; cada ítem trae el contexto del envío, la fecha sin hora y el estado real del envío si fue aceptada. Sobre un envío cancelado queda shipment_cancelled.	offers-mine.integration.test.ts	APROBADO
CP-109.03	–	**Posición competitiva** *Pre:* Tres transportistas ofertando el mismo envío. *Pasos:* Cada uno consulta sus ofertas.	Cada uno ve su propio rank; gana la mejor reputación, luego más entregas, luego la oferta más antigua. Las ofertas resueltas exponen rank nulo.	offers-mine.integration.test.ts	APROBADO
CP-109.04	–	**Detalle de una oferta** *Pre:* Oferta propia y oferta ajena. *Pasos:* GET /offers/:id.	Mismo formato enriquecido que un ítem del listado; descuenta la deuda del transportista con su motivo y tope; 403 sobre una ajena y 404 sobre una inexistente.	offers-detail.integration.test.ts	APROBADO
CP-109.05	–	**Modificar una oferta pendiente** *Pre:* Oferta pending del transportista. *Pasos:* PATCH /offers/:id con precio, fecha o franja; probar oferta aceptada, vencida y ajena.	Se actualiza el precio (neto a bruto) y se recalcula el vencimiento sin tocar createdAt; 409 OFFER_NOT_EDITABLE si está aceptada o vencida; 403 si no es el dueño; 422 ante fecha, franja o precio inválidos.	offers-update.integration.test.ts	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-109.06	-	**Retirar una oferta** *Pre:* Oferta pending del transportista. *Pasos:* POST /offers/:id/withdraw; repetir sobre una aceptada; probar un tercero.	La oferta pasa a withdrawn; sobre una ya aceptada, 409 OFFER_INVALID_TRANSITION; un tercero recibe 403. Un retiro concurrente con una edición siempre prevalece.	`offers-withdraw.integration.test.ts`	APROBADO

4.2.4 MOVO-102 · Modelo y máquina de estados de la oferta

MOVO-102 COMPLETADA SPRINT 3 3 SP

[svc-shipments] Schema y máquina de estados de la oferta (Offer)

Alcance final. Historia habilitadora, sin criterios de aceptación en Linear. Define la entidad `Offer` y su máquina de estados de seis valores (`pending`, `accepted`, `rejected`, `withdrawn`, `expired`, `superseded`), con el diagrama de transición en el documento de estados de este tomo. Una restricción única parcial impide dos ofertas `pending` del mismo transportista sobre un envío, y el estado `expired` se deriva en lectura a partir de `expires_at`.

Casos de prueba

Tabla 4.8. Casos de prueba de MOVO-102 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-102.01	-	**Transiciones válidas e inválidas** *Pre:* Cada estado de la máquina. *Pasos:* Evaluar todas las transiciones posibles.	El estado inicial es pending; solo las transiciones del diagrama son válidas; todo estado no terminal tiene al menos una salida; el error identifica origen y destino rechazados.	`offer-state-machine.test.ts`	APROBADO
CP-102.02	-	**Persistencia y vencimiento en lectura** *Pre:* Ofertas pending con vencimiento futuro y pasado. *Pasos:* Crear y leer ofertas.	Se crea en pending con su snapshot; una pending con expiresAt vencido se lee como expired sin modificar la fila; el rango de fecha propuesta se valida contra el envío.	`offer-repository.integration.test.ts`	APROBADO

05

TEMA 4 DE 7 · TRAZABILIDAD DE HISTORIAS DE USUARIO

EP-04 · Ejecución del Envío

Handshake criptográfico, evidencia fotográfica, ruta optimizada, seguimiento en tiempo real, chat, transferencia de receptor y notificaciones.

CONTENIDO

5.1	Descripción del Tema	93
5.2	Historias de Usuario y Trazabilidad	94

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: trazabilidad de las historias del tema, con criterios de aceptación de Linear, diagramas de secuencia derivados del código y casos de prueba derivados de las pruebas automatizadas.

Introducción

Este documento presenta la trazabilidad de las historias de usuario del tema *Ejecución del Envío*: para cada historia, su descripción y criterios de aceptación, los tickets que la implementaron, el diagrama de secuencia que muestra cómo la resuelve el sistema y los casos de prueba que la verifican.

Audiencia

El tribunal evaluador y los futuros mantenedores. Se asume conocimiento de las convenciones del proyecto y de UML.

5.1 Descripción del Tema

EP-04 · Ejecución del Envío es el núcleo técnico de Movo: lo que ocurre desde que el envío tiene un transportista asignado hasta que el paquete llega a manos del receptor. Incluye el **handshake criptográfico** que prueba la transferencia de custodia en el retiro y en la entrega, la **evidencia fotográfica** obligatoria en cada etapa, la **ruta optimizada** multi-parada del viaje, el **seguimiento en tiempo real** de la posición, el **chat** entre las tres partes, la **transferencia de la recepción** a otra persona y las **notificaciones push** que mantienen informados a todos.

Es la segunda capa de confianza del producto (la custodia): la firma ECDSA del dispositivo, la proximidad de 100 m y un evento inmutable reemplazan la confianza ciega entre desconocidos. Los canales en tiempo real (tracking y chat) usan WebSocket nativo con autenticación por JWT (ADR-019).

Servicios involucrados: `gateway`, `movo-svc-shipments` (handshake, evidencia, tracking, chat, transferencia), `movo-svc-pricing-logistics` (optimización de la ruta), `movo-svc-users` (claves de dispositivo, bloqueos y notificaciones), `movo-svc-payments` (liberación de fondos al entregar) y `movo-mobile`; **externos:** Google Routes API, Expo Push Service y Amazon S3.

5.2 Historias de Usuario y Trazabilidad

Tabla 5.1. Historias de usuario del tema.

Historia	Descripción	Estado	Sprint
MOVO-6	Handshake criptográfico de retiro y entrega	Completada	Sprint 4, Sprint 5, Sprint 6
MOVO-10	Mapa de ruta optimizada multi-parada	Completada	Sprint 5, Sprint 6
MOVO-11	Seguimiento en tiempo real del transportista	Completada	Sprint 5, Sprint 6
MOVO-21	Evidencia fotográfica en retiro y entrega	Completada	Sprint 5, Sprint 6
MOVO-24	Notificaciones push en los eventos del envío	Completada	Sprint 3, Sprint 6, Sprint 7, Sprint 8
MOVO-239	Configuración de notificaciones	Completada	Sprint 6
MOVO-240	Catálogo de notificaciones faltantes	Por hacer	Sprint 8
MOVO-26	Chat en tiempo real entre las partes	Completada	Sprint 7
MOVO-275	Transferencia de la recepción a otra persona	Completada	Sprint 7

5.2.1 MOVO-6 · Handshake criptográfico de retiro y entrega

MOVO-6 COMPLETADA SPRINT 4, SPRINT 5, SPRINT 6 14 SP

Como transportista, quiero ejecutar el Cryptographic Handshake para confirmar retiro y entrega segura.

Decisión de diseño (ADR-018). El cedente (emisor en el retiro, transportista en la entrega) genera un desafío con un nonce aleatorio que vive 15 segundos en Redis y lo firma en su dispositivo con ECDSA P-256; el cesionario escanea el QR y el backend verifica la firma contra la clave pública registrada del cedente, exige que ambos estén a 100 m o menos y recién entonces transiciona el envío y registra un evento inmutable con el hash del nonce. Un nonce nuevo invalida al anterior y la confirmación es un compare-and-swap en Postgres, de modo que dos confirmaciones concurrentes con el mismo nonce no pueden ganar ambas.

Criterios de aceptación

1. El emisor inicia la transferencia desde su app. El sistema genera un QR dinamico con TTL de 15 segundos.
2. El QR expira visualmente en pantalla y muestra el tiempo restante.
3. El backend valida firma criptografica correcta y que ambos dispositivos están dentro de 100m de distancia.

4. Si todas las validaciones pasan, el estado del envio se actualiza y se dispara la liberacion de fondos.
5. Si la distancia entre dispositivos supera el umbral, el handshake falla con mensaje de error explicativo.
6. Si el TTL expiro, el cedente puede generar un nuevo QR.
7. El evento de transferencia queda registrado con timestamp, coordenadas y hash del nonce (inmutable).
8. El handshake aplica tanto para el retiro (emisor → transportista) como para la entrega (transportista → receptor).

Trazabilidad de implementación

Tabla 5.2. Tickets de implementación de MOVO-6.

Ticket	Alcance	Sprint	Estado
`MOVO-157`	[svc-users] Registro de clave pública por dispositivo (par de claves asimétricas)	Sprint 4	Completada
`MOVO-158`	[svc-shipments] Handshake: QR dinámico, validación de firma/proximidad GPS y transición de estado	Sprint 4	Completada
`MOVO-159`	[movo-mobile] Pantalla de generación de QR con countdown (cedente de custodia)	Sprint 6	Completada
`MOVO-160`	[movo-mobile] Escaneo de QR y confirmación con GPS (receptor de custodia)	Sprint 6	Completada
`MOVO-195`	[movo-mobile] Par de claves en el dispositivo: generación, guardado en SecureStore y firma del nonce	Sprint 5	Completada

Diagrama de secuencia

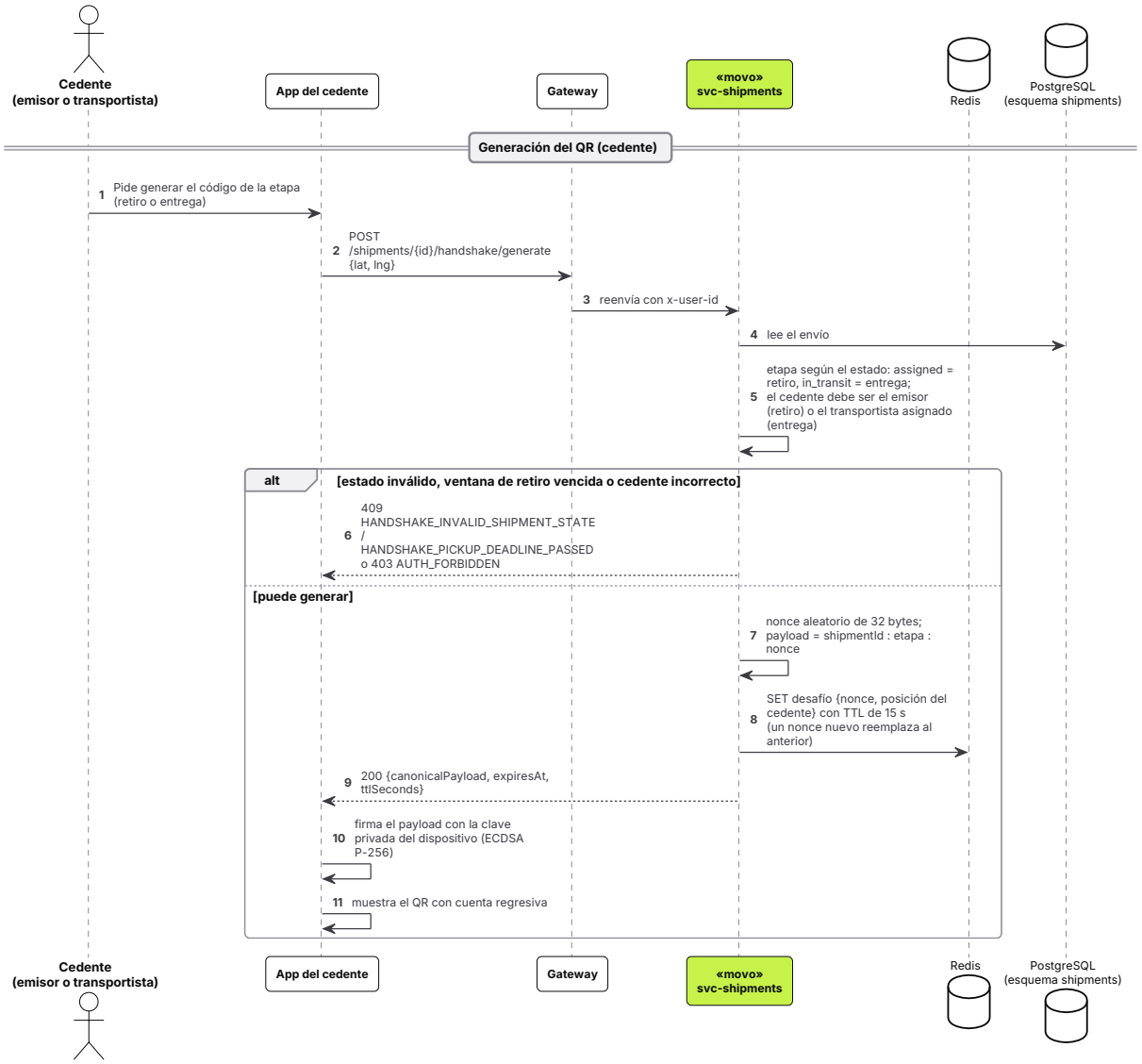


Figura 5.1. Handshake criptográfico: generación del QR y confirmación con firma y proximidad (MOVO-6, MOVO-157 a MOVO-160). Parte 1 de 3: Generación del QR (cedente).

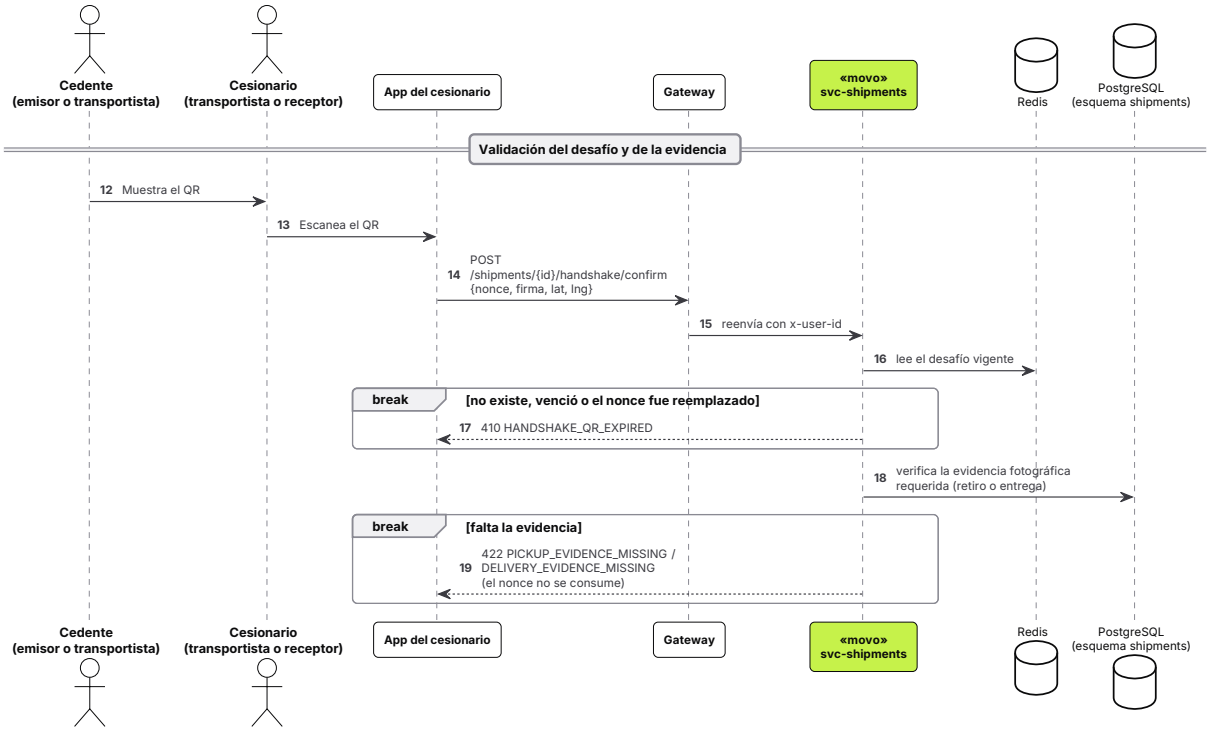


Figura 5.2. Handshake criptográfico: generación del QR y confirmación con firma y proximidad (MOVO-6, MOVO-157 a MOVO-160). Parte 2 de 3: Validación del desafío y de la evidencia.

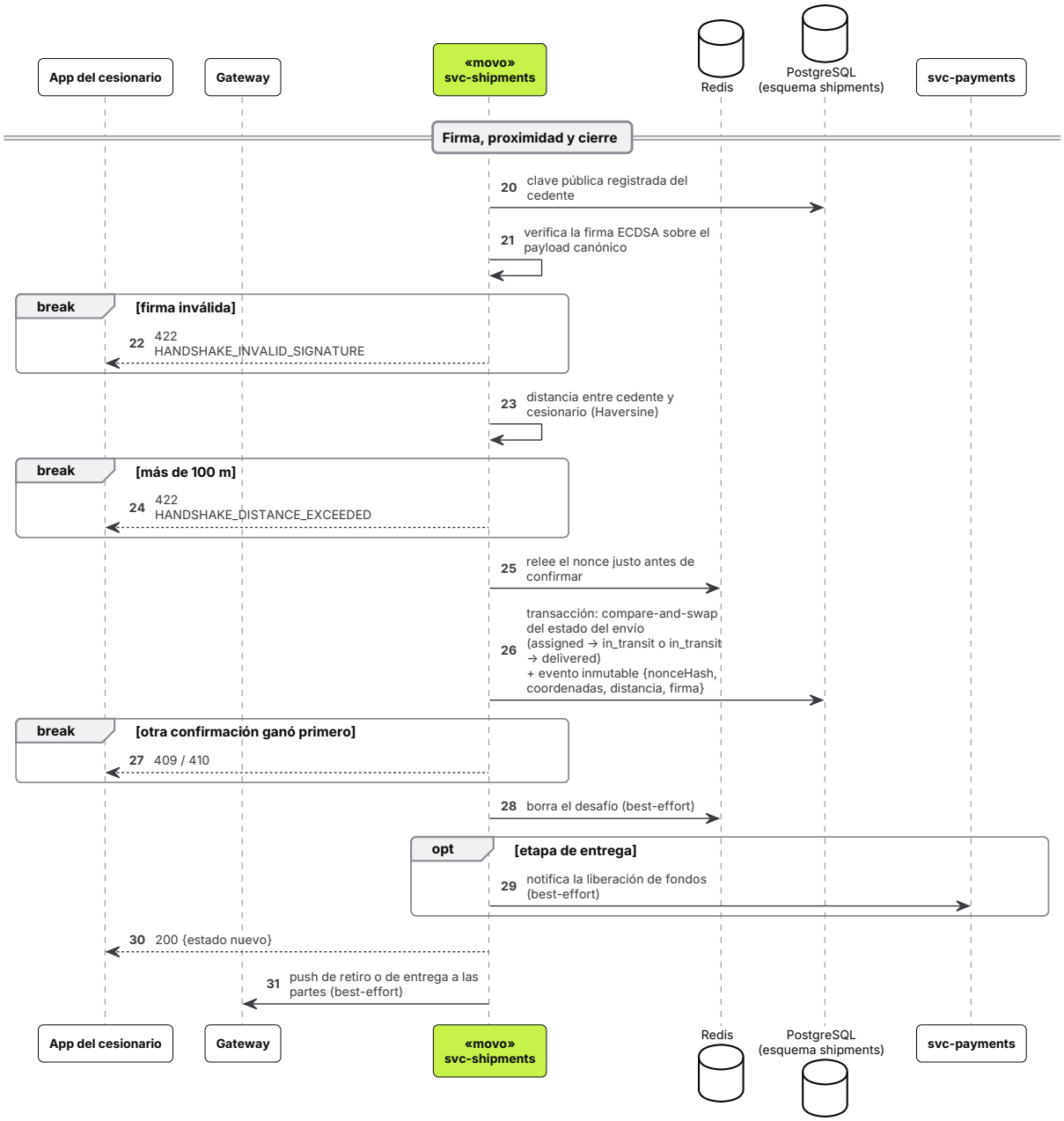


Figura 5.3. Handshake criptográfico: generación del QR y confirmación con firma y proximidad (MOVO-6, MOVO-157 a MOVO-160). Parte 3 de 3: Firma, proximidad y cierre.

Casos de prueba

Tabla 5.3. Casos de prueba de MOVO-6 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-6.01	1, 3, 4, 7, 8	**Retiro exitoso** *Pre:* Envío assigned, emisor con clave de dispositivo registrada y foto de evidencia confirmada. *Pasos:* El emisor genera el QR de retiro; el transportista lo confirma con la firma y su posición.	El envío pasa a in_transit y se persiste el evento con nonceHash, coordenadas y distancia.	`handshake.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-6.02	4, 8	**Entrega exitosa y liberación de fondos** *Pre:* Envío in_transit con foto de evidencia de entrega. *Pasos:* El transportista genera el QR de entrega; el receptor lo confirma.	El envío pasa a delivered y se notifica a svc-payments la liberación del pago.	`handshake.integration.test.ts`	APROBADO
CP-6.03	-	**Solo la parte que corresponde actúa** *Pre:* Envío en retiro o en entrega. *Pasos:* Generar o confirmar con un usuario que no es el cedente o el cesionario de la etapa.	403 AUTH_FORBIDDEN; un ajeno al envío recibe 403 y un envío inexistente 404.	`handshake.integration.test.ts` `handshake-service.test.ts`	APROBADO
CP-6.04	-	**Estados y plazo de retiro** *Pre:* Envío published, o con la ventana de retiro vencida más la gracia del barrido. *Pasos:* POST /shipments/:id/handshake/generate.	409 HANDSHAKE_INVALID_SHIPMENT_STATE para estados distintos de assigned e in_transit; 409 HANDSHAKE_PICKUP_DEADLINE_PASSED con el plazo vencido.	`handshake.integration.test.ts` `handshake-service.test.ts`	APROBADO
CP-6.05	2, 6	**QR vencido o reemplazado** *Pre:* Un QR de más de 15 s, o uno nuevo generado después. *Pasos:* Confirmar con el nonce viejo; generar un nuevo QR.	410 HANDSHAKE_QR_EXPIRED; el cedente puede generar un QR nuevo, que invalida el anterior.	`handshake.integration.test.ts` `handshake-service.test.ts`	APROBADO
CP-6.06	3	**Firma inválida** *Pre:* Desafío vigente. *Pasos:* Confirmar con una firma errónea, de otro payload, de otra clave, corrupta, o con una clave pública malformada.	422 HANDSHAKE_INVALID_SIGNATURE sin lanzar 500; el payload incluye la etapa para evitar replay entre retiro y entrega.	`handshake.integration.test.ts` `handshake-crypto.test.ts`	APROBADO
CP-6.07	3, 5	**Distancia mayor a 100 m** *Pre:* Transportista a más de 100 m del emisor. *Pasos:* Confirmar el retiro.	422 HANDSHAKE_DISTANCE_EXCEEDED con mensaje que indica la distancia; el intento es reintentable dentro del mismo TTL.	`handshake.integration.test.ts` `geo.test.ts`	APROBADO
CP-6.08	3	**Cedente sin clave de dispositivo** *Pre:* Usuario sin clave pública registrada. *Pasos:* Confirmar un handshake que generó ese usuario.	409 HANDSHAKE_CEDENTE_KEY_MISSING. La clave se registra con POST /users/me/device-key y una clave nueva reemplaza a la anterior (una sola vigente por usuario).	`handshake.integration.test.ts` `users.device-key.integration.test.ts`	APROBADO
CP-6.09	7	**Confirmaciones concurrentes** *Pre:* Desafío vigente. *Pasos:* Dos confirmaciones simultáneas con el mismo nonce.	Exactamente una gana; la otra recibe error y no duplica el evento.	`handshake.integration.test.ts`	APROBADO
CP-6.10	-	**Evidencia fotográfica obligatoria (MOVO-21)** *Pre:* Etapa sin fotos de evidencia confirmadas, o con una foto solo presignada. *Pasos:* Confirmar el retiro o la entrega.	422 PICKUP_EVIDENCE_MISSING o DELIVERY_EVIDENCE_MISSING; el nonce no se consume y el intento se reintenta apenas exista la evidencia.	`handshake.integration.test.ts`	APROBADO

5.2.2 MOVO-10 · Mapa de ruta optimizada multi-parada

MOVO-10 COMPLETADA SPRINT 5, SPRINT 6 16 SP

[movo-mobile] Pantalla y mapa de ruta optimizada multi-parada

> **Decisión de diseño.** La ruta la calcula `movo-svc-pricing-logistics` como un VRPTW con Google OR-Tools, con matriz de tiempos de Google Routes API (ADR-015). Si el optimizador no responde, `movo-svc-shipments` degrada a una ruta heurística sin optimizar en lugar de dejar al transportista sin mapa. Todo paquete ejecutable aporta al menos una parada (invariante de MOVO-277).

Criterios de aceptación

1. **Visualización de Ruta Multi-Parada:**
2. La pantalla del viaje activo consume el itinerario ordenado provisto por el backend (calculado por `svc-pricing-logistics` en MOVO-220).
3. Renderiza un mapa interactivo con la polyline completa trazada.
4. **Diferenciación de Pines:**
5. Pines diferenciados visualmente por tipo: origen del viaje, puntos de retiro (pickup), puntos de entrega (dropoff) y destino final del viaje.
6. **Detalle Cronológico:**
7. Listado ordenado cronológicamente de paradas con nombre del punto/dirección y tiempo estimado de llegada (ETA).
8. **Manejo de Estados:**
9. La ruta se actualiza dinámicamente si el transportista completa o cancela un punto del recorrido.
10. Si no hay señal de GPS o la ruta no puede calcularse, se notifica claramente al usuario.

Trazabilidad de implementación

Tabla 5.4. Tickets de implementación de MOVO-10.

Ticket	Alcance	Sprint	Estado
MOVO-205	[svc-pricing-logistics] Endpoint de optimización VRPTW multi-envío: del prototipo de ADR-013 a producción	Sprint 5	Completada
MOVO-206	[svc-shipments] Agregación de paradas del transportista y contrato con pricing-logistics	Sprint 5	Completada
MOVO-207	[movo-mobile] Mapa de ruta optimizada: paradas ordenadas, ETA y recálculo al completar una parada	Sprint 6	Completada
MOVO-235	[svc-shipments] Scopear GET /shipments/my-route al viaje iniciado (en vez de todos los envíos activos)	Sprint 5	Completada
MOVO-237	[svc-shipments][movo-mobile] Navegación turn-by-turn: RouteProvider extendido con modo per-trip/live (ADR-022) + botón deep-link a Maps/Waze	Sprint 6	Completada

Diagrama de secuencia

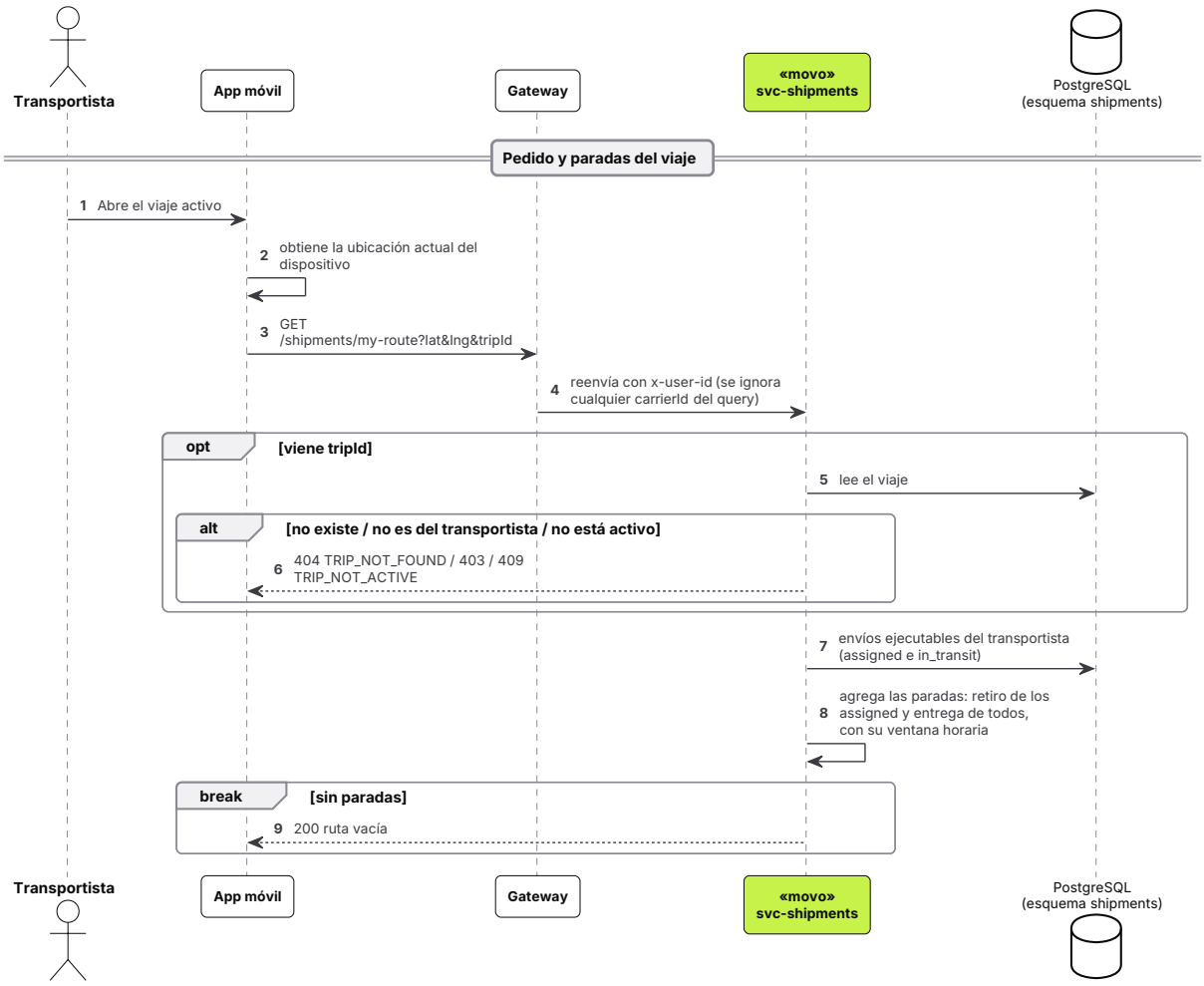


Figura 5.4. Ruta optimizada del viaje activo con paradas ordenadas y ETA (MOVO-10, MOVO-206, MOVO-235). Parte 1 de 2: Pedido y paradas del viaje.

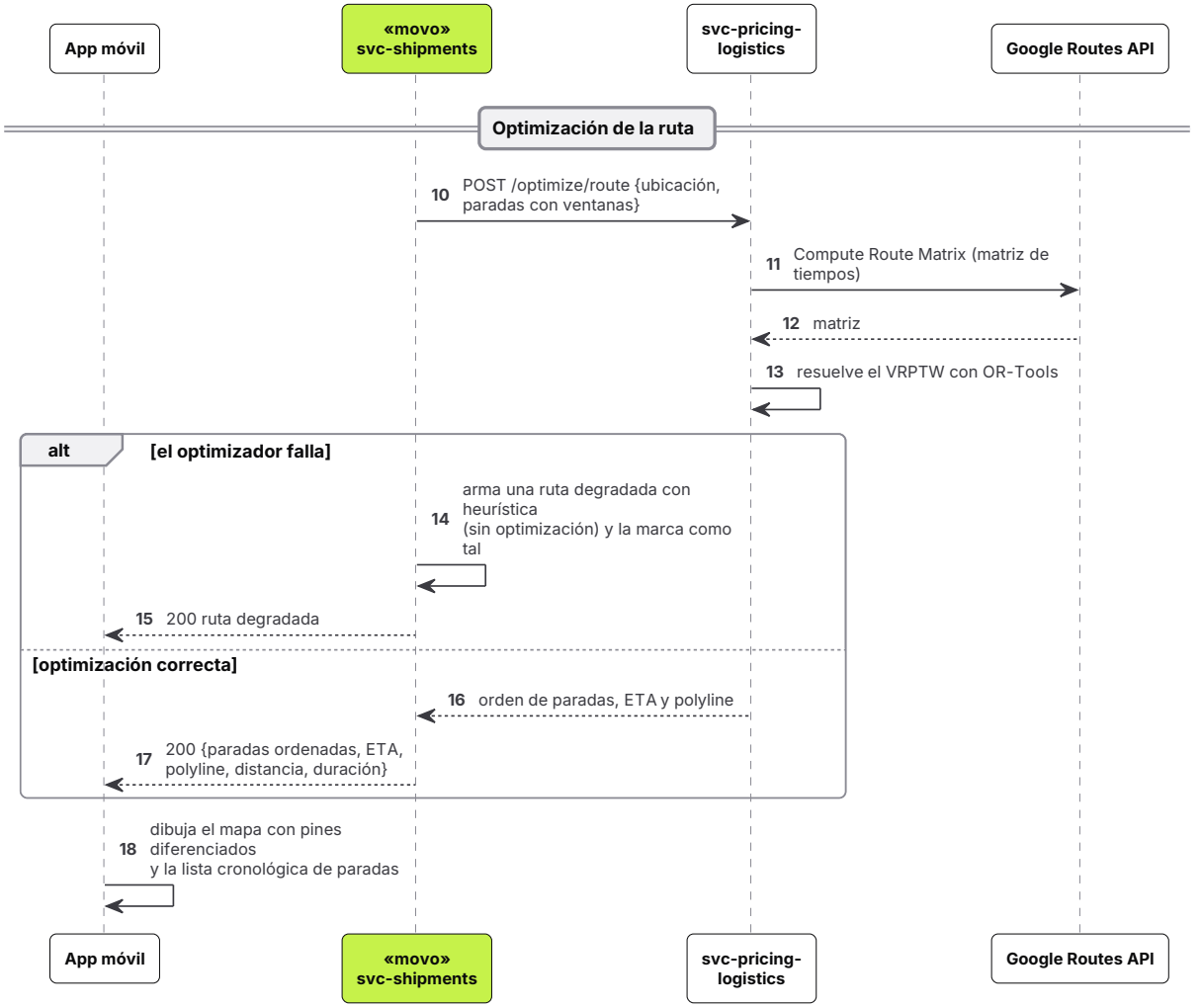


Figura 5.5. Ruta optimizada del viaje activo con paradas ordenadas y ETA (MOVO-10, MOVO-206, MOVO-235). Parte 2 de 2: Optimización de la ruta.

Casos de prueba

Tabla 5.5. Casos de prueba de MOVO-10 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-10.01	2, 3, 7	**Ruta del transportista** *Pre:* Transportista con envíos assigned e in_transit. *Pasos:* GET /shipments/my-route con su ubicación.	200 con las paradas ordenadas, ETA, polyline, distancia y duración; la fecha de retiro se combina con la hora en huso UTC-3.	shipments-my-route.routes.test.ts shipments-my-route.service.test.ts carrier-route.test.ts	APROBADO
CP-10.02	5	**Paradas por tipo de envío** *Pre:* Envíos en assigned y en in_transit, solos y combinados. *Pasos:* Agregar las paradas.	Un assigned aporta retiro y entrega; un in_transit solo entrega; se combinan varios envíos y todo paquete aporta al menos una parada.	carrier-route.test.ts	APROBADO
CP-10.03	-	**Parámetros y autenticación** *Pre:* Pedido sin identidad, sin coordenadas, con latitud inválida o con un carrierId en el query. *Pasos:* GET /shipments/my-route.	401 AUTH_REQUIRED; 400 VALIDATION_FAILED; el carrierId del query se ignora y se usa el del header.	shipments-my-route.routes.test.ts	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-10.04	10	**Viaje no iniciado** *Pre:* tripId de un viaje en declared o ya finalizado. *Pasos:* GET /shipments/my-route?tripId=...	409 TRIP_NOT_ACTIVE; con un viaje ajeno, 403; inexistente, 404.	`shipments-my-route.service.test.ts`	APROBADO
CP-10.05	10	**Degradación si el optimizador falla** *Pre:* svc-pricing-logistics caído. *Pasos:* GET /shipments/my-route.	200 con una ruta degradada que conserva todas las paradas; sin paradas, ruta vacía.	`shipments-my-route.service.test.ts`	APROBADO
CP-10.06	3	**Polyline de un trayecto** *Pre:* Origen y destino. *Pasos:* GET /shipments/route.	Devuelve polyline, distancia y duración; 401 sin autenticación y 400 con parámetros incompletos.	`shipments-route.test.ts`	APROBADO
CP-10.07	2	**Motor de optimización** *Pre:* Conjunto de paradas con ventanas horarias. *Pasos:* POST /optimize/route.	Devuelve el orden óptimo que respeta las ventanas y usa la matriz de tiempos.	`test_optimize.py` `test_evaluate.py`	APROBADO

5.2.3 MOVO-11 · Seguimiento en tiempo real del transportista

MOVO-11 COMPLETADA SPRINT 5, SPRINT 6 27 SP

Como emisor o receptor, quiero ver la ubicación del transportista en tiempo real en un mapa interactivo, para saber exactamente dónde está mi paquete en cada momento del envío.

Decisión de diseño (ADR-019). El canal usa WebSocket nativo sobre Fastify (no Socket.io), autenticado con el JWT en el handshake y con cierre por código (4001 token, 4003 acceso, 4004 inexistente, 4009 envío terminado). La última posición conocida vive en Redis para que un suscriptor nuevo vea el mapa de inmediato, y solo se persiste una fila cada cierto intervalo para acotar el almacenamiento. La historia no tiene criterios de aceptación en Linear; el alcance quedó en sus tickets hijos.

Trazabilidad de implementación

Tabla 5.6. Tickets de implementación de MOVO-11.

Ticket	Alcance	Sprint	Estado
`MOVO-200`	[Spike] Canal de tiempo real: verificar qué existe hoy, elegir tecnología y escribir el ADR	Sprint 5	Completada
`MOVO-201`	[svc-shipments] Canal de tiempo real: conexión autenticada y autorización de suscripción por envío	Sprint 5	Completada
`MOVO-202`	[svc-shipments] Ingesta de posiciones del transportista: última posición en Redis, traza persistida cada 45s y purga	Sprint 6	Completada
`MOVO-203`	[movo-mobile] Emisión de ubicación del transportista en foreground y background	Sprint 6	Completada
`MOVO-204`	[movo-mobile] Mapa de seguimiento en vivo para emisor y receptor	Sprint 6	Completada

Ticket	Alcance	Sprint	Estado
MOVO-242	[movo-mobile] Emisión de ubicación del transportista en background: permisos en dos etapas, cadencia reducida y medición de batería	Sprint 6	Completada
MOVO-250	[svc-shipments] Ajustes de ingesta y canal de tracking: orden por captured_at, cadencia sobre la traza offline, envío por lotes y eventos de estado	Sprint 6	Completada
MOVO-251	[svc-shipments] Gatear ingesta y lectura de tracking por Trip activo, no por Shipment.status	Sprint 6	Completada
MOVO-252	[movo-mobile] CTA “Iniciar viaje” para el transportista	Sprint 6	Completada

Diagrama de secuencia

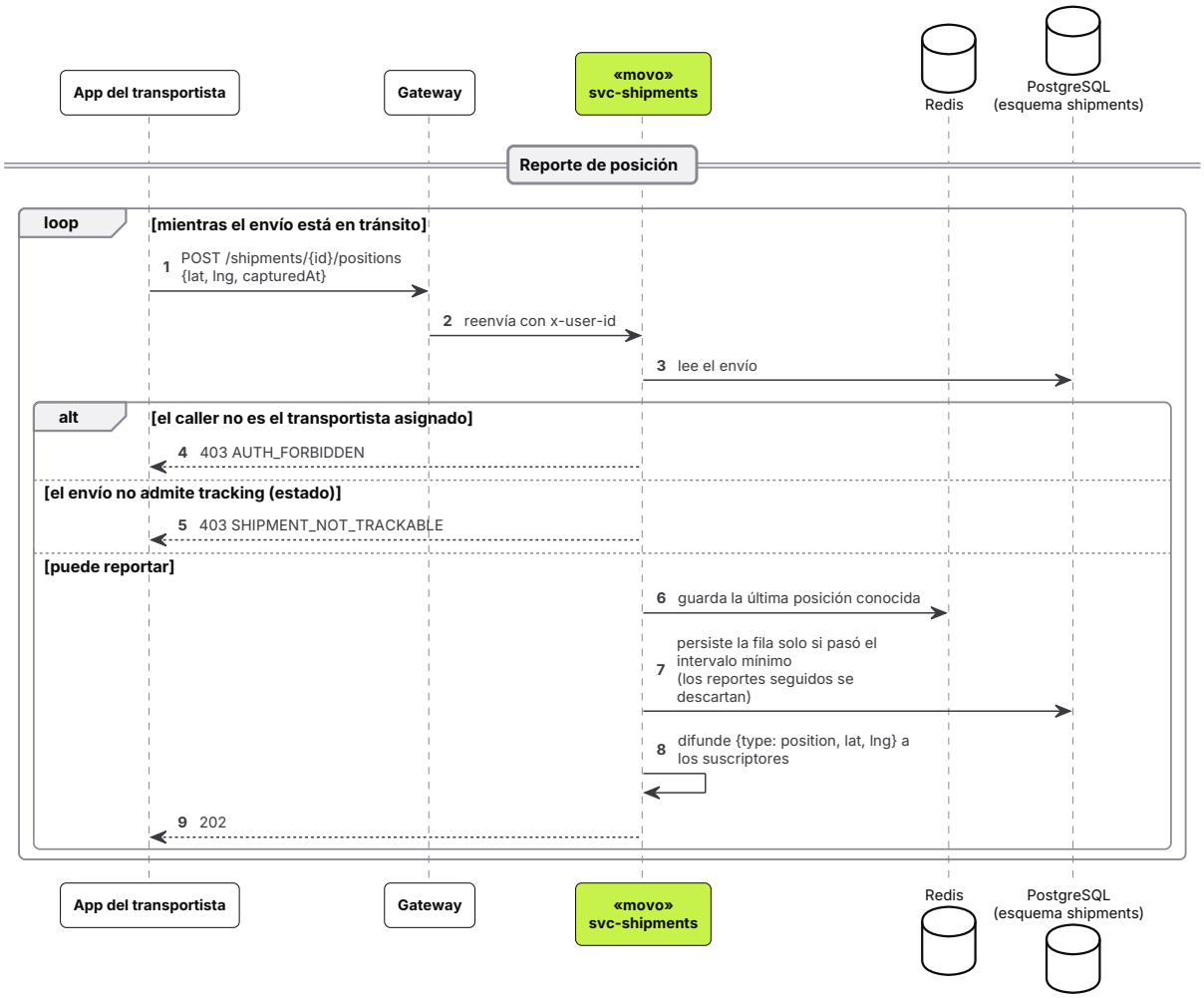


Figura 5.6. Reporte de posición y seguimiento en vivo por WebSocket (MOVO-11, MOVO-202, MOVO-251). Parte 1 de 2: Reporte de posición.

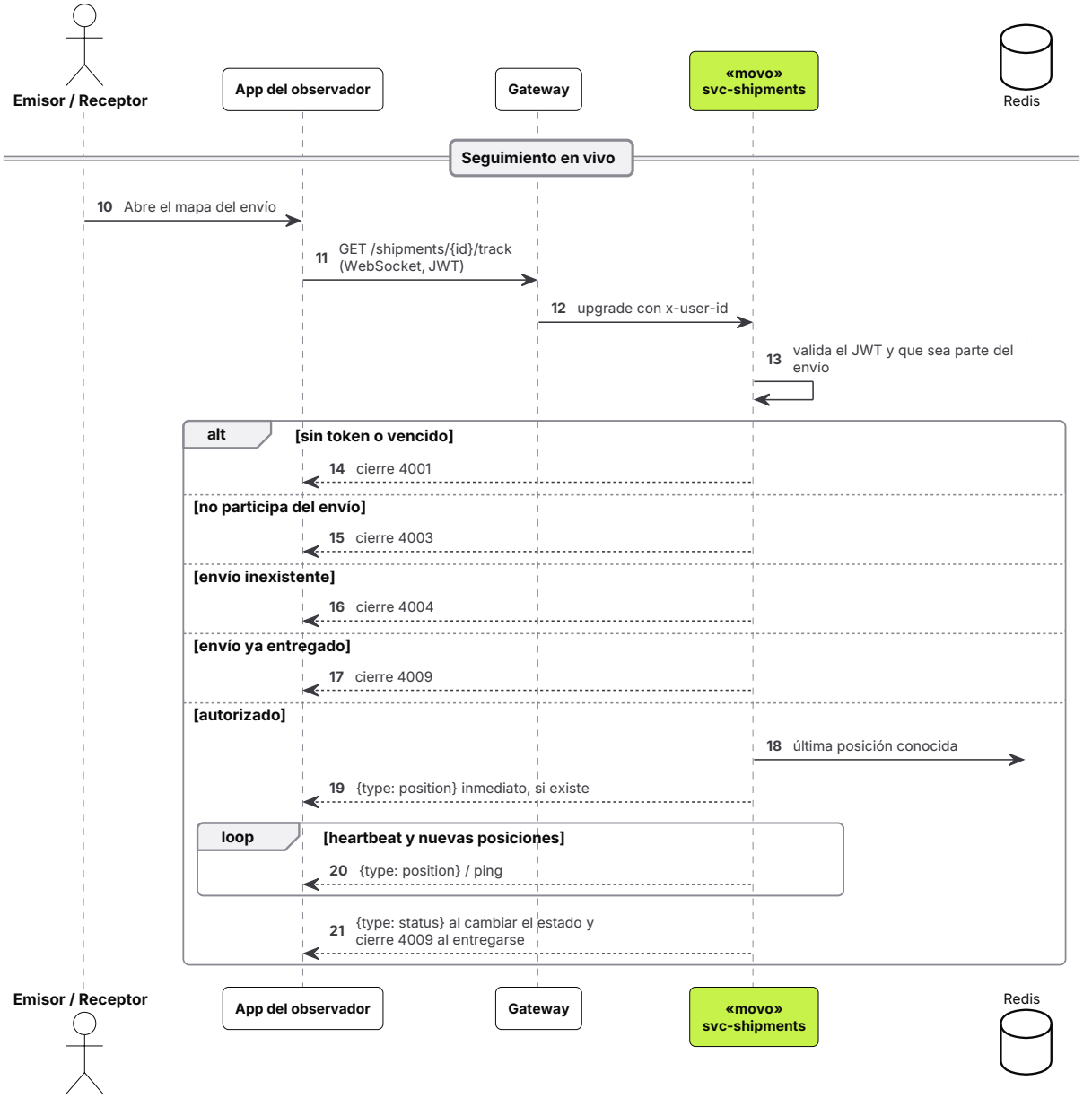


Figura 5.7. Reporte de posición y seguimiento en vivo por WebSocket (MOVO-11, MOVO-202, MOVO-251). Parte 2 de 2: Seguimiento en vivo.

Casos de prueba

Tabla 5.7. Casos de prueba de MOVO-11 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-11.01	-	**Reporte de posición del transportista** *Pre:* Envío in_transit y transportista asignado. *Pasos:* POST /shipments/:id/positions con lat, lng y capturedAt.	202; se actualiza la última posición en Redis y se persiste la fila; reportes seguidos descartan la persistencia salvo el primero.	`positions-report.integration.test.ts` `position-service.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-11.02	-	**Solo el transportista reporta** *Pre:* Emisor, receptor y transportista. *Pasos:* Reportar posición con cada uno, sin identidad, y sobre un envío inexistente.	403 para emisor y receptor; 401 sin identidad; 404 inexistente; 403 SHIPMENT_NOT_TRACKABLE si el estado no admite tracking.	`positions-report.integration.test.ts`	APROBADO
CP-11.03	-	**Validación de coordenadas y fechas** *Pre:* Pedido con lat o lng fuera de rango o capturedAt no parseable. *Pasos:* POST /shipments/:id/positions.	400; la operación figura en el Swagger generado.	`positions-report.integration.test.ts`	APROBADO
CP-11.04	-	**Viaje con varios paquetes** *Pre:* Viaje mixto con un envío in_transit y otro assigned, y un envío assigned con el viaje todavía declared. *Pasos:* Reportar la posición del viaje en cada caso.	La fila se asocia al envío in_transit; con un envío assigned y el viaje active se responde 202 sin guardar traza; con el viaje declared, 403 SHIPMENT_NOT_TRACKABLE.	`positions-report.integration.test.ts`	APROBADO
CP-11.05	-	**Conexión WebSocket autenticada** *Pre:* Token válido, ajeno, ausente o malformado; envío inexistente. *Pasos:* Abrir /shipments/:id/track.	Acepta al emisor, receptor y transportista asignado; cierra con 4003 para un ajeno, 4001 sin token o malformado y 4004 si no existe.	`tracking.integration.test.ts`	APROBADO
CP-11.06	-	**Última posición y cierre al terminar** *Pre:* Última posición conocida en Redis. *Pasos:* Conectar un suscriptor; entregar el envío.	El suscriptor recibe la última posición al conectar; al entregarse el envío (por el handshake real) recibe el estado final y la conexión se cierra con 4009; un envío ya entregado rechaza la conexión.	`tracking.integration.test.ts`	APROBADO
CP-11.07	-	**Registro de conexiones y purga** *Pre:* Conexiones abiertas y posiciones antiguas. *Pasos:* Registrar y cerrar conexiones; correr el barrido de purga.	El registro lleva las conexiones por envío; el barrido purga las posiciones más viejas que la retención configurada, con lock en Redis para que corra una sola réplica.	`realtime-registry.test.ts` `carrier-position-purge-sweep.test.ts` `position-repository.integration.test.ts`	APROBADO

5.2.4 MOVO-21 · Evidencia fotográfica en retiro y entrega

MOVO-21 COMPLETADA SPRINT 5, SPRINT 6 15 SP

Como transportista, quiero registrar fotografías al retirar y entregar cada paquete, para dejar evidencia visual del estado del paquete.

> **Decisión de diseño (ADR-007).** Las fotos viajan directo de la app a S3 con URLs prefirmadas y no son públicas: se leen con URLs de lectura temporales y solo las ven las partes y el administrador. El handshake no avanza sin al menos una foto confirmada de la etapa.

Criterios de aceptación

1. Al llegar al punto de retiro, el sistema solicita al transportista tomar al menos 1 foto del paquete antes de confirmar el retiro.
2. Al llegar al punto de entrega, el sistema solicita al transportista tomar al menos 1 foto del paquete antes de iniciar el handshake.
3. Las fotos quedan vinculadas al envio y son visibles para emisor, receptor y admin.
4. El transportista no puede avanzar al siguiente paso del flujo sin haber cargado las fotos requeridas.
5. Las fotos se almacenan de forma segura y no son publicamente accesibles por URL directa.
6. Si la camara del dispositivo no esta disponible, el sistema muestra un error claro.

Trazabilidad de implementación

Tabla 5.8. Tickets de implementación de MOVO-21.

Ticket	Alcance	Sprint	Estado
`MOVO-196`	[svc-shipments] Exigir evidencia fotográfica del stage antes de permitir el handshake	Sprint 5	Completada
`MOVO-197`	[movo-mobile] Step reusable de captura de evidencia fotográfica (retiro y entrega)	Sprint 5	Completada
`MOVO-198`	[movo-mobile] Wizard de retiro del transportista: evidencia, escaneo de QR y confirmación	Sprint 6	Completada
`MOVO-199`	[movo-mobile] Wizard de entrega del transportista: evidencia y generación de QR	Sprint 6	Completada

Casos de prueba

Tabla 5.9. Casos de prueba de MOVO-21 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-21.01	1, 2, 4	**El handshake exige la evidencia** *Pre:* Etapa de retiro o de entrega sin foto confirmada. *Pasos:* Consultar el estado de la evidencia y confirmar el handshake.	En assigned la etapa es pickup y en in_transit es delivery, con satisfied=false sin evidencia y true con ella; la confirmación se rechaza hasta que exista.	`handshake.integration.test.ts`	APROBADO
CP-21.02	1, 2	**Subida de fotos por etapa** *Pre:* Transportista asignado. *Pasos:* Presignar, subir y confirmar fotos de pickup y de delivery; probar con un transportista no asignado.	El transportista asignado puede registrarlas; uno no asignado recibe 403; se aceptan hasta 5 fotos por etapa y la sexta responde 422 PHOTO_STAGE_LIMIT_EXCEEDED.	`photos.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-21.03	5	**Acceso restringido** *Pre:* Fotos del envío. *Pasos:* Listarlas como emisor, receptor, admin, transportista asignado, transportista rechazado y tercero.	Las partes y el admin las ven con URLs de lectura temporales; transportista rechazado y terceros reciben 403.	`photos.integration.test.ts`	APROBADO
CP-21.04	3	**Confirmación idempotente y huérfanas** *Pre:* Foto subida o presignada sin confirmar. *Pasos:* Confirmar dos veces la misma clave; correr el barrido de huérfanas.	No se duplica la fila; confirmar un objeto inexistente responde 422; el barrido elimina las fotos nunca confirmadas.	`photos.integration.test.ts` `orphan-photo-sweep.test.ts`	APROBADO

5.2.5 MOVO-24 · Notificaciones push en los eventos del envío

MOVO-24 COMPLETADA SPRINT 3, SPRINT 6, SPRINT 7, SPRINT 8 15 SP

Como emisor o receptor, quiero recibir notificaciones push en los eventos clave del envío, para estar informado durante el mismo.

Alcance final. La historia no tiene criterios de aceptación en Linear. Las notificaciones salen de `svc-users` a través de Expo, y cualquier servicio las dispara con `POST /internal/notifications/push`. Cada push declara una categoría del catálogo compartido (12 categorías); un servicio emisor falla en modo best-effort y nunca bloquea la operación que la origina. Las categorías con `quietHoursExempt` (proximidad y seguridad de la cuenta) ignoran el horario de silencio.

Trazabilidad de implementación

Tabla 5.10. Tickets de implementación de MOVO-24.

Ticket	Alcance	Sprint	Estado
`MOVO-106`	[svc-users] Registro de push token y adapter PushNotificationProvider	Sprint 3	Completada
`MOVO-107`	[movo-mobile] Permisos, registro de push token y manejo de notificaciones recibidas	Sprint 3	Completada
`MOVO-108`	[svc-shipments] Disparo de notificaciones push en creación de envío y ofertas	Sprint 3	Completada
`MOVO-239`	[svc-users] [movo-mobile] Notificaciones: pantalla de configuración con toggle maestro y desglose granular (push existentes)	Sprint 6	Completada
`MOVO-240`	Notificaciones: catálogo de notificaciones faltantes	Sprint 8	Por hacer
`MOVO-274`	[svc-users] [svc-shipments] Notificaciones push: KYC, cuenta y seguridad, y calificación pendiente	Sprint 6	Completada
`MOVO-281`	[movo-mobile] [svc-shipments] Navegación al tocar las notificaciones push que hoy no llevan a ningún lado	Sprint 7	Completada

Diagrama de secuencia

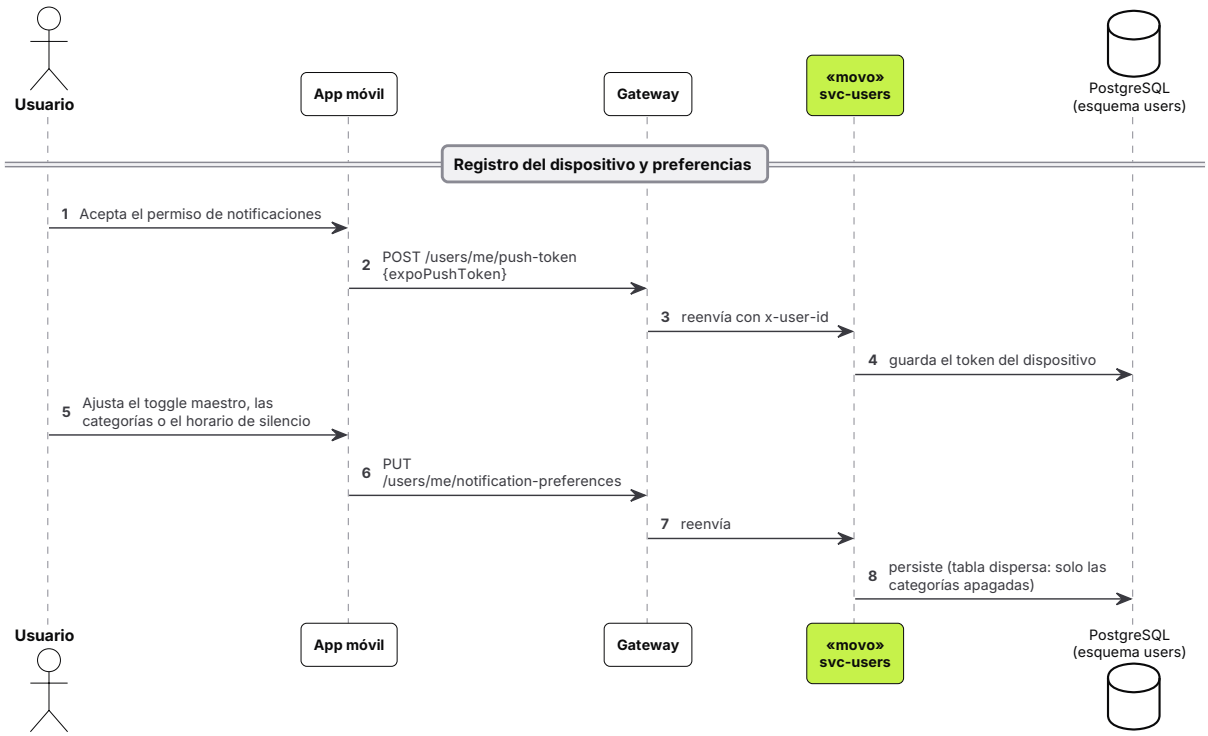


Figura 5.8. Envío de una push respetando las preferencias del usuario (MOVO-24, MOVO-239, MOVO-245). Parte 1 de 2: Registro del dispositivo y preferencias.

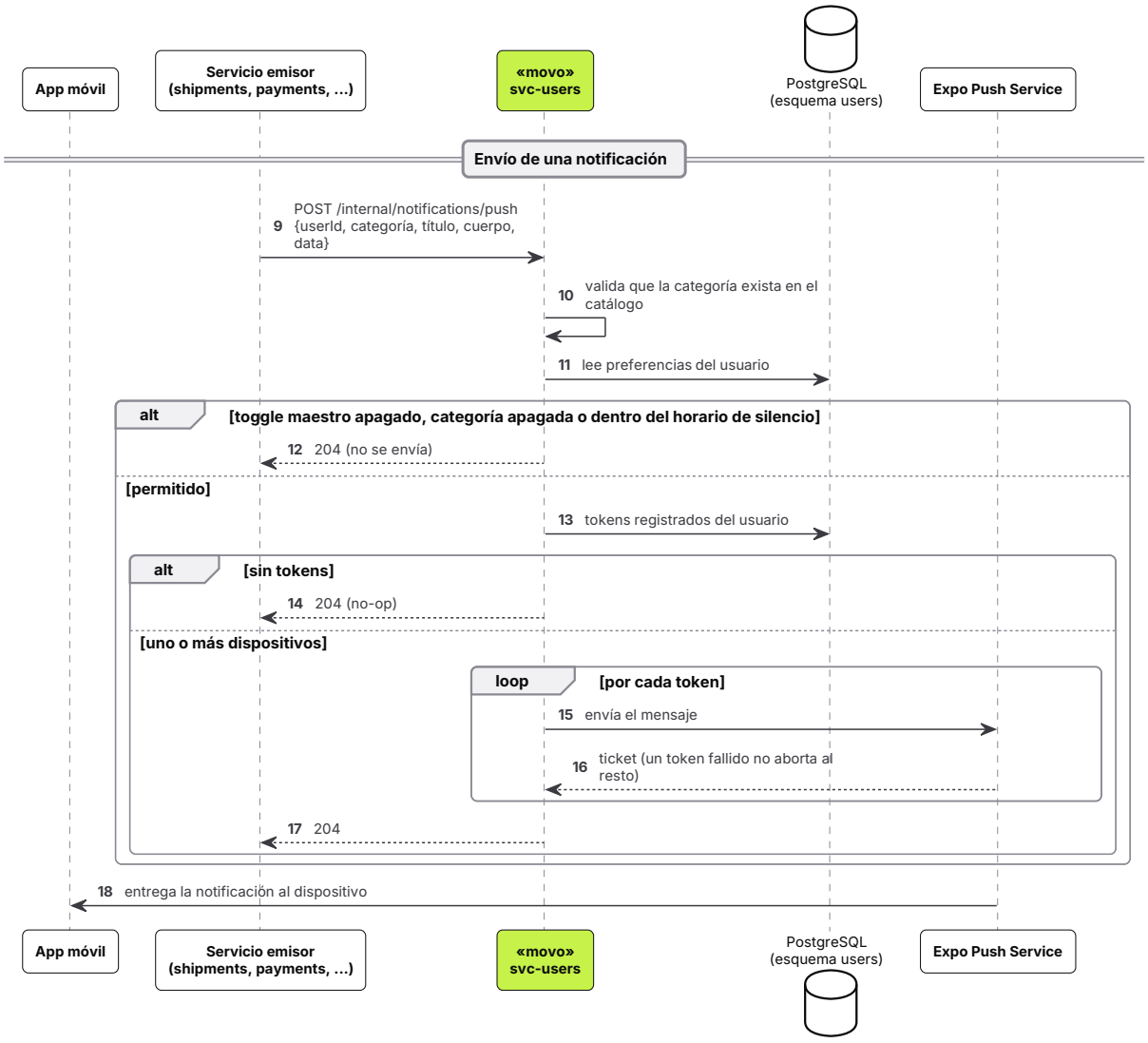


Figura 5.9. Envío de una push respetando las preferencias del usuario (MOVO-24, MOVO-239, MOVO-245). Parte 2 de 2: Envío de una notificación.

Casos de prueba

Tabla 5.11. Casos de prueba de MOVO-24 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-24.01	-	**Envío a todos los dispositivos del usuario** *Pre:* Usuario con 0, 1 o N tokens registrados. *Pasos:* POST /internal/notifications/push.	204 sin envío si no hay tokens; con uno o N tokens se envía a todos y un token que falla no aborta al resto.	`notifications.push.integration.test.ts`	APROBADO
CP-24.02	-	**Categoría obligatoria y no pública** *Pre:* Pedido sin categoría o con una categoría desconocida. *Pasos:* POST /internal/notifications/push.	400 sin category; una categoría desconocida nunca se envía; el endpoint no figura en el Swagger público.	`notifications.push.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-24.03	-	**Eventos que disparan una push** *Pre:* Evento de negocio (oferta, retiro, entrega, KYC, seguridad de la cuenta). *Pasos:* Disparar el evento en el servicio origen.	Se llama a /internal/notifications/push con el copy y la categoría del trigger; un fallo del envío se registra como warn y no rompe la operación.	`notifications-client.test.ts` `send-trigger-push.test.ts` `kyc.push.test.ts` `account-security.push.test.ts`	APROBADO

5.2.6 MOVO-239 · Configuración de notificaciones

MOVO-239 COMPLETADA SPRINT 6 5 SP

[svc-users] [movo-mobile] Notificaciones: pantalla de configuración con toggle maestro y desglose granular (push existentes)

Criterios de aceptación

1. El toggle maestro apagado impide cualquier push, sin importar el estado de los toggles individuales, lo respeta el servidor, no solo el cliente.
2. Cada uno de los 4 toggles de categoría apaga específicamente esos triggers y ninguno más (verificable con un test de integración por categoría en `svc-users`).
3. Apagar el permiso del SO no rompe la pantalla ni miente sobre el estado real de las push.
4. La preferencia se persiste server-side (sobrevive reinstalar la app / cambiar de dispositivo, a diferencia de una preferencia solo local).
5. Toda cuenta existente arranca con todo habilitado (sin regresión para quien no toca nada).

Trazabilidad de implementación

Tabla 5.12. Tickets de implementación de MOVO-239.

Ticket	Alcance	Sprint	Estado
`MOVO-245`	[svc-users] [svc-shipments] Notificaciones: modelo de preferencias, toggle maestro, horario de silencio y enforcement en el envío	Sprint 6	Completada
`MOVO-246`	[movo-mobile] Pantalla de configuración de notificaciones: toggle maestro, categorías y horario de silencio	Sprint 6	Completada

Casos de prueba

Tabla 5.13. Casos de prueba de MOVO-239 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-239.01	5	**Valores por defecto** *Pre:* Usuario que nunca tocó la configuración. *Pasos:* GET /users/me/notification-preferences.	Todo habilitado y horario de silencio apagado; sin identidad, 401.	`notification-preferences.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-239.02	1, 4	**Toggle maestro persistente** *Pre:* Usuario autenticado. *Pasos:* Apagar el toggle maestro y consultar en una request nueva.	La preferencia persiste en el servidor y ninguna push se envía, sin importar las categorías.	`notification-preferences.integration.test.ts` `notifications.push.integration.test.ts`	APROBADO
CP-239.03	2	**Categorías independientes** *Pre:* Usuario autenticado. *Pasos:* Apagar una categoría, volver a encenderla, y probar una categoría desconocida.	Solo esa categoría se silencia; al reactivarla se borra la fila (tabla dispersa); una categoría desconocida responde 400.	`notification-preferences.integration.test.ts` `notifications.push.integration.test.ts`	APROBADO
CP-239.04	–	**Horario de silencio** *Pre:* Usuario autenticado. *Pasos:* Activar el horario con una franja válida e inválida.	Se persiste la franja; un formato de hora inválido responde 400; no se alteran las preferencias de otro usuario.	`notification-preferences.integration.test.ts`	APROBADO

5.2.7 MOVO-240 · Catálogo de notificaciones faltantes

MOVO-240 POR HACER SPRINT 8

Notificaciones: catálogo de notificaciones faltantes

Estado. Historia en *Por hacer*. Es el catálogo ampliado de notificaciones: varias categorías ya se implementaron como parte de otras historias (ver MOVO-24 y MOVO-245), por lo que lo pendiente es completar los triggers que aún no tienen un `sendPush` real. Los casos de prueba se definirán junto con cada trigger.

Casos de prueba

Tabla 5.14. Casos de prueba de MOVO-240 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-240.01	–	**Trigger sin implementar** *Pre:* Categoría del catálogo con implemented=false. *Pasos:* Intentar enviar una push de esa categoría.	La categoría desconocida o no implementada nunca se envía.	`notifications.push.integration.test.ts`	PENDIENTE

5.2.8 MOVO-26 · Chat en tiempo real entre las partes

MOVO-26 COMPLETADA SPRINT 7 3 SP

[svc-shipments] Chat en tiempo real entre las partes de un envío (emisor, receptor y transportista)

> **Decisión de diseño.** El chat existe mientras el envío tiene transportista asignado y se cierra de inmediato si el envío se cancela o el receptor lo rechaza. Un bloqueo (DD-026) no impide escribir: a quien tiene el bloqueo con el autor el mensaje le llega vacío y marcado como oculto, de modo que el bloqueador no ve contenido y el bloqueado no se entera. Las palabras ofensivas se enmascaran antes de guardar y difundir.

Criterios de aceptación

1. **Disponibilidad.** Las partes (emisor, receptor y transportista asignado) escriben desde que el transportista es asignado (`assignment_pending`, `assigned_unfunded`, `assigned`, `in_transit`) hasta la entrega. Entregado (`delivered` / `completed`) el chat queda en **solo lectura** (`403 CHAT_READ_ONLY` al escribir); a las 24 h de la entrega se **archiva** (`archived: true`): sigue legible, nunca se cierra. Un envío en disputa también queda en solo lectura.
2. **Texto.** Los mensajes son de texto, de 1 a 1000 caracteres después de recortar espacios. Las imágenes quedan fuera del MVP.
3. **Tiempo real.** Los mensajes se envían por `POST /shipments/:id/chat/messages` y se reciben por el WebSocket `GET /shipments/:id/chat`, que los empuja a todos los dispositivos conectados de las partes, incluido el del autor. El envío es **idempotente** por `clientMessageId`: un reenvío devuelve el mensaje ya guardado (200) sin duplicarlo ni volver a difundirlo.
4. **Historial.** `GET /shipments/:id/chat/messages` pagina por cursor, del más reciente al más antiguo, e informa `access` (`open` / `read_only`), `archived` y si quien lo pide ya reportó la conversación (`reportedByMe`). El historial queda vinculado al envío y se conserva como evidencia.
5. **Integrantes.** `GET /shipments/:id/chat/participants` devuelve emisor, transportista y receptor con nombre, primer nombre, foto y rol, para el encabezado (“Ana, Diego y Luis”) y los avatares. Cada mensaje trae además `authorName` (primer nombre al escribir).
6. **Admin.** El administrador puede leer el historial de cualquier envío, aun cancelado, para resolver una disputa. Nunca escribe.
7. **Sin transacción en común, sin chat.** Un usuario que no es parte del envío (ni admin) no puede leer, escribir, ver los integrantes, reportar ni conectarse (403; el WebSocket cierra con `4003`). Los envíos sin transportista asignado no tienen chat.
8. **Cierre por cancelación.** Si el envío se cancela (o el receptor lo rechaza), el chat se cierra en el acto: enviar y leer responden `409 CHAT_CLOSED` y se cortan los sockets abiertos (`4009`).
9. **Receptor original.** Quien transfirió la recepción (ADR-038) conserva el acceso en solo lectura; no puede escribir (`403 CHAT_READ_ONLY`).
10. **Bloqueo (ADR-026).** Un bloqueo, en cualquier dirección, **no impide escribir**. A quien tiene un bloqueo con el autor el mensaje le llega con `hidden: true` y contenido vacío (historial, tiempo real y último mensaje de la lista), para que la app muestre “Mensaje de un usuario bloqueado”.

Ese mensaje no genera push, no suma a los no leídos y no cuenta para los tildes. Si `svc-users` no responde no se oculta nada (falla abierto).

11. **Tildes de entregado y leído.** Cada mensaje propio trae `deliveryStatus`: `sent` (enviado), `delivered` (todos los demás lo recibieron) o `read` ("Leído por todos"). Recibir es tener el chat abierto por WebSocket cuando llega el mensaje, abrir el historial o conectarse; leer es `POST /shipments/:id/chat/read`. El WebSocket empuja `chat_receipt` (`userId`, `delivered|read`, `upTo`) para actualizar los tildes en vivo. "Los demás" excluye a quienes tienen un bloqueo con el autor.
12. **Lista de chats.** `GET /shipments/chats` devuelve un chat por envío con transportista (en curso, entregado o en disputa; no los cancelados), el más reciente primero, con la ruta ("Av. Colón 1234 → Bv. San Juan 500"), el último mensaje con su autor, el contador de no leídos, `access` y `archived` (para atenuar los finalizados).
13. **Reportar conversación.** `POST /shipments/:id/chat/report` registra un reporte por persona y envío (201 la primera vez, 200 después). Solo las partes. El equipo lo revisa; el historial queda como evidencia.
14. **Notificaciones.** Cada mensaje nuevo genera una push (`chatMessageReceived`, categoría `chat`, respeta las preferencias y el horario de silencio) a las otras partes que **no** tengan el chat abierto ni un bloqueo con el autor, como máximo una por minuto y por destinatario. La push lleva el nombre de quien escribe y nunca el texto. Un fallo de la push no afecta el envío del mensaje.
15. **Conexión.** El WebSocket se autentica con el JWT en el handshake, manda heartbeat, se cierra con `4001` al vencer el token (el cliente reconecta con uno renovado) y con `4009` cuando el chat deja de estar disponible (el cliente no reconecta).
16. **Lenguaje ofensivo.** Las palabras ofensivas se tapan con asteriscos del mismo largo antes de guardar y de difundir ("sos un pelotudo" → "sos un ********"): nadie más llega a ver la palabra. El mensaje se envía igual, con `moderated: true`, y el cliente le muestra al autor el aviso `CHAT_MODERATION_WARNING` (hay que respetar a los demás miembros de la comunidad). Compara por palabra entera, ignora mayúsculas, tildes y letras repetidas, y entiende números y símbolos por letra ("put4"). La lista es acotada (insultos y agravios fuertes, no "boludo" ni "mierda", ni palabras legítimas como "retrasado") y la ajusta el equipo en `chat-moderation.ts`.
17. **Contrato para el cliente.** Los tipos (`ChatMessage`, `ChatMessagesPage`, `ChatParticipant`, `ChatListItem`, `ChatRealtimeEvent`, `ChatAccess`, `ChatDeliveryStatus`), las constantes (incluido `CHAT_MODERATION_WARNING`) y los códigos de error (`CHAT_CLOSED`, `CHAT_READ_ONLY`) quedan en `@movo/shared`, y la guía de uso en `docs/chat/README.md`, para que las pantallas de mobile (MOVO-284) los consuman.

Diagrama de secuencia

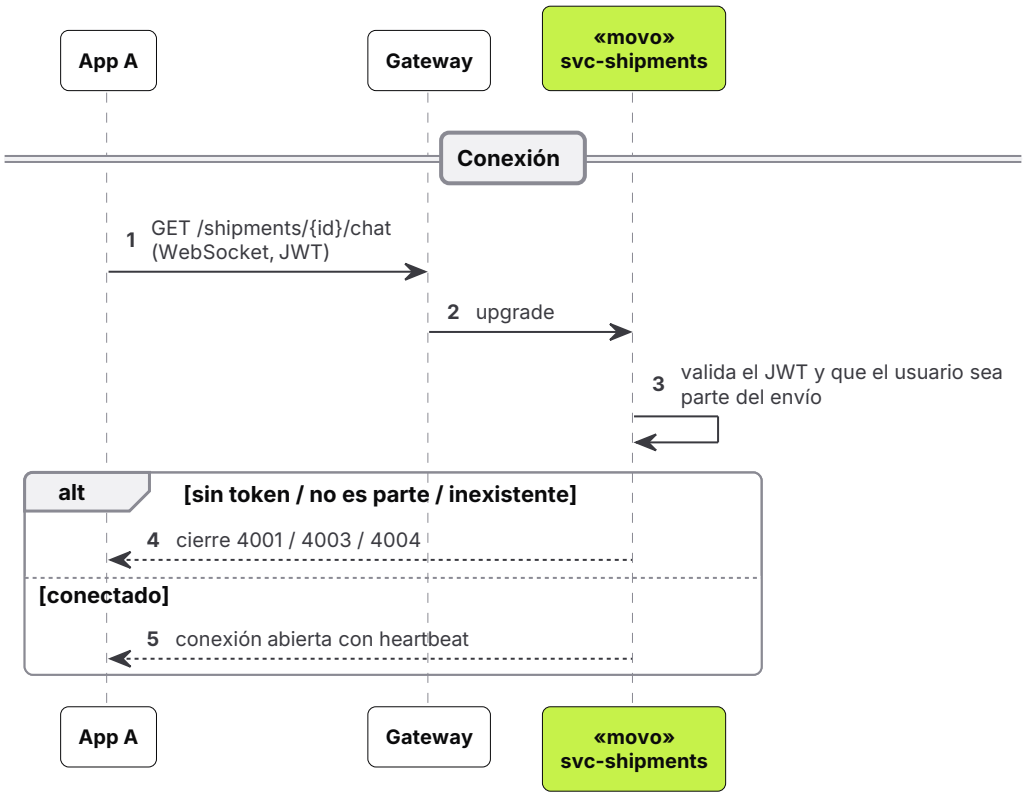


Figura 5.10. Conexión, envío de mensajes y lectura del chat del envío (MOVO-26). Parte 1 de 3: Conexión.

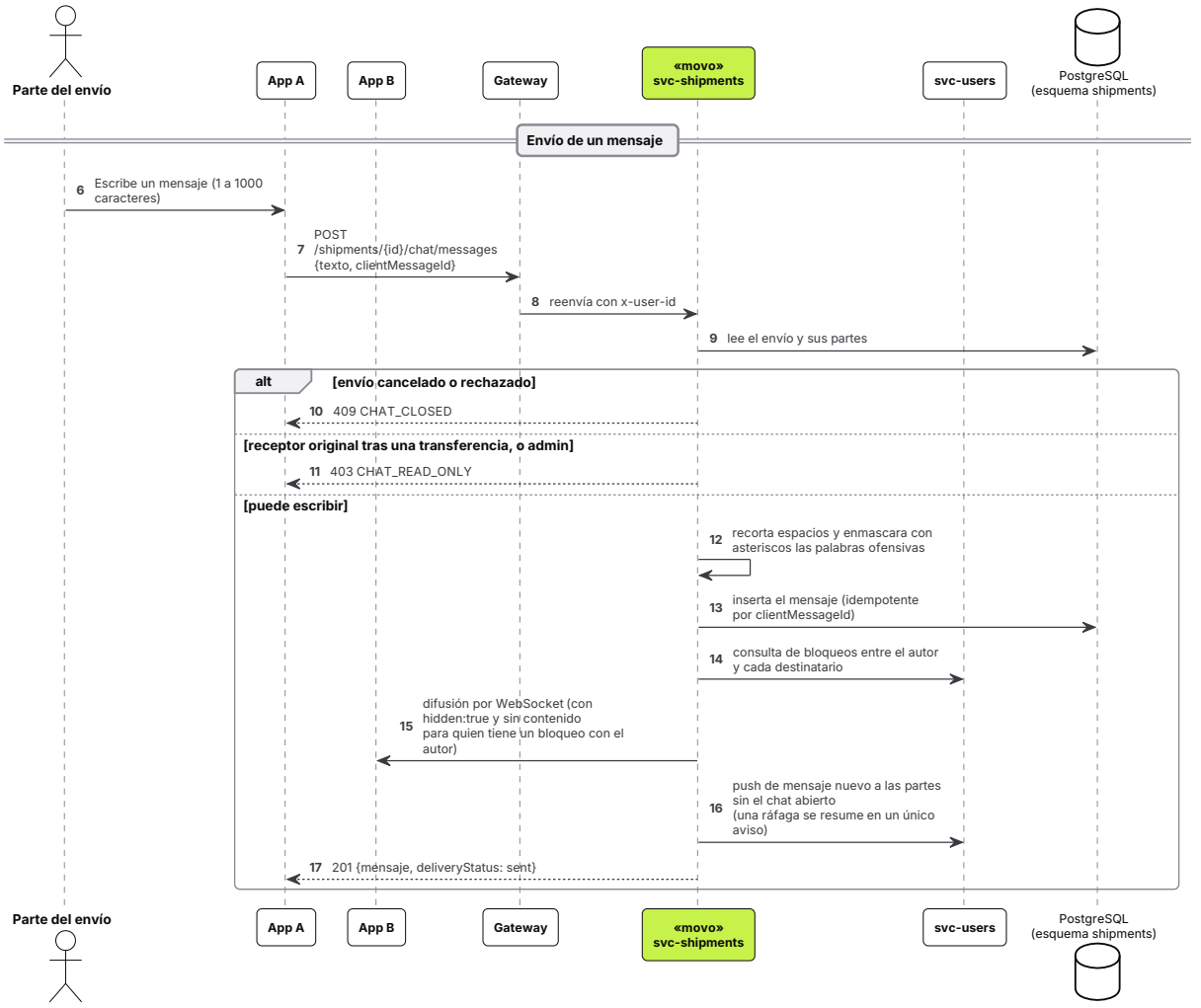


Figura 5.11. Conexión, envío de mensajes y lectura del chat del envío (MOVO-26). Parte 2 de 3: Envío de un mensaje.

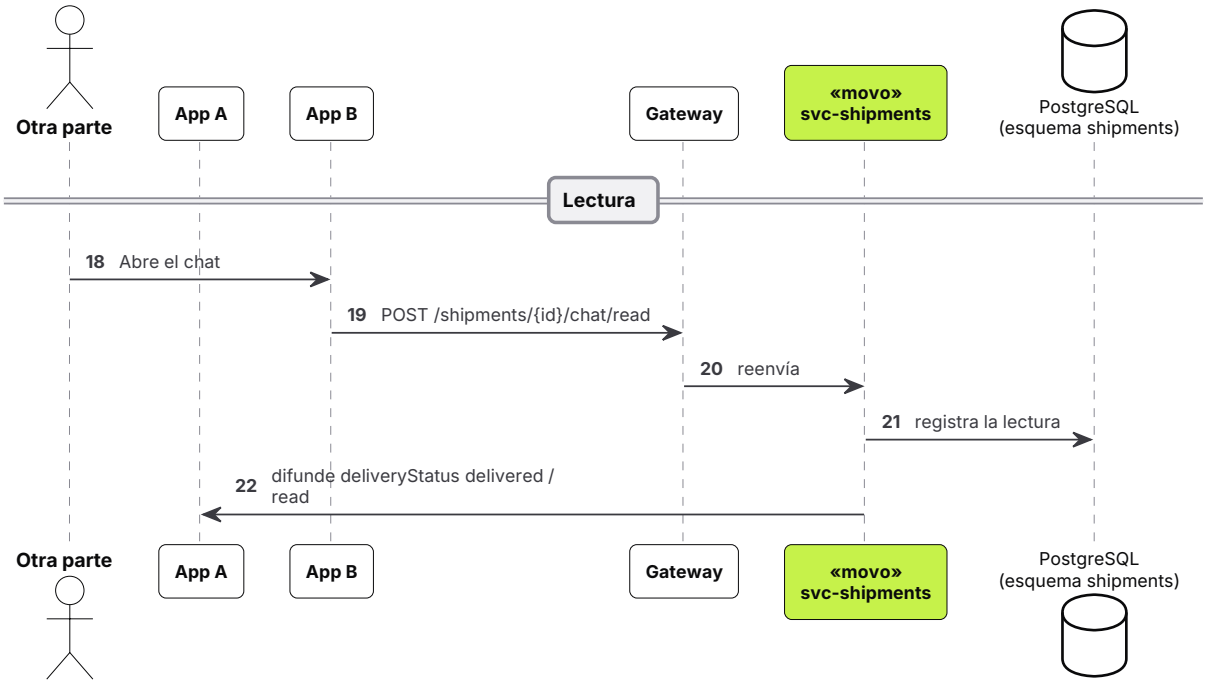


Figura 5.12. Conexión, envío de mensajes y lectura del chat del envío (MOVO-26). Parte 3 de 3: Lectura.

Casos de prueba

Tabla 5.15. Casos de prueba de MOVO-26 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-26.01	3, 14	**Mensajes en tiempo real entre las partes** *Pre:* Emisor, transportista y receptor con el chat abierto. *Pasos:* El emisor envía un mensaje; luego lo hace el receptor, también con el envío recién asignado.	El mensaje llega por WebSocket al resto y al propio emisor; el chat no se mezcla con el canal de tracking.	`chat.integration.test.ts`	APROBADO
CP-26.02	2	**Validación del texto** *Pre:* Chat abierto. *Pasos:* Enviar mensajes vacíos, solo espacios o de más de 1000 caracteres.	Se rechazan; el reenvío con el mismo clientMessageId no duplica (una fila, una difusión).	`chat.integration.test.ts`	APROBADO
CP-26.03	16	**Lenguaje ofensivo** *Pre:* Mensaje con una palabra ofensiva. *Pasos:* Enviar el mensaje y reenviarlo.	La palabra se enmascara con asteriscos del mismo largo antes de guardar y difundir; un mensaje limpio no se marca; el reenvío devuelve el mismo texto ya tapado.	`chat.integration.test.ts` `chat-moderation.test.ts`	APROBADO
CP-26.04	7, 15	**Acceso y conexión** *Pre:* Tercero ajeno al envío, sin token o con envío inexistente. *Pasos:* Leer, escribir y conectar al WebSocket.	403 al leer y escribir; 4003 al conectar; 4001 sin token; 4004 si no existe.	`chat.integration.test.ts`	APROBADO
CP-26.05	10	**Bloqueo entre las partes** *Pre:* Un usuario con bloqueo hacia el autor. *Pasos:* El autor escribe.	Escribir no se impide; a quien tiene el bloqueo le llega el contenido vacío (hidden) y los demás lo ven normal.	`chat.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-26.06	4	**Historial paginado** *Pre:* Chat con varios mensajes. *Pasos:* GET /shipments/:id/chat/messages con cursor, y con un cursor inválido.	Pagina del más reciente al más antiguo sin repetir ni saltear; cursor inválido: 400.	`chat.integration.test.ts`	APROBADO
CP-26.07	8, 9, 6	**Cierre y solo lectura** *Pre:* Envío cancelado, entregado hace más de 24 h, con receptor transferido, y un admin. *Pasos:* Leer y escribir en cada caso.	Cancelado: 409 CHAT_CLOSED y el socket se corta con 4009. Entregado: solo lectura y archivado pasadas 24 h. Receptor original: lee y recibe 403 CHAT_READ_ONLY al escribir. Admin: lee siempre y nunca escribe.	`chat.integration.test.ts` `chat-window.test.ts`	APROBADO
CP-26.08	14	**Notificación de mensajes nuevos** *Pre:* Partes sin el chat abierto. *Pasos:* Enviar un mensaje y luego una ráfaga.	El primer mensaje avisa enseguida con nombre, rol y texto; la ráfaga se resume en un único aviso con la cantidad y el último mensaje, en orden y sin dejar mensajes sin avisar.	`chat.integration.test.ts`	APROBADO

5.2.9 MOVO-275 · Transferencia de la recepción a otra persona

MOVO-275 COMPLETADA SPRINT 7 2 SP

Transferencia de receptor: el receptor designado puede pasarle la recepción a otra persona

> **Decisión de diseño (DD-038).** El cambio de receptor es atómico y se resuelve con compare-and-swap, de modo que si la entrega empieza justo antes del cambio, el receptor no se modifica. Un envío admite una sola transferencia completada y una solicitud activa a la vez. El receptor original conserva acceso de solo lectura, con el seguimiento en vivo y un resumen de la transferencia.

Criterios de aceptación

1. **AC1**: El receptor de un envío aceptado, incluso en tránsito, puede pedir la transferencia a otro usuario con KYC aprobado. Cada validación de arriba devuelve su error.
2. **AC2**: El nuevo receptor recibe la invitación y puede aceptarla o rechazarla dentro del plazo. Si la rechaza o vence el plazo, el envío sigue con el receptor anterior y este recibe el aviso.
3. **AC3**: Cuando el nuevo receptor acepta, el envío cambia de receptor de forma atómica, y el transportista, el emisor y el receptor original reciben la push. El handshake de entrega lo firma el nuevo receptor.
4. **AC4**: El emisor recibe una notificación informativa cuando se pide la transferencia y otra con el resultado. No tiene acción que tomar.
5. **AC5**: Un envío admite una sola transferencia completada y, como máximo, una solicitud activa a la vez.

6. **AC6**: Una solicitud pendiente se cancela si arranca el handshake de entrega o si la cancela el receptor original.
7. **AC7**: La línea de tiempo muestra cada solicitud como **un único item** que refleja su estado (pendiente, completada, rechazada, vencida, cancelada por el receptor o por el inicio de la entrega), con actor, motivo y horarios. El receptor original y el emisor ven todas las solicitudes; el transportista y el nuevo receptor solo la completada.
8. **AC8**: En mobile, el receptor ve la acción "Que lo reciba otra persona" en el detalle del envío. La invitación aparece en "Requiere tu atención" del nuevo receptor con el plazo restante (patrón de MOVO-253 AC5). Para el emisor es solo informativa: push y evento en la línea de tiempo, sin card de acción.
9. **AC9**: Tests de integración con Postgres real que cubran cada validación, la carrera con el handshake, el vencimiento, el límite de una transferencia, el acceso de solo lectura del receptor original y el filtrado de la línea de tiempo por quien mira.
10. **AC10**: Después de una transferencia completada, el receptor original ve el detalle completo en solo lectura (con tracking en vivo y un banner "Le pasaste la recepción a X · fecha" con el motivo), sin ninguna acción, y el envío sigue en su actividad reciente con el badge "Transferido". No puede firmar la entrega ni calificar.

Diagrama de secuencia

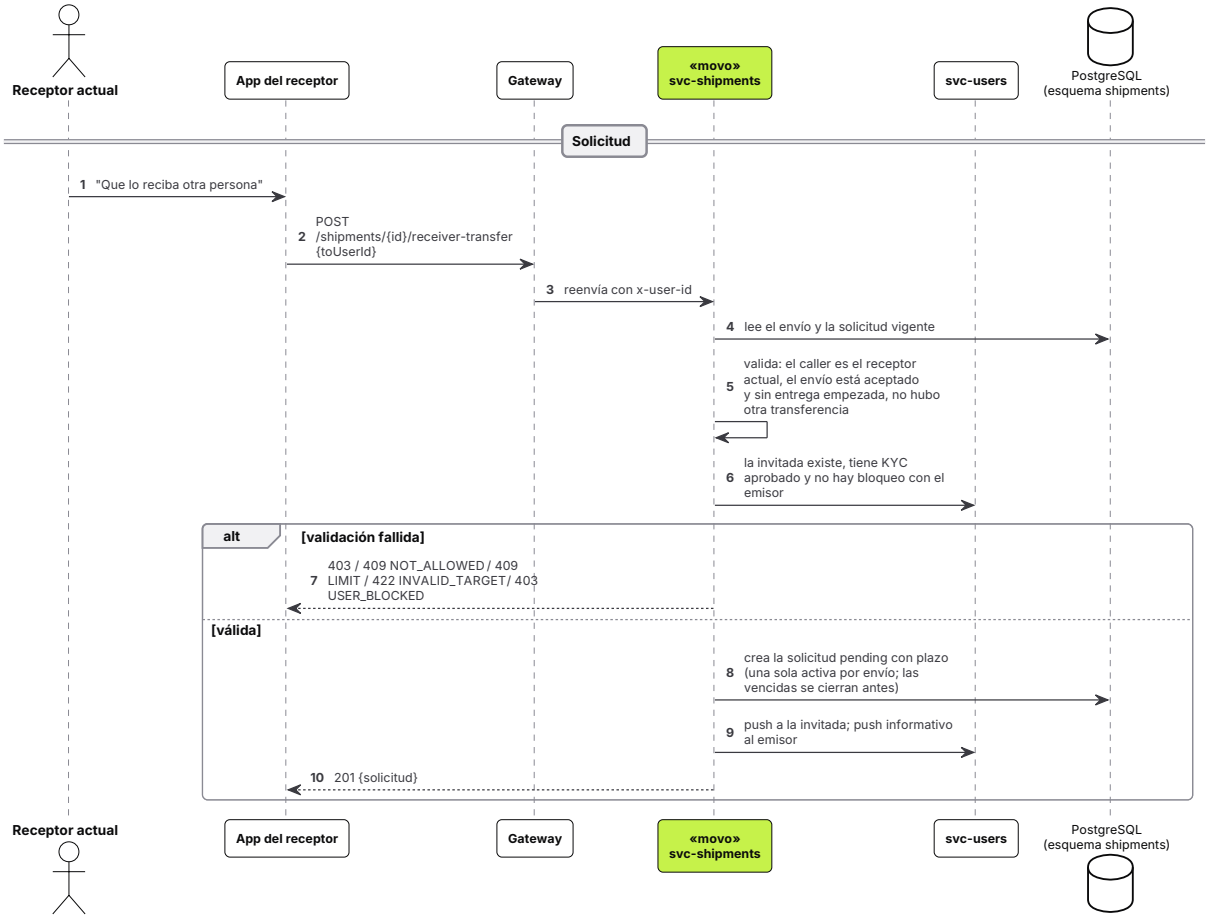


Figura 5.13. Solicitud, aceptación y rechazo de la transferencia de receptor (MOVO-275, DD-038). Parte 1 de 2: Solicitud.

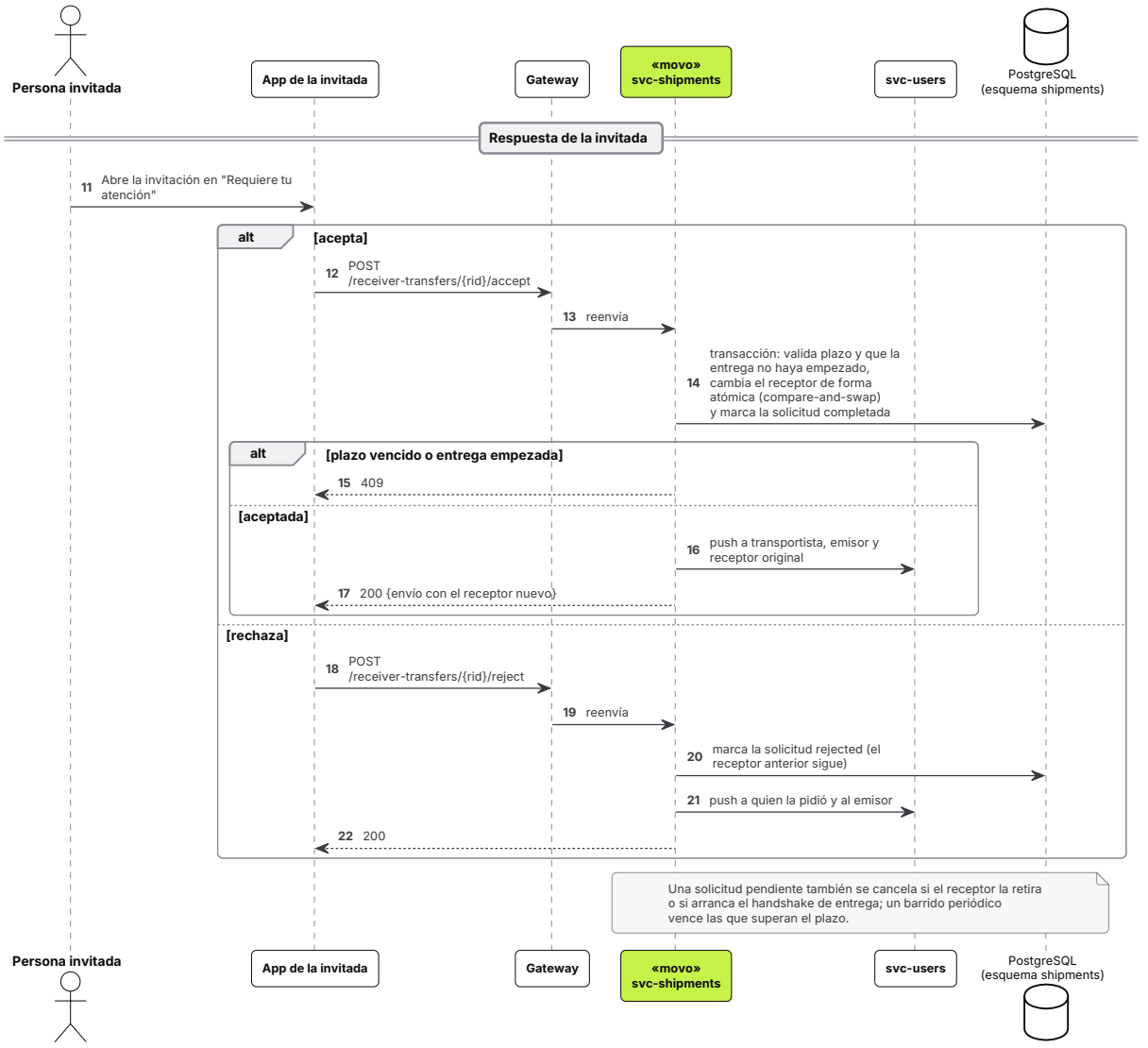


Figura 5.14. Solicitud, aceptación y rechazo de la transferencia de receptor (MOVO-275, DD-038). Parte 2 de 2: Respuesta de la invitada.

Casos de prueba

Tabla 5.16. Casos de prueba de MOVO-275 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-275.01	1, 4	**Pedir la transferencia** *Pre:* Receptor actual con envío publicado, asignado o en camino. *Pasos:* POST /shipments/:id/receiver-transfer con la persona elegida.	201; la invitada y el emisor reciben su push (el emisor, solo informativa).	`receiver-transfers.integration.test.ts`	APROBADO
CP-275.02	1	**Validaciones de la solicitud** *Pre:* Distintos usuarios y estados del envío. *Pasos:* Pedir como no receptor, antes de aceptar el envío, con el envío entregado, designando al receptor actual, al emisor o al transportista, a un usuario sin KYC, inexistente o bloqueado.	403 si no es el receptor actual; 409 NOT_ALLOWED según el estado; 422 INVALID_TARGET; 403 USER_BLOCKED por bloqueo; la persona elegida debe existir y tener KYC aprobado.	`receiver-transfers.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-275.035	5	**Una solicitud activa y una transferencia por envío** *Pre:* Solicitud pendiente, o transferencia ya completada. *Pasos:* Pedir otra, también de forma concurrente.	Como máximo una pendiente, aun con pedidos concurrentes; tras completarse una no se puede pedir otra (409 LIMIT); una pendiente vencida se cierra al pedir otra.	`receiver-transfers.integration.test.ts`	APROBADO
CP-275.043	3	**Aceptación de la invitada** *Pre:* Solicitud pendiente dentro del plazo. *Pasos:* POST /receiver-transfers/:id/accept como la invitada y como otro usuario; aceptar dos veces; aceptar fuera de plazo.	El receptor cambia de forma atómica y avisa a transportista, emisor y receptor original; otro usuario recibe 403; el segundo intento 409 NOT_PENDING; fuera de plazo 409 aunque el barrido no haya corrido.	`receiver-transfers.integration.test.ts`	APROBADO
CP-275.056	6	**Carrera con el handshake de entrega** *Pre:* Solicitud pendiente y entrega por empezar. *Pasos:* Iniciar el handshake de entrega antes o durante la aceptación.	Con la entrega empezada no se puede pedir ni aceptar (409, aunque el QR haya vencido); si empieza justo antes del UPDATE, el receptor no cambia; el handshake cancela la solicitud y valida contra el receptor vigente.	`receiver-transfers.integration.test.ts`	APROBADO
CP-275.062	2	**Rechazo, cancelación y vencimiento** *Pre:* Solicitud pendiente. *Pasos:* La invitada la rechaza; el receptor la cancela; vence el plazo.	El envío sigue con el receptor anterior, que puede volver a intentar; la invitada ya no puede aceptar una cancelada; el barrido vence las pendientes y avisa a quien la pidió y al emisor.	`receiver-transfers.integration.test.ts`	APROBADO
CP-275.0710	10	**Vista del receptor original** *Pre:* Transferencia completada. *Pasos:* El receptor original consulta el envío.	Ve el detalle completo en solo lectura con el resumen de la transferencia.	`receiver-transfers.integration.test.ts`	APROBADO

06

TEMA 5 DE 7 · TRAZABILIDAD DE HISTORIAS DE USUARIO

EP-05 · Pagos y Economía

Reserva de fondos, captura y reparto del pago, tarjetas guardadas, vinculación de Mercado Pago, penalidades de cancelación y cuenta corriente.

CONTENIDO

6.1	Descripción del Tema	125
6.2	Historias de Usuario y Trazabilidad	126

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: trazabilidad de las historias del tema, con criterios de aceptación de Linear, diagramas de secuencia derivados del código y casos de prueba derivados de las pruebas automatizadas.

Introducción

Este documento presenta la trazabilidad de las historias de usuario del tema *Pagos y Economía*: para cada historia, su descripción y criterios de aceptación, los tickets que la implementaron, el diagrama de secuencia que muestra cómo la resuelve el sistema y los casos de prueba que la verifican.

Audiencia

El tribunal evaluador y los futuros mantenedores. Se asume conocimiento de las convenciones del proyecto y de UML.

6.1 Descripción del Tema

EP-05 · Pagos y Economía reúne todo el dinero que circula por Movo, resuelto sobre Mercado Pago como procesador y sin que la plataforma custodie fondos propios. El emisor **reserva** el precio acordado en su tarjeta antes del retiro (una autorización, no un cobro); al confirmarse la entrega, el pago se **captura** y se reparte automáticamente entre el transportista y Movo (*split payment*). Si el envío se cancela, la reserva se libera o se captura parcialmente como penalidad.

Es la tercera capa de confianza (los pagos): ninguna de las partes arriesga dinero antes de que la contraparte cumpla. El tema incluye la vinculación de la cuenta de cobro del transportista (OAuth de Mercado Pago), las tarjetas guardadas del emisor (el backend nunca ve número ni CVV), el historial de pagos y cobros, las penalidades de cancelación (DD-047) y la **cuenta corriente** del usuario, donde se acumulan y cobran las deudas.

Servicios involucrados: `movo-svc-payments` (holds, capturas, tarjetas, vinculación y cuenta corriente), `movo-svc-shipments` (saga de asignación, liquidación y cancelaciones), `gateway` y `movo-mobile`; **externo:** Mercado Pago (Checkout API, Marketplace OAuth, Split Payments y webhooks).

6.2 Historias de Usuario y Trazabilidad

Tabla 6.1. Historias de usuario del tema.

Historia	Descripción	Estado	Sprint
MOVO-12	Reserva automática de fondos antes del retiro	Completada	Sprint 7, Sprint 8
MOVO-13	Pago automático al transportista (split payment)	Completada	Sprint 7, Sprint 8
MOVO-28	Historial de pagos y cobros	Completada	Sprint 7
MOVO-37	Pago de la comisión en efectivo	Por hacer	Sprint 8
MOVO-65	Tarjetas guardadas	Completada	Sprint 7
MOVO-110	Vinculación de la cuenta de Mercado Pago del transportista	Completada	Sprint 7
MOVO-226	Penalidad de cancelación y cuenta corriente	En curso	Sprint 7, Sprint 8

6.2.1 MOVO-12 · Reserva automática de fondos antes del retiro

MOVO-12 COMPLETADA SPRINT 7, SPRINT 8 19 SP

Como emisor, quiero que mis fondos se reserven automáticamente antes del retiro del paquete, para que el pago solo se ejecute cuando el paquete sea entregado.

Decisión de diseño (DD-037). Mercado Pago autoriza el pago sin capturarlo (*hold*): los fondos quedan reservados en la tarjeta del emisor pero no se cobran hasta que se confirma la entrega. La asignación sigue una saga de dos rutas según la cercanía del retiro: **cercana** (retiro dentro de N días, con N = 3 por defecto), el emisor paga en el acto y el envío espera en `assignment_pending` con un plazo de 30 minutos; **lejana**, el envío queda en `assigned_unfunded` y el pago se confirma cuando se abre la ventana de N días, con vencimiento 24 horas antes del retiro. En ambos casos, si el pago no se completa, el envío vuelve a `published` y ambas partes son notificadas. Un índice único parcial en la base impide dos holds vivos del mismo envío.

Criterios de aceptación

1. Al aceptar una oferta con retiro **dentro de N días**, el envío pasa a `assignment_pending` y el emisor ingresa su tarjeta para crear el hold por el monto acordado. Hold confirmado: `assignment_pending → assigned`.
2. Si el hold falla o el emisor no completa el pago antes del timeout, el envío vuelve a `published`, la oferta no queda aceptada y ambas partes son notificadas con el motivo.
3. Al aceptar una oferta con retiro a **más de N días**, el envío pasa a `assigned_unfunded` sin pedir tarjeta.
4. Cuando el retiro entra en N días, el emisor recibe una notificación para confirmar el pago y puede hacerlo desde el detalle del envío. Éxito: `assigned_unfunded → assigned`.

5. Si el emisor no confirma antes de T-24h de la ventana de retiro, el envío vuelve a `published` y **ambas partes** son notificadas.
6. El emisor ve el estado de la reserva en el detalle del envío (`MOVO-211`).
7. Los fondos reservados no se cobran hasta que se complete el handshake de entrega (`MOVO-13`).
8. **Un envío en `assigned_unfunded` no puede iniciar el retiro.** El wizard de retiro exige `assigned`.
9. Si el emisor cancela con un hold activo, el hold se libera.

Trazabilidad de implementación

Tabla 6.2. Tickets de implementación de MOVO-12.

Ticket	Alcance	Sprint	Estado
`MOVO-209`	[svc-payments] Hold (Auth & Capture con capture:false): crear, consultar y liberar la reserva de un envío	Sprint 7	Completada
`MOVO-210`	[svc-shipments] Saga de asignación: pago del emisor al aceptar o ventana de confirmación, vencimientos y reversión a published	Sprint 7	Completada
`MOVO-211`	[movo-mobile] Estado de la reserva de fondos en el detalle del envío	Sprint 7	Completada
`MOVO-215`	[Spike] Plazo real de la reserva (hold) de MP y valor de N	Sprint 8	Completada
`MOVO-268`	[svc-payments] Webhook de notificaciones de pagos de MP: holds cancelados o vencidos por MP y contracargos	Sprint 7	Completada
`MOVO-269`	[movo-mobile] Pago del envío con tarjeta guardada + CVV: al aceptar la oferta y al confirmar el pago de un retiro lejano	Sprint 7	Completada

Diagrama de secuencia

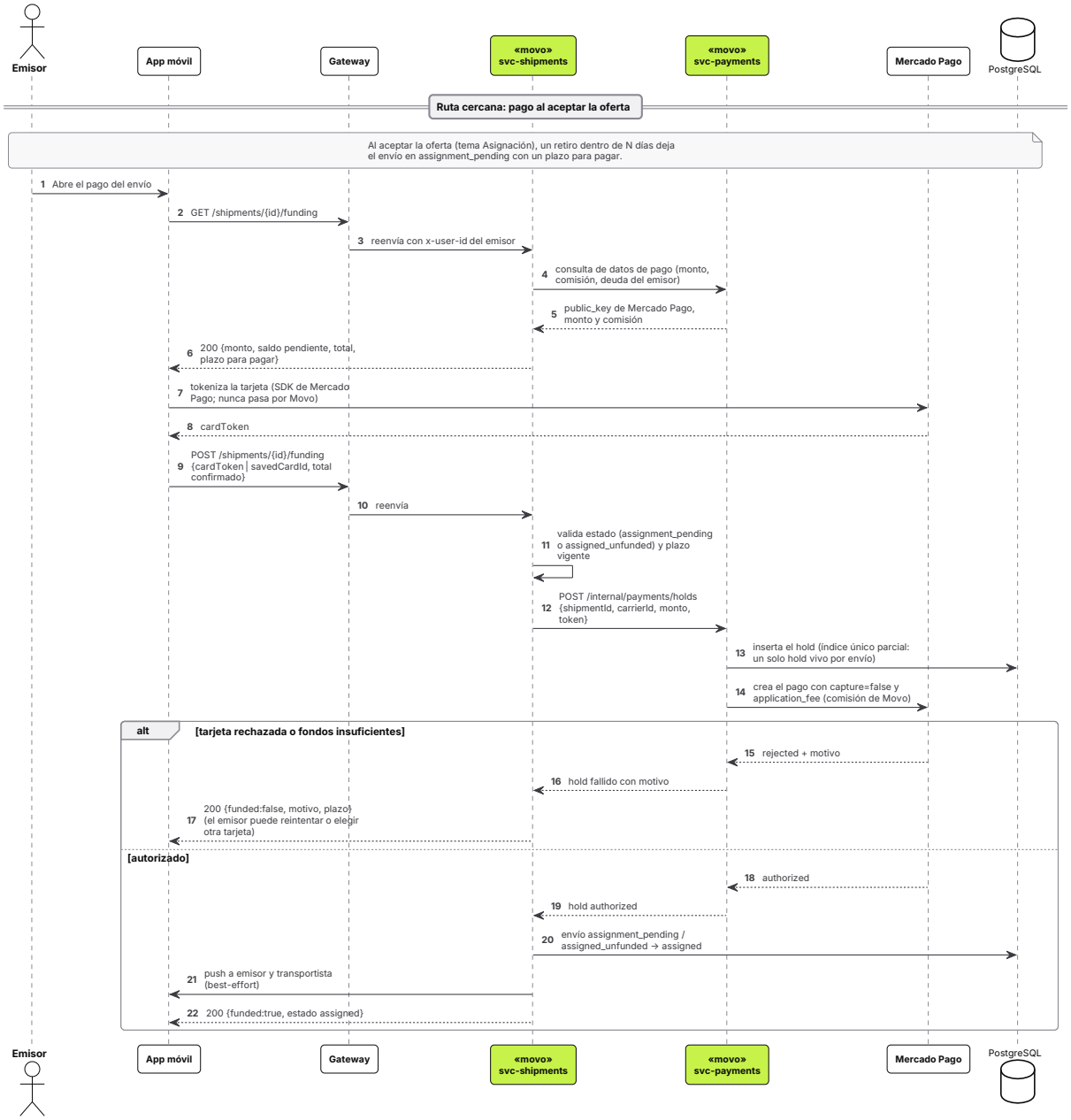


Figura 6.1. Saga de asignación y reserva de fondos (hold) con ruta cercana y lejana (MOVO-12, MOVO-210, MOVO-211, MOVO-215). Parte 1 de 3: Ruta cercana: pago al aceptar la oferta.

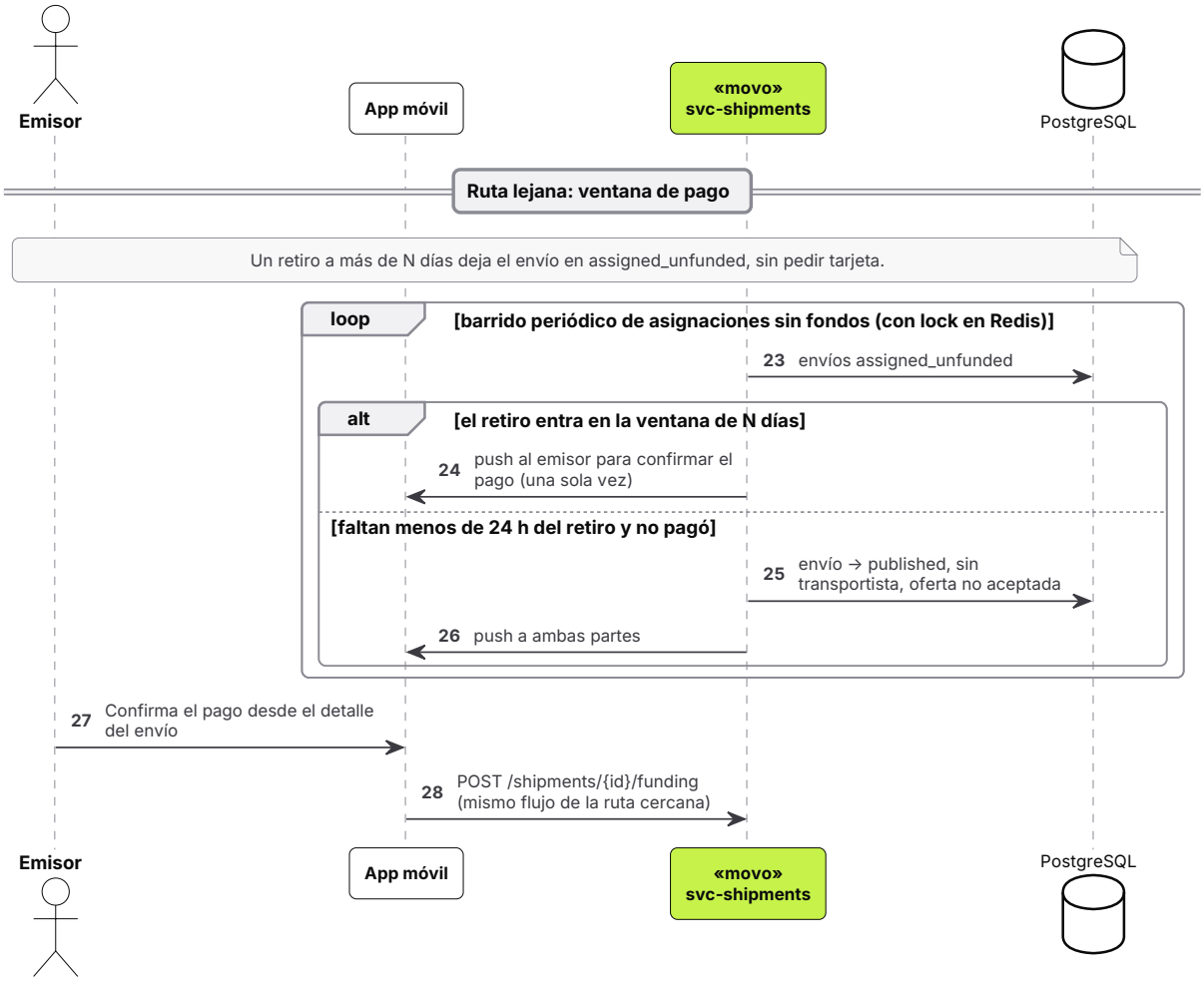


Figura 6.2. Saga de asignación y reserva de fondos (hold) con ruta cercana y lejana (MOVO-12, MOVO-210, MOVO-211, MOVO-215). Parte 2 de 3: Ruta lejana: ventana de pago.

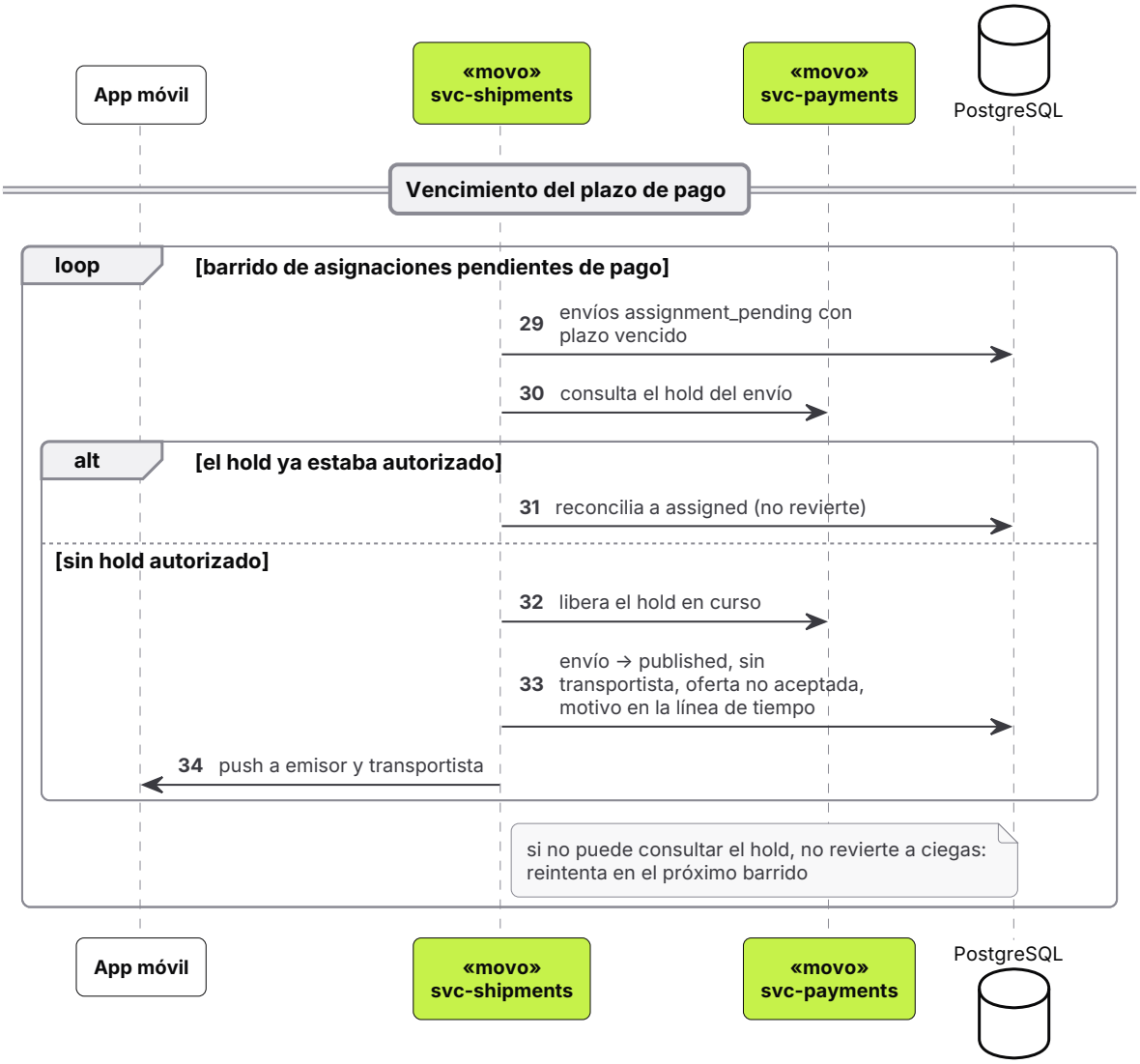


Figura 6.3. Saga de asignación y reserva de fondos (hold) con ruta cercana y lejana (MOVO-12, MOVO-210, MOVO-211, MOVO-215). Parte 3 de 3: Vencimiento del plazo de pago.

Casos de prueba

Tabla 6.3. Casos de prueba de MOVO-12 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-12.01	1	**Ruta cercana: pago al aceptar la oferta** *Pre:* Oferta aceptada con retiro dentro de la ventana cercana; transportista con Mercado Pago vinculado. *Pasos:* GET /shipments/:id/funding y luego POST /shipments/:id/funding con el token de la tarjeta.	El GET devuelve la clave pública, el monto y el plazo, solo al emisor; con hold autorizado el envío pasa de assignment_pending a assigned y se avisa a ambas partes.	`funding.integration.test.ts` `holds.integration.test.ts`	APROBADO
CP-12.02	2	**Pago rechazado** *Pre:* Envío esperando pago. *Pasos:* Pagar con una tarjeta rechazada, y luego reintentar.	El estado no cambia; la respuesta trae el motivo y permite reintentar con otra clave de idempotencia. Si payments falla al crear el hold, 502 reintentable.	`funding.integration.test.ts` `holds.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-12.03	2	**Plazo vencido** *Pre:* Envío en assignment_pending con el plazo de pago cumplido. *Pasos:* Intentar pagar; correr el barrido de vencimiento.	409 al pagar fuera de plazo. El barrido devuelve el envío a published sin transportista, deja la oferta sin aceptar, libera el hold y avisa a ambas partes; es idempotente y, si el hold ya estaba autorizado, reconcilia a assigned en lugar de revertir. Si no puede consultar el hold, no revierte a ciegas.	`funding.integration.test.ts` `shipment-expiration.integration.test.ts`	APROBADO
CP-12.04	3	**Ruta lejana** *Pre:* Oferta aceptada con retiro a más de N días. *Pasos:* Aceptar la oferta.	El envío queda en assigned_unfunded con transportista y precio, las demás ofertas siguen vivas y no se crea ningún hold.	`funding.integration.test.ts` `funding-view.integration.test.ts`	APROBADO
CP-12.05	4, 5, 8	**Ventana de pago y retiro bloqueado** *Pre:* Envío en assigned_unfunded. *Pasos:* Correr el barrido al abrirse la ventana y al faltar menos de 24 h; intentar iniciar el retiro sin fondos.	Se avisa al emisor una sola vez para confirmar el pago; sin pago antes de T-24 h el envío vuelve a published y se avisa a ambos. Un envío sin fondos no puede iniciar el retiro (el handshake exige assigned).	`funding.test.ts` `handshake.integration.test.ts`	APROBADO
CP-12.06	6	**Estado de la reserva visible** *Pre:* Envío con distintos estados de pago. *Pasos:* Consultar el detalle como emisor, transportista, receptor y tercero.	El emisor ve monto, tarjeta y motivo de un intento rechazado; el transportista solo si está pendiente o reservado; el receptor y un tercero no ven datos de pago. Si payments cae, el detalle responde igual sin el último intento.	`funding-view.integration.test.ts` `funding-view.test.ts`	APROBADO
CP-12.07	-	**Un solo hold por envío y pagos duplicados** *Pre:* Dos pedidos de pago simultáneos o un doble toque. *Pasos:* POST /shipments/:id/funding repetido.	Una sola asignación y un solo hold, con la misma clave de idempotencia; el reintento de un hold vigente devuelve el mismo hold sin nuevo cobro.	`funding.integration.test.ts` `holds.integration.test.ts`	APROBADO
CP-12.08	9	**Liberar el hold** *Pre:* Hold autorizado. *Pasos:* Liberarlo con el token del transportista, repetirlo, y probar con otro transportista o sin credenciales vigentes.	La liberación es idempotente; la de otro transportista se rechaza sin tocar el hold; sin credenciales vigentes, 409 y el hold sigue vivo.	`holds.integration.test.ts`	APROBADO
CP-12.09	-	**Eventos del hold desde Mercado Pago** *Pre:* Webhook de Mercado Pago sobre un hold. *Pasos:* Enviar el webhook con firma válida, inválida o ausente, y con distintos estados del pago.	Firma inválida o ausente: 401 sin tocar nada. Hold vencido, cancelado o rechazado: se marca y se avisa a svc-shipments; un hold perdido con el paquete en camino alerta al equipo.	`mp-webhook.integration.test.ts` `mp-webhook-signature.test.ts`	APROBADO

6.2.2 MOVO-13 · Pago automático al transportista (split payment)

MOVO-13 COMPLETADA SPRINT 7, SPRINT 8 15 SP

Como transportista, quiero recibir el pago automático con split payment tras confirmarse la entrega, para cobrar inmediatamente después de entregar sin gestiones adicionales.

Decisión de diseño. Al confirmarse la entrega, `svc-payments` captura el hold: la comisión de Movo queda como *application fee* y el neto se acredita directamente en la cuenta de Mercado Pago del transportista (split payment). Una captura fallida deja la liquidación en `failed` y el hold intacto, se reintenta con espera creciente (5, 30 y 120 minutos) y, si se agotan los reintentos, escala a revisión manual. Antes de reintentar se consulta el pago, para no capturar dos veces una operación que sí ocurrió pero cuya respuesta se perdió.

Criterios de aceptación

1. Al completarse el handshake de entrega, el sistema ejecuta la captura del hold y el split payment automaticamente.
2. El porcentaje de comision de Movo se acredita en la cuenta de Movo.
3. El saldo restante se acredita directamente en la cuenta del transportista.
4. El transportista recibe notificacion push con el monto acreditado.
5. El emisor recibe notificacion push confirmando que el pago fue procesado.
6. Si el split payment falla, el sistema reintenta automaticamente al menos 2 veces antes de escalar a un proceso manual.
7. Ambas partes pueden ver el detalle del pago (monto total, comision, neto) en el historial de la transaccion.

Trazabilidad de implementación

Tabla 6.4. Tickets de implementación de MOVO-13.

Ticket	Alcance	Sprint	Estado
`MOVO-212`	[svc-payments] Captura del hold y split payment al confirmarse la entrega (FundsReleaseNotifier real)	Sprint 7	Completada
`MOVO-213`	[svc-payments] Reintentos del split y escalado a proceso manual ante fallo persistente	Sprint 8	Completada
`MOVO-214`	[movo-mobile] Detalle del pago (bruto, comisión, neto) en el envío entregado	Sprint 7	Completada

Diagrama de secuencia

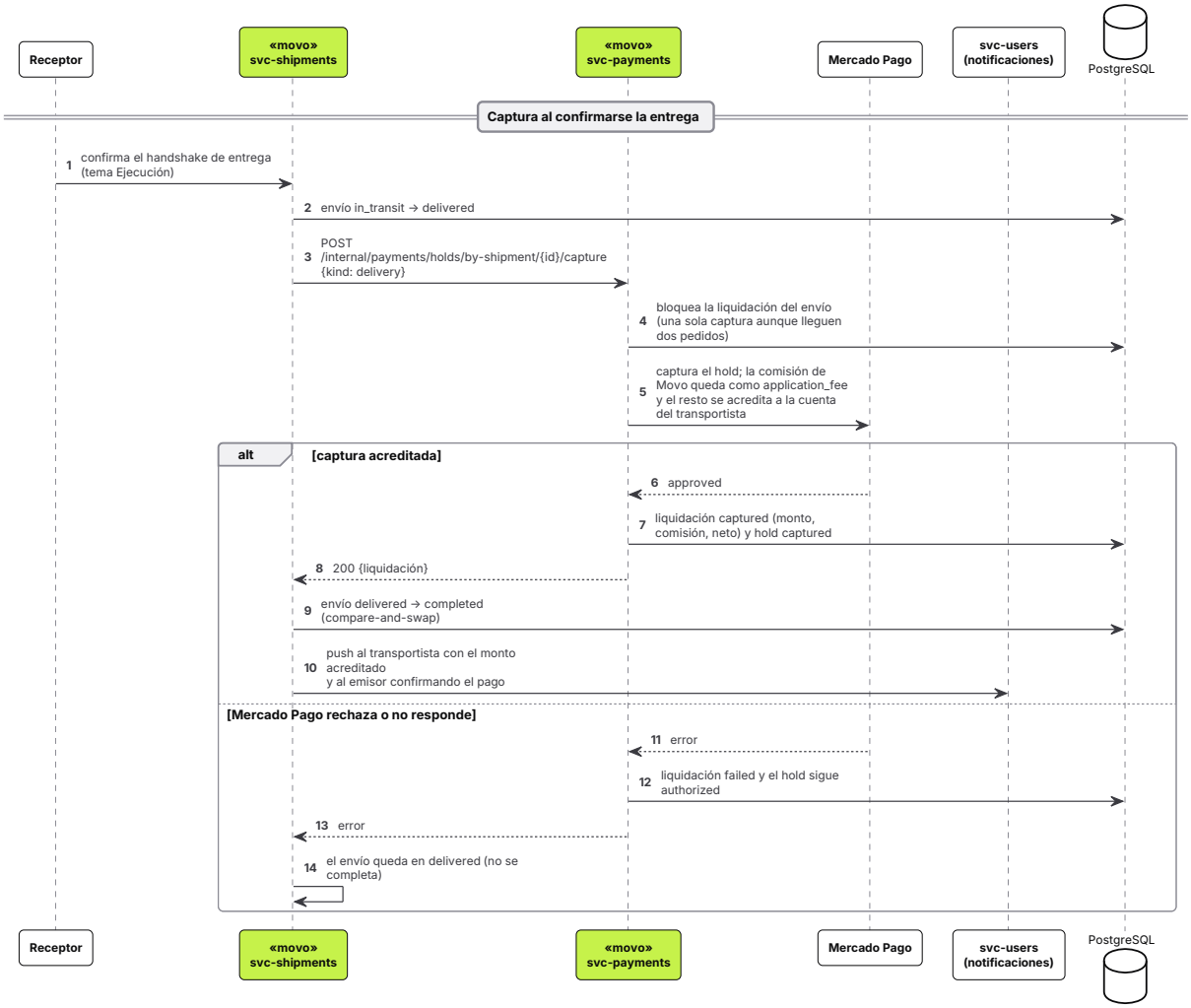


Figura 6.4. Captura del hold, reparto de fondos, reintentos y webhook (MOVO-13, MOVO-212, MOVO-213, MOVO-214). Parte 1 de 3: Captura al confirmarse la entrega.

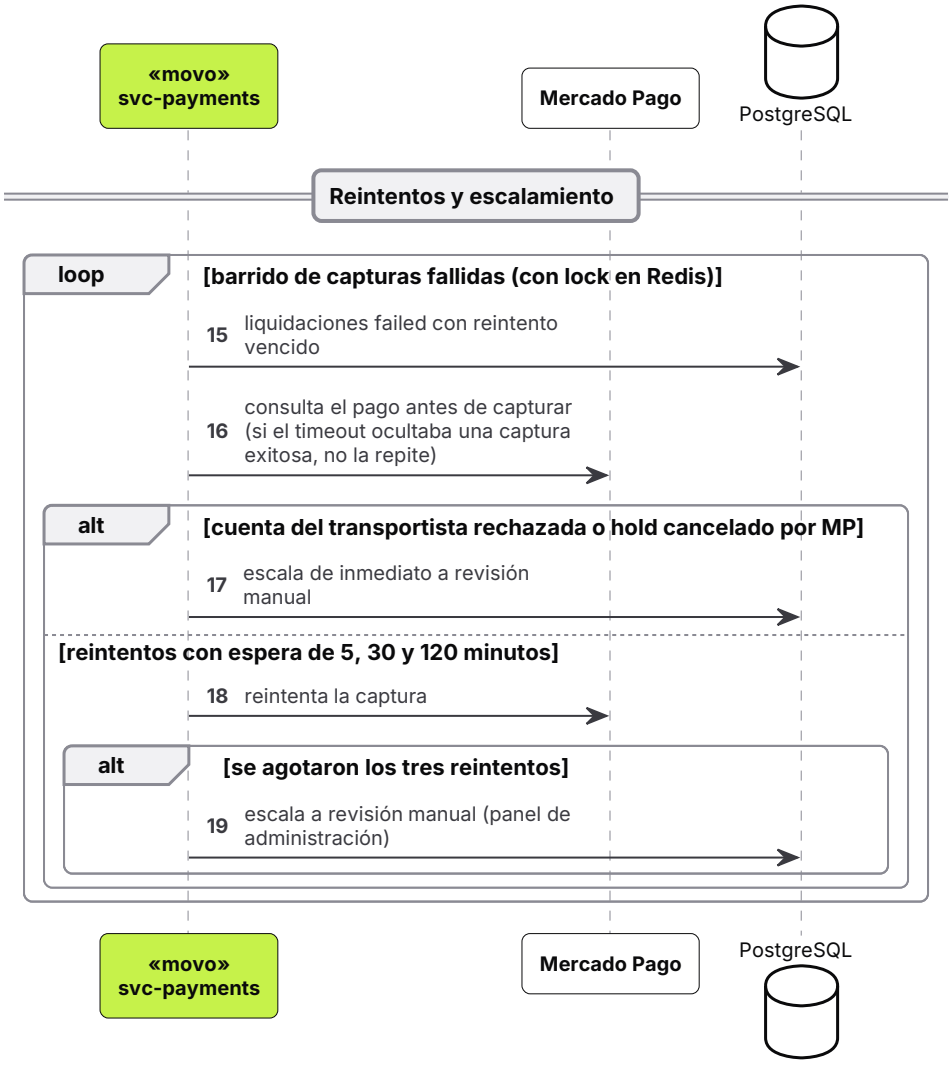


Figura 6.5. Captura del hold, reparto de fondos, reintentos y webhook (MOVO-13, MOVO-212, MOVO-213, MOVO-214). Parte 2 de 3: Reintentos y escalamiento.

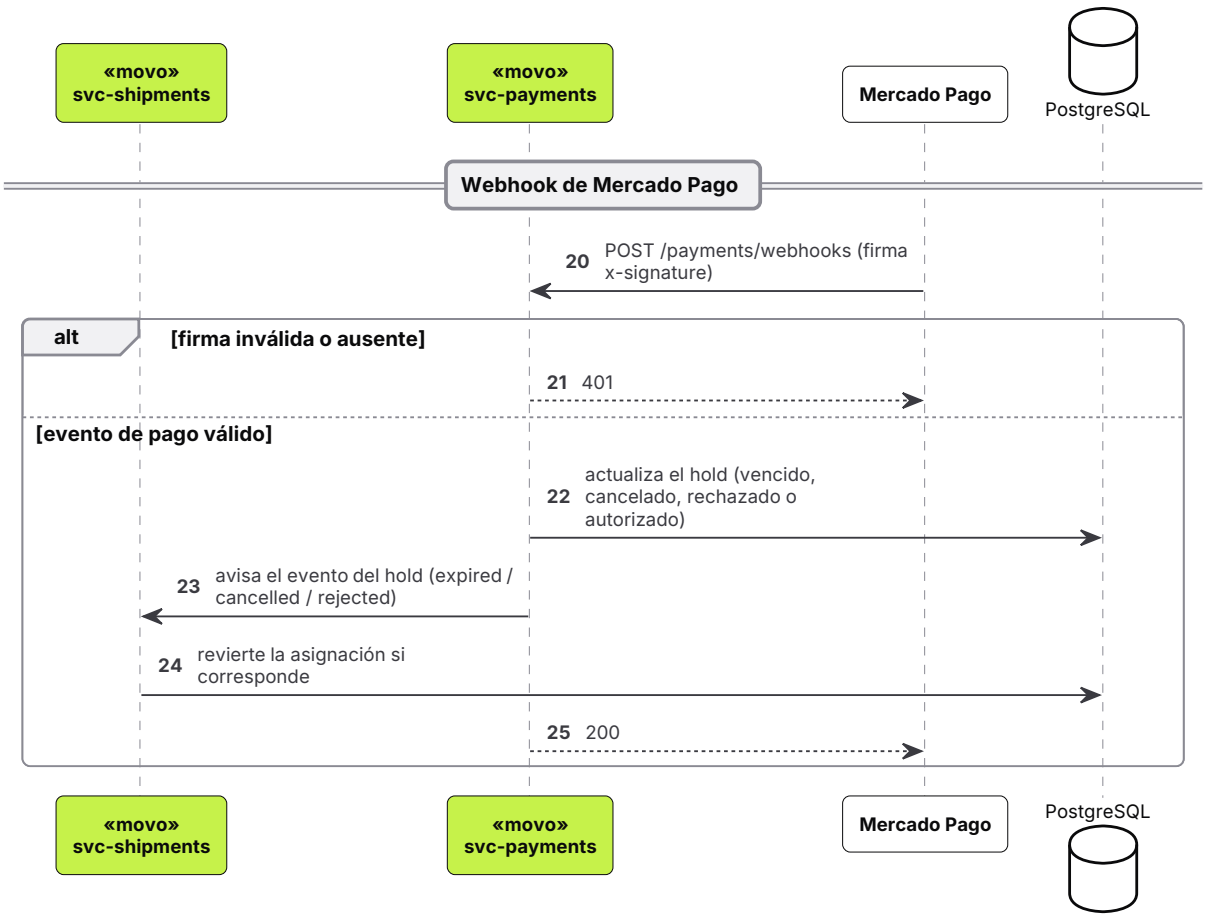


Figura 6.6. Captura del hold, reparto de fondos, reintentos y webhook (MOVO-13, MOVO-212, MOVO-213, MOVO-214). Parte 3 de 3: Webhook de Mercado Pago.

Casos de prueba

Tabla 6.5. Casos de prueba de MOVO-13 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-13.01	1, 2, 3	**Captura y liquidación** *Pre:* Hold autorizado y envío entregado. *Pasos:* POST /internal/payments/holds/by-shipment/:id/capture.	Se captura el hold, la liquidación se persiste con montos numéricos (comisión y neto) y el hold queda captured.	`captures.integration.test.ts` `capture.service.test.ts`	APROBADO
CP-13.02	1	**Captura única ante pedidos simultáneos** *Pre:* Dos pedidos de captura del mismo envío. *Pasos:* Capturar dos veces, también en paralelo.	Se cobra una sola vez; un hold rechazado no se captura y no crea liquidación.	`captures.integration.test.ts`	APROBADO
CP-13.03	4, 5	**Cierre del envío y avisos** *Pre:* Captura acreditada. *Pasos:* Cerrar la liquidación en svc-shipments, invocarla dos veces, y con una push que falla.	El envío pasa de delivered a completed; se avisa al transportista con el monto acreditado y al emisor del pago; una segunda invocación no captura ni avisa de nuevo y una push fallida no rompe el cierre.	`settlement.service.test.ts` `settlement-events.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-13.04	6	**Falla de la captura y reintentos** *Pre:* Mercado Pago rechaza o no responde. *Pasos:* Capturar y correr el barrido de reintentos.	El envío queda en delivered; la liquidación queda pendiente de reintento con motivo, intentos y próxima hora; reintenta con espera de 5, 30 y 120 minutos y, agotados, escala. No captura a ciegas si no puede consultar el pago.	`capture-retry.integration.test.ts` `capture-retry-sweep.test.ts` `capture-failure.test.ts`	APROBADO
CP-13.05	6	**Escalamiento inmediato** *Pre:* Cuenta del transportista rechazada u hold cancelado por Mercado Pago. *Pasos:* Reintentar la captura.	Escala de inmediato a revisión manual sin gastar intentos.	`capture-retry.integration.test.ts`	APROBADO
CP-13.06	7	**Detalle y consulta de la liquidación** *Pre:* Envío con y sin captura. *Pasos:* GET de la liquidación del envío y del historial de transacciones.	404 antes de capturar y 200 después; cada parte ve monto, comisión y neto.	`captures.integration.test.ts` `transactions.integration.test.ts`	APROBADO
CP-13.07	-	**Captura parcial y límites** *Pre:* Hold autorizado. *Pasos:* Capturar un monto menor, mayor al hold o inferior a la comisión.	Un monto menor captura parte del hold (cancellation_penalty); mayor: 422 CAPTURE_AMOUNT_EXCEEDS_HOLD; menor que la comisión: 422 CAPTURE_BELOW_APPLICATION_FEE; un envío ya liquidado como entrega responde 409 SETTLEMENT_CONFLICT.	`captures.integration.test.ts`	APROBADO

6.2.3 MOVO-28 · Historial de pagos y cobros

MOVO-28 COMPLETADA SPRINT 7 9 SP

Como usuario, quiero ver el historial de mis pagos y cobros, para llevar registro de lo que gasté y cobré en envíos.

Criterios de aceptación

1. El tab **Pagos** muestra las transacciones del usuario como emisor y el tab **Cobros** las del usuario como transportista. Solo ve las propias.
2. Orden del más reciente al más viejo, agrupado por mes y paginado.
3. Cada ítem muestra fecha de pago (hora de Argentina), monto, estado y referencia corta al envío (`#MOVO-xxxxx`). En Pagos suma marca y últimos 4 de la tarjeta. Tocar un ítem abre el detalle.
4. Los intentos de pago rechazados o sin respuesta no aparecen.

Trazabilidad de implementación

Tabla 6.6. Tickets de implementación de MOVO-28.

Ticket	Alcance	Sprint	Estado
`MOVO-302`	[svc-payments] Historial: guardar el pagador y la tarjeta en el hold (payer_id, card_last4, card_brand)	Sprint 7	Completada
`MOVO-303`	[svc-payments] Historial: endpoints de listado y detalle por rol (GET /payments/transactions)	Sprint 7	Completada
`MOVO-304`	[movo-mobile] Historial de pagos y cobros: tabs, filtros, detalle y estados de lista	Sprint 7	Completada

Casos de prueba

Tabla 6.7. Casos de prueba de MOVO-28 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-28.01	1	**Pagos del emisor y cobros del transportista** *Pre:* Usuario con transacciones en ambos roles. *Pasos:* GET de transacciones con role=sender y role=carrier.	role es obligatorio y se valida; cada rol ve solo lo propio y nunca las de otro usuario; los intentos creando, rechazados y cancelados sin pago no aparecen.	`transactions.integration.test.ts`	APROBADO
CP-28.02	2, 3	**Orden, paginación y formato** *Pre:* Transacciones en distintos meses y horarios. *Pasos:* Listar con paginación y rango de fechas.	Orden descendente y paginado; un pago de las 23:30 queda en su día (hora de Argentina) y el límite to es inclusivo; se rechazan fechas inválidas o from posterior a to.	`transactions.integration.test.ts`	APROBADO
CP-28.03	3	**Datos de la tarjeta y del recargo** *Pre:* Transacciones del emisor y del transportista. *Pasos:* Listar como cada uno.	La tarjeta aparece solo para el emisor (null si el hold no la tiene); el transportista no ve el recargo en el monto reintegrado ni en los movimientos.	`transactions.integration.test.ts`	APROBADO
CP-28.04	-	**Estados derivados y filtros** *Pre:* Transacciones en varios estados. *Pasos:* Filtrar por uno o varios estados.	Cada estado se deriva con su precedencia; el filtro se aplica antes de paginar; un estado desconocido se rechaza; refundedAmountArs es el último monto informado y no la suma.	`transactions.integration.test.ts` `transaction-status.test.ts`	APROBADO

6.2.4 MOVO-37 · Pago de la comisión en efectivo

MOVO-37 POR HACER SPRINT 8

Como emisor, quiero declarar que voy a pagar la comisión de Movo en efectivo al momento de la entrega, para poder pagar sin una tarjeta de crédito.

> **Estado.** Historia en *Por hacer*, fuera del alcance entregado: hoy el pago se hace solo con tarjeta mediante la reserva de fondos de MOVO-12. La documentación previa mencionaba el pago en efectivo; se corrigió porque no está implementado. Las tarjetas guardadas (MOVO-65) ya proveen la infraestructura que esta historia reutilizaría.

Casos de prueba

Tabla 6.8. Casos de prueba de MOVO-37 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-37.01	-	**Declarar pago en efectivo de la comisión** *Pre:* Historia no implementada. *Pasos:* Previsto: el emisor declara que pagará en efectivo y el transportista registra la comisión.	Casos de prueba por definir al implementarse la historia.	–	PENDIENTE

6.2.5 MOVO-65 · Tarjetas guardadas

MOVO-65 COMPLETADA SPRINT 7 13 SP

Como emisor, quiero guardar mis tarjetas en la app y pagar un envío ingresando solo el código de seguridad (CVV), para no cargar todos los datos de la tarjeta cada vez que acepto una oferta.

> **Decisión de diseño.** Movo nunca recibe ni guarda número de tarjeta ni CVV: la app tokeniza con el SDK de Mercado Pago y el backend solo conserva el identificador de la tarjeta en el *customer* de Mercado Pago y metadatos de display (marca, últimos 4 y vencimiento). Para pagar con una tarjeta guardada se pide solo el CVV, que la app vuelve a tokenizar.

Criterios de aceptación

1. El emisor puede guardar, listar, marcar como predeterminada y eliminar tarjetas desde Pagos y cobros.
2. Pagar un envío con una tarjeta guardada pide solo el CVV.
3. El backend nunca recibe ni loguea número de tarjeta ni CVV; solo `card_id` / `customer_id` de MP y metadatos de display (marca, últimos 4, vencimiento). Nunca se persiste un `card_token`.
4. Una tarjeta rechazada, vencida o con CVV incorrecto muestra un motivo claro y permite reintentar o elegir otra.
5. No se puede eliminar una tarjeta con un hold vivo asociado.
6. Las tarjetas guardadas del transportista para la comisión del pago en efectivo (`MOVO-37`) reutilizan esta misma infraestructura.

Trazabilidad de implementación

Tabla 6.9. Tickets de implementación de MOVO-65.

Ticket	Alcance	Sprint	Estado
MOVO-100	[svc-payments] Tarjetas guardadas del emisor en la cuenta de Movo: alta, listado, predeterminada, baja y datos de checkout	Sprint 7	Completada
MOVO-101	[movo-mobile] Pagos y cobros: tarjetas guardadas (alta, listado, predeterminada y baja)	Sprint 7	Completada
MOVO-282	[svc-payments][svc-shipments] Hold con tarjeta guardada: el pagador pasa a ser el customer de la tarjeta (token por CVV)	Sprint 7	Completada

Diagrama de secuencia

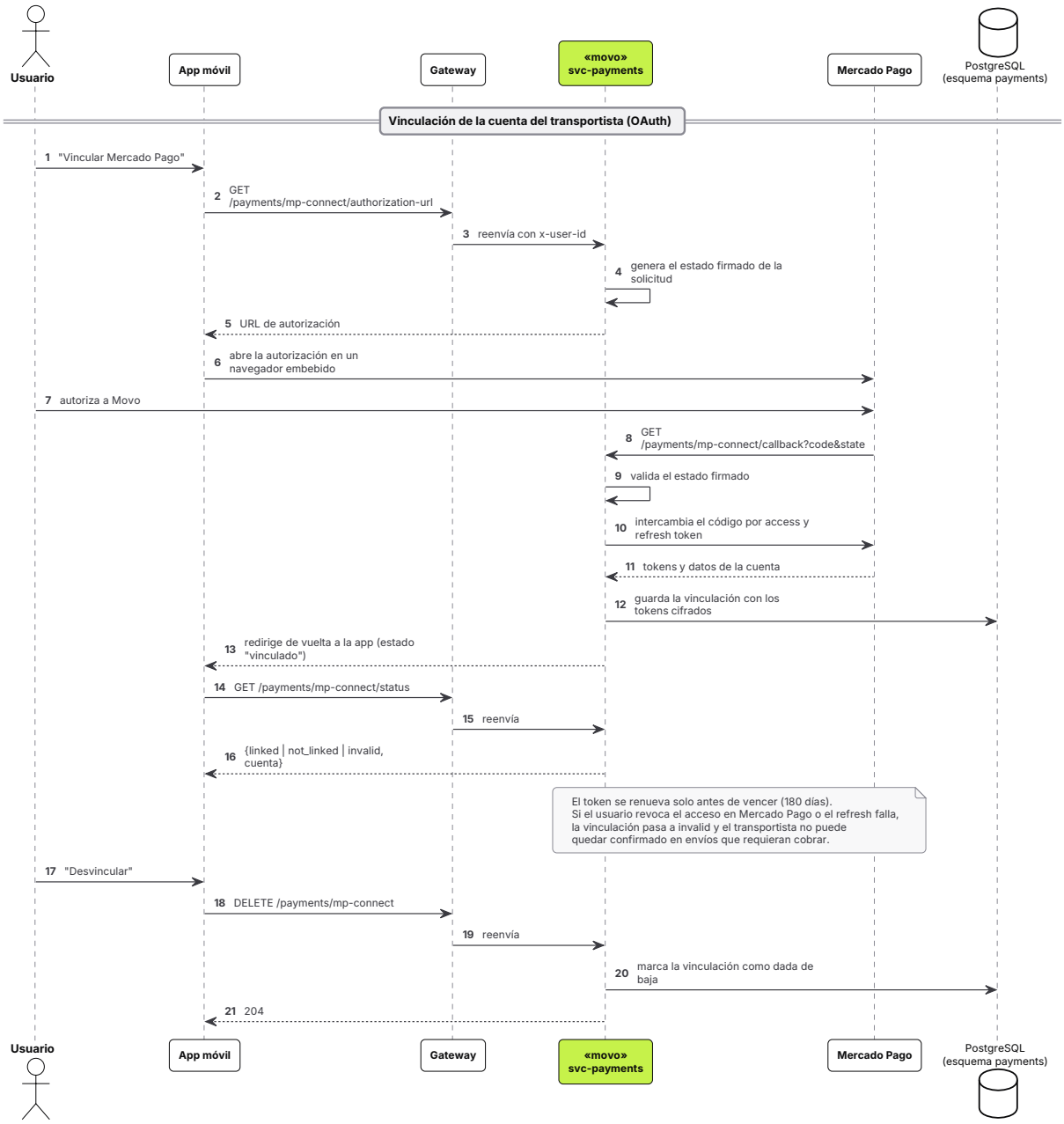


Figura 6.7. Vinculación de Mercado Pago del transportista y tarjetas guardadas del emisor (MOVO-110, MOVO-65). Parte 1 de 2: Vinculación de la cuenta del transportista (OAuth).

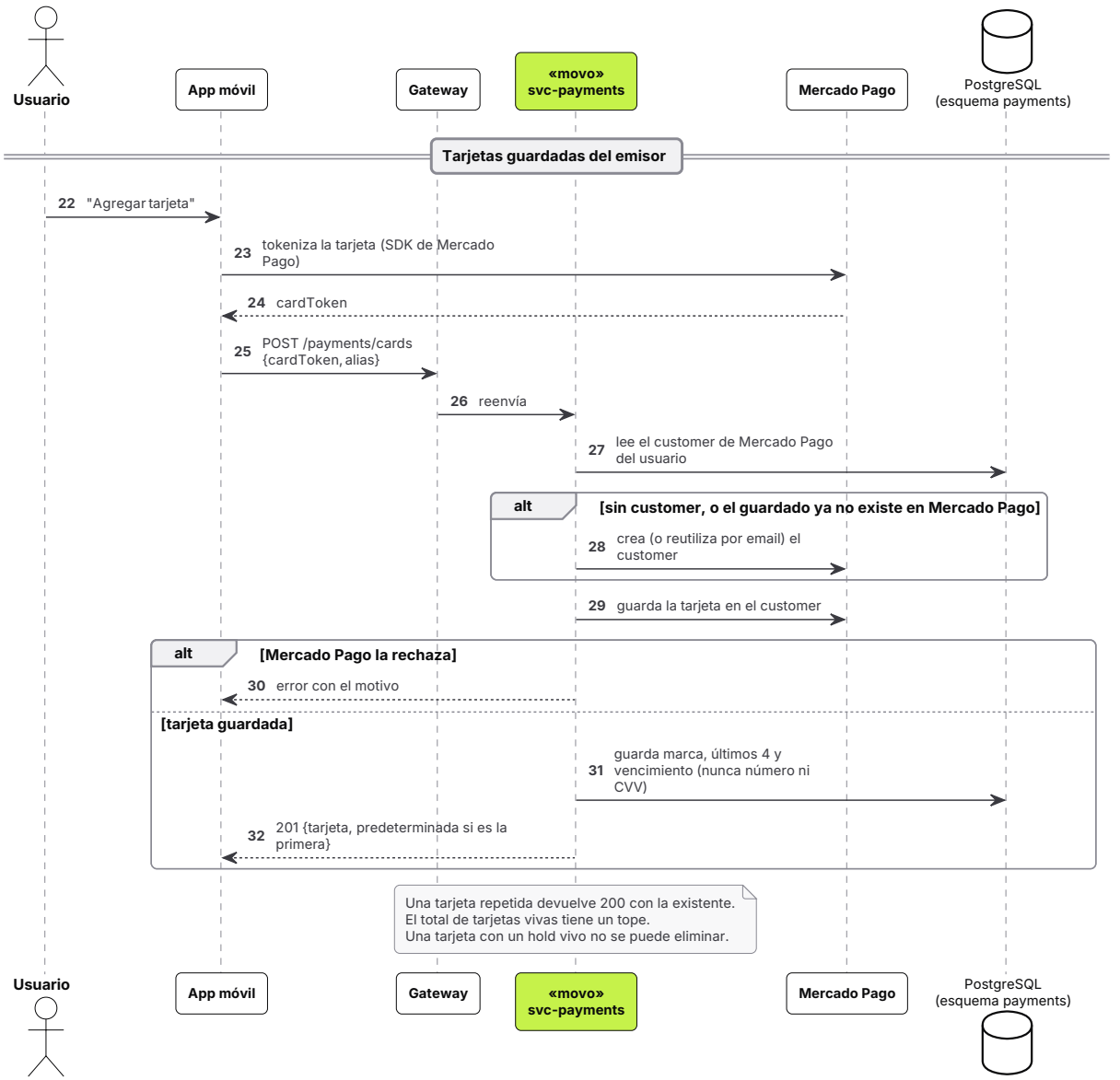


Figura 6.8. Vinculación de Mercado Pago del transportista y tarjetas guardadas del emisor (MOVO-110, MOVO-65). Parte 2 de 2: Tarjetas guardadas del emisor.

Casos de prueba

Tabla 6.10. Casos de prueba de MOVO-65 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-65.01	1, 3	**Guardar una tarjeta** *Pre:* Usuario autenticado. *Pasos:* Guardar una tarjeta nueva y una segunda.	Se crea el customer, la primera queda predeterminada y la segunda reutiliza el customer; sin alias queda en null; la respuesta no expone el número ni el CVV.	`cards.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-65.02	1	**Duplicados, tope y customer de Mercado Pago** *Pre:* Tarjeta repetida, tope alcanzado, customer ya existente o desaparecido. *Pasos:* Guardar la tarjeta en cada situación.	Una repetida devuelve 200 con la existente y borra la copia de MP; al pasar el tope no llama a MP; un customer del email ya existente se reutiliza salvo que sea de otra cuenta (409); uno inexistente se reemplaza.	`cards.integration.test.ts`	APROBADO
CP-65.03	4	**Fallas de Mercado Pago** *Pre:* Mercado Pago rechaza el alta o no responde. *Pasos:* Guardar una tarjeta.	El rechazo devuelve el motivo en el formato de error; sin respuesta, 502 sin persistir nada.	`cards.integration.test.ts`	APROBADO
CP-65.04	2	**Pagar con una tarjeta guardada** *Pre:* Tarjeta guardada del emisor. *Pasos:* POST /shipments/:id/funding con savedCardId y el token del CVV; probar un id inválido o inexistente.	El hold se crea con la tarjeta guardada; el cardToken sigue siendo obligatorio; un savedCardId que no es UUID se rechaza con 400 antes de llamar a payments; si payments responde 404 el envío no cambia.	`funding.integration.test.ts`	APROBADO
CP-65.05	5	**Integridad de las tarjetas** *Pre:* Tarjetas vivas, eliminadas y de otros usuarios. *Pasos:* Verificar los índices únicos y la propiedad del customer.	Los índices únicos son parciales sobre las filas vivas y un customer pertenece a una sola cuenta.	`cards.integration.test.ts`	APROBADO

6.2.6 MOVO-110 · Vinculación de la cuenta de Mercado Pago del transportista

MOVO-110 COMPLETADA SPRINT 7 6 SP

Como transportista, quiero vincular mi cuenta de Mercado Pago a Movo, para poder cobrar mis envíos vía split payment

> **Decisión de diseño.** La vinculación usa el flujo OAuth *Marketplace* de Mercado Pago en un navegador embebido y vuelve sola a la app mediante un callback del servidor. Los tokens se guardan cifrados, se renuevan antes de los 180 días y, si la vinculación se invalida (revocada o con el refresh fallido), el estado pasa a `invalid` y el transportista no puede quedar confirmado en envíos que requieran cobrar.

Criterios de aceptación

1. El transportista puede iniciar la vinculación de su cuenta de Mercado Pago desde la app, sin salir de la experiencia de Movo (navegador embebido, no una app de navegador aparte).
2. Al completar la autorización en Mercado Pago, la app vuelve sola al estado “vinculado”, sin pasos manuales de copiar/pegar códigos.
3. La vinculación se renueva sola del lado del servidor antes de expirar (180 días), el transportista no tiene que volver a autorizar periódicamente.

4. Si la vinculación se invalida (el transportista la revocó desde MP, o el refresh falla), el sistema lo detecta y se lo comunica claramente, ofreciendo re-vincular.
5. El transportista puede ver el estado de su vinculación y desvincular la cuenta manualmente si quiere.
6. Mientras no tenga una cuenta vinculada y vigente, el sistema no le permite quedar confirmado en envíos que requieran cobrar (evita que MOVO-12/13 lleguen al momento del split sin un `access_token` válido del transportista).

Trazabilidad de implementación

Tabla 6.11. Tickets de implementación de MOVO-110.

Ticket	Alcance	Sprint	Estado
`MOVO-111`	[svc-payments] OAuth Connect: vinculación, estado y desvinculación de la cuenta de Mercado Pago del transportista	Sprint 7	Completada
`MOVO-112`	[movo-mobile] Pantalla de vinculación de cuenta de Mercado Pago (transportista)	Sprint 7	Completada
`MOVO-243`	[svc-payments] OAuth Connect: renovación automática de tokens (cron) y marcado de vinculación revocada	Sprint 7	Completada

Casos de prueba

Tabla 6.12. Casos de prueba de MOVO-110 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-110.01	1, 2, 5	**Estado de la vinculación** *Pre:* Transportista sin vincular, vinculado y desvinculado. *Pasos:* GET /payments/mp-connect/status en cada estado.	unlinked sin vinculación; linked con la cuenta y sin tokens expuestos; tras desvincular vuelve a no vinculada con la fecha de baja.	`mp-connect.integration.test.ts`	APROBADO
CP-110.02	3, 4	**Vinculación inválida** *Pre:* Vinculación revocada o con el token vencido. *Pasos:* Consultar el estado.	invalid con el motivo (revoked o expired), devolviendo igual la cuenta para ofrecer re-vincular.	`mp-connect.integration.test.ts`	APROBADO
CP-110.03	6	**Requisito para operar como transportista** *Pre:* Transportista con cuenta vigente, revocada, vencida o sin clave pública. *Pasos:* Consultar la elegibilidad interna y el checkout.	linked: true solo con una cuenta vigente y con clave pública (el mismo criterio que el hold); el checkout responde 409 explícito si el transportista no vinculó Mercado Pago.	`mp-connect.integration.test.ts` `holds.integration.test.ts`	APROBADO
CP-110.04	-	**Intercambio OAuth y cifrado de tokens** *Pre:* Código de autorización y tokens de Mercado Pago. *Pasos:* Intercambiar el código, cifrar y descifrar los tokens, renovar.	Los tokens se guardan cifrados con una clave propia y se renuevan sin intervención; el cliente de OAuth maneja los errores de Mercado Pago.	`mercadopago-oauth-client.test.ts` `token-cipher.test.ts`	APROBADO

6.2.7 MOVO-226 · Penalidad de cancelación y cuenta corriente

MOVO-226 EN CURSO SPRINT 7, SPRINT 8 30 SP

Definir e implementar la mecánica de penalidad de cancelación con Transportista asignado (holds/liberación de pagos)

Decisión de diseño (DD-047). Cancelar con un envío ya asignado no es gratuito si el retiro está cerca: con 12 horas o más de anticipación, o con la ventana de retiro ya cerrada, no hay cargo; con menos de 12 horas se cobra el 25 % del precio acordado mediante una **captura parcial** del hold. El emisor ve y confirma el monto exacto antes de cancelar; si cambió, la cancelación se rechaza (409) y no cobra nada. Lo que no se puede cobrar en el momento queda como **deuda** en la cuenta corriente del usuario, que se descuenta de su próxima operación y, si pasan más de 14 días sin saldar, bloquea nuevas operaciones. **Historia en curso:** sus tickets de términos y condiciones (MOVO-316) siguen abiertos.

Trazabilidad de implementación

Tabla 6.13. Tickets de implementación de MOVO-226.

Ticket	Alcance	Sprint	Estado
MOVO-305	[svc-payments] Spike: captura parcial y application_fee variable en sandbox	Sprint 7	Completada
MOVO-306	[svc-payments] Captura parcial del hold por penalidad de cancelación	Sprint 7	Completada
MOVO-307	[svc-shipments] Cancelación del emisor desde assigned con penalidad (umbral 12 h)	Sprint 7	Completada
MOVO-308	[mobile] Emisor: cancelar un envío asignado con confirmación de penalidad	Sprint 7	Completada
MOVO-309	[svc-payments] Cuenta corriente del usuario (deudas, pagos y saldo con signo)	Sprint 7	Completada
MOVO-310	[svc-shipments] Cancelación del transportista de un envío asignado (vuelve a published + deuda si < 12 h)	Sprint 7	Completada
MOVO-311	[svc-payments] Cobrar la deuda en la próxima operación del usuario vía application_fee (como transportista o como emisor)	Sprint 7	Completada
MOVO-312	[svc-payments] Pagar mi saldo: el usuario salda su deuda con tarjeta guardada + CVV	Sprint 7	Completada
MOVO-313	[svc-shipments][svc-payments] Bloqueo por deuda vencida en la cuenta corriente	Sprint 7	Completada
MOVO-314	[mobile] Transportista: bajarse de un envío asignado y cancelar un viaje con paquetes	Sprint 7	Completada
MOVO-315	[mobile] Cuenta corriente: saldo, “Pagar mi saldo” y deuda al cobrar o pagar	Sprint 7	Completada

Ticket	Alcance	Sprint	Estado
MOVO-316	[docs] Términos y Condiciones: penalidades de cancelación y cuenta corriente del usuario	Sprint 8	En curso

Diagrama de secuencia

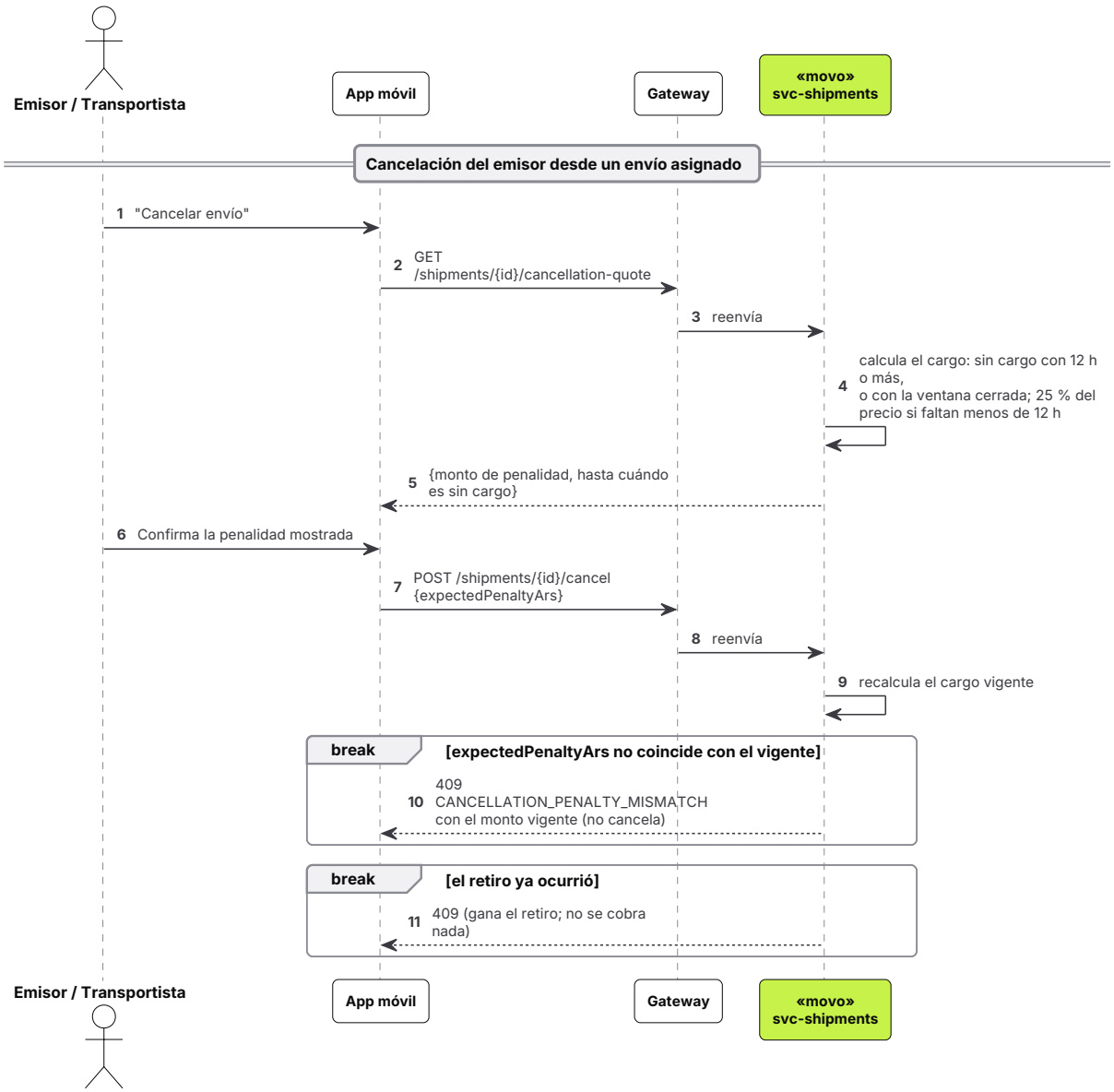


Figura 6.9. Cancelación con penalidad, baja del transportista y cuenta corriente del usuario (MOVO-226, MOVO-307, MOVO-310). Parte 1 de 4: Cancelación del emisor desde un envío asignado.

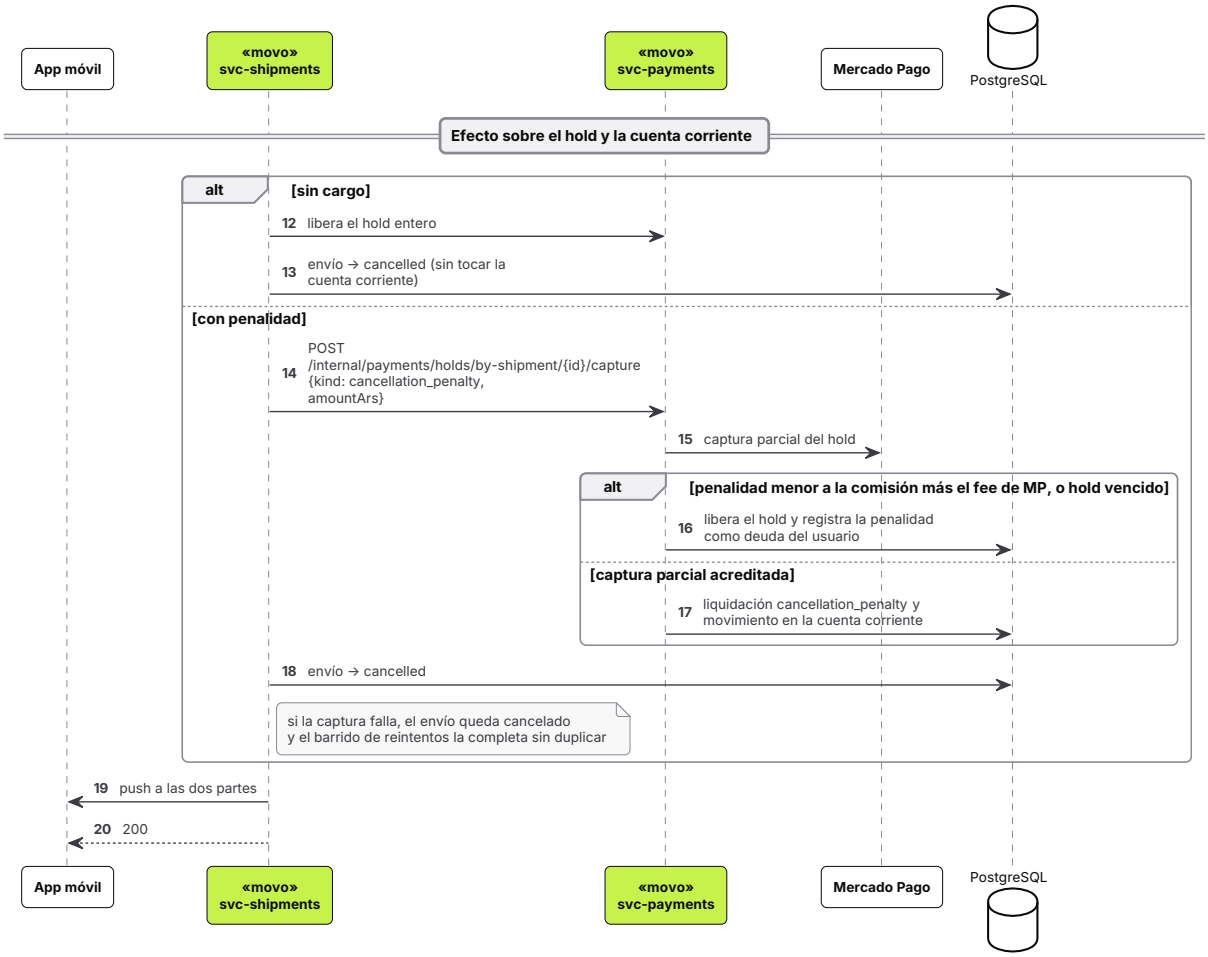


Figura 6.10. Cancelación con penalidad, baja del transportista y cuenta corriente del usuario (MOVO-226, MOVO-307, MOVO-310). Parte 2 de 4: Efecto sobre el hold y la cuenta corriente.

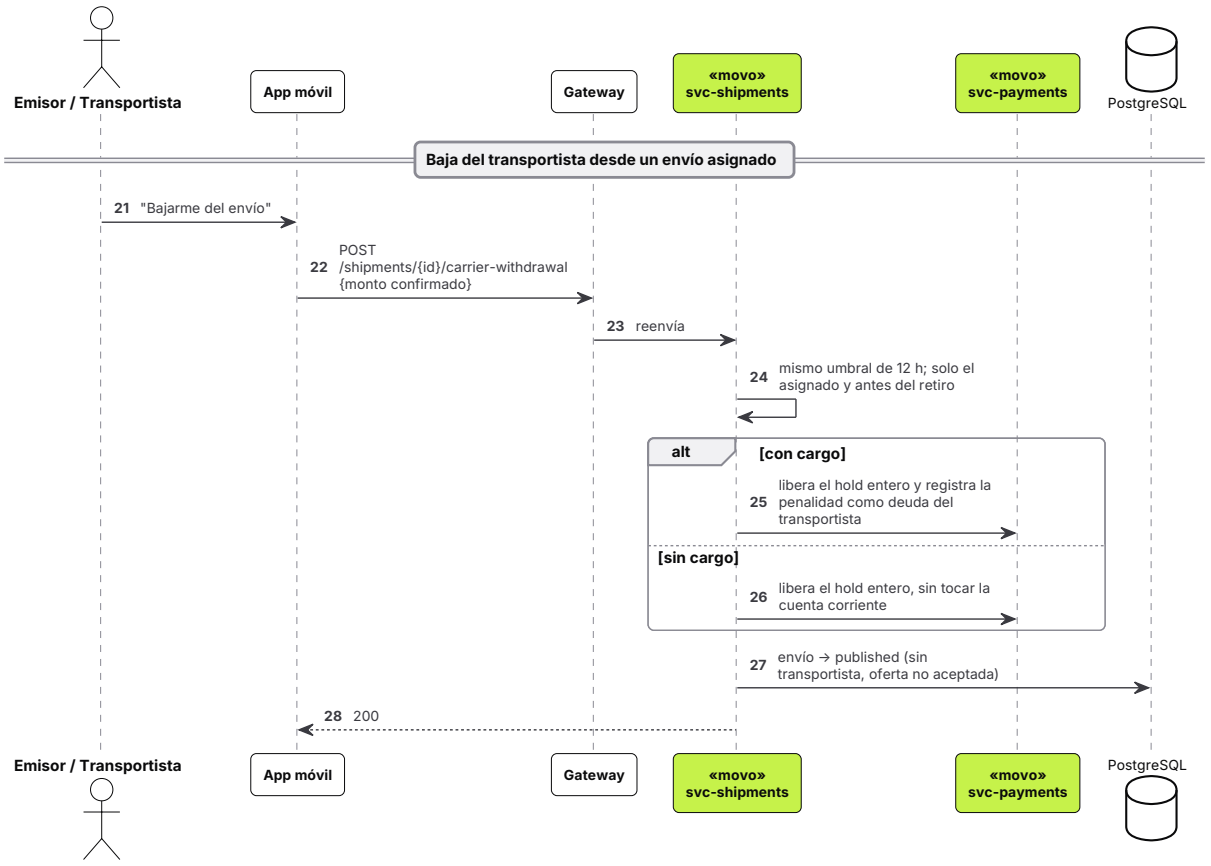


Figura 6.11. Cancelación con penalidad, baja del transportista y cuenta corriente del usuario (MOVO-226, MOVO-307, MOVO-310). Parte 3 de 4: Baja del transportista desde un envío asignado.

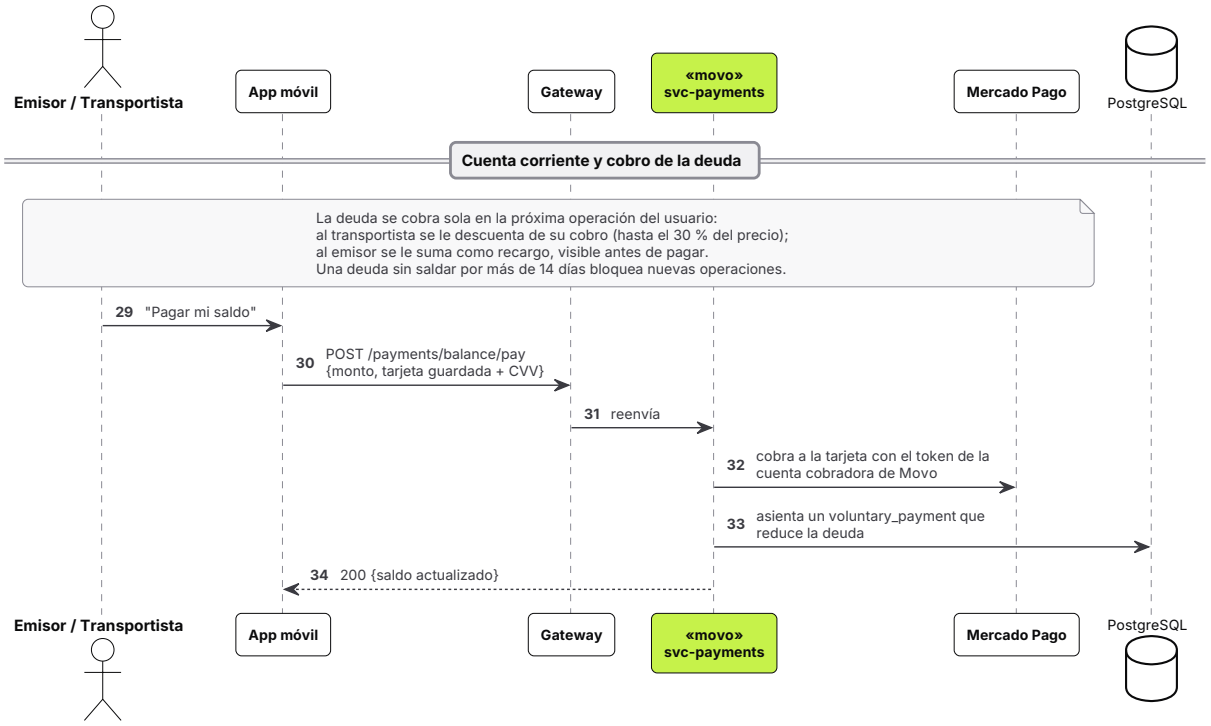


Figura 6.12. Cancelación con penalidad, baja del transportista y cuenta corriente del usuario (MOVO-226, MOVO-307, MOVO-310). Parte 4 de 4: Cuenta corriente y cobro de la deuda.

Casos de prueba

Tabla 6.14. Casos de prueba de MOVO-226 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-226.01	1, 3	**Cargo calculado según la anticipación** *Pre:* Envío assigned con distintas horas al retiro. *Pasos:* GET /shipments/:id/cancellation-quote.	Con 12 h o más: sin cargo, indicando hasta cuándo; con menos de 12 h: 25 % del precio acordado; con la ventana ya cerrada: sin cargo, salvo que el transportista haya llegado al punto de retiro (DD-052), caso en que se cobra el 25 %. Antes de assigned no hay penalidad.	`shipments-cancel-penalty.integration.test.ts`	APROBADO
CP-226.02	3	**Cancelación con penalidad** *Pre:* Envío assigned con menos de 12 h al retiro y hold autorizado. *Pasos:* POST /shipments/:id/cancel con el monto mostrado.	El envío se cancela, se captura parte del hold, se registra la deuda y el cobro y se avisa a ambas partes. Si la penalidad no cubre la comisión más el fee de MP, se libera el hold y queda como deuda.	`shipments-cancel-penalty.integration.test.ts` `captures.integration.test.ts`	APROBADO
CP-226.03	3	**Monto distinto o retiro concurrente** *Pre:* Monto desactualizado o retiro ya ocurrido. *Pasos:* Cancelar con un monto que no coincide, o con el retiro ya hecho.	409 CANCELLATION_PENALTY_MISMATCH con el monto vigente sin cancelar; si el retiro gana, 409 sin cobrar nada. Un monto viejo con la ventana cerrada también da 409.	`shipments-cancel-penalty.integration.test.ts`	APROBADO
CP-226.04	–	**Cancelación sin cargo** *Pre:* Envío assigned con 12 h o más, o hold vencido o inexistente. *Pasos:* Cancelar.	Sin cargo, libera el hold entero y no toca la cuenta corriente; con hold vencido o inexistente, la penalidad queda como deuda. Si la captura falla, el envío queda cancelado y el barrido la reintenta sin duplicar.	`shipments-cancel-penalty.integration.test.ts`	APROBADO
CP-226.05	–	**Baja del transportista** *Pre:* Transportista asignado con y sin hold. *Pasos:* POST /shipments/:id/carrier-withdrawal con distintas anticipaciones.	Republica el envío, desasigna, libera el hold entero y registra la penalidad como deuda del transportista (sin cargo con 12 h o más); solo el transportista asignado y antes del retiro; si Payments falla, conserva la asignación y se puede reintentar; con el retiro concurrente gana uno solo.	`shipments-carrier-withdrawal.integration.test.ts`	APROBADO
CP-226.06	–	**Cuenta corriente del usuario** *Pre:* Usuario con y sin movimientos. *Pasos:* Registrar movimientos y consultar saldo y deuda.	Suma con signo los movimientos no anulados, calcula la deuda más vieja, admite saldo a favor, no mezcla usuarios y es idempotente (201 y luego 200 con el mismo movimiento).	`user-ledger.integration.test.ts` `user-ledger-origins.service.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-226.07	-	**Cobro de la deuda en la próxima operación** *Pre:* Transportista o emisor con deuda. *Pasos:* Crear un hold con deuda pendiente.	Al transportista se le descuenta toda la deuda del fee si es menor al tope, o el 30 % del precio con el resto pendiente; al emisor se le suma como recargo, visible en el checkout sin reservar nada. Si el hold se libera, rechaza o vence, la reserva se anula y la deuda vuelve a pendiente; dos holds en paralelo no descuentan dos veces.	`debt-collection.integration.test.ts` `debt-discount.test.ts`	APROBADO
CP-226.08	-	**Pagar mi saldo** *Pre:* Usuario con deuda y tarjeta guardada. *Pasos:* GET /payments/balance/pay-data y POST /payments/balance/pay.	422 si no hay deuda o el monto la supera; 400 con monto inválido; un pago aprobado cobra con el token de la cuenta cobradora de Movo y asienta un único voluntary_payment; la respuesta no expone el customer ni la tarjeta.	`balance-payment.integration.test.ts`	APROBADO
CP-226.09	-	**Bloqueo por deuda vencida** *Pre:* Deuda sin saldar por más de 14 días. *Pasos:* Evaluar si la deuda está vencida.	Más de 14 días: vencida (a los 14 justos todavía no); una deuda reciente no bloquea; sin umbral de monto, una deuda de un peso bloquea igual; se respeta el umbral configurado.	`user-debt.test.ts`	APROBADO

07

TEMA 6 DE 7 · TRAZABILIDAD DE HISTORIAS DE USUARIO

EP-06 · Reputación y Comunidad

Calificaciones entre las partes, score de reputación ponderado, reportes y bloqueos entre usuarios.

CONTENIDO

7.1	Descripción del Tema	151
7.2	Historias de Usuario y Trazabilidad	152

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: trazabilidad de las historias del tema, con criterios de aceptación de Linear, diagramas de secuencia derivados del código y casos de prueba derivados de las pruebas automatizadas.

Introducción

Este documento presenta la trazabilidad de las historias de usuario del tema *Reputación y Comunidad*: para cada historia, su descripción y criterios de aceptación, los tickets que la implementaron, el diagrama de secuencia que muestra cómo la resuelve el sistema y los casos de prueba que la verifican.

Audiencia

El tribunal evaluador y los futuros mantenedores. Se asume conocimiento de las convenciones del proyecto y de UML.

7.1 Descripción del Tema

EP-06 · Reputación y Comunidad es la cuarta capa de confianza: el historial verificable de lo que cada persona hizo en la plataforma. Tras la entrega, las tres partes de un envío pueden calificarse durante 72 horas; esas calificaciones alimentan un **score de reputación** que se muestra en el perfil y que también desempata las ofertas de los transportistas (tema *Asignación y Negociación*). Además, cualquier usuario puede **reportar** a otro con evidencia fotográfica y **bloquearlo**, con efectos concretos sobre el feed, las ofertas y el chat.

El score no es un promedio simple: aplica una contracción bayesiana hacia la media global y un decaimiento temporal, para que ni una sola calificación perfecta ni una vieja distorsionen la imagen de una persona. Es un cálculo del lado de `movo-svc-shipments` que `movo-svc-users` compone en el perfil.

Servicios involucrados: `movo-svc-shipments` (calificaciones y cálculo de reputación), `movo-svc-users` (perfil, reportes y bloqueos), `gateway` y `movo-mobile`; **externo:** Amazon S3 (fotos de los reportes).

7.2 Historias de Usuario y Trazabilidad

Tabla 7.1. Historias de usuario del tema.

Historia	Descripción	Estado	Sprint
`MOVO-22`	Calificación entre las partes de un envío	Completada	Sprint 4
`MOVO-25`	Score de reputación ponderado en el perfil	Completada	Sprint 4, Sprint 5, Sprint 6
`MOVO-175`	Reportar y bloquear usuarios	Completada	Sprint 6

7.2.1 MOVO-22 · Calificación entre las partes de un envío

MOVO-22 COMPLETADA SPRINT 4 6 SP

Como usuario, quiero calificar y dejar un comentario a las otras partes post-entrega, para contribuir al sistema de confianza de la red.

> **Decisión de diseño.** Las tres partes del envío (emisor, transportista y receptor) pueden calificarse entre sí con 1 a 5 estrellas, categorías según el rol y un comentario opcional, durante 72 horas desde la entrega. Si el envío entra en disputa, esa ventana se extiende por el tiempo que duró. Un envío `completed` (entregado y pagado) se califica igual que uno `delivered` (MOVO-208).

Criterios de aceptación

1. Una vez completado el handshake de entrega, las tres partes (emisor, transportista, receptor) reciben habilitacion para calificar.
2. La calificación es de 1 a 5 estrellas y un comentario de texto libre opcional.
3. La opción de calificar expira a las 72 horas de completada la entrega.
4. Una vez enviada, la calificación puede editarse.
5. Las calificaciones de las dos partes son visibles en el perfil del usuario calificado.
6. Un usuario no puede calificar a otro usuario con quien no haya tenido una transacción completada.
7. Si hay una disputa activa, la habilitación de calificación queda suspendida hasta la resolución.
8. Cada vez que se crea o edita una calificación en la tabla ratings, se actualiza automáticamente el campo `rating_overall` (decimal) en la tabla `User` para evitar recalcular el score de reputación en cada consulta.

Trazabilidad de implementación

Tabla 7.2. Tickets de implementación de MOVO-22.

Ticket	Alcance	Sprint	Estado
MOVO-146	[svc-shipments] Schema de Rating y endpoint de calificación post-entrega	Sprint 4	Completada
MOVO-153	[movo-mobile] Calificación post-entrega de la contraparte	Sprint 4	Completada

Diagrama de secuencia

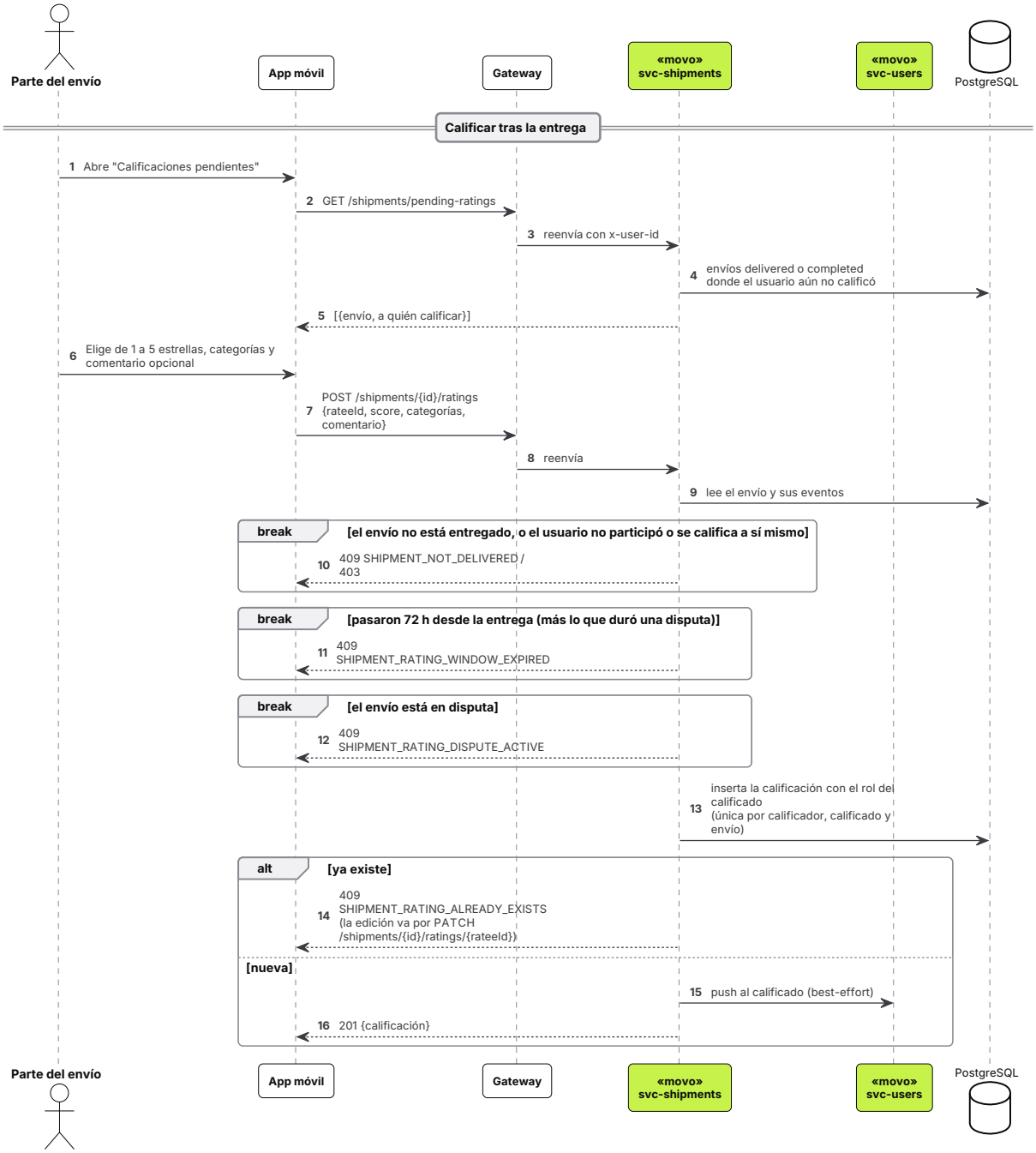


Figura 7.1. Calificación tras la entrega y reputación en el perfil (MOVO-22, MOVO-25). Parte 1 de 2: Calificar tras la entrega.

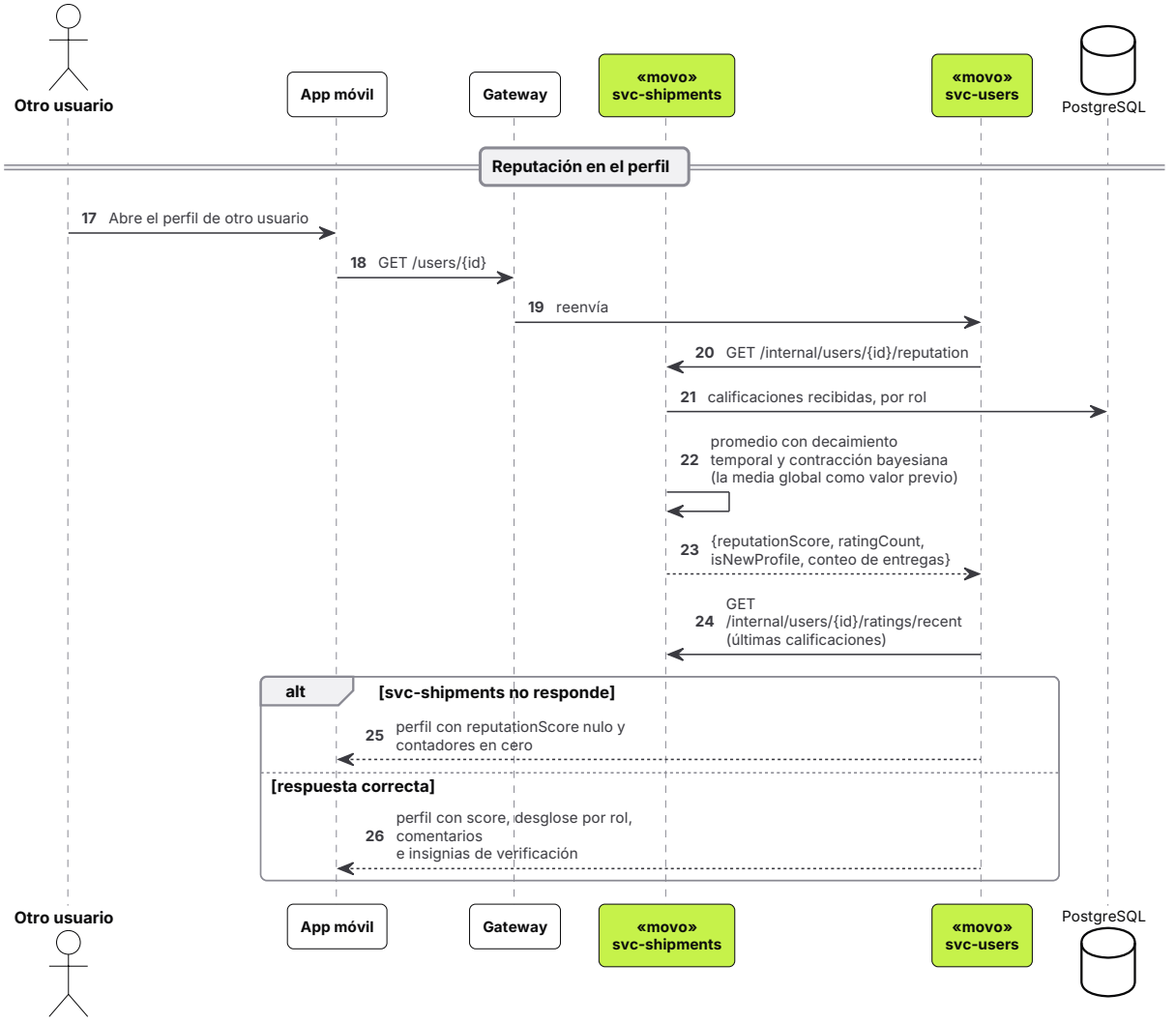


Figura 7.2. Calificación tras la entrega y reputación en el perfil (MOVO-22, MOVO-25). Parte 2 de 2: Reputación en el perfil.

Casos de prueba

Tabla 7.3. Casos de prueba de MOVO-22 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-22.01	1, 2	**Calificar a la contraparte** *Pre:* Envío entregado y participante habilitado. *Pasos:* POST /shipments/:id/ratings con score, categorías y comentario.	201; se persiste la calificación con el rol del calificado y se avisa por push. Se aceptan las categorías del emisor, transportista y receptor (puntualidad, comunicación, etc.).	`ratings.integration.test.ts` `ratings-service.test.ts`	APROBADO
CP-22.02	6	**Solo participantes de un envío entregado** *Pre:* Envío no entregado, calificador ajeno, calificado ajeno o autocalificación. *Pasos:* Calificar en cada caso.	409 SHIPMENT_NOT_DELIVERED si no está entregado; 403 si el calificador o el calificado no participó, o si se califica a sí mismo.	`ratings.integration.test.ts` `ratings-service.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-22.03	2	**Rango del puntaje** *Pre:* Participante habilitado. *Pasos:* Calificar con un puntaje fuera de 1 a 5.	400 VALIDATION_FAILED.	`ratings.integration.test.ts`	APROBADO
CP-22.04	3	**Ventana de 72 horas** *Pre:* Envío entregado hace más y menos de 72 h. *Pasos:* Calificar en cada caso.	Dentro de la ventana se acepta (inclusive hasta el instante del límite); fuera, 409 SHIPMENT_RATING_WINDOW_EXPIRED. Un período completo en disputa extiende el límite por su duración.	`ratings.integration.test.ts` `rating-window.test.ts`	APROBADO
CP-22.05	7	**Disputa activa** *Pre:* Envío en disputa. *Pasos:* Calificar.	409 SHIPMENT_RATING_DISPUTE_ACTIVE hasta la resolución.	`ratings.integration.test.ts` `ratings-service.test.ts`	APROBADO
CP-22.06	4	**Una calificación por par y edición** *Pre:* Calificación ya enviada. *Pasos:* Calificar de nuevo; editar con PATCH /shipments/:id/ratings/:rateeId.	409 SHIPMENT_RATING_ALREADY_EXISTS; la edición actualiza la fila existente en lugar de crear otra.	`ratings.integration.test.ts`	APROBADO
CP-22.07	5	**Consulta de las calificaciones de un envío** *Pre:* Envío con calificaciones. *Pasos:* GET /shipments/:id/ratings como participante y como ajeno.	Los participantes ven las calificaciones; un ajeno recibe 403.	`ratings.integration.test.ts`	APROBADO
CP-22.08	1	**Calificaciones pendientes** *Pre:* Envíos entregados con y sin calificar. *Pasos:* GET /shipments/pending-ratings.	Lista a quién falta calificar en cada envío entregado o completado; ya calificado no aparece; sin envíos, 200 con lista vacía; sin identidad, 401.	`shipments-pending-ratings.integration.test.ts`	APROBADO

7.2.2 MOVO-25 · Score de reputación ponderado en el perfil

MOVO-25 COMPLETADA SPRINT 4, SPRINT 5, SPRINT 6 26 SP

Como usuario, quiero ver el score de reputación ponderado en el perfil de cualquier otro usuario, para tomar decisiones informadas.

Decisión de diseño. El score no es un promedio simple: usa **contracción bayesiana** hacia la media global de la plataforma (una única calificación de 5 estrellas nunca da 5,0) y un **decaimiento temporal** con semivida configurable, de modo que una calificación vieja pesa menos que una reciente. Con menos de 3 calificaciones el perfil se marca como nuevo, pero el score igual se calcula. El perfil distingue lo recibido como emisor, transportista y receptor, y trae los comentarios recientes y las insignias de verificación.

Criterios de aceptación

1. El perfil de cualquier usuario muestra su score de reputacion promedio (1-5) y el numero de transacciones calificadas.

2. El score de reputación se consulta directamente desde el campo `rating_overall` (decimal) almacenado en la entidad `User`, evitando recalcular el promedio sobre la tabla de ratings en cada petición.
3. El score se calcula con un promedio ponderado que da mas peso a calificaciones recientes.
4. El perfil diferencia las calificaciones recibidas como emisor, transportista y receptor.
5. Los comentarios de las ultimas 10 calificaciones son visibles en el perfil.
6. El perfil muestra insignias de verificación (KYC completado, licencia verificada si aplica).
7. Un usuario con menos de 3 transacciones muestra 'perfil nuevo' en lugar de un score.
8. El score se actualiza en tiempo real al recibir nuevas calificaciones.

Trazabilidad de implementación

Tabla 7.4. Tickets de implementación de MOVO-25.

Ticket	Alcance	Sprint	Estado
`MOVO-147`	[svc-shipments] Score de reputación ponderado y endpoint interno de agregado	Sprint 4	Completada
`MOVO-152`	[svc-users] Perfil con reputación y contadores de transacciones reales	Sprint 4	Completada
`MOVO-154`	[movo-mobile] Reputación visible: perfil propio, perfil público y tarjetas de oferta	Sprint 4	Completada
`MOVO-170`	[svc-users][svc-shipments] Enriquecimiento de perfil con datos ya existentes (antigüedad, verificaciones públicas, historial compartido, stats básicas)	Sprint 4	Completada
`MOVO-171`	[movo-mobile][svc-users] Bio editable del perfil	Sprint 4	Completada
`MOVO-172`	[movo-mobile][svc-users] Ficha de vehículo del transportista	Sprint 5	Completada
`MOVO-173`	[movo-mobile][svc-shipments] Calificación por categorías (puntualidad/cuidado/comunicación)	Sprint 6	Completada
`MOVO-174`	[movo-mobile][svc-users] Conexiones mutuas en el perfil ("ya envió con X personas con las que vos también enviaste")	Sprint 6	Completada
`MOVO-175`	[movo-mobile][svc-users] Reportar y bloquear usuarios	Sprint 6	Completada
`MOVO-176`	[movo-mobile] Rediseño de pantalla de perfil (prototipo Claude Design): sheet → pantalla completa	Sprint 4	Completada

Casos de prueba

Tabla 7.5. Casos de prueba de MOVO-25 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-25.01	1, 3	**Cálculo del score** *Pre:* Calificaciones de distintos valores y antigüedades. *Pasos:* Calcular la reputación con 0, 1 y muchas calificaciones.	Sin calificaciones el score es nulo y el perfil es nuevo; una sola calificación de 5 queda cerca de la media global y nunca en 5,0; con muchas consistentes converge a la media real; una calificación vieja pesa menos que una reciente.	`reputation.test.ts`	APROBADO
CP-25.02	7	**Perfil nuevo** *Pre:* Usuario con menos de 3 calificaciones, y con 3 o más. *Pasos:* Consultar la reputación.	isNewProfile es verdadero con menos de 3 (el score igual viaja) y falso con 3 o más, sin contaminar un rol nunca calificado.	`ratings.integration.test.ts` `reputation.test.ts`	APROBADO
CP-25.03	4	**Desglose por rol y conteos** *Pre:* Usuario con envíos como emisor, transportista y receptor. *Pasos:* GET /internal/users/:id/reputation.	Los conteos incluyen solo envíos delivered o completed, separados por rol; el desglose por rol y por categoría se informa solo si hay datos.	`ratings.integration.test.ts`	APROBADO
CP-25.04	5	**Comentarios recientes** *Pre:* Usuario con calificaciones recibidas. *Pasos:* GET /internal/users/:id/ratings/recent con y sin cursor.	Devuelve las últimas calificaciones recibidas, paginadas con cursor, con el nombre de quien calificó.	`ratings.integration.test.ts` `users.reputation.integration.test.ts`	APROBADO
CP-25.05	2, 6	**Perfil con reputación e insignias** *Pre:* Perfil propio y perfil público. *Pasos:* GET /users/me y GET /users/:id.	Devuelve reputationScore, conteos, desglose por rol, isNewProfile y datos de miembro; si svc-shipments no responde, el perfil se devuelve igual con score nulo y contadores en cero.	`users.reputation.integration.test.ts`	APROBADO
CP-25.06	-	**Conexiones en común** *Pre:* Dos usuarios con y sin envíos en común. *Pasos:* Consultar las conexiones mutuas.	Cuenta cada contraparte compartida una sola vez, en cualquier rol, sin incluir a los dos usuarios.	`mutual-connections.integration.test.ts`	APROBADO

7.2.3 MOVO-175 · Reportar y bloquear usuarios

MOVO-175 COMPLETADA SPRINT 6 2 SP

[movo-mobile][svc-users] Reportar y bloquear usuarios

> **Alcance final.** La historia no tiene criterios de aceptación en Linear; su alcance quedó en el reporte con motivo, detalle y fotos (MOVO-256) y en el bloqueo bidireccional. El bloqueo no es simétrico en su efecto: oculta envíos y bloquea ofertas y designaciones en cualquier dirección, pero en el chat no impide escribir, solo oculta el contenido a quien tiene el bloqueo. Los reportes los revisa el equipo desde el panel de administración.

Trazabilidad de implementación

Tabla 7.6. Tickets de implementación de MOVO-175.

Ticket	Alcance	Sprint	Estado
`MOVO-256`	[movo-mobile][svc-users] Adjuntar imágenes a un reporte de usuario	Sprint 6	Completada

Diagrama de secuencia

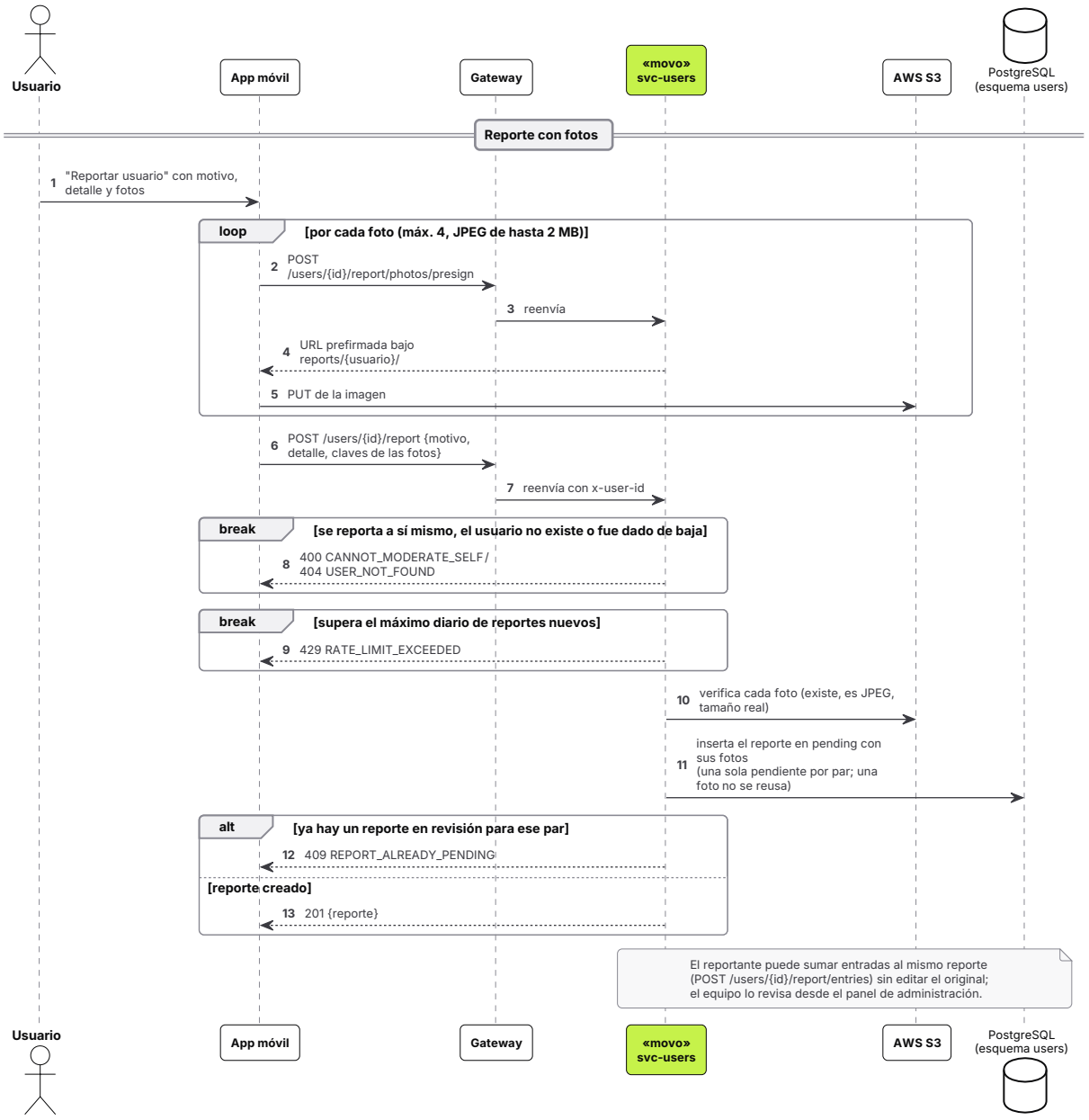


Figura 7.3. Reporte con fotos y bloqueo entre usuarios (MOVO-175, MOVO-256, DD-026). Parte 1 de 2: Reporte con fotos.

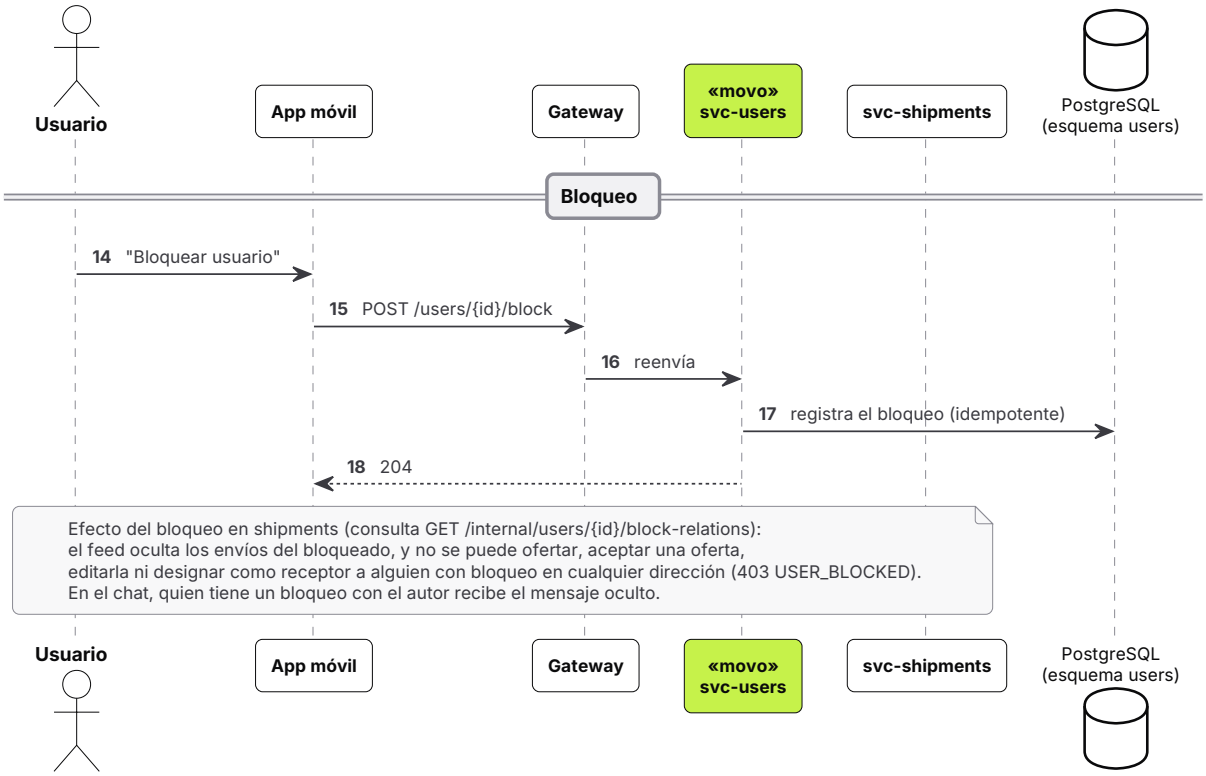


Figura 7.4. Reporte con fotos y bloqueo entre usuarios (MOVO-175, MOVO-256, DD-026). Parte 2 de 2: Bloqueo.

Casos de prueba

Tabla 7.7. Casos de prueba de MOVO-175 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-175.01	–	**Crear un reporte** *Pre:* Usuario autenticado y otro usuario. *Pasos:* POST /users/:id/report con motivo y detalle.	201 con el reporte en pending y el detalle recortado; no se puede reportar a uno mismo (400 CANNOT_MODERATE_SELF), a un usuario inexistente o dado de baja (404), ni con motivo inválido (400).	`moderation.integration.test.ts`	APROBADO
CP-175.02	–	**Un reporte pendiente por par y cupo diario** *Pre:* Reporte ya en revisión; reportes del día. *Pasos:* Reportar de nuevo y al pasar el máximo diario; dos reportes concurrentes.	409 REPORT_ALREADY_PENDING sin consumir cupo; 429 RATE_LIMIT_EXCEEDED al pasar el máximo; dos reportes concurrentes del mismo par crean una sola fila.	`moderation.integration.test.ts`	APROBADO
CP-175.03	–	**Entradas adicionales al reporte** *Pre:* Reporte propio en revisión. *Pasos:* POST /users/:id/report/entries; consultar el reporte propio y el de un tercero.	Suma entradas en orden sin editar el original; el usuario ve el reporte propio y nunca el de un tercero.	`moderation.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-175.04-		**Fotos del reporte** *Pre:* Fotos subidas, inexistentes o de otro usuario. *Pasos:* Presignar, subir y adjuntar fotos al reporte.	Solo JPEG de hasta 2 MB (se verifica el objeto real); una clave de otro usuario es 403 PHOTO_FORBIDDEN_KEY, una foto no subida 422, una reusada 409 REPORT_PHOTO_ALREADY_USED; una entrada puede ser solo fotos y no consume cupo extra.	`moderation.photos.integration.test.ts`	APROBADO
CP-175.05-		**Efectos del bloqueo en los envíos** *Pre:* Un bloqueo entre emisor o receptor y un transportista. *Pasos:* Listar el feed, ofertar, aceptar y editar ofertas, designar receptor.	El feed oculta el envío; 403 USER_BLOCKED en cualquier dirección al ofertar, aceptar, editar o designar; el emisor no ve ofertas de un bloqueado hechas antes. El feed falla abierto y crear una oferta falla cerrado (502 si users no responde).	`block-relations.integration.test.ts`	APROBADO
CP-175.06-		**Bloqueo en el chat** *Pre:* Un participante con un bloqueo hacia el autor. *Pasos:* El autor escribe en el chat del envío.	Escribir no se impide; quien tiene el bloqueo recibe el mensaje oculto y los demás lo ven normal.	`chat.integration.test.ts`	APROBADO

08

TEMA 7 DE 7 · TRAZABILIDAD DE HISTORIAS DE USUARIO

EP-07 · Gestión y Administración

Acceso seguro al panel, ficha de usuarios, sanciones de cuenta, monitoreo de envíos, disputas, soporte y cumplimiento legal.

CONTENIDO

8.1	Descripción del Tema	163
8.2	Historias de Usuario y Trazabilidad	164

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: trazabilidad de las historias del tema, con criterios de aceptación de Linear, diagramas de secuencia derivados del código y casos de prueba derivados de las pruebas automatizadas.

Introducción

Este documento presenta la trazabilidad de las historias de usuario del tema *Gestión y Administración*: para cada historia, su descripción y criterios de aceptación, los tickets que la implementaron, el diagrama de secuencia que muestra cómo la resuelve el sistema y los casos de prueba que la verifican.

Audiencia

El tribunal evaluador y los futuros mantenedores. Se asume conocimiento de las convenciones del proyecto y de UML.

8.1 Descripción del Tema

EP-07 · Gestión y Administración agrupa lo que el equipo de Movo necesita para operar la plataforma y resolver problemas: un **panel web de administración** con acceso protegido por un segundo factor, la **ficha completa de cada usuario**, el **monitoreo de envíos** (incluido el seguimiento en el mapa), las **sanciones de cuenta** (suspensión e inhabilitación), las **disputas**, el **soporte** dentro de la app y el **cumplimiento legal** (Términos, Privacidad y Ley 25.326).

Toda acción de escritura del panel pasa por `movo-svc-admin`, que aplica la regla de conflicto de interés y deja una bitácora inmutable. Es el tema con más historias todavía abiertas: la resolución de disputas con reembolsos, las métricas de oferta y demanda, la corrección manual del estado de un envío y la API de la bandeja de soporte para agentes se documentan aquí con su estado real y casos de prueba marcados como pendientes, en lugar de omitirse.

Servicios involucrados: `movo-admin` (panel Next.js), `movo-svc-admin`, `movo-svc-users`, `movo-svc-shipments`, `movo-svc-payments`, `gateway` y `movo-mobile`.

8.2 Historias de Usuario y Trazabilidad

Tabla 8.1. Historias de usuario del tema.

Historia	Descripción	Estado	Sprint
MOVO-286	Acceso al panel con segundo factor	Completada	Sprint 7
MOVO-293	Búsqueda de usuarios y ficha completa	Completada	Sprint 8
MOVO-36	Suspensión e inhabilitación de cuentas	En curso	Sprint 8
MOVO-33	Monitoreo de envíos desde el panel	Completada	Sprint 8
MOVO-30	Disputa sobre un envío	Backlog	Sprint 8
MOVO-34	Gestión y resolución de disputas por el administrador	Backlog	sin asignar
MOVO-35	Métricas de oferta y demanda	Backlog	sin asignar
MOVO-319	Soporte dentro de la aplicación	Backlog	Sprint 8
MOVO-329	Corrección manual del estado de un envío	Backlog	Sprint 8
MOVO-224	Legal y compliance (Términos, Privacidad y Ley 25.326)	Completada	Sprint 5, Sprint 7, Sprint 8

8.2.1 MOVO-286 · Acceso al panel con segundo factor

MOVO-286 COMPLETADA SPRINT 7 10 SP

Como administrador, quiero ingresar al panel con un segundo factor para operar sobre la plataforma de forma segura

> **Decisión de diseño.** El panel tiene su propia sesión, separada de la de la app: el ingreso exige contraseña más un código de un solo uso por email, el access token dura 15 minutos con una sesión máxima de 8 horas, y un token de la app nunca lleva el rol `admin` aunque el usuario lo tenga. Toda escritura desde el panel pasa por `movo-svc-admin`, que aplica la regla de conflicto de interés y deja una bitácora de solo agregar filas.

Criterios de aceptación

1. Un usuario con rol `admin`, email verificado y cuenta activa entra al panel con email, contraseña y un código OTP recibido por email.
2. Un usuario sin rol `admin`, con la cuenta suspendida o con credenciales inválidas no entra, y el error no revela cuál de las condiciones falló.
3. Un token emitido por el login de la app nunca lleva el rol `admin`, aunque el usuario lo tenga.

4. La sesión del panel vence a las 8 h como máximo, sin importar la actividad.
5. Revocar el rol `admin` o suspender la cuenta corta las sesiones del panel de inmediato.
6. Cualquier acción de escritura desde el panel queda en la bitácora y se rechaza si el admin es parte de lo que toca.
7. Existe al menos un admin en dev, creado con el script.
8. Un admin puede listar, otorgar y revocar el rol `admin` desde el panel, sin dejar el sistema sin admins.

Trazabilidad de implementación

Tabla 8.2. Tickets de implementación de MOVO-286.

Ticket	Alcance	Sprint	Estado
`MOVO-287`	[svc-users] Sesión de panel: login con email + OTP y rol admin solo en esa sesión	Sprint 7	Completada
`MOVO-288`	[gateway][svc-admin] Base de svc-admin: ruta /admin, x-user-*, Prisma, bitácora y regla de conflicto de interés	Sprint 7	Completada
`MOVO-290`	[svc-users] Alta y baja del rol admin: script CLI y endpoints internos	Sprint 7	Completada
`MOVO-291`	[movo-admin] Setup del panel: layout, login con OTP, sesión en cookies httpOnly y rutas protegidas	Sprint 7	Completada
`MOVO-292`	[movo-admin][svc-admin] Gestión de admins desde el panel: listar, otorgar y revocar el rol	Sprint 7	Completada

Diagrama de secuencia

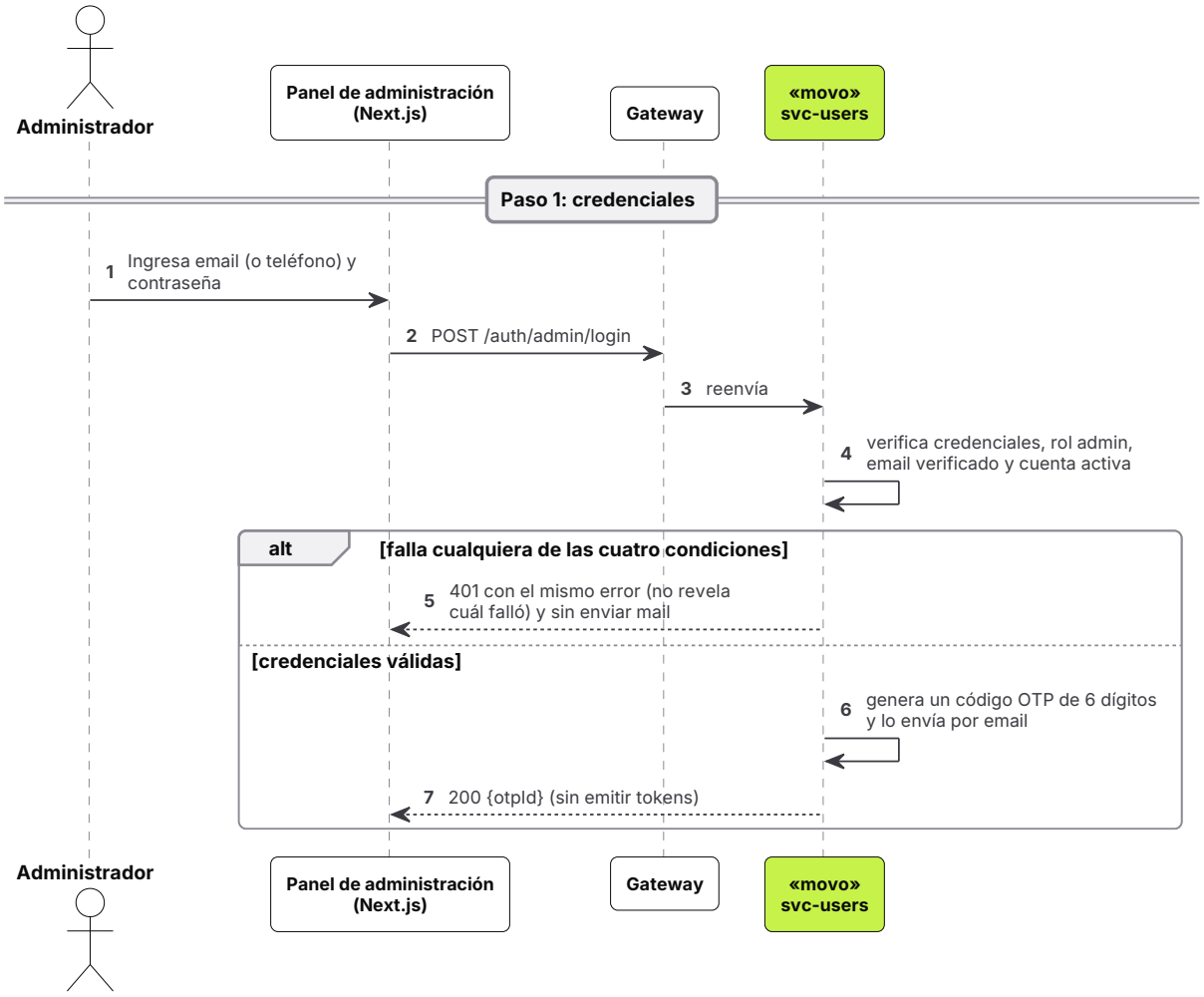


Figura 8.1. Ingreso al panel con OTP por email, sesión de 8 horas y bitácora de acciones (MOVO-286, MOVO-287 a MOVO-292). Parte 1 de 3: Paso 1: credenciales.

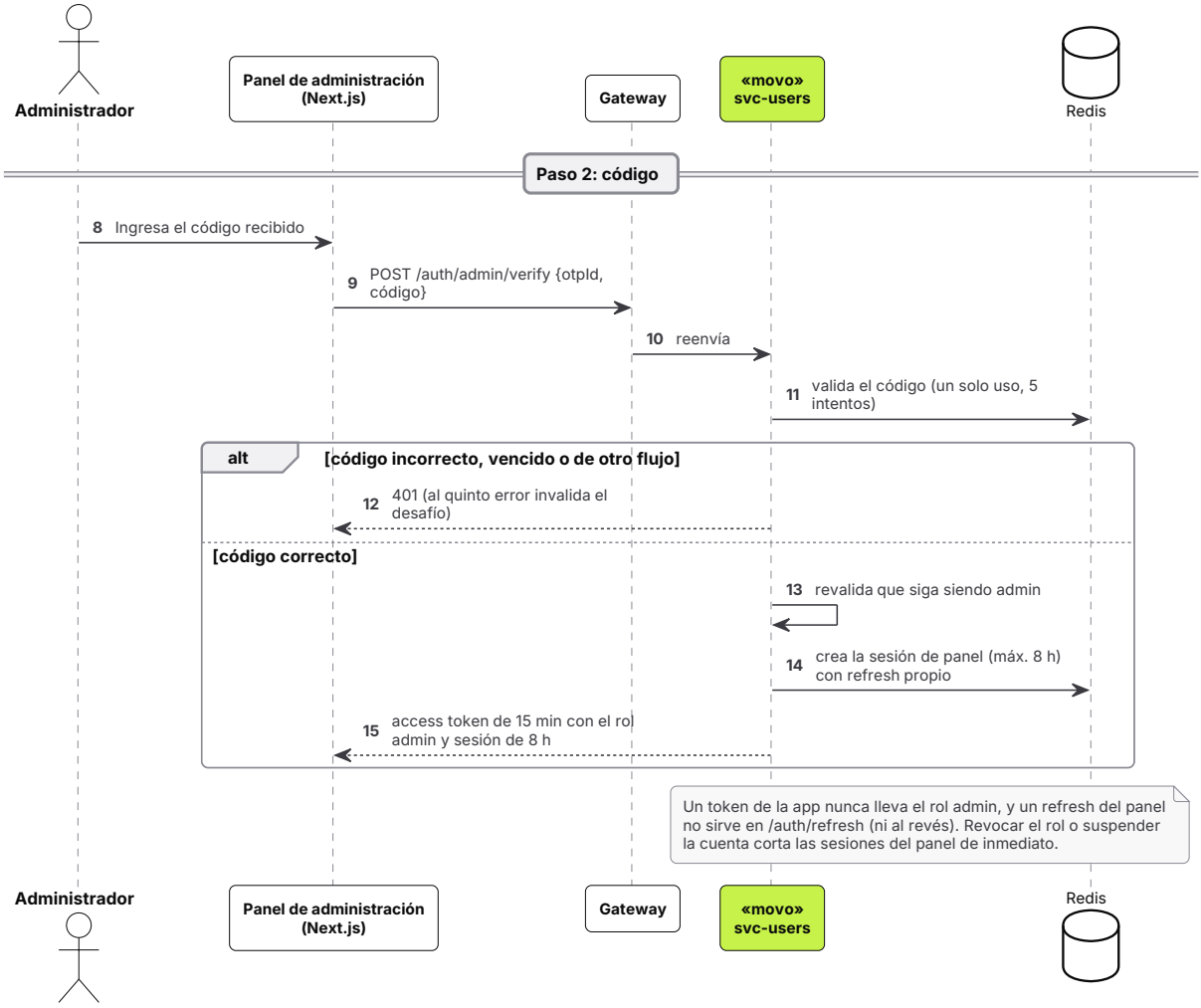


Figura 8.2. Ingreso al panel con OTP por email, sesión de 8 horas y bitácora de acciones (MOVO-286, MOVO-287 a MOVO-292). Parte 2 de 3: Paso 2: código.

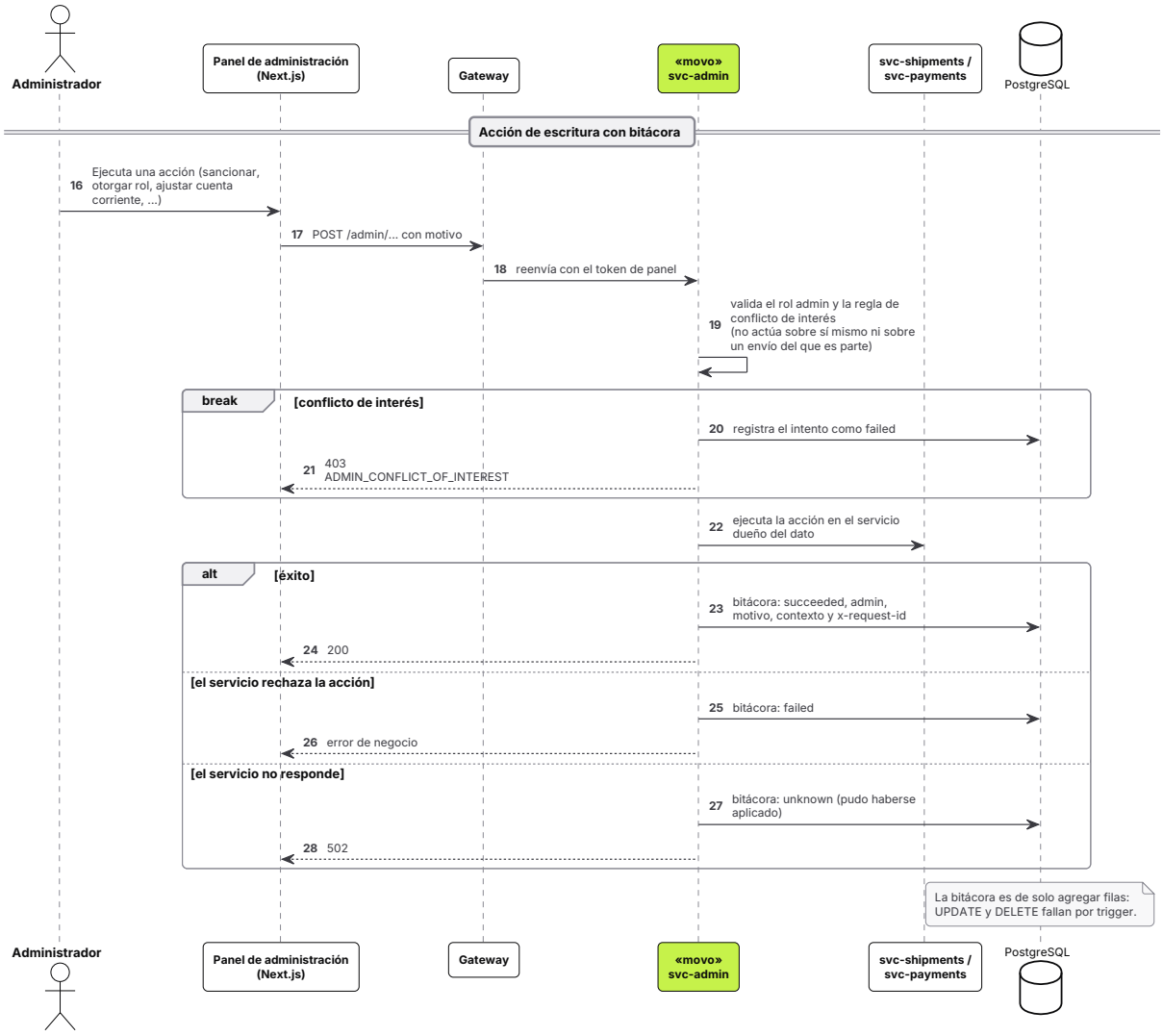


Figura 8.3. Ingreso al panel con OTP por email, sesión de 8 horas y bitácora de acciones (MOVO-286, MOVO-287 a MOVO-292). Parte 3 de 3: Acción de escritura con bitácora.

Casos de prueba

Tabla 8.3. Casos de prueba de MOVO-286 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-286.01	1	**Paso 1: credenciales y código por email** *Pre:* Usuario con rol admin, email verificado y cuenta activa. *Pasos:* POST /auth/admin/login con email o con teléfono.	El código sale por email y nunca se emiten tokens; el teléfono (formato de la app) también se acepta.	`auth.admin-session.integration.test.ts`	APROBADO
CP-286.02	2	**Rechazo sin revelar la causa** *Pre:* Usuario sin rol admin, con cuenta suspendida, con credenciales inválidas o con teléfono desconocido. *Pasos:* POST /auth/admin/login en cada caso.	Las cuatro causas responden el mismo 401 y no envían ningún mail; un body sin email ni teléfono se rechaza.	`auth.admin-session.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-286.03	1, 3, 4	**Paso 2: sesión del panel** *Pre:* Código válido recibido. *Pasos:* POST /auth/admin/verify con el código.	Se emite un access token de 15 minutos con el rol admin y una sesión de 8 horas; el mismo admin entra a la app sin el rol admin; el código es de un solo uso.	`auth.admin-session.integration.test.ts`	APROBADO
CP-286.04	2	**Código incorrecto o de otro flujo** *Pre:* Desafío vigente. *Pasos:* Verificar con un código erróneo hasta cinco veces, con un otpId ajeno, o tras perder el rol entre los dos pasos.	401; al quinto error el desafío se invalida; un otpId de otro flujo no abre sesión; si pierde el rol, el 401 es genérico.	`auth.admin-session.integration.test.ts`	APROBADO
CP-286.05	4	**Refresh separado y con rotación** *Pre:* Sesión de panel y sesión de app. *Pasos:* Refrescar cada una en la ruta contraria; refrescos concurrentes; cerrar sesión.	Un refresh de panel no sirve en /auth/refresh ni uno de la app en /auth/admin/refresh; las solicitudes concurrentes comparten una sola rotación y la revocación impide devolver el resultado compartido.	`auth.admin-session.integration.test.ts`	APROBADO
CP-286.06	3	**Confirmación de acceso desde svc-admin** *Pre:* Pedidos con y sin rol admin. *Pasos:* Consultar la sesión en svc-admin.	Sin identidad, 401; sin rol admin, 403; con rol admin confirma el acceso sin tokens ni caché.	`session.test.ts`	APROBADO
CP-286.07	6	**Bitácora de acciones** *Pre:* Acciones del admin con resultados distintos. *Pasos:* Ejecutar acciones exitosas, rechazadas, con error inesperado y con el servicio dueño caído.	Quedan registradas como succeeded, failed o unknown (nunca succeeded si el servicio no respondió); la tabla solo admite agregar filas (UPDATE y DELETE fallan); sin identidad, 401.	`audit.integration.test.ts`	APROBADO
CP-286.08	6	**Regla de conflicto de interés** *Pre:* Admin que actúa sobre sí mismo, sobre su rol o sobre un envío del que es parte. *Pasos:* Intentar la acción.	No ejecuta la acción y deja el intento como failed (403 ADMIN_CONFLICT_OF_INTEREST); sobre otro usuario o un envío ajeno sí puede; si svc-shipments no responde, falla cerrado con 502.	`conflict-of-interest.test.ts` `audit.integration.test.ts`	APROBADO
CP-286.09	8	**Gestión de administradores** *Pre:* Admin autenticado. *Pasos:* Listar, otorgar y revocar el rol admin; revocarse a sí mismo; revocar al último admin.	Se listan los admins y se autocompletan candidatos; revocar exige motivo; revocarse es 403; no se deja al sistema sin admins; otorgar y revocar dejan filas en la bitácora con el x-request-id del gateway.	`admins.integration.test.ts` `admins.integration.test.ts`	APROBADO

8.2.2 MOVO-293 · Búsqueda de usuarios y ficha completa

MOVO-293 COMPLETADA SPRINT 8

Como administrador, quiero buscar un usuario y ver su ficha completa, para entender su actividad antes de tomar una decisión sobre su cuenta

Criterios de aceptación

1. **Búsqueda** por nombre, email, teléfono o DNI, con resultados paginados: nombre, email, teléfono, estado de la cuenta, KYC de identidad y fecha de alta. Las cuentas dadas de baja (`deleted`) se pueden encontrar, marcadas como tales.
2. **Ficha** con secciones:
3. **Datos:** nombre, email y teléfono (con su estado de verificación), DNI, fecha de nacimiento, fecha de alta, roles y estado de la cuenta.
4. **Verificación:** KYC de identidad y de licencia, con el historial de intentos (sin imágenes biométricas, mismo criterio que `MOVO-32`). Estado de la cuenta de Mercado Pago (vinculada o no).
5. **Actividad:** envíos donde participa (como emisor, transportista o receptor), con estado y link al detalle; viajes; calificaciones recibidas y su promedio.
6. **Moderación:** reportes recibidos y hechos, y bloqueos en ambas direcciones.
7. **Sanciones:** historial (se completa cuando existan, en `MOVO-36`).
8. **Bitácora:** acciones de admins sobre este usuario.
9. Cada sección se carga por separado: si un servicio no responde, esa sección muestra el error y el resto de la ficha se ve igual.
10. Ver una ficha no queda en la bitácora (es lectura; la regla de conflicto no aplica).

Trazabilidad de implementación

Tabla 8.4. Tickets de implementación de MOVO-293.

Ticket	Alcance	Sprint	Estado
`MOVO-294`	[svc-admin][svc-users][svc-shipments][svc-payments] Búsqueda de usuarios y ficha: endpoints de admin e internos	Sprint 8	Completada
`MOVO-295`	[movo-admin] Pantallas de búsqueda de usuarios y ficha	Sprint 8	Completada

Casos de prueba

Tabla 8.5. Casos de prueba de MOVO-293 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-293.01	1	**Búsqueda y paginación** *Pre:* Cuentas con distintos datos, incluidas las dadas de baja. *Pasos:* Buscar por nombre, email, DNI y teléfono.	Sin consulta lista todas, las más recientes primero; la búsqueda no distingue tildes ni mayúsculas, acepta DNI con o sin puntos desde 3 dígitos y teléfonos en los formatos del login; pagina y devuelve el total.	`admin-users.integration.test.ts` `users.integration.test.ts`	APROBADO
CP-293.02	2, 3, 4, 5	**Ficha completa** *Pre:* Usuario con actividad. *Pasos:* GET /admin/users/:id y sus secciones (kyc, shipments, trips, ratings, ledger, devices, moderation).	La ficha devuelve el perfil completo (también de una cuenta dada de baja); KYC suma la tarjeta de Mercado Pago; los envíos y calificaciones traen el nombre de las partes; la cuenta corriente trae el saldo total y el nombre del admin que ajustó.	`users.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-293.03	9, 10	**Secciones independientes y lectura sin bitácora** *Pre:* Un servicio caído. *Pasos:* Consultar la ficha con svc-shipments o svc-payments caído.	La ficha responde igual: con svc-shipments caído las secciones de envíos, viajes y calificaciones son 502; con svc-payments caído la cuenta corriente es 502 y KYC responde sin la tarjeta; las lecturas no dejan filas en la bitácora.	`users.integration.test.ts`	APROBADO
CP-293.04	–	**Interfaz del panel** *Pre:* Ficha cargada en el panel. *Pasos:* Renderizar buscador, encabezado, actividad, listas largas, notas y acciones de cuenta.	Cada sección carga por separado y muestra su propio error sin romper el resto.	`user-search.test.tsx` `user-header.test.tsx` `section-error.test.tsx` `activity-panel.test.tsx`	APROBADO

8.2.3 MOVO-36 · Suspensión e inhabilitación de cuentas

MOVO-36 EN CURSO SPRINT 8 11 SP

Como administrador, quiero suspender o inhabilitar una cuenta de usuario ante comportamiento fraudulento o violación de términos, para proteger la integridad de la red.

Estado. Historia en curso: el backend (sanciones en `svc-users`, efectos en `svc-shipments`, bloqueo en el gateway) y la experiencia móvil están terminados; falta la pantalla del panel para aplicar y levantar sanciones (MOVO-299). La sanción se aplica contra la API interna de `svc-users`. **Decisión de diseño:** el estado de la cuenta vive en una marca en Redis que el gateway consulta en cada request, de modo que el efecto es inmediato aun con un token ya emitido; el historial en Postgres es de solo agregar filas.

Criterios de aceptación

1. Desde la ficha del usuario en el panel, un admin puede suspender (con fecha de fin o indefinida) o inhabilitar una cuenta. El motivo es obligatorio. Opcionalmente puede agregar una nota interna, que el usuario no ve.
2. Un usuario suspendido entra a la app, ve sus envíos, viajes y chats, y puede completar lo que tiene en curso. Cualquier acción que inicia algo nuevo responde `403 ACCOUNT_SUSPENDED` y la app explica por qué y hasta cuándo.
3. El efecto es inmediato: ni un token ya emitido permite iniciar algo nuevo después de suspender.
4. Al suspender se cancelan sus publicaciones y viajes `declared` sin paquetes, y se retiran sus ofertas pendientes.
5. Una cuenta inhabilitada no puede loguearse ni usar un token ya emitido. Inhabilitar con envíos en curso responde 409 con la lista.

6. El usuario sancionado recibe push y email con el motivo y el plazo. Las contrapartes de sus envíos en curso reciben un aviso sin el motivo.
7. El perfil público muestra el estado de la cuenta, sin el motivo.
8. Una suspensión temporal vence sola en la fecha de fin, y el usuario recibe un aviso.
9. Un admin puede levantar una suspensión o una inhabilitación antes de tiempo, con motivo. El usuario recibe un aviso.
10. Cada sanción queda en el historial del usuario: tipo, motivo, nota interna, inicio, fin, admin que la aplicó, y si se levantó, quién, cuándo y por qué. La acción también queda en la bitácora de `svc-admin`.
11. Un admin no puede sancionar ni levantar su propia sanción (`403 ADMIN_CONFLICT_OF_INTEREST`). Sancionar a otro admin le corta las sesiones del panel.

Trazabilidad de implementación

Tabla 8.6. Tickets de implementación de MOVO-36.

Ticket	Alcance	Sprint	Estado
`MOVO-296`	[svc-users][shared] Sanciones de cuenta: suspensión e inhabilitación, historial, marca en Redis y vencimiento automático	Sprint 8	Completada
`MOVO-297`	[svc-shipments] Efectos de una sanción: retirar lo no comprometido, avisar a contrapartes y listar envíos en curso	Sprint 8	Completada
`MOVO-298`	[gateway] Aplicar suspensión e inhabilitación en cada request a partir de la marca en Redis	Sprint 8	Completada
`MOVO-299`	[svc-admin][movo-admin] Suspender, inhabilitar y levantar sanciones desde la ficha del usuario	Sprint 8	En curso
`MOVO-300`	[movo-mobile] Experiencia de una cuenta suspendida o inhabilitada y estado en el perfil público	Sprint 8	Completada

Diagrama de secuencia

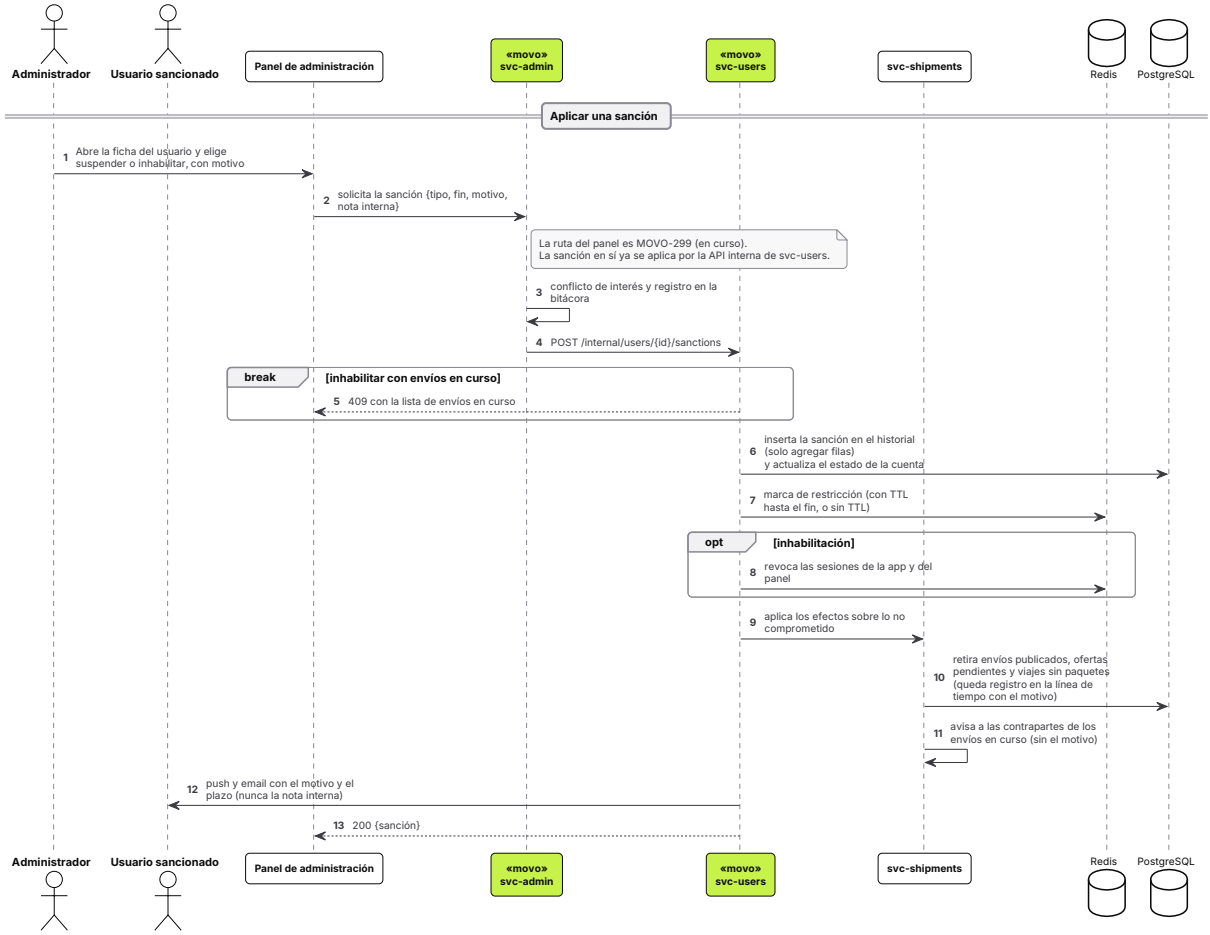


Figura 8.4. Aplicación, efecto inmediato y levantamiento de una sanción de cuenta (MOVO-36, MOVO-296 a MOVO-300). Parte 1 de 3: Aplicar una sanción.

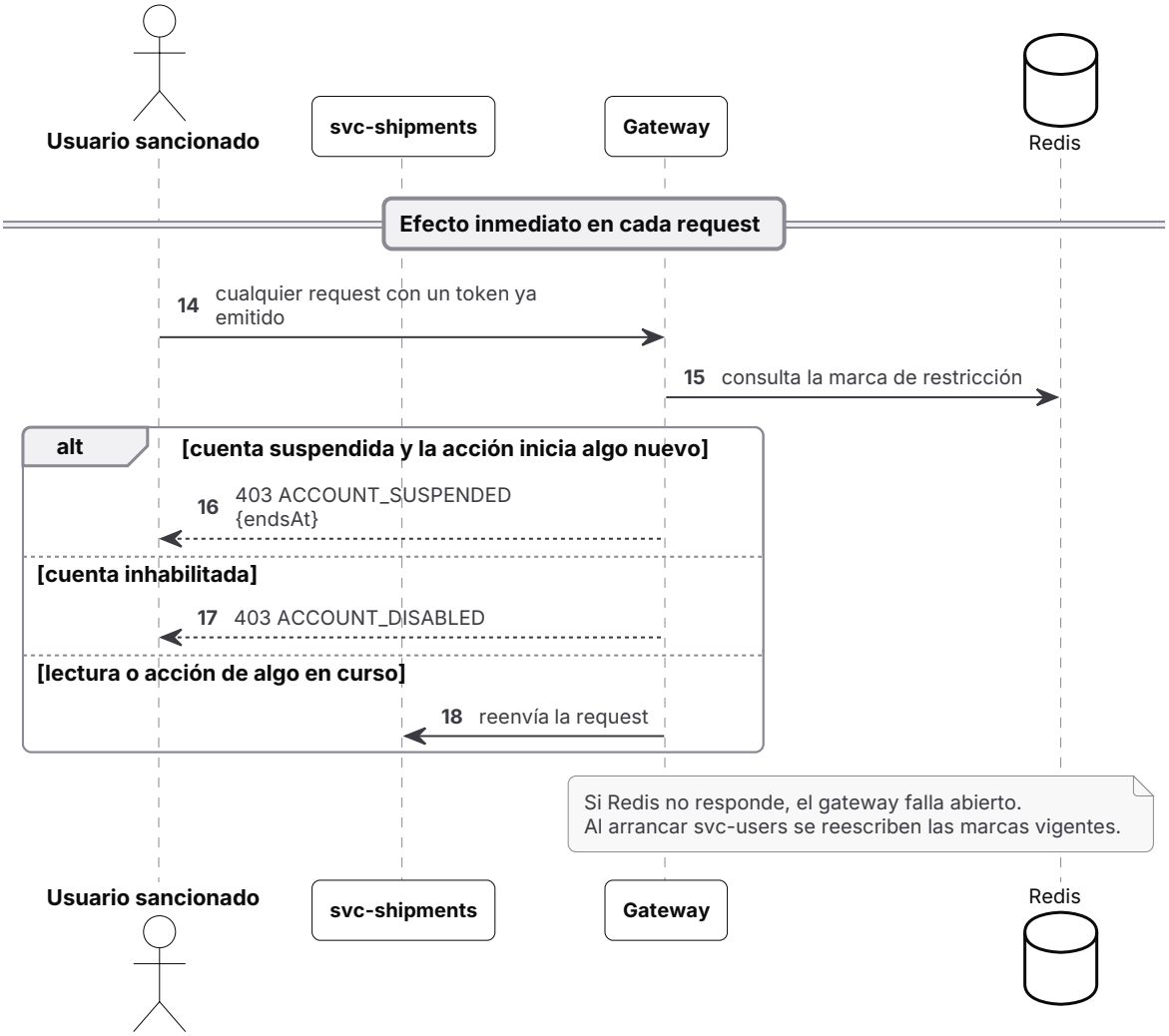


Figura 8.5. Aplicación, efecto inmediato y levantamiento de una sanción de cuenta (MOVO-36, MOVO-296 a MOVO-300). Parte 2 de 3: Efecto inmediato en cada request.

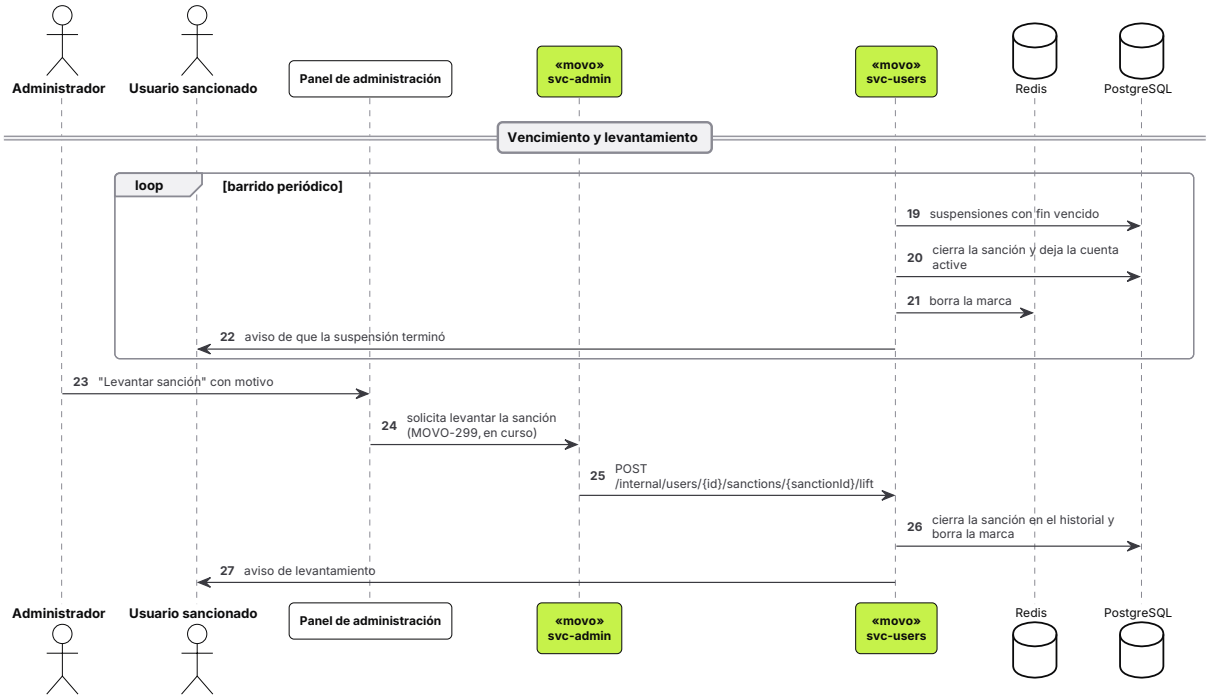


Figura 8.6. Aplicación, efecto inmediato y levantamiento de una sanción de cuenta (MOVO-36, MOVO-296 a MOVO-300). Parte 3 de 3: Vencimiento y levantamiento.

Casos de prueba

Tabla 8.7. Casos de prueba de MOVO-36 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-36.01	1, 10	**Suspender con y sin fecha de fin** *Pre:* Usuario activo. *Pasos:* POST /internal/users/:id/sanctions con tipo suspension, con y sin fin.	Queda la fila en el historial, la caché en el usuario y la marca en Redis con TTL hasta el fin (o sin TTL si es indefinida).	`sanctions.integration.test.ts`	APROBADO
CP-36.02	5	**Inhabilitar** *Pre:* Usuario activo o suspendido. *Pasos:* Inhabilitar; inhabilitar a un suspendido; inhabilitar a un inhabilitado.	La cuenta pasa a banned con marca sin TTL y se revocan las sesiones de la app y del panel; inhabilitar a un suspendido escala la sanción; sobre un inhabilitado, 409.	`sanctions.integration.test.ts`	APROBADO
CP-36.03	1	**Validación de la sanción** *Pre:* Pedido incompleto. *Pasos:* Sancionar sin motivo, con fin en el pasado o con fin en una inhabilitación.	400 por motivo obligatorio, fin futuro requerido y la inhabilitación no lleva fin; suspender a alguien ya suspendido reemplaza la sanción vigente y cierra la anterior.	`sanctions.integration.test.ts`	APROBADO
CP-36.04	2, 3, 5	**Efecto inmediato en el gateway** *Pre:* Cuenta suspendida o inhabilitada con un token ya emitido. *Pasos:* Hacer una request que inicia una acción nueva, y otra de lectura.	403 ACCOUNT_SUSPENDED (con endsAt) o ACCOUNT_DISABLED en las acciones que inician algo; las lecturas pasan; una marca vencida por TTL deja de bloquear; si Redis no responde, el gateway falla abierto.	`account-restriction.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-36.05	2, 5	**Login y refresh de una cuenta sancionada** *Pre:* Cuenta suspendida y cuenta inhabilitada. *Pasos:* Iniciar sesión y refrescar.	Un suspendido puede loguearse y refrescar; un inhabilitado no. Una suspensión vencida que el barrido no cerró se trata como activa en cualquier lectura.	`sanctions.integration.test.ts`	APROBADO
CP-36.06	4, 6	**Efectos sobre los envíos** *Pre:* Usuario con publicaciones, ofertas, viajes y envíos en curso. *Pasos:* Aplicar la sanción.	Se retiran lo publicado, las ofertas y los viajes sin paquetes y se cancelan los envíos que esperan al receptor; lo comprometido o en tránsito no se toca; las contrapartes de los envíos en curso reciben un aviso genérico sin el motivo; es idempotente.	`sanction-effects.integration.test.ts`	APROBADO
CP-36.07	6, 8, 9	**Avisos, vencimiento y levantamiento** *Pre:* Sanción vigente. *Pasos:* Aplicar, vencer y levantar.	Se avisa por push y email con el motivo y el plazo (nunca la nota interna); una suspensión vencida la levanta el barrido con aviso; levantar una sanción la cierra con motivo y deja la cuenta activa; levantar una inexistente o ajena es 404.	`sanctions.integration.test.ts`	APROBADO
CP-36.08	11	**Sanciones a administradores** *Pre:* Un admin como objetivo. *Pasos:* Sancionar a un admin y al último admin elegible.	Se le cortan las sesiones de panel; no se puede sancionar al último admin; la marca se reescribe si Redis perdió datos y al arrancar el servicio.	`sanctions.integration.test.ts`	APROBADO
CP-36.09	-	**Pantalla del panel** *Pre:* Panel con la ficha de un usuario. *Pasos:* Renderizar las acciones de cuenta.	Muestra las acciones de suspender, inhabilitar y levantar con motivo obligatorio.	`account-actions.test.tsx` `account-actions.test.ts`	PENDIENTE

8.2.4 MOVO-33 · Monitoreo de envíos desde el panel

MOVO-33 COMPLETADA SPRINT 8 15 SP

Como administrador, quiero consultar los envíos de la plataforma con su estado y el detalle de cada uno, para monitorear la operación y detectar incidentes.

Trazabilidad de implementación

Tabla 8.8. Tickets de implementación de MOVO-33.

Ticket	Alcance	Sprint	Estado
MOVO-325	[svc-shipments][svc-admin] Envíos para el panel: listado con filtros, detalle completo y chat en solo lectura	Sprint 8	Completada

Ticket	Alcance	Sprint	Estado
MOVO-326	[movo-admin] Listado y detalle de envíos (prototipo “Movo Admin Envíos”): pestañas, tabla, línea de tiempo y chat en solo lectura	Sprint 8	Completada
MOVO-327	[svc-shipments][svc-admin] Seguimiento de un envío en camino para el panel: última posición, traza y métricas	Sprint 8	Completada
MOVO-328	[movo-admin] Mapa de seguimiento del envío en el detalle: ruta recorrida, posición del transportista y métricas	Sprint 8	Completada

Casos de prueba

Tabla 8.9. Casos de prueba de MOVO-33 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-33.01	-	**Listado con filtros** *Pre:* Envíos en distintos estados y ciudades. *Pasos:* GET /admin/shipments con pestañas, búsqueda y fechas.	Por defecto muestra los activos; cuenta cada pestaña con los mismos filtros; busca por código, nombre de cualquier parte o dirección sin tildes ni mayúsculas; filtra por ciudad del retiro y por rango del último cambio de estado; ordena y pagina con cursor sin repetir ni saltear (cursor inválido: 400).	admin-shipments.integration.test.ts	APROBADO
CP-33.02	-	**Detalle y chat del envío** *Pre:* Envío con chat. *Pasos:* Consultar el detalle y el chat desde el panel.	El detalle trae partes, ciudades, precio, avisos y fechas límite; el administrador lee el chat aun cancelado y nunca escribe.	shipments.integration.test.ts chat.integration.test.ts	APROBADO
CP-33.03	-	**Seguimiento en el mapa** *Pre:* Envío en camino. *Pasos:* Consultar el seguimiento desde el panel.	Devuelve la traza submuestreada, proyecta la posición sobre la línea de la ruta (el progreso se mide por el largo acumulado, no por la recta al destino) y estima la velocidad con los puntos de los últimos 5 minutos.	admin-tracking.test.ts admin-tracking.service.test.ts tracking-map.test.tsx	APROBADO
CP-33.04	-	**Interfaz del listado y del detalle** *Pre:* Panel con datos. *Pasos:* Renderizar listado, detalle, panel de chat y de seguimiento.	Se muestran pestañas, tabla, línea de tiempo y mapa con sus estados de carga y error.	shipments-list.test.tsx shipment-detail.test.tsx chat-panel.test.tsx tracking-panel.test.tsx	APROBADO

8.2.5 MOVO-30 · Disputa sobre un envío

MOVO-30 BACKLOG SPRINT 8

Como parte de un envío, quiero iniciar una disputa sobre un envío finalizado o en curso, para reclamar formalmente ante un problema con el paquete o con la entrega.

Estado. Historia en *Backlog* en Linear: el backend (MOVO-334: entidad, apertura, descargos, notificaciones y bloqueo del transportista) está terminado, pero falta la experiencia móvil (MOVO-335). **Decisión de diseño (DD-054):** una disputa se abre sobre un envío `completed` o `delivered` dentro de los 7 días desde la entrega y no toca el hold, que ya fue capturado. La resolución por el administrador y los reembolsos están en backlog (ver MOVO-34).

Trazabilidad de implementación

Tabla 8.10. Tickets de implementación de MOVO-30.

Ticket	Alcance	Sprint	Estado
`MOVO-334`	[svc-shipments] Disputas: entidad, apertura, descargos, notificaciones y bloqueo del transportista	Sprint 8	Completada
`MOVO-335`	[movo-mobile] Disputas: botón de reclamo, wizard de apertura, detalle con descargo y pill “En disputa”	–	Backlog

Diagrama de secuencia

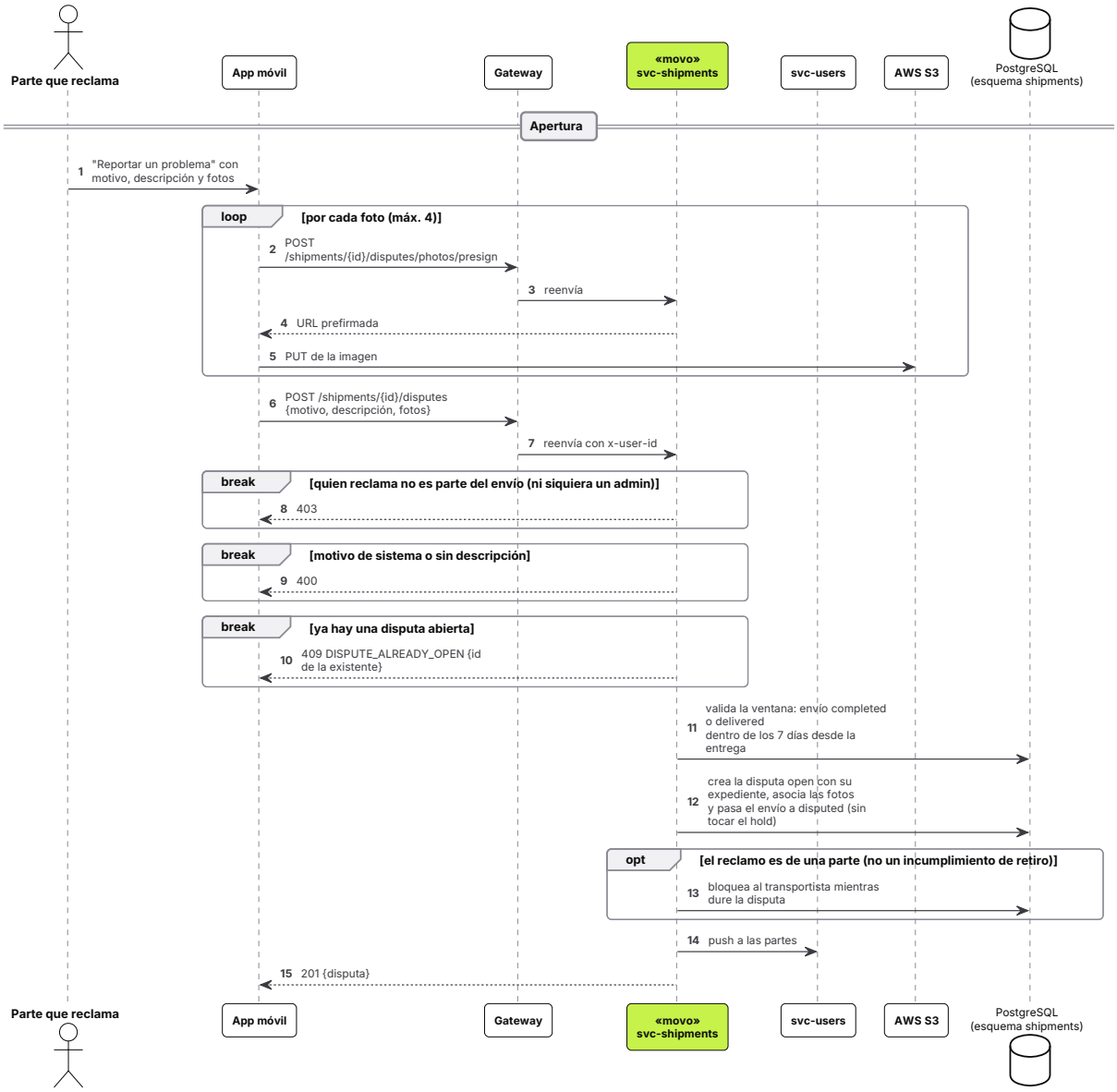


Figura 8.7. Apertura de una disputa con fotos y carga de descargos (MOVO-30, MOVO-334). Parte 1 de 2: Apertura.

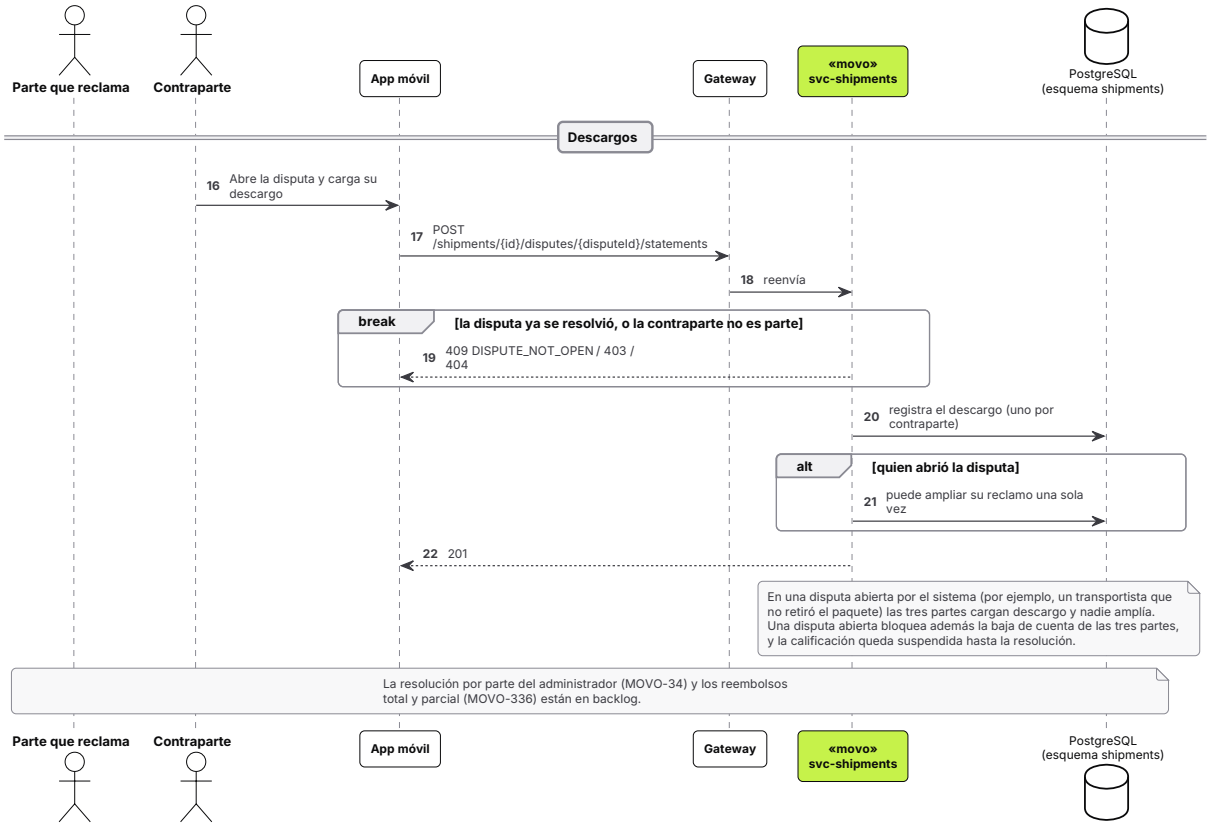


Figura 8.8. Apertura de una disputa con fotos y carga de descargos (MOVO-30, MOVO-334). Parte 2 de 2: Descargos.

Casos de prueba

Tabla 8.11. Casos de prueba de MOVO-30 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-30.01	-	**Apertura dentro de la ventana** *Pre:* Envío completed o delivered dentro de los 7 días. *Pasos:* POST /shipments/:id/disputes con motivo y descripción.	201 con la disputa abierta; el envío pasa a disputed sin tocar el hold. Quien no es parte (aunque sea admin) recibe 403.	`disputes.integration.test.ts`	APROBADO
CP-30.02	-	**Validaciones de la apertura** *Pre:* Disputa existente o datos inválidos. *Pasos:* Abrir con motivo de sistema, sin descripción, o con otra disputa abierta.	400 con motivo de sistema o sin descripción; 409 DISPUTE_ALREADY_OPEN con el id de la existente; pasar a disputed sin expediente no es posible.	`disputes.integration.test.ts`	APROBADO
CP-30.03	-	**Fotos del expediente** *Pre:* Fotos subidas, de otra parte o sin subir. *Pasos:* Presignar y adjuntar fotos a la disputa.	Las fotos se devuelven como URLs de lectura; 403 con la foto de otra parte, 422 si no se subió y 400 si pasa el tope de 4; solo las partes pueden presignar.	`disputes.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-30.04	-	**Descargos** *Pre:* Disputa abierta. *Pasos:* Cada contraparte carga un descargo; quien abrió amplía; intentos extra; disputa ya resuelta.	Cada contraparte carga un descargo y quien abrió amplía una vez, los extras son 409; en una disputa de sistema las tres partes cargan descargo y nadie amplía; 409 DISPUTE_NOT_OPEN si ya se resolvió.	`disputes.integration.test.ts`	APROBADO
CP-30.05	-	**Bloqueos asociados** *Pre:* Disputa abierta por una parte o por el sistema. *Pasos:* Consultar el estado del transportista y la baja de cuenta.	Una disputa de una parte bloquea al transportista (la de pickup_no_show no); toda disputa abierta bloquea la baja de cuenta de las tres partes.	`disputes.integration.test.ts`	APROBADO
CP-30.06	-	**Disputas de sistema por anomalías** *Pre:* Envío con anomalía de tránsito. *Pasos:* Correr el barrido de anomalías.	Crea la disputa de sistema de cada envío ya disputado, con el motivo según el evento.	`disputes.integration.test.ts` `transit-anomaly-dispute.integration.test.ts` `transit-anomaly-sweep.test.ts`	APROBADO

8.2.6 MOVO-34 · Gestión y resolución de disputas por el administrador

MOVO-34 BACKLOG SIN ASIGNAR

Como administrador, quiero gestionar las disputas abiertas para actuar como árbitro entre las partes

> **Estado.** Historia en *Backlog*: sus tres tickets (reembolsos total y parcial en `svc-payments`, matriz de resolución y bandeja de disputas del panel) no están implementados. Los casos de prueba quedan definidos a partir de los criterios de aceptación pendientes.

Criterios de aceptación

1. El detalle de la disputa resuelta muestra el resultado, el razonamiento del admin y qué pasa con la plata en lenguaje simple, según el rol (“Te devolvemos el pago”, “Se liberó la reserva”, “Se te descuenta de tu cuenta corriente”).
2. **CTA para quien ganó:** mensaje en la línea de “Ganaste la disputa, ahora contá cómo te fue con esta persona” con acceso directo a la calificación de `MOVO-153`. Se muestra en el detalle de la disputa y en el destino del push.
3. El push `dispute_resolved` navega al detalle de la disputa (`resolveNotificationRoute`).
4. Los reembolsos aparecen en el historial de pagos y cobros (`MOVO-304`) con su tipo propio.
5. El pill “En disputa” se reemplaza por el estado final del envío.
6. Tests RNTL de textos por resultado y rol, CTA solo para quien ganó y navegación del push.

Trazabilidad de implementación

Tabla 8.12. Tickets de implementación de MOVO-34.

Ticket	Alcance	Sprint	Estado
MOVO-336	[svc-payments] Reembolsos total y parcial de un pago capturado, con fallback a la cuenta corriente del transportista	–	Backlog
MOVO-337	[svc-admin][svc-shipments] Resolución de disputas: matriz de resolución, estado final del envío, desbloqueo y notificaciones	–	Backlog
MOVO-338	[movo-admin] Bandeja de disputas, expediente y pantalla de resolución	–	Backlog

Casos de prueba

Tabla 8.13. Casos de prueba de MOVO-34 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-34.01	1	**Detalle de la disputa resuelta por rol** *Pre:* Disputa resuelta. *Pasos:* Abrir el detalle como ganador y como perdedor.	Muestra resultado, razonamiento del admin y qué pasa con el dinero, en lenguaje simple según el rol.	–	PENDIENTE
CP-34.02	2, 3	**Invitación a calificar y navegación de la push** *Pre:* Disputa resuelta a favor de una parte. *Pasos:* Abrir el detalle y tocar la push dispute_resolved.	Solo quien ganó ve la invitación a calificar; la push navega al detalle de la disputa.	–	PENDIENTE
CP-34.03	4, 5	**Reembolsos y estado final** *Pre:* Reembolso total o parcial aprobado. *Pasos:* Resolver la disputa.	El reembolso aparece en el historial de pagos con su tipo propio y el estado «En disputa» se reemplaza por el estado final del envío.	–	PENDIENTE

8.2.7 MOVO-35 · Métricas de oferta y demanda

MOVO-35 BACKLOG SIN ASIGNAR

Como administrador, quiero ver métricas de oferta y demanda por ruta y región, para tomar decisiones operativas.

Estado. Historia en *Backlog*, sin tickets ni criterios de aceptación definidos. Queda fuera del alcance entregado. Hoy el motor de precios consume un contexto de demanda local (ofertas y transportistas por zona de retiro, tema Publicación), pero no hay una vista de métricas para el administrador.

Casos de prueba

Tabla 8.14. Casos de prueba de MOVO-35 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-35.01	-	**Métricas por ruta y región** *Pre:* Historia no implementada. *Pasos:* Previsto: el administrador consulta oferta y demanda por ruta y región.	Casos de prueba por definir al implementarse la historia.	–	PENDIENTE

8.2.8 MOVO-319 · Soporte dentro de la aplicación

MOVO-319 BACKLOG SPRINT 8 20 SP

Como usuario, quiero escribirle al equipo de soporte desde la app, con o sin un envío asociado, para resolver un problema sin salir de Movo.

> **Estado.** Historia en *Backlog* en Linear, aunque el lado del usuario está terminado: modelo y API de conversaciones con tiempo real (MOVO-320), adjuntos de imágenes (MOVO-321) y la experiencia móvil (MOVO-324), junto con el prototipo de la bandeja del panel (MOVO-323). Sigue en curso la API de la bandeja para agentes (MOVO-322), de modo que hoy el panel trabaja con datos de prototipo. Las notas internas del agente nunca llegan al usuario por ningún canal.

Criterios de aceptación

1. Un usuario autenticado puede abrir una consulta de soporte desde Perfil, elegir un tema y, opcionalmente, vincular un envío en el que sea parte (emisor, transportista o receptor).
2. Puede ver el listado de sus consultas con su estado (abierta, esperando al usuario, resuelta) y mensajes no leídos, y reabrir una resuelta escribiendo de nuevo.
3. Los mensajes se envían por HTTP y se reciben en tiempo real por WebSocket; el usuario recibe una push cuando un agente responde.
4. Se pueden adjuntar imágenes a los mensajes, de ambos lados.
5. Un agente (rol `ADMIN`) ve una bandeja con todas las consultas, filtros por estado y por asignación, y puede tomarla, asignarla, cambiar prioridad, etiquetar, vincular o desvincular un envío, resolver y reabrir.
6. Los agentes pueden dejar **notas internas** que el usuario nunca recibe (ni por API, ni por WebSocket, ni por push).
7. El agente ve el contexto del usuario (perfil, KYC, envíos, consultas anteriores) y puede usar respuestas rápidas.
8. El historial se conserva y es visible para los agentes (y como evidencia en una disputa, MOVO-34).

Trazabilidad de implementación

Tabla 8.15. Tickets de implementación de MOVO-319.

Ticket	Alcance	Sprint	Estado
MOVO-320	[svc-shipments] Conversaciones de soporte: modelo, API del usuario, tiempo real sobre el chat existente y ADR	Sprint 8	Completada
MOVO-321	[svc-shipments] Adjuntos de imágenes en los mensajes de soporte (S3 con presigned URLs)	Sprint 8	Completada
MOVO-322	[svc-admin][svc-shipments] API de la bandeja de soporte para agentes (rol ADMIN): asignar, priorizar, etiquetar, vincular, resolver, notas y contexto del usuario	Sprint 8	En curso
MOVO-323	[movo-admin] Bandeja de soporte (prototipo “Movo Admin Soporte”): lista, conversación y panel de contexto del usuario	Sprint 8	Completada
MOVO-324	[movo-mobile] Ayuda y soporte: nueva consulta (con o sin envío), mis consultas y conversación con el equipo de Movo	Sprint 8	Completada

Diagrama de secuencia

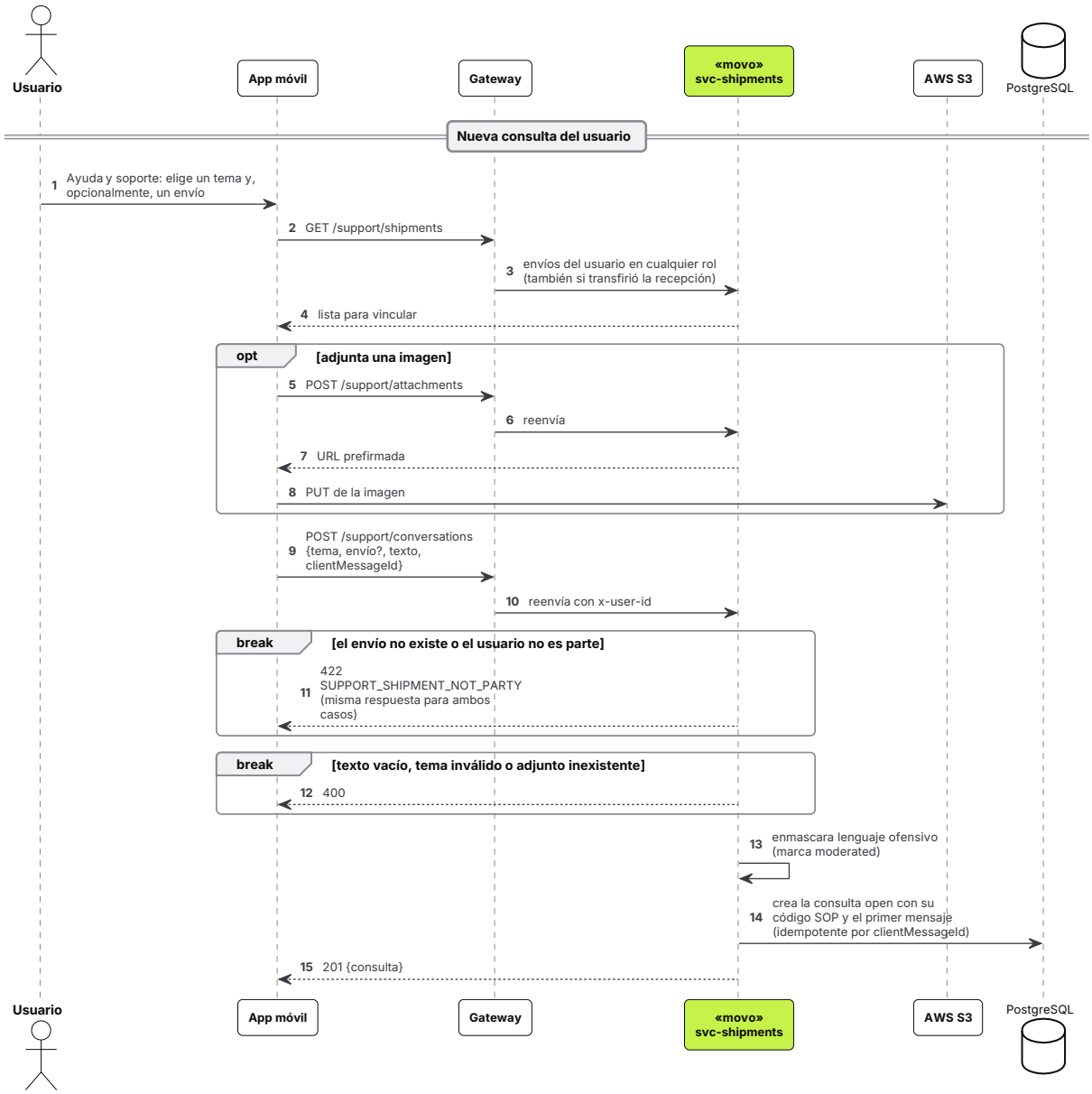


Figura 8.9. Consulta de soporte con o sin envío y respuesta del agente (MOVO-319, MOVO-320 a MOVO-324). Parte 1 de 3: Nueva consulta del usuario.

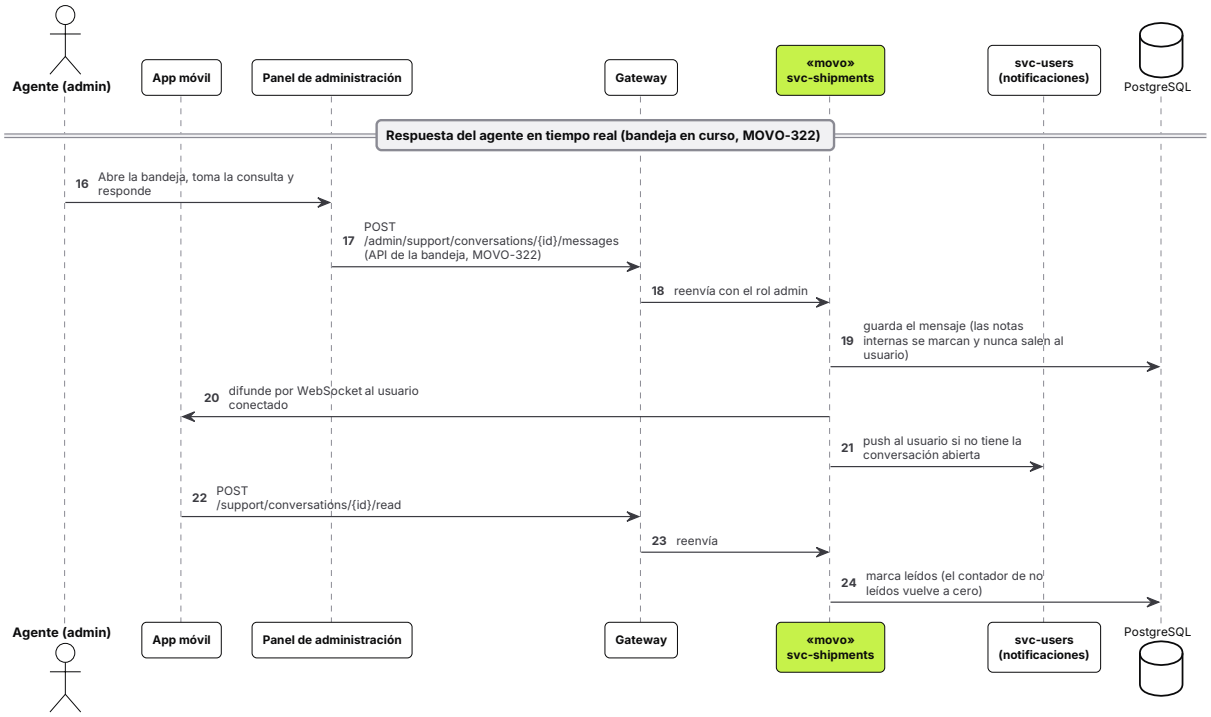


Figura 8.10. Consulta de soporte con o sin envío y respuesta del agente (MOVO-319, MOVO-320 a MOVO-324). Parte 2 de 3: Respuesta del agente en tiempo real (bandeja en curso, MOVO-322).

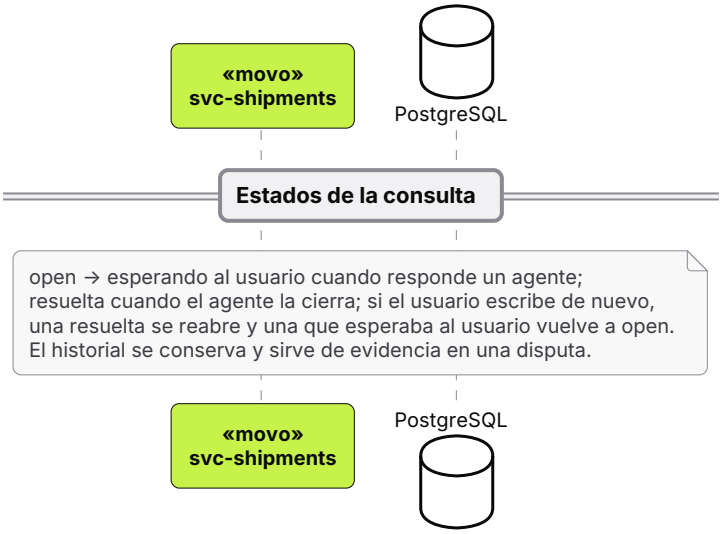


Figura 8.11. Consulta de soporte con o sin envío y respuesta del agente (MOVO-319, MOVO-320 a MOVO-324). Parte 3 de 3: Estados de la consulta.

Casos de prueba

Tabla 8.16. Casos de prueba de MOVO-319 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-319.01	1	**Crear una consulta con o sin envío** *Pre:* Usuario autenticado. *Pasos:* POST /support/conversations sin envío, y con un envío propio en cualquier rol.	201 con la consulta en open, su código SOP y el primer mensaje; con envío, se vincula el envío.	`support.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-319.02	1	**Envío ajeno o inexistente** *Pre:* Envío del que no es parte, inexistente, o un admin que no es parte. *Pasos:* Vincular el envío a la consulta.	422 SUPPORT_SHIPMENT_NOT_PARTY con la misma respuesta para ajeno e inexistente (no se filtra qué ids existen); ni un admin vincula un envío ajeno.	`support.integration.test.ts`	APROBADO
CP-319.03	1	**Idempotencia y validación** *Pre:* Reenvío del mismo clientMessageId, texto vacío, tema inválido o adjunto inexistente. *Pasos:* Crear la consulta en cada caso.	El reenvío da 200 con lo ya creado y dos pedidos simultáneos crean una sola consulta; el texto vacío, el tema inválido y el adjunto inexistente se rechazan; el lenguaje ofensivo se enmascara y se marca moderated.	`support.integration.test.ts`	APROBADO
CP-319.04	2	**Listado de consultas y no leídas** *Pre:* Consultas de varios usuarios con respuestas del equipo. *Pasos:* GET /support/conversations y marcar como leída.	Solo las propias, la de actividad más reciente primero, con el contador de respuestas del equipo no leídas que baja a cero al leer; sin consultas, lista vacía.	`support.integration.test.ts`	APROBADO
CP-319.05	2	**Estados y reapertura** *Pre:* Consulta resuelta, esperando al usuario o abierta. *Pasos:* El usuario escribe de nuevo.	Una resuelta se reabre, una que espera al usuario vuelve a open (sin contar como reapertura) y una abierta no cambia.	`support-rules.test.ts`	APROBADO
CP-319.06	1	**Envíos vinculables** *Pre:* Usuario con envíos en distintos roles, incluso terminales. *Pasos:* GET /support/shipments.	Trae los envíos en cualquier rol con código, estado, ruta y rol, incluido uno cuya recepción el usuario transfirió; no trae ajenos.	`support.integration.test.ts`	APROBADO
CP-319.07	4	**Adjuntos de imágenes** *Pre:* Imagen del usuario. *Pasos:* Presignar, subir y adjuntar a un mensaje.	Las imágenes viajan por URLs prefirmadas de S3, con las reglas de tipo y tamaño del servicio de adjuntos.	`support-attachments.integration.test.ts` `support-attachments.service.test.ts`	APROBADO

CP-319.08	5, 6, 7	**Bandeja del agente**
		Pre: Agente con rol ADMIN.
		Pasos: Operar la bandeja: tomar, asignar, cambiar prioridad, dejar notas internas, usar respuestas rápidas.
			API de la bandeja pendiente de implementación (MOVO-322); el prototipo del panel renderiza lista, conversación, composer y panel de contexto.	`conversation-list.test.tsx`
				`thread-view.test.tsx`
				`composer.test.tsx`
				`context-panel.test.tsx`
					APROBADO

8.2.9 MOVO-329 · Corrección manual del estado de un envío

MOVO-329 BACKLOG SPRINT 8 17 SP

Como administrador, quiero corregir el estado de un envío trabado o inconsistente, para destrabar casos que el flujo normal no resuelve.

> **Estado.** Historia en *Backlog*. El ticket MOVO-331 (endpoint interno de transición forzada) figura como terminado en Linear, pero no se encontró su código ni sus pruebas en la rama de integración usada para elaborar este documento, por lo que se lo trata como no verificado. Faltan el ADR con la matriz de transiciones y los riesgos (MOVO-330), la ruta de `svc-admin` con re-autenticación por OTP (MOVO-332) y el modal del panel (MOVO-333).

Criterios de aceptación

1. Un admin puede cambiar el estado de un envío desde el detalle, eligiendo entre las transiciones válidas y, aparte y con más fricción, las forzadas que permita el ADR.
2. Cada cambio exige motivo, y los de riesgo alto o crítico exigen además el código del envío tipeado; las forzadas y las críticas, un OTP nuevo.
3. El admin ve antes de confirmar qué se va a ejecutar, a quién se avisa y qué no se puede deshacer.
4. El cambio se ejecuta con los mismos efectos que la transición normal; si no hay efecto definido, el panel no ofrece ese destino.
5. Queda en la línea de tiempo del envío (visible para el admin, marcado como manual/forzado, con actor y motivo) y en la bitácora.
6. Un admin que es parte del envío no puede cambiarlo. Dos cambios simultáneos no se pisan.
7. Las partes reciben el aviso que defina el ADR.

Trazabilidad de implementación

Tabla 8.17. Tickets de implementación de MOVO-329.

Ticket	Alcance	Sprint	Estado
`MOVO-330`	[docs] ADR: cambio forzado de estado de un envío, matriz de transiciones, riesgos, efectos y bloqueos	Sprint 8	Por hacer
`MOVO-331`	[svc-shipments] Transición forzada de estado: endpoint interno que ejecuta los servicios de dominio, registra el evento y avisa a las partes	Sprint 8	Completada
`MOVO-332`	[svc-admin][svc-users] Cambio de estado desde el panel: ruta /admin, re-autenticación con OTP por acción, conflicto de interés y bitácora	Sprint 8	Por hacer
`MOVO-333`	[movo-admin] Modal de cambio de estado del envío (prototipo “Movo Admin Envíos”): elegir, ver impacto y confirmar	Sprint 8	Por hacer

Casos de prueba

Tabla 8.18. Casos de prueba de MOVO-329 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-329.01	4, 5, 6	**Transición forzada en svc-shipments** *Pre:* Envío trabado. *Pasos:* Ejecutar la transición forzada desde el endpoint interno.	Se ejecuta con los mismos efectos que la transición normal y queda en la línea de tiempo marcada como forzada.	`admin-internal.integration.test.ts`	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-329.02	1, 2, 3, 7	**Confirmaciones y re-autenticación del panel** *Pre:* Admin en el panel. *Pasos:* Elegir un cambio de estado de riesgo alto o crítico.	Exige motivo, código del envío tipeado y un OTP nuevo; muestra antes qué se ejecuta y a quién se avisa.	–	PENDIENTE

8.2.10 MOVO-224 · Legal y compliance (Términos, Privacidad y Ley 25.326)

MOVO-224 COMPLETADA SPRINT 5, SPRINT 7, SPRINT 8 5 SP

Legal y compliance: Términos y Condiciones, Política de Privacidad y protección de datos personales (Ley 25.326)

Estado. Historia terminada salvo dos tickets abiertos: la mecánica de penalidades (MOVO-226, ver el tema Pagos y Economía) y el checkbox de aceptación legal en el registro (MOVO-228, en revisión). Incluye la confirmación de la tasa de Mercado Pago para los Términos (MOVO-225), la re-aceptación de los Términos al abrir la app (MOVO-229) y el job de purga y anonimización de datos vencidos (MOVO-230). La historia no tiene criterios de aceptación en Linear.

Trazabilidad de implementación

Tabla 8.19. Tickets de implementación de MOVO-224.

Ticket	Alcance	Sprint	Estado
MOVO-225	Confirmar MP_TRANSACTION_FEE_RATE con Mercado Pago y reflejarlo en los Términos y Condiciones	Sprint 7	Completada
MOVO-226	Definir e implementar la mecánica de penalidad de cancelación con Transportista asignado (holds/liberación de pagos)	Sprint 8	En curso
MOVO-228	Checkbox obligatorio de aceptación legal en el registro + “firma electrónica” en Perfil → Legal	Sprint 8	En revisión
MOVO-229	Re-aceptación de Términos/Privacidad post-registro: gate al abrir la app + botón “Aceptar” en Perfil → Legal	Sprint 5	Completada
MOVO-230	Job automático de purga/anonimización de datos vencidos (Política de Privacidad §7)	Sprint 8	Completada

Casos de prueba

Tabla 8.20. Casos de prueba de MOVO-224 (derivados del código y de los criterios de aceptación).

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-224.01	–	**Aceptación de los Términos en el registro** *Pre:* Versiones vigentes de Términos y Privacidad. *Pasos:* Registrar con y sin las versiones legales vigentes.	El registro exige aceptar la versión vigente y la guarda con el usuario (ver MOVO-7).	auth.register.integration.test.ts	APROBADO

ID	AC	Caso, precondición y pasos	Resultado esperado	Verificación automatizada	Resultado
CP-224.02	-	**Baja de cuenta y anonimización** *Pre:* Usuario sin envíos activos. *Pasos:* Dar de baja la cuenta con la contraseña.	La cuenta queda deleted y anonimizada, sin sesiones; se borran los registros de KYC y de licencia y la clave pública del dispositivo.	`users.account-settings.integration.test.ts` `account-deletion.integration.test.ts`	APROBADO
CP-224.03	-	**Purga de datos vencidos** *Pre:* Envíos cerrados hace más de 5 años cuyas partes se dieron de baja. *Pasos:* Correr el job de retención.	Borra el envío vencido con todo lo que cuelga de él y su foto en S3; no borra si una parte sigue activa, si la última baja es reciente, si se cerró hace menos de 5 años ni si está en disputa; si falla S3, la fila queda para la próxima corrida; también purga viajes cerrados de un transportista dado de baja hace más de 5 años.	`history-retention.integration.test.ts` `history-retention.test.ts`	APROBADO

09

PRODUCTO · MODELO DE DISEÑO

Diagrama de Clases

Modelo de clases del dominio por contexto delimitado y estructura en capas de los servicios.

CONTENIDO

9.1	Modelo de Clases del Dominio	193
9.2	Contexto de Identidad	194
9.3	Contexto de Envíos	195
9.4	Contexto de Pagos	196
9.5	Estructura en Capas e Integración con Proveedores	198

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: diagramas de clases reconstruidos desde el esquema de datos y los servicios vigentes al cierre del Sprint 8.

Introducción

Este documento presenta el modelo de clases del dominio de Movo, un diagrama por contexto delimitado (identidad, envíos y pagos), y la estructura en capas de los servicios con sus interfaces de integración. Se construyó a partir del código fuente vigente.

Audiencia

El tribunal evaluador y los futuros mantenedores, con conocimiento de UML y diseño orientado a objetos.

9.1 Modelo de Clases del Dominio

El dominio de Movo se reparte en tres contextos delimitados, cada uno propiedad de un microservicio con su propio esquema de datos (ADR-001, ADR-003): **identidad** (`movo-svc-users`), **envíos** (`movo-svc-shipments`) y **pagos** (`movo-svc-payments`). Los diagramas de clases siguientes modelan cada contexto con sus entidades, atributos principales, operaciones y asociaciones. Un cuarto diagrama muestra la estructura en capas de un servicio y las interfaces de integración con proveedores externos.

Se aplican estas convenciones de lectura:

- Los **atributos** son los del esquema real de datos (Prisma, ADR-011), con los tipos del dominio (`UUID`, `Decimal`, `DateTime`); se muestran los más relevantes de cada entidad, y el listado completo está en el diccionario de datos del documento de DER.
- Las **operaciones** son los métodos reales de los servicios de cada módulo que operan sobre la entidad, no operaciones inventadas: en Movo las reglas de negocio viven en servicios (`*.service.ts`) y las entidades son datos persistidos, por lo que cada operación se asocia a la entidad sobre la que actúa.
- Las **clases marcadas con el estereotipo** `«movo»` (en verde) son los agregados raíz de cada contexto.
- Entre contextos **no hay asociaciones** en los diagramas: las referencias entre servicios (por ejemplo, el `senderId` de un envío hacia un usuario) son referencias lógicas por identificador, sin clave foránea, para que ningún servicio dependa de las tablas de otro (ADR-003, ADR-010).

9.2 Contexto de Identidad

La entidad raíz es `User`, que concentra la cuenta, el estado de verificación y los datos de contacto. Los **roles** (`sender`, `carrier`, `receiver`, `admin`) no son un campo fijo: un mismo usuario puede tener varios a la vez, por eso se modelan como una relación `UserRoleGrant`. La verificación de identidad y de licencia se registra intento por intento en `KycVerification`; `User` conserva solo el estado vigente. Las **sanciones** son un historial de solo agregar filas: suspender, inhabilitar o levantar agrega o cierra una fila, nunca modifica el pasado. La clave pública del dispositivo (`DeviceKey`) es la que verifica la firma del handshake criptográfico.

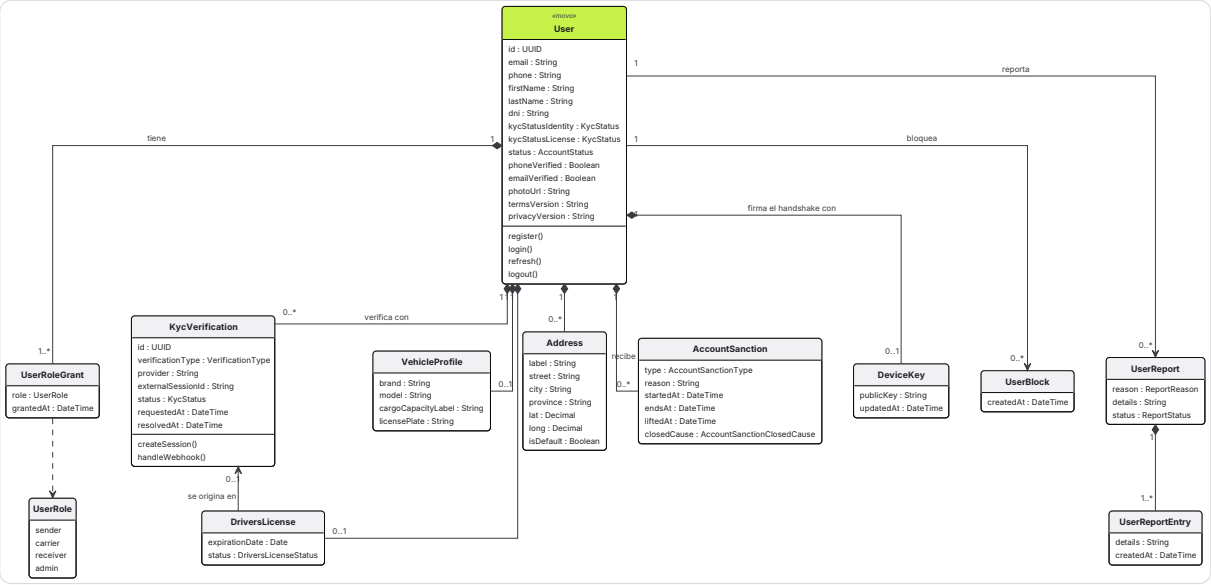


Figura 9.1. Diagrama de clases del contexto de identidad.

9.3 Contexto de Envíos

`Shipment` es el agregado raíz: su ciclo de vida lo gobierna una máquina de estados (ver el documento de diagramas de estados) y su historia queda en `ShipmentEvent`, una línea de tiempo inmutable. Los transportistas compiten por un envío mediante `Offer`; una oferta aceptada puede asociarse a un `Trip` (el viaje declarado por el transportista), que a su vez recibe la traza GPS en `CarrierPosition`. La custodia física se prueba con `HandshakeEvent` (retiro y entrega) y con las fotos de `ShipmentPhoto`. Las calificaciones, las solicitudes de transferencia del receptor, las disputas y el chat cuelgan del envío y mueren con él.

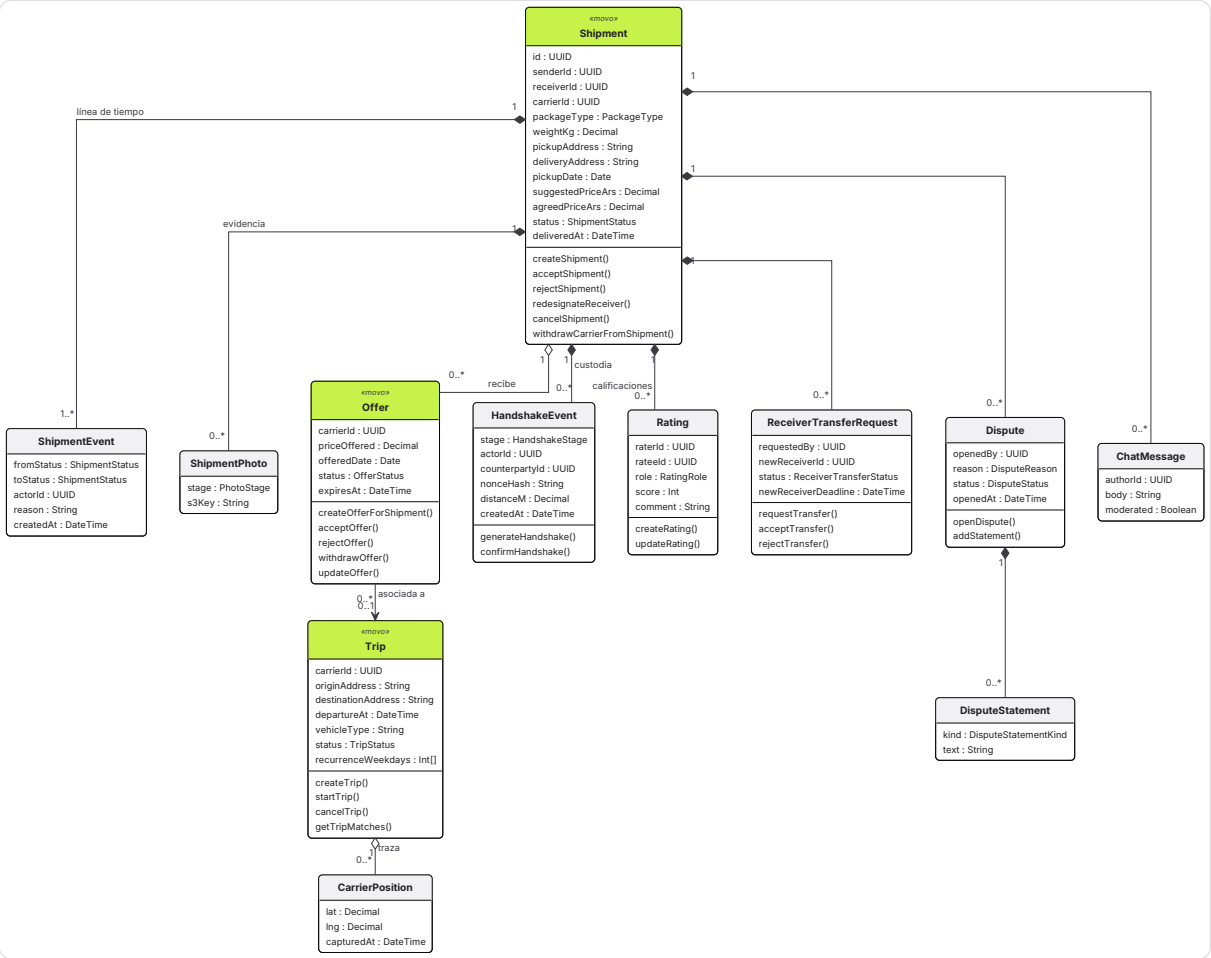


Figura 9.2. Diagrama de clases del contexto de envíos.

9.4 Contexto de Pagos

El agregado central es `Hold` (la reserva de fondos autorizada pero no capturada). Al confirmarse la entrega se liquida en un `Settlement`, que registra el reparto entre el transportista y Movo. El dinero que queda pendiente entre usuarios y la plataforma (penalidades, recargos y pagos de saldo) se lleva en `UserLedgerEntry`, una cuenta corriente por usuario con movimientos idempotentes. Las tarjetas del emisor (`SavedCard`, agrupadas bajo su `PayerCustomer` en Mercado Pago) y la cuenta de cobro del transportista (`CarrierMpAccount`) completan el contexto. Movo nunca almacena número de tarjeta ni CVV.

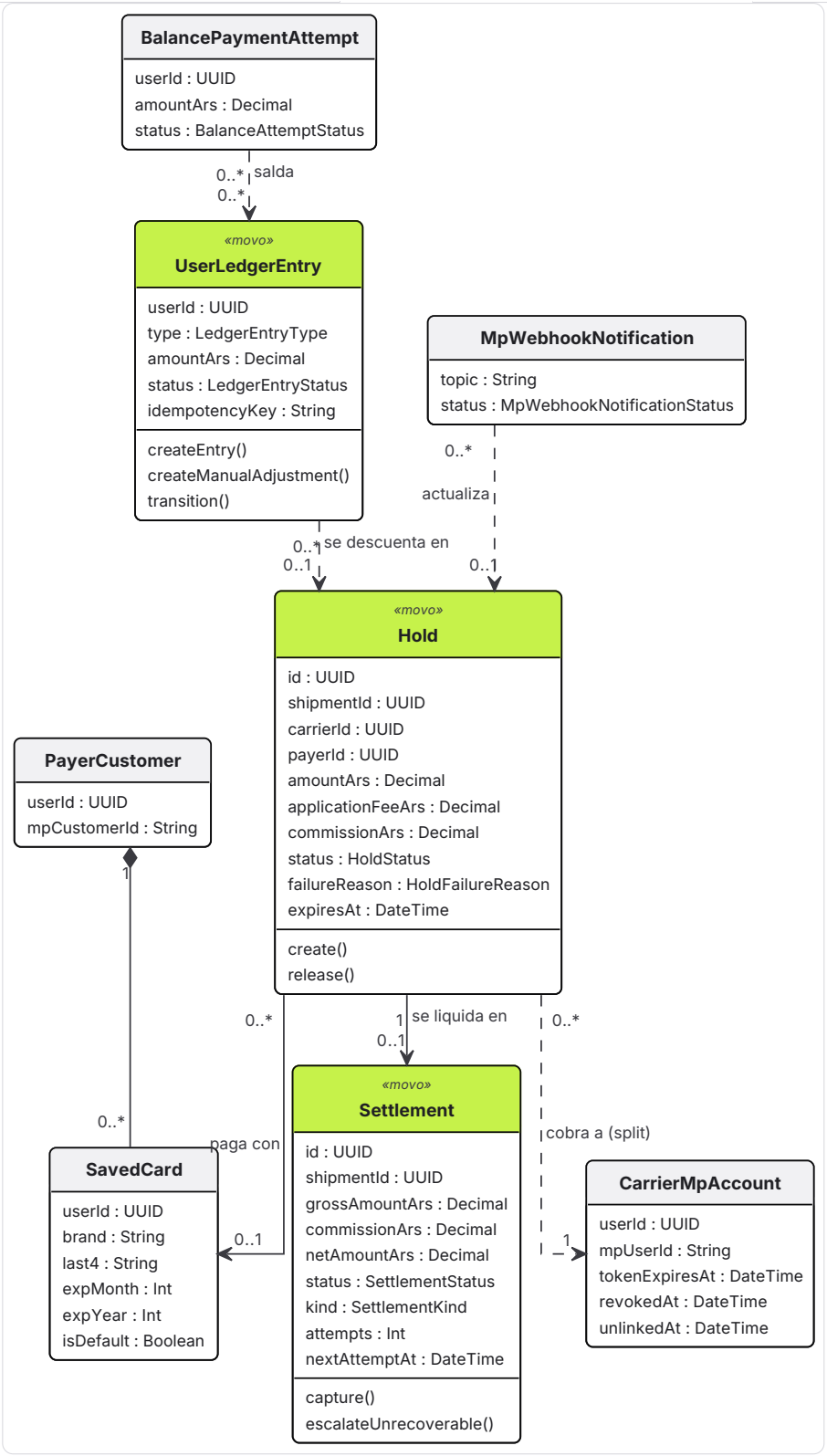


Figura 9.3. Diagrama de clases del contexto de pagos.

9.5 Estructura en Capas e Integración con Proveedores

Todos los servicios Node siguen la misma estructura en capas: las **rutas** (Fastify, con esquemas JSON que validan la entrada y generan el Swagger) delegan en **servicios** con las reglas del negocio, que usan el **dominio** (funciones puras y máquinas de estados), **repositorios** (acceso a datos con Prisma) y **adaptadores o clientes** (integración con proveedores externos y con otros servicios). Cada integración externa se oculta detrás de una interfaz con una implementación real y una implementación de consola o *mock*, elegida por una variable de entorno; es lo que permite desarrollar, probar y correr la integración continua sin depender de servicios de pago (ADR-012, ADR-014, ADR-016).

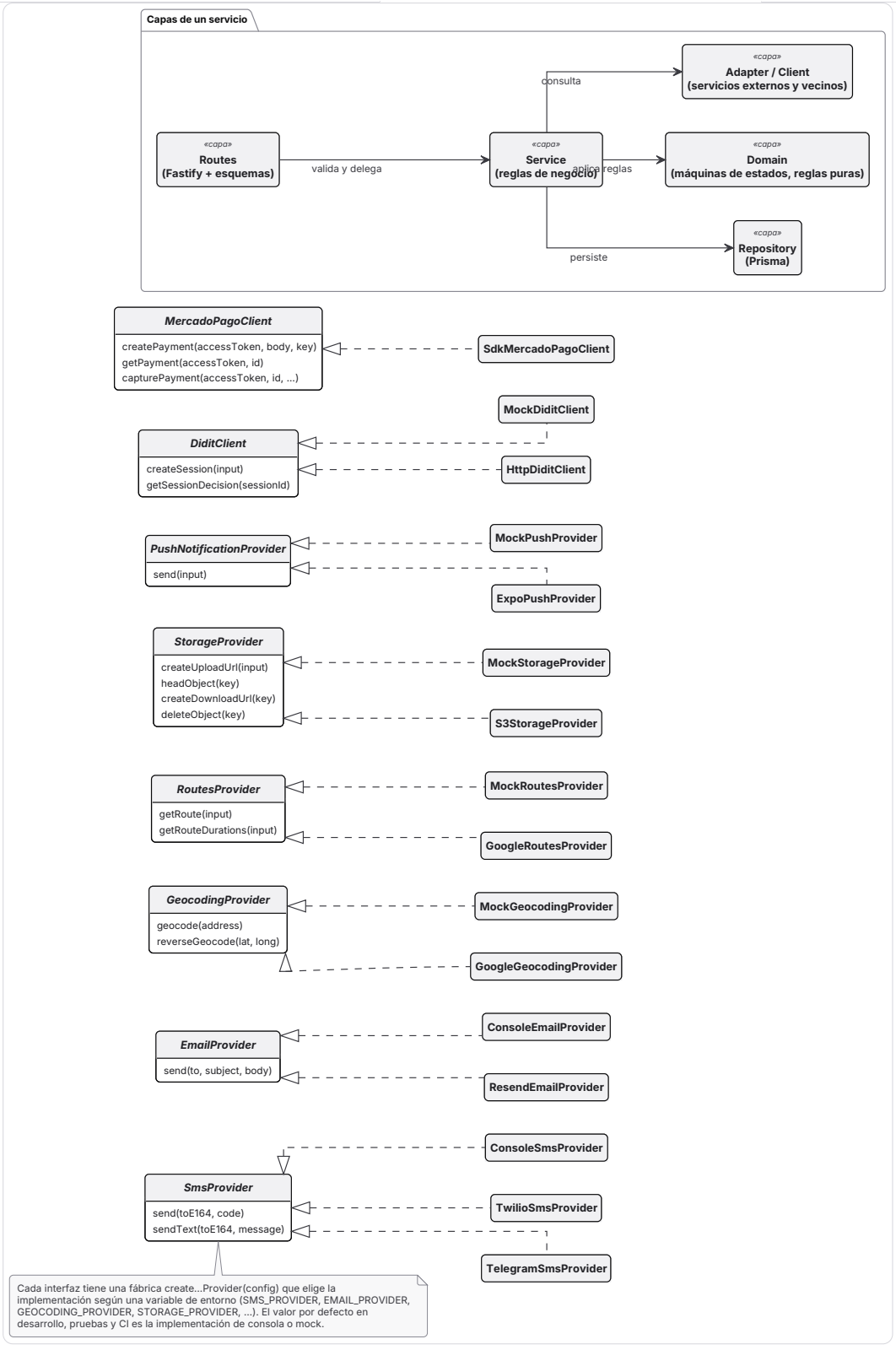


Figura 9.4. Capas de un servicio e interfaces de proveedores externos.

10

PRODUCTO · MODELO DE DISEÑO

Patrones de Diseño

Patrones aplicados en el código de Movo, con el problema que resuelven, su ubicación y el costo aceptado.

CONTENIDO

10.1	Criterio de Selección	203
10.2	Resumen	204
10.3	Creacionales y Estructurales	206
10.4	De Comportamiento	208
10.5	De Consistencia e Integración Distribuida	209
10.6	De Resiliencia y Operación	210

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: patrones relevados del código fuente vigente al cierre del Sprint 8.

Introducción

Este documento recopila los patrones de diseño que el código de Movo aplica efectivamente. Para cada patrón se indica el problema, la solución adoptada, los archivos donde se implementa y el costo que el equipo aceptó.

Audiencia

El tribunal evaluador y los futuros mantenedores, con conocimiento de patrones de diseño y de arquitectura de microservicios.

10.1 Criterio de Selección

Este documento recopila los patrones de diseño que efectivamente aplica el código de Movo, con el problema que resuelven, dónde se implementan y qué costo aceptado traen. No se listan patrones teóricos que el proyecto no use. Cada entrada cita archivos reales del repositorio para que el tribunal y los mantenedores puedan verificarla.

Los patrones se agrupan en cuatro familias: **creacionales y estructurales** (cómo se construyen y conectan los componentes), **de comportamiento** (cómo colaboran), **de consistencia y de integración distribuida** (cómo se mantiene la integridad entre servicios sin transacciones distribuidas) y **de resiliencia y operación**.

10.2 Resumen

Tabla 10.1. Patrones de diseño aplicados en Movo.

Patrón	Familia	Problema que resuelve	Dónde
Strategy y Factory Method	Creacional / comportamiento	Cambiar el proveedor de un servicio externo sin tocar la lógica	`adapters/*-provider.ts`
Adapter y Anti-Corruption Layer	Estructural	Aislar al dominio del modelo de un tercero o de otro servicio	`adapters/*-client.ts`, `mercadopago-client.ts`
Repository	Estructural	Separar la lógica de negocio del acceso a datos	`repositories/*.ts`
API Gateway y Proxy	Estructural	Punto único de entrada con autenticación y límites	`gateway/src`
Inyección de dependencias por fábricas	Creacional	Componer servicios testeables sin un contenedor de DI	`create*Service(...)`
State (máquina de estados)	Comportamiento	Impedir estados y transiciones inválidas	`domain/*-state-machine.ts`
Publicación y suscripción	Comportamiento	Difundir eventos en tiempo real a clientes conectados	`realtime/`, chat y tracking
Saga con compensación	Integración distribuida	Asignar un envío y reservar fondos entre dos servicios	`modules/funding/`
Bloqueo optimista (compare-and-swap)	Consistencia	Evitar condiciones de carrera sin bloqueos largos	`repositories/*.ts`
Receptor idempotente	Consistencia	Tolerar reintentos y doble toque sin duplicar efectos	`clientMessageId`, `idempotencyKey`

Patrón	Familia	Problema que resuelve	Dónde
Barrido periódico con candado distribuido	Operación	Ejecutar tareas de fondo en una sola réplica	`plugins/*-sweep.ts`
Degradación controlada (fail-open / fail-closed)	Resiliencia	Decidir qué hacer cuando cae un servicio vecino	clientes y gateway
Historial de solo agregar filas	Consistencia	Auditar sin posibilidad de reescribir el pasado	triggers en las migraciones
DTO y contrato compartido	Estructural	Una única definición de los tipos entre servicios	`shared/movo-shared`

10.3 Creacionales y Estructurales

10.3.1 Strategy y Factory Method: proveedores intercambiables

Problema. Movo depende de servicios de pago o de cuota limitada (Twilio, Resend, Google Maps, Didit, Mercado Pago, S3). Usarlos en desarrollo, en las pruebas y en la integración continua tendría costo, volvería lentas las pruebas y las haría dependientes de la red.

Solución. Cada capacidad externa se expresa como una **interfaz** (por ejemplo, `SmsProvider`, `EmailProvider`, `GeocodingProvider`, `RoutesProvider`, `StorageProvider`, `PushNotificationProvider`, `DiditClient`), con una implementación real y una de consola o *mock*. Una **fábrica** `create…Provider(config)` elige la estrategia según una variable de entorno (`SMS_PROVIDER`, `EMAIL_PROVIDER`, `GEOCODING_PROVIDER`, `STORAGE_PROVIDER`, `DIDIT_MODE`). El valor por defecto es siempre el *mock*, de modo que un entorno mal configurado falla del lado seguro y barato.

Dónde. `services/movo-svc-users/src/adapters/` y `services/movo-svc-shipments/src/adapters/`. Decisiones asociadas: ADR-012 (SMS), ADR-014 (geocoding), ADR-016 (email).

Costo aceptado. Hay que mantener la paridad de contrato entre la implementación real y el *mock*; y una variable de entorno olvidada en el despliegue hace que el servicio arranque sano pero con el *mock*, un fallo silencioso que ya ocurrió dos veces y motivó la regla de las tres ubicaciones en el `CLAUDE.md` del repositorio.

10.3.2 Adapter y Anti-Corruption Layer: clientes de otros servicios

Problema. Los servicios se comunican por REST síncrono (ADR-001). Si cada uno consumiera directamente el modelo de otro, un cambio interno se propagaría por todo el sistema.

Solución. Cada servicio accede a sus vecinos a través de un **cliente** que traduce el contrato HTTP al vocabulario propio del servicio y centraliza el *timeout*, los errores y la autenticación interna: `UsersClient`, `PaymentsClient`, `PricingClient`, `PricingLogisticsClient`, `NotificationsClient`. Lo mismo vale para el SDK de Mercado Pago, aislado detrás de `MercadoPagoClient`. El dominio nunca ve la forma de la respuesta del tercero.

Dónde. `services/movo-svc-shipments/src/adapters/*-client.ts`, `services/movo-svc-payments/src/adapters/`.

10.3.3 Repository

Problema. Evitar que las reglas de negocio contengan consultas y que las pruebas necesiten mockear el ORM.

Solución. Toda lectura y escritura pasa por un repositorio por agregado (`ShipmentRepository`, `OfferRepository`, `TripRepository`, `HoldRepository`, …), que encapsula Prisma y las transacciones. Los servicios reciben el repositorio por parámetro, de modo que las pruebas pueden usar la implementación real contra una base real (como exige el plan de pruebas) o una implementación en memoria.

Dónde. `services/*/src/repositories/`.

10.3.4 API Gateway y Proxy

Problema. Los clientes no deben conocer la topología interna, y la autenticación, los límites de uso y la restricción de cuentas sancionadas no deben repetirse en cada servicio.

Solución. El gateway es el único servicio expuesto: valida el JWT, aplica los límites de uso con contadores en Redis, bloquea las acciones de cuentas suspendidas o inhabilitadas leyendo su marca en Redis, inyecta los encabezados `x-user-id` y roles (descartando los del cliente) y reenvía el pedido al servicio dueño de la ruta según un mapa de rutas declarativo. También hace de *proxy* de los WebSockets. Los servicios internos confían en esos encabezados (ADR-010).

Costo aceptado. El modelo de confianza es perimetral, no *zero-trust*: si alguien llega a la red interna, puede suplantar encabezados. Es una limitación aceptada y documentada.

Dónde. `gateway/src/plugins/` (`auth.ts`, `rate-limit.ts`, `account-restriction.ts`) y `gateway/src/config/routes-map.ts`.

10.3.5 Inyección de dependencias mediante fábricas

Problema. Componer servicios con muchas colaboraciones (repositorios, clientes, proveedores) manteniéndolos fáciles de probar.

Solución. En lugar de un contenedor de inyección, cada servicio es una función fábrica `createXService(deps)` que recibe sus colaboradores y devuelve un objeto con sus operaciones. El `app.ts` de cada servicio hace la composición. Las dependencias opcionales (por ejemplo, el cliente de notificaciones) degradan el comportamiento en vez de romperlo.

10.3.6 DTO y contrato compartido

Problema. Duplicar los tipos entre servicios genera divergencias silenciosas.

Solución. El paquete `shared/movo-shared` define los tipos de dominio, los enumerados, los claims del JWT y el contrato de error `ApiError`; el gateway y los servicios Node lo importan, y los DTO de cada módulo mapean las filas de la base al contrato público sin exponer identificadores internos ni datos personales de otras partes.

10.4 De Comportamiento

10.4.1 State: máquinas de estados explícitas

Problema. El envío, la oferta, el viaje y el hold tienen ciclos de vida con reglas estrictas; un estado inválido puede significar dinero mal movido.

Solución. Cada objeto declara un grafo cerrado de transiciones válidas (`VALID_TRANSITIONS`) y toda transición pasa por una función que lo valida y lanza un error de dominio ante cualquier otra; el estado se cambia con un UPDATE condicionado al estado de origen. Los diagramas de estados de este tomo se generan leyendo ese mismo código.

Dónde. `services/movo-svc-shipments/src/domain/shipment-state-machine.ts` y `offer-state-machine.ts`; `services/movo-svc-payments/src/modules/user-ledger/user-ledger.service.ts`.

10.4.2 Publicación y suscripción para el tiempo real

Problema. Emisor y receptor deben ver la posición del transportista y los mensajes del chat sin recargar.

Solución. Un registro en memoria de conexiones WebSocket por envío (`realtime`) actúa como *hub*: los servicios publican eventos (nueva posición, nuevo mensaje, cambio de estado) y el registro los difunde a los suscriptores autorizados. Cuando un cambio de estado termina el seguimiento, el *hub* cierra las conexiones con un código propio (ADR-019).

Dónde. `services/movo-svc-shipments/src/realtime/`.

10.5 De Consistencia e Integración Distribuida

10.5.1 Saga con compensación: asignación y reserva de fondos

Problema. Asignar un transportista (en `movo-svc-shipments`) y reservar el dinero del emisor (en `movo-svc-payments`) son dos operaciones en dos servicios y dos bases, sin transacción distribuida posible.

Solución. La asignación es una **saga** orquestada por `movo-svc-shipments`: acepta la oferta y deja el envío en un estado intermedio (`assignment_pending` o `assigned_unfunded`), pide el hold a `movo-svc-payments` y, si el pago se completa, confirma la asignación; si falla, vence o llega un evento de Mercado Pago, ejecuta la **compensación**: libera el hold, devuelve el envío a `published` y deja la oferta como no aceptada. Barridos periódicos cierran los casos que quedan a mitad de camino y reconcilian el estado ante un fallo parcial (DD-037).

Dónde. `services/movo-svc-shipments/src/modules/funding/`.

10.5.2 Bloqueo optimista (compare-and-swap)

Problema. Dos aceptaciones de oferta, dos confirmaciones del mismo QR o dos capturas simultáneas no pueden ganar ambas.

Solución. Las transiciones críticas se escriben como `UPDATE … WHERE id = ? AND status = <origen>` y se verifica la cantidad de filas afectadas: si es cero, otro proceso ganó y se responde con un conflicto. No hay bloqueos de fila prolongados ni `SELECT … FOR UPDATE`. Índices únicos parciales en la base refuerzan los invariantes más importantes (un hold vivo por envío, un viaje activo por transportista, una oferta pendiente por par).

Dónde. `repositories/*.ts` de shipments y payments; migraciones con índices parciales.

10.5.3 Receptor idempotente

Problema. Las redes móviles reintentan y los usuarios tocan dos veces.

Solución. Las operaciones sensibles aceptan una clave de idempotencia y repetirlas devuelve el resultado original sin duplicar el efecto: `clientMessageId` en el chat y el soporte, la clave de idempotencia de los pagos de Mercado Pago, la clave de cada movimiento de la cuenta corriente y la huella del pedido de cada hold.

10.5.4 Historial de solo agregar filas

Problema. La auditoría debe ser confiable.

Solución. Las tablas de la bitácora de acciones del administrador, de las notas internas y del historial de sanciones rechazan `UPDATE` y `DELETE` mediante *triggers* de la base de datos; los movimientos de la cuenta corriente no se pueden borrar y solo admiten cambiar su estado. Una sanción levantada cierra su fila, no la borra. Los eventos del envío y del handshake se escriben solo por inserción desde el código.

Dónde. Migraciones de `movo-svc-admin`, `movo-svc-payments` y `movo-svc-users`.

10.6 De Resiliencia y Operación

10.6.1 Barrido periódico con candado distribuido

Problema. Vencimientos, reintentos y limpiezas deben ejecutarse sin intervención y sin duplicarse si hay más de una réplica.

Solución. Cada tarea de fondo (*sweep*) se registra con un temporizador y toma un candado en Redis (`SET NX` con TTL) antes de correr; si otra réplica ya lo tiene, omite la corrida. Cada barrido es idempotente y procesa en lotes.

Dónde. `services/movo-svc-shipments/src/plugins/*-sweep.ts` y `register-sweep.ts`.

10.6.2 Degradación controlada: fail-open y fail-closed

Problema. Los servicios dependen de sus vecinos y de terceros que pueden caer.

Solución. Cada dependencia tiene una política explícita según el riesgo: **falla abierta** cuando bloquear es peor que dejar pasar (el filtro de bloqueos del feed, la restricción de cuentas en el gateway cuando Redis no responde, el precio sugerido que pasa a «a estimar», la ruta que degrada a una heurística), y **falla cerrada** cuando dejar pasar es un riesgo (crear una oferta si no se puede consultar el bloqueo, aceptar una oferta sin poder revalidar al transportista, capturar un pago a ciegas). Los *timeouts* de cada cliente son cortos y explícitos.

11

PRODUCTO · MODELO DE COMPORTAMIENTO

Diagramas de Transición de Estados

Máquinas de estados de los objetos de dominio de Movo (envío, oferta, viaje, verificación, cuenta, pagos y moderación).

CONTENIDO

11.1 Diagramas de Transición de Estados 213

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: diagramas de estados reconstruidos desde el código vigente al cierre del Sprint 8 (reemplazan a los diagramas de Mermaid de los documentos de diseño previos).

Introducción

Este documento reúne los diagramas de transición de estados de los objetos de dominio de Movo. Para cada uno se indica el propósito de sus estados, las transiciones permitidas y el archivo del código del que se obtuvo el diagrama.

Audiencia

El tribunal evaluador y los futuros mantenedores, con conocimiento de UML.

11.1 Diagramas de Transición de Estados

Los objetos de dominio de Movo con un ciclo de vida relevante se modelan como máquinas de estados explícitas: un enumerado de estados, un conjunto cerrado de transiciones válidas y el rechazo de cualquier otra. Esto evita estados imposibles y concentra las reglas del negocio en un único lugar por objeto. Cada diagrama sigue la notación de UML (estado inicial, transiciones con su disparador, estados terminales) y cita el archivo del que se obtuvo, que es la fuente de verdad.

Tabla 11.1. Máquinas de estados del sistema.

Diagrama	Objeto	Estados
1	Envío (`ShipmentStatus`)	11
2	Oferta (`OfferStatus`)	9
3	Viaje (`TripStatus`)	5
4	Verificación de identidad (`KycStatus`)	6
5	Cuenta de usuario (`AccountStatus`)	4
6	Reserva de fondos (`HoldStatus`)	6
7	Liquidación (`SettlementStatus`)	4
8	Movimiento de la cuenta corriente (`LedgerEntryStatus`)	4
9	Transferencia de receptor (`ReceiverTransferStatus`)	5
10	Disputa (`DisputeStatus`)	2
11	Consulta de soporte (`SupportStatus`)	3
12	Reporte de usuario (`ReportStatus`)	3

11.1.1 Envío (`ShipmentStatus`)

Ciclo de vida del envío, con los 11 estados del enumerado canónico. El grafo se genera leyendo `shipment-state-machine.ts`, la única fuente de verdad: toda transición que no figure acá es rechazada con `InvalidShipmentTransitionError` (409). `cancelled` y `disputed` son terminales en este módulo; la resolución de una disputa (MOVO-34) todavía no define a qué estado vuelve el envío, por lo que no se modela una transición inventada.

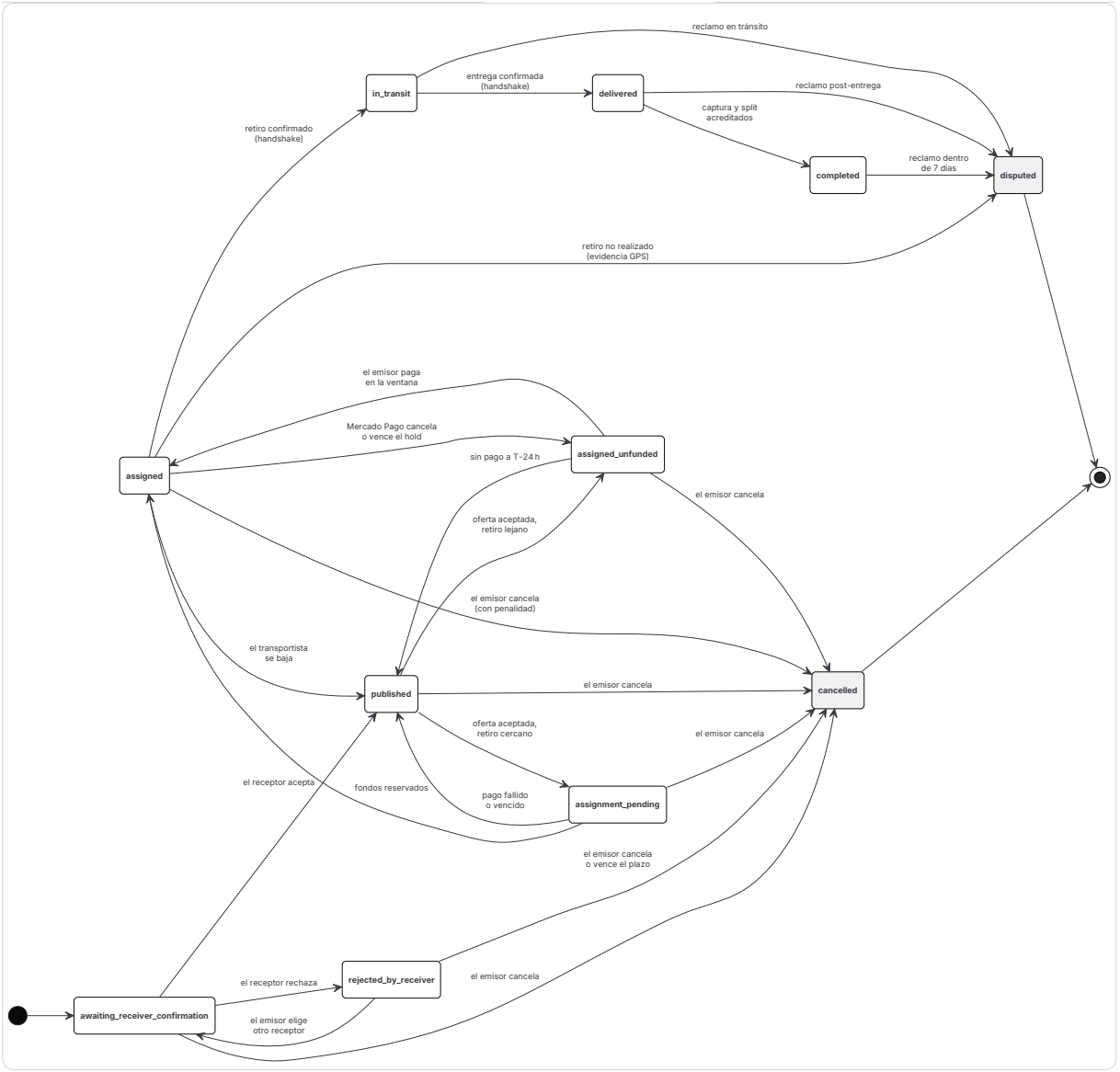


Figura 11.1. Diagrama de estados: Envío (ShipmentStatus).

Fuente: `services/movo-svc-shipments/src/domain/shipment-state-machine.ts`

11.1.2 Oferta (`OfferStatus`)

Una oferta nace `pending` y termina en uno de siete estados terminales. `expired` no se persiste como transición: se deriva en lectura comparando `expires_at` con el reloj, de modo que no hace falta ningún proceso en segundo plano para vencer ofertas. Un rechazo no bloquea una oferta nueva del mismo transportista, que es una fila nueva.

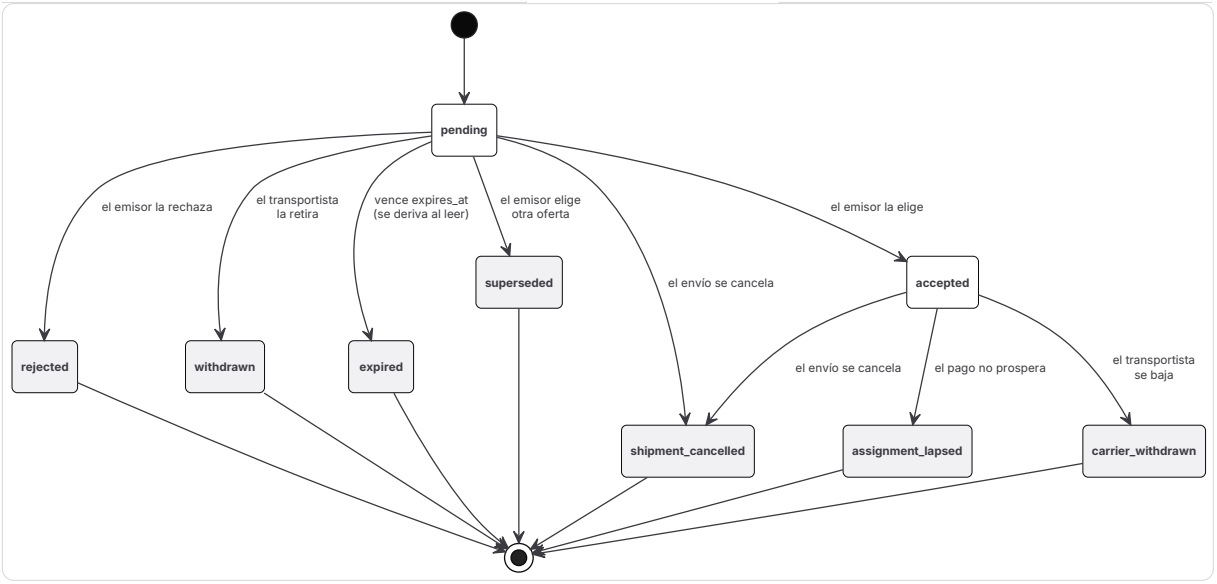


Figura 11.2. Diagrama de estados: Oferta (OfferStatus).

Fuente: `services/movo-svc-shipments/src/domain/offer-state-machine.ts`

11.1.3 Viaje (`TripStatus`)

El transportista declara un viaje (`declared`) y lo inicia explícitamente; solo puede haber un viaje `active` por transportista, garantizado por un índice único parcial. Un viaje declarado vencido sin paquetes se cancela; uno activo sin paquetes ejecutables vence; uno activo con todos sus paquetes entregados se completa.

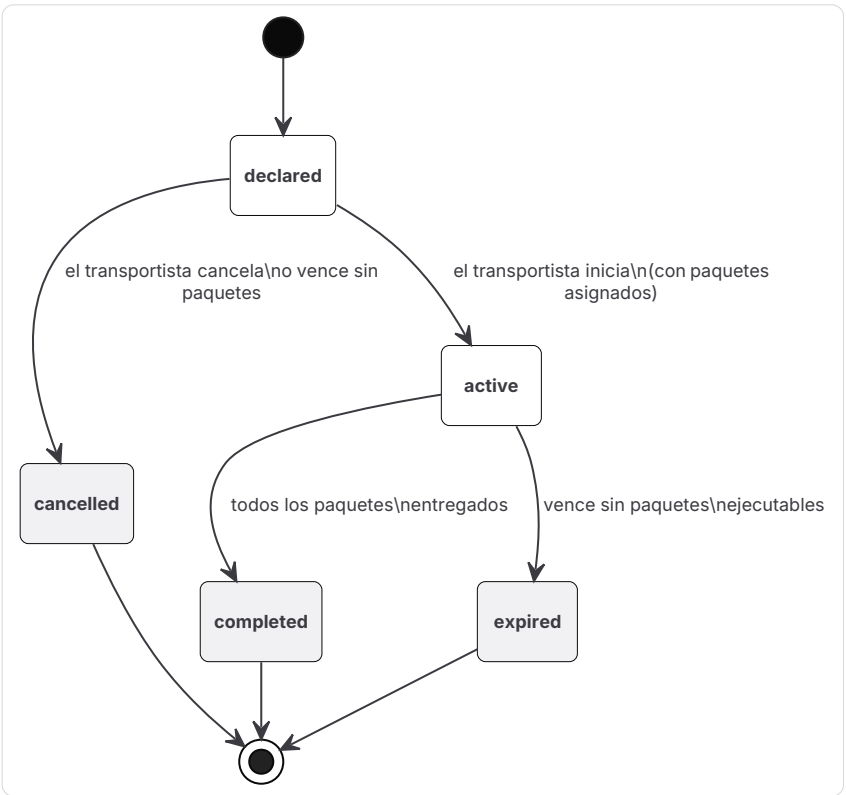


Figura 11.3. Diagrama de estados: Viaje (TripStatus).

Fuente: `services/movo-svc-shipments/src/repositories/trip-repository.ts`

11.1.4 Verificación de identidad (`KycStatus`)

Estado de la verificación delegada en Didit.me. La misma máquina de estados rige la verificación de la licencia de conducir, con `verification_type = license`. Los estados intermedios de Didit no disparan ninguna transición. La revisión manual del equipo (`manual_review`) queda registrada, pero su resolución desde el panel es la historia MOVO-32, todavía pendiente.

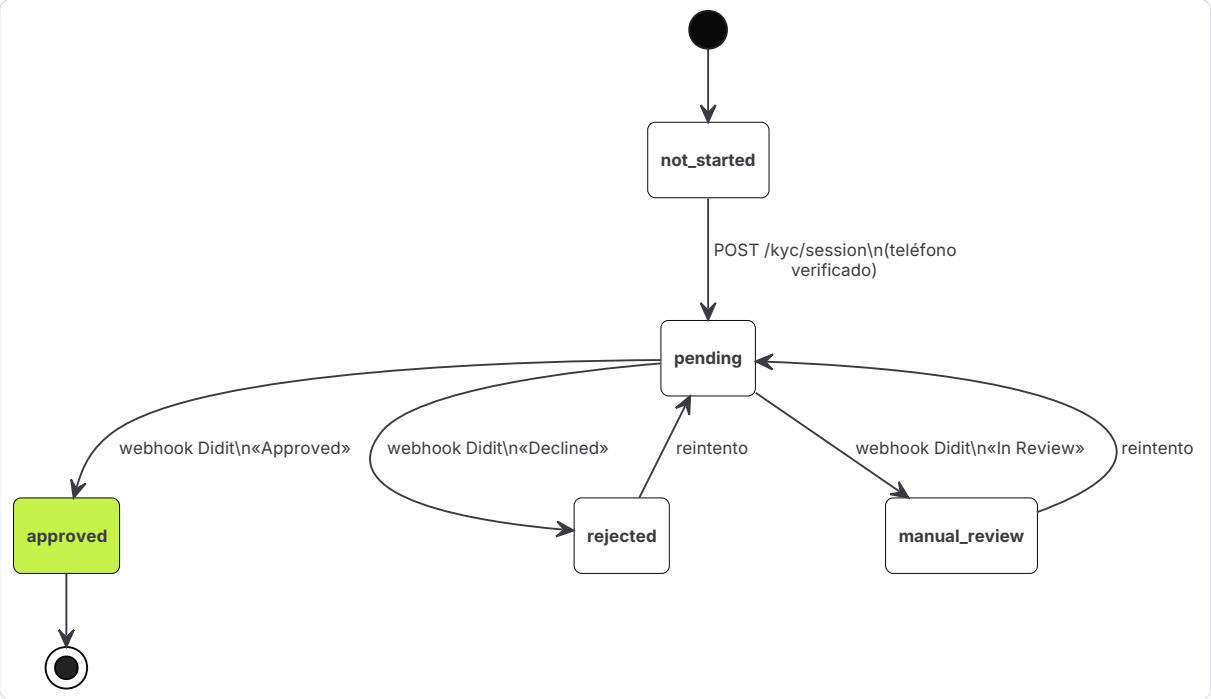


Figura 11.4. Diagrama de estados: Verificación de identidad (KycStatus).

Fuente: `docs/kyc/state-diagram.md`

11.1.5 Cuenta de usuario (`AccountStatus`)

Estado de la cuenta. `suspended` y `banned` (inhabilitada) se aplican y levantan mediante sanciones registradas en un historial de solo agregar filas; una suspensión con fecha de fin vence sola. La baja (`deleted`) anonimiza la cuenta y solo es posible sin envíos activos ni disputas abiertas.

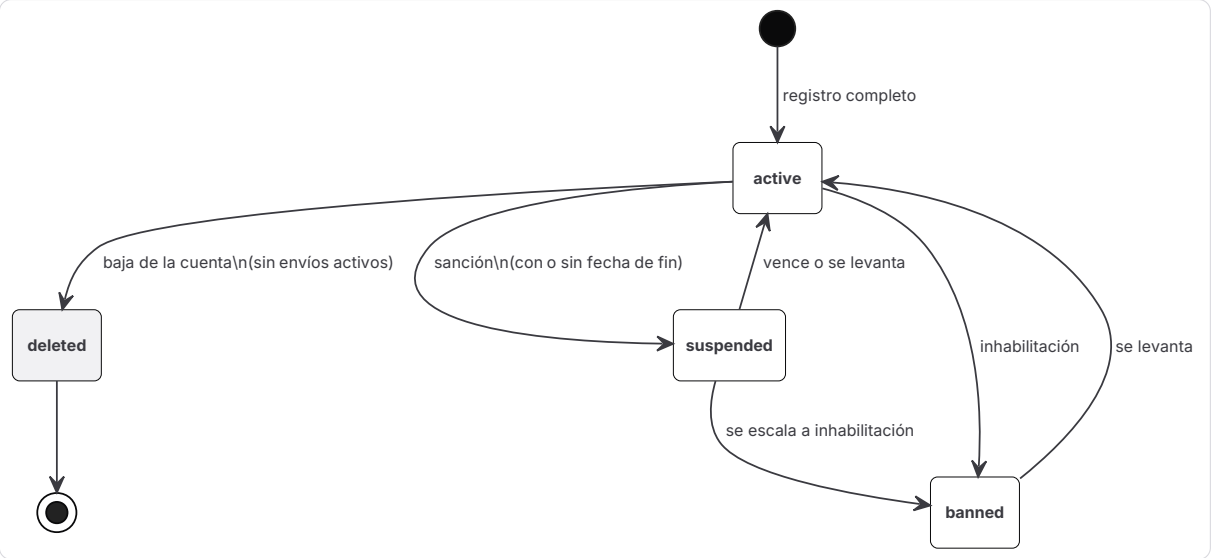


Figura 11.5. Diagrama de estados: Cuenta de usuario (AccountStatus).

Fuente: `services/movo-svc-users/prisma/schema.prisma`

11.1.6 Reserva de fondos (`HoldStatus`)

Una reserva (hold) autoriza el pago en la tarjeta del emisor sin capturarlo. Solo se captura al confirmarse la entrega, o parcialmente al cobrar una penalidad. Un índice único parcial garantiza un solo hold vivo por envío.

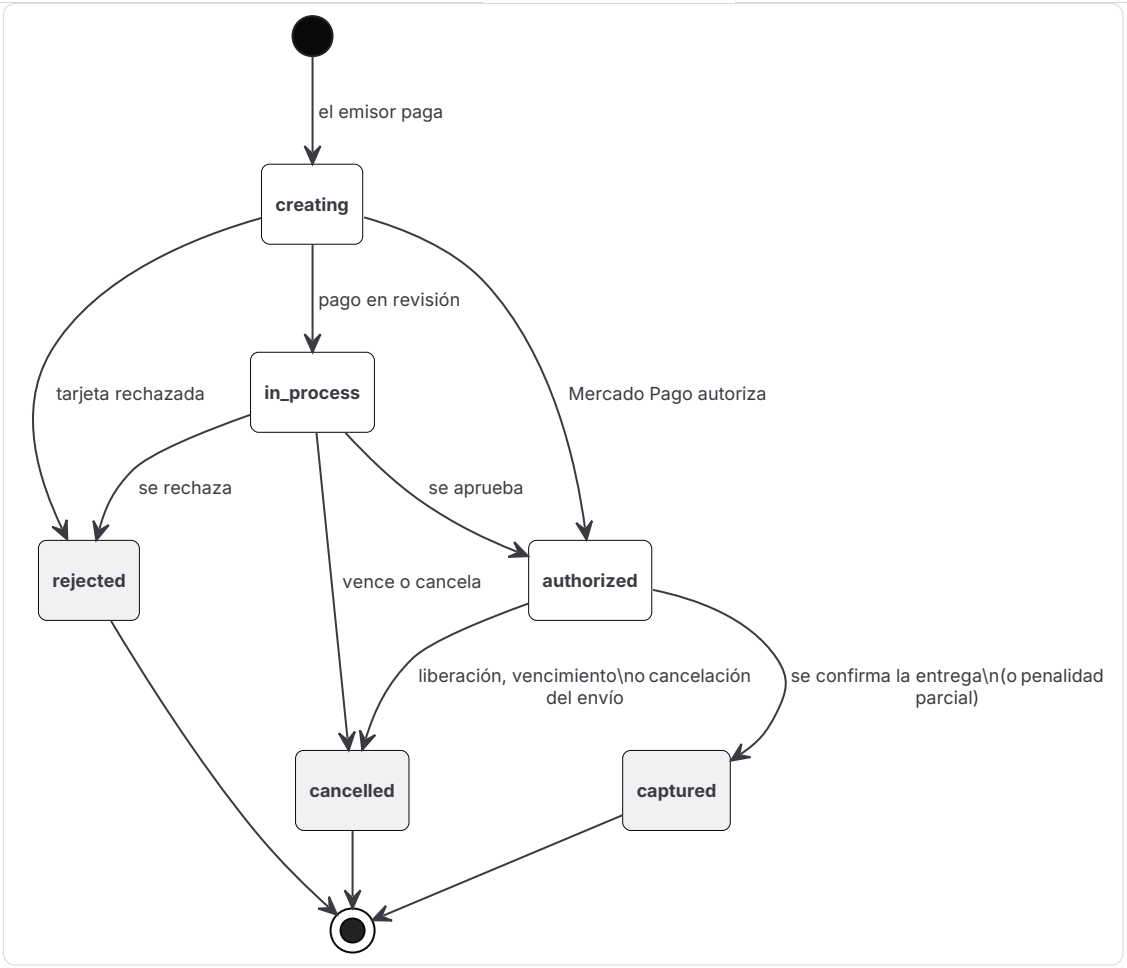


Figura 11.6. Diagrama de estados: Reserva de fondos (HoldStatus).

Fuente: `services/movo-svc-payments/prisma/schema.prisma`

11.1.7 Liquidación (`SettlementStatus`)

Resultado de capturar el hold. Una captura fallida se reintenta con espera creciente (5, 30 y 120 minutos) y, agotados los reintentos o ante una causa irrecuperable (cuenta del transportista rechazada, hold cancelado), escala a revisión manual.

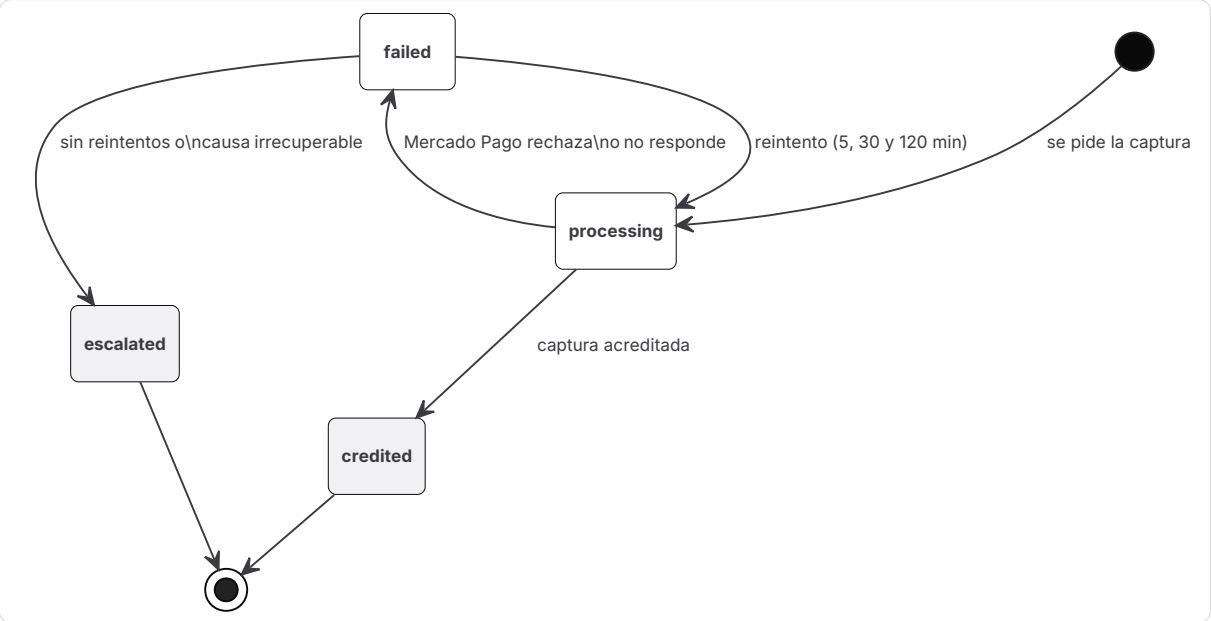


Figura 11.7. Diagrama de estados: Liquidación (SettlementStatus).

Fuente: `services/movo-svc-payments/prisma/schema.prisma`

11.1.8 Movimiento de la cuenta corriente (`LedgerEntryStatus`)

Estados de un movimiento de la cuenta corriente del usuario (penalidades, recargos, pagos voluntarios, ajustes). Las transiciones permitidas están declaradas en `user-ledger.service.ts`: de `settled` solo se sale anulando, y `voided` es terminal.

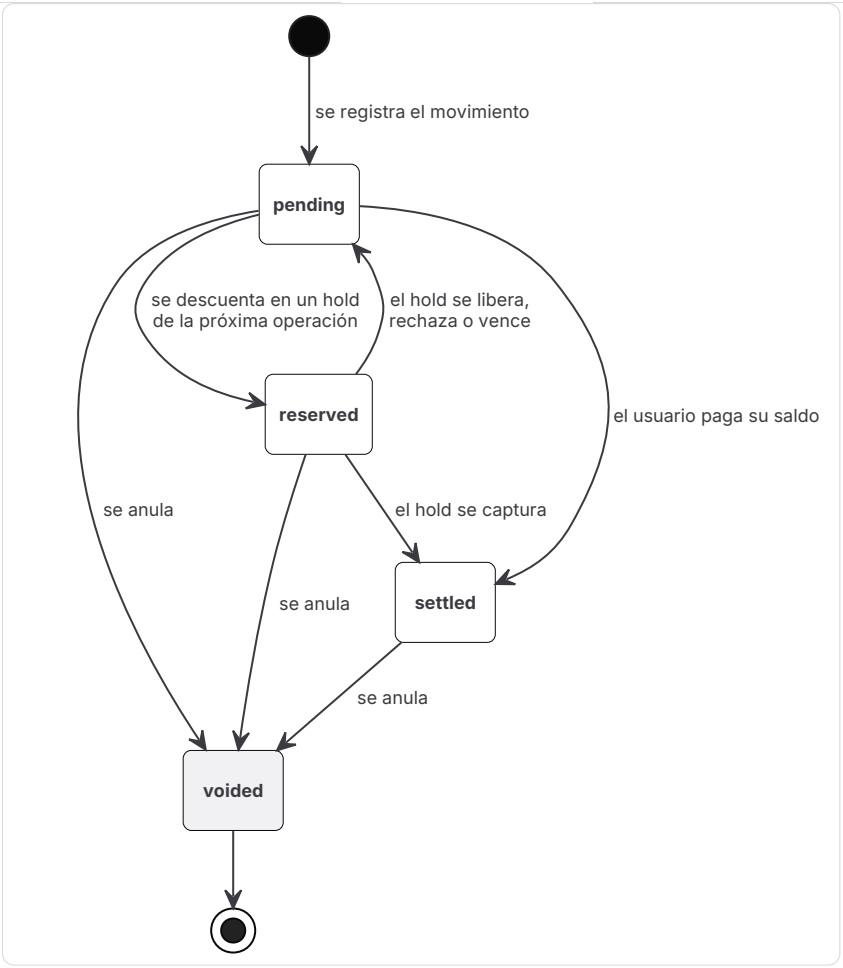


Figura 11.8. Diagrama de estados: Movimiento de la cuenta corriente (LedgerEntryStatus).

Fuente: `services/movo-svc-payments/src/modules/user-ledger/user-ledger.service.ts`

11.1.9 Transferencia de receptor (`ReceiverTransferStatus`)

Solicitud para que otra persona reciba el envío (DD-038). Solo puede haber una solicitud pendiente por envío y una transferencia completada. Una solicitud pendiente se cancela si el receptor la retira o si empieza el handshake de entrega, y un barrido vence las que superan el plazo.

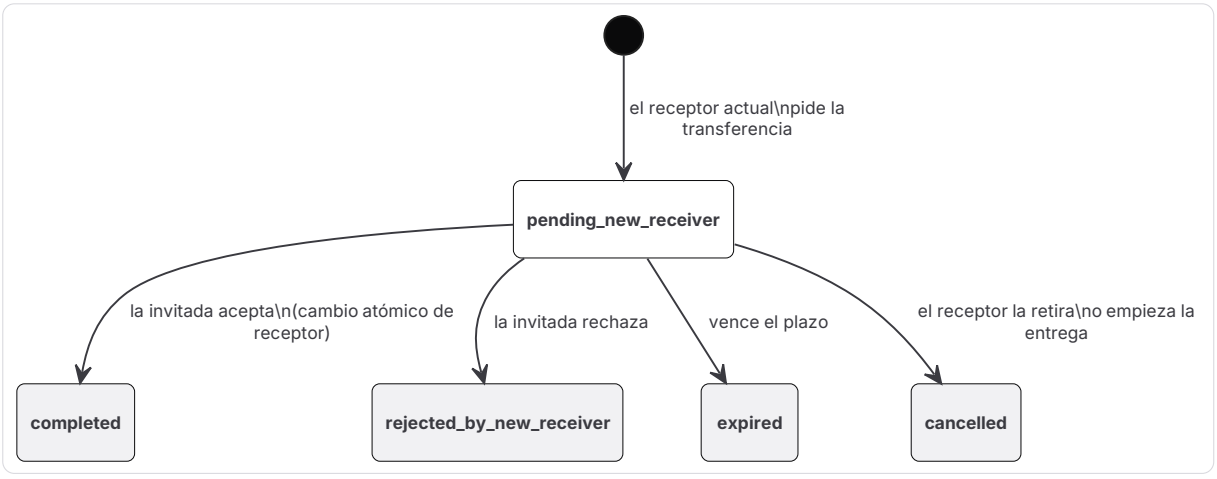


Figura 11.9. Diagrama de estados: Transferencia de receptor (ReceiverTransferStatus).

Fuente: `services/movo-svc-shipments/prisma/schema.prisma`

11.1.10 Disputa (`DisputeStatus`)

Una disputa se abre sobre un envío entregado o completado dentro de los 7 días (DD-054), o la abre el sistema ante una anomalía. Mientras está abierta, las contrapartes cargan sus descargos. El paso a `resolved` depende de la resolución del administrador (MOVO-34), todavía en backlog.

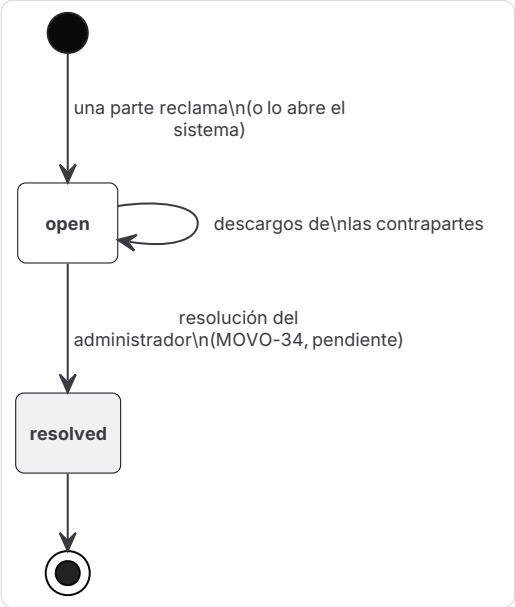


Figura 11.10. Diagrama de estados: Disputa (DisputeStatus).

Fuente: `services/movo-svc-shipments/prisma/schema.prisma`

11.1.11 Consulta de soporte (`SupportStatus`)

Ciclo de una conversación con el equipo de soporte. Cuando el usuario escribe sobre una consulta resuelta, se reabre; sobre una que esperaba su respuesta, vuelve a abierta sin contar como reapertura.

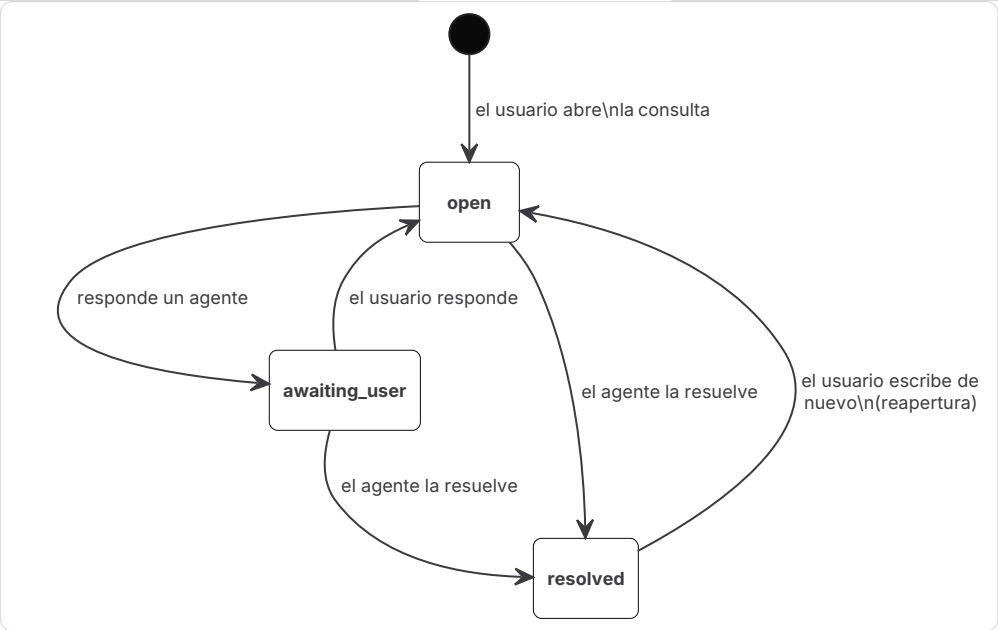


Figura 11.11. Diagrama de estados: Consulta de soporte (SupportStatus).

Fuente: `services/movo-svc-shipments/src/domain/support-rules.ts`

`11.1.12` Reporte de usuario (`ReportStatus`)

Estados de un reporte de moderación. Los estados `reviewed` y `dismissed` existen en el modelo; la pantalla del panel para revisar reportes no está implementada, por lo que hoy el equipo los consulta desde la ficha del usuario.

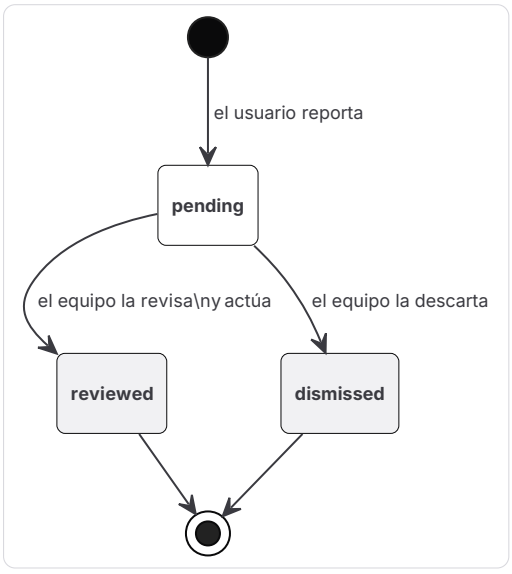


Figura 11.12. Diagrama de estados: Reporte de usuario (ReportStatus).

Fuente: `services/movo-svc-users/prisma/schema.prisma`

12

PRODUCTO · MODELO DE DATOS

Modelo de Datos (DER)

Diagramas entidad-relación por servicio y diccionario de datos de las cuatro bases de Movo.

CONTENIDO

12.1 Modelo de Datos y Diccionario 225

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: DER y diccionario de datos generados a partir de los esquemas Prisma vigentes al cierre del Sprint 8.

Introducción

Este documento presenta el modelo de datos de Movo: un diagrama entidad-relación por cada grupo de tablas, la lista de tablas con su propósito, los tipos enumerados y el diccionario de datos de cada tabla. Se genera automáticamente desde los esquemas Prisma de cada servicio.

Audiencia

El tribunal evaluador y los futuros mantenedores, con conocimiento de bases de datos relacionales.

12.1 Modelo de Datos y Diccionario

El modelo de datos relacional vive en una única instancia de PostgreSQL 16 con **un esquema por microservicio** (ADR-003). Cada servicio define su esquema con Prisma (ADR-011) y es el único que lo lee y escribe: las referencias entre esquemas (por ejemplo, el emisor de un envío o el pagador de un hold) son **referencias lógicas por identificador, sin clave foránea real**, de modo que ningún servicio depende de las tablas de otro. Los diagramas y el diccionario de este documento se generan directamente de los archivos `schema.prisma`, por lo que reflejan el estado real del código.

En total el modelo comprende **50 tablas** y **601 columnas** en cuatro esquemas.

En los diagramas, `*` marca una columna obligatoria, `PK` la clave primaria y `FK` una clave foránea. Por legibilidad cada entidad muestra su clave, sus claves foráneas y las columnas más relevantes; el detalle completo de cada tabla está en el diccionario de datos de cada sección. Las entidades de otro grupo que se referencian por clave foránea aparecen solo con su clave.

12.1.1 Identidad (esquema `users`)

El esquema contiene 16 tablas y 11 tipos enumerados.

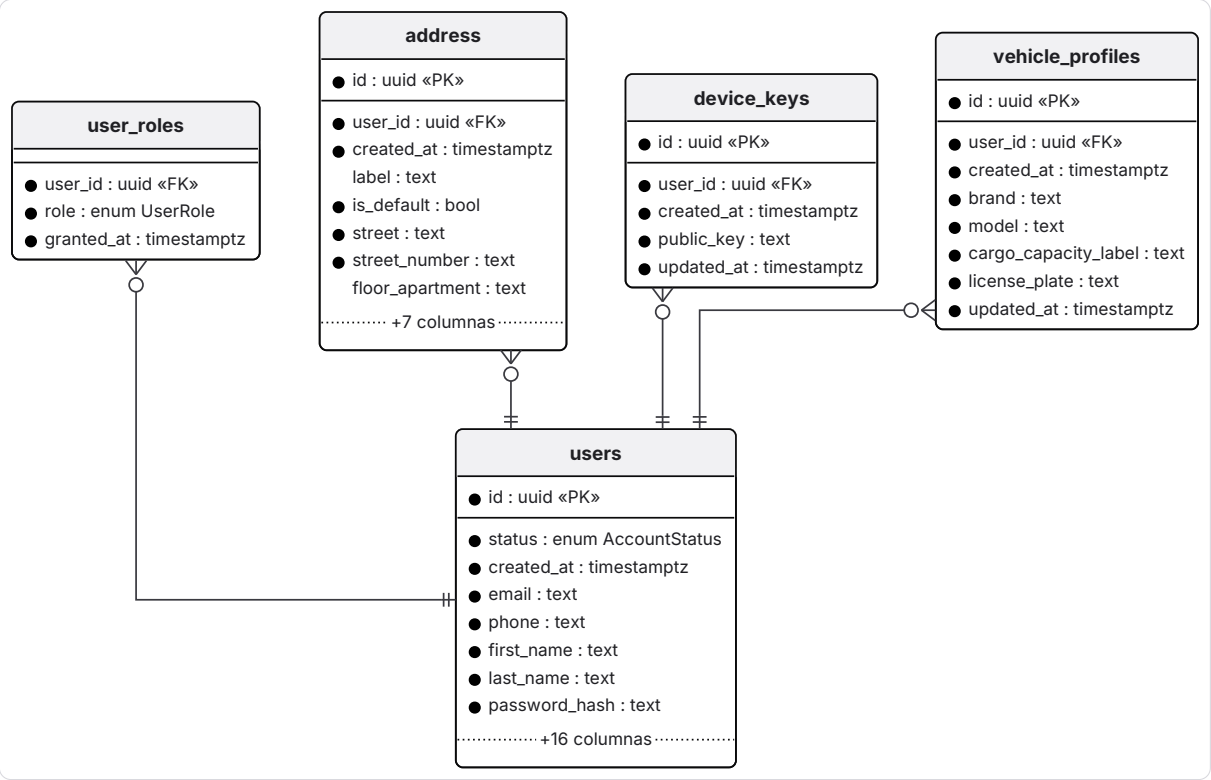


Figura 12.1. DER: Cuenta, roles, direcciones y vehículo.

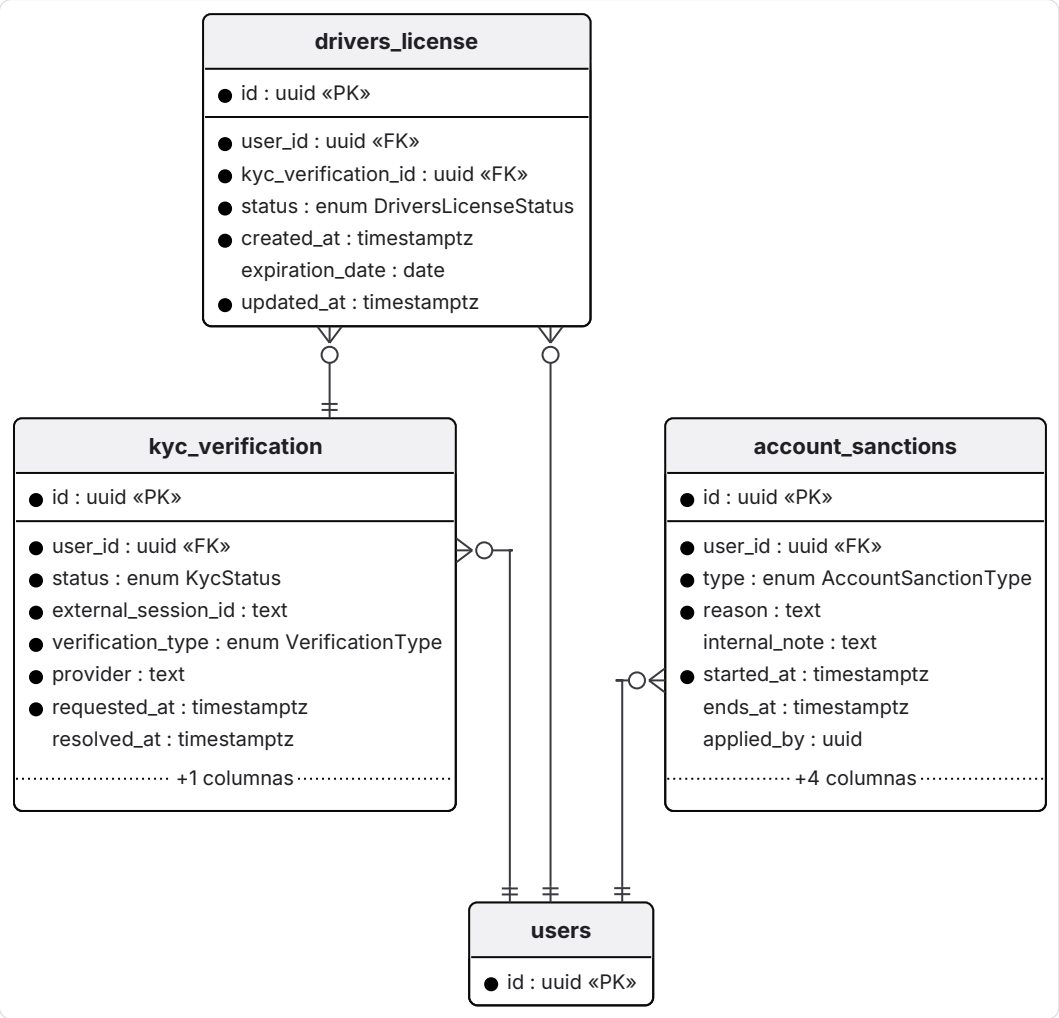


Figura 12.2. DER: Verificación de identidad, licencia y sanciones.

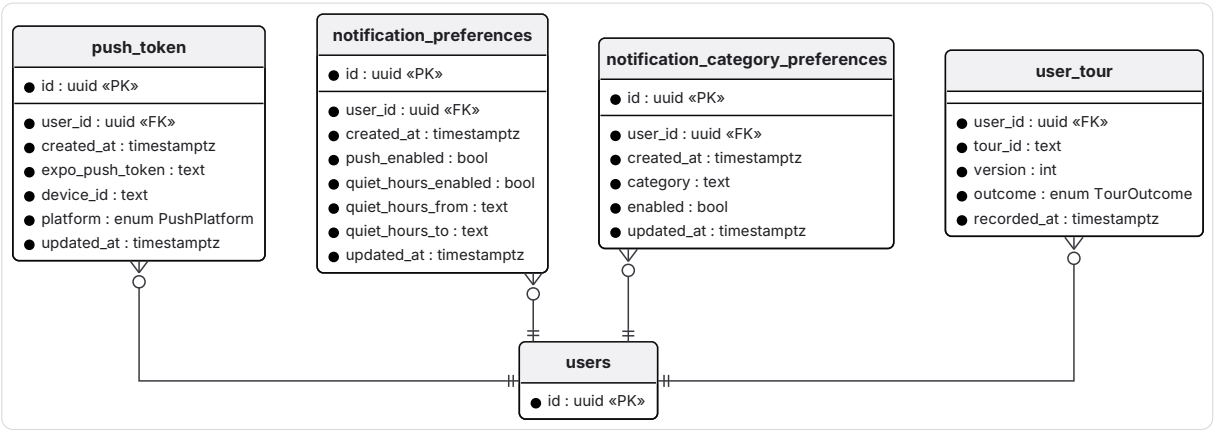


Figura 12.3. DER: Notificaciones y recorridos.

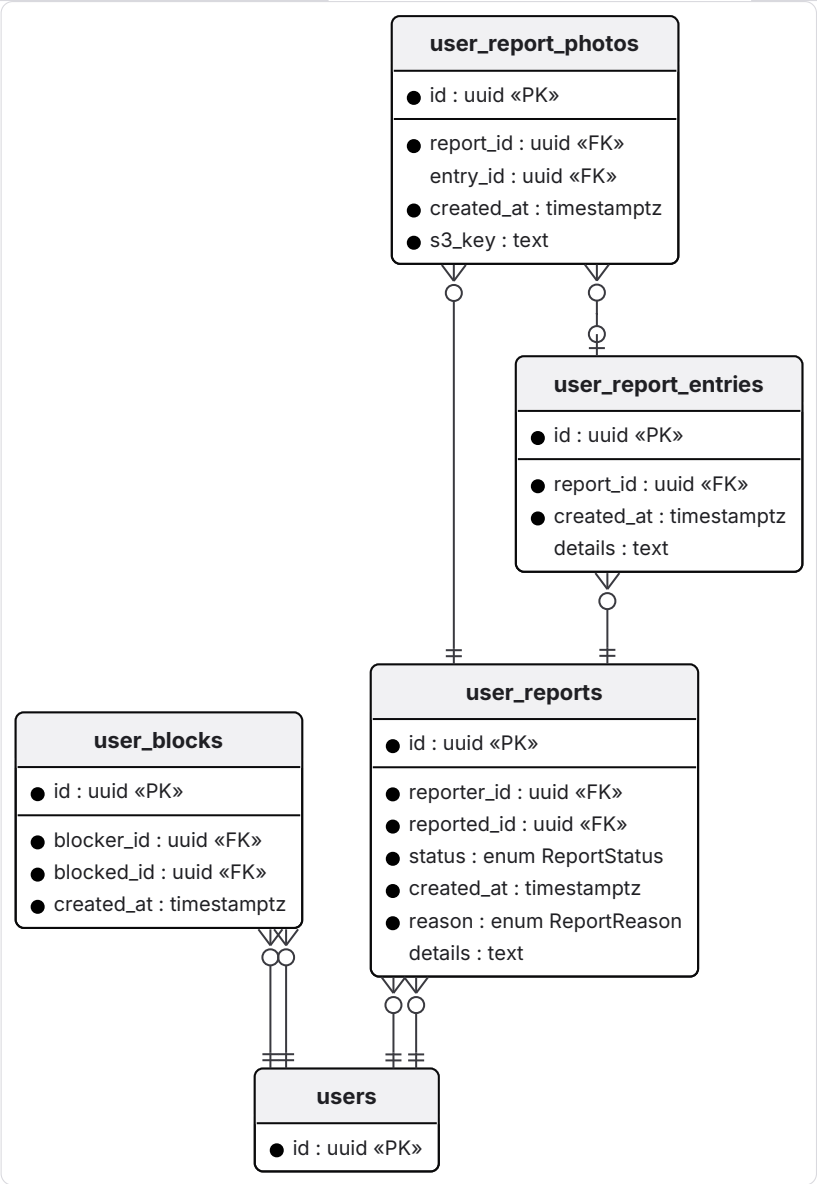


Figura 12.4. DER: Reportes y bloqueos entre usuarios.

Tablas

Tabla 12.1. Tablas del esquema users .

Tabla	Propósito	Columnas
`device_keys`	par de claves asimétricas por dispositivo, prerrequisito del handshake criptográfico de MOVO-6 (implementado en `svc-shipments` , ticket hermano MOVO-158).	5
`vehicle_profiles`	ficha de vehículo del transportista, mostrada en el perfil público (`PublicProfile.vehicle`) para que quien entrega su paquete pueda verificar el vehículo antes de subirlo.	8
`notification_preferences`	toggle maestro de push + “Horario de silencio” (quiet hours), 1 fila por usuario (mismo patrón `userId @unique` que `DeviceKey` / `VehicleProfile` – un upsert sobre la misma fila, sin...	8

Tabla	Propósito	Columnas
`notification_category_preferences`	tabla SPARSE a propósito – una fila solo cuando el usuario tocó el toggle de esa categoría.	6
`user_tour`	registro de los tours del tutorial in-app, un upsert por (usuario, tour).	5
`user_blocks`	una fila por dirección (quién bloqueó a quién), pero el EFECTO es simétrico – `listRelatedUserIds` une las dos direcciones.	4
`user_reports`	solo se persiste – la revisión (pending → reviewed/dismissed) queda para el panel de admin, fuera de alcance de esta US. Se conserva tras la baja de cuenta de cualquiera de las dos...	7
`user_report_entries`	información que el reportante suma a su reporte pendiente.	4
`user_report_photos`	fotos de evidencia de un reporte.	5
`users`	–	24
`user_roles`	–	3
`kyc_verification`	tabla `KYC_VERIFICATION` del DER (“tiene intentos de”, 1 a N desde User).	9
`address`	tabla `address` del DER (docs/movo_der.dbml), diseñada como patrón de libreta de direcciones (1:N desde User, con label/is_default).	15
`push_token`	registro de push token por dispositivo (AC1) – un usuario puede tener N dispositivos, cada uno con un solo token vigente (índice único (user_id, device_id), permite multi-dispositivo).	7
`drivers_license`	tabla `drivers_license` del DER (docs/movo_der.dbml) – el registro del carnet en sí, distinto de `kyc_verification` (que es el log de intentos, 1 a N).	7
`account_sanctions`	historial de sanciones de cuenta.	12

Tipos enumerados

Tabla 12.2. Enumerados del esquema users .

Tipo	Valores
`UserRole`	`sender` , `carrier` , `admin`
`KycStatus`	`not_started` , `pending` , `approved` , `rejected` , `expired` , `manual_review`
`VerificationType`	`identity` , `license`
`DriversLicenseStatus`	`pending` , `verified` , `expired`
`AccountStatus`	`active` , `banned` , `deleted` , `suspended`
`AccountSanctionType`	`suspension` , `disable`
`AccountSanctionClosedCause`	`lifted` , `replaced` , `escalated` , `expired`

Tipo	Valores
`PushPlatform`	`ios`, `android`
`TourOutcome`	`completed`, `skipped`
`ReportReason`	`harassment`, `no_show`, `damaged_package`, `payment_issue`, `other`
`ReportStatus`	`pending`, `reviewed`, `dismissed`

Diccionario de datos

`DEVICE_KEYS`

Tabla 12.3. Columnas de device_keys .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	FK → `users`, único, no nulo	
`public_key`	text	no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`VEHICLE_PROFILES`

Tabla 12.4. Columnas de vehicle_profiles .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	FK → `users`, único, no nulo	
`brand`	text	no nulo	
`model`	text	no nulo	
`cargo_capacity_label`	text	no nulo	
`license_plate`	text	no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`NOTIFICATION_PREFERENCES`

Tabla 12.5. Columnas de notification_preferences .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	

Columna	Tipo	Restricciones	Descripción
`user_id`	uuid	FK → `users`, único, no nulo	
`push_enabled`	bool	no nulo, por defecto true	
`quiet_hours_enabled`	bool	no nulo, por defecto false	
`quiet_hours_from`	text	no nulo, por defecto "23:00"	Hora Argentina "HH:MM" (`toArgentinaTimeOfDayString` / `isWithinQuietHours` de @movo/shared) – se guarda como texto, no `@db.Time`, porque nunca se...
`quiet_hours_to`	text	no nulo, por defecto "08:00"	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`NOTIFICATION_CATEGORY_PREFERENCES`

Tabla 12.6. Columnas de notification_category_preferences .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	FK → `users`, no nulo	
`category`	text	no nulo	
`enabled`	bool	no nulo, por defecto true	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`USER_TOUR`

Tabla 12.7. Columnas de user_tour .

Columna	Tipo	Restricciones	Descripción
`user_id`	uuid	FK → `users`, no nulo	
`tour_id`	text	no nulo	
`version`	int	no nulo	

Columna	Tipo	Restricciones	Descripción
`outcome`	enum TourOutcome	no nulo	
`recorded_at`	timestamptz	no nulo, por defecto now()	

`USER_BLOCKS`

Tabla 12.8. Columnas de user_blocks .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`blocker_id`	uuid	FK → `users`, no nulo	
`blocked_id`	uuid	FK → `users`, no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	

`USER_REPORTS`

Tabla 12.9. Columnas de user_reports .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`reporter_id`	uuid	FK → `users`, no nulo	
`reported_id`	uuid	FK → `users`, no nulo	
`reason`	enum ReportReason	no nulo	
`details`	text	nulo	
`status`	enum ReportStatus	no nulo, por defecto pending	
`created_at`	timestamptz	no nulo, por defecto now()	

`USER_REPORT_ENTRIES`

Tabla 12.10. Columnas de user_report_entries .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	

Columna	Tipo	Restricciones	Descripción
report_id	uuid	FK → user_reports, no nulo	
details	text	nulo	
created_at	timestamptz	no nulo, por defecto now()	

USER_REPORT_PHOTOS

Tabla 12.11. Columnas de user_report_photos .

Columna	Tipo	Restricciones	Descripción
id	uuid	PK, no nulo	
report_id	uuid	FK → user_reports, no nulo	
entry_id	uuid	FK → user_report_entries, nulo	
s3_key	text	no nulo	
created_at	timestamptz	no nulo, por defecto now()	

USERS

Tabla 12.12. Columnas de users .

Columna	Tipo	Restricciones	Descripción
id	uuid	PK, no nulo	
email	text	único, no nulo	
phone	text	único, no nulo	
first_name	text	no nulo	
last_name	text	no nulo	
password_hash	text	no nulo	
dni	text	nulo	
bio	text	nulo	bio de texto libre del perfil, nullable sin backfill – "" nunca se persiste, se resuelve a null en users.service.ts.
terms_accepted_at	timestamptz	nulo	"firma electrónica" de aceptación al registrarse.
terms_version	text	nulo	
privacy_accepted_at	timestamptz	nulo	
privacy_version	text	nulo	

Columna	Tipo	Restricciones	Descripción
`phone_verified`	bool	no nulo, por defecto false	
`email_verified`	bool	no nulo, por defecto false	el email pasa a ser un dato verificado (precondición de MOVO-64, recuperación de contraseña por email) y no solo un dato de contacto.
`email_verified_at`	timestamptz	nulo	
`photo_url`	text	nulo	
`kyc_status_identity`	enum KycStatus	no nulo, por defecto not_started	
`kyc_status_license`	enum KycStatus	no nulo, por defecto not_started	
`status`	enum AccountStatus	no nulo, por defecto active	
`banned_until`	timestamptz	nulo	
`deleted_at`	timestamptz	nulo	la fija `anonymizeAndDelete`. NULL en las bajas anteriores (sin backfill).
`birthdate`	date	nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	El trigger users.update_updated_at_column() de la migración original ya mantiene esta columna al día en cada UPDATE – @updatedAt hace que Prisma...

`USER_ROLES`

Tabla 12.13. Columnas de user_roles .

Columna	Tipo	Restricciones	Descripción
`user_id`	uuid	FK → `users`, no nulo	
`role`	enum UserRole	no nulo	
`granted_at`	timestamptz	no nulo, por defecto now()	

`KYC_VERIFICATION`

Tabla 12.14. Columnas de kyc_verification .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	FK → `users`, no nulo	
`verification_type`	enum VerificationType	no nulo	
`provider`	text	no nulo	
`external_session_id`	text	único, no nulo	`session_id` de Didit – único: un intento por sesión, y es la clave con la que el webhook (AC7) encuentra a qué intento corresponde un evento...
`status`	enum KycStatus	no nulo	
`requested_at`	timestamptz	no nulo, por defecto now()	
`resolved_at`	timestamptz	nulo	
`raw_decision`	jsonb	nulo	Payload del webhook de Didit REDACTADO (whitelist de campos seguros – nunca imagen de documento ni dato biométrico, AC9).

`ADDRESS`

Tabla 12.15. Columnas de address .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	FK → `users`, no nulo	
`label`	text	nulo	
`is_default`	bool	no nulo, por defecto false	
`street`	text	no nulo	
`street_number`	text	no nulo	
`floor_apartment`	text	nulo	
`city`	text	no nulo	
`province`	text	no nulo	
`postal_code`	text	no nulo	

Columna	Tipo	Restricciones	Descripción
`country`	text	no nulo	No se pide en el wizard (la app es solo Argentina hoy, mismo criterio que el regex de teléfono en auth.schema.ts), se hardcodea "AR" al crear.
`lat`	numeric	no nulo	Poblados por el paso de mapa/ geocoding del wizard de registro (MOVO-73), a diferencia del resto de las columnas, el DER no los marca nullable, así...
`long`	numeric	no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`PUSH_TOKEN`

Tabla 12.16. Columnas de push_token .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	FK → `users`, no nulo	
`expo_push_token`	text	no nulo	
`device_id`	text	no nulo	
`platform`	enum PushPlatform	no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`DRIVERS_LICENSE`

Tabla 12.17. Columnas de drivers_license .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	FK → `users`, único, no nulo	
`kyc_verification_id`	uuid	FK → `kyc_verification`, no nulo	

Columna	Tipo	Restricciones	Descripción
`expiration_date`	date	nulo	Fecha de vencimiento impresa en el documento – extraída de `decision.id_verifications[0].expiration_date` (feature “ID Verification” de Didit,...
`status`	enum DriversLicenseStatus	no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`ACCOUNT_SANCTIONS`

Tabla 12.18. Columnas de account_sanctions .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	FK → `users`, no nulo	
`type`	enum AccountSanctionType	no nulo	
`reason`	text	no nulo	
`internal_note`	text	nulo	
`started_at`	timestamptz	no nulo, por defecto now()	
`ends_at`	timestamptz	nulo	
`applied_by`	uuid	nulo	
`lifted_at`	timestamptz	nulo	
`lifted_by`	uuid	nulo	
`lift_reason`	text	nulo	
`closed_cause`	enum AccountSanctionClosedCause	nulo	

12.1.2 Envíos, viajes y soporte (esquema `shipments`)

El esquema contiene 23 tablas y 20 tipos enumerados.

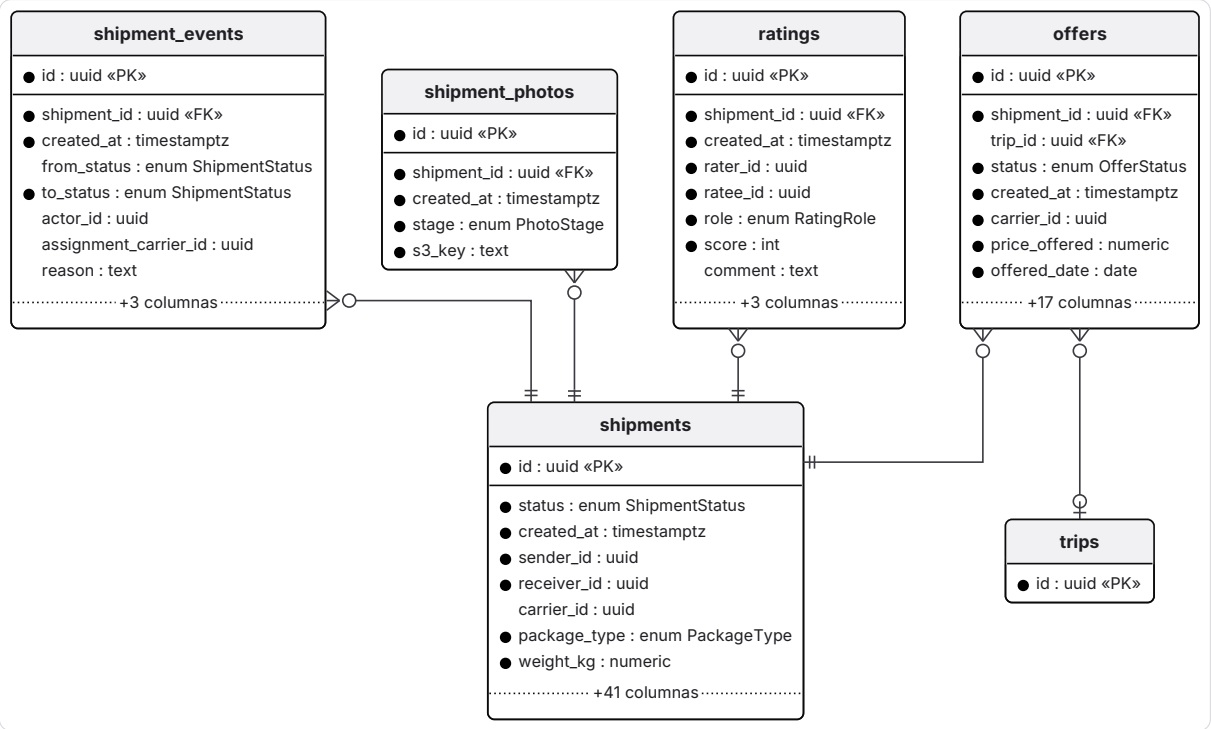


Figura 12.5. DER: Envíos, ofertas, eventos y calificaciones.

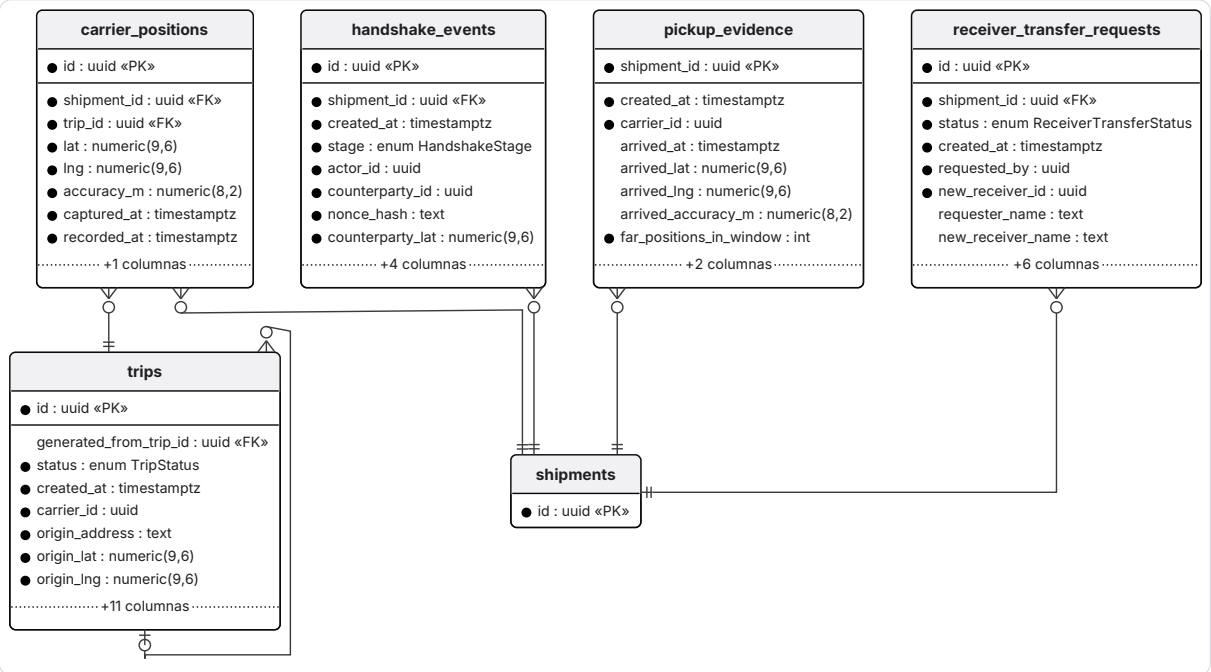


Figura 12.6. DER: Viajes, custodia y seguimiento.

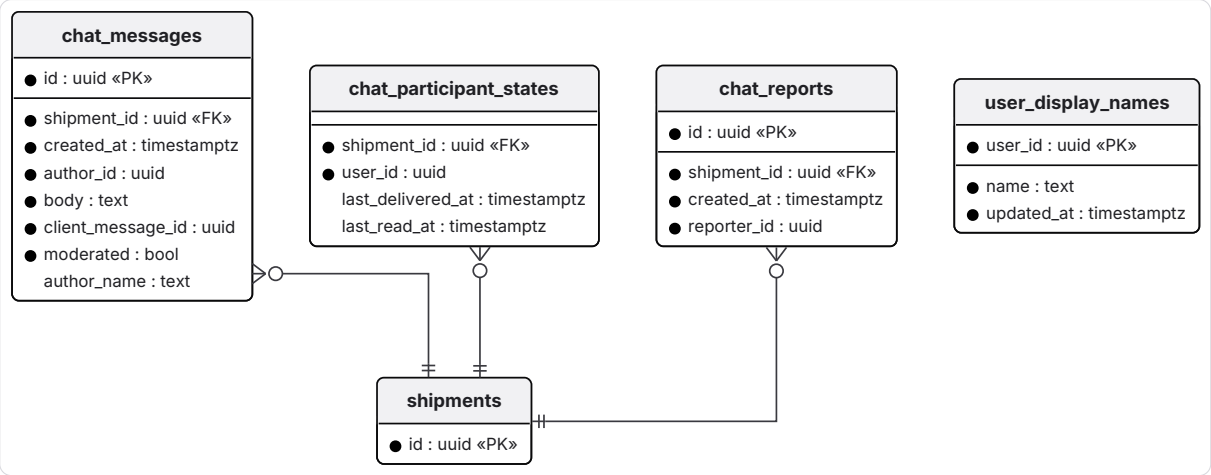


Figura 12.7. DER: Chat del envío.

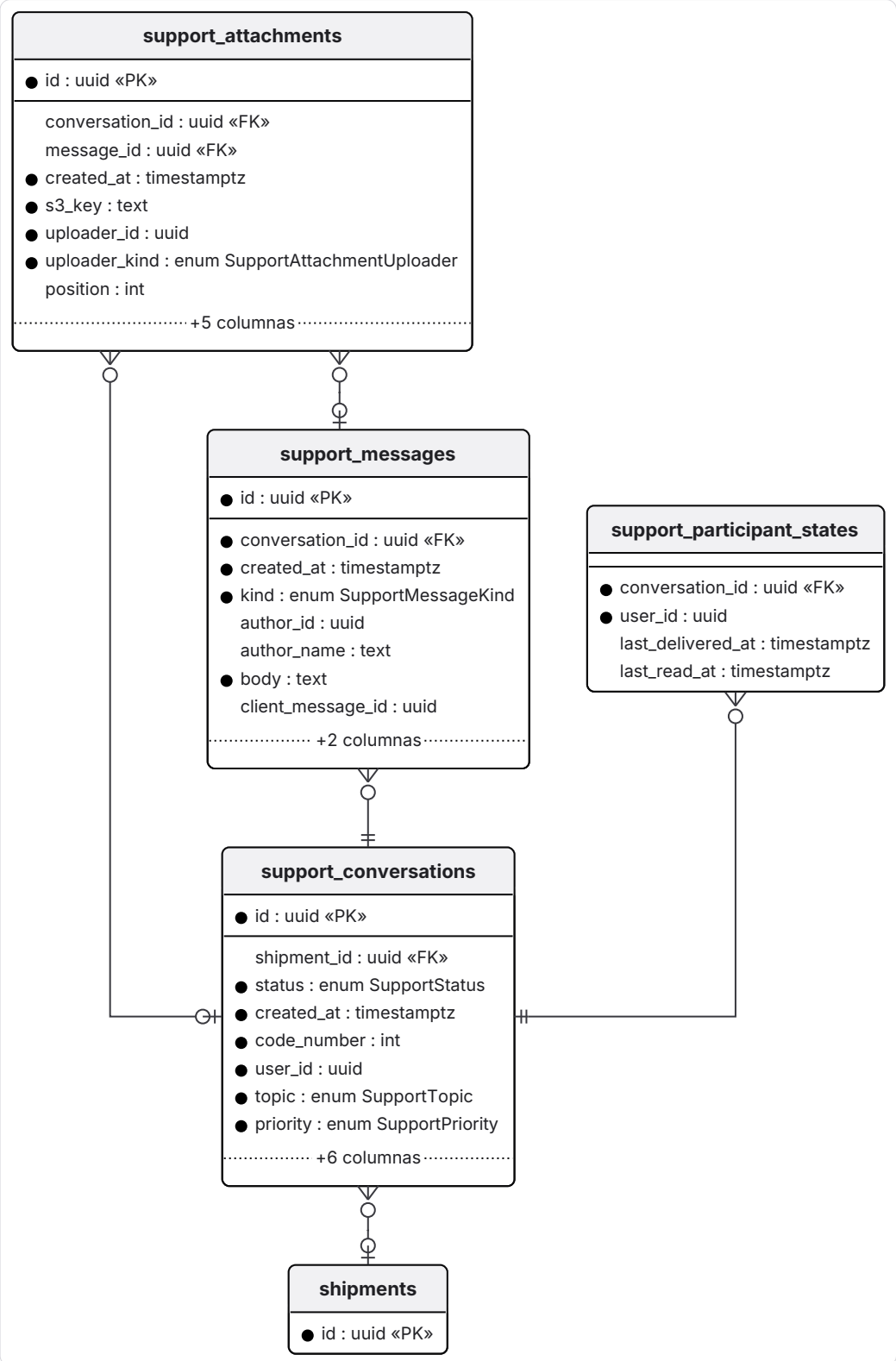


Figura 12.8. DER: Soporte.

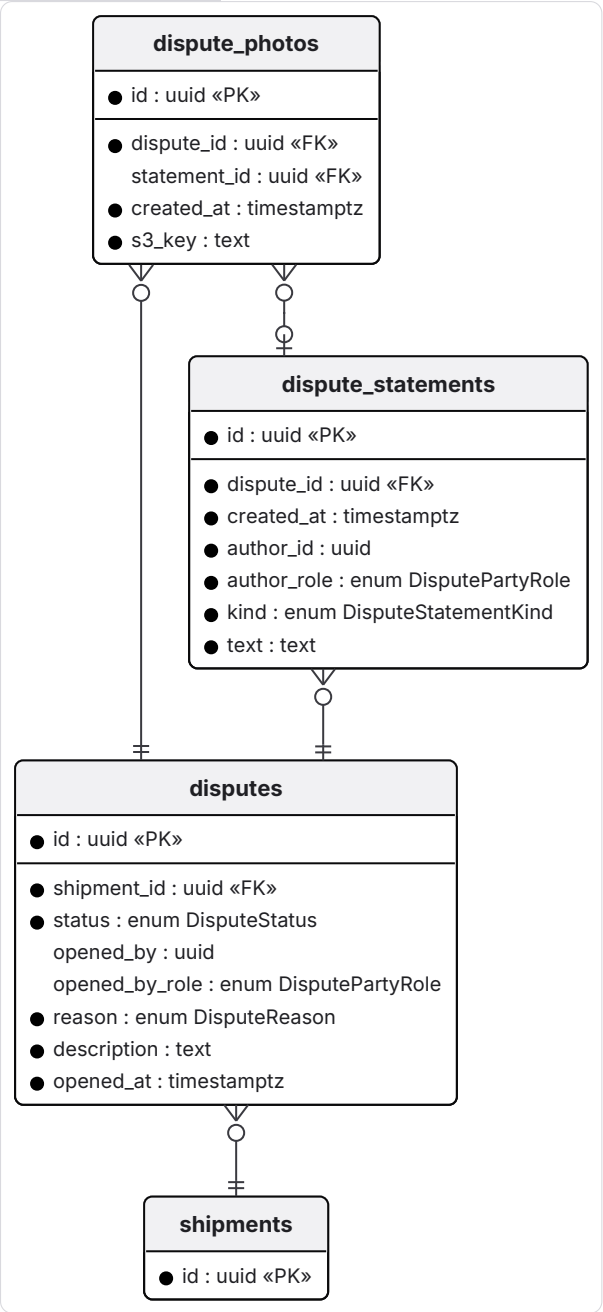


Figura 12.9. DER: Disputas.

Tablas

Tabla 12.19. Tablas del esquema shipments .

Tabla	Propósito	Columnas
`shipments`	carrier_id/urgent/agreedPriceArs/paymentMethod/ deliveredAt/ lastStatusChangedAt no estaban en el AC literal pero sí en el DER, se agregan ahora para no migrar de nuevo pronto cuando...	49
`shipment_events`	historial append-only de transiciones, nunca se actualiza ni se borra una fila ya insertada.	11
`shipment_photos`	solo la referencia a S3, nunca la imagen (ADR-007).	5

Tabla	Propósito	Columnas
`offers`	hermano de Shipment (MOVO-79/104/105) para la entidad Offer.	25
`ratings`	calificación post-entrega de una parte del envío hacia otra.	11
`handshake_events`	log append-only de handshakes de custodia CONFIRMADOS – nunca se actualiza ni se borra una fila, mismo criterio que ShipmentEvent (sin trigger de DB, por convención de repositorio:...	12
`carrier_positions`	traza de posiciones GPS del transportista durante un envío `in_transit`. Append-only (AC9): ningún código de este servicio hace UPDATE de una fila ya insertada, solo la purga periódica...	9
`pickup_evidence`	evidencia GPS pasiva del retiro de un envío `assigned`, sin guardar la traza (la traza solo se guarda con custodia, DD-023).	10
`trips`	viaje declarado por el transportista para matching geométrico de paquetes compatibles por corredor (MOVO-50) y gestión de rutas (“Mis viajes”, MOVO-18/162).	19
`pricing_game_sessions`	Juego de precios de la feria (movo-institucional, /juegos/precios): una fila por partida.	41
`route_game_sessions`	Juego del optimizador de la feria (movo-institucional /juegos/optimizador): una fila por partida.	31
`receiver_transfer_requests`	–	14
`chat_messages`	mensajes del chat de un envío.	8
`chat_participant_states`	hasta dónde recibió y leyó el chat cada integrante de un envío (los tildes del mensaje y el contador de no leídos).	4
`chat_reports`	“Reportar conversación”. Una por persona y envío; la revisa el equipo de Movo.	4
`support_conversations`	–	14
`support_messages`	Solo se agregan, nunca se editan ni se borran (evidencia).	10
`support_attachments`	imagen adjunta a un mensaje de soporte (S3 privado, ADR-007).	13
`support_participant_states`	Hasta dónde recibió y leyó la conversación el usuario (contador de no leídos).	4
`user_display_names`	nombre visible de cada usuario que participó de un envío, para el listado y la búsqueda del panel sin depender de svc-users.	3
`disputes`	Una sola disputa `open` por envío: índice único parcial agregado a mano en la migración (`disputes_shipment_open_key`), Prisma no los representa.	8
`dispute_statements`	Descargo de una parte que no abrió (`statement`) o ampliación única de quien abrió (`amendment`).	7
`dispute_photos`	Solo la referencia a S3 (ADR-007).	5

Tipos enumerados

Tabla 12.20. Enumerados del esquema shipments .

Tipo	Valores
`PackageType`	`letter_document` , `standard_package` , `fragile_item`
`ShipmentStatus`	`awaiting_receiver_confirmation` , `rejected_by_receiver` , `published` , `assignment_pending` , `assigned_unfunded` , `assigned` , `in_transit` , `delivered` , `completed` , `cancelled` , `disputed`
`PhotoStage`	`creation` , `pickup` , `delivery`
`RatingRole`	`sender` , `carrier` , `receiver`
`OfferStatus`	`pending` , `accepted` , `rejected` , `withdrawn` , `expired` , `superseded` , `shipment_cancelled` , `assignment_lapsed` , `carrier_withdrawn`
`HandshakeStage`	`pickup` , `delivery`
`CancellationPenaltyStatus`	`pending` , `captured` , `debt`
`TripStatus`	`declared` , `active` , `cancelled` , `expired` , `completed`
`ShipmentEventKind`	`normal` , `manual` , `forced`
`PickupMissedAttribution`	`carrier` , `none`
`ReceiverTransferStatus`	`pending_new_receiver` , `completed` , `rejected_by_new_receiver` , `expired` , `cancelled`
`SupportTopic`	`shipment` , `payments` , `account` , `carrier` , `security`
`SupportStatus`	`open` , `awaiting_user` , `resolved`
`SupportPriority`	`urgent` , `high` , `normal` , `low`
`SupportMessageKind`	`user` , `agent` , `note` , `system`
`SupportAttachmentUploader`	`user` , `staff`
`DisputeReason`	`damaged` , `not_delivered` , `wrong_delivery` , `other` , `transit_anomaly` , `pickup_no_show`
`DisputeStatus`	`open` , `resolved`
`DisputePartyRole`	`sender` , `receiver` , `carrier`
`DisputeStatementKind`	`statement` , `amendment`

Diccionario de datos

`SHIPMENTS`

Tabla 12.21. Columnas de shipments .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`sender_id`	uuid	no nulo	Referencias lógicas a users.users, sin FK real (AC5, ADR-003).

Columna	Tipo	Restricciones	Descripción
`receiver_id`	uuid	no nulo	
`carrier_id`	uuid	nulo	
`package_type`	enum PackageType	no nulo	
`weight_kg`	numeric	no nulo	
`length_cm`	numeric	no nulo	
`width_cm`	numeric	no nulo	
`height_cm`	numeric	no nulo	
`oversized`	bool	no nulo, por defecto false	el emisor eligió “Más de 300 cm” en alguno de los lados.
`description`	text	nulo	
`urgent`	bool	no nulo, por defecto false	
`pickup_address`	text	no nulo	
`pickup_lat`	numeric(9,6)	no nulo	
`pickup_lng`	numeric(9,6)	no nulo	
`pickup_city`	text	nulo	ciudad del punto, del reverse geocoding de svc-users al crear (best-effort).
`delivery_address`	text	no nulo	
`delivery_lat`	numeric(9,6)	no nulo	
`delivery_lng`	numeric(9,6)	no nulo	
`delivery_city`	text	nulo	
`pickup_date`	date	no nulo	
`pickup_time_window_start`	time	no nulo	
`pickup_time_window_end`	time	no nulo	
`original_pickup_date`	date	nulo	ventana que pidió el emisor, guardada por `acceptOffer` antes de pisar `pickupDate` / `pickupTimeWindow*` con la acordada.
`original_pickup_time_window_start`	time	nulo	

Columna	Tipo	Restricciones	Descripción
original_pickup_time_window_end	time	nulo	
suggested_price_ars	numeric	nulo	nullable = “precio a estimar” (AC6), cuando movo-svc-pricing-logistics no respondió o faltaban datos al crear el envío.
calculation_method	text	nulo	Versión del algoritmo que produjo suggestedPriceArs (PriceCalculationMethod de @movo/shared, ej.
high_demand	bool	nulo	true si la cotización llevó recargo por alta demanda.
agreed_price_ars	numeric	nulo	
payment_method	text	nulo	DER: “tipado como enum en el Miro pero sin valores definidos”, se deja texto libre a propósito, no se inventan valores que nadie definió.
estimated_delivery_date	date	nulo	entrega estimada (día + franja horaria) de la oferta ganadora, copiada acá al aceptarla (offer-repository.ts#acceptOffer) – mismo criterio que...
estimated_delivery_time_window_start	varchar(8)	nulo	
estimated_delivery_time_window_end	varchar(8)	nulo	
status	enum ShipmentStatus	no nulo, por defecto awaiting_receiver_confirmation	
carrier_withdrawal_pending	bool	no nulo, por defecto false	Bloquea las transiciones mientras se libera el hold de la asignación que se retira.
carrier_withdrawal_penalty_ars	numeric(12, 2)	nulo	
last_status_changed_at	timestamptz	nulo	
delivered_at	timestamptz	nulo	
receiver_confirmation_deadline	timestamptz	nulo	deadline de confirmación del receptor (createdAt + RECEIVER_CONFIRMATION_TIMEOUT_HOURS).
receiver_redesignation_deadline	timestamptz	nulo	plazo del emisor para elegir otro receptor tras un rechazo, seteado al pasar a rejected_by_receiver.

Columna	Tipo	Restricciones	Descripción
transit_anomaly_flagged_at	timestamptz	nulo	instante en que el barrido de in_transit anómalo marcó el envío para revisión (no lo cancela).
transit_anomaly_warned_at	timestamptz	nulo	instante en que se le avisó al transportista que el envío marcado pasa a disputa si no confirma la entrega.
chat_started_at	timestamptz	nulo	cuándo empezó la última asignación (acceptOffer).
delivery_handshake_started_at	timestamptz	nulo	instante en que el transportista generó por primera vez el QR de entrega.
cancellation_penalty_ars	numeric(12, 2)	nulo	penalidad que el emisor confirmó al cancelar un assigned con menos de 12 h, y qué pasó con ella.
cancellation_penalty_status	enum CancellationPenaltyStatus	nulo	
created_at	timestamptz	no nulo, por defecto now()	
updated_at	timestamptz	no nulo	Mantenida por trigger de Postgres (shipments.update_updated_at_column()), mismo patrón que users.users, @updatedAt hace que Prisma también la pise...

SHIPMENT_EVENTS

Tabla 12.22. Columnas de shipment_events .

Columna	Tipo	Restricciones	Descripción
id	uuid	PK, no nulo	
shipment_id	uuid	FK → shipments , no nulo	
from_status	enum ShipmentStatus	nulo	NULL solo en el evento de creación (no hay “from” antes del primer estado).
to_status	enum ShipmentStatus	no nulo	
actor_id	uuid	nulo	Referencia lógica (sender/receiver/carrier/admin), sin FK, nullable porque una transición futura disparada por el sistema (ej.
assignment_carrier_id	uuid	nulo	Transportista cuya asignación terminó al volver a published por falta de pago.

Columna	Tipo	Restricciones	Descripción
`reason`	text	nulo	
`attribution`	enum PickupMissedAttribution	nulo	solo en la cancelación del barrido de retiro no realizado.
`kind`	enum ShipmentEventKind	no nulo, por defecto normal	cambio hecho desde el panel.
`actor_name`	text	nulo	
`created_at`	timestamptz	no nulo, por defecto now()	

`SHIPMENT_PHOTOS`

Tabla 12.23. Columnas de shipment_photos .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`shipment_id`	uuid	FK → `shipments` , no nulo	
`stage`	enum PhotoStage	no nulo	
`s3_key`	text	no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	

`OFFERS`

Tabla 12.24. Columnas de offers .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`shipment_id`	uuid	FK → `shipments` , no nulo	
`carrier_id`	uuid	no nulo	Referencia lógica a users.users, sin FK real (AC4, ADR-003), mismo criterio que Shipment.senderId/receiverId/carrierId.
`price_offered`	numeric	no nulo	AC3: numeric, nunca float.
`offered_date`	date	no nulo	
`offered_pickup_time_window_start`	varchar(8)	nulo	franja horaria alternativa de retiro, solo cuando el transportista propone un día/horario distinto al pedido por el emisor (`offeredDate` distinto...

Columna	Tipo	Restricciones	Descripción
`offered_pickup_time_window_end`	varchar(8)	nulo	
`message`	text	nulo	AC2: mensaje opcional del transportista.
`carrier_rating_at_offer`	numeric	nulo	AC2: snapshot del transportista al momento de ofertar, evita el N+1 al listar (MOVO-17) y es el dato correcto para una disputa futura (“qué...
`carrier_name_at_offer`	text	nulo	
`sender_name_at_offer`	text	nulo	snapshot simétrico del EMISOR al momento de ofertar – mismo criterio que el snapshot del transportista de arriba (congelado, no lectura en vivo).
`sender_verified_at_offer`	bool	nulo	
`sender_rating_at_offer`	numeric	nulo	
`trip_id`	uuid	FK → `trips`, nulo	
`estimated_delivery_date`	date	nulo	entrega estimada (día + franja horaria) declarada por el transportista al ofertar – opcional, complementa offeredDate (que es de RETIRO, no de...
`estimated_delivery_time_window_start`	varchar(8)	nulo	
`estimated_delivery_time_window_end`	varchar(8)	nulo	
`status`	enum OfferStatus	no nulo, por defecto pending	
`expires_at`	timestamptz	nulo	AC2: sostiene la expiración perezosa (AC11).
`created_at`	timestamptz	no nulo, por defecto now()	
`responded_at`	timestamptz	nulo	
`viewed_at_by_sender`	timestamptz	nulo	instante en que el EMISOR vio esta oferta por primera vez (nunca “última vez”, ver deriveEffectiveOfferStatus/ AC1) – se completa con un updateMany...
`carrier_withdrawn_at`	timestamptz	nulo	el transportista se bajó del envío ya asignado (`carrier_withdrawn`).
`withdrawal_penalty_ars`	numeric(12, 2)	nulo	

Columna	Tipo	Restricciones	Descripción
withdrawal_penalty_status	enum CancellationPenaltyStatus	nulo	

RATINGS

Tabla 12.25. Columnas de ratings .

Columna	Tipo	Restricciones	Descripción
id	uuid	PK, no nulo	
shipment_id	uuid	FK → shipments , no nulo	
rater_id	uuid	no nulo	Referencias lógicas a users.users, sin FK real (mismo criterio que Shipment.senderId/receiverId/carrierId, ADR-003).
ratee_id	uuid	no nulo	
role	enum RatingRole	no nulo	Rol del CALIFICADO dentro de este envío en particular – no un rol de cuenta.
score	int	no nulo	AC4: entero 1..5, sin medias estrellas – validado en el schema de la ruta (@movo/shared no define un enum acá, es un número simple).
comment	text	nulo	
punctuality_score	int	nulo	sub-scores opcionales (1..5, validados en el schema de la ruta).
care_score	int	nulo	
communication_score	int	nulo	
created_at	timestamptz	no nulo, por defecto now()	

HANDSHAKE_EVENTS

Tabla 12.26. Columnas de handshake_events .

Columna	Tipo	Restricciones	Descripción
id	uuid	PK, no nulo	
shipment_id	uuid	FK → shipments , no nulo	
stage	enum HandshakeStage	no nulo	

Columna	Tipo	Restricciones	Descripción
`actor_id`	uuid	no nulo	actor_id = quien confirma la custodia (llama a /confirm); counterparty_id = quien generó el QR (el cedente, llamó a /generate).
`counterparty_id`	uuid	no nulo	
`nonce_hash`	text	no nulo	SHA-256 hex del nonce – nunca se persiste el nonce en texto plano, ya cumplió su función de un solo uso en Redis antes de llegar acá.
`counterparty_lat`	numeric(9,6)	no nulo	
`counterparty_lng`	numeric(9,6)	no nulo	
`actor_lat`	numeric(9,6)	no nulo	
`actor_lng`	numeric(9,6)	no nulo	
`distance_m`	numeric(8,2)	no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	

`CARRIER_POSITIONS`

Tabla 12.27. Columnas de carrier_positions .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`shipment_id`	uuid	FK → `shipments`, no nulo	
`trip_id`	uuid	FK → `trips`, no nulo	asociar al viaje (Trip) para agrupar y controlar cadencia por viaje (VRPTW).
`lat`	numeric(9,6)	no nulo	
`lng`	numeric(9,6)	no nulo	
`accuracy_m`	numeric(8,2)	no nulo	Precisión reportada por el dispositivo (metros) – mismo formato que `HandshakeEvent.distanceM`.
`captured_at`	timestamptz	no nulo	Timestamp del DISPOSITIVO (AC1) – puede ser anterior a `recordedAt` si el mobile encoló posiciones sin red y las mandó después; ahí es donde la...

Columna	Tipo	Restricciones	Descripción
`recorded_at`	timestamptz	no nulo, por defecto now()	Timestamp del SERVIDOR al persistir – ancla real de la cadencia de ~45s y de la retención (AC6/AC8), nunca `capturedAt` (no confiable, viene del...
`mocked`	bool	nulo	flag `mocked` de expo-location (solo Android; NULL si el dispositivo no lo informa).

`PICKUP_EVIDENCE`

Tabla 12.28. Columnas de pickup_evidence .

Columna	Tipo	Restricciones	Descripción
`shipment_id`	uuid	PK, FK → `shipments`, no nulo	
`carrier_id`	uuid	no nulo	Transportista de la asignación a la que pertenece la evidencia: si el envío se reasigna, la fila de otro transportista no se lee ni se suma (se...
`arrived_at`	timestamptz	nulo	Timestamp del DISPOSITIVO de la primera posición dentro del radio de llegada.
`arrived_lat`	numeric(9,6)	nulo	
`arrived_lng`	numeric(9,6)	nulo	
`arrived_accuracy_m`	numeric(8,2)	nulo	
`far_positions_in_window`	int	no nulo, por defecto 0	
`gray_positions_in_window`	int	no nulo, por defecto 0	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`TRIPS`

Tabla 12.29. Columnas de trips .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`carrier_id`	uuid	no nulo	Referencia lógica a users.users, sin FK real (ADR-003).
`origin_address`	text	no nulo	

Columna	Tipo	Restricciones	Descripción
origin_lat	numeric(9,6)	no nulo	
origin_lng	numeric(9,6)	no nulo	
destination_address	text	no nulo	
destination_lat	numeric(9,6)	no nulo	
destination_lng	numeric(9,6)	no nulo	
departure_at	timestamptz	no nulo	
vehicle_type	text	no nulo	
status	enum TripStatus	no nulo, por defecto declared	default pasa de active a declared.
cancelled_at	timestamptz	nulo	
started_at	timestamptz	nulo	cuándo pasó a active (lo escribe start()).
recurrence_weekdays	int[]	no nulo, por defecto []	recurrencia como atributo del trip.
recurrence_until	date	nulo	Inclusive, calendario argentino.
series_id	uuid	nulo	El primer trip de la serie usa su propio id; se conserva al apagar la recurrencia.
generated_from_trip_id	uuid	FK → trips , único, nulo	UNIQUE: cada trip tiene a lo sumo un sucesor.
created_at	timestamptz	no nulo, por defecto now()	
updated_at	timestamptz	no nulo	

PRICING_GAME_SESSIONS

Tabla 12.30. Columnas de pricing_game_sessions .

Columna	Tipo	Restricciones	Descripción
id	uuid	PK, no nulo	
event_tag	varchar(64)	no nulo	
device_id	varchar(64)	nulo	
started_at	timestamptz	no nulo	
ended_at	timestamptz	no nulo	
duration_sec	int	no nulo	

Columna	Tipo	Restricciones	Descripción
`completed`	bool	no nulo	
`last_screen`	varchar(32)	no nulo	
`origin_name`	varchar(120)	nulo	
`origin_province`	varchar(120)	nulo	
`origin_lat`	numeric(9,6)	nulo	
`origin_lng`	numeric(9,6)	nulo	
`destination_name`	varchar(120)	nulo	
`destination_province`	varchar(120)	nulo	
`destination_lat`	numeric(9,6)	nulo	
`destination_lng`	numeric(9,6)	nulo	
`package_preset`	varchar(16)	nulo	
`package_type`	enum PackageType	nulo	
`weight_kg`	numeric(6,2)	nulo	
`quote_id`	uuid	nulo	Cotización.
`quote_verified`	bool	no nulo, por defecto false	
`calculation_method`	varchar(32)	nulo	
`suggested_price_ars`	numeric(12,2)	nulo	
`high_demand`	bool	nulo	
`distance_km`	numeric(8,2)	nulo	
`distance_source`	varchar(32)	nulo	
`fuel_ars_per_liter`	numeric(10,2)	nulo	
`breakdown`	jsonb	nulo	Desglose completo de pricing (`QuoteBreakdown`), tal cual lo devolvió el servicio.

Columna	Tipo	Restricciones	Descripción
sender_answer	varchar(8)	nulo	Emisor: yes | maybe | no.
sender_alt_choice	varchar(8)	nulo	
sender_wtp_ars	numeric(12, 2)	nulo	
commission_rate	numeric(5,4)	nulo	Transportista: misma escala; courierA ltChoice : +10 | +20 | +30 | custom | none.
courier_earn_ars	numeric(12, 2)	nulo	
courier_answer	varchar(8)	nulo	
courier_alt_choice	varchar(8)	nulo	
courier_wta_ars	numeric(12, 2)	nulo	
uber_estimate_ars	numeric(12, 2)	nulo	
email	varchar(25 4)	nulo	Solo con consentimiento explícito (sorteo); nunca se expone en stats .
user_agent	varchar(51 2)	nulo	
created_at	timestamptz	no nulo, por defecto now()	
updated_at	timestamptz	no nulo	

ROUTE_GAME_SESSIONS

Tabla 12.31. Columnas de route_game_sessions .

Columna	Tipo	Restricciones	Descripción
id	uuid	PK, no nulo	
event_tag	varchar(64)	no nulo	
device_id	varchar(64)	nulo	
started_at	timestamptz	no nulo	
ended_at	timestamptz	no nulo	
scenario_id	varchar(16)	no nulo	
city	varchar(64)	no nulo	
points	jsonb	no nulo	{ start, stops[], end } con nombre, barrio y coordenadas.
stop_count	int	no nulo	
user_order	jsonb	no nulo	

Columna	Tipo	Restricciones	Descripción
`optimal_order`	jsonb	no nulo	
`user_km`	numeric(8,2)	no nulo	
`optimal_km`	numeric(8,2)	no nulo	
`extra_km`	numeric(8,2)	no nulo	
`user_min`	numeric(8,1)	no nulo	
`optimal_min`	numeric(8,1)	no nulo	
`extra_min`	numeric(8,1)	no nulo	
`efficiency_pct`	numeric(5,1)	no nulo	
`tie`	bool	no nulo	
`time_used_sec`	int	no nulo	
`time_limit_sec`	int	nulo	
`timed_out`	bool	no nulo	
`distance_method`	varchar(32)	nulo	
`computed_by`	varchar(8)	no nulo	
`name`	varchar(18)	nulo	Ranking del día: el nombre se muestra en el stand; `inRanking` lo apaga el reset.
`in_ranking`	bool	no nulo, por defecto false	
`ranking_hidden_at`	timestamptz	nulo	Cuándo la sacó el reset: un reenvío posterior no la vuelve a meter al ranking.
`email`	varchar(254)	nulo	Solo con consentimiento explícito (sorteo + newsletter).
`user_agent`	varchar(512)	nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`RECEIVER_TRANSFER_REQUESTS`

Tabla 12.32. Columnas de receiver_transfer_requests .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`shipment_id`	uuid	FK → `shipments`, no nulo	
`requested_by`	uuid	no nulo	Referencias lógicas a users.users, sin FK real (ADR-003).
`new_receiver_id`	uuid	no nulo	
`requester_name`	text	nulo	Snapshots de nombre al pedirla (mismo criterio que carrierNameAtOffer): la línea de tiempo y el banner del receptor original no dependen de...
`new_receiver_name`	text	nulo	
`reason`	text	nulo	
`response_reason`	text	nulo	
`status`	enum ReceiverTransferStatus	no nulo, por defecto pending_new_receiver	
`cancel_reason`	varchar(32)	nulo	`requester` | `delivery_started`, solo con status `cancelled`.
`new_receiver_deadline`	timestamptz	no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`resolved_at`	timestamptz	nulo	
`resolved_by`	uuid	nulo	

`CHAT_MESSAGES`

Tabla 12.33. Columnas de chat_messages .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`shipment_id`	uuid	FK → `shipments`, no nulo	
`author_id`	uuid	no nulo	Referencia lógica a users.users, sin FK real (ADR-003).
`body`	text	no nulo	
`client_message_id`	uuid	no nulo	Lo genera el cliente: reenviar el mismo mensaje (cola offline, reintento) no lo duplica.

Columna	Tipo	Restricciones	Descripción
`moderated`	bool	no nulo, por defecto false	El filtro de lenguaje tapó alguna palabra de `body` (que se guarda ya enmascarado).
`author_name`	text	nulo	Primer nombre del autor al escribir: la lista de chats muestra “Diego: …” sin consultar a svc-users.
`created_at`	timestamptz	no nulo, por defecto now()	

`CHAT_PARTICIPANT_STATES`

Tabla 12.34. Columnas de chat_participant_states .

Columna	Tipo	Restricciones	Descripción
`shipment_id`	uuid	FK → `shipments` , no nulo	
`user_id`	uuid	no nulo	Referencia lógica a users.users, sin FK real (ADR-003).
`last_delivered_at`	timestamptz	nulo	
`last_read_at`	timestamptz	nulo	

`CHAT_REPORTS`

Tabla 12.35. Columnas de chat_reports .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`shipment_id`	uuid	FK → `shipments` , no nulo	
`reporter_id`	uuid	no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	

`SUPPORT_CONVERSATIONS`

Tabla 12.36. Columnas de support_conversations .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`code_number`	int	único, no nulo, por defecto autoincrement()	Número del código de display `SOP-NNNN` : la secuencia `support_conversation_code_seq` (arranca en 1001).
`user_id`	uuid	no nulo	Referencias lógicas a users.users, sin FK real (ADR-003).

Columna	Tipo	Restricciones	Descripción
`topic`	enum SupportTopic	no nulo	
`status`	enum SupportStatus	no nulo, por defecto open	
`priority`	enum SupportPriority	no nulo, por defecto normal	
`assigned_to`	uuid	nulo	
`shipment_id`	uuid	FK → `shipments`, nulo	Sin onDelete cascade: la conversación es evidencia y sobrevive al envío (retención, DD-051).
`tags`	text[]	no nulo, por defecto []	
`created_at`	timestamptz	no nulo, por defecto now()	
`last_message_at`	timestamptz	no nulo, por defecto now()	
`last_user_message_at`	timestamptz	no nulo, por defecto now()	Último mensaje del usuario: base del “esperando hace X min” del panel.
`last_agent_read_at`	timestamptz	nulo	Hasta dónde leyó un agente (los tildes “leído” del usuario).
`resolved_at`	timestamptz	nulo	

`SUPPORT_MESSAGES`

Tabla 12.37. Columnas de support_messages .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`conversation_id`	uuid	FK → `support_conversations`, no nulo	
`kind`	enum SupportMessageKind	no nulo	
`author_id`	uuid	nulo	Referencia lógica a users.users (sin FK real, ADR-003).
`author_name`	text	nulo	Nombre de pila del agente al escribir (“Lucía”): el usuario lo ve sin consultar a svc-users.
`body`	text	no nulo	

Columna	Tipo	Restricciones	Descripción
`client_message_id`	uuid	nulo	Lo genera el cliente (usuario o panel): reenviar el mismo mensaje no lo duplica.
`moderated`	bool	no nulo, por defecto false	El filtro de lenguaje tapó alguna palabra de `body` (solo mensajes del usuario).
`attachment_count`	int	no nulo, por defecto 0	cuántos adjuntos se asociaron al crearlo (se fija en la misma transacción y no cambia).
`created_at`	timestamptz	no nulo, por defecto now()	

`SUPPORT_ATTACHMENTS`

Tabla 12.38. Columnas de support_attachments .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`uploader_id`	uuid	no nulo	Referencia lógica a users.users (sin FK real, ADR-003).
`uploader_kind`	enum SupportAttachmentUploader	no nulo	
`conversation_id`	uuid	FK → `support_conversations`, nulo	Nulo solo mientras es de la primera consulta del usuario y todavía no se envió.
`message_id`	uuid	FK → `support_messages` , nulo	
`position`	int	nulo	Orden dentro del mensaje (el de `attachmentIds`).
`s3_key`	text	único, no nulo	
`content_type`	text	no nulo	
`size_bytes`	int	no nulo	
`width`	int	nulo	
`height`	int	nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`deleting_at`	timestamptz	nulo	El sweep lo marcó para borrar: ya no se puede asociar.

`SUPPORT_PARTICIPANT_STATES`

Tabla 12.39. Columnas de support_participant_states .

Columna	Tipo	Restricciones	Descripción
`conversation_id`	uuid	FK → `support_conversations` , no nulo	
`user_id`	uuid	no nulo	
`last_delivered_at`	timestamptz	nulo	
`last_read_at`	timestamptz	nulo	

`USER_DISPLAY_NAMES`

Tabla 12.40. Columnas de user_display_names .

Columna	Tipo	Restricciones	Descripción
`user_id`	uuid	PK, no nulo	
`name`	text	no nulo	
`updated_at`	timestamptz	no nulo, por defecto now()	

`DISPUTES`

Tabla 12.41. Columnas de disputes .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`shipment_id`	uuid	FK → `shipments` , no nulo	
`opened_by`	uuid	nulo	NULL: la abrió un barrido del sistema (`transit_anomaly` , `pickup_no_show`).
`opened_by_role`	enum DisputePartyRole	nulo	
`reason`	enum DisputeReason	no nulo	
`description`	text	no nulo	
`status`	enum DisputeStatus	no nulo, por defecto open	
`opened_at`	timestamptz	no nulo, por defecto now()	

`DISPUTE_STATEMENTS`

Tabla 12.42. Columnas de dispute_statements .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`dispute_id`	uuid	FK → `disputes`, no nulo	
`author_id`	uuid	no nulo	
`author_role`	enum DisputePartyRole	no nulo	
`kind`	enum DisputeStatementKind	no nulo	
`text`	text	no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	

`DISPUTE_PHOTOS`

Tabla 12.43. Columnas de dispute_photos .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`dispute_id`	uuid	FK → `disputes`, no nulo	
`statement_id`	uuid	FK → `dispute_statements`, nulo	
`s3_key`	text	único, no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	

12.1.3 Pagos (esquema `payments`)

El esquema contiene 9 tablas y 11 tipos enumerados.

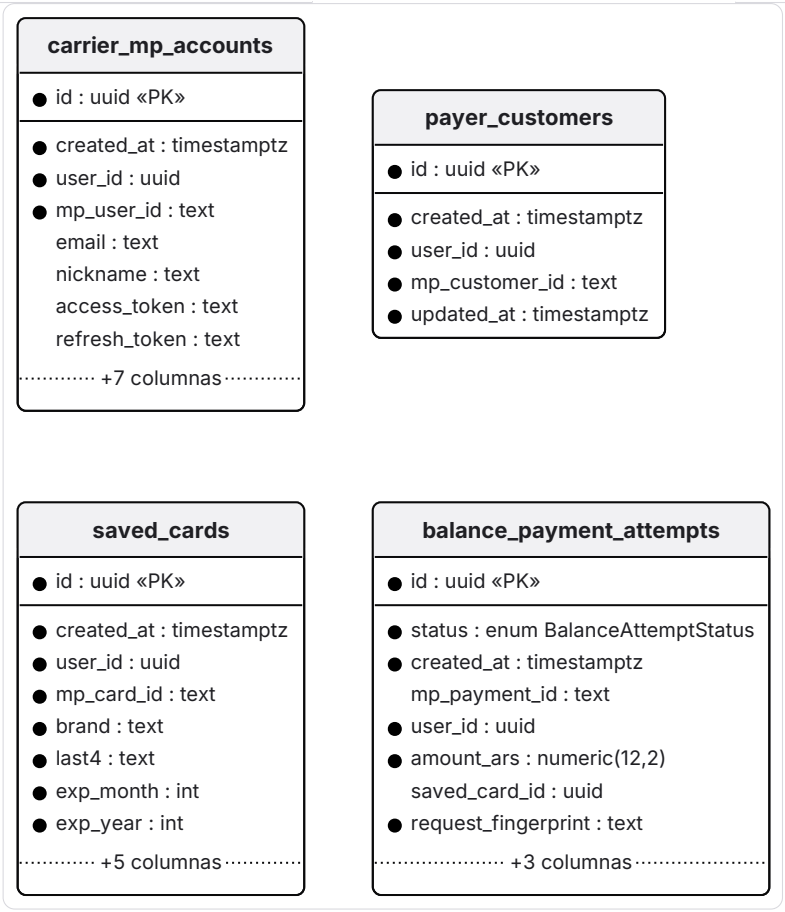


Figura 12.10. DER: Cuentas de cobro, tarjetas y pago de saldo.

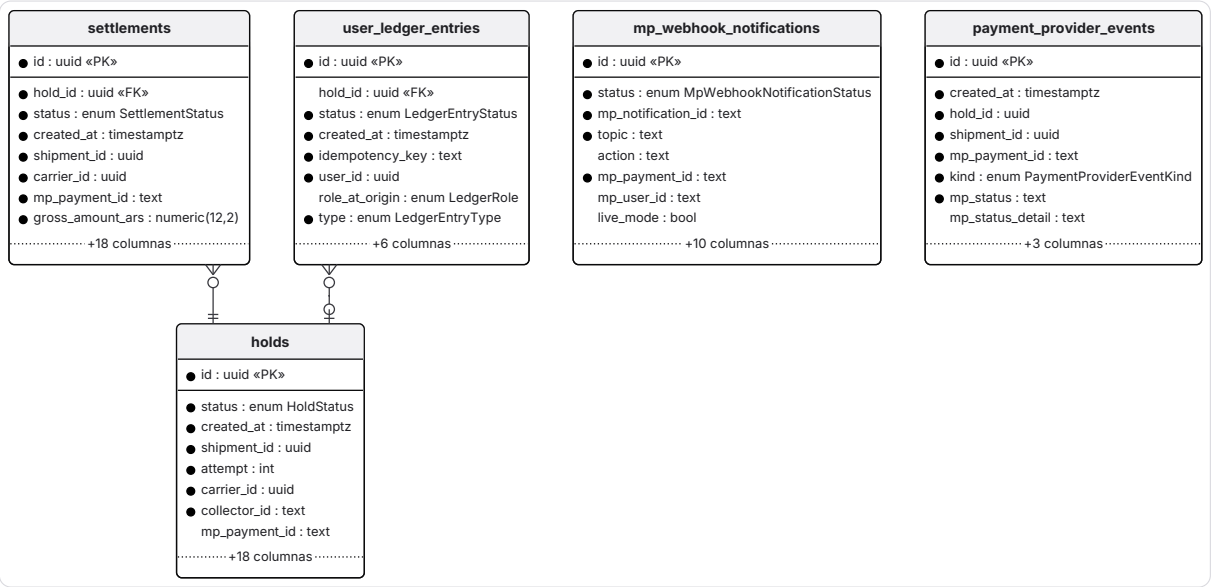


Figura 12.11. DER: Reservas, liquidaciones, eventos y cuenta corriente.

Tablas

Tabla 12.44. Tablas del esquema payments .

Tabla	Propósito	Columnas
carrier_mp_accounts	cuenta de Mercado Pago vinculada por OAuth de cada transportista.	15
holds	reserva de fondos (Auth & Capture, capture: false) de un envío.	26
payer_customers	customer de la cuenta de Movo S.A. en Mercado Pago, uno por emisor.	5
saved_cards	tarjeta guardada del emisor en su customer de Movo.	13
settlements	liquidación de un envío (captura del hold + split).	26
mp_webhook_notifications	inbox de las notificaciones del webhook de Mercado Pago (tópico payment).	18
payment_provider_events	lo que MP hizo sobre un pago por fuera de Movo (contracargo, reembolso, mediación abierta por el pagador).	11
user_ledger_entries	cuenta corriente del USUARIO (no de un rol).	14
balance_payment_attempts	un intento de “Pagar mi saldo” (Checkout API: tarjeta guardada + CVV, pago directo a la cuenta cobradora de Movo).	11

Tipos enumerados

Tabla 12.45. Enumerados del esquema payments .

Tipo	Valores
HoldStatus	creating , in_process , authorized , captured , cancelled , rejected
HoldFailureReason	insufficient_funds , card_rejected , invalid_data , platform_error
SettlementStatus	processing , credited , failed , escalated
SettlementKind	delivery , cancellation_penalty
MpWebhookNotificationStatus	pending , processed , ignored , unverifiable , failed
HoldProviderEventKind	cancelled , expired , rejected
PaymentProviderEventKind	chargeback , refund , mediation
LedgerEntryType	cancellation_penalty , penalty_capture , cash_commission , fee_offset , payment_surcharge , voluntary_payment , manual_adjustment
LedgerEntryStatus	pending , reserved , settled , voided
LedgerRole	sender , carrier
BalanceAttemptStatus	creating , pending , approved , rejected , failed

Diccionario de datos

`CARRIER_MP_ACCOUNTS`

Tabla 12.46. Columnas de carrier_mp_accounts .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	único, no nulo	Referencia lógica a users.users, sin FK real (ADR-003).
`mp_user_id`	text	no nulo	
`email`	text	nulo	
`nickname`	text	nulo	
`access_token`	text	nulo	
`refresh_token`	text	nulo	
`public_key`	text	nulo	Del vendedor: la usa el mobile para tokenizar la tarjeta del emisor (MOVO-209).
`scope`	text	nulo	
`token_expires_at`	timestamptz	nulo	
`connected_at`	timestamptz	no nulo	
`revoked_at`	timestamptz	nulo	
`unlinked_at`	timestamptz	nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`HOLDS`

Tabla 12.47. Columnas de holds .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`shipment_id`	uuid	no nulo	
`attempt`	int	no nulo	
`carrier_id`	uuid	no nulo	
`collector_id`	text	no nulo	
`mp_payment_id`	text	nulo	
`amount_ars`	numeric(12, 2)	no nulo	

Columna	Tipo	Restricciones	Descripción
application_fee_ars	numeric(12, 2)	no nulo	
base_amount_ars	numeric(12, 2)	no nulo	amountArs es lo que se le cobra al emisor (precio + recargo por su deuda) y applicationFeeArs la comisión + la deuda que se cobra.
commission_ars	numeric(12, 2)	no nulo	
debt_offset_ars	numeric(12, 2)	no nulo, por defecto 0	
debt_surcharge_ars	numeric(12, 2)	no nulo, por defecto 0	
status	enum HoldStatus	no nulo	
was_authorized	bool	no nulo, por defecto false	Persiste si MP autorizó alguna vez el intento; cancelled también puede venir de in_process.
status_detail	text	nulo	status_detail crudo de MP (pending_capture, cc_rejected_*, ...).
failure_reason	enum HoldFailure Reason	nulo	
request_fingerprint	text	nulo	sha256 de (card_token, email del pagador, medio de pago) del intento.
saved_card_id	uuid	nulo	tarjeta guardada (saved_cards.id) con la que se creó el hold, sin FK (la fila local se borra soft).
payer_id	uuid	nulo	quién pagó y con qué tarjeta, para listar "lo que pagó cada emisor". payerId es el emisor (payerUserId de MOVO-282), referencia lógica a...
card_last4	varchar(4)	nulo	
card_brand	varchar(32)	nulo	
mp_fee_ars	numeric(12, 2)	nulo	comisión de MP de este pago (mercadopago_fee de charges_details , o de fee_details si el pago ya se capturó) sobre el total que paga el emisor.
expires_at	timestamptz	nulo	
released_at	timestamptz	nulo	la fija un trigger la primera vez que el hold pasa a cancelled (ver la migración hold_released_at).

Columna	Tipo	Restricciones	Descripción
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`PAYER_CUSTOMERS`

Tabla 12.48. Columnas de payer_customers .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	único, no nulo	
`mp_customer_id`	text	único, no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`SAVED_CARDS`

Tabla 12.49. Columnas de saved_cards .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	no nulo	
`mp_card_id`	text	no nulo	
`brand`	text	no nulo	
`last4`	text	no nulo	
`exp_month`	int	no nulo	
`exp_year`	int	no nulo	
`issuer_name`	text	nulo	banco y tipo que devuelve MP al guardar, solo para mostrar.
`payment_type`	text	nulo	
`is_default`	bool	no nulo, por defecto false	
`alias`	text	nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`deleted_at`	timestamptz	nulo	

`SETTLEMENTS`

Tabla 12.50. Columnas de settlements .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`shipment_id`	uuid	único, no nulo	
`hold_id`	uuid	FK → `holds`, único, no nulo	
`carrier_id`	uuid	no nulo	
`mp_payment_id`	text	no nulo	
`gross_amount_ars`	numeric(12, 2)	no nulo	
`mp_fee_ars`	numeric(12, 2)	nulo	
`commission_ars`	numeric(12, 2)	no nulo	
`net_amount_ars`	numeric(12, 2)	nulo	
`debt_offset_ars`	numeric(12, 2)	no nulo, por defecto 0	deuda de la cuenta corriente cobrada en este envío, aparte de `commissionArs` (que queda solo la comisión de Movo): el descuento al transportista y...
`debt_surcharge_ars`	numeric(12, 2)	no nulo, por defecto 0	
`debt_surcharge_fee_ars`	numeric(12, 2)	no nulo, por defecto 0	Parte del `application_fee` retenido que corresponde al recargo (recargo menos la comisión de MP que absorbe Movo): para mostrarle al transportista...
`carrier_gross_amount_ars`	numeric(12, 2)	nulo	Bruto y fee de MP tal como los ve el transportista (sin el recargo del emisor), fijados al acreditar.
`carrier_mp_fee_ars`	numeric(12, 2)	nulo	
`status`	enum SettlementStatus	no nulo	
`kind`	enum SettlementKind	no nulo, por defecto delivery	
`failure_reason`	text	nulo	Motivo si quedó `failed` (ver `SettlementFailureReason` en @movo/shared).

Columna	Tipo	Restricciones	Descripción
status_detail	text	nulo	status_detail crudo de MP al capturar (accredited , ...).
attempts	int	no nulo, por defecto 0	reintentos de la captura.
next_attempt_at	timestamptz	nulo	
last_error	text	nulo	
escalated_at	timestamptz	nulo	
escalation_reason	text	nulo	
captured_at	timestamptz	nulo	
created_at	timestamptz	no nulo, por defecto now()	
updated_at	timestamptz	no nulo	

MP_WEBHOOK_NOTIFICATIONS

Tabla 12.51. Columnas de mp_webhook_notifications .

Columna	Tipo	Restricciones	Descripción
id	uuid	PK, no nulo	
mp_notification_id	text	único, no nulo	
topic	text	no nulo	
action	text	nulo	
mp_payment_id	text	no nulo	
mp_user_id	text	nulo	user_id del body: el vendedor (transportista) dueño del pago.
live_mode	bool	nulo	
status	enum MpWebhookNotificationStatus	no nulo, por defecto pending	
attempts	int	no nulo, por defecto 0	
next_attempt_at	timestamptz	no nulo, por defecto now()	
outcome	text	nulo	Qué hizo el procesamiento (hold_cancelled , chargeback_recorded , ...), para auditar.
last_error	text	nulo	
hold_id	uuid	nulo	

Columna	Tipo	Restricciones	Descripción
`pending_hold_event`	enum HoldProviderEventKind	nulo	
`pending_hold_event_detail`	text	nulo	
`processed_at`	timestamptz	nulo	
`received_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo	

`PAYMENT_PROVIDER_EVENTS`

Tabla 12.52. Columnas de payment_provider_events .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`hold_id`	uuid	no nulo	Referencias lógicas, sin FK (mismo criterio que `MpWebhookNotification.holdId`).
`shipment_id`	uuid	no nulo	
`mp_payment_id`	text	no nulo	
`kind`	enum PaymentProviderEventKind	no nulo	
`mp_status`	text	no nulo	
`mp_status_detail`	text	nulo	
`amount_ars`	numeric(12, 2)	nulo	Lo que MP informa como devuelto (`transaction_amount_refunded`), si viene.
`hold_status`	enum HoldStatus	no nulo	
`alerted_at`	timestamptz	nulo	
`created_at`	timestamptz	no nulo, por defecto now()	

`USER_LEDGER_ENTRIES`

Tabla 12.53. Columnas de user_ledger_entries .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	no nulo	

Columna	Tipo	Restricciones	Descripción
`role_at_origin`	enum LedgerRole	nulo	
`type`	enum LedgerEntryType	no nulo	
`amount_ars`	numeric(12, 2)	no nulo	Con signo, nunca float.
`status`	enum LedgerEntryStatus	no nulo, por defecto pending	
`shipment_id`	uuid	nulo	
`hold_id`	uuid	FK → `holds`, nulo	
`payment_id`	text	nulo	
`reason`	text	nulo	
`admin_id`	uuid	nulo	
`idempotency_key`	text	único, no nulo	
`request_fingerprint`	text	no nulo	Hash del cuerpo, para distinguir un reintento legítimo de una key reusada con otros datos.
`created_at`	timestamptz	no nulo, por defecto now()	Milisegundos: es la precisión del cursor de paginación (la app la fija al insertar).

`BALANCE_PAYMENT_ATTEMPTS`

Tabla 12.54. Columnas de balance_payment_attempts .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	no nulo	
`amount_ars`	numeric(12, 2)	no nulo	Lo que el usuario quiso pagar en este intento: siempre positivo (el ledger lo asienta negativo).
`status`	enum BalanceAttemptStatus	no nulo, por defecto creating	
`saved_card_id`	uuid	nulo	Tarjeta guardada (`saved_cards.id`) con la que se pagó; sin FK, solo para el historial.

Columna	Tipo	Restricciones	Descripción
`request_fingerprint`	text	no nulo	Hash del cuerpo del pedido (token, tarjeta, medio de pago): un reintento con la misma key solo es seguro si coincide.
`mp_payment_id`	text	único, nulo	Pago de MP: único, así un mismo pago no puede asentar dos intentos.
`status_detail`	text	nulo	
`failure_reason`	enum HoldFailureReason	nulo	
`created_at`	timestamptz	no nulo, por defecto now()	
`updated_at`	timestamptz	no nulo, por defecto now()	

12.1.4 Administración (esquema `admin`)

El esquema contiene 2 tablas y 2 tipos enumerados.

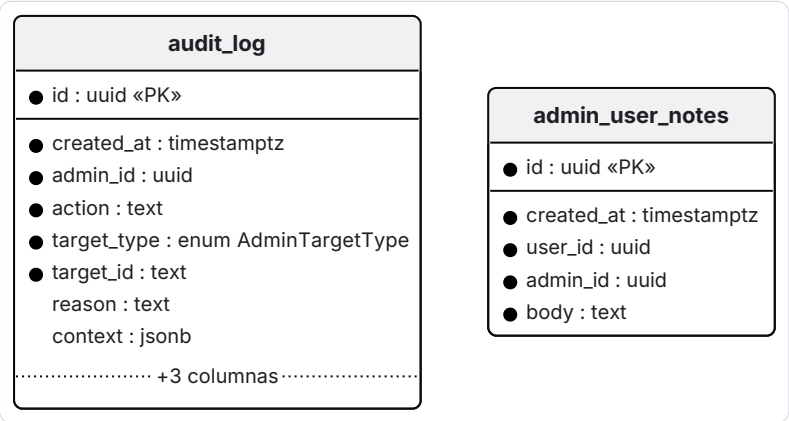


Figura 12.12. DER: Administración.

Tablas

Tabla 12.55. Tablas del esquema admin .

Tabla	Propósito	Columnas
`audit_log`	una fila por acción de escritura del panel (decisión 4 de MOVO-286).	11
`admin_user_notes`	notas internas del panel sobre un usuario (sección “Notas” de la ficha).	5

Tipos enumerados

Tabla 12.56. Enumerados del esquema admin .

Tipo	Valores
`AuditOutcome`	`succeeded` , `failed` , `unknown`
`AdminTargetType`	`user` , `admin_role` , `shipment` , `chat_report` , `user_report`

Diccionario de datos

`AUDIT_LOG`

Tabla 12.57. Columnas de audit_log .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`admin_id`	uuid	no nulo	
`action`	text	no nulo	Libre a propósito: cada US de admin suma las suyas (ej.
`target_type`	enum AdminTargetType	no nulo	
`target_id`	text	no nulo	
`reason`	text	nulo	
`context`	jsonb	nulo	
`outcome`	enum AuditOutcome	no nulo	
`error_code`	text	nulo	Código de ApiError con el que falló la acción (o el rechazo por conflicto).
`request_id`	text	nulo	
`created_at`	timestamptz	no nulo, por defecto now()	

`ADMIN_USER_NOTES`

Tabla 12.58. Columnas de admin_user_notes .

Columna	Tipo	Restricciones	Descripción
`id`	uuid	PK, no nulo	
`user_id`	uuid	no nulo	
`admin_id`	uuid	no nulo	
`body`	text	no nulo	
`created_at`	timestamptz	no nulo, por defecto now()	

13

PRODUCTO · ARQUITECTURA

Arquitectura del Sistema

Vistas de contexto, contenedores y despliegue, ambiente de implementación, seguridad y decisiones de arquitectura (ADR).

CONTENIDO

13.1	Visión General	275
13.2	Vista de Contexto	276
13.3	Vista de Contenedores	278
13.4	Vista de Despliegue	280
13.5	Ambiente de Implementación	282
13.6	Seguridad	283
13.7	Limitaciones Aceptadas	284
13.8	Registro de Decisiones de Arquitectura	285

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: vistas C4, ambiente de implementación y índice de ADR vigentes al cierre del Sprint 8.
2	02/11/2026	Equipo completo	Se incorpora el desarrollo completo de los 23 ADR vigentes (contexto, alternativas, decisión y consecuencias) desde el documento de Sprint 0.

Introducción

Este documento describe la arquitectura de Movo: sus tres vistas (contexto, contenedores y despliegue), el ambiente de implementación (software, hardware y proceso de despliegue), las decisiones de seguridad y el registro de decisiones de arquitectura.

Audiencia

El tribunal evaluador y los futuros mantenedores, con conocimiento de arquitectura de software, microservicios y despliegue con contenedores.

13.1 Visión General

Movo es una plataforma de logística distribuida punto a punto: conecta a quien necesita enviar un paquete con quien ya viaja esa ruta. Su arquitectura sigue un estilo de **microservicios** con un único punto de entrada público (ADR-001): una aplicación móvil y un panel web consumen un **API Gateway** que autentica, aplica límites de uso y enruta cada pedido al servicio dueño del dominio. Los servicios se comunican entre sí por REST síncrono sobre la red interna, sin *message broker*, y comparten una única instancia de PostgreSQL con **un esquema por servicio** (ADR-003), de modo que cada servicio es el único que escribe en sus tablas.

Las decisiones que más condicionan el diseño son las que sostienen las cuatro capas de confianza del producto: la **identidad** verificada por un tercero, la **custodia** probada con firma criptográfica y proximidad (ADR-018), el **pago** reservado y capturado solo al cumplirse la entrega (DD-037) y la **reputación** acumulada. Este documento describe la arquitectura en tres vistas (contexto, contenedores y despliegue, siguiendo el modelo C4), el ambiente de implementación y el registro de decisiones.

13.2 Vista de Contexto

El sistema Movo es usado por cuatro tipos de actores (emisor, transportista, receptor y administrador) y depende de ocho servicios externos, todos aislados detrás de interfaces que permiten reemplazarlos por simulaciones en desarrollo y pruebas.

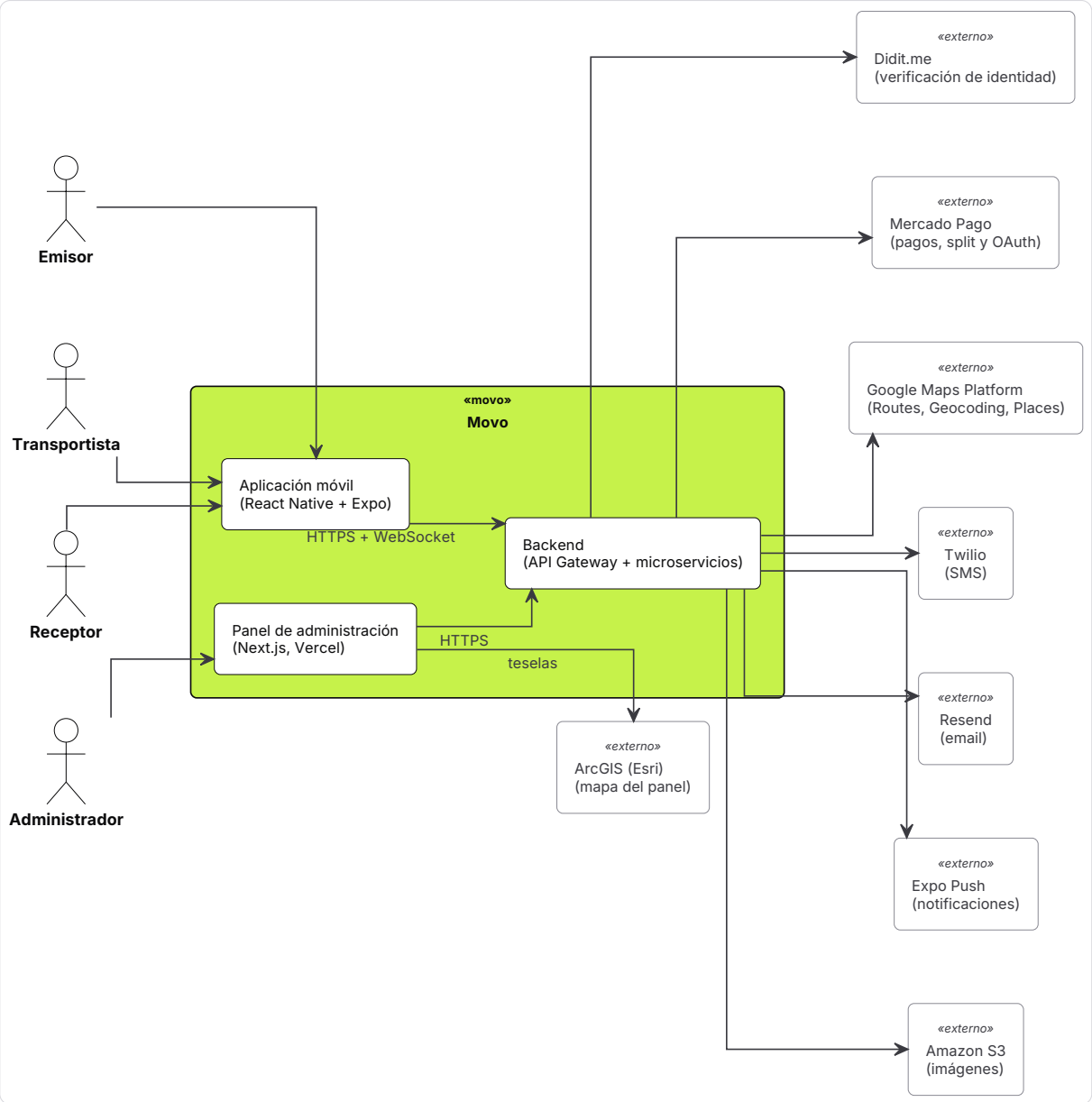


Figura 13.1. Vista de contexto: actores y sistemas externos.

Tabla 13.1. Sistemas externos de los que depende Movo.

Sistema externo	Uso	Decisión
Didit.me	Verificación de identidad y de licencia de conducir (documento y prueba de vida)	n/a
Mercado Pago	Reserva y captura del pago, reparto entre transportista y Movo, vinculación de la cuenta del transportista por OAuth	ADR-022

Sistema externo	Uso	Decisión
Google Maps Platform	Distancias y matriz de tiempos (Routes API), geocodificación y autocompletado de direcciones	ADR-015, ADR-014
Twilio	Envío del código por SMS para verificar el teléfono	ADR-012
Resend	Envío de correo electrónico (códigos y avisos)	ADR-016
Expo Push Service	Entrega de notificaciones push a los dispositivos	n/a
ArcGIS Location Platform (Esri)	Teselas del mapa de seguimiento de envíos en el panel de administración	n/a
Amazon S3	Fotos de envíos, de perfil, de reportes y adjuntos, con URLs prefirmadas	ADR-007, DD-016

13.3 Vista de Contenedores

El backend se compone de un gateway y cinco servicios de dominio, más Postgres y Redis. Solo el gateway (detrás de Nginx) es alcanzable desde fuera de la red interna.

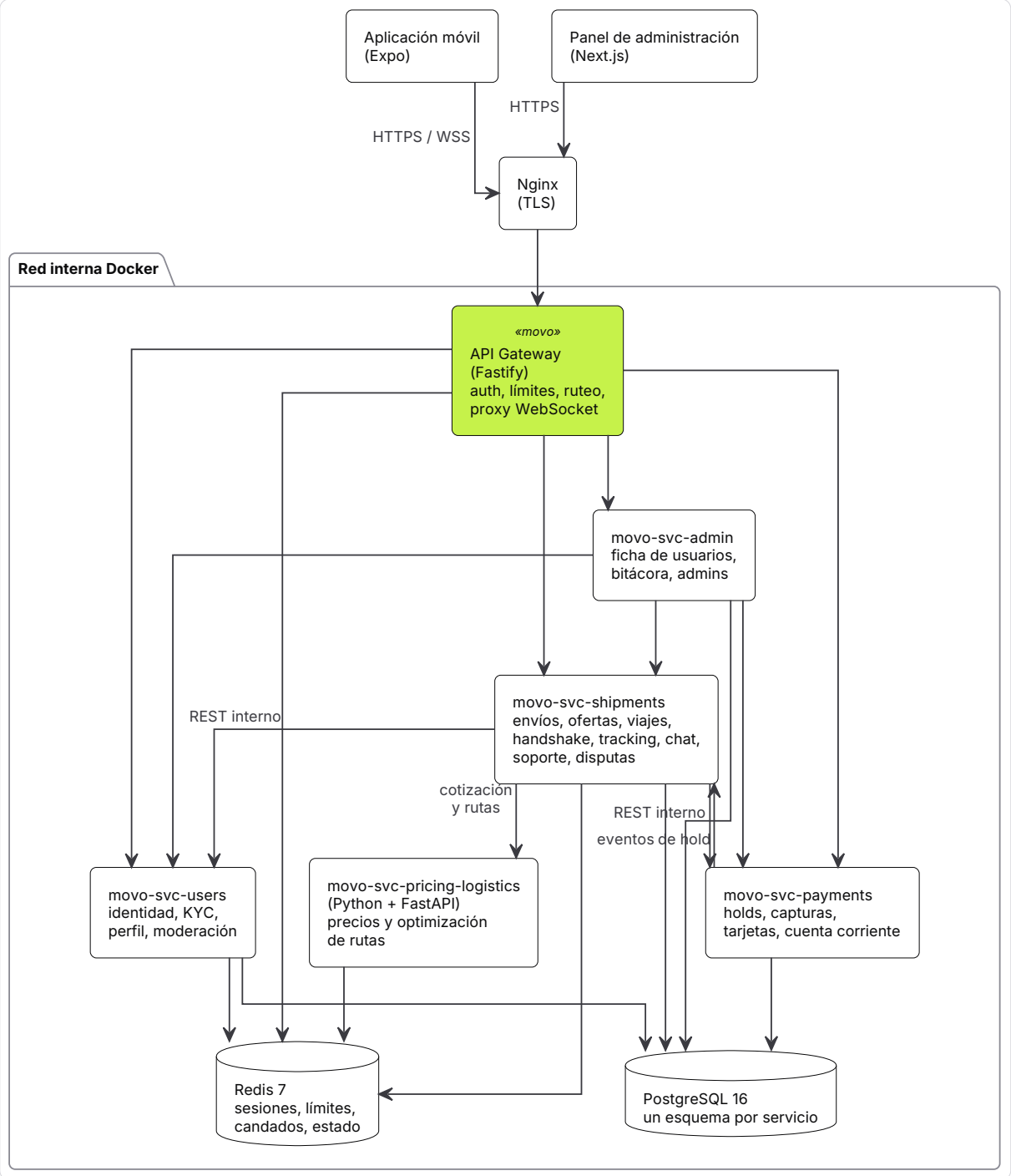


Figura 13.2. Vista de contenedores: gateway, servicios y almacenes de datos.

Tabla 13.2. Contenedores del sistema.

Contenedor	Tecnología	Responsabilidad
`movo-mobile`	React Native + Expo	Aplicación para emisores, transportistas y receptores
`movo-admin`	Next.js (Vercel)	Panel de administración
`gateway`	Node.js + Fastify	Autenticación del JWT, límites de uso, restricción de cuentas sancionadas, ruteo y proxy de WebSockets
`movo-svc-users`	Node.js + Fastify	Identidad, sesiones, verificación (KYC), perfiles, notificaciones, reportes y bloqueos, sanciones
`movo-svc-shipments`	Node.js + Fastify	Envíos, ofertas, viajes, handshake, tracking, chat, soporte, disputas y valoraciones
`movo-svc-payments`	Node.js + Fastify	Reservas de fondos, capturas y reparto, tarjetas, vinculación de Mercado Pago, cuenta corriente
`movo-svc-admin`	Node.js + Fastify	Operaciones del panel: ficha de usuarios, administradores y bitácora de acciones
`movo-svc-pricing-logistics`	Python + FastAPI + OR-Tools	Motor de precios y optimización de rutas con ventanas horarias (VRPTW)
PostgreSQL 16	n/a	Datos persistentes, un esquema por servicio
Redis 7	n/a	Sesiones y refresh tokens, límites de uso, desafíos del handshake, candados de barridos, marcas de cuentas sancionadas

El servicio de precios no se expone por el gateway: solo lo consume `movo-svc-shipments`. Todas las rutas públicas viven bajo el prefijo `/api/v1/`, con la excepción del chequeo de salud (`GET /health`).

13.4 Vista de Despliegue

Cada ambiente (desarrollo y producción) es una instancia EC2 de AWS en la región `sa-east-1` que ejecuta todo el backend con Docker Compose (ADR-006). Nginx termina TLS con certificados de Let's Encrypt y reenvía al gateway; el DNS lo resuelve Cloudflare en modo *DNS-only* hacia una dirección IP elástica. El panel de administración se despliega aparte en Vercel. La aplicación móvil se distribuye mediante builds de EAS.

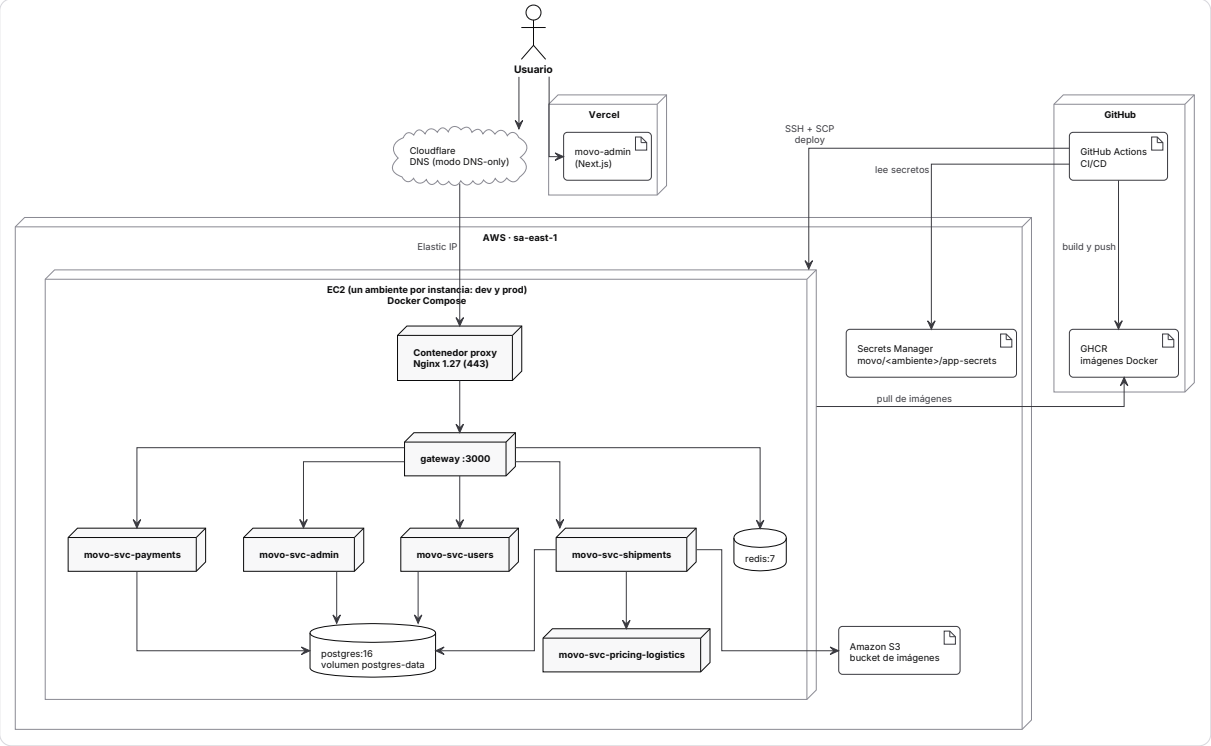


Figura 13.3. Vista de despliegue: infraestructura de un ambiente.

La infraestructura se aprovisiona con Terraform en un repositorio separado, `movo-infra` (ADR-009), que define, entre otros recursos, la instancia, el bucket de imágenes y los registros DNS de cada ambiente.

13.4.1 Integración y despliegue continuos

Los *workflows* de GitHub Actions automatizan la verificación y el despliegue:

1. **Pull request.** Con detección de cambios por ruta, se ejecutan *lint*, verificación de tipos y pruebas de cada paquete afectado contra PostgreSQL y Redis reales. El chequeo `tests-summary` bloquea el *merge* a `develop` y `main`.
2. **Integración en `develop`.** Se construye y publica en GHCR una imagen Docker por servicio con la etiqueta `dev`; luego se genera el archivo `.env` desde AWS Secrets Manager (`movo/dev/app-secrets`), se copian a la instancia el archivo de Compose y las plantillas de Nginx, se aplican las migraciones de Prisma servicio por servicio, se hace *pull* de las imágenes, se reinician los contenedores y se limpian las imágenes huérfanas.

3. **Producción.** El flujo equivalente se dispara al integrar `develop` en `main`, con sus propios secretos y su propia instancia.

Los secretos nunca están en el repositorio: en desarrollo local viven en un `.env` ignorado por Git, en los ambientes desplegados en Secrets Manager y en CI en GitHub Secrets.

13.5 Ambiente de Implementación

13.5.1 Software

Tabla 13.3. Versiones del software del ambiente.

Componente	Versión
Node.js (servicios y gateway)	20 (imagen `node:20-alpine`)
Fastify	5
TypeScript	6 (modo estricto)
Prisma	7 (con *driver adapter* para PostgreSQL)
Python (servicio de precios)	3.12 (imagen `python:3.12-slim`)
FastAPI y OR-Tools	0.115 y 9.15
PostgreSQL	16
Redis	7
Nginx	1.27 (imagen Alpine)
React Native / Expo (aplicación móvil)	0.86 / SDK 57
Next.js (panel de administración)	16
Docker Compose	Plugin v2

13.5.2 Hardware y red

El backend completo corre en una única instancia EC2 por ambiente, por lo que no hay balanceo ni escalado automático: es una limitación aceptada para el alcance del proyecto (ADR-006), igual que la ausencia de alta disponibilidad. Los dos servicios de datos (PostgreSQL y Redis) corren como contenedores de la misma instancia, con el volumen `postgres-data` persistente. Los puertos 80 y 443 de Nginx son los únicos publicados; todo lo demás vive en la red de Docker `movo-net`. El puerto 22 (SSH) permanece abierto a cualquier dirección porque el despliegue desde GitHub Actions no puede limitarse a un rango fijo de direcciones; es una limitación aceptada y documentada.

13.5.3 Operación

Los contenedores tienen política de reinicio automático y rotación de *logs* (`json-file`, 10 MB por archivo y 3 archivos). Al final de cada despliegue se ejecuta `docker image prune -af`, porque el disco reducido de la instancia se llena de imágenes sin etiqueta si no se limpia. El detalle de instalación y operación está en el Manual de Instalación y Operación (Tomo III).

13.6 Seguridad

- **Autenticación.** Un JWT de acceso de 60 minutos y un *refresh token* opaco de 90 días en Redis, de un solo uso con detección de reuso: presentar un refresh ya canjeado revoca todas las sesiones del usuario (ADR-004, ADR-013). El panel tiene una sesión propia, con segundo factor por correo y duración máxima de 8 horas.
- **Autorización.** Los roles viajan en el token como un arreglo y el gateway inyecta la identidad verificada en encabezados internos que los servicios no revalidan (ADR-010). Es un modelo de confianza perimetral, no *zero-trust*.
- **Custodia.** Firma ECDSA P-256 del dispositivo, desafío efímero de 15 segundos y verificación de proximidad a 100 metros (ADR-018).
- **Datos sensibles.** Movo no almacena documentos ni datos biométricos de la verificación de identidad (solo el estado y la decisión redactada) ni números de tarjeta ni CVV; los tokens de Mercado Pago se guardan cifrados.
- **Limitación de uso.** Contadores en Redis por usuario y por ruta en el gateway, más límites específicos de cada flujo sensible (envío de códigos, cotizaciones, reportes).
- **Cumplimiento.** Términos y Condiciones y Política de Privacidad versionados con aceptación registrada, y purga automática de datos vencidos conforme a la Ley 25.326.

13.7 Limitaciones Aceptadas

El equipo documentó explícitamente estas limitaciones de arquitectura como aceptadas, en lugar de dejarlas implícitas: ausencia de alta disponibilidad y de escalado automático (una instancia por ambiente); punto único de falla en la instancia de base de datos compartida, mitigado con esquemas separados y *snapshots*; acoplamiento temporal por el uso de REST síncrono, mitigado con el identificador `x-request-id` en los *logs*; modelo de confianza perimetral entre servicios; puerto SSH abierto; y dependencia de red de los servicios externos en el camino crítico de algunas operaciones, con las políticas de degradación descriptas en el documento de patrones de diseño.

13.8 Registro de Decisiones de Arquitectura

Los *Architecture Decision Records* (ADR) documentan cada decisión técnica significativa: el contexto que la motivó, las alternativas evaluadas con criterios explícitos, la decisión tomada y sus consecuencias, incluidos los costos que el equipo aceptó de forma consciente. Se mantienen **23 ADR**, numerados del 001 al 023, que fijan el stack, un proveedor o una integración, la estructura entre servicios o el modelo de seguridad. Un ADR aceptado no se modifica: si la decisión cambia, se crea uno nuevo que referencia y depreca al anterior.

Las decisiones de alcance acotado (la regla de un módulo, el contrato de un endpoint, un parámetro) no son ADR: están en el Registro de Decisiones de Diseño, con el prefijo **DD** y el número que tenían originalmente.

13.8.1 Índice de ADR

Tabla 13.4. Índice de los ADR vigentes.

ADR	Decisión	Estado	Núm. anterior
ADR-001	Patrón Arquitectónico del Backend	Aceptado	–
ADR-002	Lenguaje y Framework del Backend	Aceptado	–
ADR-003	Motor y Estrategia de Base de Datos	Aceptado	–
ADR-004	Mecanismo de Autenticación y Autorización	Aceptado (actualizado por ADR-013)	–
ADR-005	Diseño y Protocolo de la API	Aceptado (actualizado por ADR-019)	–
ADR-006	Infraestructura y Estrategia de Despliegue	Aceptado	–
ADR-007	Estrategia de Almacenamiento de Imágenes y Archivos Multimedia	Aceptado	–
ADR-008	Proveedor de Datos de Distancia para el Motor Logístico	Deprecado (Ver ADR-015)	–
ADR-009	Infraestructura como Código (Terraform) y Gestión de DNS	Aceptado	–
ADR-010	Confianza en la Red Interna del API Gateway	Aceptado	–
ADR-011	Adopción de Prisma como ORM para los servicios de MOVO	Aceptado	–
ADR-012	Twilio como proveedor de SMS para verificación de OTP	Aceptado	–
ADR-013	Refresh token con TTL de 90 días, rotación de un solo uso y detección de reuso	Aceptado (MOVO-75)	–
ADR-014	Google Maps como proveedor de geocoding detrás de GeocodingProvider	Aceptado (MOVO-73)	–

ADR	Decisión	Estado	Núm. anterior
ADR-015	Migración de Distance Matrix API (Legacy) a Routes API (Compute Route Matrix)	Aceptado	–
ADR-016	Resend como proveedor de email detrás de EmailProvider	Aceptado (MOVO-139)	017
ADR-017	Statelessness de movo-svc-pricing-logistics y ubicación de la entidad Offer	Aceptado	019
ADR-018	Handshake de custodia: firma ECDSA P-256 generada en el dispositivo	Aceptado (MOVO-158, MOVO-195)	020
ADR-019	Canal de tiempo real – WebSocket nativo (@fastify/websocket)	Aceptado	022
ADR-020	Tracking en tiempo real: ingesta por HTTP, difusión por WebSocket	Aceptado (MOVO-250)	024
ADR-021	Motor de precios dinámico: combustible desde datos abiertos, demanda por zona y contrato simplificado	Aceptado	025
ADR-022	Arquitectura de cobro: Payments API + OAuth Connect + application_fee	Aceptado (MOVO-209)	035
ADR-023	Chat propio sobre el canal WebSocket (MOVO-26)	Aceptado (MOVO-26); parcialmente reemplazado por DD-046	040

La columna *Núm. anterior* indica el número con el que estas decisiones se registraron antes de depurar el registro a 23 ADR; los ADR 001 a 015 conservan su número. Las referencias a un número anterior que hoy corresponde a un DD se leen con el prefijo DD.

13.8.2 ADR-001 · Patrón Arquitectónico del Backend

ACEPTADO

CONTEXTO

Situación

Movo es una plataforma con dominios claramente diferenciados (identidad/KYC, envíos, pagos, logística/pricing) que tienen ciclos de vida, equipos responsables y tecnologías distintas. Se necesita una arquitectura que permita desarrollar, escalar y desplegar estos dominios de forma independiente. La restricción más importante es que migrar desde un monolito hacia microservicios en el futuro sería prohibitivamente costoso dado que implicaría refactoring masivo de código ya escrito.

CRITERIOS DE COMPARACIÓN

Criterios

Independencia de despliegue por dominio.

Posibilidad de usar distintas tecnologías por servicio (Node.js vs Python).

Claridad de responsabilidades y fronteras de dominio.

Complejidad operacional manejable para un equipo de 5 personas.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Opción A: Monolito modular

Un único proceso con módulos internos bien definidos.

Pros: menor complejidad operacional, un solo repositorio, deploy más simple.

Contras: imposibilita usar Python para el servicio de logística, cualquier cambio en un módulo requiere redeployar todo el sistema, escalabilidad monolítica.

Alternativa 2

Opción B: Microservicios ELEGIDA

Cada dominio es un servicio desplegable de forma independiente.

Pros: permite mezclar Node.js y Python según conveniencia técnica, fronteras de dominio claras, escalabilidad por servicio, un fallo no tumba toda la plataforma.

Contras: mayor complejidad operacional (gestión de múltiples contenedores, comunicación entre servicios, troubleshooting distribuido).

Alternativa 3

Opción C: Serverless (AWS Lambda)

Funciones como servicio sin gestión de infraestructura.

Pros: escalabilidad automática, sin administración de servidores.

Contras: cold starts problemáticos para WebSockets de tracking en tiempo real, costos impredecibles con alto volumen, mayor vendor lock-in, complejidad para gestionar estado de conexiones.

DECISIÓN

Decisión tomada

Microservicios (Opción B) con 5 servicios más el API Gateway: movo-api-gateway, movo-svc-users, movo-svc-shipments, movo-svc-payments, movo-svc-pricing-logistics, movo-svc-admin. La comunicación entre servicios es sincrónica vía REST HTTP, sin message broker, para mantener la complejidad acotada al nivel del Proyecto Final.

CONSECUENCIAS

Trade-offs

Positivo: fronteras claras que facilitan el desarrollo paralelo del equipo.

Positivo: permite usar FastAPI + Python para el servicio de logística sin impactar el resto del stack.

Positivo: un fallo en movo-svc-pricing-logistics no afecta la capacidad de procesar pagos.

Trade-off negativo: la comunicación sincrónica entre servicios introduce acoplamiento temporal; si movo-svc-users está caído, el gateway no puede autenticar requests.

Trade-off negativo: el debugging distribuido es más complejo que en un monolito. Se mitiga con correlación de IDs de request en todos los logs.

Trade-off negativo: Docker Compose en una sola EC2 implica que un fallo de hardware tumba todos los servicios. Para el alcance del Proyecto Final se acepta este trade-off.

13.8.3 ADR-002 · Lenguaje y Framework del Backend

ACEPTADO

CONTEXTO

Situación

El equipo necesita elegir el lenguaje y framework para los servicios backend. Existen dos requisitos que no pueden satisfacerse con un único lenguaje: la mayoría de los servicios requieren alto throughput de I/O (requests HTTP, WebSockets, queries a base de datos), mientras que el servicio de logística y precios requiere capacidades de cómputo numérico, acceso a librerías de optimización combinatoria (OR-Tools, scipy) y consumo de la Google Maps Distance Matrix API para obtener tiempos y distancias reales de ruta entre los puntos del problema.

CRITERIOS DE COMPARACIÓN

Criterios

Conocimiento previo del equipo.

Performance en I/O concurrente.

Ecosistema de librerías para optimización combinatoria.

Consistencia de tipado y tooling.

Velocidad de desarrollo.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Opción A: Node.js + Fastify (TypeScript) - para todos los servicios

Pros: uniforme, el equipo lo conoce, excelente para I/O concurrente.

Contras: librerías de optimización combinatoria en Node.js son pobres; implementar VRPTW en TypeScript sería reinventar la rueda.

Alternativa 2

Opción B: Node.js (Fastify/TypeScript) + Python (FastAPI) - ELEGIDA

Node.js + Fastify para: api-gateway, svc-users, svc-shipments, svc-payments.

Python + FastAPI para: svc-pricing-logistics.

Pros: cada lenguaje en el dominio donde es más fuerte. FastAPI tiene validación automática con Pydantic y documentación Swagger generada automáticamente igual que Fastify.

Contras: el equipo debe mantener dos stacks tecnológicos distintos.

Alternativa 3

Opción C: Python + FastAPI para todos los servicios

Pros: acceso nativo a todas las librerías de optimización y data science.

Contras: el GIL de Python limita la concurrencia real en I/O; para WebSockets de tracking en tiempo real con muchos clientes simultáneos, Node.js es sustancialmente mejor.

Alternativa 4

Opción D: Go para todos los servicios

Pros: performance excepcional, tipado estático, excelente para sistemas distribuidos.

Contras: el equipo no tiene experiencia en Go; la curva de aprendizaje en el contexto de un Proyecto Final es inviable.

DECISIÓN

Decisión tomada

Opción B: Node.js con Fastify y TypeScript para los servicios de I/O intensivo; Python con FastAPI para el servicio de logística y precios.

CONSECUENCIAS

Trade-offs

Positivo: TypeScript en todo el stack frontend/backend Node.js permite compartir tipos entre repositorios via el paquete movo-shared.

Positivo: FastAPI genera documentación OpenAPI automáticamente al igual que Fastify, lo que mantiene consistencia en la estrategia de documentación.

Trade-off negativo: el equipo debe mantener dos entornos de desarrollo distintos (Node.js y Python), con sus respectivos linters, testers y configuraciones.

Trade-off negativo: la comunicación entre el gateway y el servicio Python requiere serialización/deserialización JSON que agrega latencia marginal.

13.8.4 ADR-003 · Motor y Estrategia de Base de Datos

ACEPTADO

CONTEXTO

Situación

El sistema maneja datos con distintas características: datos transaccionales fuertemente relacionados (envíos, pagos, usuarios, reputación), datos de sesión y estado de WebSockets que requieren acceso extremadamente rápido, y potencialmente datos de series de tiempo para el historial de posición GPS de envíos activos.

CRITERIOS DE COMPARACIÓN

Criterios

Consistencia y soporte para transacciones ACID.

Familiaridad del equipo.

Costo operacional.

Complejidad de gestión.

Capacidad para datos relacionales complejos.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Opción A: PostgreSQL única instancia compartida (ELEGIDA para datos relacionales)

Pros: ACID completo, el equipo tiene experiencia, excelente soporte para JSON cuando se necesita flexibilidad, extensiones como PostGIS si en el futuro se incorpora análisis geoespacial complejo.

Contras: punto único de fallo; en una arquitectura de microservicios 'pura' cada servicio debería tener su propia DB.

Alternativa 2

Opción B: Una instancia de PostgreSQL por microservicio

Pros: aislamiento completo entre servicios, patrón 'correcto' para microservicios.

Contras: para el alcance del Proyecto Final es sobredimensión operacional. Gestionar 5 instancias de DB separadas implica 5 veces el costo, 5 veces la complejidad de backup y 5 veces las migraciones a coordinar.

Alternativa 3

Opción C: MongoDB para datos de dominio + Redis para caché

Pros: flexibilidad de schema para datos variables.

Contras: el dominio de Movo es inherentemente relacional (un envío tiene UN emisor, UN transportista, MÚLTIPLES pagos). Un modelo documental fuerza joins en la aplicación que PostgreSQL hace nativamente con mejor performance.

Alternativa 4

Opción D: TimescaleDB para datos de tracking GPS

Pros: query de series de tiempo muy eficientes.

Contras: el historial de GPS de envíos activos puede manejarse razonablemente con Redis durante el envío activo y PostgreSQL para el histórico. Agregar TimescaleDB solo para esto es sobredimensión.

DECISIÓN

Decisión tomada

PostgreSQL 16 como única base de datos relacional. Redis 7 para datos que requieren acceso en microsegundos: sesiones, refresh tokens y estado de WebSockets para tracking en tiempo real. No se usa base de datos de series de tiempo; el historial de posición GPS se persiste en PostgreSQL al finalizar cada envío.

CONSECUENCIAS

Trade-offs

Positivo: una sola DB simplifica enormemente las migraciones, los backups y el monitoreo.

Positivo: Redis como segunda capa resuelve el caso de uso de alta frecuencia sin contaminar la DB principal.

Trade-off negativo: una única instancia de PostgreSQL es punto único de fallo. Se mitiga con snapshots automáticos de AWS.

Trade-off negativo: no se tiene aislamiento de datos entre servicios. Un bug en movo-svc-payments podría en teoría corromper datos de movo-svc-shipments si no se usan esquemas separados por servicio dentro de la misma instancia. Se mitiga usando esquemas de PostgreSQL distintos por servicio.

13.8.5 ADR-004 · Mecanismo de Autenticación y Autorización

ACEPTADO

> El plazo del refresh token (7 días) fue reemplazado por 90 días en el ADR-013.

CONTEXTO

Situación

El sistema tiene tres tipos de actores (emisor, transportista, receptor/admin) con distintos permisos sobre los recursos. Se necesita un mecanismo que funcione tanto para la app móvil (clientes sin estado persistente del lado del servidor) como para el panel de administración web. Además, el tiempo de sesión debe ser gestionable: un JWT que no expira nunca es un riesgo de seguridad, pero obligar al usuario a re-loguearse frecuentemente es mala UX.

CRITERIOS DE COMPARACIÓN

Criterios

Seguridad: resistencia a robo de tokens.

Escalabilidad: no requerir consulta a DB en cada request.

Experiencia de usuario: sesiones de larga duración sin re-login constante.

Compatibilidad con entorno móvil (React Native).

Simplicidad de implementación.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Opción A: Sessions con cookies (session-based auth)

Pros: revocación inmediata de sesiones, patrón conocido.

Contras: requiere estado del lado del servidor (session store); en microservicios cada servicio necesitaría acceder al session store o el gateway debería resolver la sesión antes de rutear. Problemático con apps móviles donde las cookies tienen limitaciones.

Alternativa 2

Opción B: JWT de larga duración (sin refresh token)

Pros: simplísimo, stateless.

Contras: si el token es robado, no puede revocarse hasta que expire. Un JWT de 30 días es una ventana de ataque inaceptable.

Alternativa 3

Opción C: JWT de corta duración + Refresh Token ELEGIDA

Access token JWT con expiración corta (15-60 minutos). Refresh token de larga duración almacenado en Redis.

	Pros: si el access token es robado, expira rápido. El refresh token puede revocarse inmediatamente eliminándolo de Redis. Stateless en la validación del access token. **Contras:** mayor complejidad que un JWT simple; el cliente debe implementar la lógica de renovación automática.
Alternativa 4	**Opción D:** OAuth 2.0 con proveedor externo (Google, Apple) **Pros:** delega la seguridad a un tercero con más experiencia. **Contras:** dependencia crítica de un tercero para el login. Si Google/Apple cambia sus condiciones de OAuth, el sistema queda afectado. Para el Proyecto Final la implementación propia es más pertinente académicamente.

DECISIÓN

Decisión tomada	**Opción C:** JWT de corta duración (access token, TTL 60 minutos) + Refresh Token almacenado en Redis (TTL 7 días). El API Gateway valida el access token en cada request. La autorización por roles (RBAC) se implementa vía claims en el payload del JWT. El mismo mecanismo aplica para la app mobile y el panel de administración. Se usan los roles como array no como string singular. AccessTokenClaims.roles: UserRole[]. En Movo el mismo usuario puede ser emisor y transportista simultáneamente. Refresh token opaco, no JWT: signRefreshToken() genera crypto.randomBytes(32).toString('base64url') + un tokenId (randomUUID()). No es autodescriptivo ni verificable sin consultar Redis. La librería no persiste nada, eso queda a cargo del servicio consumidor (guardar el hash bajo tokenId). Esto preserva la capacidad de revocación inmediata que ADR-004 declara como beneficio del diseño; un refresh como JWT firmado la hubiera anulado.

CONSECUENCIAS

Trade-offs	**Positivo:** la revocación de sesiones es inmediata (se borra el refresh token de Redis). **Positivo:** el access token es stateless; el gateway puede validarlo sin consultar la DB. **Positivo:** el RBAC en claims del JWT permite que cada servicio verifique permisos sin llamar a movo-svc-users. **Trade-off negativo:** el access token no puede revocarse antes de su TTL de 60 minutos. Si un usuario es baneado, puede seguir operando hasta que su token actual expire. Se acepta este trade-off para el alcance del Proyecto Final. **Trade-off negativo:** el cliente móvil debe implementar lógica de refresh automático, lo que agrega complejidad al código del frontend.

13.8.6 ADR-005 · Diseño y Protocolo de la API

ACEPTADO

> La mención de Socket.io fue reemplazada por WebSocket nativo en el ADR-019.

CONTEXTO

Situación

El sistema necesita definir cómo se comunican los clientes (app móvil, panel admin) con el backend, y cómo se manejan las actualizaciones en tiempo real del tracking de envíos. Se debe elegir entre paradigmas de consulta/respuesta y un protocolo para comunicación bidireccional.

CRITERIOS DE COMPARACIÓN

Criterios

Curva de aprendizaje del equipo.

Soporte en React Native y Next.js.

Generación automática de documentación.

Capacidad para la comunicación en tiempo real.

Facilidad de testeo.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Opción A: REST + WebSockets ELEGIDA

REST para todas las operaciones CRUD y transaccionales.

Socket.io para el tracking de posición en tiempo real.

Pros: el equipo conoce REST, generación automática de Swagger con Fastify y FastAPI, Socket.io tiene abstracción sobre WebSockets nativos con fallback automático.

Contras: para casos de uso con muchos campos opcionales, REST puede resultar en over-fetching o under-fetching.

Alternativa 2

Opción B: GraphQL + Subscriptions

Pros: el cliente solicita exactamente los campos que necesita, subscriptions para tiempo real.

Contras: el equipo no tiene experiencia en GraphQL; la curva de aprendizaje en el contexto del Proyecto Final es un riesgo real de cronograma. La generación de documentación automática es más compleja. El N+1 problem requiere DataLoader para resolverse correctamente.

Alternativa 3

Opción C: gRPC entre servicios internos

Pros: performance superior para comunicación interna, contratos fuertemente tipados vía Protocol Buffers.

Contras: complejidad de setup significativa, tooling más complejo para debugging, incompatible con llamadas directas desde el browser sin grpc-web.

DECISIÓN

Decisión tomada

REST sobre HTTP/1.1 para todas las operaciones, con prefijo de versión /api/v1/ en todas las rutas. Socket.io para el canal de tracking en tiempo real. La documentación OpenAPI/Swagger se genera automáticamente: Fastify vía @fastify/swagger, FastAPI nativamente. El versionado permite introducir /api/v2/ en el futuro sin romper clientes existentes.

CONSECUENCIAS

Trade-offs

Positivo: el equipo puede comenzar a desarrollar inmediatamente sin curva de aprendizaje.

Positivo: Swagger generado automáticamente reduce el riesgo de documentación desactualizada.

Positivo: Socket.io provee fallback a HTTP long-polling en redes con firewalls que bloquean WebSockets.

Trade-off negativo: REST puede generar over-fetching en endpoints que devuelven objetos grandes. Se mitiga con query params de proyección (?fields=id,status,origin).

Trade-off negativo: la comunicación interna entre microservicios también usa REST, lo que es menos eficiente que gRPC pero más simple de implementar y debuggear.

13.8.7 ADR-006 · Infraestructura y Estrategia de Despliegue

ACEPTADO

CONTEXTO

Situación

El sistema tiene entre 6 y 8 contenedores Docker que deben correr coordinados, con configuraciones distintas para desarrollo y producción. Se necesita una estrategia de despliegue que sea manejable por un equipo de 5 estudiantes sin experiencia previa en DevOps avanzado, que tenga costos controlados y que permita CI/CD automatizado.

CRITERIOS DE COMPARACIÓN

Criterios

Costo mensual operacional.

Complejidad de administración para el equipo.

Tiempo de setup inicial.

Soporte para CI/CD automatizado.

Capacidad de gestionar secretos de forma segura.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Opción A: EC2 + Docker Compose ELEGIDA

Una instancia EC2 por ambiente (dev y prod). Docker Compose orquesta todos los contenedores.

Pros: el equipo ya conoce Docker Compose de los cursos anteriores, costo predecible (se paga la instancia, no por llamada), setup inicial relativamente simple, suficiente para el volumen del Proyecto Final.

Contras: no tiene auto-scaling, Docker Compose no es un orquestador de producción enterprise.

Alternativa 2

Opción B: Kubernetes (EKS en AWS)

Pros: orquestación de nivel enterprise, auto-scaling, auto-healing.

Contras: la curva de aprendizaje de Kubernetes para un equipo sin experiencia previa es de semanas. El overhead de gestión (YAML de deployments, services, ingresses, namespaces) duplicaría el tiempo de setup. Completamente sobredimensionado para el alcance del Proyecto Final.

Alternativa 3

Opción C: Railway / Render / Fly.io (PaaS)

Pros: zero DevOps, deploy con git push.

Contras: menor control sobre la infraestructura, costos más impredecibles con escala, vendor lock-in significativo, limitaciones en la configuración de red entre servicios.

Alternativa 4

Opción D: AWS ECS (Fargate)

Pros: orquestación de contenedores sin gestionar servidores.

Contras: precio por vCPU/hora es mayor que EC2; la configuración de task definitions, services y ALB es más compleja que Docker Compose para un resultado similar.

DECISIÓN

Decisión tomada

EC2 + Docker Compose para el backend. Dos ambientes: dev (branch develop) y prod (branch main). GitHub Actions ejecuta el pipeline de CI/CD: en cada push a develop/main, corre lint, tests, build de imagen Docker y deploy vía SSH a la EC2 correspondiente. Los secretos del backend se gestionan en AWS Secrets Manager y se inyectan como variables de entorno al arrancar los contenedores.

Los frontends Next.js (movo-admin y movo-institucional) se despliegan en Vercel. Esta separación responde a que Next.js está optimizado para el entorno de Vercel: SSR, ISR, image optimization y CDN global funcionan de forma nativa sin configuración adicional. Correr Next.js en Docker Compose en EC2 es técnicamente viable pero introduce fricción operacional innecesaria y elimina las optimizaciones automáticas de la plataforma. Vercel se conecta directamente al repositorio GitHub y genera deploys automáticos en cada push, con URLs de preview por Pull Request. Las variables de entorno de cada frontend (principalmente la URL del API Gateway) se configuran en el dashboard de Vercel por ambiente.

CONSECUENCIAS

Trade-offs

Positivo: el equipo puede enfocarse en el código del producto en lugar de en la infraestructura.

Positivo: Docker Compose garantiza que el ambiente de desarrollo local es idéntico al de producción.

Positivo: sacar los dos frontends Next.js de Docker Compose simplifica el stack del EC2 y reduce la cantidad de contenedores a gestionar. Positivo: Vercel provee CDN global, previews automáticas por PR y zero-config para Next.js, sin costo operacional para el equipo.

Trade-off negativo: los frontends y el backend pasan a estar en plataformas distintas, lo que introduce CORS como requisito explícito de configuración en el API Gateway. El gateway debe aceptar requests desde los dominios de Vercel (*.vercel.app y el dominio de producción).

Trade-off negativo: los deploys de preview de Vercel generan URLs públicas accesibles para cada PR. Para movo-admin esto implica que versiones intermedias del panel de administración son accesibles en internet. Se mitiga garantizando que la autenticación en el API Gateway esté implementada desde el primer endpoint, sin rutas desprotegidas en ningún estado del desarrollo.

Trade-off negativo: una sola EC2 por ambiente es un punto único de fallo de hardware. No hay alta disponibilidad. Para el Proyecto Final esto es aceptable.

Trade-off negativo: Docker Compose no tiene capacidad de auto-scaling. Si el tráfico supera la capacidad de la instancia, se requiere intervención manual.

Trade-off negativo: los deployments tienen un breve periodo de downtime mientras los contenedores se reinician. Se acepta para el alcance del proyecto.

13.8.8 ADR-007 · Estrategia de Almacenamiento de Imágenes y Archivos Multimedia

ACEPTADO

CONTEXTO

Situación

El flujo operativo de Movo requiere almacenar imágenes en múltiples etapas del ciclo de vida de un envío: fotografías del paquete al momento del retiro, fotografías al momento de la entrega, e imágenes de evidencia adjuntadas durante la apertura de una disputa. Estas imágenes son generadas desde dispositivos móviles y deben ser accesibles posteriormente por emisor, receptor, transportista y administrador según corresponda. El volumen estimado es de 2 a 6 imágenes por envío activo. El sistema necesita una estrategia de almacenamiento que sea durable, segura, de bajo costo operacional y que no impacte la performance de la base de datos principal.

CRITERIOS DE COMPARACIÓN

Criterios

Durabilidad y disponibilidad.

Seguridad y control de acceso.

Costo operacional para el volumen del Proyecto Final.

Complejidad de integración con el stack existente.

Performance en upload desde dispositivos móviles con conectividad variable.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Opción A: Almacenamiento en filesystem de la EC2

Guardar las imágenes directamente en el sistema de archivos de la instancia EC2, sirviendo los archivos a través del API Gateway.

Pros: cero configuración adicional, sin costo extra.

Contras: los archivos son efímeros si se reemplaza o reinicia la instancia. No escala. Toda la transferencia de imágenes pasa por el servidor, consumiendo ancho de banda y memoria del proceso Node.js.

Inaceptable para producción.

Alternativa 2

Opción B: Almacenar imágenes como BLOBs en PostgreSQL

Guardar el binario de cada imagen directamente en la base de datos relacional.

Pros: simplifica el modelo de datos, todo en un solo lugar.

Contras: antipatrón conocido de performance. Las imágenes inflan el tamaño de la base de datos, degradan la performance de queries y complican los backups. PostgreSQL no está optimizado para servir archivos binarios grandes.

Alternativa 3

Opción C: AWS S3 con presigned URLs ELEGIDA

Almacenar las imágenes en un bucket S3 privado. El backend genera URLs prefirmadas de corta duración para upload y visualización. La imagen nunca transita por los servidores de Movo.

Pros: durabilidad de 99.999999999% garantizada por AWS. Costo marginal para el volumen del PY (primer GB gratis, fracciones de centavo por GB adicional). Upload directo desde el dispositivo móvil a S3 sin pasar por el servidor, eliminando latencia y carga en EC2. Control de acceso centralizado: cada URL prefirmada tiene un TTL configurable y solo puede ser generada por el backend autenticado. Integración nativa con el stack AWS ya elegido en ADR-006.

Contras: agrega una dependencia adicional de AWS. Requiere que el cliente implemente la lógica de upload directo a S3 usando la presigned URL recibida del backend.

Alternativa 4

Opción D: Servicio de terceros (Cloudinary, Uploadcare)

Plataformas especializadas en gestión de imágenes con transformaciones automáticas (resize, compresión, CDN).

Pros: funcionalidades avanzadas como resize automático y optimización de formato. SDK mobile disponible.

Contras: dependencia externa adicional fuera del ecosistema AWS ya adoptado. Costo impredecible con escala. Las funcionalidades avanzadas están fuera del alcance del Proyecto Final. Sobredimensionado para el caso de uso.

DECISIÓN

Decisión tomada

AWS S3 con presigned URLs (Opción C). Se crea un bucket privado por ambiente (movo-shipment-media-dev y movo-shipment-media-prod). Las imágenes se organizan con el prefijo shipments/{shipment_id}/{etapa}/{timestamp}.jpg, donde etapa puede ser order, pickup, delivery o dispute. El backend (movo-svc-shipments) es el único servicio autorizado a generar presigned URLs, tanto para upload (PUT, TTL 5 minutos) como para visualización (GET, TTL 15 minutos). La referencia a cada imagen (su S3 key) se persiste en PostgreSQL vinculada al envío. Ninguna URL de S3 se almacena en la base de datos; las URLs se generan en tiempo de request.

Flujo de upload:

1. El cliente solicita al backend una presigned URL para upload.
2. El backend valida que el usuario tiene permiso para subir una foto en esa etapa del envío y genera la URL prefirmada con TTL de 5 minutos.
3. El cliente sube la imagen directamente a S3 usando la presigned URL (PUT request).
4. El cliente notifica al backend que el upload fue completado.
5. El backend registra la S3 key en PostgreSQL y avanza el estado del envío si corresponde.

CONSECUENCIAS

Trade-offs

Positivo: las imágenes son durables e independientes del ciclo de vida de la EC2.

Positivo: el upload directo desde el dispositivo a S3 elimina la carga de transferencia del servidor.

Positivo: el control de acceso mediante presigned URLs con TTL garantiza que solo usuarios autenticados con permiso sobre ese envío pueden ver o subir imágenes.

Positivo: el costo operacional para el volumen esperado del PY es prácticamente cero.

Trade-off negativo: el cliente móvil debe implementar un flujo de dos pasos para el upload (solicitar URL al backend, luego hacer PUT a S3), lo que agrega complejidad al código del frontend comparado con un upload directo al API.

Trade-off negativo: si el cliente completa el upload a S3 pero falla la notificación al backend en el paso 4, la imagen existe en S3 pero no queda registrada en la base de datos. Se mitiga con un timeout en el cliente que reintenta la notificación, y con una tarea de limpieza periódica que detecta objetos S3 sin referencia en la DB.

13.8.9 ADR-008 · Proveedor de Datos de Distancia para el Motor Logístico

DEPRECADO Ver ADR-015

CONTEXTO

Situación

OR-Tools resuelve el problema VRPTW trabajando sobre una matriz de costos entre puntos. Esa matriz debe representar el tiempo o distancia real de desplazamiento entre cada par de puntos (origen del transportista, puntos de retiro y entrega de paquetes). La calidad de la ruta optimizada depende directamente de la precisión de esa matriz: datos incorrectos producen rutas subóptimas que degradan la experiencia del transportista.

CRITERIOS DE COMPARACIÓN

Criterios

Precisión de los datos (considera calles reales, sentidos viales, tráfico). Costo por uso. Dependencia de red en el camino crítico. Complejidad de integración. Disponibilidad y SLA del proveedor.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Opción A: Distancia haversine con factor de corrección.

Calcula distancia en línea recta entre coordenadas GPS aplicando un factor de aproximación (~1.3x). Pros: sin dependencia de red, sin costo, implementación en una función Python de 10 líneas, testable de forma determinista.

Contras: no considera calles ni sentidos viales; en ciudades con ríos, vías de tren o zonas sin acceso directo, el error puede ser significativo.

Alternativa 2

Opción B: OSRM self-hosted (Open Source Routing Machine).

Motor de ruteo de código abierto basado en OpenStreetMap que se puede alojar en la propia infraestructura.

Pros: datos reales de calles, sin costo por llamada, sin dependencia de red externa.

Contras: requiere levantar y mantener un contenedor adicional con los datos de OSM para Argentina (~500 MB), agrega complejidad operacional al Docker Compose.

Alternativa 3

Opción C: Google Maps Distance Matrix API. ELEGIDA

Servicio REST de Google que devuelve tiempos y distancias reales de ruta entre múltiples orígenes y destinos.

Pros: alta precisión considerando calles, sentidos viales y tráfico en tiempo real; SDK oficial para Python; documentación extensa; sin infraestructura adicional a mantener.

Contras: costo por elemento de la matriz (N^2 llamadas por optimización); dependencia de red en el camino crítico del motor logístico; requiere API key con billing habilitado en Google Cloud.

DECISIÓN

Decisión tomada

Google Maps Distance Matrix API. El servicio se consume exclusivamente desde movo-svc-pricing-logistics. Cada llamada incluye un máximo configurable de puntos para acotar el costo. La API key se gestiona en AWS Secrets Manager.

CONSECUENCIAS

Trade-offs

Positivo: la matriz de distancias refleja condiciones reales de ruta, mejorando la calidad de las rutas optimizadas que el sistema sugiere al transportista.

Positivo: sin infraestructura adicional a operar.

Trade-off negativo: si la API de Google Maps no responde, el motor de optimización no puede completar el cálculo. Estrategia de fallback: se usa automáticamente un cálculo con distancia haversine con menos precisión.

Trade-off negativo: el costo escala cuadráticamente con la cantidad de paradas. Se mitiga definiendo un límite máximo de paradas por llamada (configurable vía variable de entorno GOOGLE_MAPS_MAX_ELEMENTS). Trade-off negativo: dependencia de un servicio externo de pago en una función crítica del producto.

13.8.10 ADR-009 · Infraestructura como Código (Terraform) y Gestión de DNS

ACEPTADO

CONTEXTO

Situación

El ADR-006 define que la infraestructura de backend corre en EC2 con Docker Compose, con un ambiente dev y un ambiente prod, pero no define cómo se aprovisionan esos recursos de AWS (VPC, EC2, security groups, buckets S3, secrets, roles IAM). Hasta ahora la creación de estos recursos se asume manual o ad-hoc, sin versionado ni reproducibilidad. Adicionalmente, el dominio del proyecto está registrado y gestionado en Cloudflare, y no existe una decisión documentada sobre cómo se administran los registros DNS que apuntan a cada ambiente.

CRITERIOS DE COMPARACIÓN

Criterios

Reproducibilidad del entorno (poder recrear dev o prod desde cero sin pasos manuales).

Curva de aprendizaje para un equipo de 5, uno de ellos con conocimientos de DevOps.

Costo operacional adicional.

Consistencia con las decisiones ya tomadas en ADR-006 (EC2 + Docker Compose, no contenedores gestionados).

Evitar introducir dependencias o migraciones innecesarias, como mover el dominio de proveedor de DNS sin una razón técnica real.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Opción A: Aprovisionamiento manual documentado

Crear los recursos a mano desde la consola de AWS y documentar los pasos en un runbook.

Pros: cero curva de aprendizaje adicional, no depende de una herramienta nueva.

Contras: no es reproducible de forma confiable, alto riesgo de drift entre dev y prod, recrear un ambiente ante un desastre depende de que el runbook esté perfectamente actualizado y de que alguien lo siga al pie de la letra.

Alternativa 2

Opción B: Terraform ELEGIDA

Herramienta de IaC declarativa, con providers oficiales tanto para AWS como para Cloudflare, lo que permite gestionar cómputo y DNS desde el mismo lugar.

Pros: reproducibilidad total de dev y prod a partir del mismo código, historial de cambios versionado en Git, el equipo puede revisar cambios de infraestructura vía Pull Request igual que el código de aplicación, un mismo módulo parametrizado sirve para ambos ambientes.

Contras: requiere que el equipo aprenda HCL y los conceptos de state management, agrega un backend remoto (S3 + DynamoDB) como nueva pieza de infraestructura a mantener.

Alternativa 3

Opción C: AWS CloudFormation

Nativo de AWS, sin necesidad de gestionar un backend de state por separado.

Pros: integración directa con la consola de AWS, sin costo adicional de backend.

Contras: sintaxis YAML/JSON más verbosa que HCL, no tiene provider nativo para Cloudflare, lo que obligaría a gestionar DNS por fuera de la herramienta de todos modos, menos portable si en el futuro se evalúa otro proveedor cloud.

DECISIÓN

Decisión tomada

Se adopta Terraform como herramienta de Infraestructura como Código para los ambientes de AWS (dev y prod), reemplazando el aprovisionamiento manual. El state se almacena en un backend remoto (S3 + DynamoDB para locking). La infraestructura se organiza en módulos reutilizables (networking, compute, storage, secrets, IAM, DNS) parametrizados por ambiente. El dominio permanece en Cloudflare y sus registros DNS se gestionan con el provider oficial de Cloudflare para Terraform, en el mismo repositorio de infraestructura. No se migra el dominio a Route53. El TLS sigue resolviéndose con Let's Encrypt en cada instancia EC2, sin cambios respecto a lo definido en ADR-006.

CONSECUENCIAS

Trade-offs

Positivo: dev y prod quedan definidos por el mismo código parametrizado, reduciendo el riesgo de drift entre ambientes.

Positivo: cualquier cambio de infraestructura pasa por Pull Request y queda en el historial de Git, mejorando la trazabilidad exigida por la cátedra.

Positivo: se evita una migración de dominio sin justificación técnica, ahorrando tiempo del equipo.

Positivo: gestionar AWS y Cloudflare desde el mismo Terraform simplifica el proceso de crear un ambiente nuevo desde cero.

Trade-off negativo: el equipo debe aprender Terraform y HCL, lo que consume tiempo del cronograma del Proyecto Final. Se mitiga empezando por el módulo más simple (storage o DNS) antes de abordar networking y compute.

13.8.11 ADR-010 · Confianza en la Red Interna del API Gateway

ACEPTADO

CONTEXTO

Situación

Se implementó el middleware del gateway: valida el access token una única vez y propaga la identidad verificada hacia los microservicios internos mediante los headers x-user-id, x-user-roles y x-kyc-status, en vez de que cada servicio revalide el JWT por su cuenta. Esto evita duplicar la lógica de autenticación (ADR-004) en cinco servicios distintos, pero abre una pregunta de diseño: ¿deben los microservicios internos volver a validar esos headers de alguna forma (por ejemplo, una firma que solo el gateway pueda generar), o alcanza con confiar en que solo el gateway puede alcanzarlos?

CRITERIOS DE COMPARACIÓN

Criterios

Complejidad de implementación adicional. Latencia agregada por validaciones extra en cada request interno. Nivel de aislamiento de red real ya garantizado por la infraestructura (ADR-006, ADR-009). Alcance apropiado para un Proyecto Final frente a una arquitectura zero-trust de nivel productivo.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Opción A: Confianza total en la red interna ELEGIDA Los microservicios internos confían en los headers x-user-* sin revalidarlos, apoyados en que la red Docker interna no es alcanzable desde afuera: por ADR-006 y ADR-009, solo el API Gateway expone un puerto público (443), y todos los demás contenedores (svc-users, svc-shipments, svc-payments, svc-pricing-logistics, svc-admin, postgres, redis) solo son alcanzables dentro de esa red. Pros: cero overhead de validación adicional, ningún microservicio necesita conocer JWT_SECRET ni reimplementar lógica de verificación. Contras: si un atacante compromete la red interna (una vulnerabilidad en cualquier servicio, o una mala configuración de red), puede falsificar cualquier identidad hacia cualquier servicio sin que nada lo detecte. Es un modelo de seguridad perimetral, no zero-trust.

Alternativa 2

Opción B: Firma de los headers de identidad El gateway firma los headers x-user-* (HMAC con clave compartida, o los reempaqueta en un JWT interno de corta vida) y cada microservicio valida esa firma antes de confiar en la identidad recibida. Pros: aislamiento real; un atacante en la red interna no puede falsificar identidad sin la clave de firma. Contras: duplica exactamente el problema que

este diseño buscaba evitar, cada servicio vuelve a implementar lógica de verificación -, agrega gestión de una clave compartida y latencia extra en cada request interno.

Alternativa 3

Opción C: mTLS entre el gateway y los servicios internos Cada contenedor tiene certificados propios; la comunicación interna se autentica y cifra mutuamente. Pros: estándar de industria para zero-trust en microservicios. Contras: requiere una CA interna y rotación de certificados, y Docker Compose no tiene soporte nativo para esto sin un service mesh (Istio, Linkerd), completamente fuera de alcance para un equipo de 5 personas con una EC2 por ambiente (ADR-006).

DECISIÓN

Decisión tomada

Opción A: los microservicios internos confían en los headers x-user-id, x-user-roles y x-kyc-status inyectados por el gateway, sin revalidación criptográfica adicional. Como defensa adicional, el gateway elimina explícitamente cualquier header x-user-* que llegue de un cliente externo antes de inyectar los propios derivados del JWT verificado, para que nadie pueda inyectar identidad falsa desde afuera de la red interna.

CONSECUENCIAS

Trade-offs

Positivo: cero overhead de validación adicional en cada microservicio; ninguno necesita conocer JWT_SECRET ni implementar verificación de tokens, coherente con el objetivo de MOVO-68 de centralizar la autenticación en un único punto. Positivo: consistente con la topología de red ya decidida en ADR-006 y ADR-009, sin introducir infraestructura nueva. Trade-off negativo (aceptado conscientemente): si un atacante logra acceso a la red Docker interna de una EC2, puede alcanzar cualquier microservicio directamente y falsificar cualquier identidad vía los headers x-user-*, sin que ningún servicio lo detecte. Es un modelo de seguridad perimetral, no zero-trust; se acepta para el alcance de un Proyecto Final y se documenta acá para que la limitación quede explícita y no implícita. Si el proyecto escalara a un entorno productivo real, correspondería evaluar la Opción B o C como siguiente paso.

13.8.12 ADR-011 · Adopción de Prisma como ORM para los servicios de MOVO

ACEPTADO

CONTEXTO

Situación

Los 4 servicios Node de MOVO acceden a Postgres con pg a mano y versionan migraciones como.sql aplicados manualmente vía scripts/run-migrations.sh, que no lleva registro de qué se aplicó, solo "funcionaba" porque la única migración real (movo-svc-users) usa guards IF NOT EXISTS. Automatizar migraciones en el deploy expone el problema: cualquier migración futura no-idempotente rompería

en el segundo deploy. movo-svc-users es hoy el único servicio con dominio real; los otros tres siguen siendo placeholders, momento más barato para fijar la convención antes de que acumulen código propio.

CRITERIOS DE COMPARACIÓN

Criterios

Costo de adopción ahora vs. después en cada servicio. Tracking automático de migraciones. Consistencia entre los 4 servicios Node, que no cada uno resuelva persistencia distinto. Curva de aprendizaje. Consistencia con el stack (ADR-002). Compatibilidad con schema-por-servicio (ADR-003).

ALTERNATIVAS CONSIDERADAS

Alternativa 1

SQL a mano + tracking propio en los 4. Reinventa lo que un migration runner ya resuelve, sin tipado.

Alternativa 2

Prisma como ORM estándar del proyecto ELEGIDA. Tracking nativo, tipado end-to-end, multi-schema nativo, baselining explícito, una sola convención repetible entre servicios en vez de que cada uno la resuelva por su cuenta.

Alternativa 3

TypeORM/Drizzle/Kysely. Ninguno tan maduro para baselinear una DB existente; sin experiencia previa del equipo, sin ventaja clara sobre Prisma.

DECISIÓN

Decisión tomada

Se adopta Prisma como ORM estándar para los 4 servicios Node (svc-users, svc-shipments, svc-payments, svc-admin), no una decisión puntual de uno solo. svc-pricing-logistics (Python) queda fuera. Arranca en movo-svc-users por ser el único con dominio real hoy; los otros tres adoptan el mismo patrón desde el arranque cuando modelen su dominio, en vez de escribir SQL a mano y migrar después.

CONSECUENCIAS

Trade-offs

Positivo: idempotencia resuelta en los 4 servicios sin mantenimiento propio; tipado de punta a punta en todo el backend Node; una sola convención de persistencia; punto más barato del proyecto para fijarlo. Negativo: herramienta nueva a mitad de proyecto (mitigado escalonando la adopción, no big-bang); casos de borde de Prisma 7 no documentados oficialmente (documentados en el código para los siguientes servicios). Fuera de esta implementación: cómo correr prisma migrate deploy contra la EC2 real (Postgres no expone puerto público, ADR-010).

13.8.13 ADR-012 · Twilio como proveedor de SMS para verificación de OTP

ACEPTADO

CONTEXTO

Situación

MOVO-71 requiere enviar códigos OTP de 6 dígitos por SMS para verificar el teléfono del usuario antes del alta de cuenta (Módulo 1 del alcance). El envío de SMS real tiene costo por mensaje y requiere alta de cuenta con un proveedor externo, para el alcance de un Proyecto Final, sostener ese costo durante todo el desarrollo (múltiples sprints, testing repetido, CI) no se justifica frente al riesgo ya identificado (R10: costos de APIs externas; R11: necesidad de abstraer integraciones externas para poder testear sin red).

CRITERIOS DE COMPARACIÓN

Criterios

Costo operativo durante desarrollo/testing. Facilidad de integración (SDK oficial, documentación). Posibilidad de reforzar la demo final con WhatsApp Business API. Testeabilidad sin red (no depender de un proveedor real en CI). Consistencia con el patrón de abstracción ya usado en el proyecto para integraciones externas (ADR-007, S3 presigned URLs).

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Enviar SMS real en todo ambiente (dev/test/CI/prod) contra un único proveedor. Simplicidad de una sola implementación, pero costo recurrente en cada corrida de tests/CI y dependencia de red en el camino crítico de testing, contradice R10/R11.

Alternativa 2

Interfaz SmsProvider con dos implementaciones, ConsoleSmsProvider (default en dev/test/CI, loguea el código sin costo) y TwilioSmsProvider (real, activado solo con SMS_PROVIDER=twilio, reservado para la demo final). Sin costo durante el desarrollo, testeable sin red, cambio de proveedor a una sola variable de entorno.

Alternativa 3

WhatsApp Business API en vez de SMS. Requiere aprobación de plantillas de mensaje y alta de cuenta de Meta Business, con lead time y fricción mayores a los de Twilio para el alcance de este sprint, queda como opción a evaluar para la demo final, no para la implementación base.

DECISIÓN

Decisión tomada

Twilio como proveedor concreto de envío de SMS (AC8 de MOVO-71), detrás de una interfaz SmsProvider (src/adapters/sms-provider.ts en movo-svc-users) con dos implementaciones intercambiables por la variable de entorno SMS_PROVIDER (console | twilio, default console). El código de verificación nunca se expone por HTTP en ningún caso, independientemente del proveedor activo.

Autenticación contra la API de Twilio vía API Key + Secret, no Account SID + Auth Token (recomendación explícita de Twilio al crear la cuenta): el Auth Token da acceso total a la cuenta y no se puede limitar en permisos ni revocar sin regenerarlo entero; una API Key se puede acotar por producto/permiso y revocar individualmente sin afectar el resto de las integraciones. El Account SID se sigue necesitando (identifica la cuenta ante la API), pero pasa a ser un dato de configuración, no una credencial, las credenciales son TWILIO_API_KEY_SID/TWILIO_API_KEY_SECRET.

CONSECUENCIAS

Trade-offs

- **Positivo:** cero costo de SMS durante todo el desarrollo/testing/CI; el patrón de adapter (ya usado conceptualmente en ADR-007 para S3) queda establecido como convención para integraciones externas futuras (ej. Didit.me en MOVO-72); los tests de integración corren sin red.
- Negativo / limitación aceptada: Twilio real nunca se ejercita en CI ni en desarrollo, solo se activa manualmente para la demo final, así que un bug específico de la integración real (formato de número, rate limits del proveedor, etc.) podría no detectarse hasta ese momento. Mitigado con un test unitario del adapter de Twilio con el SDK mockeado (única excepción documentada a la regla de "nunca mockear" del plan de testing, justificada por tratarse de una API de pago externa).
- Riesgo confirmado en la práctica: al probar contra Twilio real (cuenta trial, número de prueba verificado) apareció exactamente el tipo de problema anticipado arriba, Twilio no reconoce el "9" que Argentina inserta en el E.164 de un celular (+549XXXXXXXXXX, convención de marcado para voz) como el mismo destinatario que +54XXXXXXXXXX: un número verificado como recipient de la cuenta trial devolvía error 572002 ("destinatario no verificado") en formato +549..., y pasaba ese chequeo en +54.... Se resolvió quitando el "9" únicamente al armar el to de la llamada a Twilio, sin tocar la normalización +549... que el resto del sistema usa para unicidad/persistencia (MOVO-70). Vale como precedente para cualquier integración futura con un proveedor de SMS/voz externo sobre números argentinos.

13.8.14 ADR-013 · Refresh token con TTL de 90 días, rotación de un solo uso y detección de reuso

ACEPTADO MOVO-75

CONTEXTO

Situación

ADR-004 fijó el refresh token en 7 días. Al implementar el login (MOVO-74/MOVO-75) el equipo priorizó minimizar cuánto tienen que volver a loguearse los usuarios en una app que no maneja datos bancarios. Un TTL más largo agranda la ventana de exposición si un refresh token es robado, por lo que la misma US agrega rotación y detección de reuso.

Alcance

Reemplaza el valor de TTL de ADR-004.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Mantener el TTL de 7 días (valor original de ADR-004). Descartada: obliga a loguearse con más frecuencia, contrario a la prioridad del equipo para una app sin datos bancarios.

Alternativa 2

TTL de 90 días con rotación de un solo uso y detección de reuso ELEGIDA. Compensa la ventana de exposición mayor con una revocación automática ante el robo.

DECISIÓN

Decisión tomada

El refresh token vive 90 días en Redis. Es un token opaco con la forma {userId}.{tokenId}.{secret}, de modo que se puede ubicar la key en Redis a partir de lo que ve el cliente. La rotación es de un solo uso: al refrescar, la sesión vieja queda como tombstone (used: true); un refresh ya canjeado que se vuelve a presentar revoca todas las sesiones del usuario (detección de robo). El logout recibe el refresh token en el body (los claims del access token no incluyen tokenId) y siempre responde 204, incluso con un token ajeno o inválido, para no filtrar información. El access token mantiene su TTL de 60 minutos (MOVO-74).

En el mobile (MOVO-76) el interceptor HTTP refresca con single-flight ante un 401 y no reintenta si el 401 viene de un Authorization explícito del llamador, para no competir con el refresh proactivo ni disparar la detección de reuso.

CONSECUENCIAS

Trade-offs

Ventana de exposición mayor si un refresh token es robado; se mitiga con la rotación de un solo uso y la detección de reuso (reusar un refresh ya canjeado revoca todas las sesiones del usuario). Un cliente que dispare dos refrescos concurrentes con el mismo token puede ser interpretado como reuso; por eso el mobile usa single-flight. DD-043 (propuesto) revisa este punto para el panel web, que sí puede enviar solicitudes simultáneas con la misma cookie.

REFERENCIAS

Referencias

MOVO-74 (login), MOVO-75 (este ticket), MOVO-76 (refresh automático en mobile), ADR-004 (valor reemplazado), DD-043 (refresh idempotente del panel).

13.8.15 ADR-014 · Google Maps como proveedor de geocoding detrás de GeocodingProvider

ACEPTADO MOVO-73

CONTEXTO

Situación

El wizard de registro (MOVO-73) incluye un paso de mapa: la dirección que carga el usuario se geocodifica a latitud/longitud y se guarda en la tabla nueva users.address (una por registro). Es la primera implementación real de un servicio de Google Maps en el proyecto, aunque ADR-008 y ADR-015 ya habían decidido Google Maps para movo-svc-pricing-logistics, que todavía era un esqueleto.

DECISIÓN

Decisión tomada

Google Maps (Geocoding API) como proveedor, detrás de una interfaz GeocodingProvider en movo-svc-users. Un mock determinístico es el default de dev/test/CI y Google real se activa con GEOCODING_PROVIDER=google (mismo molde que SmsProvider, ADR-012). El mobile no llama a Google directamente para geocodificar: usa el endpoint público POST /geocode, que proxea la Geocoding API

del lado del servidor. Las keys de Google Maps del cliente van en app.config.js / variables de EAS, nunca en app.json ni eas.json, porque esos archivos se trackean en git.

CONSECUENCIAS

Trade-offs Hay que provisionar dos API keys de Google distintas: Geocoding API del lado servidor (restringida por IP) y Maps SDK del lado cliente del mobile (restringida por bundle id y SHA). Ninguna estaba cargada al momento de la decisión, mismo estado pendiente que las credenciales de Twilio y Didit. El endpoint /geocode es público a propósito (el registro ocurre antes de tener sesión); los proxies de Places que le siguieron (MOVO-120) siguieron el mismo criterio.

REFERENCIAS

Referencias MOVO-73 (onboarding mobile), MOVO-120 (proxy de Places Autocomplete), ADR-008, ADR-012 (molde de proveedor), ADR-015.

13.8.16 ADR-015 · Migración de Distance Matrix API (Legacy) a Routes API (Compute Route Matrix)

ACEPTADO

CONTEXTO

Situación ADR-008 (Aceptado) definió Google Maps Distance Matrix API como proveedor de datos de distancia para el motor VRPTW, consumido exclusivamente desde movo-svc-pricing-logistics.

En agosto de 2026 se confirmó que Distance Matrix API y Directions API fueron designadas "Legacy" por Google desde marzo de 2025: quedan feature-frozen (sin nuevas funcionalidades), no están disponibles para proyectos nuevos de Cloud, y no tienen fecha de baja fijada, Google se compromete a dar 12 meses de aviso antes de discontinuarlas. Google recomienda explícitamente que los proyectos nuevos usen Routes API desde el arranque.

A la fecha de esta revisión, ningún servicio del monorepo tiene código de integración real con Google Maps: movo-svc-pricing-logistics sigue usando el mock de distancia haversine definido para desarrollo (spike MOVO-50). Es el punto más barato del proyecto para fijar esta decisión, antes de escribir la integración real, mismo criterio que llevó a adoptar Prisma en ADR-011.

CRITERIOS DE COMPARACIÓN

Criterios Vigencia del producto a largo plazo (no arrancar sobre un servicio que el proveedor ya desalienta para proyectos nuevos).

Costo por elemento y previsibilidad del gasto, crítico para un equipo de estudiantes sin margen de error en USD.

Límite de elementos por request y latencia.

Consistencia con la decisión ya tomada en ADR-008 de no introducir Directions API como dependencia separada.

Mecanismos disponibles para poner topes duros de gasto (no solo alertas).

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Mantener Distance Matrix API (Legacy), decisión original de ADR-008.

Pros: cero esfuerzo de cambio; la documentación y el presupuesto del Plan de Proyecto ya están redactados sobre esta opción.

Contras: Google no la recomienda para proyectos nuevos; está feature-frozen; ante una migración forzada eventual (sin fecha, pero con 12 meses de aviso) el trabajo se termina haciendo igual, más tarde y con código en producción para re-escribir. Dado que hoy no hay código escrito, no hay ninguna razón real para no partir directamente de la opción vigente.

Alternativa 2

Routes API, método Compute Route Matrix, tier Basic.

Pros: es la opción que Google recomienda activamente para proyectos nuevos. Mismo orden de magnitud de costo que la legacy en tier Basic ($5 por 1.000 elementos, sin campos de tráfico en vivo ni peajes). Límite de 625 elementos por request contra 100 de la legacy. Respuesta en streaming. Mecanismos de control de costo documentados oficialmente por Google a nivel de plataforma (cuota diaria dura, alertas de presupuesto).

Contras: formato de request/response distinto (POST JSON + header X-Goog-FieldMask obligatorio), lo que implica escribir el cliente HTTP contra este formato desde el arranque (sin costo de migración real, porque no existía integración previa). La estructura de tier gratuito por SKU (vigente desde marzo 2025) reemplaza al crédito plano de USD 200/mes que contemplaba el Plan de Proyecto, el monto exacto para este SKU queda pendiente de confirmación (ver sección de Costos).

Alternativa 3

Proveedor de ruteo no-Google (ej. OSRM self-hosted).

Se descarta por el mismo motivo ya documentado en ADR-008: requiere levantar y mantener un contenedor adicional con datos de OpenStreetMap para Argentina, complejidad operacional no justificada para el alcance del equipo. Sin cambios respecto al análisis original.

DECISIÓN

Decisión tomada

Se adopta Routes API, método Compute Route Matrix, consumido exclusivamente desde movo-svc-pricing-logistics (mismo aislamiento que definía ADR-008). Las requests se limitan al tier Basic (sin tráfico en tiempo real, sin peajes, sin modificadores de vehículo de dos ruedas) para mantener el costo previsible en $5 por cada 1.000 elementos facturables.

Se mantiene la variable de entorno GOOGLE_MAPS_MAX_ELEMENTS (ya prevista en ADR-008) como límite máximo configurable de elementos por request.

Como capa adicional de protección de costo, nueva respecto a ADR-008, se configura un daily quota limit en Google Cloud Console sobre el proyecto, en dev y en prod, de forma que al alcanzar el tope diario configurado la API deja de responder en lugar de seguir facturando. Se suma un billing budget alert como alarma temprana independiente del tope de cuota.

Pendiente antes de cerrar el número final en el Plan de Proyecto: confirmar en la calculadora de precios de Google Cloud a qué categoría de SKU (Essentials/Pro/Enterprise) corresponde Compute Route Matrix en tier Basic y cuántos eventos gratuitos mensuales incluye, dado que reemplaza al crédito plano de USD 200/mes ya presupuestado.

CONSECUENCIAS

Trade-offs

Positivo: se evita una migración forzada más adelante, aplicando el mismo principio que llevó a adoptar Prisma en ADR-011, fijar la convención correcta en el punto más barato del proyecto.

Positivo: el límite de 625 elementos por request (vs. 100 de la legacy) da más margen para las matrices VRPTW sin trocear llamadas.

Positivo: el mecanismo de cuota diaria dura elimina el riesgo de un cargo inesperado en USD, una preocupación legítima para un equipo de estudiantes sin margen de error presupuestario.

Positivo: académicamente defendible, la elección sigue la recomendación activa del proveedor en lugar de una API que el proveedor mismo desalienta para proyectos nuevos.

Trade-off negativo: el cliente HTTP se escribe contra Routes API sin haber podido validar antes con la legacy, se acepta porque de cualquier manera no había integración escrita todavía.

Trade-off negativo: el monto exacto del tier gratuito mensual queda pendiente de confirmación antes de cerrar el presupuesto definitivo del Plan de Proyecto, se documenta como acción de seguimiento, no como bloqueante.

13.8.17 ADR-016 · Resend como proveedor de email detrás de EmailProvider

ACEPTADO MOVO-139

CONTEXTO

Situación

MOVO-139 incorpora el primer canal de email del proyecto. Cierra la limitación que MOVO-133 y MOVO-134 dejaron documentada (sin EmailProvider no se podía notificar al email anterior) y es precondición de MOVO-64 (recuperación de contraseña por email), además de habilitar la verificación de email por OTP.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

AWS SES. Descartada: salir del sandbox de SES exige una aprobación manual de AWS con tiempos impredecibles.

Alternativa 2	Resend ELEGIDA. El free tier (3.000 mails por mes) cubre de sobra el TFG.

DECISIÓN

Decisión tomada	Resend detrás de una interfaz EmailProvider, con el molde de ADR-012: factory por EMAIL_PROVIDER, implementación de consola como default de dev/test/CI y Resend real con EMAIL_PROVIDER=resend, con fail-fast al arrancar si faltan RESEND_API_KEY o EMAIL_FROM. Resend se consume por fetch contra su API HTTP, no por su SDK (mismo criterio que otros proveedores del repo). El cuerpo viaja como { text, html } (un mail transaccional sin parte de texto plano es disparador de filtros de spam) y los templates viven en código, no en la UI de Resend, para que se versionen y se testeen. El canal se persiste en el registro de Redis del OTP (OtpRecord.channel) para que el reenvío sepa si el destino es teléfono o email. El OTP de cambio de email va al email nuevo, y el aviso al email anterior es best-effort después del UPDATE.

CONSECUENCIAS

Trade-offs	Un proveedor externo más del que depender. El dominio de envío necesita SPF/DKIM propios (un terraform apply en movo-infra; el DNS ya se maneja por Cloudflare) y la cuenta queda sin verificar hasta la demo, igual que Twilio y Didit.

REFERENCIAS

Referencias	MOVO-139, MOVO-133, MOVO-134, MOVO-64, ADR-012 (molde de proveedor).

13.8.18 ADR-017 · Statelessness de movo-svc-pricing-logistics y ubicación de la entidad Offer

ACEPTADO

CONTEXTO

Situación	El motor de precios y el algoritmo VRPTW de movo-svc-pricing-logistics necesitan ser invocados en dos momentos del flujo: al crear un envío (cotización sugerida, ver DD-018) y al declarar un viaje / crear una oferta (matching y optimización de ruta). La entidad Offer (oferta de un transportista sobre un envío) participa de una transacción que también actualiza el estado del envío (Shipment): al aceptar una oferta, esta pasa a `accepted`, las demás ofertas pendientes del mismo envío pasan a `superseded`, y el envío pasa a `assignment_pending`, las tres cosas en una sola operación atómica. Definir en qué servicio vive `Offer`, y si pricing-logistics necesita base de datos propia, condiciona la arquitectura de todo el dominio de negociación.
Criterios	Atomicidad transaccional para operaciones que tocan `Offer` y `Shipment` simultáneamente. Consistencia con ADR-001 (microservicios) y ADR-003 (schema por servicio dentro de la misma instancia Postgres). Complejidad operacional de su-

mar un schema a un servicio que hoy es puramente de cómputo. Fijar la decisión en el punto más barato del proyecto, antes de escribir código que dependa de ella (mismo criterio que ADR-011/013/016).

ALTERNATIVAS CONSIDERADAS

Alternativa 1

`Offer` vive en movo-svc-pricing-logistics, con schema Postgres propio. Pros: agrupa toda la lógica de "negociación y precio" en un solo servicio. Contras: aceptar una oferta requeriría actualizar `Offer` (en pricing-logistics) y `Shipment` (en shipments) en la misma operación lógica, al ser schemas de servicios distintos, no hay transacción ACID nativa entre ambos, y habría que implementar un patrón de saga o compensación para algo que hoy es una sola escritura atómica. Además, pricing-logistics dejaría de ser un servicio de cómputo puro, contradiciendo la razón original de separarlo en Python (ADR-002: cómputo numérico y OR-Tools, no persistencia transaccional).

Alternativa 2

`Offer` vive en movo-svc-shipments, en el mismo schema que `Shipment` ELEGIDA. Pros: aceptar una oferta es una única transacción SQL dentro del mismo schema (`shipments.offers` y `shipments.shipments`): todo o nada. pricing-logistics se mantiene sin base de datos propia, recibe un request HTTP, calcula, devuelve la respuesta, sin persistir nada. Contras: `price_offered` (en shipments) y `suggested_price_ars` calculado por pricing-logistics quedan en servicios distintos; cualquier reporte que cruce ambos requiere una llamada HTTP entre servicios (trade-off ya aceptado en ADR-001).

DECISIÓN

Decisión tomada

movo-svc-pricing-logistics es un servicio completamente stateless: no tiene schema propio en PostgreSQL, no persiste ofertas ni ningún otro dato entre requests. Su única responsabilidad es recibir inputs (`POST /quote`, matriz de rutas) y devolver un resultado calculado. La entidad `Offer` vive en movo-svc-shipments, en el schema `shipments`, junto a `Shipment`, precisamente para que aceptar una oferta sea una transacción atómica de base de datos y no una saga distribuida entre dos servicios.

CONSECUENCIAS

Trade-offs

Positivo: la operación más sensible del dominio de negociación (aceptar una oferta) es una transacción SQL simple con bloqueo optimista, sin necesidad de compensación ni sagas. Positivo: pricing-logistics es trivial de escalar horizontalmente, al no tener estado, cualquier instancia atiende cualquier request, sin sticky sessions ni replicación de datos. Positivo: consistente con la razón original de elegir Python/FastAPI para este servicio en ADR-002. Trade-off negativo: el acoplamiento de negocio entre `shipments` y la lógica de ofertas queda en el mismo servicio que el ciclo de vida del envío, en vez de aislado en un dominio de "pricing/negociación" separado, se acepta porque la atomicidad transaccional pesa más que la pureza de bounded context para esta operación específica.

Nota técnica: limitación de silent push notifications en iOS

El modelo de tracking on-demand (ver Estudio Inicial, sección P10) depende de que el backend pueda despertar a la app del transportista en segundo plano mediante una notificación push silenciosa (background push) para solicitar su posición actual. En iOS, Apple no garantiza la entrega ni el tiempo de entrega de este tipo de notificaciones: el sistema operativo puede demorarlas, agruparlas o directamente no entregarlas si evalúa que el dispositivo está en bajo consumo de batería, si la app fue recientemente forzada a cerrarse por el usuario, o por throttling general. Es una restricción de la plataforma, no un bug del diseño. Mitigación: el cliente define un timeout configurable (valor de referencia inicial: 10 segundos) tras el cual, si no llegó una respuesta de posición actualizada, la UI cae automáticamente al fallback ya contemplado en el diseño (última posición conocida + indicador de antigüedad), sin bloquear la pantalla ni mostrar error. En Android esta limitación es sustancialmente menor gracias a Firebase Cloud Messaging con prioridad alta (high-priority FCM), por lo que el mismo valor de timeout puede usarse en ambas plataformas sin perjudicar la experiencia en Android.

13.8.19 ADR-018 · Handshake de custodia: firma ECDSA P-256 generada en el dispositivo

ACEPTADO MOVO-158, MOVO-195

CONTEXTO

Situación

El Cryptographic Handshake (MOVO-6) confirma el cambio de custodia (emisor a transportista en el retiro, transportista a receptor en la entrega) combinando una firma asimétrica del dispositivo con una validación de proximidad GPS. Es la primera criptografía asimétrica del repo y no había precedente propio que reusar. MOVO-157 registra la clave pública por dispositivo en svc-users; MOVO-158 valida firma y GPS en svc-shipments; MOVO-195 genera el par de claves y firma en el mobile.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Ed25519 (recomendado en el ticket MOVO-195, a definir). Descartada: el backend ya validaba ECDSA P-256 desde MOVO-158 y habría rechazado toda firma Ed25519 con 422.

Alternativa 2

react-native-quick-crypto o una clave no exportable en Keychain/Keystore nativo. Evaluadas con el usuario y descartadas: requieren módulo nativo y dev client/rebuild.

Alternativa 3

@noble/curves (ELEGIDA para el mobile). JavaScript puro, sin módulo nativo.

DECISIÓN

Decisión tomada

El handshake firma con ECDSA P-256 / SHA-256 vía WebCrypto (webcrypto.subtle de node:crypto en el backend). La clave pública viaja en formato raw (punto EC sin comprimir, 65 bytes) y la firma en formato IEEE P1363 (raw r‖s, 64

bytes), no DER. La clave privada nunca sale del dispositivo: por eso POST /shipments/:id/handshake/generate nunca firma, solo crea el nonce (Redis, TTL 15 s) con las coordenadas GPS del cedente y devuelve el string canónico {shipmentId}:{stage}:{nonce} para que el mobile lo firme y arme el QR. El stage se infiere del estado del envío, nunca lo manda el cliente. El estado pendiente vive en Redis y el commit de la transición es atómico en Postgres (compare-and-swap más inserts en shipment_events y handshake_events). En el mobile, @noble/curves (p256) genera y firma; el par se crea una vez, se persiste en expo-secure-store y sobrevive al logout porque identifica al dispositivo, no a la sesión.

CONSECUENCIAS

Trade-offs

La firma en formato no-DER es una convención propia del proyecto: el mobile tiene que hablar exactamente el mismo formato y no hay un estándar externo que un tercero pueda verificar directamente. La clave privada pasa por JavaScript del lado mobile en vez de quedar aislada en hardware; se acepta por simplicidad y alcance del TFG (sin dev client ni rebuild nativo). El modelo de MOVO-157 no soporta múltiples dispositivos: un login en un dispositivo nuevo rota la clave pública vigente y el dispositivo viejo queda con firmas que el backend ya no valida. El handshake incluyó un bug de carrera detectado con Postgres real (el stage se releyó en vivo); se corrigió guardando el stage dentro del desafío pendiente de Redis.

REFERENCIAS

Referencias

MOVO-6 (Cryptographic Handshake), MOVO-157 (registro de clave pública), MOVO-158 (validación en backend), MOVO-159/160 (QR y UI mobile), MOVO-195 (par de claves en el dispositivo).

13.8.20 ADR-019 · Canal de tiempo real – WebSocket nativo (@fastify/websocket)

ACEPTADO

CONTEXTO

Situación

El tracking en tiempo real es uno de los diferenciadores declarados de MOVO y no tenía una sola línea de código escrita. En el relevamiento del 08/09/2026 (MOVO-200) no apareció ninguna tarea de WebSocket en todo el backlog – la única mención en el board era MOVO-159 AC4, en condicional ("hace polling, o se suscribe si hay canal de Socket.io disponible"). Ese condicional era el síntoma de que nadie sabía con certeza si el canal existía. El relevamiento del monorepo (gateway, movo-svc-shipments, movo-mobile) confirmó por escrito que no hay ninguna dependencia de WebSocket/Socket.io/SSE instalada ni código de conexión persistente en ningún lado – se partía de cero. Se identificaron tres consumidores reales del canal: MOVO-11 (tracking, in scope de este ADR), MOVO-26

(chat, fuera de la implementación de este spike pero con un AC ya escrito por producto que el ticket pedía considerar) y MOVO-33 (panel de admin, envíos activos en tiempo real).

Criterios

Soporte en React Native/Expo sin polyfills; reconexión automática de fábrica vs. a mano; complejidad de autenticación con el JWT existente (MOVO-68/ADR-004); comportamiento detrás del gateway, nginx y Cloudflare; si el canal elegido sirve también para el chat (MOVO-26) sin mantener dos mecanismos de tiempo real en paralelo, porque migrar más adelante sale caro; no incorporar aparato (salas, ack, namespaces) que ningún caso de uso comprometido hoy necesita.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Server-Sent Events (SSE): Pros: cambio de infraestructura mínimo (solo timeout de nginx); nativo en el browser, sirve directo para MOVO-33 sin polyfill; no trae bidireccionalidad que hoy no se usa; auth sin cambios (Authorization Bearer). Contras: no es nativo en Expo/React Native – requiere el polyfill react-native-sse (basado en XHR, sí soporta headers custom); el chat (MOVO-26) queda partido en dos mecanismos (SSE para recibir + un POST REST aparte para enviar, ya que el historial de todos modos se persiste en Postgres); no cumple la letra del AC de MOVO-26, que pide WebSockets explícitamente; sin margen para bidireccional a futuro (typing, presencia) sin cambiar de tecnología.

Alternativa 2

WebSocket nativo, @fastify/websocket **ELEGIDA**: Pros: bidireccional – un solo protocolo para tracking, chat y admin, evitando mantener dos mecanismos de tiempo real y dos configuraciones de infraestructura en paralelo; nativo tanto en el browser como en Expo/React Native, sin polyfills en ningún cliente; cumple el AC de MOVO-26 tal cual está escrito, sin necesidad de justificar un desvío del requisito en la defensa del PF; menos dependencias que Socket.io; el upgrade ocurre sobre una request HTTP normal, así que el JWT viaja en el Authorization header igual que hoy, sin fricción extra de autenticación. Contras: sin reconexión automática de fábrica – backoff y resync se programan a mano del lado del cliente; sin salas ni ack de fábrica – si el chat los necesita en el futuro, se construyen a medida; mismo cambio de infraestructura completo en nginx que necesitaría Socket.io (headers Upgrade/Connection + subir proxy_read_timeout).

Alternativa 3

Socket.io: Pros: reconexión automática con backoff configurable de fábrica; salas, namespaces y ack de mensajes sin código propio; fallback automático a long-polling en redes restrictivas; adapter con Redis (ya presente en el stack, ADR-003) si algún día se escala a más de una instancia de backend; librería madura y muy documentada. Contras: ninguno de los tres casos de uso comprometidos hoy (tracking, chat, admin) necesita salas, ack ni multi-instancia – sumar esa complejidad ahora es exactamente el scope creep que el propio documento de riesgos del proyecto identifica como riesgo R07; protocolo propio sobre WebSocket – cliente y servidor deben ser ambos Socket.io, no interopera con WS puro; dependencia más pesada de mantener y actualizar en un proyecto de un solo servidor (ADR-006); mismo requisito de infraestructura en nginx que WebSocket

nativo, no ahorra nada ahí; difícil de justificar por escrito ante la cátedra sin caer en "se eligió por comodidad", justo lo que el ticket (MOVO-200) pedía evitar explícitamente.

DECISIÓN

Decisión tomada

Se adopta WebSocket nativo, implementado con @fastify/websocket sobre el gateway (proxy, MOVO-201) y movo-svc-shipments (origen del canal). Cubre los tres consumidores identificados (MOVO-11, MOVO-26, MOVO-33) con un solo protocolo y una sola configuración de infraestructura. Autenticación: el JWT viaja en el Authorization header de la request HTTP sobre la que ocurre el upgrade, mismo mecanismo que usa hoy el gateway para requests normales (gateway/src/plugins/auth.ts) – sin esquema nuevo. Autorización: el cliente indica a qué envío quiere suscribirse (shipmentId); el servidor valida que el usuario autenticado sea emisor, transportista o receptor de ese envío (misma verificación que assertShipmentAccess en movo-svc-shipments) antes de agregar la conexión a cualquier grupo de difusión – si falla, la conexión se rechaza antes de esa suscripción. Para el panel de admin (MOVO-33) el chequeo es de rol, no de pertenencia al envío. Infraestructura: nginx (infra/nginx/templates/default.conf.template) necesita sumar los headers Upgrade/Connection en el bloque de proxy y subir proxy_read_timeout (hoy 30s) a un valor acorde a la duración esperada de una conexión de tracking/chat – Cloudflare está en modo DNS-only (confirmado en infra/docker-compose.yml) y no participa en el camino de la conexión persistente, así que no hay que investigar su comportamiento en modo proxy. Una prueba de concepto mínima (canal que empuja un mensaje desde movo-svc-shipments a un cliente Expo en dev build, con el JWT validado en el handshake) quedó implementada en una rama con instrucciones de cómo correrla (docs/tracking-poc/README.md), sin ser la implementación final. Como consecuencia directa, MOVO-159 AC4 queda resuelto a favor de reemplazar el polling por suscripción una vez que el canal esté disponible (MOVO-201) – ese reemplazo se trata como parte de los tickets de implementación de MOVO-159/MOVO-199 cuando corresponda, no de este spike.

CONSECUENCIAS

Trade-offs

Positivo – un solo protocolo y una sola configuración de infraestructura para los tres consumidores del canal (tracking, chat, admin), en vez de mantener dos mecanismos de tiempo real en paralelo. Positivo – nativo tanto en Expo/React Native como en el browser del panel de admin, sin polyfills en ningún cliente. Positivo – cumple el AC de MOVO-26 (chat vía WebSockets) tal cual está escrito por producto, sin tener que justificar un desvío del requisito ante la cátedra. Positivo – no incorpora funcionalidad (salas, ack, multi-instancia) que ningún caso de uso comprometido hoy necesita, evitando el scope creep de elegir Socket.io por comodidad (riesgo R07 del documento de riesgos del proyecto). Trade-off negativo – sin reconexión automática ni salas/ack de fábrica: ese costo se asume conscientemente y queda documentado como decisión explícita, manejable en la escala actual del proyecto (un servidor, tres casos de uso acotados), no como una carencia no evaluada. Trade-off negativo – nginx necesita un cambio de configu-

ración real (headers de upgrade + timeout) antes de que el canal funcione de punta a punta en producción; es el riesgo que el propio ticket señalaba como el más caro de descubrir tarde (funciona en local, se corta en producción por un timeout de proxy) – identificado y con la corrección puntual a aplicar, documentado como pendiente explícito en vez de una sorpresa. Alternativa a reabrir si cambian las condiciones: si el proyecto necesita escalar movo-svc-shipments a más de una instancia, o si aparecen features que dependen de lo que Socket.io da de fábrica (typing indicator, confirmación de lectura, subasta en vivo entre transportistas), ahí sí hay una razón concreta para reevaluar Socket.io – mientras esas condiciones no existan, agregar esa complejidad no está justificada.

13.8.21 ADR-020 · Tracking en tiempo real: ingesta por HTTP, difusión por WebSocket

ACEPTADO MOVO-250

CONTEXTO

Situación ADR-019 definió el canal WebSocket nativo. MOVO-250 fija el contrato entre backend y mobile para la cola offline (MOVO-203) y el envío en segundo plano (MOVO-242): el transportista puede reportar posiciones capturadas horas antes de que lleguen al servidor, y el backend tiene que tratarlas correctamente.

Alcance Complementa ADR-019 sin modificarlo.

ALTERNATIVAS CONSIDERADAS

Alternativa 1 WebSocket bidireccional para emitir y recibir. Descartada: dejaría la emisión atada a un socket que el sistema operativo mata en segundo plano mucho antes que una tarea de red puntual.

Alternativa 2 Ingesta por HTTP y difusión por WebSocket ELEGIDA. El WebSocket queda como canal de recepción.

DECISIÓN

Decisión tomada El transportista reporta su posición con POST /shipments/:id/positions (o el lote POST /shipments/positions, de 1 a 100 ítems, con un resultado por ítem) y el servidor la difunde por el WebSocket de ADR-019, que queda como canal de recepción (mensajes connected, position y status en cada transición de estado). Se ordena y agrupa por capturedAt (hora del dispositivo), no por hora de llegada: la última posición conocida solo avanza si capturedAt es estrictamente más reciente (script Lua atómico en Redis) y la traza persiste una posición por tramo de 45 s de capturedAt (SET NX por tramo). Se rechaza con 422 un capturedAt a más de 2 minutos en el futuro o a más de 2 minutos anterior al último cambio de estado del

envío. El lote tiene su propio límite en el gateway (30 requests por minuto por IP). El socket se cierra con 4001 al vencer el JWT, 4003/4004 ante un envío ajeno o inexistente y 4009 cuando el envío llega a un estado que corta el tracking.

CONSECUENCIAS

Trade-offs

La emisión no tiene ack en tiempo real; el resultado por ítem del lote lo reemplaza. La cola offline del mobile depende de que el backend sea idempotente ante reenvíos, y lo es: un tramo ya persistido no se duplica y el marcador nunca retrocede. Los claims de tramo viven 7 días en Redis, por lo que una cola vaciada después de ese plazo podría duplicar un tramo; se acepta porque coincide con el TTL de la última posición. Un error de infraestructura (base o Redis caídos) falla el request entero en vez de convertirse en un rechazo por ítem, y el cliente reintenta de forma segura.

REFERENCIAS

Referencias

MOVO-200 y MOVO-201 (canal WebSocket), MOVO-202 (ingesta), MOVO-203 (cola offline), MOVO-242 (envío en segundo plano), MOVO-250 (este ticket), ADR-019, DD-023.

13.8.22 ADR-021 · Motor de precios dinámico: combustible desde datos abiertos, demanda por zona y contrato simplificado

ACEPTADO

CONTEXTO

Situación

El endpoint POST /quote de movo-svc-pricing-logistics calcula hoy el precio sugerido de un envío con euclidean_linear_v1 (DD-018). Es una fórmula lineal provisoria que combina la distancia en línea recta entre origen y destino, el peso y un factor según el tipo de paquete, con coeficientes fijos expresados en pesos. MOVO-138 tiene que reemplazarla por el motor de precios real. Ese motor incorpora el precio del combustible, un recargo cuando en una zona hay muchos envíos esperando y pocos transportistas disponibles, y la distancia real por ruta que provee Routes API (ADR-015).

Antes de implementarlo hacía falta resolver cuatro preguntas. La primera, de dónde sale el precio del combustible y con qué frecuencia se consulta. La segunda, cómo se mide la demanda, teniendo en cuenta que pricing-logistics no tiene base de datos propia (ADR-017) y que los datos de envíos y viajes viven en el esquema shipments. La tercera, qué información ve el emisor sobre cómo se formó el precio. La cuarta, qué pasa con la cotización si Google falla. El spike MOVO-216 respondió las cuatro de forma empírica, con consultas reales a la API y un prototipo ejecutable. El informe completo y el prototipo están en docs/pricing/ del repositorio.

Criterios

Se priorizó un costo nulo o mínimo en proveedores externos, en línea con el riesgo R10 del plan de proyecto. También se buscó que los precios no se desactualicen con la inflación sin que alguien tenga que intervenir a mano. Hacía falta mantener pricing-logistics sin estado (ADR-017) y respetar un esquema por servicio (ADR-003). Se buscó que el emisor nunca vea un recargo sin explicación ni una explicación sin recargo. Y como el precio es solo una sugerencia que el emisor puede editar, una cotización no debería perderse por la caída de un proveedor.

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Fuente del precio del combustible: Se evaluaron cuatro fuentes. La API de datos abiertos de la Secretaría de Energía (dataset "Precios en surtidor, Resolución 314/2016") es oficial y gratuita, no requiere API key, tiene licencia CC-BY-4.0, se actualiza a diario y permite hacer consultas SQL en el servidor. Como contra, solo responde por HTTP, no ofrece SLA y los precios los declaran las propias estaciones, así que contienen datos desactualizados. El scraping de los precios que publican las petroleras se descartó: no hay API pública, se rompe con cualquier cambio en el sitio y los términos de uso son dudosos. Las APIs privadas que agregan precios se descartaron por ser pagas o no garantizar continuidad. Un precio fijo en la configuración, actualizado a mano, es trivial, pero en Argentina queda desactualizado en semanas. Se eligió la API de la Secretaría de Energía y el precio fijo en configuración quedó solo como respaldo de último recurso.

Alternativa 2

Dónde se calcula la demanda: La primera opción era que movo-svc-shipments cuente los envíos y transportistas de la zona y los envíe dentro del request de /quote. Hoy ese endpoint solo lo llama shipments, que ya tiene los datos a mano. La segunda era que pricing-logistics le pidiera los conteos a shipments por HTTP, pero eso genera una dependencia circular dentro del mismo request (shipments llama a pricing, que llama a shipments) y agrega un salto de red al crear un envío. La tercera era que pricing-logistics leyera directamente el esquema shipments, lo que contradice ADR-003 y ADR-017. Se eligió la primera: pricing-logistics sigue siendo una función pura que recibe dos números enteros.

Alternativa 3

Forma de la función de demanda: Un multiplicador continuo sin umbral le sumaría a todo envío algún recargo mínimo sin un aviso que lo explique. Una grilla fija de zonas, como geohash o H3, tiene efecto borde: dos retiros a doscientos metros de distancia pero en celdas distintas cotizarían distinto. Se eligió una zona definida por un radio alrededor del punto de retiro, con un umbral a partir del cual se aplica un recargo escalonado y con tope. Así el aviso de alta demanda aparece si y solo si hay recargo.

Alternativa 4

Comportamiento si falla Google Routes API al cotizar: Propagar el error es coherente con la política de no degradar que ya rige para la optimización de rutas (MOVO-205), pero deja el envío en "precio a estimar" cada vez que se agota la cuota diaria de Google. La alternativa es calcular una distancia aproximada (línea recta multiplicada por 1,3) y dejar registro en el log. Se eligió degradar.

DECISIÓN

Decisión tomada

El precio de referencia del combustible es la mediana nacional de la nafta súper que publica la API de la Secretaría de Energía, calculada sólo con las declaraciones de los últimos 30 días. El filtro de frescura es obligatorio: el dataset de precios "vigentes" conserva la última declaración de cada estación aunque tenga años. Sin filtrar, la mediana subestima el precio real en torno al 40% (en Córdoba daba $1.346 por litro contra unos $2.249 reales al 24/09/2026). Se usa la mediana nacional y no la provincial porque varias provincias tienen apenas una o dos estaciones con datos recientes, y en las provincias con más operación la diferencia es de ±2%. Se utiliza la nafta y no el gasoil porque, al momento de cotizar, todavía no hay un transportista asignado, y el transportista típico de una plataforma P2P viaja en auto particular.

El valor se guarda en Redis durante 24 horas, que coincide con la frecuencia de actualización del dataset, así que hay a lo sumo una consulta por día. Si la API falla, se usa el último valor bueno durante 7 días, y después un valor de configuración. Como la API solo responde por HTTP, se descarta cualquier respuesta con menos de 50 muestras, con un precio fuera del rango de $500 a $10.000 por litro o con una variación mayor al 25% respecto del valor anterior.

Todos los coeficientes de la tarifa (base, costo por kilómetro y costo por kilo) se expresan en litros de nafta y no en pesos, de modo que la tarifa entera se indexa sola con el precio del surtidor. Están calibrados para que, al precio actual, el resultado coincida con euclidean_linear_v1 para una misma distancia, así el cambio de método no produce un salto de precio. El emisor paga el 60% del combustible del trayecto más un componente por desgaste, porque el viaje del transportista iba a ocurrir de todas formas, igual que en el carpooling.

La demanda se mide contando los envíos publicados a 15 km o menos del punto de retiro, más el envío que se está cotizando. La oferta se mide contando los transportistas distintos con viajes declarados o activos que salen entre 6 horas antes y 72 horas después y cuyo trayecto pasa a 15 km o menos del retiro. Es el mismo umbral que usa el prefiltro de corredor de MOVO-50. Hay alta demanda cuando se juntan al menos 4 envíos y la relación entre envíos y transportistas llega a 3. En ese caso el recargo arranca en 10%, suma 5% por cada punto de relación por encima de 3 y tiene un tope de 30%. El umbral se calibró con una simulación Monte Carlo sobre mercados sintéticos: el aviso aparece en alrededor del 9% de las zonas con oferta holgada, el 22% en un mercado equilibrado y el 49% cuando faltan transportistas. Todos los parámetros quedan en configuración. Los conteos los calcula movo-svc-shipments y los envía en un campo opcional del request. Si ese campo no llega, no se aplica recargo.

El endpoint POST /quote responde únicamente el precio sugerido, un indicador de alta demanda y la versión del método de cálculo (demand_fuel_routes_v1). El desglose que devolvía la versión anterior deja de viajar en la respuesta, porque ningún consumidor lo usaba. En su lugar, el desglose completo queda en un log estructurado que sirve como evidencia ante disputas. El texto del aviso ("Alta demanda en tu zona") es responsabilidad de la app mobile. El indicador de alta demanda se guarda en el envío, junto al precio sugerido, como una foto del momento de la cotización que no se recalcula. Un valor vacío significa que no hubo cotización y no equivale a que no hubiera alta demanda.

Si Google Routes API falla al cotizar, /quote no devuelve error: calcula la distancia en línea recta multiplicada por 1,3 y lo deja registrado en el log. Para que el respaldo llegue a tiempo, la llamada a Google tiene un timeout de alrededor de 1,5 segundos. Es una excepción acotada a la cotización. La política de no degradar sigue vigente para la optimización de rutas y la evaluación de candidatos, porque ahí una distancia aproximada se traduciría en instrucciones de ruta falsas para el transportista.

CONSECUENCIAS

Trade-offs

Positivo: los precios siguen a la inflación sin que nadie tenga que actualizar coeficientes. El proveedor de combustible no tiene costo y se consulta como mucho una vez por día. pricing-logistics sigue sin base de datos y su función de precio es determinística y fácil de testear. El emisor ve un solo precio y, como mucho, un aviso que siempre corresponde a un recargo real. Y ni la caída de Google ni la del portal de Energía dejan un envío sin precio.

Trade-off negativo: la API de Energía no tiene SLA y, al responder solo por HTTP, una respuesta podría alterarse en tránsito. Los controles de cordura y el cache lo mitigan, con el costo de que un aumento real mayor al 25% en un solo día tarda hasta 7 días en reflejarse. El umbral de demanda se calibró con datos sintéticos porque todavía no hay producción, y habrá que recalibrarlo con los logs reales cambiando la configuración. Usar el precio nacional desvía el precio final hasta un 21% en provincias con nafta más barata, como La Pampa. Cuando Google falla, la distancia es aproximada, lo que se acepta porque el precio es sugerido y editable y el caso queda identificado en el log. Por último, movo-svc-shipments suma dos consultas de conteo al crear un envío, apoyadas en índices que ya existen.

13.8.23 ADR-022 · Arquitectura de cobro: Payments API + OAuth Connect + application_fee

ACEPTADO MOVO-209

CONTEXTO

Situación

MOVO conecta a un emisor que paga con un transportista que cobra, y no quiere tocar el dinero ni ser un intermediario financiero. Hay que reservar el monto al aceptar una oferta, cobrarlo recién al entregar, y repartirlo en el mismo acto (comisión de Movo y neto del transportista). El spike MOVO-49 verificó en sandbox qué combinaciones de la API de Mercado Pago lo permiten (docs/payments/mercadopago-spike/SOLUCION-FINAL.md).

ALTERNATIVAS CONSIDERADAS

Alternativa 1

Alternativa	Por qué no
Movo recibe el pago en su cuenta y le transfiere al transportista	Movo pasa a custodiar dinero ajeno (obligaciones regulatorias, riesgo de contraparte) y hay que programar y conciliar los payouts.

Alternativa	Por qué no
Checkout Pro / Preferences	No permite reservar sin cobrar con split por transportista con este modelo de cuentas.
Pago en efectivo con cobro de comisión posterior	Queda en el backlog (MOVO-37): no reserva nada y depende de cobrarle después al transportista.
Tarjeta guardada + hold off-session	No validado en marketplace; ver punto 5.
Renovar holds vencidos	Puede fallar con el paquete en tránsito, bloquea doble monto y parece un doble cobro (decisión de MOVO-12).

DECISIÓN

Decisión tomada

1. Marketplace de Mercado Pago: el pago se crea en la Payments API con el access_token OAuth del transportista (vendedor), y la comisión de Movo viaja como application_fee. MP acredita el neto directo al transportista y retiene la comisión para Movo en la misma operación: Movo nunca custodia el dinero.
2. Vinculación por OAuth Connect + PKCE (MOVO-111). Los tokens se guardan cifrados (AES-256-GCM) en svc-payments, único servicio que habla con MP.
3. Hold = Auth & Capture (capture: false): reserva sin cobrar. Se libera con status: cancelled y se captura al confirmarse la entrega (MOVO-212).
4. SDK oficial mercadopago v3 para pagos, con X-Idempotency-Key obligatoria en cada escritura. El canje y el refresh de OAuth van con fetch propio (el SDK no tipa code_verifier/test_token y exige un access token que /oauth/token no usa).
5. El hold se crea siempre con el emisor presente: el mobile tokeniza la tarjeta con la public_key del transportista y el backend nunca ve el número. Cobrar sobre una tarjeta guardada sin el emisor (card-on-file / off-session) se descartó: no se pudo validar en el modelo marketplace (error 128 al guardarla, SOLUCION-FINAL §7).
6. Un intento de reserva = una fila y una idempotency key (movo-hold-<shipmentId>-<attempt>). Un reintento del mismo intento reusa la key; un rechazo o una liberación abre el intento siguiente. Un índice único parcial en Postgres garantiza un solo hold vivo por envío.
7. Comisión: application_fee = decomposeOfferGrossPrice(monto).commissionAmountArs de @movo/shared. El monto que paga el emisor es el bruto (Offer.priceOffered) y Movo cobra su porcentaje sobre el neto del transportista (MOVO-143 AC6), no sobre el bruto.

CONSECUENCIAS

Trade-offs

- Dependencia de que el transportista tenga la cuenta vinculada y vigente: sin ella no hay public_key ni token para cobrar (CARRIER_MP_ACCOUNT_NOT_LINKED). Los tokens vencen a los 180 días; renovarlos es MOVO-243.
- El emisor tiene que estar presente al crear el hold: la reserva de un retiro lejano se difiere hasta que el emisor confirma el pago cerca de la fecha (DD-021, MOVO-210).

- El plazo del hold lo decide Mercado Pago y su documentación se contradice (5 o 7 días). Se parametriza (MP_HOLD_VALIDITY_DAYS); MOVO-215 fijó 5 días (DD-050).
- Dos requests casi simultáneos o una respuesta perdida se resuelven por idempotencia y por un índice único parcial, no con locks distribuidos. Un intento sin respuesta de MP (creating) se reintenta con la misma key si el cuerpo coincide; si cambió, se busca el pago en MP por external_reference antes de abrir otro intento. Caso residual aceptado: si no hay credenciales del transportista para preguntarle a MP, el intento viejo se cierra igual y, si MP llegó a crearlo, queda un hold huérfano que vence solo (MOVO-268 lo reconcilia).
- Liberar un hold exige el access_token del transportista (MP solo deja cancelar al cobrador). Si desvinculó o MP revocó el acceso, el hold no se puede liberar desde Movo y vence solo en MP, con los fondos del emisor retenidos hasta entonces. Pendiente de decisión: impedir la desvinculación con holds vivos.
- El sandbox exige que las tres partes sean cuentas de prueba y que el pagador use el email real de una cuenta Comprador; con uno inventado MP crea un invitado y responde 2034. Producción puede requerir homologar el modelo Marketplace en la cuenta real de Movo (pregunta abierta en el hilo de soporte de MP).
- Credenciales: nunca se loguean tokens ni datos de tarjeta (redacción en el logger, MOVO-267).
- Captura y liquidación (MOVO-212): al entregarse el envío, svc-shipments pide a svc-payments capturar el hold SIN amount (cobra exactamente lo autorizado) con el access_token del transportista; application_fee ya se fijó al crear el hold. El desglose real (fee_details + net_received_amount) solo existe después de capturar y se guarda en payments.settlements (una fila por envío, numeric(12,2)), no se calcula con una tasa. Idempotencia en tres capas: fila única por envío creada en processing antes de llamar a MP, key de MP por hold y adopción de un pago ya approved. La entrega es un hecho inmutable: una captura fallida deja el envío en delivered y solo una captura acreditada lo pasa a completed; reintentos con backoff y escalado manual quedan para MOVO-213.

REFERENCIAS

Referencias

- docs/payments/flujo-de-pagos.md
- docs/payments/mercadopago-spike/SOLUCION-FINAL.md
- ADR-004 (JWT/tokens), ADR-010 (confianza perimetral), ADR-017, DD-021
- Tickets: MOVO-49, MOVO-111, MOVO-209, MOVO-210, MOVO-212, MOVO-215, MOVO-268

13.8.24 ADR-023 · Chat propio sobre el canal WebSocket (MOVO-26)

ACEPTADO MOVO-26); parcialmente reemplazado por DD-046

CONTEXTO

Situación	MOVO-26 pide chat de texto en tiempo real entre emisor, receptor y transportista, disponible desde la asignación hasta 24 h después de la entrega, con el historial vinculado al envío y visible para el admin en una disputa. Evaluamos proveedores con plan gratuito: Stream Chat (Maker Account: 2.000 MAU y 100 conexiones simultáneas, por aprobación, para equipos chicos y de bajos ingresos; al llegar al tope los usuarios nuevos no pueden conectarse), Sendbird y CometChat (~100 MAU), Tencent RTC (1.000 MAU) y Firebase (una base en tiempo real, no un chat). Cifras de las páginas públicas de cada proveedor, a reconfirmar.
Alcance	Parcialmente reemplazado por DD-046 (dd-046-notificaciones-de-chat.md): la push del chat ahora lleva el texto del mensaje y las ráfagas se juntan en un resumen. El resto de este ADR sigue vigente.

DECISIÓN

Decisión tomada	Chat propio dentro de movo-svc-shipments, sobre el canal de ADR-019. Un hilo por envío; se envía por HTTP (POST /shipments/:id/chat/messages, idempotente por clientMessageId) y se recibe por WebSocket (GET /shipments/:id/chat), como la ingesta de posiciones de ADR-020. Persistencia en shipments.chat_messages. Solo texto en el MVP. Instancia propia del registro de sockets para no mezclar el chat con el tracking. Push por mensaje, sin el texto, a quien no tiene el chat abierto.

CONSECUENCIAS

Trade-offs	Sin ack en tiempo real por el socket (el ack es la respuesta HTTP), sin presencia, indicador de escritura ni leído/no leído; el registro de sockets está en memoria y es de una sola réplica (ADR-006/019); el bloqueo (DD-026) oculta el contenido de quien tiene un bloqueo con el autor (mensaje vacío, sin push ni tildes) en vez de impedir escribir, porque es un hilo único por envío y no uno por par; entregado el envío el hilo queda en solo lectura y a las 24 h se archiva (sigue legible). cada asignación arranca una conversación limpia (shipments.chat_started_at, fijado por acceptOffer): tras un pago vencido y una reasignación el transportista nuevo no lee lo hablado con el anterior, aunque los mensajes viejos se conservan y el admin los ve. Retención y purga del historial: pendiente de definir con el equipo.

Por Qué No Un Proveedor	
Detalle	La autorización del chat depende de datos que solo tiene svc-shipments (partes del envío, receptor transferido por DD-038, ventana de 24 h, bloqueos de DD-026). Un tercero obligaría a duplicar usuarios y permisos, a sincronizar el historial para el admin y a depender de un cupo y de una credencial más, sin ahorrar el trabajo de reglas. El canal WebSocket, la autenticación del handshake y el push ya existen.

14

PRODUCTO · DECISIONES

Registro de Decisiones de Diseño

Decisiones de alcance acotado (DD) tomadas durante los sprints: reglas de módulo, contratos de endpoint y ajustes de comportamiento.

CONTENIDO

14.1 Registro de Decisiones de Diseño 327

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	24/10/2026	Equipo completo	Se trasladan desde el registro de ADR del Sprint 0 las decisiones de alcance acotado, con el prefijo DD y su número original.
2	02/11/2026	Equipo completo	Incorporación al Tomo II de la documentación final.

Introducción

Complementa al registro de ADR del documento de Arquitectura: allí están las decisiones que fijan el stack, un proveedor, una integración, la estructura entre servicios o el modelo de seguridad; acá, las de alcance acotado.

Audiencia

El tribunal evaluador y los futuros mantenedores que necesiten la justificación de una regla de negocio o de un contrato puntual del sistema.

14.1 Registro de Decisiones de Diseño

Este registro reúne las decisiones de diseño de alcance acotado (las reglas de un módulo, el contrato de un endpoint, el ajuste de un comportamiento) que se tomaron durante los sprints y que no definen la arquitectura ni el stack del sistema. Por eso no forman parte del registro de ADR.

Cada decisión lleva el prefijo **DD** y conserva el número que tenía cuando se registró como ADR, para que las referencias existentes en el código, en los tickets y en los demás documentos sigan siendo válidas: un «ADR» anterior con uno de estos números se lee como DD con el mismo número. Los ADR vigentes están en el documento de Arquitectura del Sistema y se numeran del 001 al 023.

14.1.1 Índice de decisiones de diseño

Tabla 14.1. Índice de las 32 decisiones de diseño.

DD	Decisión
DD-016	Foto de perfil: prefijo profile-photos/ público en S3 con key aleatoria
DD-018	Precio sugerido: implementación lineal provisoria versionada, antes del motor de precios real
DD-021	Set canónico de ShipmentStatus ampliado de 9 a 11 estados
DD-023	Retención de la traza GPS del transportista
DD-026	Bloqueo de usuarios: efecto simétrico sobre envíos y ofertas
DD-027	rejected_by_receiver deja de ser terminal: el emisor elige otro receptor o cancela
DD-028	Precio comprometido: cotización congelada de 15 minutos con quoteId
DD-029	Separación de los estados expired y cancelled en viajes
DD-030	Juegos del sitio institucional: prefijo /demo autenticado con API key
DD-031	Juego del optimizador de la feria contra OR-Tools real
DD-032	Expiración y cierre automático de envíos y viajes
DD-033	Navegación turn-by-turn del transportista por deep-link
DD-034	Inicio de viaje: día calendario y paquete ejecutable
DD-036	Requisitos habilitantes del transportista: licencia aprobada + cuenta de Mercado Pago vinculada
DD-037	Saga de asignación: ofertas vivas hasta confirmar el hold y reversión a published
DD-038	Transferencia de receptor como entidad paralela al ciclo de vida del envío
DD-039	Tarjetas guardadas del emisor con re-ingreso de CVV
DD-041	Pricing por peso facturable con distancia achatada
DD-042	Viajes recurrentes: recurrencia como atributo del Trip con generación de la siguiente ocurrencia
DD-043	Refresh idempotente del panel en una ventana de 10 segundos
DD-044	Cuenta corriente del usuario

DD	Decisión
DD-045	Salida para un envío in_transit demorado: aviso, disputa automática y bloqueo del transportista
DD-046	Notificaciones push del chat con el texto y ráfagas en un resumen (MOVO-301)
DD-047	Penalidad por cancelar un envío asignado
DD-048	Cuenta cobradora de Movo configurada por credenciales
DD-049	Sanciones de cuenta: suspended y banned
DD-050	Plazo del hold de Mercado Pago (5 días) y N de la saga (3 días)
DD-051	Soporte al usuario sobre el chat existente (MOVO-320)
DD-052	Atribución del no-retiro con evidencia GPS pasiva
DD-053	Oferta de transportistas del recargo por demanda: solo viajes declared
DD-054	Disputa sobre un envío completed: la arista completed → disputed
DD-055	Cambio forzado de estado de un envío desde el panel

14.1.2 DD-016 · Foto de perfil: prefijo profile-photos/ público en S3 con key aleatoria

ACEPTADO (MOVO-97)

CONTEXTO

Situación: MOVO-97 agrega la foto de perfil. El mobile recorta (1:1), comprime en cliente y sube directo a S3 con una presigned URL. La foto es información pública por diseño (criterio de aceptación 9 de MOVO-97): la contraparte de un envío la usa para reconocer a la persona.

Alcance: Primera implementación real de ADR-007.

ALTERNATIVAS CONSIDERADAS

Alternativa 1: Bucket 100 % privado con presigned GET en cada lectura. Descartada: photo_url no sería una URL estable ni cacheable por el cliente.

Alternativa 2: Prefijo profile-photos/ de lectura pública, resto del bucket privado (ELEGIDA). La policy de bucket abre solo profile-photos/* y no hay ListBucket público.

DECISIÓN

Decisión tomada: Bucket S3 con policy de lectura pública acotada al prefijo profile-photos/* y el resto privado, con key de objeto basada en un UUID aleatorio. photo_url queda como URL estable y cacheable por el cliente. La subida usa una presigned URL que firma también content-type y content-length (el cliente tiene que mandar esos headers exactos o S3 rechaza la firma). Detrás hay una interfaz StorageProvider con S3StorageProvider y MockStorageProvider; la key del objeto se deriva parseando el photo_url ya guardado, sin columna nueva. Las credenciales AWS llegan por el IAM role de la EC2. El bucket y su policy están en Terraform (movo-infra), aplicados en dev; prod tiene el código listo sin terraform apply corrido al momento de la decisión.

CONSECUENCIAS

Trade-offs: Quien tenga la URL exacta ve la foto sin autenticarse; se acepta porque la foto ya es pública por diseño. A cambio, el cliente cachea la URL sin pedir una firmada nueva en cada lectura. Quedó documentada una ventana de carrera aceptada desde MOVO-97 (cerrada después, ver HISTORIAL de movo-svc-users).

REFERENCIAS

Referencias: MOVO-97, ADR-007 (almacenamiento de archivos multimedia).

14.1.3 DD-018 · Precio sugerido: implementación lineal provisoria versionada, antes del motor de precios real

ACEPTADO

CONTEXTO

Situación: La US padre (MOVO-8, "Como emisor, quiero crear un envío...") exige que el sistema calcule y muestre un precio sugerido en ARS al completar peso, dimensiones y destino, considerando al menos distancia, peso y tipo de paquete, y que muestre "precio a estimar" si alguno de esos datos no está completo.

El alcance del proyecto ([Movo] 002-Estudio Inicial.md) describe un motor de precios dinámico que además incorpora índice de demanda, precio de combustible y distancia real por calle vía Google Routes API (ADR-015). Implementar ese motor completo junto con el resto de la creación de envíos (MOVO-79/80/81) en el sprint 1 excedía la capacidad del equipo, el propio ticket MOVO-82 lo señala explícitamente: "meter eso completo en el sprint 1 se come el sprint".

A la fecha de esta revisión, movo-svc-pricing-logistics seguía siendo un esqueleto (GET /health únicamente, sin ningún endpoint de negocio), el único código relacionado era el spike de VRPTW con OR-Tools (MOVO-50), que no calcula precio. Es el punto más barato del proyecto para fijar el contrato correcto antes de escribir el algoritmo final, mismo criterio que ya justificó adoptar Prisma en ADR-011 y migrar a Routes API en ADR-015.

Criterios: - Cumplimiento completo del AC de la US padre (precio sugerido, o "precio a estimar" si faltan datos) sin bloquear el resto del flujo de creación de envíos.

- Esfuerzo de implementación dentro de la capacidad real del sprint 1, con un solo dev disponible para movo-svc-pricing-logistics en paralelo al track de svc-shipments.
- Honestidad académica: no presentar una fórmula simplificada como si fuera el motor completo en la defensa oral, la memoria escrita necesita justificación documentada de cualquier limitación aceptada.
- Costo de reemplazo futuro: que el contrato HTTP no tenga que migrarse cuando el motor real exista.
- Separar "levantar el primer servicio de negocio en Python/FastAPI del monorepo" de "escribir el algoritmo definitivo", son dos riesgos distintos que conviene no acoplar en un mismo ticket.

ALTERNATIVAS CONSIDERADAS

Alternativa 1: Bloquear la creación de envíos hasta tener el motor de precios real completo.

Pros: nunca se le muestra al emisor un precio inexacto.

Contras: viola el AC explícito de la US padre, “precio a estimar” es un estado válido y esperado, no una degradación a evitar. Introduce un acoplamiento duro entre movo-svc-shipments y movo-svc-pricing-logistics sin justificación de negocio (ADR-001 ya acepta el acoplamiento temporal de REST síncrono, pero no bloquear un flujo completo por un servicio no crítico). El sprint 1 se extendería indefinidamente: ni el índice de demanda ni el precio de combustible tienen todavía fuente de datos ni definición de negocio, y las credenciales de Google Maps seguían sin cargar (mismo estado que Twilio/Didit).

Alternativa 2: Implementar el motor de precios real completo (demanda + combustible + Google Routes API) en este mismo ticket.

Pros: no hay que volver a tocar el cálculo de precio más adelante.

Contras: sobre-alcance declarado explícitamente por el propio ticket MOVO-82, se come el sprint completo. Google Routes API todavía no tenía credenciales provisionadas. El índice de demanda no tiene ni fuente de datos ni criterio de negocio definido (¿oferta de transportistas por zona? ¿por franja horaria? nadie lo había refinado). Se descartó por el mismo motivo que llevó a acotar el alcance del resto de MOVO-8 (MOVO-83 diferido a sprint 2).

Alternativa 3: Contrato estable POST /quote (tipado en shared/movo-shared, consumido por movo-svc-shipments vía un cliente HTTP dedicado) con una implementación provisoria versionada explícitamente, y fallback a “precio a estimar” si el servicio no responde o los datos están incompletos.

Pros: cumple el AC completo de la US padre, incluida la rama de datos incompletos. Separa contrato de implementación, el motor real reemplaza el algoritmo sin migrar a los consumidores. Primera pieza de negocio real de movo-svc-pricing-logistics (antes solo un esqueleto), lo que además habilita a MOVO-83 (wizard mobile) a integrarse contra un contrato estable. La honestidad queda explícita en el propio payload (calculationMethod), no relegada a un comentario en el código.

Contras: el precio mostrado no refleja demanda, combustible ni tráfico real hasta que el motor real lo reemplace. La distancia euclidiana (no geodésica, no por calle) introduce un margen de error adicional, aceptado a la escala urbana/interurbana en la que opera MOVO.

DECISIÓN

Decisión tomada: Se adopta el contrato POST /quote en movo-svc-pricing-logistics, tipado en shared/movo-shared/src/types/pricing.ts (QuoteRequest/QuoteResponse/PriceCalculationMethod) y consumido desde movo-svc-shipments vía src/adapters/pricing-client.ts.

La implementación de este sprint calcula el precio con una fórmula lineal (base + distanciaKm·tarifaKm + pesoKg·tarifaKg, escalada por un factor según packageType), usando distancia euclidiana entre coordenadas (proyección equirectangular: los grados se convierten a km escalando la longitud por cos(latitud promedio), y recién ahí se aplica Pitágoras). Los coeficientes de la fórmula viven en configuración (env vars con prefijo PRICING_), no hardcodeados, para poder ajustarlos durante las pruebas sin rebuildear la imagen.

La versión del algoritmo queda identificada explícitamente en cada respuesta (calculationMethod: “euclidean_linear_v1”) y se persiste junto al precio en shipments.shipments.calculation_method, un envío ya creado nunca se recalcula retroactivamente, así que un cambio de fórmula a futuro no altera precios ya cotizados.

Si movo-svc-pricing-logistics no responde, hace timeout, o faltan datos necesarios (peso, dimensiones o coordenadas), movo-svc-shipments no bloquea la creación del envío: persiste suggested_price_ars = null (junto con calculation_method = null), que la API expone como el indicador de "precio a estimar" exigido por el AC de la US padre.

Se deja registrado en el backlog (MOVO-138, sin refinar todavía) el motor de precios real, índice de demanda, precio de combustible y distancia real por calle vía Google Routes API (ADR-015), que reemplazará euclidean_linear_v1 con un nuevo valor de PriceCalculationMethod, nunca reusando ni renombrando el existente.

CONSECUENCIAS

Trade-offs: Positivo: cumple el AC completo de la US padre (MOVO-8), incluida la rama de "precio a estimar" para datos incompletos, no es un recorte de alcance silencioso.

Positivo: movo-svc-pricing-logistics deja de ser un esqueleto vacío, primer endpoint de negocio real del servicio, y primer contacto real del equipo con FastAPI en el monorepo.

Positivo: el contrato queda listo para el motor real sin requerir cambios en los consumidores (movo-svc-shipments, y a futuro el wizard mobile de MOVO-83), reemplazarlo es agregar un valor de enum, no renegociar un contrato.

Positivo: académicamente defendible, la implementación provisoria está versionada y expuesta explícitamente en cada respuesta (calculationMethod), nunca presentada como el motor completo ante la cátedra.

Trade-off negativo: el precio sugerido que ve el emisor es inexacto (no refleja demanda, combustible ni tráfico real) hasta que MOVO-138 reemplace euclidean_linear_v1, mitigado por el propio nombre del calculationMethod, visible en cada respuesta y persistido.

Trade-off negativo: MOVO-138 (el motor real) queda en el backlog sin refinar, sin criterios de aceptación detallados ni estimación, con el riesgo de quedar indefinidamente pospuesto si no se prioriza en un sprint futuro.

14.1.4 DD-021 · Set canónico de ShipmentStatus ampliado de 9 a 11 estados

ACEPTADO (MOVO-208)

CONTEXTO

Situación: El set de 9 estados se cerró en MOVO-79 y se modeló en el DTE de MOVO-105. Dos decisiones posteriores necesitaban estados propios: el hold de fondos anclado cerca del retiro en lugar de en la aceptación de la oferta (MOVO-12) y distinguir "entregado" de "entregado y cobrado" (MOVO-212). El diseño original de MOVO-12 (método de pago validado al aceptar más un hold programado automático a T-24h) se descartó por el spike MOVO-49: tarjeta guardada más cobro sin el emisor presente no se pudo validar en marketplace.

Alcance: Su arista assigned → assigned_unfunded fue agregada después por DD-037.

ALTERNATIVAS CONSIDERADAS

Alternativa 1: delivery_failed. Evaluado y descartado explícitamente: quedaban cinco preguntas de negocio sin responder (ver docs/shipments/state-diagram.md).

Alternativa 2: completed como resultado alternativo de delivered. Descartado: completed es una consecuencia posterior de delivered, no una alternativa.

DECISIÓN

Decisión tomada: El enum pasa a 11 valores, en @movo/shared, en Postgres y en la máquina de estados, en el mismo PR. assigned_unfunded: transportista asignado sin hold creado porque el retiro es a más de N días; cuando el retiro entra en N días se abre una ventana para que el emisor confirme el pago con la app abierta (crea el hold y pasa a assigned), y sin pago a T-24h vuelve a published. Es una rama alternativa a assignment_pending desde published, y nunca sale directo a in_transit. completed: entrega confirmada y pago liberado, terminal, solo entra desde delivered. La migración se escribió a mano con dos ALTER TYPE ... ADD VALUE; la reversibilidad (Postgres no soporta DROP VALUE) está documentada en la propia migración y probada en un test de integración. Cuatro lugares que comparaban status === DELIVERED pasan a tratar completed como equivalente (FULFILLED_SHIPMENT_STATUSES).

CONSECUENCIAS

Trade-offs: Al momento de la decisión ninguna de las dos transiciones nuevas se disparaba todavía (bloqueadas por Mercado Pago, MOVO-210 y MOVO-212): la máquina de estados permitía un camino que ningún endpoint HTTP disparaba, documentado como "disponible y probado, no disparado" en vez de dejarlo implícito. Revisiones posteriores: completed dejó de ser terminal con DD-054 (arista completed → disputed) y DD-037 agrega assigned → assigned_unfunded.

REFERENCIAS

Referencias: MOVO-79, MOVO-105 (DTE), MOVO-208 (este ticket), MOVO-12, MOVO-49, MOVO-210, MOVO-212, DD-037, DD-054, docs/shipments/state-diagram.md.

14.1.5 DD-023 · Retención de la traza GPS del transportista

ACEPTADO

CONTEXTO

Situación: MOVO-202 (ingesta de posiciones del transportista durante in_transit) exige persistir la traza completa, no solo la última posición, decisión de producto del 08/09/2026, con el propósito declarado de dejar evidencia para disputas (MOVO-30), nunca analítica ni perfilado. Persistir el desplazamiento georreferenciado de una persona física es tratamiento de datos personales sensibles (Ley 25.326), el proyecto ya implementó el derecho de supresión (MOVO-39, Done), así que el criterio de minimización y borrado ya existe en el proyecto y este ticket tiene que respetarlo, no inventar uno nuevo. El propio ticket advertía explícitamente: sin retención acotada y purga automática real, esto es un hallazgo directo en la defensa del PF y un problema real de cumplimiento, no solo un detalle técnico.

Criterios: Propósito declarado (evidencia operativa de disputas, no analítica) como techo de cuánto conviene retener; principio de minimización de datos (Ley 25.326) sobre geolocalización, la categoría de dato más sensible que maneja el proyecto hasta ahora; que el plazo sea ejecutable por un job real dentro del alcance del PF (un servidor, sin infraestructura de colas/cron externa, ADR-006) y no quede como intención sin código; que la existencia de una disputa abierta

(MOVO-30) no permita que la purga borre evidencia que todavía hace falta; consistencia con el precedente de ventanas cortas ya aceptado en el proyecto (72hs para calificar, MOVO-146) en vez de inventar un criterio aislado.

ALTERNATIVAS CONSIDERADAS

Alternativa 1: Sin plazo fijo / purga manual a demanda: Pros: no había que definir un número arbitrario; simple de implementar (nada que programar). Contras: exactamente lo que el ticket advertía evitar, sin un job real y probado, la retención queda como intención, no como control efectivo; acumula datos de geolocalización sensibles indefinidamente sin justificación de propósito, el escenario de mayor riesgo de cumplimiento; ningún costo de mantenimiento pero el mayor pasivo legal.

Alternativa 2: 30 días desde el cierre del envío (ELEGIDA): Pros: cubre con margen amplio la ventana típica en que una disputa se abre después de una entrega, bien por encima de las 72hs que ya rigen la ventana de calificación (MOVO-146); minimiza el tiempo que se retiene un dato sensible sin necesidad real, más alineado al principio de minimización que un plazo largo; job de purga simple (setInterval + lock de Redis, mismo patrón ya aceptado en el proyecto para MOVO-124/MOVO-130) sin infraestructura nueva. Contras: corta frente a un litigio civil que escale mucho después de esos 30 días, mitigado por la excepción de disputed (ver Decisión); es un número de equipo, no derivado de una norma específica, y así queda documentado, no presentado como obligación legal que no es.

Alternativa 3: 90 días desde el cierre del envío: Pros: más margen si una disputa demora en escalar o el usuario tarda en reclamar. Contras: retiene geolocalización sensible el triple de tiempo sin que el propósito declarado (evidencia para disputas, no analítica) lo justifique, contradice el principio de minimización que el propio ticket invocaba; sin ningún caso de uso comprometido hoy que necesite ese margen extra.

DECISIÓN

Decisión tomada: Se adopta un plazo de 30 días desde que el envío cierra (estados delivered/completed/cancelled/rejected by receiver, POSITION_PURGE_ELIGIBLE_STATUSES en shipment-state-machine.ts), anclado en Shipment.lastStatusChangedAt (columna ya existente, mantenida por updateStatus() desde MOVO-104/105, sin necesidad de una columna nueva). Un envío en disputed nunca es candidato a purga mientras siga en ese estado, la traza puede ser evidencia de la disputa en curso. Como disputed no tiene hoy ninguna transición de salida modelada (mismo vacío que dejó pendiente DD-021), en la práctica la traza de un envío disputado queda retenida sin límite mientras dure; el día que exista una transición real de salida, el plazo de 30 días cuenta desde ese momento (la resolución), nunca desde el cierre original anterior a la disputa, consecuencia directa de anclar en lastStatusChangedAt en vez de en un timestamp de "primer cierre", sin requerir ningún cambio de código cuando esa transición se modele. La purga corre por job periódico (carrier-position-purge-sweep.ts, cada 60 minutos por default, configurable vía CARRIER_POSITION_PURGE_SWEEP_INTERVAL_MINUTES), mismo mecanismo ya aceptado en el proyecto para los barridos de MOVO-124/MOVO-130 y el de retiro vencido de published (MOVO-39). La supresión de cuenta no espera los 30 días: al dar de baja una cuenta, la traza del usuario como transportista se borra de inmediato mediante un endpoint interno nuevo (DELETE /internal/account-deletion/users/:userId/carrier-positions), llamado best-effort desde movo-svc-users, es supresión de datos personales a pedido, un caso distinto del ciclo de vida normal de retención, y no bloquea la baja de cuenta si movo-svc-shipments no responde (el barrido periódico la alcanza más tarde si el envío ya cerró)..

CONSECUENCIAS

Trade-offs: Positivo, minimiza el tiempo que se retiene un dato de geolocalización sensible sin necesidad real, alineado al propósito declarado (evidencia operativa, no analítica). Positivo, la excepción de disputed evita el escenario más grave (borrar evidencia mientras una disputa sigue abierta) sin necesitar todavía una fecha de resolución real, que no existe en el proyecto. Positivo, el job de purga existe, está probado (incluido el caso disputed) y corre con el mismo mecanismo ya validado en el proyecto (Redis lock + setInterval), no queda como intención.

Trade-off negativo, 30 días es corto frente a un litigio civil que escale mucho después de ese plazo; se acepta porque el propósito declarado de esta traza es evidencia operativa de una disputa gestionada por la plataforma (MOVO-30), no evidencia judicial de largo plazo, y la excepción de disputed cubre el caso donde la evidencia todavía hace falta. Trade-off negativo, el borrado cross-service de la supresión de cuenta es best-effort, no transaccional (no hay 2PC entre movo-svc-users y movo-svc-shipments, ADR-001), si falla, depende del barrido periódico para llegar más tarde; el escenario de un transportista con un envío todavía activo dándose de baja ya está bloqueado aguas arriba (MOVO-134 que cerró envíos activos), así que en la práctica el riesgo real es bajo.

Alternativa a reabrir si cambian las condiciones: si el equipo define una transición real de salida de disputed (resolución de disputas, MOVO-30/32), o si surge un requisito legal/contractual que exija un plazo distinto, ahí hay una razón concreta para revisar el número, mientras esas condiciones no existan, no está justificado extenderlo.

14.1.6 DD-026 · Bloqueo de usuarios: efecto simétrico sobre envíos y ofertas

ACEPTADO

CONTEXTO

Situación: El rediseño de la pantalla de perfil (MOVO-176) incorpora un menú para reportar y bloquear a otro usuario (MOVO-175). Hasta ese momento no existía ningún concepto de trust & safety en el sistema: ni en el backlog, ni en el esquema de datos. La relación de bloqueo es un dato de identidad y por lo tanto vive en movo-svc-users, pero su efecto recae casi por completo en movo-svc-shipments: el feed de envíos disponibles, los envíos compatibles con un viaje declarado, las ofertas, la asignación del transportista y la designación del receptor. Hay que definir qué significa bloquear para el resto del sistema, en qué dirección aplica, qué ve la persona bloqueada, qué pasa con los envíos que ya están en curso entre ambas partes y cómo se comporta el sistema si movo-svc-users no responde en el momento de consultar el bloqueo.

Criterios: Protección real de quien bloquea: que la otra persona no pueda seguir iniciando interacciones con ella, no solo dejar de verla. Consistencia con ADR-001 (comunicación REST síncrona, sin message broker) y ADR-010 (endpoints internos protegidos de forma perimetral). Consistencia con el criterio ya aceptado en MOVO-134 para la baja de cuenta: nada se cancela en cascada, cada parte resuelve lo que ya tiene en curso. Disponibilidad: una falla de movo-svc-users no debería dejar inutilizable el feed de envíos. Paginación correcta de los listados filtrados. Costo de implementación acotado al alcance del PF.

ALTERNATIVAS CONSIDERADAS

Alternativa 1: Alcance del bloqueo. (a) Solo ocultar a la otra persona de los listados propios. Pros: implementación mínima, sin llamadas nuevas entre servicios en las escrituras. Contras: el bloqueado podría seguir ofertando sobre los envíos de quien lo bloqueó, o ser designado como receptor; el bloqueo sería una preferencia visual y no una protección. (b) Limitarlo a la relación social (perfil, y en el futuro chat) sin tocar envíos. Pros: cero impacto en movo-svc-shipments. Contras: en MOVO la interacción real entre desconocidos ocurre justamente en envíos y ofertas, así que el bloqueo no cubriría el caso de uso principal. (c) Ocultar e impedir nuevas interacciones (ELEGIDA). Pros: el bloqueo protege de verdad. Contras: movo-svc-shipments pasa a depender de movo-svc-users en operaciones de escritura.

Alternativa 2: Dirección del bloqueo. (a) Unidireccional: solo quien bloquea deja de ver a la otra persona. Contras: el bloqueado seguiría viendo los envíos de quien lo bloqueó en el feed Transportar, y la única barrera sería un error al intentar ofertar, un camino que invita a reintentar. (b) Simétrica (ELEGIDA): un bloqueo en cualquier dirección separa a los dos usuarios en todos los listados e interacciones. Se persiste igualmente una fila por dirección (quién bloqueó a quién), porque la dirección sí importa para mostrar "Desbloquear" y para la lista propia de bloqueados; la simetría se resuelve al consultar, uniendo ambas direcciones.

Alternativa 3: Qué se comunica al bloqueado. (a) Error genérico ("el envío ya no está disponible"), sin revelar el bloqueo. Pros: discreción, al estilo de las redes sociales. Contras: mensajes engañosos que llevan a reintentar o a reportar un bug, y un comportamiento difícil de explicar en la defensa del proyecto. (b) Error explícito 403 USER_BLOCKED (ELEGIDA). Pros: claridad para el usuario y para el soporte. Contras: revela que hubo un bloqueo. El perfil público de quien bloqueó se sigue mostrando con normalidad: es público por diseño (DD-016) y la contraparte de un envío en curso necesita poder reconocer a la persona.

Alternativa 4: Comportamiento si movo-svc-users no responde. (a) Fallar siempre cerrado: una caída de movo-svc-users también tira el feed de envíos y las ofertas recibidas. (b) Fallar siempre abierto: el bloqueo pasa a ser best-effort incluso en las escrituras, y durante una caída el bloqueado podría ofertar o ser asignado. (c) Mixto (ELEGIDA): las escrituras (crear oferta, aceptar oferta, designar receptor) fallan cerradas con 502, igual que el chequeo de KYC ya existente; los listados y el push de envío compatible fallan abiertos, se muestran sin filtrar y dejan un warning en los logs.

Alternativa 5: Dónde se consulta el bloqueo. (a) Replicar la relación en el schema shipments (tabla espejo o cache). Contras: sin message broker (ADR-001), mantener la réplica consistente requiere escrituras duales o sincronización periódica, con una ventana de inconsistencia en la dirección insegura. (b) Consultar a movo-svc-users por HTTP en cada operación (ELEGIDA): un endpoint interno GET /internal/users/:id/block-relations devuelve la unión simétrica de ids. Contras: una llamada de red extra en cada operación afectada.

DECISIÓN

Decisión tomada: La relación de bloqueo vive en movo-svc-users (users.user_blocks, una fila por dirección) y su efecto es simétrico. movo-svc-users expone POST y DELETE /users/:id/block, GET /users/me/blocked (solo la lista propia, nunca quién bloqueó al usuario) y el endpoint interno GET /internal/users/:id/block-relations, que devuelve la unión de ambas direcciones. El perfil público incluye isBlockedByMe solo al mirar a otro usuario, sin revelar nunca la dirección inversa, y la búsqueda de receptor excluye a cualquier usuario con un bloqueo en cualquier dirección. movo-svc-shipments aplica el bloqueo en dos modos. Falla cerrado, con 403 USER_BLOCKED explícito, al crear una oferta (transportista contra emisor y receptor), al aceptar una oferta hecha antes del

bloqueo y al designar el receptor de un envío nuevo. Falla abierto al filtrar el feed de envíos disponibles y los envíos compatibles con un viaje, con el filtro dentro del WHERE compartido entre la consulta de datos y la de conteo para que la paginación sea correcta; también al filtrar las ofertas pendientes que ve el emisor y el push de envío compatible con un viaje. Los envíos ya en curso entre ambas partes no se cancelan ni se modifican. Al dar de baja una cuenta se borran sus bloqueos en ambas direcciones; los reportes (users.user_reports) se conservan como evidencia para administración. Los reportes solo se persisten: se admite un único reporte pendiente por par de usuarios (reportar de nuevo es idempotente) y un máximo de 10 reportes nuevos por día por usuario.

CONSECUENCIAS

Trade-offs: Positivo: el bloqueo protege efectivamente a quien bloquea: la otra persona deja de verla y no puede iniciar ninguna interacción nueva. Positivo: el feed de envíos sigue disponible aunque movo-svc-users no responda, y ninguna interacción se concreta sin haber pasado por un chequeo que falla cerrado. Positivo: una única fuente de verdad, sin réplicas que sincronizar. Trade-off negativo: la persona bloqueada se entera del bloqueo al intentar interactuar; se prioriza la claridad sobre la discreción. Trade-off negativo: durante una caída de movo-svc-users los listados pueden mostrar a alguien bloqueado; se acepta porque cualquier acción posterior vuelve a chequear y falla cerrada. Trade-off negativo: movo-svc-shipments suma una llamada HTTP síncrona a movo-svc-users en cada creación o aceptación de oferta, en cada creación de envío y en cada consulta del feed; es un acoplamiento temporal adicional, ya aceptado como costo general de ADR-001. Trade-off negativo: la relación de bloqueo no se cancela en cascada sobre envíos en curso, así que dos usuarios bloqueados pueden seguir vinculados por un envío existente hasta que termine o lo cancelen (mismo criterio que MOVO-134). Limitación aceptada: los reportes se persisten sin una interfaz de revisión; la moderación desde el panel de administración queda para una US posterior.

14.1.7 DD-027 · rejected_by_receiver deja de ser terminal: el emisor elige otro receptor o cancela

ACEPTADO

DECISIÓN

Decisión tomada: rejected_by_receiver deja de ser terminal (MOVO-253, reemplaza en ese punto a MOVO-79/DD-021): el emisor tiene un plazo (receiver_redesignation_deadline, 48hs con tope en la ventana de retiro) para elegir otro receptor (POST /shipments/:id/receiver, vuelve a awaiting_receiver_confirmation) o cancelar; si no, el barrido de MOVO-130 lo pasa a cancelled. Mismo molde “estado con deadline + barrido” que la confirmación del receptor, en vez de decidir en el cliente según la hora

CONSECUENCIAS

Trade-off aceptado: El terminal real de un rechazo pasa a ser cancelled (el rechazo queda como evento en la línea de tiempo); quien rechazó pierde acceso al envío apenas se elige a otra persona (el acceso sigue a receiverId); solo se cambia la persona, nunca la dirección de entrega, así el precio sugerido sigue sin recalcularse (MOVO-82 AC8)

Fuente: fila de resumen del registro de ADRs del repositorio; no tiene borrador extendido.

14.1.8 DD-028 · Precio comprometido: cotización congelada de 15 minutos con quoteId

ACEPTADO (MOVO-255)

CONTEXTO

Situación: MOVO-255 hace que el resumen del wizard de envío muestre una cotización real y que el envío se cree exactamente a ese precio. Para eso la cotización usa la misma función que la creación (quoteShipment, extraída en MOVO-138).

Alcance: Complementa ADR-021.

DECISIÓN

Decisión tomada: POST /shipments/quote cotiza con la misma función que la creación y, si hay precio, lo guarda en Redis (shipment_quote:{quoteId}, TTL de 15 minutos, constante y no variable de entorno) ligado al usuario y a un fingerprint de los datos que afectan el precio (SHA-256 de tipo, peso, dimensiones y coordenadas redondeadas a 6 decimales). POST /shipments acepta un quoteId opcional: con él crea el envío exactamente a ese precio, sin llamar a pricing. La cotización es de un solo uso y se consume con un script Lua que compara antes de borrar, de modo que un quoteId ajeno o mandado con otros datos no quema la cotización del dueño. Un quoteId inexistente, vencido, usado o de otro usuario responde 409 QUOTE_EXPIRED (sin revelar que el id existe) y uno con fingerprint distinto 409 QUOTE_MISMATCH (la cotización queda viva); nunca se recalcula en silencio. Se consume después de las validaciones de createShipment (receptor, KYC, bloqueo, franja) para que un 422 no la queme. Descripción, direcciones escritas y franja no entran al fingerprint porque no afectan el precio.

CONSECUENCIAS

Trade-offs: El emisor que tarda más de 15 minutos o cambia datos que afectan el precio tiene que volver a confirmar uno nuevo. Sin quoteId (pricing caído o builds viejos del mobile) el backend sigue cotizando al crear, con un precio que el emisor no vio (“precio a estimar”).

REFERENCIAS

Referencias: MOVO-138 (quoteShipment), MOVO-255 (este ticket), ADR-021 (motor de precios dinámico), DD-041 (pricing por peso facturable).

14.1.9 DD-029 · Separación de los estados expired y cancelled en viajes

ACEPTADO

DECISIÓN

Decisión tomada: Separación de estado expired y cancelled en viajes (MOVO-260): se introduce TripStatus.EXPIRED para viajes vencidos por el cronjob trip-expiry-sweep.ts, diferenciándolos de TripStatus.CANCELLED que ahora se usa para la cancelación lógica explícita del transportista (POST /trips/:id/cancel). El borrado físico (DELETE /trips/:id) queda deprecado

CONSECUENCIAS

Trade-off aceptado: Desacoplamiento semántico que permite a la UI diferenciar un viaje vencido por inactividad de uno cancelado manualmente; mantiene historial completo de la actividad del transportista, lo que sirve para trazabilidad y métricas

Fuente: fila de resumen del registro de ADRs del repositorio; no tiene borrador extendido.

14.1.10 DD-030 · Juegos del sitio institucional: prefijo /demo autenticado con API key

ACEPTADO

DECISIÓN

Decisión tomada: Juegos del sitio institucional (movo-institucional /juegos, primero el juego de precios de la feria): el gateway autentica el prefijo /demo con API key (DEMO_API_KEYS, header x-api-key) en vez de JWT, solo desde el servidor de Next (sin CORS); POST /quote de pricing acepta includeBreakdown opt-in, que solo pide el módulo demo de svc-shipments (el emisor sigue sin desglose, ADR-021 intacto); partidas en shipments.pricing_game_sessions

CONSECUENCIAS

Trade-off aceptado: Primer cliente sin usuario: una key filtrada permite cotizar (Google Routes, pago) y escribir partidas hasta rotarla, mitigado con rate limit por visitante y keys por cliente; el desglose de la fórmula queda visible en el juego (es justamente lo que se quiere mostrar en la feria)

Fuente: fila de resumen del registro de ADRs del repositorio; no tiene borrador extendido.

14.1.11 DD-031 · Juego del optimizador de la feria contra OR-Tools real

ACEPTADO

DECISIÓN

Decisión tomada: Juego del optimizador de la feria (movo-institucional /juegos/optimizador): compite contra el OR-Tools real de POST /optimize/route, con dos opt-in que solo usa el módulo demo de svc-shipments (objective: “distance” y matrix precalculada); la matriz de cada ciudad (pool fijo) se pide una vez a POST /routes/matrix y se cachea 30 días en Redis, así la feria completa factura ~487 elementos de Google en vez de ~49 por partida. Ranking del día compartido entre iPads en shipments.route_game_sessions

CONSECUENCIAS

Trade-off aceptado: La matriz cacheada no refleja cambios de tránsito durante el evento (irrelevante para comparar órdenes); el ranking muestra nombres que el visitante elige en un stand público, y quien tenga la key demo puede reiniciarlo (mismo riesgo que DD-030)

Fuente: fila de resumen del registro de ADRs del repositorio; no tiene borrador extendido.

14.1.12 DD-032 · Expiración y cierre automático de envíos y viajes

ACEPTADO

DECISIÓN

Decisión tomada: Expiración y cierre automático (MOVO-258, revisa "bloquea, no cascadea" de MOVO-238): todo envío con transportista (assignment_pending/assigned_unfunded/assigned) se cancela sin culpables 24h después de cerrar su ventana de retiro efectiva (la acordada, copiada al envío al aceptar la oferta); cancelar un envío cierra sus ofertas (pending vigentes y accepted → nuevo OfferStatus.shipment_cancelled) en la misma transacción y eso destraba al viaje sin cascada explícita; un published sigue vivo mientras tenga ofertas vigentes (solo lo protege de la cancelación: deja de ofrecerse y no recibe ofertas nuevas); un in_transit que no se entrega se MARCA para revisión (entrega estimada + 50% de la duración) y nunca se cancela; los paquetes de un viaje quedan fijos al iniciarlo (start() exige ≥1 paquete; solo un viaje declared recibe ofertas, matches y el push de paquete compatible) y un viaje active pasa a completed cuando todos sus paquetes terminaron (disputed cuenta como terminado: sin salida modelada, retenerlo trabaría al transportista para siempre), o se cancela si se queda sin ninguno

CONSECUENCIAS

Trade-off aceptado: Sin penalización ni atribución de responsabilidad (quién faltó queda para un ticket aparte con geolocalización); delivered→completed y la reversión por hold fallido siguen dependiendo de Mercado Pago (MOVO-210/212); el in_transit anómalo solo avisa al transportista y deja la marca, sin disputa automática ni alerta de admin todavía; el conteo de oferta de transportistas del pricing (ADR-021) sigue contando viajes active pese a que ya no toman paquetes, pendiente en MOVO-270

Fuente: fila de resumen del registro de ADRs del repositorio; no tiene borrador extendido.

14.1.13 DD-033 · Navegación turn-by-turn del transportista por deep-link

ACEPTADO

DECISIÓN

Decisión tomada: Navegación turn-by-turn del transportista (MOVO-237): se delega por deep-link a la app de mapas del dispositivo (Google Maps nativo → Waze → Google Maps web, movo-mobile/src/lib/navigation-deeplink.ts) en vez del Navigation SDK de Google o re-routing propio. RoutesProvider (svc-shipments) gana mode: "per_trip" | "live": per_trip es el polyline estático de siempre (default, MOVO-123) y live queda tipado pero falla con 501 ROUTE_MODE_NOT_IMPLEMENTED, como placeholder intencional para una navegación 100% in-app futura

CONSECUENCIAS

Trade-off aceptado: Costo cero para Movo a cambio de que la navegación salga de la app (el tracking en segundo plano de MOVO-242 sigue emitiendo, pero el transportista tiene que volver a Movo para confirmar cada parada); el Navigation SDK (~US$25/1.000, 5x Compute Routes) queda descartado hasta que haya una razón de producto para pagarlo. El ticket lo numeraba ADR-019, ya ocupado por el canal WebSocket

Fuente: fila de resumen del registro de ADRs del repositorio; no tiene borrador extendido.

14.1.14 DD-034 · Inicio de viaje: día calendario y paquete ejecutable

ACEPTADO

DECISIÓN

Decisión tomada: Inicio de viaje (MOVO-277, complementa DD-032 sin modificarlo): POST /trips/:id/start exige que el día calendario argentino de departureAt ya haya llegado (canStartTripOn, @movo/shared, la misma función que decide el botón en el mobile) y al menos un paquete ejecutable. "Paquete ejecutable" (TRIP_EXECUTABLE_SHIPMENT_STATUSES = assigned/in_transit, shipment-state-machine.ts) pasa a ser la única definición que usan start(), la ruta del viaje, el tracking y el conteo executablePackagesCount de /trips; "paquete aceptado" (ACCEPTED_OFFER_FILTER) sigue rigiendo lo que bloquea editar/cancelar y el cierre de un viaje active

CONSECUENCIAS

Trade-off aceptado: Mientras no exista el hold de fondos (MOVO-210) ningún envío llega a assigned, así que ningún viaje se puede iniciar desde la app (en dev hay que forzar el estado); a cambio, un viaje ya no queda active semanas antes ni con "N paquetes" y "Sin paradas asignadas", bloqueando al transportista por el índice de 1 viaje activo. Trade-off aceptado: un viaje "mixto" (al menos un paquete ejecutable más otros en assignment_pending/assigned_unfunded) se puede iniciar, y los no ejecutables quedan fuera de la ruta y del retiro pero siguen contando como aceptados, así que completeFinishedActive no lo cierra hasta que el barrido de DD-032 los cancele (24h después de su ventana de retiro); mientras tanto el índice de 1 viaje activo mantiene bloqueado al transportista. Hoy es teórico (ningún envío llega a assigned hasta MOVO-210); la alternativa descartada por ahora es desasociar los no ejecutables al iniciar, como ya se hace con las ofertas pending

Fuente: fila de resumen del registro de ADRs del repositorio; no tiene borrador extendido.

14.1.15 DD-036 · Requisitos habilitantes del transportista: licencia aprobada + cuenta de Mercado Pago vinculada

ACEPTADO (MOVO-116)

CONTEXTO

Situación: Hasta MOVO-116, operar como transportista (declarar un viaje, ofertar) solo exigía el rol carrier y el KYC de identidad aprobado. MOVO-142 había decidido explícitamente no exigir la licencia de conducir: alguien sin auto puede llevar un paquete en micro, tren o avión.

Dos problemas con eso:

- Mercado Pago: sin una cuenta vinculada y vigente (MOVO-111), el transportista no puede cobrar. Hoy lo descubre recién el hold (MOVO-209, 409 CARRIER_MP_ACCOUNT_NOT_LINKED), cuando el emisor ya eligió su oferta: el envío queda trabado y el emisor tiene que volver a elegir.
- Licencia: el equipo decidió que la insignia de "transportista verificado" pase a ser condición para operar (MOVO-116 lo pide explícito), aceptando que se pierde el caso del transportista sin vehículo propio.

Alcance: Revierte, en ese punto, la decisión de MOVO-142 (“la licencia de conducir es una insignia de confianza, no un permiso de acceso”).

ALTERNATIVAS CONSIDERADAS

Alternativa 1: Mantener la licencia como insignia (MOVO-142) y exigir solo Mercado Pago: descartado por decisión de producto.

Alternativa 2: Leer la licencia de PublicProfile.badges (license_verified) con la llamada que ya existía: cero endpoints nuevos, pero ata un control de acceso a una derivación pensada para mostrar en el perfil, y GET /users/:id además pide la reputación a svc-shipments.

Alternativa 3: Cachear el resultado en Redis (criterio 4 del ticket): cachear un negativo deja bloqueado a quien acaba de vincular su cuenta; cachear solo los positivos deja operar unos minutos a quien acaba de desvincularse. Las escrituras protegidas son pocas por transportista, así que el costo de no cachear es bajo.

Alternativa 4: Un código combinado para “faltan los dos”: combinatoria rígida que crece con cada requisito nuevo; details escala sin códigos nuevos.

DECISIÓN

Decisión tomada: 1. Para operar como transportista hacen falta tres cosas: KYC de identidad aprobado (prerequisito, igual que antes), licencia de conducir aprobada (kyc_status_license = approved, svc-users) y una cuenta de Mercado Pago con la que hoy se pueda cobrar (svc-payments, mismo criterio que el hold: no desvinculada, no revocada, token sin vencer y con public_key).

2. Dónde se exige (svc-shipments, utils/carrier-gate.ts): declarar viaje (POST /trips), ofertar (POST /shipments/:id/offers), editar una oferta (PATCH /offers/:id) e iniciar un viaje (POST /trips/:id/start, sobre el dueño del viaje). Al aceptar una oferta (POST /offers/:id/accept) se revalida al transportista y, si ya no cumple, el emisor recibe 409 OFFER_CARRIER_NOT_ELIGIBLE sin detalle y la oferta sigue pending. Las lecturas (feed, detalle de un envío publicado, mis viajes, matches) siguen pidiendo solo identidad: se puede mirar antes de completar los requisitos.

3. Error explícito y accionable: 403 con code = el primer requisito que falta (CARRIER_LICENSE_NOT_APPROVED, si no CARRIER_MP_ACCOUNT_NOT_LINKED) y details.missingRequirements con todos ([“license”, “mp_account”]). Para eso ApiError (@movo/shared) gana un campo details opcional, compatible hacia atrás.

4. Consulta síncrona REST sin cache (ADR-001): dos endpoints internos nuevos, GET /internal/users/:id/kyc-status y GET /internal/payments/mp-connect/:userId/status, consultados en paralelo. Si cualquiera no responde, el bloqueo falla cerrado (502), mismo criterio que el bloqueo de usuarios en escrituras (DD-026).

CONSECUENCIAS

Trade-offs: • Se pierde el transportista sin vehículo propio (el caso que MOVO-142 quería cubrir).

- Dos llamadas síncronas más por escritura de transportista: si svc-users o svc-payments están caídos, nadie puede declarar viajes ni ofertar (falla cerrado).
- Perder la cuenta de MP a mitad de un viaje en curso no se trata acá (regla de negocio sin definir, fuera del alcance de MOVO-116): solo se revalida al iniciar el viaje y al aceptar una oferta. El hold de MOVO-209 sigue revalidando al cobrar.

- Hoy nada escribe revoked_at (MOVO-243), así que el único motivo real de "cuenta no vigente", además de no haber vinculado, es el vencimiento del token.

REQUISITOS SUMADOS DESPUÉS

Detalle: • no_open_dispute (MOVO-278, DD-045): ningún envío en disputed como transportista. Va primero en missingRequirements y responde 403 CARRIER_DISPUTE_OPEN. Es una consulta local (misma DB) y falla cerrado igual que los otros dos. Se levanta cuando se resuelve la disputa (MOVO-34).

- no_overdue_debt (MOVO-313, DD-044): ninguna deuda de la cuenta corriente sin saldar hace más de USER_DEBT_OVERDUE_DAYS (14, env var de svc-shipments). Responde 403 USER_DEBT_OVERDUE y va al final de missingRequirements: es lo último que se resuelve (con un pago) y, como la disputa, no es un paso de la checklist de MOVO-117.
- La deuda es de la persona, el bloqueo es del rol: solo corta las operaciones de transportista de arriba. Como emisor no se bloquea (crear envíos, pagar, aceptar ofertas): su próximo pago ya incluye la deuda (MOVO-311), así que se cobra ahí.
- Umbral por antigüedad, sin monto mínimo: balanceArs > 0 y oldestUnpaidAt hace más de 14 días (> estricto). Quien sigue operando con tarjeta la salda sola; el bloqueo apunta a quien no lo hace por esa vía (efectivo, MOVO-37).
- Mismo trato que los otros requisitos: consulta síncrona sin cache a GET /internal/payments/users/:userId/balance (MOVO-309), en el mismo Promise.all, y falla cerrado con 502 si svc-payments no responde. Al saldarla (MOVO-311/312) el bloqueo se levanta en la siguiente consulta. No corta envíos en curso: ni el handshake ni el tracking pasan por este gate.
- Un saldo positivo sin oldestUnpaidAt (solo pasa si lo compone un movimiento ya settled sin contrapartida) bloquea: no se puede probar que la deuda es reciente y no se puede deber plata y operar como si nada. Se levanta al compensar el saldo.
- Consecuencias por usar la misma función: el emisor que intenta aceptar la oferta de un transportista con deuda vencida recibe el 409 OFFER_CARRIER_NOT_ELIGIBLE sin detalle (igual que con la licencia o una disputa). Es a propósito: el emisor solo se entera de que ese transportista no puede tomar el envío, no del motivo; la deuda es privada del transportista.
- Viajes recurrentes (DD-042): la deuda frena la ocurrencia, no corta la serie. Si lo único que falta es no_overdue_debt (cuenta activa, identidad aprobada), el barrido posterga la generación (deferred, log trip_recurrence_generation_blocked_by_debt) y la serie queda intacta. Se avisa al transportista una sola vez por ocurrencia frenada (tripRecurrenceBlockedByDebt: "no pudimos declarar tu próximo viaje porque tenés una deuda pendiente"). Al saldar, la próxima corrida declara la siguiente ocurrencia (sale de max(departureAt, ahora), no nace vencida ni se acumulan atrasadas). Con cualquier otro requisito faltante además de la deuda, la serie se corta como antes.

Referencias: MOVO-116 (este ticket), MOVO-117 (UI de requisitos pendientes), MOVO-142 (decisión que se revierte), MOVO-15 (licencia), MOVO-111/112 (vinculación de MP), MOVO-209 (hold), DD-026 (criterio de fallar cerrado).

14.1.16 DD-037 · Saga de asignación: ofertas vivas hasta confirmar el hold y reversión a published

ACEPTADO (MOVO-210)

CONTEXTO

Situación: MOVO-210 implementa la asignación: aceptar una oferta, pagar el hold de fondos del emisor en Mercado Pago (ADR-022) y recién entonces asignar. svc-shipments es dueño de la saga y única puerta del mobile; llama a los endpoints internos de holds de svc-payments. La ruta depende de cuándo es el retiro: a N días o menos va por assignment_pending (el emisor paga al aceptar) y a más por assigned_unfunded (ventana de confirmación, DD-021).

Alcance: Complementa ADR-022 y DD-021.

DECISIÓN

Decisión tomada: Las demás ofertas no se cierran al aceptar sino al confirmarse el hold: pasan a superseded en la misma transacción que → assigned, con aviso a cada transportista. Así una asignación que se cae devuelve el envío a published con sus ofertas vivas (decisión del equipo en el review de la PR #227). La oferta aceptada que no prospera pasa a assignment_lapsed (valor nuevo de OfferStatus), no a rejected, porque no fue un rechazo del emisor. Volver a published desasigna de verdad: carrierId, precio y entrega estimada pasan a null y se restaura la ventana de retiro original. Se agrega la arista assigned → assigned_unfunded (revierte "assigned nunca vuelve atrás" de DD-021, solo hacia ese estado) para que, si Mercado Pago pierde el hold ya confirmado, el emisor reconfirme el pago dentro de la ventana en vez de perder la asignación. updateStatus suma expectedFrom, expectedCarrierId y expectedAgreedPriceArs para cerrar las carreras entre pagar, cancelar y revertir. Un barrido (cada 5 minutos por defecto) expira los assignment_pending impagos, avisa y recuerda en la ventana y revierte a T-24h del retiro; antes de revertir reconcilia con payments y no revierte a ciegas si payments no responde. La cancelación cancela primero y libera el hold después.

CONSECUENCIAS

Trade-offs: Una oferta puede vencer (expiresAt) mientras espera el pago. Un hold perdido con el envío ya in_transit no tiene estado al que volver: queda como error detectable (funding_hold_lost_in_transit) hasta que MOVO-212 y MOVO-268 definan la captura. Un assigned puede volver a pedir pago, relajando la garantía de DD-021. Si la liberación del hold falla al cancelar, el envío igual queda cancelado y el hold vence solo en Mercado Pago, con un error logueado.

REFERENCIAS

Referencias: MOVO-12, MOVO-209 (holds en payments), MOVO-210 (este ticket), MOVO-212 (captura), MOVO-215 y DD-050 (plazo del hold), MOVO-258, DD-021, ADR-022.

14.1.17 DD-038 · Transferencia de receptor como entidad paralela al ciclo de vida del envío

ACEPTADO (MOVO-275)

Complementa DD-027 (elegir otro receptor tras un rechazo) sin modificarlo.

CONTEXTO

Una vez que el receptor aceptó un envío, y sobre todo con el paquete viajando, no había forma de cambiar quién lo recibe. DD-027 resuelve otro caso: el emisor elige otra persona después de un rechazo, antes del retiro, volviendo el envío a awaiting_receiver_confirmation. Acá lo inicia el receptor, sobre un envío que ya aceptó, y puede pasar en tránsito.

DECISIÓN

1. Entidad propia, no un estado del envío. shipments.receiver_transfer_requests (pending_new_receiver → completed , con rejected_by_new_receiver , expired y cancelled como cierres alternativos). La transferencia corre en paralelo y nunca toca Shipment.status : con el paquete en tránsito, volver a esperar confirmación rompería el tracking (MOVO-251) y la máquina de estados. Al aceptar, en una sola transacción la solicitud pasa a completed y el envío cambia de receiverId (compare-and-swap contra el receptor que la pidió y un estado que todavía la admite).

2. La persona invitada tiene que aceptar y tener KYC aprobado, sin bloqueo con el emisor (DD-026). No puede ser el receptor actual, el emisor ni el transportista.

3. El emisor no aprueba. Se evaluó exigir su aprobación (paga y eligió al receptor, y sería una defensa ante una cuenta de receptor comprometida). Se descartó por la fricción: si el emisor tarda, la transferencia llega tarde para quien no va a estar. Mitigación: KYC y aceptación de la persona invitada, push informativa al emisor en el momento y todo queda en la línea de tiempo como evidencia para una disputa (MOVO-30).

4. El receptor original conserva acceso de solo lectura (decisión del equipo, distinta de DD-027, donde quien rechazó pierde el acceso). Sigue viendo el detalle, la línea de tiempo, las fotos y el tracking en vivo, y el envío sigue en su lista con el badge “Transferido”. No tiene ninguna acción: no firma la entrega, no califica, no puede volver a transferir. Se implementa en hasShipmentAccess (lectura); los asserts de acción siguen mirando el receiverId vigente.

5. Una sola transferencia efectiva por envío. Solo la completada consume el cupo. A lo sumo una pendiente y una completada por envío, garantizado por índices únicos parciales.

6. Plazo y carrera con la entrega. La persona invitada tiene RECEIVER_TRANSFER_TIMEOUT_HOURS (6 h); la vence el mismo barrido que la confirmación del receptor (MOVO-130), y aceptar fuera de plazo da 409 aunque el barrido no haya corrido (pedir otra transferencia también vence inline la pendiente pasada de plazo). La entrega “empieza” cuando el transportista genera el QR de entrega por primera vez: eso deja una marca durable (shipments.delivery_handshake_started_at , no el QR de Redis, que vence a los 15 s), cancela la solicitud pendiente y desde ahí no se puede pedir ni aceptar un cambio; el UPDATE de receiverId también exige la marca vacía. El handshake siempre valida contra el receiverId vigente.

7. Línea de tiempo: un item por solicitud. shipment_events sigue siendo “evento = transición de estado”. Las solicitudes se exponen aparte (GET /shipments/:id/receiver-transfers , sin cambiar la respuesta de /events) y el mobile las intercala por fecha como un solo item que cambia según su estado, con texto según quién mira. El emisor y quien pidió cada solicitud ven todas; el transportista y el receptor vigente, solo la completada.

8. Chat del envío (MOVO-340). Al completarse, el receptor nuevo se suma al chat (ADR-023) en el acto y ve el historial previo : el hilo es uno por envío, es la evidencia de la coordinación y suele tener indicaciones de entrega que le sirven. Lo anterior le cuenta como leído (solo suman como no

leídos los mensajes posteriores), y los tildes de un mensaje previo se miden contra los integrantes de ese momento, con el corte en resolved_at de la transferencia. Los sockets abiertos reciben chat_participants_changed ; el receptor original pasa a solo lectura sin que se cierre el suyo, y deja de recibir la push del chat.

CONSECUENCIAS

- Sin aprobación del emisor, una cuenta de receptor comprometida puede desviar el envío hacia otra persona verificada. Queda la evidencia y la disputa.
- El receptor original sigue viendo la ubicación del transportista hasta la entrega aunque ya no reciba el paquete (decisión del equipo: le sirve saber que llegó a quien eligió).
- Dos llamadas a svc-users más al pedir y una al aceptar (perfil y bloqueos), fallando cerrado si no responde.
- El receptor nuevo lee lo que el original escribió en el chat antes de transferir (por ejemplo, un teléfono). Fue el original quien eligió transferir.
- Una solicitud pendiente cuenta como actividad de la persona invitada para la baja de cuenta (MOVO-134).

REFERENCIAS

MOVO-275, DD-027 (MOVO-253), DD-026 (MOVO-175), DD-023 (traza GPS), MOVO-130, MOVO-158.

14.1.18 DD-039 · Tarjetas guardadas del emisor con re-ingreso de CVV

PROPUESTO (MOVO-100)

CONTEXTO

Situación: ADR-022 descartó las tarjetas guardadas porque el único intento (spike MOVO-49, 29/09, SOLUCION-FINAL.md §7) falló al guardar la tarjeta en la cuenta de un transportista con 128 “payment method response is empty”. La consecuencia de producto es mala: el emisor tiene que escribir número, vencimiento, titular y CVV cada vez que acepta una oferta.

El 13/10/2026 se repitió la prueba contra el sandbox con un Vendedor de prueba vinculado en dev (token TEST-), con un script propio y no el del spike. El 128 no se reprodujo. Lo que se verificó, con Visa y Mastercard de prueba:

Paso | Resultado

Guardar la tarjeta en un customer de la cuenta del Vendedor (POST /v1/customers/{id}/cards) | 201

Re-tokenizar con card_id + security_code y la public_key del Vendedor (POST /v1/card_tokens) | 201

Crear el hold con ese token, capture:false y application_fee (POST /v1/payments, token del Vendedor) | 201, authorized / pending_capture (los dos holds se cancelaron: cancelled / by_collector)

Control: re-tokenizar sin CVV y crear el hold | Visa: 500 internal_error. Mastercard: 400 3031 “security_code_id can’t be null”

Conclusiones firmes: (1) guardar la tarjeta y pagar con solo el CVV funciona con el hold y el split de ADR-022; (2) pagar sin CVV no es posible, así que el cobro automático sin el emisor (la opción B original de MOVO-12) sigue descartado.

No se verificó: que application_fee efectivamente se aplique (la consulta posterior devolvió fee_details vacío; lo comprueba la captura, MOVO-212), ni el comportamiento días después, ni más de un Vendedor.

Alcance: Revisa el punto 5 de ADR-022 (“el hold se crea siempre con el emisor presente, sin tarjetas guardadas”) solo en lo que dice de tarjetas guardadas: el emisor sigue presente en cada pago, pero ya no re-ingresa todos los datos. ADR-022 no se modifica (un ADR aceptado no se edita); este lo reemplaza en ese punto una vez que pase a “aceptado”.; Actualización 13/10/2026: la ubicación de la tarjeta (opción A, cuenta de Movo) quedó validada en sandbox con un token TEST- de Movo S.A. (ver “Dónde vive la tarjeta”). Falta confirmar en producción y los puntos de “Pasos para cerrar” antes de pasar a “aceptado”.

DECISIÓN

Decisión tomada: 1. El emisor puede guardar tarjetas y, al pagar, solo re-ingresa el CVV. El CVV nunca se guarda: el mobile lo usa para pedir a MP un token de un solo uso (card_id + CVV + public_key correspondiente) y lo manda como hoy a POST /shipments/:id/funding. El contrato de svc-shipments y svc-payments no cambia: ya recibe un cardToken.

2. Lo que se persiste de la tarjeta es solo customer_id + card_id de MP más los metadatos de display (marca, últimos 4, vencimiento y un nombre de referencia opcional que elige el usuario). Nunca el card_token (de un solo uso, vence a los 7 días) ni número ni CVV.

3. Dueño de la entidad: movo-svc-payments, no svc-users como decía MOVO-100. Es el único servicio que habla con MP (ADR-022) y quien alta/baja la tarjeta tiene que llamar a la API de Customers & Cards; ponerla en svc-users obligaría a svc-users a conocer a MP o a hacer un hop extra por cada operación.

4. Ubicación de la tarjeta: la cuenta de Movo S.A. (opción A), un customer por emisor, creado con el token TEST-/producción de la app y no con el de ningún transportista. El customer_id y el card_id se guardan en svc-payments; el payer del hold es ese customer.

5. El pago en efectivo del transportista (MOVO-37/101) sigue siendo un caso aparte, con Movo como cobrador directo, sin split. Comparte con este ADR el modelo de datos de tarjeta guardada pero no la ubicación (esa sí es la cuenta de Movo por definición).

CONSECUENCIAS

Trade-offs: • Cumplimiento (PCI): no cambia. Número y CVV se capturan y tokenizan en el dispositivo contra MP; el backend nunca los ve. Se suma que se guardan identificadores de MP (customer_id, card_id), no datos de tarjeta.

- El CVV sigue siendo obligatorio en cada pago: sin él MP no crea el hold. El emisor no puede pagar “en segundo plano” y el hold de un retiro lejano sigue requiriendo que abra la app (MOVO-12, DD-021).
- Más superficie: pantalla de alta/baja de tarjetas, endpoints y una entidad nueva, a cambio de un pago de un paso (elegir tarjeta + CVV).

- Baja de tarjeta: no se puede borrar una tarjeta con un hold vivo asociado a un envío del emisor. Si MP rechaza la baja o hay un hold vivo, la app muestra el motivo.
- Tarjetas vencidas o dadas de baja del lado del emisor/banco: la re-tokenización falla y el pago cae al formulario completo (misma pantalla de MOVO-269).
- El sandbox no refleja del todo producción en este tema (credenciales live de la app): parte de la validación queda para producción, igual que la homologación del modelo Marketplace (ADR-022).

DÓNDE VIVE LA TARJETA

Detalle: En la primera prueba el customer y la tarjeta se crearon con el access_token del Vendedor, es decir en la cuenta del transportista. Eso no sirve para el modelo que se quiere (guardar tarjetas en Ajustes → Pagos y cobros y pedir solo el CVV al aceptar una oferta): en Ajustes no hay transportista, y cada oferta puede venir de uno distinto. La alternativa es guardar la tarjeta en la cuenta de Movo S.A. (un customer por emisor) y derivar de ahí un token por transportista.

Por qué parecía imposible y no lo era. El spike había dado 401 300 “Unauthorized use of live credentials” al crear el customer con la app. La causa no era que Movo S.A. sea una cuenta de prueba, sino que se usaba el token de la app (APP_USR-...), que MP trata como producción. Pidiendo un token con POST /oauth/token (grant_type=client_credentials, test_token=true) Movo obtiene un token TEST- y puede crear customers (201). Con test_token=false se reproduce el 300.

Verificado el 13/10/2026 (opción A):

Paso | Resultado

Token TEST- de Movo S.A. (client_credentials + test_token:true) | 200, TEST-

Crear el customer del emisor en la cuenta de Movo | 201

Guardar la tarjeta en ese customer (token de la tarjeta generado con una public_key TEST-) | 201 (visa)

Re-tokenizar con card_id de la cuenta de Movo + CVV y la public_key del Vendedor A | 201

Hold con el access_token del Vendedor A, payer: {type: “customer”, id: <customer de Movo>}, capture:false + application_fee | 201, authorized / pending_capture (cancelado después)

Mismo hold con payer: {email} en vez del customer | 400 2002 “Customer not found”: el pagador tiene que ser el customer de la tarjeta

Re-tokenizar con la public_key del Vendedor B | 201 (el hold de B no se probó: sin sus credenciales)

Detalles que condicionan la implementación:

- Tokenizar la tarjeta con el Bearer de Movo (sin public_key) da 403 G001; hace falta una public_key TEST- en sandbox. La public_key APP_USR- de la app genera un token que el customer TEST- no reconoce (404 card not found). En producción se espera usar la public_key de Movo.
- El payer del hold debe ser { type: “customer”, id }, por lo que payerEmail solo no alcanza: hay que guardar el customer_id por emisor.

Opción **Cómo funciona** **Estado**

A. Tarjeta en la cuenta de Movo (un customer por emisor), token por transportista derivado de ella	Ajustes guarda la tarjeta una vez; al aceptar una oferta se pide el token al transportista con card_id + CVV	Validada en sandbox con el Vendedor A. Falta el hold de otro Vendedor (B) y confirmar en producción
B. card_id de la cuenta del Vendedor A con la public_key de B	Una tarjeta guardada en una sola cuenta de transportista	Descartada: A resuelve el problema de raíz
C. Tarjeta por transportista	Ingresar la tarjeta completa la primera vez con cada transportista	Validada pero sin valor para el emisor; queda como respaldo si A fallara en producción

PASOS PARA CERRAR LA DECISIÓN

Detalle: 1. Crear el hold con las credenciales de un segundo Vendedor (B) usando la misma tarjeta guardada en la cuenta de Movo (hoy solo se probó con A; el token de B se emite, falta el hold).

2. Repetir el recorrido en producción con una cuenta real y bajo monto: la public_key y el token de Movo son distintos fuera del sandbox, y la homologación del modelo Marketplace sigue pendiente (ADR-022).

3. Confirmar que application_fee se aplica al capturar (MOVO-212).

4. Con eso, pasar el ADR a "aceptado". Si falla en producción, la alternativa es la opción C.

REFERENCIAS

Referencias: • docs/payments/adr-022-arquitectura-de-cobro.md (punto 5 y alternativas)

- docs/payments/mercadopago-spike/SOLUCION-FINAL.md §7
- docs/payments/flujo-de-pagos.md §7
- Tickets: MOVO-65, MOVO-100, MOVO-101, MOVO-12, MOVO-269, MOVO-37, MOVO-49, MOVO-212

14.1.19 DD-041 · Pricing por peso facturable con distancia achatada

ACEPTADO (MOVO-276)

CONTEXTO

Situación: demand_fuel_routes_v1 casi no usaba el paquete para cotizar: el peso entraba lineal y barato (~300 $/kg) y el volumen no entraba. Un televisor de 105 × 65 × 15 cm costaba apenas más que una caja de zapatos, y un cubo de 1 m³ lo mismo que una caja compacta de 10 kg. La distancia ya se había achatado con un hotfix (tramos decrecientes, 10/10/2026) pero esa curva nunca se había documentado ni calibrado contra una referencia.

Correo Argentino, Andreani, OCA y Mercado Envíos cotizan con el mismo modelo, aunque sus números no sean públicos: peso facturable (máximo entre real y volumétrico, L × A × H / 5000), distancia por zonas, tramos de peso y límites duros de peso y medidas.

Alcance: Reemplaza la parte de cálculo de ADR-021 (demand_fuel_routes_v1); combustible, recargo por demanda, fallback a Haversine × 1,3 y contrato de /quote siguen como los dejó ADR-021.

ALTERNATIVAS CONSIDERADAS

Alternativa 1: Forma del achatamiento de la distancia

Alternativa 2: Zonas escalonadas (lo que hace la industria): fácil de explicar, pero el precio salta en los bordes de cada zona (un km más puede costar un 20% más) y MOVO no tiene una matriz de zonas por código postal. Descartada.

Alternativa 3: Curva cóncava (ej. raíz o logaritmo de la distancia): suave, pero difícil de explicar en la defensa y de calibrar contra una referencia por tramos. Descartada.

Alternativa 4: Tramos de km con tarifa marginal decreciente (elegida): continua, monótona y cóncava por construcción, y cada parámetro se lee como "cuánto cuesta un km más en este tramo".

Base del precio

Alternativa 5: Desvío marginal del transportista como base (cobrar lo que le cuesta desviarse): exigiría un precio por transportista, distinto del que vio el emisor, y chocaría con la cotización congelada de DD-028. Descartada: el desvío se informa al transportista y él lo traduce en su oferta.

Frágil: categoría o atributo

Alternativa 6: Convertir frágil en un booleano combinable con la categoría separaría tamaño de manipulación ("un frasco y un televisor frágiles no deberían costar lo mismo"), pero cambia el modelo de datos del envío y el wizard mobile. Con el peso facturable el tamaño ya entra en el precio de un fragile_item, así que el problema que motivaba el cambio desaparece sin migrar nada. Se mantiene como categoría.

Límites duros

Alternativa 7: Andreani acepta 50 kg reales y 300 cm de suma de lados (165 cm por lado). Se tomó la suma de 300 cm pero 30 kg y 150 cm por lado: el transportista de MOVO es una persona en su auto, no un camión de reparto, y 30 kg/150 cm ya eran los máximos que validaba el schema del envío desde MOVO-80.

Alternativa 8: Límites según el vehículo del transportista (una moto no carga lo que un auto): queda fuera; el límite es del paquete. Lo que el transportista no puede llevar lo resuelve no ofertando.

DECISIÓN

Decisión tomada: 1. Método nuevo package_tapered_v2 (PriceCalculationMethod, @movo/shared y Python). Los envíos ya creados conservan su precio congelado y su método (demand_fuel_routes_v1 o euclidean_linear_v1): nada recalcula un precio persistido (MOVO-82 AC8, MOVO-255).

2. Peso facturable = máx(peso real, L × A × H / PRICING_VOLUMETRIC_DIVISOR), divisor 5000 por default (Correo/Andreani/OCA; Mercado Libre usa 6000). Los primeros PRICING_WEIGHT_TIER_KG (10) se cobran a tarifa llena y el resto al PRICING_WEIGHT_TIER_RATE (35%): sin ese tramo, el cubo de 1 m³ que admiten los límites (200 kg volumétricos) cotizaba ~$135.000 a cualquier destino.

3. Distancia: se conservan los tramos de km del hotfix (primeros 30 km al 100%, hasta 300 km al 20%, el resto al 1,5%), ahora formalizados y calibrados.

4. Categoría: letter_document tiene una tarifa base propia y menor (PRICING_BASE_FARE_DOCUMENT_L); fragile_item lleva un recargo porcentual (PRICING_FRAGILE_SURCHARGE, 20%) sobre el mismo paquete no frágil.

5. Un solo precio sugerido, origen → destino, calculado al cotizar y congelado con el quoteId. El desvío marginal del transportista (MOVO-219) se muestra, no se cobra.

6. Límites duros: 30 kg reales, 150 cm por lado y 300 cm de suma de lados (PACKAGE_MAX_*, mismas env vars y defaults en svc-shipments y svc-pricing-logistics). svc-shipments los valida al cotizar y al crear el envío (422 PACKAGE_LIMITS_EXCEEDED con details.violations) antes de cualquier I/O, y /quote responde 422 con el mismo criterio como defensa.

7. Coeficientes en litros de nafta (como en ADR-021), calibrados con una grilla de referencia: para standard_package de hasta 10 kg facturables en rutas de 200 km o más, el precio cae entre 0,6× y 1,0× del Correo clásico a domicilio. La grilla (docs/pricing/grilla-referencia.md, tests/pricing_grid.py) queda como golden tests del motor, con 100% de cobertura exigida en CI sobre los módulos de pricing.

CONSECUENCIAS

Trade-offs: • El documento a larga distancia sigue siendo caro (~$15.500 a 1.500 km, más que 1 kg por Correo): su precio es casi todo distancia y una base menor no alcanza. Corregirlo exigiría una tarifa por km por categoría; queda para recalibrar con datos reales.

- La caja de zapatos a CABA queda en 0,99× del Correo, al borde del techo de la banda. Con la distancia real de Google (menor que la del mock de la grilla) el cociente baja, pero cualquier recalibración tiene que volver a pasar los golden tests.
- La banda solo cubre standard_package de hasta 10 kg facturables en rutas de 200 km o más: el Correo no publica tramos mayores ni sirve de referencia para envíos urbanos (ahí la competencia es la mensajería en moto). El resto queda cubierto solo por las propiedades de forma: monotonía en distancia y peso facturable, y concavidad en distancia.
- Las medidas siguen siendo las que declara el emisor: quien subdeclara paga menos. No hay verificación más allá de las fotos del retiro.
- Los límites se configuran en dos servicios: si se recalibran, hay que cambiarlos en los dos (y el mobile replica 150 cm por lado y 300 cm de suma para el aviso en línea; el 422 del backend sigue siendo la fuente de verdad).

Referencias: MOVO-276, MOVO-138/ADR-021, MOVO-216 (spike), MOVO-219 (desvío marginal), MOVO-255/DD-028 (cotización congelada), MOVO-82/DD-018.

14.1.20 DD-042 · Viajes recurrentes: recurrencia como atributo del Trip con generación de la siguiente ocurrencia

ACEPTADO (MOVO-264, spike)

Implementación en MOVO-265 (svc-shipments) y MOVO-266 (movo-mobile). El ticket reservaba DD-029 (lo tomó MOVO-260) y después DD-041 (lo tomó MOVO-276), así que queda 042.

CONTEXTO

El PO pide viajes recurrentes (“todos los viernes viajo a Villa María”). Hoy un Trip es un viaje con una fecha concreta, y varias piezas dependen de eso: las ofertas aceptadas, el filtro de matching por día de departureAt (MOVO-163), el barrido de vencimiento (MOVO-238/DD-032), el cierre a completed y el índice de un solo viaje active por transportista. Por eso un Trip no puede representar “todos los viernes” en una sola fila.

ALTERNATIVAS CONSIDERADAS

Entidad TripSchedule + job que materializa trips con N días de anticipación. Cubre más casos (por ejemplo, matchear paquetes de dentro de dos semanas), pero suma una entidad, un job nuevo y el problema de "editar la serie o solo esta ocurrencia". Para el alcance del TFG no compensa.

DECISIÓN

1. Un trip por ocurrencia, generados de a uno. La recurrencia es un atributo del propio Trip : recurrenceWeekdays (int[] , valores 0-6, nullable = no recurrente), recurrenceUntil (date , nullable, inclusive, interpretada en America/Argentina/Cordoba) y seriesId (uuid ; el primer trip de la serie usa su propio id). Cuando pasa el departureAt del trip vigente, el sistema genera solo la ocurrencia siguiente como un Trip común en declared . La genera sin importar cómo termine el vigente (declared todavía sin vencer, active , expired , cancelled , completed).

2. Convención de días: 0 = domingo … 6 = sábado , como getDay() de JS y EXTRACT(DOW) de Postgres. Se fija como constante documentada en @movo/shared . El mobile ordena los chips arrancando en lunes, pero el mapeo nunca depende de la posición.

3. Trip vigente e idempotencia por sucesor. El trip generado guarda generatedFromTripId (uuid , FK a Trip , UNIQUE): cada trip tiene a lo sumo un sucesor. El "trip vigente" de una serie es el que nadie referencia. Se descartó el índice único parcial "un solo trip no terminal por seriesId": cuando el vigente pasa a active y se genera el siguiente en declared , los dos son legítimos a la vez. El insert usa ON CONFLICT DO NOTHING , así que una segunda corrida del barrido o dos instancias en paralelo no duplican, y no hace falta un nextGeneratedAt . Como el candidato se lee antes de dos llamadas HTTP por serie, el insert bloquea la fila origen (FOR UPDATE), la relee como cabeza de una serie activa y copia los datos de esa fila: un PATCH / cancel con stopRecurrence que llegó en el medio gana y no se genera nada (review de PR #246). La API expone isSeriesHead para que el mobile no tenga que reconstruir la cadena.

4. Cuándo y dónde se genera. Es un paso nuevo del barrido existente (trip-expiry-sweep.ts , runStep propio), sin job nuevo. Corre después de cancel_overdue_declared , así el vigente que vence en esa misma corrida ya cuenta como expired para la regla de serie fantasma (punto 8).

5. Fecha de la siguiente ocurrencia. Es el próximo día de recurrenceWeekdays estrictamente posterior a max(departureAt, now) , a la misma hora de pared, calculado en America/Argentina/Cordoba con el helper de @movo/shared/utils/argentina-date (el mismo huso que canStartTripOn , DD-034). Nunca se suman 7×24 h en UTC. Tomar el máximo con now evita una cascada de trips que nacen vencidos si el barrido estuvo caído varios días. Si la fecha supera recurrenceUntil , no se genera nada y la serie termina sola. El trip generado copia ruta, vehicleType , la recurrencia y seriesId .

6. Editar. La recurrencia solo se edita en el trip vigente (en otro trip de la serie responde 409 TRIP_NOT_SERIES_HEAD). Editar el vigente (ruta, hora, días) define el patrón de las ocurrencias siguientes: no hay una edición "de la serie" aparte. Un viaje con paquetes aceptados no se edita (409 TRIP_HAS_ACCEPTED_PACKAGES), tampoco su recurrencia: si el transportista quiere otro patrón, declara un viaje nuevo. La única excepción es apagar la recurrencia (recurrenceWeekdays: null , con o sin recurrenceUntil: null), que no cambia ese viaje concreto y es la única forma de cortar una serie cuyo vigente ya tiene paquetes (review de PR #246: la primera versión dejaba pasar cualquier PATCH de solo recurrencia, y entonces cambiar los días de un vigente ya active chocaba con la regla del día de salida de abajo, pensada para un viaje que todavía se puede edi-

tar). Validaciones: al menos un día, sin repetidos; el día de semana de departureAt (huso Córdoba) tiene que estar en recurrenceWeekdays ; recurrenceUntil tiene que ser posterior a la fecha de departureAt . Prender o cambiar la recurrencia exige que el viaje esté declared o active (409 TRIP_NOT_EDITABLE si ya terminó: un expired / cancelled / completed sin sucesor sería candidato del barrido y se declararía uno nuevo desde el historial). Un trip auto-creado por MOVO-234 se puede volver recurrente con PATCH , como cualquier otro.

7. Cancelar la ocurrencia o cortar la serie. POST /trips/:id/cancel (MOVO-260) suma { stopRecurrence?: boolean } . Sin el flag cancela solo la ocurrencia, y la siguiente se genera igual cuando pasa su departureAt . Con true también apaga la recurrencia, en la misma transacción. Sobre un trip no recurrente o que no es vigente, el flag se ignora. El cancel sigue limitado a viajes declared sin paquetes aceptados (TRIP_NOT_DECLARED / TRIP_HAS_ACCEPTED_PACKAGES), así que la elección “solo este / este y dejar de repetir” del mobile solo aparece ahí. Para cortar una serie cuyo vigente ya tiene paquetes, está “Dejar de repetir” (el PATCH que la apaga, única excepción del punto 6). Apagar la recurrencia (recurrenceWeekdays = null) conserva seriesId como histórico.

8. Cortes automáticos (siempre recurrenceWeekdays = null en el vigente + push):

- Serie fantasma: si las últimas 3 ocurrencias consecutivas de la cadena (generatedFromTripId) terminaron expired , no se genera la siguiente. Cuenta cualquier expired , sea un declared vencido o un active que se quedó sin paquetes: las dos rutas significan “el viaje no llevó nada” y no hay un startedAt que permita distinguirlas. Una cancelled por el usuario no cuenta y corta la racha.

El resto de esta decisión (los demás cortes automáticos, la alternativa descartada y las consecuencias completas) está en el repositorio: docs/shipments/dd-042-viajes-recurrentes.md.

Trade-off aceptado: Solo existe la próxima ocurrencia, así que no se matchean paquetes a 2 o más semanas. La hora sale del viaje vigente y, si ya tiene paquetes aceptados, ni la hora ni los días se pueden cambiar (solo se puede apagar la serie). Un problema temporal de requisitos al correr el barrido apaga la serie, porque no hay un disparador para saltear y reintentar. No hay control de solapamiento, igual que POST /trips.

14.1.21 DD-043 · Refresh idempotente del panel en una ventana de 10 segundos

PROPUESTO · pendiente de aceptación en Sprint 0

DECISIÓN

Decisión tomada: Refresh idempotente del panel durante 10 s con la misma clave y el mismo token, coordinado en Redis y con resultado cifrado (MOVO-291). Conserva el reuso fuera de ventana y la política de la app.

CONSECUENCIAS

Trade-off aceptado: Es una excepción acotada a la rotación estricta, pensada para solicitudes concurrentes de Next. Queda pendiente de aceptación en Sprint 0.

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); texto completo en docs/auth/dd-043-refresh-idempotente-panel.md.

14.1.22 DD-044 · Cuenta corriente del usuario

ACEPTADO (MOVO-309, 15/10/2026)

DECISIÓN

Decisión tomada: Cuenta corriente por usuario (no por rol), en payments.user_ledger_entries, con saldo con signo (positivo = debe, negativo = a favor); el saldo es la suma de lo que no está voided. Es de solo inserción, impuesta por triggers (de un UPDATE solo pasa status), e idempotente por idempotencyKey más el hash del cuerpo. Incluye oldestUnpaidAt para el bloqueo por deuda (MOVO-313). Es la base de las penalidades (MOVO-226) y de la comisión en efectivo (MOVO-37).

CONSECUENCIAS

Trade-off aceptado: La plata de las penalidades cobradas como deuda queda para Movo, sin crédito al emisor perjudicado. El signo no se deduce del tipo: lo manda quien registra. La deuda se cobra en el application_fee del próximo hold (MOVO-311, tope del 30 % del precio, con una reserva negativa que un trigger anula si el hold se cae) o con un pago voluntario (MOVO-312).

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); texto completo en docs/payments/dd-044-cuenta-corriente-del-usuario.md.

14.1.23 DD-045 · Salida para un envío in_transit demorado: aviso, disputa automática y bloqueo del transportista

ACEPTADO (MOVO-278)

DECISIÓN

Decisión tomada: Complementa DD-032 sin modificarlo. La marca de anomalía de tránsito se expone (transitAnomalyFlaggedAt; “Demorado” en la app y “No tenemos la ubicación del transportista” en el mapa). A las 12 h de la marca se avisa al transportista (una sola vez) y a las 24 h el barrido pasa el envío a disputed sin actor, lo que cierra el viaje solo. Se suma a DD-036 el requisito no_open_dispute (403 CARRIER_DISPUTE_OPEN). Los plazos se configuran por variables de entorno.

CONSECUENCIAS

Trade-off aceptado: El transportista con una disputa abierta queda bloqueado sin salida hasta MOVO-34. La disputa no toca el hold, que puede vencer sin reintento de captura (riesgo conocido: el cobro posterior a la resolución va por la cuenta corriente de MOVO-309). “Demorado” no promete una revisión que hoy nadie hace.

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); texto completo en docs/shipments/dd-045-salida-in-transit-demorado.md.

14.1.24 DD-046 · Notificaciones push del chat con el texto y ráfagas en un resumen (MOVO-301)

ACEPTADO (MOVO-301)

DECISIÓN

Decisión tomada: Las notificaciones push del chat llevan nombre, rol y texto (140 caracteres), al estilo WhatsApp; reemplaza el punto "push sin el texto" de ADR-023. El primer mensaje avisa enseguida y los que llegan en los 30 s siguientes se juntan en un único resumen que sale al cerrarse la ventana, se omite si ya se leyó y vuelve a filtrar bloqueos y envío al salir. El mobile saca de la bandeja las notificaciones del chat al abrirlo.

CONSECUENCIAS

Trade-off aceptado: El texto se ve en la pantalla bloqueada. No hay reemplazo de notificaciones (Expo Push no lo permite), así que una ráfaga son hasta dos avisos por ventana. El resumen vive en un setTimeout del proceso y un reinicio lo pierde.

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); texto completo en docs/chat/dd-046-notificaciones-de-chat.md.

14.1.25 DD-047 · Penalidad por cancelar un envío asignado

PROPUESTO (MOVO-307, 15/10/2026)

DECISIÓN

Decisión tomada: Si se cancela un envío assigned con menos de 12 h hasta el inicio de la ventana de retiro acordada y la ventana abierta, se cobra el 25 % del precio acordado; con 12 h o más, o con la ventana cerrada, es sin cargo. La regla es única y vive en @movo/shared (computeCancellationPenalty), y el emisor confirma el monto que vio (409 CANCELLATION_PENALTY_MISMATCH si cambió). Primero se cancela y después se cobra: deuda en la cuenta corriente (DD-044) más captura parcial del hold (MOVO-306); si el hold no la soporta, se libera entero y queda como deuda. Desde MOVO-310 los mismos tramos aplican al transportista que se baja de un envío asignado (POST /shipments/:id/carrier-withdrawal): el envío vuelve a published y la penalidad va como deuda a su cuenta corriente.

CONSECUENCIAS

Trade-off aceptado: Mercado Pago cobra el application_fee entero en la captura parcial (spike MOVO-305), por lo que el transportista recibe cerca del 11 % del precio y no el 25 %. Con el plan B (incluido un hold con deuda previa) o con el hold vencido, la penalidad se cobra después (MOVO-311) y no compensa al transportista.

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); texto completo en docs/payments/dd-047-penalidad-de-cancelacion.md.

14.1.26 DD-048 · Cuenta cobradora de Movo configurada por credenciales

ACEPTADO (MOVO-315, 17/10/2026)

DECISIÓN

Decisión tomada: "Pagar mi saldo" cobra con las credenciales de la propia cuenta de Mercado Pago de Movo S.A. (MP_COLLECTOR_ACCESS_TOKEN y las claves asociadas), sin un usuario interno de Movo ni OAuth en la app; reemplaza la cobradora vinculada a un usuario de MOVO-312. En producción se usan las credenciales productivas del panel; en sandbox, el token TEST- del OAuth de la cuenta de prueba (scripts/collector-oauth.mjs), que es el único que cobra con tarjeta.

CONSECUENCIAS

Trade-off aceptado: En dev el token vence a los 180 días y se renueva a mano. Dev y prod cobran con distinto tipo de credencial, y producción no se puede probar en el sandbox (queda para MOVO-283).

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); texto completo en docs/payments/dd-048-cuenta-cobradora-de-movo.md.

14.1.27 DD-049 · Sanciones de cuenta: suspended y banned

ACEPTADO (MOVO-296)

DECISIÓN

Decisión tomada: AccountStatus.suspended bloquea las operaciones pero no el login, y banned pasa a significar inhabilitada (corta el login). El historial vive en users.account_sanctions (solo altas y cierre, una vigente por usuario) y User.status/bannedUntil son la caché de la vigente; una suspensión vencida se lee como active. El gateway lee una clave Redis account:restriction:<userId> (suspended o disabled) con TTL hasta el fin de la suspensión; Redis no es la fuente de verdad: svc-users la reescribe al arrancar y en un barrido periódico que también cierra las suspensiones vencidas. Los avisos de sanción no se pueden silenciar desde las preferencias.

CONSECUENCIAS

Trade-off aceptado: Las marcas pueden quedar desfasadas hasta el próximo barrido (5 min por defecto) si Redis falla justo después del commit. Una suspensión pausa (no apaga) las series recurrentes.

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); no tiene borrador extendido.

14.1.28 DD-050 · Plazo del hold de Mercado Pago (5 días) y N de la saga (3 días)

ACEPTADO (MOVO-215)

DECISIÓN

Decisión tomada: Complementa ADR-022. MP_HOLD_VALIDITY_DAYS = 5 (el mínimo documentado por MP; el sandbox no venció dos holds en unas 188 h, así que la medición no permite subirlo) y FUNDING_NEAR_PICKUP_DAYS = 3, de modo que N + tránsito máximo no supere el plazo del hold (2 días entre retiro y vencimiento, uno de tránsito y uno de margen). Ambos valores son configuración.

CONSECUENCIAS

Trade-off aceptado: El plazo no está confirmado en producción ni por banco emisor, y solo se midió Visa crédito. El status_detail de un hold vencido que asume MOVO-268 sigue sin confirmarse. Con N = 3, más envíos van por la ruta lejana.

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); texto completo en docs/payments/dd-050-plazo-del-hold.md.

14.1.29 DD-051 · Soporte al usuario sobre el chat existente (MOVO-320)

ACEPTADO (MOVO-320)

DECISIÓN

Decisión tomada: Las consultas de soporte viven en movo-svc-shipments (support_conversations, support_messages, support_participant_states) y reusan el ChatHub con otra clave de sala (support:<id>), el handshake WS, el filtro de lenguaje y la push de DD-046 (con categoría propia support). Las notas internas del equipo no llegan al usuario por ningún camino: un único punto (toUserFacingMessage) arma lo que ve y la consulta usa una lista blanca de kind. Escribir sobre una conversación resolved la reabre; un envío ajeno es 422 SUPPORT_SHIPMENT_NOT_PARTY y una consulta ajena es 404. No hay borrado ni archivo al resolver.

CONSECUENCIAS

Trade-off aceptado: Es una excepción más a "un esquema por servicio" (ADR-003). El registro de sockets está en memoria y en una réplica (ADR-006/019). El plazo de retención de las conversaciones queda por definir con el equipo legal (MOVO-224) y no hay job de purga. El lado del agente es MOVO-322.

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); texto completo en docs/support/dd-051-soporte-al-usuario.md.

14.1.30 DD-052 · Atribución del no-retiro con evidencia GPS pasiva

ACEPTADO (MOVO-270)

DECISIÓN

Decisión tomada: Complementa DD-023/ADR-020, DD-032, DD-045 y DD-047 sin modificarlos. Cada posición de un envío assigned con el viaje activo se clasifica contra el punto de retiro (llegada, lejos o zona gris, según la distancia menos la precisión) y se guarda en shipments.pickup_evidence, sin guardar la traza. El barrido de retiro no realizado atribuye: si el transportista no llegó, se cancela con attribution: carrier y la penalidad de baja de DD-047 como deuda; si llegó sin handshake, el envío pasa de assigned a disputed con el hold liberado y sin bloquear al transportista; sin evidencia, se cancela sin culpables. El emisor que cancela con la ventana cerrada y la llegada registrada paga el 25 %. Pasado el plazo del barrido, el retiro ya no se confirma.

CONSECUENCIAS

Trade-off aceptado: La falta de datos nunca culpa a nadie: con mala señal o el pin mal geocodificado, un transportista que llegó puede quedar "sin evidencia". iOS no informa mocked. El emisor no se trackea (por eso su falta va a disputa y no a cobro). La penalidad es plata para Movo y la disputa sigue sin salida hasta MOVO-34.

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); texto completo en docs/shipments/dd-052-atribucion-del-no-retiro.md.

14.1.31 DD-053 · Oferta de transportistas del recargo por demanda: solo viajes declared

ACEPTADO (MOVO-270)

DECISIÓN

Decisión tomada: Ajusta el insumo de ADR-021 (DD-041 no cambia el recargo): countAvailableCarriersNear cuenta solo viajes declared, porque un viaje active ya no toma paquetes (DD-032). El umbral ((publicados + 1) / transportistas ≥ 3, mínimo 4) queda igual, y v2 aplica el mismo multiplicador sobre su subtotal.

CONSECUENCIAS

Trade-off aceptado: Las ocurrencias autogeneradas de viajes recurrentes (DD-042) y los transportistas inhabilitados (MOVO-313, MOVO-278) siguen contando: filtrarlos exigiría consultar otros servicios en cada cotización.

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); texto completo en docs/pricing/dd-053-conteo-de-oferta.md.

14.1.32 DD-054 · Disputa sobre un envío completed: la arista completed → disputed

ACEPTADO (MOVO-334)

DECISIÓN

Decisión tomada: Complementa DD-021 y DD-045. Se agrega la arista completed → disputed, porque el handshake de entrega captura el hold y pasa el envío a completed en el momento (MOVO-212), y delivered → disputed casi nunca se alcanza. La usa solo una de las partes dentro de los 7 días desde deliveredAt (DISPUTE_WINDOW_DAYS, la misma ventana que para delivered), nunca un barrido, y abrirla no toca la plata. Mientras dure, el envío deja de contar como cumplido (reputación, ranking, historial compartido, calificación). La disputa es una entidad (shipments.disputes) creada en la misma transacción, una abierta por envío.

CONSECUENCIAS

Trade-off aceptado: Con el cobro ya acreditado no hay hold que liberar: recuperar la plata queda para MOVO-337 (cuenta corriente, DD-044). La reputación baja temporalmente aunque la disputa se resuelva a favor del transportista.

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); texto completo en docs/shipments/dd-054-disputa-desde-completed.md.

14.1.33 DD-055 · Cambio forzado de estado de un envío desde el panel

PROPUESTO (MOVO-330/331)

DECISIÓN

Decisión tomada: Complementa DD-021, ADR-022, DD-044, DD-045, DD-047 y DD-054. Una matriz FORCED_TRANSITION_MATRIX en @movo/shared es la fuente de verdad (el panel solo la lee) y POST /internal/admin/shipments/:id/force-status ejecuta el servicio de dominio de cada celda, nunca un UPDATE. Solo se usan aristas que ya existen; la cancelación y la baja hechas por un admin condonan la penalidad; pasar a assigned exige un hold autorizado en payments; disputed, completed, cancelled, in_transit y delivered no se fuerzan. Lleva expectedStatus obligatorio (409), lock por envío e idempotencyKey; el evento registra el tipo (manual o forced) y al admin, y las partes no ven ni su identidad ni el motivo. Se envía un aviso crítico a las tres partes.

CONSECUENCIAS

Trade-off aceptado: Un envío in_transit o disputed trabado sigue sin salida por el panel (el primero necesita una decisión sobre evidencia, el segundo es MOVO-34). Los avisos críticos saltan las preferencias del usuario.

Fuente: fila de resumen del registro de decisiones del repositorio (docs/decisiones/README.md); texto completo en docs/shipments/dd-055-cambio-forzado-de-estado.md.

15

PRODUCTO · GLOSARIO

Glosario

Lenguaje ubicuo del proyecto, con las definiciones vigentes al cierre del Sprint 8.

CONTENIDO

15.1 Lenguaje del Dominio 361

Historial de revisión

Versión	Fecha	Autor	Descripción del cambio
1	02/11/2026	Equipo completo	Primera versión: fusión del glosario del Estudio Inicial y del Sprint 0, actualizado con la terminología del producto construido.

Introducción

Este glosario define los términos del negocio y de la técnica que usa el equipo, con las definiciones vigentes al cierre del proyecto.

Audiencia

Todos los lectores del tomo; sirve de referencia para interpretar el resto de los documentos.

15.1 Lenguaje del Dominio

Este glosario reúne los términos del negocio y de la técnica que usa el equipo, con las definiciones vigentes al cierre del proyecto. Constituye el **lenguaje ubicuo** de Movo: se usa de forma consistente en el código, en la documentación y en la comunicación del equipo. Donde una definición del Sprint 0 quedó superada por la implementación, aquí figura la versión actual (por ejemplo, el ciclo de vida del envío, el momento de la reserva de fondos o la resolución de disputas).

Tabla 15.1. Glosario del dominio y de la técnica.

Término	Definición
Emisor	Usuario registrado que crea un envío, designa a un receptor y paga. Prepara el paquete y es quien elige al transportista entre las ofertas recibidas.
Receptor	Usuario registrado designado por el emisor como destinatario. Debe aceptar o rechazar el envío en 48 horas, antes de que se publique; no puede editar sus datos. Puede transferir la recepción a otra persona (DD-038).
Transportista	Usuario con identidad y licencia de conducir verificadas y una cuenta de Mercado Pago vinculada, que traslada físicamente el paquete y cobra el neto del envío.
Administrador	Usuario con rol `admin` que opera el panel de administración con un segundo factor; sus acciones de escritura quedan registradas en una bitácora.
Envío	Unidad central del sistema: el acuerdo entre emisor, receptor y transportista para trasladar un paquete. Su ciclo de vida tiene 11 estados (ver el documento de diagramas de estados).
Viaje (declarado)	Trayecto planificado por un transportista (origen, destino, salida y vehículo). Permite sugerirle paquetes compatibles con su ruta y agrupar los envíos que va a llevar. Puede repetirse semanalmente (DD-042).
Oferta	Propuesta de un transportista para trasladar un envío publicado, con el monto neto que quiere cobrar. El sistema le suma la comisión de Movo para mostrar al emisor el precio bruto.
Precio sugerido	Valor que calcula el motor de precios para un envío a partir del peso facturable, la distancia, el tipo de paquete y la demanda de la zona (`package_tapered_v2` , DD-041).
Cotización congelada	Precio sugerido que se reserva durante 15 minutos bajo un identificador de un solo uso, para que el emisor pague el valor que vio (DD-028).
Peso facturable	El mayor entre el peso real y el peso volumétrico del paquete, base del cálculo del precio.
Alta demanda	Recargo que se aplica al precio sugerido cuando la relación entre paquetes y transportistas de la zona supera un umbral.
Comisión de Movo	Porcentaje fijo y configurable del precio que retiene la plataforma al repartir el pago (15 % por defecto).
Feed de paquetes	Lista de envíos publicados cercanos a un punto de retiro, visible para transportistas verificados, sin datos personales de las partes.
Handshake criptográfico	Protocolo de transferencia de custodia en el retiro y en la entrega: el cedente genera un desafío efímero, lo firma con la clave de su dispositivo y el cesionario lo confirma estando a menos de 100 metros (ADR-018).
Cedente / Cesionario	Quien entrega el paquete en una etapa del handshake (emisor en el retiro, transportista en la entrega) y quien lo recibe (transportista en el retiro, receptor en la entrega).

Término	Definición
Nonce	Número aleatorio de un solo uso que identifica un desafío del handshake; se guarda solo su hash en el evento de custodia.
ECDSA P-256	Algoritmo de firma digital de curva elíptica con el que el dispositivo del cedente firma el desafío del handshake.
TTL (QR)	Vigencia del código QR del handshake: 15 segundos; pasado ese tiempo el cedente debe generar uno nuevo.
Proximidad (geofencing)	Verificación de que cedente y cesionario están dentro de un radio de 100 metros al confirmar el handshake.
Evidencia fotográfica	Fotos del paquete que el transportista debe cargar al retirar y al entregar para poder confirmar el handshake.
Tracking	Seguimiento de la posición del transportista durante el tránsito, transmitido en tiempo real por WebSocket a las partes del envío.
Ruta optimizada	Orden de retiros y entregas de un viaje calculado como un VRPTW con ventanas horarias.
VRPTW	*Vehicle Routing Problem with Time Windows*: problema de optimización de rutas con ventanas de tiempo, resuelto con Google OR-Tools.
Chat del envío	Conversación en tiempo real entre emisor, receptor y transportista asignado, vigente mientras el envío no se cancele ni se rechace.
Transferencia de receptor	Solicitud del receptor para que otra persona reciba el envío; se completa de forma atómica si la invitada acepta antes de que empiece la entrega.
Redesignación de receptor	Elección de otro receptor por parte del emisor después de un rechazo, dentro de 48 horas (DD-027).
KYC (Know Your Customer)	Verificación de identidad previa a operar, delegada en Didit.me (documento y prueba de vida). Movo no almacena documentos ni biometría.
Insignia	Marca visual del perfil que certifica un nivel de verificación: identidad verificada y licencia verificada.
Reputación	Score de 1 a 5 de cada usuario, calculado con contracción bayesiana hacia la media global y decaimiento temporal, discriminado por rol. Con menos de 3 calificaciones el perfil se muestra como nuevo.
Calificación	Valoración de 1 a 5 estrellas con comentario opcional entre las partes de un envío entregado, posible durante 72 horas.
Reporte y bloqueo	Herramientas de moderación entre usuarios: el reporte llega al equipo; el bloqueo oculta envíos y bloquea ofertas y designaciones en ambos sentidos.
Sanción	Suspensión (temporal o indefinida) o inhabilitación de una cuenta, aplicada por un administrador y registrada en un historial de solo agregar filas.
Disputa	Reclamo formal de una parte sobre un envío entregado o completado, dentro de 7 días, con motivo, descripción, fotos y descargos de las contrapartes (DD-054).
Soporte	Conversación del usuario con el equipo de Movo desde la aplicación, con o sin un envío asociado.
Hold (reserva)	Autorización del pago en la tarjeta del emisor sin cobrarlo. Se crea al asignar el transportista y se captura recién al confirmarse la entrega.

Término	Definición
Captura	Cobro efectivo de un hold. Puede ser total (al entregar) o parcial (al cobrar una penalidad).
Split payment	Reparto automático del pago capturado: la comisión de Movo queda como *application fee* y el neto se acredita en la cuenta de Mercado Pago del transportista.
Liquidación	Registro del resultado de una captura: montos bruto, comisión y neto, y su estado (en proceso, acreditada, fallida o escalada).
Saga de asignación	Secuencia de pasos entre `movo-svc-shipments` y `movo-svc-payments` para asignar un transportista y reservar los fondos, con compensación si el pago falla o vence (DD-037).
Ruta cercana / lejana	Camino de la saga según los días al retiro: con retiro cercano (dentro de 3 días) el emisor paga al aceptar la oferta, con 30 minutos de plazo; con retiro lejano el pago se confirma cuando se abre una ventana previa al retiro.
Penalidad de cancelación	Cargo del 25 % del precio acordado cuando el emisor cancela un envío asignado con menos de 12 horas de anticipación al retiro (DD-047).
Cuenta corriente	Registro por usuario de penalidades, recargos, pagos voluntarios y ajustes que generan una deuda o un saldo a favor (DD-044).
Deuda vencida	Deuda sin saldar por más de 14 días, que impide al transportista ofertar y declarar viajes hasta saldarla.
Vinculación de Mercado Pago	Autorización OAuth que permite a Movo cobrar en nombre del transportista y repartir el pago.
Tarjeta guardada	Tarjeta del emisor registrada en Mercado Pago, con la que se paga ingresando solo el código de seguridad. Movo conserva únicamente la marca, los últimos cuatro dígitos y el vencimiento.
Access token / Refresh token	Credenciales de sesión: un JWT de 60 minutos y un token opaco de 90 días, de un solo uso, que se renueva con rotación (ADR-004, ADR-013).
OTP	Código de un solo uso enviado por SMS o correo para verificar un teléfono, recuperar la contraseña o ingresar al panel.
API Gateway	Único servicio expuesto a Internet: autentica, limita el uso y enruta cada pedido al servicio dueño.
Referencia lógica	Referencia a una entidad de otro servicio por su identificador, sin clave foránea, para que ningún servicio dependa de las tablas de otro (ADR-003).
ADR	*Architecture Decision Record*: registro de una decisión de arquitectura con su contexto y el costo aceptado.
Compare-and-swap	Escritura condicionada al estado esperado (`UPDATE … WHERE status = …`) con la que se evitan condiciones de carrera sin bloqueos largos.
Idempotencia	Propiedad por la que repetir una operación con la misma clave devuelve el resultado original sin duplicar su efecto.
Barrido (*sweep*)	Tarea periódica en segundo plano (vencimientos, reintentos, limpieza) que toma un candado en Redis para correr en una sola réplica.
Fail-open / Fail-closed	Política ante la caída de una dependencia: dejar pasar la operación (por ejemplo, el filtro de bloqueos del feed) o rechazarla (por ejemplo, crear una oferta sin poder verificar el bloqueo).

Término	Definición
URL prefirmada	URL temporal de Amazon S3 para subir o leer un archivo sin exponer credenciales ni pasar el archivo por el backend (ADR-007).
WebSocket	Canal bidireccional persistente usado para el tracking, el chat y el soporte, autenticado con el JWT en el establecimiento de la conexión (ADR-019).
Webhook	Llamada que un proveedor externo (Didit, Mercado Pago) hace a Movo para informar un evento; se valida con una firma y una ventana de tiempo para evitar repeticiones.
TFG	Trabajo Final de Grado: proyecto integrador de la carrera, en el que se enmarca Movo.

movo

Cinco estudiantes, un solo equipo.

Movo empezó con una pregunta: ¿por qué enviar algo a otro lado tendría que depender de si vivís cerca de una sucursal? Nuestra respuesta fue una red de personas que se mueven y aprovechan el camino para llevar los paquetes de otros.

La construimos entre cinco. Sin jerarquías fijas y sin silos: los cinco somos, ante todo, desarrolladores, las decisiones técnicas se toman en equipo y el rol de Scrum Master rotó cada mes para que todos viviéramos las dos perspectivas.

Hubo bloqueos de proveedores, rediseños y sprints que no cerraron como esperábamos. También hubo un sistema que funciona de punta a punta y una feria donde Movo llegó entre los tres finalistas. Lo que más nos enorgullece no cabe en un tomo: lo hicimos juntos.



Alena Ariza
Leg. 95359



Juan Cruz Bordino Blanche
Leg. 95008



Lucas Dalmagro
Leg. 94366



Tomás Ignacio Vergara
Leg. 94197



Pedro Yorlano
Leg. 95197

Cinco personas. Un proyecto. Logística para personas.



Fin del tomo II

Logística para personas. Movo es un proyecto independiente de Ingeniería en Sistemas de Información, Universidad Tecnológica Nacional, Facultad Regional Córdoba. Entrega final: 2 de noviembre de 2026.